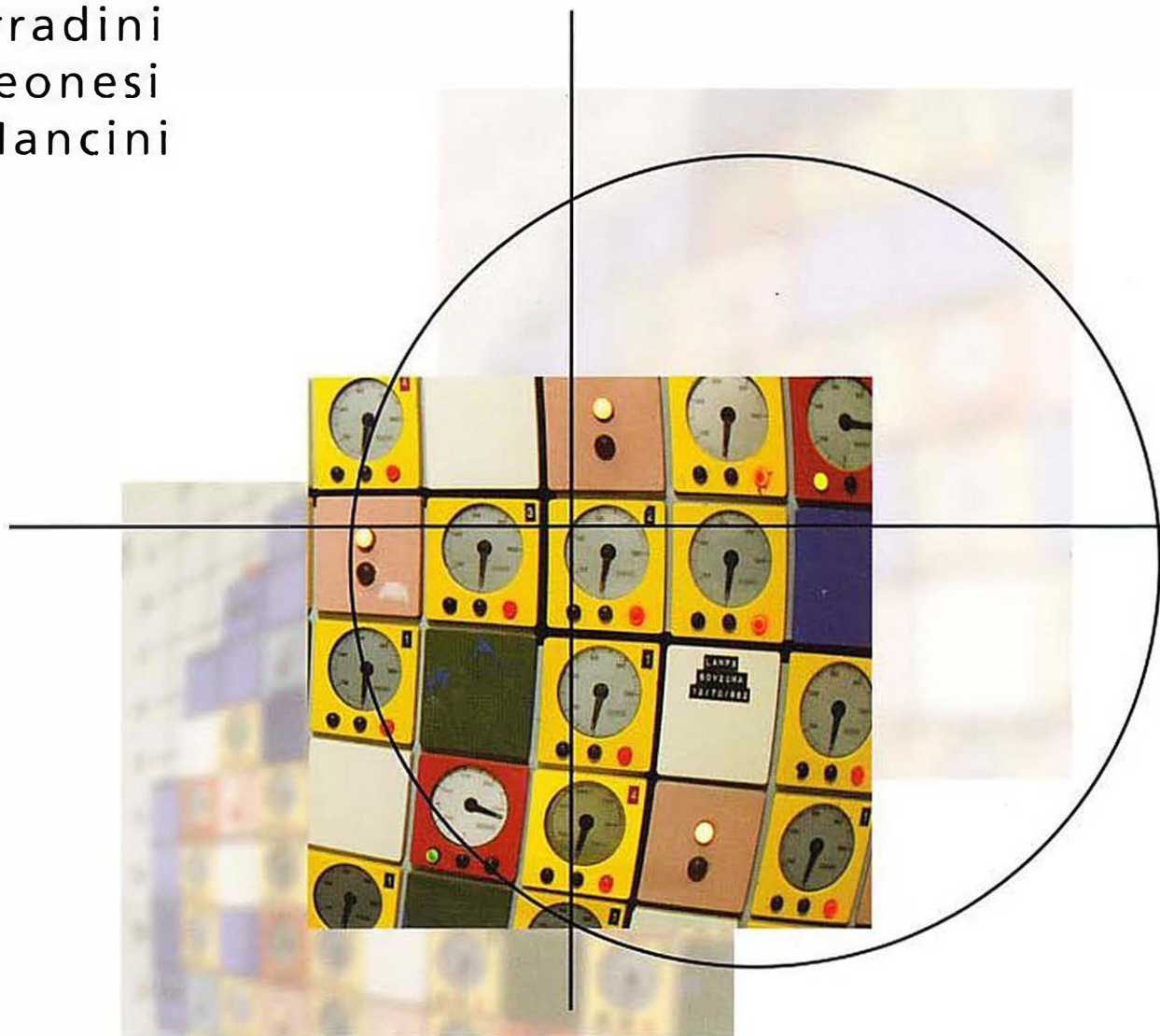
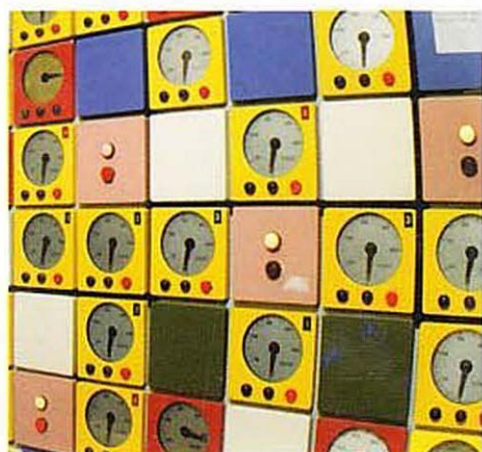


Carlo Toffalori
Flavio Corradini
Stefano Leonesi
Stefano Mancini



Teoria della computabilità e della complessità

McGraw-Hill



Carlo Toffalori
Flavio Corradini
Stefano Leonesi
Stefano Mancini

Teoria della computabilità e della complessità

Che cos'è un algoritmo? Quali problemi si possono risolvere con un algoritmo? E in quali casi un algoritmo di soluzione richiede risorse accessibili nella pratica?

Il libro affronta questi problemi, trattando anzitutto la teoria della computabilità sia dal punto di vista classico – macchine di Turing, funzioni ricorsive, grammatiche – che da prospettive più attuali, come quella dei programmi while, legate ai linguaggi di programmazione e in particolare al Pascal. Si discutono poi i vari possibili criteri di misura delle risorse disponibili (tempo, memoria, casualità) e si sviluppa la corrispondente teoria della complessità computazionale, dalla classica questione $P = NP$ al teorema di Savitch, dai circuiti booleani alle procedure probabilistiche e interattive. Un moderno approccio alla computazione e alla complessità basato sulla teoria quantistica è introdotto nella parte finale del volume.

Il testo si rivolge agli studenti di Informatica, Matematica, Fisica o Ingegneria interessati a corsi di Informatica teorica.

Gli autori insegnano presso la Facoltà di Scienze e Tecnologie dell'Università di Camerino.

Carlo Toffalori è docente di Logica Matematica.

Flavio Corradini è docente di Ingegneria del Software.

Stefano Leonesi è docente di Didattica della Matematica.

Stefano Mancini è docente di Informazione Quantistica.



www.mcgraw-hill.it



www.ateneonline.it



www.hyperbook.it

ISBN 88-386-6228-2



9 788838 662287

Carlo Toffalori
Flavio Corradini
Stefano Leonesi
Stefano Mancini

Teoria della computabilità e della complessità

McGraw-Hill

Milano • New York • San Francisco • Washington D.C. • Auckland
Bogotá • Lisboa • London • Madrid • Mexico City • Montreal
New Delhi • San Juan • Singapore • Sydney • Tokyo • Toronto

Copyright © 2005 The McGraw-Hill Companies, S.r.l.
Publishing Group Italia
via Ripamonti 89 – 20139 Milano

McGraw-Hill



A Division of The McGraw-Hill Companies

I diritti di traduzione, di riproduzione, di memorizzazione elettronica e di adattamento totale e parziale con qualsiasi mezzo (compresi i microfilm e le copie fotostatiche) sono riservati per tutti i Paesi.

Date le caratteristiche intrinseche di Internet, l'Editore non è responsabile per eventuali variazioni negli indirizzi e nei contenuti dei siti Internet riportati.

Nomi e marchi citati nel testo sono generalmente depositati o registrati dalle rispettive case produttrici.

Editor: Chiara Tartara
Produzione: Donatella Giuliani
Impaginazione: a cura degli Autori
Grafica di copertina: G & G
Stampa: Cromografica Europea, Rho (MI)

ISBN 88-386-6228-2
Printed in Italy
123456789CRORRV98765

A	Teoria della computabilità	1
1	Introduzione	3
1.1	Breve preistoria della computabilità	3
1.2	Problemi di matematica	5
1.3	Un decalogo per i bravi algoritmi	7
2	Le Macchine di Turing	11
2.1	Macchine che calcolano	11
2.2	Alfabeti, stringhe, linguaggi	12
2.3	La Macchina di Turing	15
2.4	Macchine di Turing e linguaggi	20
2.5	Macchine di Turing e funzioni	22
2.6	Codifiche di stringhe	23
2.7	Numeri e coppie	26
2.8	Numeri e macchine	28
2.9	La Tesi di Church-Turing	31
2.10	Macchine di Turing a più nastri	33
2.11	Macchine di Turing non deterministiche	37
2.12	Esercizi	39
3	Problemi senza Soluzione	41
3.1	Diagonalizzare e ridurre	41
3.2	Problemi risolvibili algoritmicamente	42
3.3	La Macchina Universale	44
3.4	Il Problema dell'Arresto	45
3.5	Insiemi decidibili e semidecidibili	49
3.6	Un'altra funzione non calcolabile	52
3.7	Il Decimo Problema di Hilbert	56
3.8	I teoremi di Kleene e di Rice	58
3.9	Esercizi	61

4	Funzioni Ricorsive	63
4.1	Calcolabilità secondo Church	63
4.2	Le funzioni parziali ricorsive	64
4.3	Esempi a volontà	68
4.4	Church o Turing?	71
4.5	Esercizi	74
5	Calcolabilità e Grammatiche	77
5.1	Grammatiche e Automi	77
5.2	Linguaggi	78
5.3	Grammatiche e Linguaggi	79
5.4	La Gerarchia di Chomsky	81
5.5	Alberi di Derivazione	83
5.6	Linguaggi regolari	85
5.7	Linguaggi Liberi dal Contesto	89
5.8	Linguaggi Dipendenti dal Contesto	93
5.9	Linguaggi e Macchine di Turing	96
5.10	Automi di Riconoscimento	98
5.11	Esercizi	100
6	Calcolabilità e Linguaggi di Programmazione	103
6.1	Il linguaggio WHILE	103
6.2	La Sintassi del Linguaggio WHILE	103
6.3	La Semantica dei linguaggi WHILE	108
6.4	Funzioni WHILE-Calcolabili	110
6.5	Programmi e Macchine di Turing	111
6.6	Il Teorema di Kleene per i Programmi	112
6.7	Esercizi	114
B	Teoria della complessità	117
7	Complessità	119
7.1	Introduzione	119
7.2	Come misurare la complessità	123
7.3	Un esempio	128
7.4	Esercizi	130
8	Classi di Complessità Temporale	133
8.1	La classe P e la Tesi di Edmonds-Cook-Karp	133
8.2	La classe NP	143
8.3	Un'altra caratterizzazione di NP: le macchine di Turing non de-terministiche	149
8.4	Il problema $P = NP$	151
8.5	Problemi NP-completi	152
8.6	Il teorema di Cook-Levin	154

8.7	Altri problemi NP -completi	159
8.8	P e NP : qualche commento ulteriore	165
8.9	coNP e la gerarchia polinomiale	167
8.10	Oracoli	174
8.11	Tempi esponenziali	175
8.12	P = NP nella pratica	175
8.13	Esercizi	180
9	Complessità, Logica e Circuiti	183
9.1	Un po' di logica	183
9.2	Ancora logica: le formule Booleane quantificate	190
9.3	Circuiti	191
9.4	Circuiti e complessità	198
9.5	Circuiti e NP -completezza	204
9.6	Esercizi	204
10	Classi di Complessità Spaziale	207
10.1	Il parametro spazio	207
10.2	PSPACE e L	210
10.3	NPSPACE e NL	212
10.4	Il teorema di Savitch	216
10.5	NL = coNL	219
10.6	PSPACE -completezza	221
10.7	Esercizi	226
11	Classi di Complessità Probabilistiche	229
11.1	Probabilmente primi	229
11.2	Montecarlo o Las Vegas?	233
11.3	La classe PP , e altre variazioni sul tema	234
11.4	BPP e NP	246
11.5	Esercizi	250
12	Contare e Approssimare	251
12.1	Soddisfacibilità unica	251
12.2	Contare	252
12.3	Approssimare	256
12.4	Il teorema di Valiant-Vazirani	261
12.5	Esercizi	267
13	Algoritmi Interattivi	269
13.1	Artù e Merlino	269
13.2	La classe IP	275
13.3	Il teorema di Shamir	281
13.4	Un tentativo di conclusione	286
13.5	Esercizi	288

C	Teoria quantistica della computazione	289
14	Dal Bit al QuantumBit	291
14.1	Nozioni preliminari	291
14.2	I postulati della Meccanica Quantistica	295
14.3	Qubits versus Bits	300
14.4	Esercizi	300
15	Modelli di Computazione Quantistica	303
15.1	Dalla macchina di Turing classica a quella quantistica	303
15.1.1	Macchina di Turing reversibile	304
15.1.2	Macchina di Turing quantistica	305
15.2	Classi di complessità quantistiche	310
15.3	Porte logiche quantistiche	312
15.4	Circuiti quantistici	315
15.4.1	Insiemi universali	315
15.4.2	Aritmetica con circuiti quantistici	320
15.5	Esercizi	321
16	Algoritmi Quantistici	323
16.1	Algoritmo di Deutsch-Jozsa	323
16.2	Algoritmo di Simon	325
16.3	Trasformata di Fourier quantistica	326
16.4	Algoritmo di fattorizzazione (Shor)	330
16.4.2	Il caso generale	334
16.4.4	Il problema del sottogruppo nascosto	338
16.5	Algoritmo di ricerca (Grover)	338
16.5.1	Interpretazione geometrica	340
16.6	I limiti della computazione quantistica	342
16.7	Conclusioni	345
16.8	Esercizi	346

Premessa

Il libro introduce e discute due argomenti fondamentali, che hanno accompagnato e anzi preceduto la nascita e lo sviluppo dell'Informatica e dei moderni "calcolatori":

- 1) che cosa si può calcolare?
- 2) che cosa si può calcolare a costi accessibili?

L'esistenza di problemi che non si possono risolvere in alcun modo, oppure richiedono risorse praticamente indisponibili per la loro soluzione, avvalora l'interesse ad approfondire e possibilmente chiarire le due questioni. A esse si aggiunge in modo naturale una terza domanda:

- 3) può il progresso delle teorie fisiche intervenire anche sugli aspetti teorici della computazione?

Il libro affronta nell'ordine i tre argomenti, dedicando a ciascuno una delle sue parti:

- 1) tratta dapprima la *teoria della computabilità*, fornisce e confronta varie possibili risposte alla prima domanda, da quelle classiche (macchine di Turing, grammatiche, funzioni ricorsive) ad altre più recenti, legate all'evoluzione dei linguaggi di programmazione;
- 2) considera poi la *teoria della complessità computazionale*, discute cioè il tema dei costi di una computazione, dei possibili criteri (tempo, memoria, casualità) che li misurano, delle varie classi di problemi che si possono conseguentemente formare e delle relazioni che intercorrono tra queste classi;
- 3) introduce infine l'argomento relativamente nuovo della *computazione quantistica*, mostrando come i progressi della meccanica quantistica possano indurre a profonde revisioni e rivisitazioni del concetto di complessità.

Le tre parti del libro corrispondono in modo naturale a tre corsi separati e successivi secondo l'attuale ordinamento universitario:

- 1) è argomento per un corso di primo livello in una Laurea Triennale in Informatica, ma interessa anche Matematica e Fisica;
- 2) fornisce materiale per un corso specialistico di Informatica, o eventualmente per un corso finale della laurea triennale in Informatica, ma, di nuovo, si adatta senza difficoltà a lauree in Matematica e Fisica;
- 3) è la base per corsi specialistici o di dottorato, ancora in Fisica, Matematica, Informatica.

Intersezioni con lauree in Ingegneria Informatica sono anche possibili. Anche i prerequisiti richiesti per le tre parti variano; in particolare le nozioni e gli strumenti di Matematica e Fisica necessari per 3) sono ovviamente superiori a quelli richiesti in precedenza.

Gli autori confidano che il loro lavoro possa riuscire di utilità e interesse a quanti vorranno considerarlo.

Parte A

Teoria della computabilità

1.1 Breve preistoria della computabilità

Il nostro libro si interessa anzitutto del tema della *computabilità*: parola forse impegnativa e difficile, e pur tuttavia ovviamente collegata all'informatica, capace di richiamare l'idea del *computer* e del *calcolatore*. Vale allora la pena di spiegarne subito la rilevanza, anche per introdurre in maniera adeguata i temi della prima parte del libro.

Nella accezione originaria e comune, *computare*, o *calcolare*, significa far di conto con i numeri usuali, e cioè gli interi $0, \pm 1, \pm 2, \dots$: sommarli, moltiplicarli, semmai sottrarli e dividerli. Per chi ha qualche maggior dimestichezza con un po' di matematica, l'arte di far di conto si può estendere ad operazioni più sofisticate e a contesti più ampi (limiti, derivate, integrali). Ma, in realtà, tutti i problemi che si prestano a formalizzazioni tramite modelli matematici possono essere "computati". Già René Descartes (Cartesio), secoli fa, all'inizio del Seicento, vagheggiava la possibilità di tradurre in termini matematici ogni problema, e dunque di risolverlo tramite equazioni e computazioni; applicava poi l'intuizione al campo geometrico, identificando, ad esempio, punti del piano con coppie ordinate di numeri (reali), rette del piano con equazioni di primo grado in due incognite, e così via. Pochi anni dopo, nel 1666, Gottfried Wilhelm Leibniz esortava appassionatamente "*Calculemus*" (calcoliamo!), invitando ad usare conti e computazioni anche per risolvere oggettivamente gli stessi conflitti umani e condurre a termine le controversie trasferendole "*dai ragionamenti complicati ai calcoli semplici, dai vocaboli di significato incerto e vago a caratteri determinati*". Del resto lo stesso Leibniz provvedeva a chiarire di intendere per *calcolo* "*qualunque notazione che rappresenti il ragionamento, quand'anche non avesse alcun rapporto con i numeri*".

A queste applicazioni computazionali, rivolte ad orizzonti sempre più larghi, si accompagnava il tentativo di costruire macchine calcolatrici capaci di aiutare, simulare e, talora, sostituire, la mente umana nel suo lavoro. Proprio al Seicento risalgono vari esempi di meccanismi automatici per fare di conto e svolgere almeno le tradizionali operazioni di somma, prodotto, sottrazione e divisione. Possiamo così ricordare che proprio Leibniz aveva concepito nel 1671 una *ruota* adatta a sviluppare questo genere di computazione, preceduto comunque in questo da

Blaise Pascal e da Wilhelm Schickard che, rispettivamente nel 1643 e nel 1623, avevano inventato analoghi procedimenti meccanici. Nei secoli successivi ci furono anche casi di macchine capaci di svolgere computazioni fuori dal solo ambito numerico. Si parla, ad esempio, di un automa di Kempelen capace di giocare a scacchi (1768) e di anticipare in questo senso il moderno *Deep Blue* (il computer IBM capace di sconfiggere qualche anno fa anche il campione del mondo degli scacchi): ma, nel caso di Kempelen, non fu mai chiaro quale fosse il segreto programma della macchina e ci fu chi fondatamente dubitò trattarsi soltanto di un imbroglio ben riuscito (i lettori appassionati di libri polizieschi potranno trovare ampia discussione dell'argomento nel giallo "*L'Automa*" di J. Dickson Carr). Nell'Ottocento, tuttavia, Charles Babbage concepì teoricamente un meccanismo automatico, da lui chiamato *macchina analitica* (*analytical engine*), virtualmente capace di adattarsi non solo a numeri e a mosse di scacchi, ma ad ogni possibile contesto, dunque una sorta di hardware universale, che Ada Lovelace Byron si preoccupò in quegli stessi anni di corredare di quel che oggi chiameremmo il software.

A questi progressi tecnici dovevano del resto accompagnarsi corrispondenti approfondimenti teorici, a proposito di una questione che diventa fondamentale non appena si affronta il tema del rapporto uomo-macchina: si tratta infatti di

- identificare un linguaggio astratto appropriato, con simboli opportuni, in cui formulare i problemi (matematici e non) da risolvere, per poterli poi proporre alla macchina che deve computarli,
- individuare, ancora, le leggi che la macchina deve seguire per svolgere i suoi calcoli.

Entrambe le questioni sono tutto men che trascurabili. Oggi, ad esempio, siamo abituati ad usare le cifre 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 per rappresentare i numeri interi, e i simboli +, ·, −, : per indicare le loro usuali operazioni. Ma la genesi di questi simboli non è stato processo banale ed ha richiesto talora il progresso di secoli; del resto, gli antichi Greci e Romani non utilizzavano ancora questi caratteri, ma altri ben più complicati. Ancora nel Rinascimento, o nello stesso Seicento, queste notazioni si affacciavano timidamente. Anzi, proprio a Leibniz si attribuisce l'idea di un linguaggio universale ("*lingua characteristica*" nella denominazione latina da lui adoperata) con cui formulare le comunicazioni scientifiche tra uomini di idiomi diversi, e dunque anche tra uomo e macchina.

Sempre Leibniz propugnò l'idea di un calcolo della ragione ("*calculus ratiocinator*" in latino) atto ad individuare le leggi fondamentali di sviluppo del pensiero (e delle computazioni), utile dunque anche nella programmazione di nuove macchine pensanti. Come si vede, siamo qui agli albori di quelle che oggi si chiamano *intelligenza artificiale* e *deduzione automatica*, della capacità, cioè, di sviluppare calcoli e pensieri in modo meccanico, dunque delegabile ai "*computers*". Del resto, l'idea di uno studio dei procedimenti del pensiero (quello che in termini ufficiali si chiama Logica) è ben precedente Leibniz, e risale almeno ad Aristotele e al 300 avanti Cristo. Il *calculus ratiocinator* fu comunque ampiamente sviluppato già nell'Ottocento da George Boole, che formulò una sorta di teoria algebrica del pensiero, oggi comunemente conosciuta proprio come *algebra Booleana*.

Come si vede, il tema della *computabilità* si collega a svariati argomenti come

- le macchine calcolatrici per attuare le computazioni,
- i linguaggi e le regole con cui favorire la loro collaborazione,

ed altri ancora. Tra l'altro, è interessante osservare come queste riflessioni anticipino largamente la nascita dei moderni *computers* e si sviluppino nei secoli precedenti. Infatti i primi *calcolatori* (come oggi comunemente li intendiamo) risalgono a tempi relativamente recenti, al periodo della seconda guerra mondiale e agli anni immediatamente successivi se è vero, come è vero, che il primo computer elettronico, l'*ENIAC* (architettato da Von Neumann) è del 1946.

La computabilità che vogliamo qui trattare si interessa ovviamente a tutte queste problematiche, ma vuole anche sottolinearne un aspetto ulteriore. In effetti, il suo ideale punto di partenza e, se vogliamo, l'anno stesso di nascita della moderna Informatica Teorica (se pure ha senso istituire un'anagrafe delle correnti scientifiche) si colloca in una data intermedia, che segue tutti i germi computazionali che abbiamo descritto, ma precede di una decina di anni la nascita di *ENIAC*. Infatti, la data che attira il nostro interesse è il 1936. Nel prossimo paragrafo tenteremo di spiegare il perchè.

1.2 Problemi di matematica

Apriamo una piccola parentesi, e parliamo ancora dei numeri interi $0, \pm 1, \pm 2, \pm 3, \dots$ e delle operazioni $+, \cdot, -$ che li riguardano. In particolare, costruiamo attraverso $+, \cdot, -$ polinomi a coefficienti interi, come

$$2x + 3, \quad x + 2, \quad x^2 + y^2 + 1, \quad x^2 + y^2 - 2, \quad x^2 - 2, \quad x^2 + 1$$

e così via. Notiamo che alcuni di questi polinomi (il secondo e il quarto, per la precisione) hanno anche radici intere: ad esempio:

- $x + 2 = 0$ è risolto da $x = -2$,
- $x^2 + y^2 - 2 = 0$ da $x = \pm 1, y = \pm 1$.

Tuttavia gli altri polinomi della lista, pur avendo coefficienti interi, non hanno radici intere:

- non $2x + 3$, perché 3 è dispari;
- non $x^2 - 2, x^2 + 1$, perché 2 e -1 non hanno radici quadrate tra gli interi;
- non $x^2 + y^2 + 1$ per analoghi motivi (per ogni scelta di x, y interi, si ha $x^2 + y^2 + 1 > x^2 + y^2 \geq 0$ dunque $x^2 + y^2 + 1$ non si annulla mai tra gli interi).

Anzi, proprio per ovviare a queste deficienze si è portati ad allargare l'ambito numerico degli interi e formare rispettivamente gli insiemi dei razionali (con $\frac{3}{2}$), dei reali (con $\sqrt{2}$), dei complessi (con $i = \sqrt{-1}$).

Comunque, rimanendo nell'ambito dei polinomi a coefficienti interi, potrebbe interessarci distinguere quali tra essi hanno anche radici intere, e quali no. Si tratta di questione tutto men che banale se è vero, come è vero, che un matematico illustre come David Hilbert la segnalò nel 1900 in una lista di 23 problemi matematici meritevoli a suo avviso di lavoro e di attenzione. Più precisamente la questione era collocata al posto 10 nella lista.

Decimo Problema di Hilbert. Determinare un procedimento capace di stabilire, per ogni polinomio a coefficienti interi, se esso ha o no radici intere.

Nel linguaggio odierno ci viene dunque richiesto un algoritmo che abbia

- INPUT: un polinomio a coefficienti interi;
- OUTPUT: SÌ/NO, secondo che il polinomio abbia o no radici intere.

Si noti che il polinomio non ha limitazioni né sul grado (1, o 2, o anche maggiore) né sul numero delle variabili coinvolte (x , eventualmente y , ma anche $z, t, \dots$).

Il decimo problema di Hilbert, formulato, come detto, nel 1900, non ebbe certo soluzione nel 1936. Anzi, si dovette attendere il 1970 perché potesse trovare una risposta per molti versi inattesa e sorprendente. Ma gli sviluppi del 1936 ebbero un ruolo fondamentale in questa soluzione del 1970. Vediamo perché, e collochiamoci idealmente negli anni '30, tre decenni dopo la proposizione del quesito di Hilbert.

Di fronte alla difficoltà di determinare l'algoritmo richiesto, si poteva reagire in due modi:

- pensare che i tempi (e le conoscenze matematiche) non erano ancora così maturi da permetterne la soluzione, e che ulteriori progressi scientifici erano richiesti;
- dubitare che l'algoritmo potesse davvero trovarsi e che nessuno, nel 1930 e negli anni successivi, fosse effettivamente in grado di concepirlo.

La prima reazione sembra più ragionevole e realistica; la seconda, più pessimista, aveva comunque all'epoca fondate motivazioni. Infatti altre questioni di matematica e logica condividevano la stessa situazione del Decimo Problema di Hilbert: una soluzione che tardava a venire, ostacolata da formidabili complicazioni tecniche e teoriche. Ad esempio, era questo il caso del fondamentale problema di logica che lo stesso Hilbert aveva sollevato, chiamandolo *Entscheidungsproblem* (problema di decisione). Chi ha dimestichezza con la Logica Matematica ne conosce i termini; gli altri lettori saranno, se non altro, soddisfatti di sapere che esso chiede di riconoscere quali affermazioni di un usuale linguaggio logico (quello denominato "del primo ordine") sono da ritenersi valide (cioè universalmente accettabili) e quali no.

Del resto, a favore della visione più "pessimista" contribuiva in quegli anni '30 la fresca dimostrazione (proprio del 1931) di due famosissimi risultati di Logica Matematica - i Teoremi di Incompletezza di Kurt Gödel - che stabilivano in qualche modo l'incapacità umana di cogliere i reali fondamenti dei numeri interi con le loro usuali operazioni $+$, $\cdot$, e conseguentemente l'impossibilità di delegare

completamente la ricerca matematica alle macchine, secondo le idee accennate nel paragrafo precedente.

Non ci attardiamo comunque su queste pur affascinanti connessioni e torniamo al nostro problema delle radici intere, e alla possibilità di una risposta negativa del tipo: “*l’algoritmo richiesto non esiste*”. Ora, per convincere che un algoritmo per il Decimo problema di Hilbert c’è, basta proporlo: stabilirne le istruzioni e verificarne la correttezza. Invece, per escluderne l’esistenza, non ci basta formulare qualche tentativo più o meno maldestro e poi denunciarne il fallimento. Dobbiamo infatti stabilire una volta per tutte che nessuno, nel 1930 o oggi o in futuro, saprà mai escogitare un procedimento di calcolo. Ma allora dobbiamo anticipatamente convenire una qualche definizione generale di

- ciò che è computabile (il problema)

e, parallelamente, di

- chi computa (l’algoritmo che affronta il problema e la macchina che lo svolge),
- come si computa (le regole che algoritmo e macchina devono rispettare);

dimostrare conseguentemente che il Decimo problema di Hilbert, o l’*Entscheidungsproblem* non rientrano in questa classe di questioni “*calcolabili*” (se davvero questa è la loro sorte). Così la questione si allarga dai contesti particolari dei numeri interi o delle proposizioni logiche, e va ad introdurre temi basilari di informatica:

- che cosa è un “calcolatore”?
- quali sono i problemi che i calcolatori sanno (o non sanno) risolvere?

Fu nel 1936 che queste domande ebbero una prima risposta grazie a personaggi illustri degli albori dell’informatica moderna, come Turing, Church, Kleene, Gödel e altri.

1.3 Un decalogo per i bravi algoritmi

Il grande scrittore di fantascienza Isaac Asimov fissò nei suoi racconti le 3 leggi fondamentali che dovevano regolare il comportamento dei robot nei loro rapporti con l’Uomo, ispirandole a principi inderogabili di rispetto, obbedienza e collaborazione. Il nostro obiettivo qui è per certi versi analogo. Come già detto, vogliamo infatti descrivere in modo rigoroso come avviene una computazione e precisare:

- anzitutto chi computa (l’algoritmo che la svolge e la macchina su cui si svolge),
- ma anche e soprattutto **come** si computa (le regole che algoritmo e macchina devono rispettare nel loro lavoro);

formulare in conclusione una specie di decalogo del bravo algoritmo e del bravo calcolatore. Dunque il nostro contesto richiama in qualche senso le leggi dei robot androidi di Asimov. Ma per ottenere il nostro obiettivo seguiamo, più che i

romanzi di fantascienza, le riflessioni che gli autorevoli scienziati citati alla fine dello scorso paragrafo, e tra essi segnatamente Alan Turing, svolsero intorno al 1936.

Notiamo anzitutto che, per calcolare un dato problema, serve anzitutto un alfabeto appropriato con cui formalizzarne i contenuti (una *lingua caratteristica*, per dirla alla Leibniz): una sequenza finita di simboli che descrivano i termini della questione. Questo alfabeto può essere semplicemente quello della lingua italiana, o magari quello della lingua scientifica dominante (l'inglese), ma può anche talora allargarsi in ragione del contesto. Ad esempio, se trattiamo i polinomi a coefficienti interi, come nel caso del Decimo Problema di Hilbert, è bene che prevediamo anche i simboli 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, ·, −. Altri ambiti ci potranno poi suggerire alfabeti ancor più specialistici e mirati.

Concordato dunque il nostro alfabeto, possiamo formulare in modo adeguato il nostro problema e trasmetterlo all'algoritmo e alla macchina calcolatrice per la loro computazione.

A questo punto dobbiamo finalmente pensare a concordare con precisione a quali modelli di algoritmo e di macchina intendiamo ricorrere. In questo possiamo fare momentaneo riferimento alla nostra esperienza personale e a un'idea intuitiva di algoritmo e di calcolatore, per immaginare quali regole di buon comportamento ci aspettiamo che essi rispettino durante il loro lavoro. Semmai, nei casi più delicati ed incerti, proviamo a riferirci al modo di agire della mente umana come possibile termine di paragone. Possiamo allora convenire quanto segue.

1. Un algoritmo è di lunghezza finita.
2. Esiste un agente di calcolo (la *macchina calcolatrice*, appunto) che sviluppa la computazione eseguendo le istruzioni dell'algoritmo.
3. L'agente di calcolo ha a disposizione una memoria, dove vengono immagazzinati i risultati intermedi del calcolo.
4. Il calcolo avviene per passi discreti.
5. Il calcolo non è probabilistico.

I punti 1-3 hanno un'ovvia interpretazione. Il punto 4 afferma che il calcolo non avviene mediante dispositivi analogici, ma si svolge ordinatamente, un passo dietro l'altro. Il punto 5 afferma che il calcolo non è soggetto a fattori casuali, ma si svolge in modo assolutamente preciso e rigoroso (o anche, come in genere si dice, "*deterministico*"). Altre caratteristiche degli algoritmi sono le seguenti.

6. Non deve esserci alcun limite finito alla lunghezza dei dati di ingresso.
7. Allo stesso modo, non deve esserci alcun limite alla quantità di memoria disponibile.

Il punto 6 ci dice che l'input del problema può essere arbitrariamente lungo. La condizione è assolutamente ragionevole: ad esempio, un algoritmo di somma tra gli interi si deve applicare ad ogni possibile addendo, quindi ad ogni numero intero, comunque grande. Anche il punto 7 è meritevole di qualche chiarimento. Asserisce la possibilità di accedere ad una memoria potenzialmente illimitata. Per sostenere la plausibilità di questa assunzione, proviamo a pensare che cosa accade in caso contrario, se ammettiamo di limitare preliminarmente ad un livello

uniforme prefissato le potenzialità di memoria. In queste condizioni può capitare che algoritmi anche elementari, costruiti per eseguire semplici computazioni, si trovino talora nell'impossibilità di completarle. Ad esempio, la funzione che ad ogni intero associa il suo quadrato $f(x) = x^2$ non è più calcolabile, perchè lo spazio di memoria necessario per computare il quadrato di x dipende ovviamente da x e dunque, per x molto grande, si trova ad infrangere i limiti eventualmente prefissati. Così 7 risulta del tutto plausibile.

Anche le seguenti osservazioni sono essenziali per cogliere la natura del calcolo.

8. Deve esserci un limite finito alla complessità delle istruzioni eseguibili dal dispositivo.
9. Il numero di passi della computazione è finito ma non limitato.
10. Sono ammesse computazioni senza fine.

Il punto 8 ribadisce, insieme al punto 1, l'intrinseca finitezza del dispositivo di calcolo. Ci deve essere una limitazione finita non solo per il numero delle istruzioni di un algoritmo (come 1 sostiene), ma anche per la complessità delle singole istruzioni; in altre parole, solo una porzione finita della memoria dell'agente di calcolo deve essere occupata da queste istruzioni iniziali al momento in cui la computazione su un dato input si avvia. La condizione non contraddice la condizione 7, che riguarda i successivi passi del calcolo e afferma la possibilità di accoglierne senza limite le informazioni nella restante porzione di memoria. D'altra parte 8 si può intuitivamente giustificare proprio in riferimento alle caratteristiche della mente umana, considerandone l'intrinseca limitatezza (ma anche le possibilità potenzialmente illimitate di crescita).

Il punto 9, poi, ci dice che non è possibile stabilire a priori un limite massimo per il numero dei passi richiesti per eseguire un generico algoritmo su un certo input. Il punto 10, infine, fa riferimento all'eventualità di dover affrontare problemi senza soluzione, come quelli accennati nei precedenti paragrafi: in questi casi una computazione è destinata a prolungarsi indefinitamente senza riuscire a produrre risposte soddisfacenti.

Queste sono le condizioni che possiamo ragionevolmente richiedere ad un qualunque sistema di computazione (che includa sia il programma di calcolo che la macchina che lo esegue). Sono 10, e quindi completano il nostro decalogo nel senso stretto del termine. Sono anche sufficienti ad individuare il nostro obiettivo. Infatti, proprio a partire da queste riflessioni, Alan Turing affrontò nel 1936 il problema di definire rigorosamente che cosa possa intendersi per *computabile*, proponendo un semplicissimo modello di calcolatore ante litteram (la *macchina di Turing*, appunto) e sostenendo che computabile è esattamente quanto una macchina di Turing riesce a computare; dimostrò conseguentemente l'impossibilità di risolvere, ad esempio, l'Entscheidungsproblem di Hilbert, perchè non ci sono macchine di Turing che lo soddisfino e, di conseguenza, neppure algoritmi di altro genere con questa capacità. Tra parentesi, potrà essere interessante anticipare che qualche anno dopo, nel 1970, anche il Decimo Problema di Hilbert ottenne la stessa risposta negativa, come racconteremo in maggior dettaglio nelle prossime pagine.

In realtà, nei capitoli che verranno, avremo modo di presentare, descrivere e discutere vari modelli di computazione, anzitutto quello di Turing, ma anche altri, classici come quello delle *funzioni ricorsive* o delle *grammatiche*, ed altri relativamente più recenti, collegati ai linguaggi di programmazione. Avremo modo di confrontare questi approcci e di mostrarne la sostanziale equivalenza. Incontreremo anche (come già anticipato nelle scorse righe) casi di funzioni che non si possono calcolare e di problemi che non si possono risolvere (perché estranei a qualunque trattamento mediante i nostri modelli di calcolo). La teoria della *computabilità* si interessa principalmente a tutte queste tematiche; in questo senso precorre ed inaugura, come già accennato, la moderna *Informatica Teorica*; concorre ancora sostanzialmente al suo sviluppo.

Le Macchine di Turing

2.1 Macchine che calcolano

Un processo di calcolo è finalizzato ad elaborare informazioni. Può essere semplice quanto una stima della durata di un percorso tra due città, e complicato quanto una previsione metereologica. Lo studio della *calcolabilità* si prefigge l'obiettivo di fornire l'intuizione degli elementi e delle proprietà salienti che caratterizzano il processo. Tale intuizione può essere usata per predire la complessità della computazione, per scegliere in modo consapevole un tipo di calcolo piuttosto che un altro, e per sviluppare strumenti che facilitino il progetto del processo stesso.

Lo studio della computabilità rivela addirittura che ci sono problemi che non possono avere risposta; inoltre, ci mostra che tra i problemi che invece possono essere risolti, ce ne sono alcuni che richiedono risorse così ingenti (ad esempio, un tempo di calcolo dell'ordine del milione di anni) da scoraggiare ogni tentativo di soluzione pratica. Lo studio della computabilità riesce a identificare queste situazioni; permette così di determinare con opportuni strumenti i casi "positivi" in cui la soluzione si ottiene, magari anche a costi accessibili.

In questa prima parte del libro ci concentreremo in realtà solo sul primo aspetto, ci dedicheremo cioè a distinguere quali problemi si possono risolvere, e quali no. Presenteremo quattro modelli astratti di calcolo che permettono questa identificazione: nell'ordine, *macchine di Turing*, *funzioni ricorsive*, *grammatiche*, *programmi*. Non faremo invece alcun riferimento al tema della complessità computazionale, non ci interesseremo dunque dei costi dei problemi che hanno soluzione: questo aspetto sarà ampiamente trattato nella seconda parte del libro.

In particolare, in questo capitolo verranno inizialmente introdotti le nozioni di *stringa* e di *linguaggio*, e il ruolo che essi hanno nel rappresentare l'informazione. Verrà poi presentato il formalismo delle Macchine di Turing: dapprima sarà definita la versione più semplice, si passerà poi a varianti più complesse, utili per approfondire aspetti particolari del concetto di calcolo. Verranno poi presentate le nozioni di calcolabilità e di decidibilità indotte dal modello della macchina di Turing.

2.2 Alfabeti, stringhe, linguaggi

L'abilità nel rappresentare l'informazione è cruciale nel processo di calcolo dell'informazione stessa. Le società umane hanno creato linguaggi parlati per comunicare ad un livello elementare, e hanno sviluppato il metodo della scrittura per raggiungere un livello ben più sofisticato. La lingua italiana, ad esempio, nella sua forma parlata si basa su un insieme finito di suoni elementari, le parole sono definite in termini di sequenze finite di tali suoni, le frasi vengono derivate da sequenze finite di parole, i discorsi sono ottenuti da sequenze di frasi, e così via. L'italiano scritto usa un insieme finito di simboli come insieme di primitive, le parole sono definite da sequenze finite di simboli, i periodi vengono derivati da sequenze finite di parole, i paragrafi da sequenze finite di periodi, e così via. Approcci simili sono stati sviluppati anche in altri contesti; per esempio, abbiamo già visto nel precedente capitolo che un numero naturale può essere rappresentato come una sequenza finita di cifre decimali 0, 1, 2, 3, 4, 5, 6, 7, 8, 9; ma anche alternativamente in forma binaria cioè come sequenza finita di bit 0, 1, quando

$$0, 1, 2, 3, 4, 5, \dots, 10, 11, 12, \dots$$

diventano rispettivamente

$$0, 1, 10, 11, 100, 101, \dots, 1010, 1011, 1100, \dots$$

Ci sono del resto altre possibili rappresentazioni dei naturali, rispetto a basi ancora diverse; ad esempio un naturale si può esprimere in modo molto rozzo con l'uso del solo simbolo 1, come sequenza finita di 1: in questo caso

$$0, 1, 2, 3, \dots$$

diventano

$$1, 11, 111, 1111, \dots$$

e così via.

Se poi parliamo di polinomi a coefficienti interi, possiamo espandere i simboli precedenti accogliendo $+$, $-$, x , y , z , t , $\dots$ e costruire l'oggetto della nostra attenzione - i polinomi, appunto - come sequenze finite di tutti questi elementi.

Ovviamente ci si aspetta che i processi di calcolo trattino l'informazione anche in ambiti più generali: infatti essi manipolano non solo interi, ma anche grafi, programmi, e molti altri tipi di entità. Comunque questi oggetti possono essere tutti descritti come sequenze finite di lettere su opportuni alfabeti; dunque possiamo assumere in astratto che i processi di calcolo considerino stringhe di *simboli* scelti in un insieme finito e non vuoto (che viene in genere chiamato *alfabeto*).

Una sequenza finita di simboli di un dato alfabeto A viene chiamata *stringa* o *parola* su A . La stringa, formata dalla sequenza di simboli $a_1, a_2, \dots, a_n$, viene usualmente denotata $a_1 a_2 \dots a_n$. Ammettiamo anche l'eventualità di una stringa costituita da nessun simbolo, chiamata *stringa vuota* e denotata dalla lettera greca λ .

Esempio. $A = \{a, b, c, \dots, z\}$ e $B = \{0, 1, 2, \dots, 9\}$ sono alfabeti. abb è una stringa su A , mentre 123 è una stringa su B . $ba12$ non è una stringa su A , in quanto contiene simboli non in A . Analogamente, l'allineamento delle cifre decimali di π $1415\dots$ non è una stringa su B , perchè non è una sequenza finita.

Ribadiamo che un alfabeto A non è mai vuoto: infatti, per costruire stringhe, dobbiamo avere dei simboli che le compongano. Inoltre si assume di solito A finito. In realtà, già nei casi del Decimo problema di Hilbert e dei polinomi a coefficienti interi, si ammettono infinite indeterminate $x, y, z, t, \dots$ e dunque infiniti simboli; ma si può facilmente rimediare con qualche artificio, ad esempio adoperando per le indeterminate due soli simboli $x, |$ e convenendo di rappresentare $x, y, z, t, \dots$ come

$$x, x|, x||, x|||, \dots$$

cioè come stringhe su $x, |$.

La lunghezza di una stringa α - denotata $l(\alpha)$ - è il numero dei simboli che la compongono: per $\alpha = a_1 a_2 \dots a_n$, $l(\alpha) = n$.

Esempio. aa è una stringa di lunghezza 2; $l(\lambda) = 0$; $l(ab) + l(b) = 3$.

Date due stringhe α, β se ne può formare un'altra in cui α è seguita da β : essa è denotata $\alpha\beta$ e chiamata la *concatenazione* di α e β . La notazione α^i è usata per la stringa ottenuta concatenando i copie della stringa α quando i è un intero positivo. Così $\alpha^2 = \alpha\alpha$, $\alpha^3 = \alpha\alpha\alpha$ e via dicendo. Si intende poi $\alpha^1 = \alpha$ e $\alpha^0 = \lambda$. La concatenazione si applica ovviamente anche alle lettere di A , viste come parole su A , e anzi rappresenta la via secondo cui esse generano le altre stringhe.

Esempio. La concatenazione di ab con baa produce la stringa $abbaa$, quella di baa con ab determina $baaab$, cioè ba^3b . Le concatenazioni $\lambda\alpha$ e $\alpha\lambda$, per ogni stringa α , producono la stessa stringa α . In particolare, $\lambda\lambda = \lambda$.

Diciamo che una stringa α è una *sottostringa* di β se $\beta = \gamma\alpha\rho$, per qualche scelta di stringhe γ e ρ . Una sottostringa α di una stringa β si chiama *prefisso* di β se $\beta = \alpha\rho$ per qualche stringa ρ (dunque per $\gamma = \lambda$); α è un *prefisso proprio* se $\rho \neq \lambda$. Una sottostringa α di una stringa β è un *suffisso* di β se $\beta = \gamma\alpha$ per qualche stringa γ ; α è un *suffisso proprio* se $\gamma \neq \lambda$.

Esempio. λ, a, b, ab, abb sono sottostringhe di abb . λ, a, ab sono prefissi propri di abb . λ, b, bb sono suffissi propri di abb . abb è prefisso e suffisso di abb .

Se $\alpha = a_1 \dots a_n$, allora $a_n \dots a_1$ è chiamata l'*inversa* di α e viene denotata α^{rev} . L'insieme di tutte le stringhe su un alfabeto A viene indicato A^* , mentre A^+ denota l'insieme $A^* - \{\lambda\}$ delle stringhe non vuote su A .

Un alfabeto A , come insieme finito di simboli, si può sempre ordinare in più modi possibili. Sarà spesso conveniente nel seguito di questo libro fissare un tale ordinamento totale $<$ e, per $A = \{a_1, \dots, a_n\}$, assumere ad esempio

$$a_1 < a_2 < \dots < a_n.$$

In realtà è bene concordare sin d'ora un qualche ordinamento anche per le parole su A . Infatti in Scienza dell'Informazione si affrontano comunemente tecniche di ricerca sequenziale e binaria, *insertion sort*, *quick sort* e *merge sort* che trattano stringhe e si riferiscono a un loro prefissato ordinamento. Una strategia frequentemente usata è quella *lessicografica*, definita nel modo che segue.

Definizione. Sia A un alfabeto (ordinato da qualche relazione $\leq$) e siano α e β stringhe in A^* . Si dice che α è *lessicograficamente minore* di β , $\alpha < \beta$, o equivalentemente β *lessicograficamente maggiore* di α , $\beta > \alpha$, se vale uno dei due casi:

- a. α è prefisso proprio di β ,
- b. α ha un prefisso γa , β ha un prefisso γb , per lo stesso $\gamma \in A^*$, con $a, b \in A$ e $a < b$ in A .

Esempi.

1. Se A è l'alfabeto della lingua italiana, ordinato ponendo $a < b < c < \dots$, l'ordine lessicografico in A^* non è quello comune dei vocabolari; infatti $aba < abb$ perché $a < b$, e abb precede $abba$ perché ha lunghezza minore, proprio come nei dizionari, ma z precede ab perché ha lunghezza minore.
2. Consideriamo ora l'alfabeto $A = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ dove si fissa, come di consueto, $0 < 1 < 2 < \dots < 9$. L'ordine che ne deriva sui naturali, intesi come stringhe in A^* è il loro ordine abituale: ad esempio
 - $151 < 153$ perché $1 < 3$,
 - $151 < 1327$ perché 151 ha lunghezza minore.

Consideriamo ancora i numeri naturali $\mathbb{N}$, intesi come parole su $A = \{0, 1, 2, \dots, 9\}$ rispetto alla loro tradizionale rappresentazione in base 10. È ben noto che N è primo se $N \geq 2$ e gli unici divisori di N sono 1 e N , e che ogni $N \geq 2$ si decompone in modo unico nel prodotto di fattori primi. Nascono così due classici problemi: entrambi hanno per input il nostro $N \geq 2$,

- il primo chiede se N è primo o no,
- il secondo di decomporre N nei suoi fattori primi.

Allora la prima domanda intende riconoscere tra le stringhe su A quelle che corrispondono ai numeri primi; la seconda vuole calcolare, per ogni N , quelle stringhe che rappresentano numeri primi che dividono N . Nel primo caso si vuole individuare un sottoinsieme di A^* , nel secondo computare una funzione che da parole su A genera nuove sequenze di parole su A .

Sotto questo punto di vista, il Decimo problema di Hilbert presenta molte analogie col quesito di riconoscere i primi. È vero che tratta polinomi piuttosto che numeri, e che adopera tutt'altro criterio di selezione (l'esistenza di radici intere piuttosto che la primalità); tuttavia ha ancora a che fare con un alfabeto A (quello dei polinomi a coefficienti interi) e con parole su A (i polinomi, appunto) e

vuole scegliere quelle parole che soddisfano il criterio fissato (l'esistenza di radici intere), individuare quindi un particolare sottoinsieme di A^* .

In generale, dato un alfabeto A , un sottoinsieme L di A^* si chiama *linguaggio formale*, o più semplicemente *linguaggio*, o anche *problema*. La prima notazione fa riferimento alla sua natura di insieme di parole, come ogni lingua tradizionale; la seconda alla questione computazionale che L ovviamente propone, e cioè riconoscere le stringhe su A che stanno in L e escludere le altre.

Esempio. L'insieme dei numeri primi è un linguaggio su $A = \{0, 1, 2, \dots, 9\}$ (e su qualunque alfabeto per i naturali). L'insieme dei polinomi a coefficienti e radici intere è un linguaggio sull'alfabeto del Decimo problema di Hilbert.

Non dimentichiamo che, accanto al “problema” di riconoscere un linguaggio, c'è anche quello di computare funzioni su stringhe su un alfabeto A , come il caso della decomposizione in fattori primi ci ha sopra illustrato.

2.3 La Macchina di Turing

La *Macchina di Turing*, qui abbreviata MdT, prende il nome dal matematico inglese Alan Turing, che la introdusse nel 1936 precedendo di almeno un decennio l'era del computer.

È il primo dei modelli di calcolo che vogliamo trattare. L'idea di Turing è infatti quella di immaginare un semplice meccanismo astratto che riassume e simuli tutte le potenzialità computazionali dell'uomo comune. Così Turing prende a esplicito modello l’“*impiegato diligente*”, che svolge con ordine e cura gli incarichi assegnatigli, ma non fa niente di più: all'ora stabilita timbra il cartellino e torna a casa. Vediamo come la MdT traduce questo comportamento.

Da un punto di vista fisico la MdT può essere pensata come composta da una unità di controllo a stati finiti, un nastro di lunghezza infinita e una testina di lettura e scrittura, che permette la comunicazione tra controllo e nastro.

Il nastro è infinito e suddiviso in *celle* (anche chiamate *quadri*). Ogni cella è in grado di memorizzare un simbolo di un certo alfabeto $A = \{a_0, a_1, \dots, a_n\}$, oppure un simbolo “bianco” $\star$ che denota l'assenza di scrittura in una cella. Il nastro contiene solo un numero finito di simboli di A ; tutte le altre celle contengono comunque il simbolo bianco $\star$. La testina di lettura e scrittura permette all'unità di controllo di leggere e scrivere un simbolo per volta dal nastro, quindi a ogni istante la testina può indicare una sola cella del nastro (e leggere o scrivere il simbolo che la riguarda). L'unità di controllo, oltre ad avere gli organi meccanici per lo spostamento del nastro e della testina, contiene il programma secondo cui verrà eseguito il calcolo e mantiene lo stato della macchina. L'insieme di possibili stati della macchina è un insieme finito $Q = \{q_0, q_1, \dots, q_m\}$.

Se vogliamo riprendere il paragone con l'impiegato diligente, A rappresenta l'insieme di lettere con cui egli legge la domanda di partenza e svolge i suoi calcoli successivi; gli stati di Q corrispondono invece alle possibilità che l'impiegato ha di scrivere la stessa lettera in più modi distinti, sottolineandola, o evidenziandola, o colorandola per darle maggiore risalto.

La computazione della MdT avviene per passi discreti. Si concorda che, all'avvio di ogni sua computazione, la macchina si trovi in uno *stato iniziale* prefissato q . Ad ogni passo l'unità di controllo prende atto dello stato in cui si trova e dal simbolo contenuto nella cella che la testina indica, e di conseguenza esegue le operazioni sotto elencate:

- rivede il suo stato;
- scrive un simbolo nella cella indicata dalla testina, sostituendo il simbolo esistente (ricordiamo che tra i simboli ammessi c'è anche $\star$);
- sposta la testina di una posizione a sinistra o a destra.

Il nuovo stato assunto dall'unità di controllo, il simbolo da scrivere sulla cella indicata dalla testina e lo spostamento della testina a sinistra o a destra sono determinati dal programma della MdT, che stabilisce il comportamento della macchina stessa. Il programma di una MdT può quindi essere pensato come un insieme di quintuple della forma (q, a, q', a', x) , dove q indica lo stato dell'unità di controllo, a il simbolo nella cella indicata dalla testina mentre q' , a' , x specificano l'azione che la MdT deve intraprendere: in dettaglio q' rappresenta il nuovo stato dell'unità di controllo, a' il simbolo da scrivere nella cella esaminata dalla testina e $x = \pm 1$ lo spostamento della testina: una posizione a sinistra se $x = -1$, a destra se $x = +1$. $+1$ sarà talora denotato semplicemente con 1 nel seguito.

Ovviamente ogni singola MdT si riconosce, non tanto dal suo aspetto esteriore di nastri, celle e testine, quanto proprio da queste quintuple, e cioè dal programma che le si richiede di svolgere e dalle istruzioni che esso prevede, tutte ridotte a semplici ordini di cambiamento di stato, di scrittura e di spostamento: così Turing schematizzava il comportamento della mente dell'impiegato diligente, sintetizzandone le capacità operative minime, e secondo questo modello si prevede di riconoscere linguaggi o computare funzioni.

Notiamo poi che un impiegato diligente esegue ordinatamente le sue istruzioni finché ne ha e purché non ne abbia troppe tra loro in concorrenza. Nel primo caso, quando gli ordini mancano, l'impiegato si ferma e ritiene concluso il suo lavoro; nel secondo, invece, resta nell'imbarazzo della *scelta*: responsabilità che non si può scaricare sulle sue spalle.

È dunque importante che le istruzioni relative ad una coppia (q, a) , quando esistono, siano univoche. Si può ammettere che manchino, non che si moltiplichino. Il passaggio da (q, a) alla corrispondente terna (q', a', x) deve essere assolutamente deterministico e lontano da ogni ambiguità.

Il programma di una MdT può essere descritto mediante una tabella a righe e colonne, la cosiddetta *matrice funzionale*; le righe corrispondono ai possibili stati q della macchina mentre le colonne corrispondono ai possibili simboli a dell'alfabeto (incluso $\star$). All'incrocio di ciascuna coppia q e a si pone l'istruzione che deve essere eseguita dalla macchina quando l'unità di controllo si trova nello stato q e la testina legge il simbolo a : la terna che descrive in che stato entrare, che cosa scrivere, dove spostarsi. È ammesso che una casella della matrice funzionale sia bianca. In tal caso, si conviene che la macchina non ha azioni da compiere e quindi termina il calcolo.

La matrice funzionale della macchina rappresenta una specie di tabella in cui sono concentrate tutte le istruzioni: l'impiegato diligente esegue il suo lavoro applicandola pedissequamente.

Ma è tempo di dare una definizione formalmente rigorosa delle MdT, che ne riassume i caratteri essenziali, l'alfabeto, gli stati, le istruzioni. Possiamo allora dire:

Definizione. Una *Macchina di Turing* M è una quadrupla (Q, A, δ, q_0) , dove:

- Q è un insieme finito di stati;
- A è un alfabeto, cui si aggiunge il simbolo bianco $\star$;
- δ è una funzione da $Q \times (A \cup \{\star\})$ a $Q \times (A \cup \{\star\}) \times \{-1, +1\}$: δ è chiamata *funzione di transizione*, e gli elementi (q, a, q', a', x) di δ sono chiamati *regole di transizione* o *istruzioni* di M ;
- $q_0 \in Q$ è lo stato iniziale.

Come già sottolineato, l'informazione cruciale su una MdT è quella fornita dalla funzione di transizione δ , quella che descrive il programma delle istruzioni. È δ che regola il comportamento di M . Così, se M si trova nello stato $q \in Q$, la testina legge il simbolo $a \in A \cup \{\star\}$ e $\delta(q, a) = (q', a', x)$, allora M si sposta nello stato q' , il simbolo a' sostituisce a nella cella in esame e la testina si sposta di una posizione a sinistra se $x = -1$, o a destra se $x = +1$. Se invece M si trova nello stato $q \in Q$, la testina legge il simbolo $a \in A \cup \{\star\}$ ma $\delta(q, a)$ non è definito, allora M si arresta. In questo senso, la matrice sopra descritta altro non è che il grafico di δ . Spesso, quando chiaro dal contesto, identificheremo una macchina di Turing M con la sua funzione di transizione δ . L'istruzione di δ che genera la terna (q', a', x) da (q, a) si scriverà talora

$$(q, a) \xrightarrow{\delta} (q', a', x)$$

invece che $\delta(q, a) = (q', a', x)$ o $(q, a, q', a', x) \in \delta$; qualche volta, quando non ci sono rischi di ambiguità, ometteremo δ e scriveremo semplicemente

$$(q, a) \rightarrow (q', a', x).$$

Una MdT, così come l'abbiamo appena definita, viene anche detta *deterministica*, a sottolineare che ogni tripla (q', a', x) , se esiste, è unica e univocamente determinata dalla coppia (q, a) . Tra qualche paragrafo incontreremo anche MdT *non deterministiche* e ne spiegheremo le differenze rispetto a quanto appena descritto. Torniamo adesso ad una descrizione informale del modo in cui una MdT esegue le sue computazioni a partire da un dato input e restituisce il relativo output. Ci è utile introdurre anche il concetto di *configurazione istantanea* di una MdT: una sorta di "fotografia" che ritrae la macchina ad un dato istante di lavoro, illustrandone lo stato, il contenuto del nastro e la posizione della testina.

Poichè il nastro di una MdT è sempre vuoto, salvo che per un numero finito di celle, è possibile riassumerne il contenuto con una stringa finita di simboli e fornire la descrizione istantanea di una MdT con una quadrupla del tipo (ξ, q, a, η) , dove

q rappresenta lo stato della macchina all'istante in considerazione, a è il simbolo in lettura, $\xi \in (A \cup \{\star\})^*$ è la stringa di simboli a sinistra del simbolo in lettura ed $\eta \in (A \cup \{\star\})^*$ è la stringa di simboli a destra del simbolo in lettura prima che il nastro diventi definitivamente bianco. Per esempio, per $A = \{1\}$, $(11, q_0, 1, \lambda)$ ci dice che la macchina esamina una cella con 1 nello stato q_0 , che il nastro è bianco a destra della cella considerata, contiene 11 e poi diventa bianco a sinistra.

Poniamo adesso:

Definizione. Una *configurazione istantanea* di una Macchina di Turing $M = (Q, A, \delta, q_0)$ è un elemento dell'insieme: $(A \cup \{\star\})^* \times Q \times (A \cup \{\star\}) \times (A \cup \{\star\})^*$.

Riepiloghiamo allora come avviene una computazione di una macchina M su una stringa di input $w \in A^*$.

- Al passo iniziale 0, M esamina una sequenza w scritta da sinistra verso destra sul nastro di input: la testina indica il primo carattere a_0 di w e lo stato interno della macchina è quello iniziale q_0 . Questa configurazione è detta *configurazione iniziale* su w . Se $w = a_0 w'$, la 4-upla che la rappresenta è (λ, q_0, a_0, w') .
- Ammettiamo che al passo i della computazione la macchina si trovi nella configurazione $C_i = (\xi, q, a, \eta)$. In base alla funzione δ , se esiste una regola di transizione $(q, a) \xrightarrow{\delta} (q', a', x)$, allora la macchina passa nella nuova configurazione C_{i+1} derivata da C_i secondo questa istruzione. Se invece $\delta(q, a)$ non è definita M si arresta. Per esempio, se C_i è come sopra $(11, q_0, 1, \lambda)$ e $(q_0, 1) \xrightarrow{\delta} (q_1, 1, 1)$, si ha $C_{i+1} = (111, q_1, \star, \lambda)$; se invece $(q_0, 1) \xrightarrow{\delta} (q_1, 1, -1)$, allora $C_{i+1} = (1, q_1, 1, 1)$.
- Se M si ferma dopo un numero n finito di passi, allora si dice che M *converge* sull'input w (in notazione, $M \downarrow w$); C_n si chiama *configurazione finale*, la sequenza di configurazioni $C_0, C_1, \dots, C_n$ è una computazione completa (finita) di M sull'input w e la stringa contenuta sul nastro di input è l'*output* di tale computazione. Per esempio, se $C_n = (\xi, q, a, \eta)$, allora $\xi a \eta$ è l'output.
- Se M non si arresta mai, allora si dice che M *diverge* sull'input w e si scrive $M \uparrow w$.

Per formalizzare rigorosamente questo comportamento computazionale possiamo introdurre una relazione binaria $\vdash_M$ tra le configurazioni di M . Sostanzialmente $\vdash_M$ associa ad ogni configurazione di M quella che la segue dopo l'esecuzione dell'istruzione di M corrispondente. Per esempio, se C è, come sopra, $(11, q_0, 1, \lambda)$, C' è $(111, q_1, \star, \lambda)$ e in M si ha l'istruzione $(q_0, 1) \xrightarrow{\delta} (q_1, 1, 1)$, si pone $C \vdash_M C'$. Il lettore potrà scrivere per **esercizio**, se ha voglia e pazienza, tutti i dettagli della definizione di $\vdash_M$.

Se poi per una data C non esiste alcuna configurazione C' tale che $C \vdash_M C'$, allora scriveremo $C \not\vdash_M$.

Sia $\vdash_M^*$ la chiusura riflessiva e transitiva di $\vdash_M$; vale a dire, per C, C' configura-

zioni, $C \vdash_M^* C'$ significa che esistono un naturale n e configurazioni $C_0, C_1, \dots, C_n$, tali che $C = C_0 \vdash_M C_1 \vdash_M \dots \vdash_M C_n = C'$.

Chiamiamo poi *computazione* di una MdT M su un input w una sequenza (eventualmente infinita) di configurazioni

$$C_0 \vdash_M C_1 \vdash_M \dots \vdash_M C_i \vdash_M \dots$$

tale che C_0 è una configurazione iniziale su w . Chiaramente $M \downarrow w$ se questa computazione è finita

$$C_0 \vdash_M C_1 \vdash_M \dots \vdash_M C_n$$

e inoltre $C_n \not\vdash_M$; C_n è la *configurazione finale* di M su w . Altrimenti, se $M \uparrow w$, la computazione non ha mai fine.

Esempio. Si consideri l'alfabeto $A = \{\star, 1\}$ composto dal solo 1 e dal simbolo bianco $\star$. Definiamo una MdT M che, presa come input una successione di 1 consecutivi, restituisce come output tale successione aumentata di un elemento. In dettaglio M dispone di due stati interni, lo stato iniziale q_0 e uno stato di "arresto" q_1 , e la funzione di transizione δ è composta dalle istruzioni

$$(q_0, 1) \rightarrow (q_0, 1, +1), \quad (q_0, \star) \rightarrow (q_1, 1, +1).$$

Secondo le convenzioni stabilite, alla partenza M ha lo stato q_0 e la testina indica il primo simbolo a sinistra dell'input. Fintanto che la testina trova celle segnate con 1, allora, in virtù della prima regola di transizione, M scrive 1 sulla cella osservata (cioè il simbolo letto viene lasciato inalterato) e la testina si sposta a destra di una cella, mantenendo lo stato q_0 . Quando la testina arriva ad una cella vuota, M passa alla seconda regola di transizione, in virtù della quale la macchina segna con 1 la cella osservata, sposta la testina a destra (la scelta della direzione di spostamento è in questo caso influente) e assume lo stato q_1 , per il quale non è definita nessuna regola di transizione e, quindi, la macchina si arresta.

Esempio. Consideriamo adesso una MdT che ha lo stesso alfabeto $A = \{1, \star\}$ dell'esempio precedente, ancora due stati q_0, q_1 , ma una sola istruzione $(q_0, \star) \rightarrow (q_0, \star, +1)$. Ammettiamo che la macchina abbia come input il nastro bianco, e dunque si trovi a leggere $\star$ nello stato q_0 al momento in cui la computazione si avvia. È facile verificare che la macchina scivola verso destra continuando a ristampare $\star$ su ogni nuova cella considerata e a rimanere nello stato q_0 . Quindi la computazione diverge.

In conclusione, la macchina di Turing costituisce un soddisfacente modello di calcolo, e comunque corrisponde al decalogo fissato nel Capitolo 1. Infatti si ha quanto segue.

- La funzione di transizione di una MdT costituisce un insieme finito di istruzioni.
- La MdT così descritta è anche l'agente di calcolo che esegue le istruzioni di cui al punto 1).

- La MdT può utilizzare il nastro per memorizzare i risultati intermedi.
- La MdT opera in modo discreto.
- La MdT opera in modo deterministico, con istruzioni univocamente specificate.
- Non esiste nessuna limitazione sulla lunghezza delle stringhe di ingresso, in quanto si assume che il nastro è illimitato.
- Il nastro della MdT costituisce, appunto, una memoria di capacità non limitata.
- Le operazioni che la MdT può eseguire sono molto semplici, quindi di complessità finita.
- Non esiste nessun limite al numero delle istruzioni eseguite durante una computazione: infatti, le istruzioni sono di numero finito, ma è ammessa la possibilità di usare più volte la stessa istruzione.
- Esiste la possibilità di computazioni infinite, come l'esempio precedente ci mostra.

2.4 Macchine di Turing e linguaggi

Vediamo adesso come una MdT M gestisce la questione di identificare un linguaggio L in un alfabeto A .

Ovviamente M avrà lo stesso alfabeto A di L , e quindi sarà $M = (Q, A, \delta, \phi)$ per opportuni Q e δ . Possiamo immaginare per M due possibili strategie per distinguere gli elementi di L .

1. M fissa preventivamente, a prescindere da L , due stringhe di output tra loro distinte, da intendersi una volta per tutte come “SÌ” e “NO”. Si chiede poi a M , per ogni input $w \in A^*$, di:
 - convergere su SÌ se $w \in L$,
 - convergere su NO se $w \notin L$.

Si dice allora che M decide L .

2. Alternativamente si ammette che, per ogni $w \in A^*$, M converge su w se e solo se $w \in L$; dunque
 - $M \downarrow w$ se $w \in L$,
 - $M \uparrow w$ se $w \notin L$.

Si dice allora che M accetta L .

Notiamo che le due strategie non sono tra loro equivalenti e che in realtà solo la prima è adeguata a distinguere effettivamente gli elementi di L da quelli fuori di L . Infatti, se M si limita ad accettare L , M sa riconoscere gli elementi di L , perché converge su di essi; ma non sa escludere gli elementi fuori di L , per i quali segue l'evoluzione della corrispondente computazione, prende atto che essa non si conclude né dopo 10, né dopo 100, né dopo 10^n passi, ma non può su questa base prevedere che essa prosegua indefinitamente (come in realtà accade), o non si arresti proprio al passo $10^n + 1$, o poco dopo.

Quando M accetta un linguaggio L e dunque L si compone delle parole $w \in A^*$ per cui $M \downarrow w$, diremo comunque che (almeno) L è *riconosciuto* da M .

Diciamo poi:

Definizione. Un linguaggio L è *decidibile secondo Turing* (o, più semplicemente, *decidibile*) se esiste una macchina di Turing M che decide L , *indecidibile (secondo Turing)* altrimenti.

Definizione. Un linguaggio L è detto *semidecidibile secondo Turing* (o, più semplicemente, *semidecidibile*) se esiste una macchina di Turing M che accetta L .

Nei prossimi capitoli studieremo formalmente le relazioni tra decidibilità e semidecidibilità. Non è difficile convincersi, comunque, che se un linguaggio è decidibile allora è anche semidecidibile. È facile, infatti, trasformare una MdT M che risponde “SÌ” alle stringhe che appartengono al linguaggio e “NO” alle stringhe che non appartengono al linguaggio in una macchina di Turing M che converge quando M dichiara “SÌ” e diverge quando M dice “NO”. Nel seguito saranno proposti molti esempi di linguaggi decidibili e semidecidibili. Per il momento ci accontentiamo di un caso molto semplice.

Esempio. Sia $A = \{1\}$. Sappiamo che i numeri naturali $0, 1, 2, 3, 4, \dots$ si possono rappresentare, in verità in modo assai rozzo, come parole su A $1, 11, 111, 1111$, e così via: dunque ogni naturale N diventa una stringa di 1 di lunghezza $N + 1$. Sia L l'insieme dei pari. Vogliamo provare che L è decidibile e quindi costruisce una MdT M su $\{1\}$ che lo decide. Identifichiamo anzitutto le risposte SÌ e NO rispettivamente con i numeri 1 e 0 , e dunque con le codifiche 11 e 1 . Informalmente M conta la parità di un dato input N e risponde 1 o 11 di conseguenza. Per ottenere questo obiettivo ci servono quattro stati q_0, q_1, q_2, q_3 e una funzione di transizione δ con le seguenti istruzioni

- (a) $(q_0, \star) \rightarrow (q_3, 1, +1), (q_1, \star) \rightarrow (q_2, 1, +1),$
- (b) $(q_0, 1) \rightarrow (q_1, \star, +1), (q_1, 1) \rightarrow (q_0, \star, +1),$
- (c) $(q_2, \star) \rightarrow (q_3, 1, +1).$

Vediamo allora come M verifica che 4 , cioè 11111 , è pari. Anzitutto M usa le istruzioni in (b) per cancellare ogni 1 e giungere finalmente a $\star$ nello stato q_1 , secondo la computazione che adesso mostriamo:

$$(\lambda, q_0, 1, 1111) \vdash_M (\lambda, q_1, 1, 111) \vdash_M (\lambda, q_0, 1, 11) \vdash_M (\lambda, q_1, 1, 1) \vdash_M \\ \vdash_M (\lambda, q_0, 1, \lambda) \vdash_M (\lambda, q_1, \star, \lambda);$$

a questo punto (a), (c) intervengono e implicano

$$(\lambda, q_1, \star, \lambda) \vdash_M (1, q_2, \star, \lambda) \vdash_M (11, q_3, \star, \lambda),$$

dopo di che, in assenza di istruzioni su q_3 , M si arresta con l'output atteso 11 , cioè SÌ.

Consideriamo ora la computazione di M sull'input 3, cioè 1111. Stavolta (b) implica

$$\begin{aligned} &(\lambda, q_0, 1, 111) \vdash_M (\lambda, q_1, 1, 11) \vdash_M (\lambda, q_0, 1, 1) \vdash_M (\lambda, q_1, 1, \lambda) \\ &\vdash_M (\lambda, q_0, \star, \lambda) \end{aligned}$$

cui (a) aggiunge

$$(\lambda, q_0, \star, \lambda) \vdash_M (1, q_3, \star, \lambda),$$

dopo di che si arresta con l'output 1, cioè NO.

Il lettore verifichi da solo che una MdT M' con due soli stati q_0, q_1 e le tre istruzioni $(q_0, 1) \rightarrow (q_1, \star, +1)$, $(q_1, 1) \rightarrow (q_0, \star, +1)$, $(q_0, \star) \rightarrow (q_0, \star, +1)$ accetta L , e conferma così che L è (almeno) semidecidibile.

2.5 Macchine di Turing e funzioni

In questo paragrafo discutiamo come le Macchine di Turing possano calcolare le funzioni sulle stringhe di un certo alfabeto A .

Definizione. Sia f una funzione con dominio e immagine in A^* . Si dice che una MdT $M = (Q, A, \delta, q_0)$ *calcola* o *computa* f se e solo se, $\forall w \in A^*$,

- i. se w è nel dominio di f , allora $M \downarrow w$ con output $f(w)$;
- ii. se invece $f(w)$ non è definita, allora $M \uparrow w$.

Definizione. La funzione f è detta *calcolabile secondo Turing* o *computabile secondo Turing* se esiste una macchina di Turing M che la calcola; *non calcolabile*, o *non computabile secondo Turing* altrimenti.

Le precedenti definizioni si estendono facilmente al caso in cui il dominio o l'immagine di f si compongono di coppie, o terne, o n -uple di parole invece che direttamente di stringhe. Basterà convenire di rappresentare una coppia (w_1, w_2) di parole su A come una stringa $w_1 \star w_2$ in cui w_2 succede a w_1 , separata da un simbolo bianco, e procedere analogamente per terne, o sequenze più complicate. Si noti poi che tra le funzioni computabili secondo Turing sono ammessi anche esempi parziali, definiti solo su porzioni di A^* , e non sulla sua totalità. Questa scelta non deve sorprendere né scandalizzare. Si osservi che molte operazioni comuni ed elementari, come la divisione tra i naturali, non sempre si possono eseguire; per esempio non c'è verso di calcolare $5 : 2$. Nel caso specifico, si può cercare di rimediare ammettendo un eventuale resto minore del divisore. Così, per $5 : 2$, si ottiene quoziente approssimato 2 e resto 1

$$5 = 2 \cdot 2 + 1.$$

Pur tuttavia, certe divisioni, quelle con divisore 0 come $5 : 0$, restano ancora impossibili. La divisione è, quindi, una funzione solo parziale.

Esempio. Presentiamo una MdT M che computa la funzione totale di addizione tra i naturali. Come già in precedenza, lavoriamo sull'alfabeto $A = \{1\}$. Assumiamo che M abbia quattro stati q_0, q_1, q_2, q_3 e le istruzioni

$$(q_0, \star) \rightarrow (q_1, 1, +1), (q_0, 1) \rightarrow (q_0, 1, +1), (q_1, \star) \rightarrow (q_2, \star, -1),$$

$$(q_1, 1) \rightarrow (q_1, 1, +1), (q_2, 1) \rightarrow (q_3, \star, -1), (q_3, 1) \rightarrow (q_3, \star, 1).$$

Mostriamo come, per esempio, M verifichi che $2 + 3$ fa 5, cioè produca l'output 111111 quando il suo input è la coppia $(2, 3)$, cioè $111 \star 1111$. Infatti le istruzioni relative a q_0 fanno ricopiare a M la prima sequenza di 1 senza cambiare stato

$$(\lambda, q_0, 1, 11 \star 1111) \vdash_M^* (111, q_0, \star, 1111),$$

poi le fanno aggiungere un ulteriore 1 e passare ad esaminare la seconda sequenza di 1 nello stato q_1

$$(111, q_0, \star, 1111) \vdash_M (1111, q_1, 1, 111).$$

L'istruzione su $(q_1, 1)$ ha ancora l'effetto di riprodurre inalterata questa seconda sequenza

$$(1111, q_1, 1, 111) \vdash_M^* (11111111, q_1, \star, \lambda);$$

a questo punto M cancella gli ultimi due 1, passa allo stato q_3 e si ferma producendo l'output desiderato 111111

$$(11111111, q_1, \star, \lambda) \vdash_M (1111111, q_2, 1, \lambda) \vdash_M \\ \vdash_M (111111, q_3, 1, \lambda) \vdash_M (111111\star, q_3, \star, \lambda).$$

Finalmente notiamo che ogni MdT M con alfabeto A computa una funzione, eventualmente parziale, da A^* a $(A \cup \{\star\})^*$, e più in generale dall'insieme delle sequenze finite di stringhe di A^* e $\star$ all'insieme stesso: la funzione associa ad ogni stringa (o sequenza di stringhe) l'output della eventuale computazione convergente di M su di essa.

2.6 Codifiche di stringhe

Abbiamo sin qui considerato stringhe su alfabeti A arbitrari. In realtà è possibile ridurre il nostro ambito ai numeri naturali e a uno degli alfabeti che servono a rappresentarli, come $\{1\}$ o $\{0, 1\}$ o $\{0, 1, 2, \dots\}$. L'idea che si può seguire è la stessa in base alla quale

- in un teatro ogni poltrona ha un numero di riconoscimento,
- o in una biblioteca ogni volume riceve un suo numero di etichetta.

I numeri non sono né poltrone né libri, ma servono a identificarli senza ambiguità. Procedimenti effettivi di codifica relativamente elementari svolgono la stessa funzione per le stringhe su un qualunque alfabeto A . Vediamone alcuni. Per A arbitrario, cerchiamo di definire in modo esplicito una funzione di numerazione $\sharp$ da A^* a $\mathbb{N}$.

1. Una prima strategia può fare riferimento alla rappresentazione dei naturali in base 10, o 2, o qualunque valore prefissato > 0 . Ammettiamo, ad esempio, per semplicità, che l'alfabeto A abbia nove elementi. È immediato identificarli con le cifre $1, 2, \dots, 9$ e magari indicarli con $a_1, a_2, \dots, a_9$. Così

$$j \mapsto a_j \text{ per ogni } j = 1, \dots, 9$$

è una corrispondenza biunivoca tra $\{1, 2, \dots, 9\}$ e A . A questo punto ad ogni parola non vuota $w = a_{j_1} \cdots a_{j_k}$ su A possiamo associare $j_1 \cdots j_k$ in base 10, cioè porre

$$\sharp(w) = j_1 \cdot 10^{k-1} + j_2 \cdot 10^{k-2} + \cdots + j_{k-1} \cdot 10^1 + j_k \cdot 10^0;$$

gli indici dei simboli di w determinano i coefficienti di questa rappresentazione, la lunghezza k di w ne regola il numero degli addendi (k , appunto, da 10^0 a 10^{k-1}). Si noti che $\sharp(w) \neq 0$ perché $j_1, \dots, j_k > 0$ e $k > 0$. Alla parola vuota λ , che non ha simboli e ha lunghezza 0, associamo allora 0:

$$\sharp(\lambda) = 0.$$

Parole distinte ricevono così in modo effettivo numeri distinti. Per esempio

$$\sharp(a_1 a_2 a_3) = 123, \quad \sharp(a_2 a_4) = 24,$$

e via dicendo. Viceversa 2154 è il numero di $a_2 a_1 a_5 a_4$, mentre 1004 non corrisponde a nessuna parola perché contiene 0, ma non è 0. Quindi $\sharp$ non è suriettiva. Tuttavia per ogni naturale n è possibile stabilire in modo effettivo se n è o no nell'immagine di $\sharp$ e, se sì, a quale parola corrisponde.

Il procedimento si estende facilmente al caso in cui l'alfabeto A ha non più nove, ma un numero arbitrario $N > 0$ di simboli. Si scrivono gli elementi di A come $a_1, \dots, a_N$ e poi si fa riferimento a $N+1 > 1$ e alla rappresentazione dei naturali in base $N+1$, al fatto cioè che ogni j si scrive in modo unico come

$$j = j_1 \cdot (N+1)^{k-1} + j_2 \cdot (N+1)^{k-2} + \cdots + j_{k-1} \cdot (N+1)^1 + j_k \cdot (N+1)^0$$

per $k > 0$, $j_1, \dots, j_k$ naturali opportuni. Si pone allora, per $w \in A^*$

- se $w = \lambda$ è vuota, $\sharp(w) = 0$,
- se $w = a_{j_1} \cdots a_{j_k}$ non è vuota,

$$\sharp(w) = j_1 \cdot (N+1)^{k-1} + j_2 \cdot (N+1)^{k-2} + \cdots + j_{k-1} \cdot (N+1)^1 + j_k \cdot (N+1)^0.$$

Le proprietà osservate per $N = 9$ si preservano. In particolare ogni parola su A riceve il “suo” numero $\sharp(w)$ e, viceversa, da ogni naturale si può recuperare in modo effettivo la parola corrispondente, se esiste.

I pignoli potrebbero semmai obiettare che la numerazione data da $\sharp$ ha il difetto di non saper distinguere simboli da parole. Per spiegarci meglio, riferiamoci per un attimo alla lingua italiana, dove a ha ruolo sia di lettera (vocale) che di parola (preposizione: andare “a” Roma). Questa duplice veste meriterebbe un duplice numero di codice, uno come simbolo e uno come stringa. Ma $\sharp$ le assegna soltanto il valore 1, come prima lettera dell’alfabeto.

2. Cerchiamo di ovviare all’ultima obiezione proponendo un’altra strategia di numerazione che fa stavolta ricorso al teorema fondamentale dell’Aritmetica, quello secondo cui ogni naturale ≥ 2 si scompone in modo unico nel prodotto di fattori primi: ad esempio $12 = 2^2 \cdot 3$, $15 = 3 \cdot 5$, e così via. Cominciamo allora col numerare i simboli dell’alfabeto A , cui assegniamo valori *dispari* (per motivi che saranno chiari più tardi). Definiamo dunque una funzione $\sharp_0$ da A a $\mathbb{N}$ ponendo, per $A = \{a_1, \dots, a_N\}$ e per $j = 1, \dots, N$,

$$\sharp_0(a_j) = 2 \cdot j + 1,$$

cioè $\sharp_0(a_1) = 3$, $\sharp_0(a_2) = 5$ e via dicendo.

Passiamo poi a numerare le stringhe di A^* , introducendo la seguente funzione $\sharp$ da A^* a $\mathbb{N}$. Sia $w \in A^*$:

- se $w = \lambda$, conveniamo $\sharp(w) = 1$,
- altrimenti sia $w = a_{j_1} \cdots a_{j_k}$; in riferimento ai valori $2 \cdot j_1 + 1, \dots, 2 \cdot j_k + 1$ già associati da $\sharp_0$ a $j_1, \dots, j_k$, poniamo

$$\sharp_1(w) = 2^{2 \cdot j_1 + 1} \cdot 3^{2 \cdot j_2 + 1} \cdots p_k^{2 \cdot j_k + 1}$$

dove p_k denota il k -mo numero della sequenza dei primi.

Assegniamo quindi le codifiche (tramite $\sharp_0$) dei simboli di w come esponenti a $\sharp_1(w)$ nella sua decomposizione in fattori primi. Per esempio

$$\sharp_1(a_1 a_4 a_2 a_1 a_1) = 2^3 \cdot 3^9 \cdot 5^5 \cdot 7^3 \cdot 11^3.$$

In questo modo, stringhe distinte ricevono in modo effettivo valori distinti. Viceversa, per ogni numero naturale n , è possibile stabilire in modo effettivo se n è o no immagine di $\sharp_1$ e, se sì, risalire alla parola corrispondente: basta fare riferimento alla decomposizione di n in fattori primi, oppure constatare che $n = 1$. Per esempio, il numero dei fattori primi coinvolti rivela la lunghezza della eventuale stringa corrispondente. Stavolta, poi, gli elementi a_j di A ricevono due numeri distinti

- l’uno $2 \cdot j + 1$ come simboli,
- l’altro $2^{2 \cdot j + 1}$ come parole.

Né c'è pericolo di confondere questi valori, perché i numeri dei simboli sono dispari ≥ 3 , mentre quelli delle parole sono 1 oppure pari, perché esplicitamente divisibili per 2.

Questo secondo procedimento di numerazione fu ideato da Kurt Gödel nella sua dimostrazione dei Teoremi di Incompletezza cui si è accennato nel primo capitolo.

Una volta trattati simboli e stringhe, esso permette anche di numerare coppie, o terne, o n -uple ordinate di stringhe. A questo proposito, abbiamo già visto una possibile strategia: tradurre queste sequenze in stringhe in un alfabeto più lungo che utilizzi anche il simbolo bianco $\star$. Così (w_1, w_2) diviene $w_1 \star w_2$, (w_1, w_2, w_3) diviene $w_1 \star w_2 \star w_3$ e via dicendo; in questo modo (w_1, w_2) , $(w_1, w_2, w_3), \dots$ vengono a condividere il numero che $w_1 \star w_2, w_1 \star w_2 \star w_3, \dots$ ricevono in $A \cup \{\star\}$.

Il procedimento di Gödel consente tuttavia di evitare il ricorso al simbolo estraneo $\star$. Infatti, di fronte alla n -upla $(w_1, w_2, \dots, w_n)$ di stringhe di A (con $n \geq 2$), possiamo

- ricordare che $w_1, w_2, \dots, w_n$ hanno già un loro numero di codice, $\sharp_1(w_1), \sharp_1(w_2), \dots, \sharp_1(w_n)$ rispettivamente,
- ricorrere allora nuovamente al teorema fondamentale dell'Aritmetica,
- associare a $(w_1, w_2, \dots, w_n)$ il numero

$$\sharp_2(w_1, w_2, \dots, w_n) = 2^{\sharp_1(w_1)} \cdot 3^{\sharp_1(w_2)} \dots p_k^{\sharp_1(w_n)}.$$

Si ottiene così una funzione effettiva $\sharp_2$ dall'insieme delle sequenze finite di stringhe su A di ogni possibile lunghezza $n \geq 2$ a $\mathbb{N}$. Sequenze distinte ricevono da $\sharp_2$ numeri distinti, per il teorema fondamentale dell'Aritmetica e l'unicità della decomposizione in fattori primi. Viceversa, per ogni naturale m , è possibile effettivamente riconoscere se m è o no immagine di $\sharp_2$ di qualche sequenza di stringhe e, se sì, di quale: di nuovo basta ricorrere alla decomposizione di m in fattori primi, il numero dei fattori primi distinti coinvolti rivela la lunghezza della sequenza, gli esponenti di questi fattori dicono le stringhe della sequenza. Di più, non c'è pericolo di confondere i valori assegnati alle sequenze di stringhe e quelli associati in precedenza a simboli o stringhe, perché questi ultimi sono dispari oppure pari con esponenti dispari ≥ 3 nella loro decomposizione in fattori primi, i nuovi sono pari e con esponenti pari.

2.7 Numeri e coppie

Visto che stiamo trattando coppie, o terne, o n -uple di elementi, varrà la pena di citare un'osservazione importante sulle coppie di numeri naturali, che ci sarà presto utile nel seguito. Si tratta di un risultato dimostrato dal matematico Georg Cantor nella seconda metà dell'Ottocento.

Teorema 2.7.1 *C'è una corrispondenza biunivoca r tra $\mathbb{N}^2$ e $\mathbb{N}$: specificamente,*

per ogni scelta di n, m naturali,

$$r(n, m) = \frac{(n + m)(n + m + 1)}{2} + n.$$

L'affermazione è per certi versi sorprendente: si può infatti osservare che ogni naturale n compare infinite volte come ascissa di una coppia $(n, m) \in \mathbb{N}^2$, al variare di m . Così $\mathbb{N}^2$ “sembra” infinitamente più grande di $\mathbb{N}$. Eppure è in corrispondenza biunivoca con $\mathbb{N}$ tramite r e quindi, in questo senso, ci sono tante coppie ordinate di naturali quanti naturali.
Al di là della complessità della formula che definisce r , cerchiamo di capire l'idea che Cantor seguì per ottenerla.
Supponiamo allora di disporre le coppie ordinate di numeri naturali in una tabella (infinita):

$(0, 0)$	$(0, 1)$	$(0, 2)$	$\dots$	$(0, n)$	$\dots$
$(1, 0)$	$(1, 1)$	$(1, 2)$	$\dots$	$(1, n)$	$\dots$
$(2, 0)$	$(2, 1)$	$(2, 2)$	$\dots$	$(2, n)$	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$(n, 0)$	$(n, 1)$	$(n, 2)$	$\dots$	(n, n)	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Le righe corrispondono alle ascisse, le colonne alle ordinate. Per enumerare le coppia di numeri naturali si percorra la tabella sulle diagonali come indicato nella figura che segue:

✓	✓	✓	✓	$\dots$	$\dots$
✓	✓	✓	$\dots$	$\dots$	$\dots$
✓	✓	$\dots$	$\dots$	$\dots$	$\dots$
✓	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

La prima diagonale contiene solo $(0, 0)$, la seconda $(0, 1)$ e $(1, 0)$, ..., la k -esima contiene $(0, k - 1)$, $(1, k - 2)$, ..., $(k - 2, 1)$ e $(k - 1, 0)$ e così via. La generica diagonale k contiene k coppie, precisamente quelle in cui la somma delle coordinate è $k - 1$. In questo modo, le coppie (n, m) di $\mathbb{N}^2$ risultano enumerate come segue:

$$(0, 0), (0, 1), (1, 0), (0, 2), (1, 1), (2, 0), (0, 3), (1, 2), (2, 1), (3, 0), \dots$$

e così via, dunque *prima* secondo $m + n$ e poi secondo n . Non è difficile controllare che la funzione r associa alle coppie sopra elencate nell'ordine

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9, \dots$$

cioè proprio la sequenza dei naturali. In generale r associa ad ogni coppia (n, m) il numero di passi necessari per raggiungerla meno 1. Dalla costruzione segue allora che la funzione r è una biezione: ogni coppia (n, m) compare una sola volta nella tabella e quindi è univocamente determinato il numero di passi necessario per raggiungerla lungo le varie diagonali. Inoltre r è data effettivamente e ammette, come ogni biezione, la sua funzione inversa: una corrispondenza biunivoca da $\mathbb{N}$ a $\mathbb{N}^2$, ancora effettiva. Sono quindi anche note le proiezioni di tale inversa su $\mathbb{N}$ cioè le due funzioni (sempre effettive) $\pi_1 : \mathbb{N} \rightarrow \mathbb{N}$ e $\pi_2 : \mathbb{N} \rightarrow \mathbb{N}$ tali che, per ogni t naturale, $\pi_1(t)$, $\pi_2(t)$ sono rispettivamente quei naturali n, m per cui $r(n, m) = t$. A proposito, un insieme che - come $\mathbb{N}^2$ - è in corrispondenza biunivoca con $\mathbb{N}$ si dice *numerabile*

Esercizio. Si provi che, per ogni $m \geq 2$, c'è una biezione effettiva tra $\mathbb{N}^m$ e $\mathbb{N}$.

2.8 Numeri e macchine

Adesso mostriamo che, per ogni alfabeto A , anche le Macchine di Turing che corrispondono ad A formano un insieme che è numerabile, e cioè è in corrispondenza biunivoca con $\mathbb{N}$. Anzi, è possibile dare un esempio esplicito di tale biezione. Vediamo come. Precisiamo anzitutto il contesto. Se A è fissato, una MdT M su A è determinata da

- l'insieme Q dei suoi stati,
- la funzione di transizione δ , con le sue istruzioni.

Possiamo convenire per semplicità di disporre a priori di un serbatoio infinito (numerabile) di stati

$$q_0, q_1, \dots, q_n, \dots$$

da cui ogni M estrae quegli $n + 1$

$$q_0, q_1, \dots, q_n$$

che le sono necessari per formare Q . Notiamo che il numero degli stati di M è sempre finito, ma può essere arbitrariamente grande; quindi non possiamo porre limiti al serbatoio cui M attinge e dobbiamo ammetterlo infinito. M risulta allora determinata da

- il numero $n + 1$ dei suoi stati,
- la sua funzione di transizione.

Possiamo ora provare:

Teorema 2.8.1 *C'è una corrispondenza biunivoca effettiva tra le MdT su un alfabeto A e i numeri naturali.*

Dimostrazione. Poniamo anzitutto $A = \{a_1, \dots, a_N\}$ con $a_1 \neq \dots \neq a_N$. Procediamo in cinque passi consecutivi, in modo per certi versi analogo a quello seguito per numerare simboli, stringhe e sequenze di stringhe su un alfabeto. Per la precisione numeriamo successivamente

- 1) i simboli che compaiono nelle istruzioni delle MdT su A ,
- 2) le istruzioni stesse,
- 3) le funzioni di transizione,
- 4) le MdT su A

e, finalmente,

- 5) costruiamo la biiezione richiesta tra MdT e numeri.

Ecco i dettagli dei vari passi.

- 1) Ricordiamo che i simboli che compaiono nelle istruzioni di una MdT su A sono

- gli elementi $a_1, \dots, a_N$ di A e il simbolo bianco $\star$;
- stati tra $q_0, q_1, \dots, q_n, \dots$;
- indici di spostamento ± 1 .

Li numeriamo con una funzione $\#_0$ che opera come segue:

-1	$+1$	$\star$	a_1	a_2	$\dots$	a_N	q_0	q_1	$\dots$	q_N	$\dots$
$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\dots$	$\downarrow$	$\downarrow$	$\downarrow$	$\dots$	$\downarrow$	$\dots$
3	5	7	9	11	$\dots$	$2N+7$	$2N+9$	$2N+11$	$\dots$	$2N+2n+2$	$\dots$

Così i valori di $\#_0$ sono tutti dispari ≥ 3 ; $\#_0$ è iniettiva ed effettiva.

2) Passiamo adesso alle istruzioni. Ciascuna di esse è una 5-upla $\Omega = (q, a, q', x)$ dove q, q' sono tra $q_0, q_1, \dots, q_n, \dots$, a, a' appartengono ad $A \cup \{\star\}$, $x = \pm 1$. Definiamo una funzione $\#_1$ che associa ad ogni Ω il numero naturale $\#_1(\Omega)$ in cui i codici dei cinque simboli componenti di Ω compaiono in ordine come esponenti dei primi 2, 3, 5, 7, 11

$$\#_1(\Omega) = 2^{\#_0(q)} 3^{\#_0(a)} 5^{\#_0(q')} 7^{\#_0(a')} 11^{\#_0(x)}.$$

Si noti che i codici assegnati da $\#_1$ sono numeri pari che hanno gli esponenti dispari o nulli nella loro decomposizione in fattori primi: dunque, non si confondono con quelli di $\#_0$, che sono dispari. Anche $\#_1$ è una funzione iniettiva, come conseguenza del teorema fondamentale dell'aritmetica e dell'injectività di $\#_0$; $\#_1$ è anche effettiva.

3) A questo punto siamo in grado di associare un numero naturale anche ad ogni funzione di transizione δ . Supponiamo che δ si componga delle quintuple $\Omega_1, \Omega_2, \dots, \Omega_k$. Ordiniamo le componenti delle 5-uple convenendo, per esempio,

- $q_0 < q_1 < q_2 < \dots < q_n < \dots$ tra gli stati,
- $\star < a_1 < \dots < a_N$ per l'alfabeto,

- $-1 < +1$ tra gli spostamenti.

Deduciamo su δ l'ordine lessicografico: cos , per esempio, $(q_0, a_1, q_1, a_2, 1)$ precede $(q_1, a_2, q_0, a_1, -1)$ perch  $q_0 < q_1$; $(q_1, a_1, q_0, a_1, -1)$ precede $(q_1, a_2, q_0, a_0, 1)$ perch  $a_1 < a_2$. Supponiamo che si abbia per tal via $\Omega_1 < \Omega_2 < \dots < \Omega_k$. Allora poniamo

$$\mathbb{h}_2(\delta) = \prod_{1 \leq i \leq k} p_i^{\mathbb{h}_1(\Omega_i)}$$

dove p_i  , al solito, l' i -esimo numero primo. Otteniamo una funzione iniettiva ed effettiva $\mathbb{h}_2$ che non confonde i suoi valori con quelli di $\mathbb{h}$ o di $\mathbb{h}_1$ (grazie alle consuete argomentazioni). Per questo motivo $\mathbb{h}_2$ non   suriettiva.

4) A questo punto ogni MdT M su A risulta determinata da:

- il numero $n + 1$ dei suoi stati,
- il codice $\mathbb{h}_2(\delta)$ della sua funzione di transizione,

dunque da una coppia ordinata di naturali; anzi, tramite la biiezione r di $\mathbb{N}^2$ su $\mathbb{N}$, risulta definita una funzione iniettiva $\mathbb{h}$ da MdT a naturali, quella che associa a ogni M l'immagine in r della coppia di numeri che determina M .

5) $\mathbb{h}$   effettiva e iniettiva, ma non suriettiva perch  neppure $\mathbb{h}_2$ lo  . Comunque possiamo ordinare gli elementi dell'immagine di $\mathbb{h}$

$$r_0, r_1, r_2, \dots, r_j, \dots \quad (j \in \mathbb{N});$$

ogni r_j corrisponde in $\mathbb{h}$ a un'unica MdT su A , che denotiamo M_j . Quindi $\mathbb{h}(M_j) = r_j$ per ogni j . Cos  anche le MdT su A risultano enumerate come

$$M_0, M_1, M_2, \dots, M_j, \dots;$$

la funzione $j \mapsto M_j$ (per $j \in \mathbb{N}$)   la biiezione richiesta. $\square$

L'enumerazione delle MdT appena ottenuta implica immediatamente una enumerazione anche delle funzioni calcolabili da MdT su un alfabeto A

0	1	2	3	4	...	i	...
$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	$\downarrow$	...	$\downarrow$	...
ϕ_0	ϕ_1	ϕ_2	ϕ_3	ϕ_4	...	ϕ_i	...

dove la funzione ϕ_i denota la funzione calcolata dalla i -esima MdT M_i su A . Stavolta perch  l'enumerazione delle funzioni calcolabili pu  non essere una corrispondenza biunivoca poich  pi  MdT possono computare la stessa funzione. Nel seguito, quindi, parleremo per ogni alfabeto A della i -esima MdT M_i su A e della i -esima funzione calcolabile ϕ_i (per intendere la funzione calcolata da M_i). Quando una funzione ϕ_i   definita su un dato input w (cio  M_i converge sull'input w) allora scriveremo $\phi_i(w) \downarrow$, altrimenti scriveremo $\phi_i(w) \uparrow$.

2.9 La Tesi di Church-Turing

Come già accennato nel Capitolo 1, la macchina di Turing non è l'unico modello astratto di computabilità finora comparso. Anzi, durante gli anni '30 e all'inizio degli anni '40, molte importanti caratterizzazioni della calcolabilità hanno visto la luce: oltre a quella di Turing, anche quella di Kleene, basata sulle equazioni funzionali, quella di Church, fondata sul λ -calcolo, ed altre ancora. In ogni caso, gli autori hanno elaborato modelli elementari di computazione e hanno formulato l'ipotesi che la loro nozione di calcolabilità fosse la più generale possibile e che non potessero esistere procedimenti algoritmici di calcolo non esprimibili nel loro formalismo. Alcuni esempi di questi modelli alternativi saranno presentati nei capitoli successivi.

D'altra parte, tutti i tentativi fatti nell'ambito della logica matematica e dell'informatica teorica di definire modelli e paradigmi alternativi alle MdT hanno condotto a caratterizzazioni di insiemi decidibili, semidecidibili e di funzioni calcolabili equivalenti a quelle di Turing (esempi di queste equivalenze verranno dimostrati nei prossimi capitoli). Questa circostanza ha condotto alla formulazione di quella che oggi è nota come *Tesi di Church-Turing* e viene ancora ritenuta universalmente valida. Questa tesi afferma che ogni procedimento algoritmico, espresso in un qualunque modello di calcolo, è realizzabile mediante una macchina di Turing. In realtà il contributo di Alonzo Church alla formulazione di questa affermazione può sembrare misterioso, e si potrebbe ritenere più consono chiamarla semplicemente *Tesi di Turing*. Spiegheremo nel Capitolo 4 il ruolo di Church. In conclusione si ha:

Tesi di Church-Turing. *Una funzione è calcolabile se e solo se esiste una macchina di Turing che la calcola (e cioè se e solo se la funzione è calcolabile secondo Turing).*

Che ogni funzione calcolata da una MdT sia effettivamente calcolabile è banale. Ciò che è invece rilevante e problematico è l'implicazione inversa, per la quale ogni procedimento algoritmico è riconducibile a una MdT. "Algoritmo" e "funzione calcolabile" sono concetti intuitivi, non specificati in modo formale, per cui non è possibile una dimostrazione rigorosa di equivalenza con il concetto di macchina di Turing. La Tesi di Church-Turing non è dunque una congettura che, in linea di principio, potrebbe un giorno diventare un teorema. Tuttavia, la nozione intuitiva di funzione calcolabile è contraddistinta da un insieme di caratteristiche (quali determinismo, finitezza di calcolo, etc.), che possono essere considerate in larga misura "oggettive". Questo fa sì che sia praticamente sempre possibile una valutazione concorde nel decidere se un dato procedimento di calcolo possa essere considerato algoritmico o meno. Quindi, almeno in linea di principio, è ammissibile che venga "scoperto" un controesempio alla Tesi di Church-Turing: che si individui cioè una funzione che sia effettivamente calcolabile secondo questi parametri informali, ma che non sia computata da nessuna MdT. Almeno finora, però, nessun controesempio alla tesi di Church-Turing è stato trovato nonostante gli ovvi progressi teorici e pratici che l'Informatica ha avuto dagli anni '30. In

effetti, gli argomenti a favore della Tesi di Church-Turing possono essere raccolti in tre gruppi.

a. *Evidenza euristica.*

- Per ogni singola funzione calcolabile che sia stata esaminata, è sempre stato possibile trovare una MdT in grado di computarla. Lo stesso si può dire dei procedimenti che producono effettivamente nuove funzioni a partire da altre. Eppure questa indagine è stata condotta per un gran numero di funzioni, di classi di funzioni e di procedure, con l'intento di renderla la più esaustiva possibile.
- I metodi per dimostrare che le funzioni computabili sono calcolabili secondo Turing sono stati sviluppati con un grado di generalità tale da far ritenere improbabile che possa essere scoperta una funzione calcolabile cui l'approccio di Turing non possa essere applicato.
- I vari metodi tentati per costruire funzioni effettivamente calcolabili, ma non computabili da MdT, hanno condotto tutti al fallimento, nel senso che le funzioni ottenute si dimostrano a loro volta calcolabili secondo Turing, oppure in realtà non calcolabili.

b. *Equivalenza delle diverse formulazioni proposte.* Tutti i tentativi che sono stati elaborati per caratterizzare in modo rigoroso la classe di tutte le funzioni effettivamente calcolabili si sono rivelati tra loro equivalenti. Ciò che è particolarmente rilevante è la diversità degli strumenti e dei concetti impiegati nelle diverse formulazioni; in molti casi esse traggono la loro origine da concetti matematici preesistenti. Che punti di vista talmente diversi e vasti convergano concentricamente alla stessa conclusione (e siano comunque equivalenti al modello di Turing) può essere inteso come forte argomento a sostegno di ognuno di loro e in particolare della tesi di Church-Turing.

c. *L'impiegato diligente.* In realtà, i precedenti ragionamenti si applicano indifferentemente a tutti i vari approcci equivalenti alla calcolabilità. Ma c'è un'ulteriore riflessione che serve in particolare a sostenere l'approccio di Turing. In effetti, le caratteristiche che lo distinguono dagli altri sono la sua naturalezza, la sua minore astrazione, e anzi il suo dichiarato proposito di simulare meccanicamente il comportamento di un essere umano che esegue un calcolo (l'impiegato diligente). In questo senso il modello di Turing si lascia preferire. Del resto, avremo modo nei prossimi capitoli di presentare altri approcci alla calcolabilità e di constatare direttamente le loro maggiori complicazioni teoriche. Anzi, ci furono matematici illustri, come Kurt Gödel, che, pur perplessi di fronte all'astrazione di questi metodi alternativi, si dichiararono tuttavia pienamente convinti e conquistati dalla naturalezza del modello di Turing.

Queste argomentazioni hanno indotto la teoria della computabilità a sviluppare la tendenza di considerare la Tesi di Church-Turing come una sorta di "*legge empirica*", piuttosto che come un enunciato a carattere "*logico-formale*".

In altre parole, si è autorizzati a procedere come segue. Se un certo linguaggio ammette un qualche algoritmo (di qualunque natura) che lo decide o accetta, oppure se una certa funzione ha un algoritmo che la calcola, allora, sulla base della

tesi di Church-Turing, possiamo assumere che ci sia una macchina di Turing che decide o accetta quel linguaggio, o calcola quella funzione: non c'è bisogno di descrivere in dettaglio la MdT, possiamo affidarci all'autorevolezza della tesi e non attardarci in ulteriori verifiche.

La tesi di Church-Turing permette allora, per ogni algoritmo che sia intuitivamente tale (anche se espresso in maniera informale), di dedurre l'esistenza di una MdT che lavora in modo equivalente. L'uso della tesi di Church-Turing consente di esprimere rapidamente risultati che un approccio più formale costringerebbe a raggiungere più faticosamente. Per esempio, lavoriamo con numeri naturali x, y, z e con un alfabeto che li rappresenta, e consideriamo la funzione così definita:

$$f(x, y, z) = \begin{cases} 1 & \text{se l'x-esima macchina } M_x \text{ converge in } z \text{ passi sull'input } y, \\ 0 & \text{altrimenti;} \end{cases}$$

sulla base della tesi di Church-Turing, possiamo dire che essa è calcolabile senza fornire necessariamente una MdT che la computa. È infatti sufficiente fornire un algoritmo che la calcoli anche ricorrendo a procedure di più alto livello. Per esempio: si manda in esecuzione M_x sull'input y per z passi di computazione; a ogni passo si incrementa di 1 un contatore C (che inizialmente viene posto a 0) e si controlla se la macchina di Turing ha terminato la sua esecuzione (cioè ha raggiunto una configurazione terminale). Se M_x si arresta quando il contatore non ha ancora raggiunto z allora l'algoritmo dà in output il valore 1. Se invece il contatore supera z senza che M_x si sia fermata, l'algoritmo termina e restituisce in output 0.

2.10 Macchine di Turing a più nastri

La descrizione informale della macchina di Turing data nel paragrafo 2 in termini di nastro, celle e testina è assai elementare, essenziale e schematica, certamente passibile di adattamenti, modifiche e migliorie. Per esempio, tra le varianti ammissibili, possiamo considerare l'eventualità di disporre di più nastri di lavoro. Ma il meccanismo che ne deriva, anche se apparentemente più duttile e potente della semplice MdT, mantiene in realtà le medesime capacità computazionali: i

linguaggi decisi o accettati restano esattamente gli stessi, così come le funzioni calcolate. Vediamo perché.

Consideriamo allora una macchina di Turing M a più nastri (si veda Figura 2.1) immaginata come composta da:

- una unità di controllo a stati finiti,
- un nastro N_0 di input-output di lunghezza infinita con relativa testina,
- m nastri ausiliari $N_1, \dots, N_m$ di lunghezza infinita ($m \geq 0$), ciascuno dotato di propria testina.

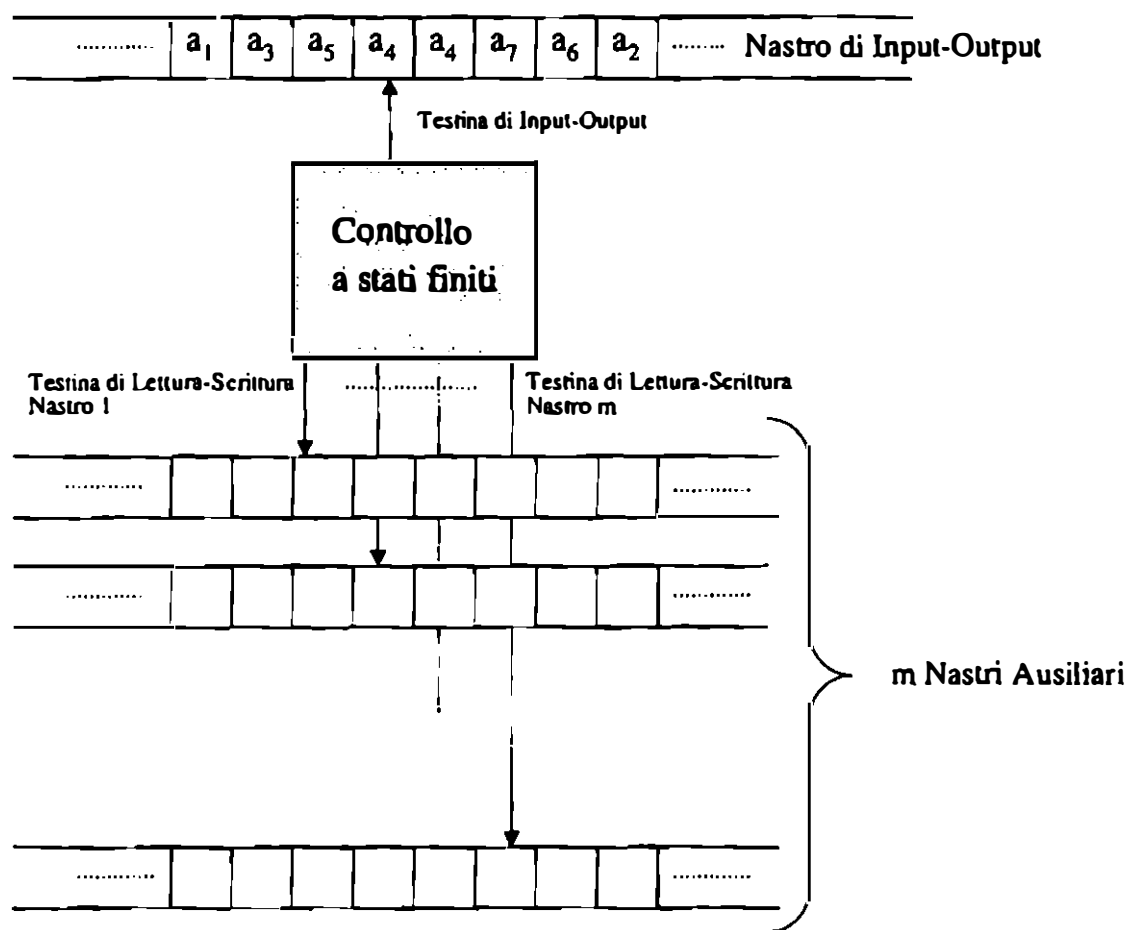


Figura 2.1 Schema di una Macchina di Turing a più nastri

Ogni nastro è suddiviso in celle, e ogni cella è in grado di memorizzare un simbolo di un certo alfabeto $A = \{a_0, a_1, \dots, a_n\}$, o il simbolo bianco $\star$.

Il nastro di input-output N_0 contiene l'input iniziale e l'eventuale output finale di ogni computazione; ma, durante il lavoro, anche gli altri nastri $N_1, \dots, N_m$ possono essere coinvolti e adoperati. Ogni nastro è scandito dalla sua testina, secondo le convenzioni usuali per MdT semplici. L'unità di controllo, oltre a contenere il programma secondo cui la computazione deve essere eseguita, controlla lo stato della macchina.

All'inizio della computazione, l'input è scritto su N_0 e la testina ne indica il simbolo più a sinistra; gli altri nastri sono vuoti e la testina ne esamina una cella arbitraria; la macchina M è nello stato q_0 . Ogni successiva mossa di M è determinata da

- il suo stato,

- i simboli indicati dalle testine sui vari nastri $N_0, N_1, \dots, N_m$,

e consiste in

- aggiornare lo stato,
- riscrivere i simboli esaminati su ogni nastro,
- spostare in ogni nastro la testina di un passo verso destra o verso sinistra.

Come già detto, si conviene che l'eventuale output della computazione sia la stringa scritta in N_0 se e quando M si arresta.

Tutte le definizioni già date per la MdT semplice possono essere adattate al nuovo modello, ovviamente con le opportune modifiche. In particolare, le regole di transizione di una macchina M a m nastri ausiliari sono fatalmente più elaborate, perché devono tenere conto non solo dello stato di M e del simbolo indicato su N_0 , ma anche dei simboli considerati sui nastri ausiliari $N_1, \dots, N_m$; la transizione sarà caratterizzata, come detto, dalla scelta di un nuovo stato e dalle istruzioni di scrittura e spostamento per ciascun nastro, sia di N_0 che di $N_1, \dots, N_m$. Una regola di transizione sarà allora della forma

$$\delta(q, b_0, b_1, \dots, b_m) = (q', b'_0, x_0, b'_1, x_1, \dots, b'_m, x_m)$$

dove:

- q, q' rappresentano gli stati di M rispettivamente prima e dopo la transizione,
- per ogni $i \in \{0, 1, \dots, m\}$, $b_i \in A \cup \{\star\}$ è il simbolo indicato dalla testina sul nastro N_i , $b'_i \in A \cup \{\star\}$ il simbolo che lo sostituisce, $x_i \in \{-1, +1\}$ lo spostamento che la testina deve compiere.

Il modello che si ottiene sembra più potente ed espressivo delle semplici MdT.

Esempio. Vediamo in particolare come una MdT con un nastro ausiliario N_1 sappia controllare, per ogni parola $w = a_{j_1} \dots a_{j_k}$ su A , se $w = w^{rev}$ e cioè se

$$a_{j_1} a_{j_2} \dots a_{j_k} = a_{j_k} a_{j_{k-1}} \dots a_{j_1};$$

sappia cioè decidere il linguaggio

$$\{(w, w^{rev}) | w \in A^*, w = w^{rev}\}.$$

La computazione di M su un generico w avviene come segue: M inizia il suo lavoro muovendo simultaneamente la testina di N_0 e quella del nastro ausiliario N_1 verso destra, una cella per volta, finché la testina di N_0 non incontra $\star$, cioè termina di leggere w . Durante questo movimento, M copia su N_1 i simboli letti; quindi M scorre all'indietro N_1 e individua il primo simbolo non bianco. Finalmente, M legge simultaneamente il nastro N_0 da destra a sinistra e N_1 da sinistra a destra e controlla che i simboli esaminati coincidano uno a uno. A seconda dell'esito della verifica scrive *SÌ* o *NO* su N_0 .

Nonostante le apparenze, le capacità computazionali di una MdT a più nastri sono le stesse di quelle a un solo nastro, come adesso proviamo.

Teorema 2.10.1 *Per ogni MdT $M = (Q, A, \delta, q_0)$ a m nastri ausiliari, esiste una MdT $M' = (Q', A', \delta', q_0)$ con $A \supseteq A'$ tale che:*

- *per ogni linguaggio L su A , L è accettato o deciso da M se e solo se lo è da M' ;*
- *per ogni funzione f su stringhe in A^* , f è calcolata da M se e solo se lo è da M' .*

Dimostrazione. Dobbiamo simulare il comportamento di M con una MdT “tradizionale” M' , nel senso che, per ogni $w \in A^*$,

- i. se $M \downarrow w$, allora $M' \downarrow w$ e M, M' hanno lo stesso output su w ;
- ii. se $M \uparrow w$, allora $M' \uparrow w$.

Suddividiamo allora l'unico nastro di M' in $2m + 2$ tracce consecutive

$$N'_0, N'_1, \dots, N'_m, N''_0, N''_1, \dots, N''_m.$$

Un nuovo simbolo Δ nell'alfabeto serve a separarle, segnalando inizio e fine di ognuna; un ulteriore simbolo ∇ delimita a sinistra e a destra la sequenza delle tracce. Per ogni $i = 0, 1, \dots, m$, le tracce N'_i, N''_i corrispondono al nastro N_i di M , in particolare

- N'_i ne ricopia il contenuto significativo (cioè non vuoto),
- N''_i provvede a indicare la posizione della testina di N_i .

A quest'ultimo proposito, ammettiamo che A' includa due ulteriori simboli $+, -$ e conveniamo che la cella di N''_i corrispondente a quella indicata dalla testina di N_i contenga $+$, le altre $-$. Per esempio, se N'_i contiene la stringa $a_0a_1a_0a_2$ e la testina indica a_1 , N'_i presenta $a_0a_1a_0a_2$ e $N''_i = +---$. Naturalmente il numero delle celle nella traccia N'_i è lo stesso che in N''_i , ma questo comune valore può variare durante la computazione in ragione dei movimenti di M su N_i : i delimitatori Δ, ∇ possono però spostarsi e segnalare con chiarezza i confini. All'inizio della computazione su un dato input w , il nastro di M' si presenta allora nel seguente modo:

- N'_0 contiene w , N''_0 ha $+$ nella prima cella, $-$ nelle altre;
- per $i = 1, \dots, m$, N'_i è bianca, cioè contiene $\star$, N''_i ha $+$ nella prima cella e $-$ in tutte le altre.

A questo punto M' può iniziare la simulazione della macchina M . Ad ogni regola di transizione di M

$$\delta(q, b_0, b_1, \dots, b_m) = (q', b'_0, x_0, b'_1, x_1, \dots, b'_m, x_m)$$

M' fa corrispondere le istruzioni adeguate a svolgere il seguente lavoro:

- M' entra nello stato q' ;

- per ogni $i = 1, \dots, m$, M' sostituisce b_i con b'_i su N'_i , e registra lo spostamento x_i su N''_i con l'uso appropriato di $+$ e $-$.

Quando M converge, M' opera una opportuna pulizia del nastro lasciando solo il contenuto di N'_0 .

Ovviamente questa descrizione del comportamento di M' è forzatamente imprecisa. Le operazioni sopra descritte necessiterebbero di una implementazione in termini di quintuple secondo quanto richiesto da una MdT tradizionale. Ma, con un minimo di pazienza, il discorso può essere precisato. In conclusione M' simula adeguatamente il comportamento di M , e corrisponde alle richieste dell'enunciato del teorema. $\square$

Viceversa, ogni MdT tradizionale può essere facilmente intesa come MdT a più nastri (che in realtà non ha bisogno di ricorrere ai nastri ausiliari). Dunque i due modelli, con o senza nastri ausiliari, si rivelano equivalenti.

2.11 Macchine di Turing non deterministiche

In questa sezione mostreremo un'altra possibile estensione delle MdT, quella delle così dette macchine di Turing “*non deterministiche*”; ne discuteremo brevemente utilità e motivazione, ma dimostreremo che anche questa generalizzazione equivale in definitiva al modello delle macchine di Turing tradizionali.

In una comune MdT $M = (Q, A, \delta, q_0)$, per ogni stato $q \in Q$ e per ogni simbolo $a \in A \cup \{\star\}$, c'è al più un'istruzione da eseguire a proposito di (q, a) . In altre parole, δ è una “funzione”:

- associa a (q, a) un'unica terna (q', a', x) con $q' \in Q$, $a' \in A \cup \{\star\}$, $x = \pm 1$,
- oppure esclude (q, a) dal suo dominio (nel qual caso M si ferma non appena incontra (q, a)).

Del resto, queste convenzioni sono stabilite per simulare in modo adeguato il comportamento tipico di un impiegato diligente che, se non ha istruzioni, si ferma; se ne ha una sola, la esegue; se ne ha più d'una, resta nel dubbio e non si prende responsabilità.

Ma a un impiegato diligente e *fortunato* si può anche concedere la facoltà di scegliere, confidando che la buona sorte gli ispiri l'opzione migliore. Corrispondentemente a una MdT non deterministica si concederà, per certe - o per tutte - le coppie (q, a) , la possibilità di scegliere tra più possibili terne (q', a', x) , tra loro concorrenziali. In altre parole, si ritirerà a δ la qualifica di funzione da coppie in $Q \times (A \cup \{\star\})$ a terne in $Q \times ((A \cup \{\star\}) \times \{-1, +1\})$, e la si ridurrà al rango di una semplice relazione tra coppie e terne, quindi a un insieme di 5-uple in

$$Q \times (A \cup \{\star\}) \times Q \times ((A \cup \{\star\}) \times \{-1, +1\}).$$

Si potrà in conclusione

Definizione. Una macchina di Turing *non deterministica* M è una 4-upla (Q, A, δ, q_0) come nella definizione delle MdT tradizionali, dove però δ è una relazione in

$$Q \times (A \cup \{\star\}) \times Q \times ((A \cup \{\star\}) \times \{-1, +1\}).$$

In effetti le MdT tradizionali si chiamano talora “*deterministiche*”, a sottolineare l’univocità delle loro istruzioni. Va però osservato che le MdT deterministiche rientrano tra quelle non deterministiche: hanno maggiore rigidità e meno libertà rispetto alle altre - al più una istruzione da eseguire per una data coppia (q, a) -, ma non ne contraddicono la definizione.

In ragione della molteplicità di istruzioni, una MdT non deterministica M può svolgere più possibili computazioni e produrre diversi possibili output per lo stesso input w . Diremo che M *converge* su w se almeno una di queste computazioni ha fine, e dedurremo che M *diverge* su w se tutte queste computazioni divergono.

Esempio. Sia M la MdT non deterministica che ha due stati q_0, q_1 , l’alfabeto $\{1\}$ e due sole istruzioni

$$(q_0, \star) \rightarrow (q_1, 1, +1), \quad (q_0, \star) \rightarrow (q_0, \star, +1),$$

tutte relative a $(q_0, \star)$. Ammettiamo che M inizi la computazione dal nastro bianco e quindi si trovi a considerare al primo passo proprio la coppia $(q_0, \star)$. Se M esegue la prima istruzione, M entra nello stato q_1 e subito dopo si arresta. Se invece M sceglie la seconda istruzione, M ricopia $\star$, va a destra e resta nello stato q_0 , cioè si ritrova nella situazione di partenza, una cella più a destra. Se M continua a eseguire indefinitamente la seconda istruzione su $(q_0, \star)$, M diverge. Comunque M converge sul nastro bianco perché almeno una delle sue computazioni sul nastro bianco ha fine.

È evidente che questa capacità di computazioni plurime dà a una MdT non deterministica M maggiori potenzialità, almeno apparenti: dove una MdT deterministica ha un’unica via da seguire, M ha più alternative e quindi la possibilità di concludere più rapidamente scegliendo la strada migliore. Così, M accetta un linguaggio L se e solo se gli elementi di L coincidono con gli input di M su cui almeno una computazione di M converge. Tuttavia, alla resa dei conti, questi presunti vantaggi non aumentano realmente il potere computazionale delle macchine, nel senso del seguente

Teorema 2.11.1 *Per ogni MdT non deterministica M , c’è una MdT deterministica M' che accetta lo stesso insieme di L .*

Dimostrazione. (Ci limitiamo a cenni informali). Per ogni input w , M' segue contemporaneamente tutte le computazioni di M su w , e si arresta non appena una di queste si arresta; continua a osservare il comportamento di M altrimenti. $\square$

2.12 Esercizi

1. Nella definizione di MdT si allarghino i movimenti consentiti alla testina, prevedendo che, oltre che spostarsi a destra o sinistra (-1 e $+1$), possa anche restare ferma su una cella dopo un passo (0). Si provi che questo modello rivisto di MdT decide e accetta gli stessi linguaggi e calcola le stesse funzioni di quello definito nel corso del capitolo.
2. Rappresentiamo i numeri naturali con l'unico simbolo 1 , in modo che $0, 1, 2, 3, \dots$ diventano $1, 11, 111, 1111, \dots$ rispettivamente. Si costruiscano MdT sull'alfabeto $\{1\}$ che calcolino le funzioni seguenti:
 - a) la funzione *successore* $s : \mathbb{N} \rightarrow \mathbb{N}$ tale che $s(n) = n + 1$ per ogni naturale n ,
 - b) la funzione *zero* $z : \mathbb{N} \rightarrow \mathbb{N}$ tale che $z(n) = 0$ per ogni naturale n ,
 - c) la funzione *proiezione* $p : \mathbb{N}^2 \rightarrow \mathbb{N}$ tale che $p(n, m) = n$ per ogni scelta di n, m naturali.
3. Consideriamo adesso i naturali come parole su $\{0, 1\}$ rispetto alla loro rappresentazione in base 2. Si costruisca una MdT che decida l'insieme dei pari, e una che lo accetti.
4. Stesso quesito dell'esercizio 3, ma rispetto alla base 10, cioè con alfabeto $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$.
5. Consideriamo ancora i numeri naturali rappresentati su qualche alfabeto A . Enumeriamo le funzioni che sono calcolate da MdT su A $\phi_0, \phi_1, \phi_2, \dots$. Definiamo poi una funzione parziale f da $\mathbb{N}$ a $\mathbb{N}$ ponendo, per ogni $n \in \mathbb{N}$,

$$f(n) = \begin{cases} \phi_n(n) + 1 & \text{se } \phi_n(n) \downarrow, \\ \uparrow & \text{altrimenti.} \end{cases}$$

- a) Stabilire se f è calcolabile.
- b) Dimostrare che f non può essere estesa ad una funzione g calcolabile e totale (cioè non esiste una funzione g che ha dominio $\mathbb{N}$, soddisfa $f(n) = g(n)$ per ogni n nel dominio di f ed è calcolabile).

Riferimenti bibliografici Le macchine di Turing sono introdotte in [108]. Una loro ampia discussione, utile anche per i capitoli futuri si trova in [52]. Si vedano anche le trattazioni del modello di Turing in [7, 51, 76, 80, 84].

Problemi senza Soluzione

3.1 Diagonalizzare e ridurre

Nel capitolo precedente abbiamo introdotto il modello di calcolo della macchina di Turing e abbiamo conseguentemente definito la computabilità secondo Turing (per linguaggi o per funzioni). Nel presente capitolo approfondiamo questa teoria. Uno dei suoi aspetti più rilevanti è l'esistenza di problemi non computabili: per la precisione, emergono dal modello di Turing

- funzioni che non si possono calcolare,
- linguaggi che non si possono accettare, o che si accettano ma non si decidono.

Esperimenteremo d'altra parte la possibilità di costruire per ogni alfabeto A una MdT “*universale*”, capace di svolgere ogni computazione proponibile su A . In realtà, limiteremo per semplicità la nostra analisi ai numeri naturali, visti come parole su qualche opportuno alfabeto $\{0, 1, \dots, 9\}$ o $\{0, 1\}$, o $\{1\}$: questa assunzione è implicita in tutto il capitolo, salvo avviso contrario. Del resto, le strategie di codifica apprese nel Capitolo 2 ci mostrano come ogni alfabeto A si riduca effettivamente al contesto dei naturali. Ricordiamo poi che le MdT sull'alfabeto fissato si enumerano (in più possibili modi)

$$M_0, M_1, M_2, \dots, M_n, \dots \quad (n \in \mathbb{N}),$$

così come le funzioni da esse calcolate

$$\phi_0, \phi_1, \phi_2, \dots, \phi_n, \dots \quad (n \in \mathbb{N});$$

(ϕ_n è in particolare la funzione calcolata da M_n). Nel nostro caso, quando lavoriamo su $\mathbb{N}$, possiamo assumere che ciascuna ϕ_n sia una funzione, eventualmente parziale, da $\mathbb{N}$ a $\mathbb{N}$. Tanto l'enumerazione delle MdT quanto quella conseguente delle funzioni calcolabili sono da ritenersi fissate per tutto il capitolo. I risultati che otterremo non dipendono comunque dalla loro scelta. Ricordiamo anche la seguente notazione: per f funzione parziale da $\mathbb{N}$ a $\mathbb{N}$ e n naturale, $f(n) \downarrow$ significa che n è nel dominio di f , cioè $f(n)$ è definita, $f(n) \uparrow$ il contrario. Il riferimento ai naturali permette alcune utili semplificazioni del contesto. Per esempio, per $L \subseteq \mathbb{N}$, e quindi L linguaggio sull'alfabeto fissato, è definita la *funzione*

caratteristica di L , cioè la funzione totale f_L da $\mathbb{N}$ a $\mathbb{N}$ tale che, per ogni $x \in \mathbb{N}$,

$$f_L(x) = \begin{cases} 1 & \text{se } x \in L, \\ 0 & \text{altrimenti.} \end{cases}$$

Si ha allora:

Lemma 3.1.1 Per $L \subseteq \mathbb{N}$, L è decidibile se e solo se f_L è calcolabile.

Dimostrazione. Esercizio. □

Il risultato si estende facilmente al caso in cui L è sottoinsieme di $\mathbb{N}^2$, $\mathbb{N}^3$ e così via, tramite le biiezioni effettive tra questi insiemi e $\mathbb{N}$.

Quanto ai risultati negativi di incomputabilità sopra menzionati, incontreremo due tecniche particolari che permettono di provarli:

1. La prima si chiama *diagonalizzazione* e riesce a produrre, ad esempio, funzioni non calcolabili manipolando opportunamente la lista di tutte le funzioni calcolabili;
2. la seconda prende il nome di *riduzione*: essenzialmente deduce l'incapacità di calcolare una funzione f o di decidere un linguaggio L riferendoli a situazioni negative già note e provando che queste ultime avrebbero soluzione positiva se altrettanto valesse per f e L .

3.2 Problemi risolubili algoritmicamente

Il paragrafo è dedicato a presentare e discutere alcuni esempi di funzioni calcolabili secondo Turing. Per cominciare non è difficile vedere che le familiari operazioni dell'aritmetica, come l'addizione, la moltiplicazione, la divisione - intesa come la coppia di funzioni parziali quoziente e resto - sono tutte calcolabili; del resto, il Capitolo 2 include già esempi ed esercizi a questo proposito.

Ecco un caso più sofisticato e astratto di funzione calcolabile.

Esempio. Sia f la funzione da $\mathbb{N}$ a $\mathbb{N}$ definita come segue: per ogni $x \in \mathbb{N}$,

$$f(x) = \begin{cases} 1 & \text{se } \phi_x(x) \downarrow, \\ \uparrow & \text{altrimenti.} \end{cases}$$

Dimostriamo che f è una funzione calcolabile. Usiamo la Tesi di Church-Turing proponendo un algoritmo che la calcola a un alto livello di astrazione e deducendo che c'è una MdT che la computa. L'algoritmo procede così. Dato un input x , si decodifica x per ottenere la MdT M_x che calcola ϕ_x . A questo punto si manda in esecuzione M_x sull'input x . Se e quando l'esecuzione termina, si converge sull'output convenuto 1. Altrimenti la divergenza di M_x induce la divergenza dell'algoritmo che calcola f .

Ci sono poi casi sorprendenti e “anomali” di funzioni che si dimostrano calcolabili senza che si riesca a determinare con precisione una MdT, o anche un algoritmo, che le computi. Eccone un esempio.

Esempio. Consideriamo la funzione totale $g : \mathbb{N} \rightarrow \mathbb{N}$ tale che, per ogni $x \in \mathbb{N}$,

$$g(x) = \begin{cases} 1 & \text{se nell'espansione decimale di } \pi \\ & \text{esistono almeno } x \text{ 5 consecutivi,} \\ 0 & \text{altrimenti} \end{cases}$$

La funzione g è calcolabile perché esiste infatti un algoritmo che genera l'espansione decimale di π . Allora per g ci sono due casi possibili. Ammettiamo dapprima che l'espansione decimale di π riesca a includere un numero arbitrariamente grande di occorrenze successive di '5'; allora g coincide con la funzione costante 1, che è chiaramente calcolabile. Altrimenti c'è un massimo numero k di '5' consecutivi nello sviluppo decimale di π , e quindi, per ogni $x \in \mathbb{N}$,

$$g(x) = \begin{cases} 1 & \text{se } x \leq k, \\ 0 & \text{altrimenti.} \end{cases}$$

Anche questa seconda opzione è calcolabile.

Pertanto g corrisponde ad una delle due possibilità elencate, e in ogni caso g è calcolabile. D'altra parte siccome lo sviluppo decimale di π è infinito aperiodico, non è dato sapere direttamente da una sua lettura parziale se vale il primo o il secondo caso per g e, semmai, quale è k nel secondo caso, nè si conoscono attualmente altre strategie per chiarire la questione. In conclusione, sappiamo che g ha un algoritmo di calcolo, ma non sappiamo quale è questo algoritmo.

Esistono poi funzioni di cui non è ancora noto se siano calcolabili o non calcolabili.

Esempio. Consideriamo la funzione totale $g' : \mathbb{N} \rightarrow \mathbb{N}$ definita come segue: per ogni $x \in \mathbb{N}$,

$$g'(x) = \begin{cases} 1 & \text{se nell'espansione decimale di } \pi \\ & \text{esistono esattamente } x \text{ 5 consecutivi,} \\ 0 & \text{altrimenti.} \end{cases}$$

Chi potesse leggere per esteso lo sviluppo decimale di π potrebbe anche calcolare g' , esaminandovi le occorrenze di 5; ma purtroppo lo sviluppo di π è infinito e sfugge ai nostri occhi. Potrebbero tuttavia esistere altre strategie più mirate per risolvere il problema. Ma ad oggi non è ancora noto alcun algoritmo convincente. Quindi, per quanto ne sappiamo, g' può essere calcolabile, ma può anche non esserlo.

Esempi di funzioni non calcolabili in alcun modo saranno presentati nei prossimi paragrafi.

3.3 La Macchina Universale

I moderni calcolatori, così come li intendiamo comunemente, sono “macchine” capaci di eseguire svariati programmi. Una macchina di Turing invece, al di là del nome e della sua descrizione esteriore, è determinata dalla funzione δ che ne stabilisce le istruzioni ed è quindi un programma di computazione, delegato a calcolare sempre la stessa funzione. C’è tuttavia modo di costruire una MdT $\mathcal{U}$ che riesce a simulare il lavoro di ogni altra MdT M , nel senso che, per ogni M , e per ogni input naturale y , $\mathcal{U}$ accoglie (M, y) come input e riproduce la computazione di M su y . Per questo $\mathcal{U}$ è chiamata “universale”. Vediamo come è possibile definirla.

C’è una ovvia perplessità relativa a questa ipotetica $\mathcal{U}$, e cioè: come si fa a fornire a $\mathcal{U}$ come input un’altra MdT M ? Infatti l’alfabeto di $\mathcal{U}$ tratta numeri e non macchine. D’altra parte, abbiamo visto nel Capitolo 2 come ogni MdT riceva un suo numero di etichetta, secondo una biiezione effettiva tra numeri e macchine. In altre parole $M = M_x$ per un unico naturale x che può essere esplicitamente calcolato da M . Quindi possiamo proporre a $\mathcal{U}$ come input

- il numero x come codice di M ,
- il numero y come input di M .

Ci aspettiamo che $\mathcal{U}$ ci fornisca come output della coppia (x, y) quello di $M_x = M$ su y , e cioè $\phi_x(y)$. Dimostriamo allora l’esistenza di $\mathcal{U}$.

Teorema 3.3.1 *Esiste una MdT universale $\mathcal{U}$. In altre parole, la funzione $g : \mathbb{N}^2 \rightarrow \mathbb{N}$ definita ponendo, per ogni $x, y \in \mathbb{N}$,*

$$g(x, y) = \phi_x(y)$$

è calcolabile secondo Turing.

Dimostrazione. Informalmente, ci basta, per ogni scelta di $x, y \in \mathbb{N}$, prima decodificare x per ottenere la MdT x -esima M_x , poi far lavorare M_x su y . Entrambi i passaggi sono possibili, in quando la decodifica è algoritmica e le regole di funzionamento di una macchina di Turing sono anch’esse algoritmiche.

Comunque, per non far sembrare questa soluzione un gioco di prestigio, diamo una descrizione più dettagliata della costruzione di $\mathcal{U}$. $\mathcal{U}$ può essere pensata come una macchina con due nastri ausiliari (sappiamo infatti che questo modello è equivalente a quello di una MdT con un solo nastro). Dapprima $\mathcal{U}$ controlla che il suo input sia una coppia (M, y) , per qualche macchina di Turing $M = M_x$ e qualche input y per M . Poi $\mathcal{U}$ passa a simulare M sull’input y . In particolare, $\mathcal{U}$ adopera il primo nastro ausiliario per mantenere traccia degli stati e della posizione della testina di M e il secondo nastro per memorizzare le istruzioni di M . Così $\mathcal{U}$ copia, di volta in volta, una configurazione (ξ, q, a, η) di M dal nastro di input/output sul primo nastro ausiliario; poi per determinare la regola di transizione (q, a, q', a', x) di M da usare, $\mathcal{U}$ estrae q e a , quindi determina (q', a', x) cercando nel secondo nastro l’istruzione per (q, a) . A quel punto si comporta come dettato dalla terna (q', a', x) per sviluppare sul nastro di input/output la sua computazione. $\square$

3.4 Il Problema dell'Arresto

Esistono funzioni che le macchine di Turing non riescono a calcolare, e linguaggi che non riescono ad accettare, oppure accettano e non riescono a decidere. Ne abbiamo già accennato alla fine del paragrafo 3.2. Così, se accogliamo la tesi di Church-Turing, queste funzioni e questi linguaggi non ammettono alcun algoritmo capaci di trattarli.

Il prototipo di queste situazioni è il seguente

Problema dell'Arresto. Determinare un algoritmo per stabilire, per ogni scelta di una MdT M e di un input $y \in \mathbb{N}$, se $M \downarrow y$ o no.

Allora, se prestiamo fede alla tesi di Church-Turing secondo cui tutto quello che è alitmicamente risolubile può essere trattato con successo da MdT, il Problema dell'Arresto ci chiede una MdT che, ricevuti in input $x, y \in \mathbb{N}$ (e in particolare, tramite x , la MdT M_x), risponda

- *SI* se $M \downarrow y$,
- *NO* se $M \uparrow y$,

decida cioè l'insieme

$$K' = \{(x, y) \in \mathbb{N}^2 : M \downarrow y\}.$$

Notiamo che la macchina cercata non può essere quella universale $\mathcal{U}$. Infatti $\mathcal{U}$, dati x, y , simula la computazione di M_x su y , ma non può prevederne l'esito e, se M_x diverge su y , è condannata a divergere su (x, y) , a proseguire cioè indefinitamente la sua computazione senza potersi accorgere a nessun passo dell'assenza di una conclusione. Invece la MdT cercata converge per ogni scelta di x, y e anzi calcola la seguente funzione totale h' da $\mathbb{N}^2$ a $\mathbb{N}$: per $x, y \in \mathbb{N}$,

$$h'(x, y) = \begin{cases} 1 & \text{se } \phi_x(y) \downarrow, \\ 0 & \text{se } \phi_x(y) \uparrow. \end{cases}$$

Ebbene questa macchina non esiste. Anzi, vale un risultato per certi versi più negativo.

Teorema 3.4.1 *La funzione h tale che, per ogni input $x \in \mathbb{N}$,*

$$h(x) = \begin{cases} 1 & \text{se } \phi_x(x) \downarrow, \\ 0 & \text{se } \phi_x(x) \uparrow \end{cases}$$

non è calcolabile da nessuna MdT.

Dimostrazione. Supponiamo per assurdo il contrario, che esista cioè una MdT con questa capacità. Consideriamo la seguente funzione k per $x \in \mathbb{N}$,

$$k(x) = \begin{cases} \uparrow & \text{se } h(x) = 1, \\ 0 & \text{se } h(x) = 0. \end{cases}$$

Dalla calcolabilità della funzione h segue facilmente quella di k . Dunque c'è una MdT che calcola k ; sia $j \in \mathbb{N}$ il suo codice; in altre parole sia $\phi_j = k$. Ma allora abbiamo

$$\phi_j(j) \downarrow \iff k(j) = 0 \iff h(j) = 0 \iff \phi_j(j) \uparrow,$$

il che costituisce una evidente contraddizione. Quindi h non è calcolabile. $\square$

Il metodo usato per trovare una contraddizione nella prova del precedente teorema è un esempio della tecnica della *diagonalizzazione* già introdotta nel paragrafo 3.1; dall'elenco

$$\phi_0, \phi_1, \phi_2, \dots$$

delle funzioni calcolabili e dall'apparentemente innocua equivalenza, valida per ogni naturale x ,

$$k(x) \downarrow \text{ se e solo se } \phi_x(x) \uparrow$$

si deduce, per $k = \phi_j$ e per $x = j$, la contraddizione

$$\phi_j(j) \downarrow \text{ se e solo se } \phi_j(j) \uparrow$$

e dunque si conclude che k non può essere calcolabile.

Corollario 3.4.2 *L'insieme $K = \{x \in \mathbb{N} : M_x \downarrow x\}$ non è decidibile.*

Dimostrazione. La funzione h del teorema 3.4.1 non è altro che la funzione caratteristica di K . $\square$

Corollario 3.4.3 *La funzione h' tale che, per ogni scelta di $x, y \in \mathbb{N}$,*

$$h'(x, y) = \begin{cases} 1 & \text{se } \phi_x(y) \downarrow, \\ 0 & \text{se } \phi_x(y) \uparrow \end{cases}$$

non è calcolabile da nessuna MdT.

Dimostrazione. Altrimenti si vede facilmente che anche h è calcolabile; infatti $h(x) = h'(x, x)$ per ogni $x \in \mathbb{N}$ $\square$

Corollario 3.4.4 *L'insieme $K' = \{(x, y) \in \mathbb{N}^2 : M_x \downarrow y\}$ non è decidibile.*

Dimostrazione. Basta notare che $h' = f_{K'}$. $\square$

La prova di questi corollari illustra il metodo della *riduzione* introdotto sempre nel paragrafo 3.1: per esempio l'incapacità di calcolare h' è dedotta dall'analogo risultato per h . Diagonalizzazione e riduzione costituiscono entrambe utili strumenti teorici per dimostrare l'indcidibilità di linguaggi e l'incomputabilità di funzioni.

La soluzione negativa del problema dell'arresto ha delle importanti conseguenze anche nell'ambito della teoria della programmazione. In particolare, essa indica chiaramente che non è possibile costruire un perfetto sistema di “*debugging*”, che sia in grado di stabilire se un programma termini o meno sull'input dato.

Notiamo poi che, sebbene il problema dell'arresto, nella sua piena generalità, sia stato dimostrato indecidibile, è tuttavia possibile ottenere risultati positivi in casi parziali. Per esempio abbiamo già affermato nel capitolo precedente la calcolabilità della funzione f che a ogni terna (x, y, z) di naturali associa

$$f(x, y, z) = \begin{cases} 1 & \text{se } M_x \text{ sull'input } y \text{ termina in } z \text{ passi,} \\ 0 & \text{altrimenti.} \end{cases}$$

In questo caso, infatti, si limita con z il numero di passi di computazione di M_x , e questa specificazione consente il calcolo effettivo dell'immagine di f in ogni situazione.

Il problema dell'arresto non è l'unico esempio indecidibile in Informatica Teorica. Altre questioni condividono la sua assenza di soluzioni algoritmiche. Citiamo qui il caso del *Problema dell'equivalenza tra MdT* che adesso enunciamo.

Problema dell'Equivalenza tra Macchine di Turing. Determinare un algoritmo per decidere, per ogni scelta di due naturali x, y (e delle corrispondenti MdT M_x e M_y), se M_x e M_y calcolano o no la stessa funzione, cioè se $\phi_x = \phi_y$ o no.

In altre parole, sulla base della tesi di Church-Turing, si domanda se $E = \{(x, y) \in \mathbb{N}^2 : \phi_x = \phi_y\}$ è o no decidibile. Si intende con $\phi_x = \phi_y$ che ϕ_x e ϕ_y hanno lo stesso dominio e agiscono allo stesso modo sui suoi elementi. Ma si prova che il problema non è decidibile secondo Turing e dunque, per la tesi di Church-Turing, non è risolubile da nessun modello di calcolo.

Un caso particolarmente critico in questo ambito più “generale” è quello dei linguaggi di programmazione cui dedicheremo il Capitolo 6. Qui la soluzione negativa del problema dell'equivalenza implica che non sarà mai possibile decidere se due generici programmi calcolano la stessa funzione o, in altri termini, se hanno la stessa semantica. Ciò preclude la speranza di trovare un algoritmo per l'analisi e la verifica di correttezza automatica dei programmi: le tecniche di *testing* rimangono così le uniche strategie per scoprire errori in un dato programma.

La nostra dimostrazione dell'indecidibilità del problema dell'equivalenza per MdT procede in tre passi.

- a) Dapprima mostriamo che non è possibile decidere se una funzione calcolabile è totale (cioè converge su tutti gli input) o meno.
- b) Proviamo poi che non è possibile riconoscere quali funzioni calcolabili coincidono con l'identità id di $\mathbb{N}$, cioè con la funzione di $\mathbb{N}$ in $\mathbb{N}$ che lascia fisso ogni naturale.
- c) Dimostreremo infine l'indecidibilità dell'equivalenza tra MdT.

Si noti che anche i passi a), b) costituiscono per proprio conto risultati di indecidibilità. Procediamo con la prova di a). Usiamo un argomento di *diagonalizzazione*:

Teorema 3.4.5 *L'insieme*

$$T = \{x \in \mathbb{N} : \phi_x \text{ è totale (cioè } \phi_x \text{ converge su ogni input)}\}$$

non è decidibile.

Dimostrazione. Ammettiamo per assurdo che T sia decidibile. Allora siamo capaci di riconoscere effettivamente nell'enumerazione $\phi_0, \phi_1, \phi_2, \dots$ di tutte le funzioni calcolabili quelle che sono anche totali. C'è dunque una funzione totale e calcolabile f tale che, per ogni $x \in \mathbb{N}$,

$$f(x) = \text{indice della } x\text{-ma funzione totale nell'enumerazione.}$$

Così ogni funzione calcolabile totale è della forma $\phi_{f(x)}$ per un unico naturale x . Poniamo adesso

$$g(x) = \phi_{f(x)}(x) + 1, \text{ per ogni } x \in \mathbb{N}.$$

Allora, per ogni x , g calcola l'output $\phi_{f(x)}$ di $M_{f(x)}$ su x e poi gli aggiunge 1. Ovviamente g è calcolabile totale. Così esiste un unico $x_0 \in \mathbb{N}$ tale che $g = \phi_{f(x_0)}$. Ma allora

$$\phi_{f(x_0)}(x_0) + 1 = g(x_0) = \phi_{f(x_0)}(x_0),$$

e questo è assurdo. Segue che T è indecidibile. □

Passiamo alla prova di b). Usiamo qui una tecnica di *riduzione* ad a).

Teorema 3.4.6 *L'insieme* $I = \{x \in \mathbb{N} : \phi_x = id\}$ *non è decidibile.*

Dimostrazione. Mostriamo che, se I è decidibile, anche l'insieme T del precedente teorema lo è; così possiamo concludere che I non è decidibile.

Consideriamo la funzione parziale f da $\mathbb{N}^2$ a $\mathbb{N}$ tale che, per ogni scelta di x, y naturali,

$$f(x, y) = \begin{cases} y & \text{se } \phi_x(y) \downarrow, \\ \uparrow & \text{altrimenti.} \end{cases}$$

Non è difficile vedere che f è calcolabile. Supponiamo di fissare il primo argomento x della funzione f . Otteniamo allora per ogni x una funzione calcolabile g_x da $\mathbb{N}$ a $\mathbb{N}$ così definita: per ogni y ,

$$g_x(y) = \begin{cases} y & \text{se } \phi_x(y) \downarrow, \\ \uparrow & \text{altrimenti.} \end{cases}$$

Tutte le funzioni g_x sono calcolabili e quindi compaiono nell'enumerazione delle funzioni calcolabili. Così c'è una funzione calcolabile r da $\mathbb{N}$ a $\mathbb{N}$ tale che, per ogni $x \in \mathbb{N}$,

$$\phi_{r(x)} = g_x.$$

Osserviamo poi che, per ogni x ,

$$g_x = id \text{ se e solo se } \phi_x \text{ è totale.}$$

Segue che, per ogni x , $x \in T$ se e solo se ϕ_x è totale, cioè se e solo se $\phi_{r(x)} = id$ e in conclusione se e solo se $r(x) \in I$. Così un algoritmo che decide I ne produce uno che decide T : basta che, per ogni $x \in \mathbb{N}$, prima si calcoli $r(x)$ e poi si applichi a $r(x)$ l'algoritmo di I ; se esso ci dice $r(x) \in I$, deduciamo $x \in T$, e viceversa. $\square$

Teorema 3.4.7 *L'insieme $E = \{(x, y) \in \mathbb{N}^2 : \phi_x = \phi_y\}$ non è decidibile.*

Dimostrazione. Di nuovo usiamo una tecnica di *riduzione*, stavolta riferita a b). Osserviamo che la funzione id è certamente calcolabile e dunque $id = \phi_j$ per qualche j . Ma allora, per ogni x ,

$$x \in I \text{ se e solo se } (x, j) \in E.$$

Quindi E non è decidibile perché non lo è neppure I . $\square$

3.5 Insiemi decidibili e semidecidibili

Nel Capitolo 2 abbiamo introdotto non solo linguaggi decidibili, ma anche semidecidibili. Ricordiamo la definizione di entrambi questi concetti (assumendo semmai per semplicità di trattare linguaggi $L \subseteq \mathbb{N}$). Allora si ha:

- L è decidibile se e solo se c'è una MdT che sa decidere, per ogni $x \in \mathbb{N}$, se $x \in L$ o no (e dunque, in senso lato, se e solo se c'è un algoritmo che sa svolgere questo compito),
- L è semidecidibile se e solo se c'è una MdT che lo accetta e cioè converge su tutti e soli gli elementi di L .

Cerchiamo di cogliere meglio il senso di questa seconda nozione e mostriamo:

Teorema 3.5.1 *Sia $L \subseteq \mathbb{N}$. Allora sono equivalenti le proposizioni*

- L è semidecidibile,
- L è il dominio di una funzione calcolabile,
- L è vuoto o c'è una funzione totale calcolabile f da $\mathbb{N}$ a $\mathbb{N}$ tale che L è l'immagine di f (e dunque gli elementi di L sono enumerati effettivamente come $f(0), f(1), f(2)$ e così via).

Dimostrazione. L'equivalenza tra (a) e (b) è diretta conseguenza della definizione. Se $M = M_x$ è la MdT che accetta L , L è il dominio di ϕ_x , e viceversa. Passiamo allora a confrontare (a) e (c).

(a) $\Rightarrow$ (c). Supponiamo dapprima L semidecidibile e fissiamo una MdT M che converge su tutti e soli gli elementi di L . Enumeriamo L seguendo le computazioni di M sui naturali $0, 1, 2, \dots, m, \dots$: non appena ci si accorge che la computazione di M su uno di questi elementi m ha termine, si aggiunge m all'elenco di L . C'è però un'ovvia obiezione che si può sollevare per questo procedimento, e cioè come controllare contemporaneamente il lavoro di M sugli infiniti input m , tanto più che ogni computazione richiede i suoi passi $0, 1, 2, \dots, n, \dots$ e talora, nei casi di divergenza, infiniti passi. Per esempio, se M diverge su 0 come facciamo ad avviare la computazione su $1, 2, \dots$ e verificarne l'esito? La situazione richiama però le difficoltà già affrontate nel Capitolo 2 per costruire una biiezione tra coppie (m, n) di naturali e naturali, e risolte tramite la sequenza $(0, 0), (0, 1), (1, 0)$ e così via. Proprio con l'uso della stessa idea si riesce a gestire il nuovo ambito: si segue il passo 0 di M su 0 , poi il passo 1 su 0 , il passo 0 su 1 , e via dicendo. In questo modo, se $L \neq \emptyset$, si generano i suoi elementi

$$l_0, l_1, l_2, \dots, l_j, \dots$$

(se $L = \emptyset$, L già soddisfa (c)). La lista così ottenuta è finita per L finito ma, ammettendovi ripetizioni, possiamo supporla infinitamente lunga, cioè con j che varia tra tutti i naturali. Poniamo adesso, per ogni $j \in \mathbb{N}$,

$$f(j) = l_j.$$

Allora f è una funzione totale calcolabile, e la sua immagine coincide proprio con L .

(c) $\Rightarrow$ (a). Supponiamo viceversa che ci sia una funzione f con queste proprietà. Costruiamo una MdT M che converge su tutti e soli gli elementi x di L procedendo come segue. Per ogni $x \in \mathbb{N}$, M computa finché necessario $f(0), f(1), f(2), \dots$ e per ognuno controlla contestualmente se coincide o no con x . Se la verifica è positiva, M si arresta, se no prosegue il suo lavoro. Grazie a M , L è semidecidibile. Lo stesso accade, ovviamente, se $L = \emptyset$: in tal caso L è accettato da ogni MdT che diverge sempre (il lettore può provare a definirne una). $\square$

In conclusione, L (se non vuoto), è semidecidibile se e solo se c'è un programma (e dunque una MdT) che enumera effettivamente i suoi elementi. Abbiamo così un'ulteriore conferma del seguente fatto, già osservato nel Capitolo 2.

Osservazione. Se L è decidibile, allora L è semidecidibile.

Infatti una MdT che sa riconoscere gli elementi di L e scartare gli altri si adatta facilmente a elencare gli elementi di L (se ve ne sono).

Non sembra invece chiaro il contrario, se cioè un insieme semidecidibile non vuoto sia anche decidibile. Ammettiamo infatti di avere un algoritmo che produce successivamente tutti gli elementi di L . Ogni suo output è ovviamente in L . Tuttavia non c'è verso di riconoscere per suo tramite gli elementi x fuori di L : chi osserva gli elementi prodotti dall'algoritmo, prende atto che, dopo 10 , o 10^2 , o 10^n passi, x non è ancora presente tra essi, ma non può dedurre che x non comparirà nei passi successivi.

Visto che ci siamo, notiamo anche:

Osservazione. L è decidibile se e solo se $\mathbb{N} - L$ lo è.

Infatti, per ogni $x \in \mathbb{N}$, $x \in \mathbb{N} - L$ se e solo se $x \notin L$. Così una MdT che decide L si può facilmente adattare a decidere $\mathbb{N} - L$ (e viceversa): basta scambiare le risposte finali, dire NO (a $x \in \mathbb{N} - L$?) quando si diceva $SÌ$ (a $x \in L$?), e viceversa.

Di nuovo, non sembra affatto chiaro che altrettanto valga per la semidecidibilità. Infatti, se L è semidecidibile e dunque sappiamo riconoscere gli elementi di L come quelli su cui un'opportuna MdT M converge, non è detto che sappiamo riconoscere per tal via anche gli altri, quelli su cui M diverge.

In effetti si ha:

Teorema (Post) 3.5.2 *Sia $L \subseteq \mathbb{N}$. Allora L è decidibile se e solo se tanto L quanto $\mathbb{N} - L$ sono semidecidibili.*

Dimostrazione. Sappiamo già che, se L è decidibile, allora L è semidecidibile. Del resto, se L è decidibile, anche $\mathbb{N} - L$ lo è, quindi $\mathbb{N} - L$ è semidecidibile.

Viceversa, assumiamo $L, \mathbb{N} - L$ semidecidibili. Ci sono due MdT $M(L)$ e $M(\mathbb{N} - L)$ che accettano $L, \mathbb{N} - L$ rispettivamente. Per ogni $x \in \mathbb{N}$, seguiamo le computazioni di $M(L)$ e $M(\mathbb{N} - L)$ su x . Una e una sola delle due converge, perché x è in uno e uno solo degli insiemi $L, \mathbb{N} - L$. Se converge $M(L)$, si dichiara $x \in L$; se converge $M(\mathbb{N} - L)$, si conclude $x \in \mathbb{N} - L$. $\square$

Il precedente paragrafo ci ha fornito vari esempi di insiemi decidibili, e tra questi

$$K = \{x \in \mathbb{N} : M_x \downarrow x\}.$$

D'altra parte si ha:

Teorema 3.5.3 *K è semidecidibile. In particolare ci sono linguaggi semidecidibili e non decidibili.*

Dimostrazione. Ecco la descrizione informale di una MdT M che accetta K : per ogni $x \in \mathbb{N}$, M costruisce M_x e ne segue la computazione su x ; se e quando M_x converge, anche M converge. Altrimenti, M diverge insieme a M_x . $\square$

Corollario 3.5.4 *$\mathbb{N} - K$ non è semidecidibile (e dunque esistono linguaggi non semidecidibili).*

Dimostrazione. Altrimenti dal teorema di Post segue che K è decidibile. $\square$

Il lettore potrà controllare per **esercizio** l'eventuale semidecidibilità degli altri insiemi indecidibili incontrati nel precedente paragrafo

$$T = \{x \in \mathbb{N} : \phi_x \text{ è totale}\}, \quad I = \{x \in \mathbb{N} : \phi_x = id\}$$

e dei loro complementari $\mathbb{N} - T$, $\mathbb{N} - I$; potrà anche allargare l'esame in $\mathbb{N}^2$ a

$$K' = \{(x, y) \in \mathbb{N}^2 : M_x \downarrow y\}, \quad E = \{(x, y) \in \mathbb{N}^2 : \phi_x = \phi_y\}.$$

Noi dedichiamo invece la parte finale del paragrafo a un'appendice del problema dell'arresto. Abbiamo stabilito che la questione di decidere, per ogni MdT M e per ogni input x , se $M \downarrow x$ o no è priva di algoritmi di soluzione. Si può pensare che, se si fissa M e ci si limita a controllare per quali x $M \downarrow x$ e per quali no, la situazione si semplifichi e ammette risposta positiva. Certe volte è davvero così. Per esempio, se M è una MdT senza istruzione, allora M converge su ogni input e quindi

$$\{x \in \mathbb{N} : M \downarrow x\}$$

coincide con $\mathbb{N}$ ed è ovviamente decidibile. Ma talora la situazione resta così complicata come in generale. Infatti si ha:

Corollario 3.5.5 *Ci sono MdT M tali che $\{x \in \mathbb{N} : M \downarrow x\}$ è indecidibile.*

Dimostrazione. Basta sfruttare la semidecidibilità di K e prendere una MdT che accetta K . Allora $\{x \in \mathbb{N} : M \downarrow x\} = K$ è indecidibile. Lo stesso ragionamento si applica, ovviamente, a ogni linguaggio semidecidibile e indecidibile. $\square$

3.6 Un'altra funzione non calcolabile

Negli scorsi paragrafi abbiamo presentato casi di funzioni che non sono calcolabile e di linguaggi che non sono decidibili secondo la definizione di Turing. In genere, questi esempi sono stati relativamente tecnici e poco naturali, scarsamente legati ai familiari contesti numerici. Nei prossimi due paragrafi, introduciamo due esempi più semplici ed accessibili, l'uno di una funzione che non è calcolabile, l'altro di un insieme che non è decidibile. Cominciamo con il caso della funzione. Va detto che non è affatto facile immaginare funzioni non calcolabili; infatti le funzioni che ci vengono in mente sono per lo più calcolabili (e proprio in quanto tali si affacciano alla nostra percezione). Nel 1962, comunque, il matematico ungherese Tibor Rado riuscì a ideare un esempio relativamente semplice ed anche simpatico (per quanto può essere simpatica una funzione di Matematica). La funzione in questione viene chiamata Σ di Rado, va dall'insieme $\mathbb{N}$ dei naturali a $\mathbb{N}$ stesso ed è definita nel modo che segue.

Sia dato un numero naturale n . Dobbiamo costruire $\Sigma(n)$. Costruiamo allora tutte le macchine di Turing $M = (Q, A, \delta, q_0)$ dove A è un alfabeto con un solo simbolo $A = \{1\}$, Q si compone di $n + 1$ stati $q_0, q_1, \dots, q_n$ e, inoltre,

- (i) δ non ha istruzioni sulle coppie $(q_n, \star)$, $(q_n, 1)$ relative allo stato q_n ,
- (ii) M converge sul nastro bianco.

Tra tutte queste MdT organizziamo il seguente gioco (che ebbe da Rado il nome inglese di “*busy beaver competition*”, in italiano “*gioco del castoro laborioso*”):

ogni MdT ottiene per punteggio il numero di 1 che riesce a scrivere sull'output della sua computazione (convergente!) sul nastro bianco. $\Sigma(n)$ è il punteggio vincente in questo torneo.

Notiamo che almeno la macchina vuota (quella priva di istruzioni) è ammessa a giocare per $\Sigma(n)$ perchè soddisfa ovviamente le due condizioni (i) e (ii); tra l'altro, essendo priva di istruzioni, si arresta ancor prima di partire, non riesce a stampare nessun 1 e quindi ottiene punteggio 0. Così il gioco di $\Sigma(n)$ trova almeno un concorrente. D'altra parte, perchè ci sia un punteggio vincente e quindi $\Sigma(n)$ sia correttamente definita, è bene escludere che i concorrenti siano troppi, cioè infiniti (in tal caso potrebbe anche non esserci un risultato massimo). Ma, per questo, basta osservare che tutte le macchine di Turing sull'alfabeto $\{1\}$ con $n + 1$ stati sono complessivamente un numero finito, e dunque anche quelle che rispettano (i) e (ii) e sono di conseguenza ammesse a giocare rientrano nell'ambito finito.

Vediamo adesso alcuni possibili valori di Σ .

Esempio.

1. È facile vedere che $\Sigma(0) = 0$. Infatti, per $n = 0$, le macchine ammesse a giocare hanno un solo stato q_0 e nessuna istruzione che lo riguardi, dunque si arrestano immediatamente su qualunque input, in particolare su quello bianco. In altre parole, l'unica MdT concorrente è quella priva di istruzioni ed ottiene punteggio 0.
2. È relativamente semplice anche controllare che $\Sigma(1) = 1$. Infatti stavolta le MdT M concorrenti hanno due stati q_0 e q_1 e mancano di istruzioni per q_1 . Se però la funzione di transizione δ di M ha un'istruzione del tipo $\delta(q_0, \star) = (q_0, \dots, \dots)$ (e quindi resta nello stato q_0 quando legge il simbolo bianco nello stato q_0), allora M , indipendentemente da quanto le viene ordinato di scrivere e da dove le viene comandato di spostarsi, finisce per divergere sul nastro bianco, e dunque è squalificata dal gioco. Ad esempio, se l'istruzione di δ è $\delta(q_0, \star) = (q_0, 1, +1)$, M finisce per spostarsi indefinitamente verso destra stampando 1 su ogni quadro incontrato, e comunque diverge. Lo stesso capita se altra è l'istruzione su che cosa scrivere o dove spostarsi, ma si mantiene l'ordine di rimanere nello stato q_0 . Dunque, tra le MdT M partecipanti al gioco soltanto due comportamenti sono ammessi: o δ non ha istruzioni su $(q_0, \star)$ (nel qual caso M converge subito con punteggio 0), oppure δ ha un'istruzione $\delta(q_0, \star) = (q_1, \dots, \dots)$, e allora M prima la esegue, stampando $\star$ o 1 sul nastro, poi entra nello stato q_1 e, non avendo disposizioni che lo riguardano, si arresta, con punteggio 0 o 1 rispettivamente. In conclusione, $\Sigma(1) = 1$, come detto.
3. Tuttavia, già il calcolo di $\Sigma(2)$ comincia a complicarsi. In effetti si riesce a verificare che $\Sigma(2) = 4$, ma si devono affrontare e superare seri ostacoli computazionali. Il motivo è semplice. Proviamo infatti a contare le MdT M a 3 stati q_0, q_1, q_2 che sono prive di istruzioni relative a q_2 e, in questo senso, sono potenziali concorrenti al gioco di $\Sigma(2)$. Esse corrispondono alle funzioni di transizione δ che vanno da coppie ordinate con q_0 e q_1 (ma non q_2) come prima componente e $\star$ o 1 come seconda e giungono a terne che

- hanno q_0 , q_1 o q_2 come prima componente, ancora $\star$ o 1 come seconda e, finalmente, ± 1 come terza. Restano in questo modo coinvolte $2 \times 2 = 4$ coppie e $3 \times 2 \times 2 = 12$ terne. Le funzioni totali che vanno dalle prime alle seconde sono allora $12^4 = 20.736$ (e ad esse andrebbero aggiunte le altre funzioni parziali, quelle il cui dominio accoglie solo alcune delle coppie sopra elencate): tante sono, indicativamente, le potenziali partecipanti al gioco di $\Sigma(2)$ e per ognuna di esse va esaminato il comportamento sul nastro bianco per computare $\Sigma(2)$. La loro analisi produce comunque il risultato $\Sigma(2) = 4$.
4. Anche i valori di Σ per $n = 3$ e $n = 4$ sono conosciuti, ma il loro calcolo è formidabile. $\Sigma(3) = 6$ fu provato da Lin e Rado nel 1965, ma richiede l'esame di oltre 16 milioni di MdT (al posto delle 20.736 di $\Sigma(2)$). È, invece, un teorema di Brady del 1983 il fatto che $\Sigma(4) = 13$: stavolta le potenziali concorrenti da analizzare sono altre 25 miliardi e stabilirne il punteggio non è affatto facile; pur tuttavia, una accurata classificazione dei loro comportamenti (cui si conferiscono nomi fantasiosi, come *alberi di Natale*, o *draghi che si mangiano la coda*) e un largo ricorso all'ausilio dei calcolatori permette la conclusione $\Sigma(4) = 13$.
5. Ma, per $n \geq 5$, i valori di Σ non sono ancora noti. Calcoli di Marxen e Buntrock del 2002 permettono al massimo di stabilire che $\Sigma(5) \geq 4.098$ e che $\Sigma(6)$ supera addirittura $1,29 \cdot 10^{865}$.

Del resto, non c'è da stupirsi di tutte queste complicazioni. Infatti, come Rado osservò sin dal 1962, Σ non è calcolabile, anzi supera asintoticamente qualunque funzione calcolabile. Ad esempio, provate a considerare le funzioni $f(n) = 2^n$, oppure 2^{2^n} , o ancora $2^{2^{2^n}}$, e così via. Un attimo di riflessione assicura che queste funzioni sono tutte calcolabili, ma crescenti rapidissimamente. Eppure il Teorema di Rado (che adesso dimostreremo) assicura che, da un certo n in poi, Σ le supera definitivamente. Non c'è allora da stupirsi che Σ finisca fuori da ogni ragionevole controllo umano.

Per dimostrare il Teorema di Rado ci serve prima premettere la semplice osservazione che anche Σ è una funzione crescente, nel senso che, per ogni naturale n ,

$$\Sigma(n) \leq \Sigma(n+1).$$

Infatti tutte le MdT M a $n+1$ stati ammesse a giocare per $\Sigma(n)$ possono essere facilmente adattate per partecipare al torneo di $\Sigma(n+1)$: basta aggiungere loro un nuovo stato q_{n+1} senza istruzioni che lo riguardino. In tal modo M resta sostanzialmente la stessa e quindi converge ancora sull'input bianco, producendo lo stesso output di prima; non ha istruzioni sul nuovo stato q_{n+1} (anzi, non ne ha neppure per q_n), quindi gioca anche per $\Sigma(n+1)$ ottenendo lo stesso punteggio che per $\Sigma(n)$. D'altra parte, il torneo di $\Sigma(n+1)$ ammette anche altre partecipanti assolutamente nuove, che possono raggiungere anche risultati superiori. Quindi $\Sigma(n) \leq \Sigma(n+1)$, come detto.

Possiamo adesso provare:

Teorema (Rado) 3.6.1 Σ non è calcolabile. Anzi, per ogni funzione calcolabile f da $\mathbb{N}$ a $\mathbb{N}$, esiste un naturale $k(f)$ tale che, per ogni $k > k(f)$, $\Sigma(k) > f(k)$.

Dimostrazione. Rappresentiamo per semplicità i naturali n sull'alfabeto $\{1\}$, come stringhe di 1: quindi 0, 1, 2, 3, ... diventano rispettivamente 1, 11, 111, 1111, e così via. In genere ogni n viene espresso da una stringa di $n + 1$ caratteri uguali a 1. Osserviamo poi che, se f è calcolabile, anche la funzione g che ad ogni naturale n associa il valore massimo tra $f(2n + 1)$ e $f(2n + 2)$ è chiaramente calcolabile. C'è dunque una MdT $M(g)$ su $\{1\}$ che la computa. Supponiamo che $M(g)$ ammetta $n(g)$ stati. Consideriamo adesso un qualunque numero naturale n . Possiamo costruire una MdT M_n che, a partire dall'input bianco, prima vi stampa n e poi simula $M(g)$, computando quindi in conclusione $g(n)$. Le istruzioni per M_n sono relativamente semplici da organizzare. Anzitutto dobbiamo consentirle di scrivere n , dunque $n + 1$ cifre consecutive tutte uguali a 1 (dopo di che la macchina deve tornare al simbolo 1 più a sinistra per poter simulare $M(g)$). Dobbiamo conseguentemente prevedere per M_n $n + 1$ stati $q_0, \dots, q_n$ in aggiunta a quelli di $M(g)$ e, per quanto riguarda la sua funzione di transizione δ_n , l'istruzione $\delta_n(q_i, \star) = (q_{i+1}, 1, -1)$ per ogni $i < n$: si ottiene così che M_n si muova verso sinistra e scriva esattamente n cifre uguali a 1. Aggiungiamo le ulteriori istruzioni

$$\delta_n(q_n, \star) = (q_n, 1, +1), \quad \delta_n(q_n, 1) = (q, 1, -1)$$

dove q è lo stato di partenza di $M(g)$: il loro effetto è di far stampare l'ultimo 1 a sinistra dei precedenti, e poi di portare M su questo 1 nel primo stato di $M(g)$. A questo punto aggiungiamo a M_n tutte le istruzioni di $M(g)$ nei relativi $n(g)$ stati, avendo cura di distinguere questi stati da quelli già adoperati per scrivere n . In questo modo M_n viene ad ammettere complessivamente $n(g) + n + 1$ stati e a convergere sull'input bianco con output $g(n)$, come richiesto. Comunque, se vogliamo coinvolgerla nel gioco del castoro laborioso, dobbiamo accrescerla di un ulteriore stato, ovviamente privo di istruzioni, e quindi prevedere in totale $n(g) + n + 2$ stati per M_n ; in questo modo, M_n concorre al torneo di $\Sigma(n(g) + n + 1)$ con punteggio $g(n) + 1$ (il numero degli 1 nella sequenza che rappresenta $g(n)$). Così $g(n) < g(n) + 1 \leq \Sigma(n(g) + n + 1)$. A questo punto è facile concludere: per $n \geq n(g)$, si ha infatti

$$f(2n + 1) \leq g(n) < \Sigma(n(g) + n + 1) \leq \Sigma(2n + 1),$$

$$f(2n + 2) \leq g(n) < \Sigma(n(g) + n + 1) \leq \Sigma(2n + 2)$$

(in ciascun caso, la prima disuguaglianza segue dalla definizione di g , la seconda è stata appena provata, l'ultima dipende dal fatto che Σ è crescente). La tesi è quindi dimostrata: se vogliamo essere scrupolosi e riferirci precisamente all'enunciato del teorema, fissiamo $k(f) = 2n(g)$ e, per $k > k(f)$, distinguiamo i due casi in cui k è dispari oppure pari (dunque $k = 2n + 1$ oppure $k = 2n + 2$ con $n \geq n(g)$) per concludere comunque $\Sigma(k) > f(k)$. $\square$

3.7 Il Decimo Problema di Hilbert

Adesso proponiamo l'esempio di un classico problema di Matematica che non ha algoritmo di soluzione perchè non è decidibile secondo Turing. Ci riferiamo al Decimo Problema di Hilbert, già trattato proprio all'inizio del libro. Ricordiamone l'enunciato.

Decimo Problema di Hilbert (1900). Determinare un algoritmo per decidere, per ogni intero positivo k e per ogni polinomio $p(x_1, \dots, x_k)$ con coefficienti interi e k indeterminate $x_1, \dots, x_k$, se $p(x_1, \dots, x_k)$ ha o no radici intere (se cioè esistono o no interi $z_1, \dots, z_k$ tali che $p(z_1, \dots, z_k) = 0$).

Si noti che il problema non pone limitazioni nè sul numero delle indeterminate nè sul grado complessivo del polinomio. Nel 1970, Yuri Matijasevic, coronando il lavoro di Julia Robinson, Martin Davis e altri negli anni precedenti, dimostrò che un algoritmo così generale non può esistere. Discutiamo brevemente la prova di Matijasevic.

Anzitutto si osservi che è sufficiente mostrare che non esiste alcun algoritmo per decidere, per intero positivo k e per ogni polinomio $p(x_1, \dots, x_k)$ a coefficienti interi, se $p(x_1, \dots, x_k)$ ha o no radici **naturali** (e cioè intere non negative). Infatti un classico teorema di Lagrange assicura che un numero intero z è ≥ 0 se e solo se si può esprimere in $\mathbb{Z}$ come somma di quattro quadrati (ad esempio, $4 = 2^2 + 0^2 + 0^2 + 0^2$, $7 = 2^2 + 1^2 + 1^2 + 1^2$, e così via). Di conseguenza, un eventuale algoritmo di decisione per le radici intere ne fornisce uno per $\mathbb{N}$: infatti un arbitrario polinomio $p(x_1, \dots, x_k)$ ammette radici naturali se e solo se il polinomio

$$p(x_{1,1}^2 + x_{1,2}^2 + x_{1,3}^2 + x_{1,4}^2, \dots, x_{k,1}^2 + x_{k,2}^2 + x_{k,3}^2 + x_{k,4}^2)$$

(in cui ogni vecchia indeterminata x_i ($1 \leq i \leq k$) è sostituita dalla sua espressione come somma dei quadrati di 4 nuove indeterminate $x_{i,1}^2 + x_{i,2}^2 + x_{i,3}^2 + x_{i,4}^2$, dunque si hanno $4k$ indeterminate e, ancora, coefficienti interi) ha radici intere.

A questo punto diciamo che un insieme S di naturali è *Diofanteo* se e solo se esistono un naturale k ed un polinomio $p(x, x_1, \dots, x_k)$ a coefficienti interi in $k+1$ indeterminate $x, x_1, \dots, x_k$ tali che S coincide con l'insieme di quei naturali n per cui $p(n, x_1, \dots, x_k)$ ha radici naturali. L'aggettivo *Diofanteo* è un tributo a Diofanto, matematico alessandrino dell'antichità che si interessò della soluzione dei polinomi a coefficienti interi.

Il risultato chiave della ricerca di Matijasevic è la seguente, sorprendente caratterizzazione dei sottoinsiemi Diofantei di $\mathbb{N}$.

Lemma (Matijasevic) 3.7.1 $S \subseteq \mathbb{N}$ è *Diofanteo* se e solo se è *semidecidibile*.

Ovviamente, pensiamo qui S come un insieme finito di stringhe su un qualche alfabeto per i naturali, ad esempio su $\{0, 1, \dots, 9\}$. Il risultato precedente è la chiave fondamentale di tutta la dimostrazione di Matijasevic. La difficoltà degli argomenti usati ci impediscono di darne resoconto qui.

Accettiamolo dunque per buono, e consideriamo l'insieme semidecidibile ma non decidibile $K \subseteq \mathbb{N}$ incontrato nella trattazione del problema dell'arresto per le MdT. Per il lemma, K è Diofanteo, dunque esiste un polinomio

$$p_K(x, x_1, \dots, x_k)$$

a coefficienti interi tale che K coincide con l'insieme di quei naturali n per i quali $p_K(n, x_1, \dots, x_k)$ ammette radici naturali; il polinomio $p_K(x, x_1, \dots, x_k)$ può essere effettivamente calcolato conoscendo K . A questo punto, se esiste un algoritmo per stabilire se un polinomio a coefficienti interi ammette o non radici naturali, si può facilmente derivarne un algoritmo di decisione per K : per ogni n naturale, si costruisce il polinomio $p_K(n, x_1, \dots, x_k)$ nelle indeterminate $x_1, \dots, x_k$ e si controlla se $p_K(n, x_1, \dots, x_k)$ ha o no radici naturali. Ma allora K è decidibile, e questo è assurdo.

In conclusione si ha:

Teorema (Matijasevic, 1970) 3.7.2 *Non esiste alcun algoritmo per risolvere il Decimo Problema di Hilbert.*

Come già accennato, il risultato negativo di Matijasevic si applica a polinomi di grado e numero di indeterminate arbitrari. Ove si pongano limitazioni sul grado dei polinomi oppure sul numero delle loro indeterminate, un algoritmo per la esistenza di radici intere può ben essere trovato. Ad esempio, possiamo attingere dai nostri ricordi di Matematica elementare e rammentare che, se poniamo $k = 1$ e quindi ci restringiamo ad una sola indeterminata x_1 , allora c'è un semplice algoritmo per gestire soddisfacentemente la situazione. Accenniamolo brevemente. Sia dunque

$$p(x_1) = a_0 + a_1x_1 + \dots + a_mx_1^m$$

il nostro polinomio a coefficienti interi nell'unica indeterminata x_1 . Se $a_0 = 0$, allora $p(x_1)$ ha la radice 0. Se invece $p(x_1) = a_0 \neq 0$, allora $p(x_1)$ non può avere radici. Assumiamo allora $a_0 \neq 0$, $p(x_1)$ di grado $m \geq 1$ (e dunque $a_m \neq 0$). Se $z \in \mathbb{Z}$ è radice di $p(x_1)$, si ha

$$a_0 = -z \cdot (a_1 + \dots + a_m z^{m-1}),$$

da cui segue che z divide a_0 . Perciò le eventuali radici intere di $p(x_1)$ si trovano tra i divisori del suo termine noto a_0 (che sono al più un numero finito). Allora un algoritmo in questo caso particolare è il seguente: determinare i divisori di a_0 e poi controllare se tra di essi si trova o no una radice di $p(x_1)$.

Per polinomi a 2 indeterminate, c'è ancora un algoritmo abbastanza generale, mentre il problema per 3 indeterminate è ancora aperto.

Algoritmi parziali di soluzione del Decimo Problema di Hilbert si hanno anche ove si limiti il grado del polinomio (ma non il numero delle indeterminate), almeno per grado 1 o 2. Per polinomi di grado 3, il problema è ancora aperto; per polinomi di grado 4 o maggiore, la soluzione, invece, è negativa come nel caso generale.

3.8 I teoremi di Kleene e di Rice

In questa sezione presentiamo due risultati di notevole importanza sulle funzioni calcolabili e sui linguaggi decidibili. Il primo è dovuto a Kleene ed è anche noto come *Teorema del punto fisso*, il secondo a Rice.

Il *Teorema di Kleene* contiene uno dei risultati più affascinanti della teoria della calcolabilità. Utilizza e sottolinea, infatti, una proprietà per certi verso paradossale che la codifica delle macchine di Turing conferisce alle MdT stesse, e cioè la capacità di riprodursi e determinare nuovi algoritmi. In effetti ogni funzione calcolabile totale t da $\mathbb{N}$ a $\mathbb{N}$ permette, per ogni $x \in \mathbb{N}$, alla MdT M_x e alla corrispondente funzione ϕ_x di “generare” rispettivamente la MdT $M_{t(x)}$ e la funzione $\phi_{t(x)}$. Anzi, se prestiamo fede alla tesi di Church-Turing, ogni procedimento effettivo che a partire da vecchi algoritmi ne produce di nuovi corrisponde a una tale funzione t .

Il contesto suggerisce in particolare la possibilità di definire una funzione calcolabile non solo esplicitamente, ma anche implicitamente, tramite condizioni quali

$$\phi_x = \phi_{x^2}$$

o, più in generale,

$$\phi_x = \phi_{t(x)}$$

per t totale calcolabile, identificando quindi ϕ_x come una delle funzioni che operano allo stesso modo di quelle che esse determinano tramite $x \mapsto x^2$, o una generica trasformazione t .

Il meccanismo di codifica conduce poi al notevole fenomeno dell'*autoriferimento* di un algoritmo: infatti ogni funzione ϕ_x può applicarsi a se stessa, più precisamente al proprio indice x , e usare x al proprio interno. Possiamo allora immaginare di identificare le funzioni calcolabili ϕ_x tramite condizioni di autoriferimento, per esempio chiedendo

$$\phi_x(y) = x, \text{ per ogni } y \in \mathbb{N}$$

e cioè che la MdT M_x produca come output di ogni input il proprio indice x . In realtà, a proposito di quest'ultimo esempio, l'intuizione provoca qualche fondato dubbio sulla sua attuabilità: infatti, una MdT M che include x nelle sue istruzioni riceve una codifica maggiore di x . Ma possiamo comunque ammettere che una MdT M possa calcolare un certo indice x e poi coinvolgere ϕ_x nelle sue computazioni, come suo sottoprogramma. Del resto, la dimostrazione del teorema di Kleene si basa essenzialmente su questa idea.

Il teorema di Kleene si riferisce ai meccanismi di produzione di programmi appena descritti e afferma che, per ogni procedura t con cui ogni MdT M_x genera una MdT $M_{t(x)}$, c'è sempre un programma che resta invariato, e cioè un $e \in \mathbb{N}$ tale che

$$\phi_e = \phi_{t(e)},$$

dunque $M_e, M_{t(e)}$ calcolano la stessa funzione. Per questo motivo il risultato di Kleene è anche chiamato *Teorema del punto fisso*: infatti t lascia invariato, se non il numero e , almeno il programma ϕ_e che gli corrisponde.

Teorema (Kleene) 3.8.1 *Per ogni funzione calcolabile totale t da $\mathbb{N}$ in $\mathbb{N}$ esiste $e \in \mathbb{N}$ tale che*

$$\phi_e = \phi_{t(e)}.$$

Dimostrazione. Per ogni naturale u , si consideri la MdT $M(u)$ che, dato un input x , calcola dapprima $\phi_u(u)$ e poi, se $\phi_u(u) \downarrow$, applica la funzione $\phi_{\phi_u(u)}$ a x con relativo output $\phi_{\phi_u(u)}(x)$; se invece $\phi_u(u) \uparrow$, anche $M(u)$ diverge su x . Così $M(u)$ prima computa l'indice $\phi_u(u)$ di un programma e poi applica questo programma. È facile convincersi che per ogni u una MdT $M(u)$ come descritta si può effettivamente costruire. Anzi la funzione g che associa ad ogni u il codice di $M(u)$, cioè $M(u) = M_{g(u)}$, è totale e calcolabile. Si noti che, per ogni u , la funzione $\phi_{g(u)}$ è definita ponendo, per ogni $x \in \mathbb{N}$,

$$\phi_{g(u)}(x) = \begin{cases} \phi_{\phi_u(u)}(x) & \text{se } \phi_u(u) \text{ è definita,} \\ \uparrow & \text{altrimenti} \end{cases}$$

(ovviamente $\phi_{g(u)}(x)$ può essere divergente anche nel primo caso, se x non è nel dominio di $\phi_{\phi_u(u)}$). Sia ora t una funzione calcolabile totale. Allora anche la computazione $t \circ g$ è calcolabile totale, e dunque $t \circ g = \phi_v$ per qualche naturale v . Inoltre, per ogni $x \in \mathbb{N}$,

$$\phi_{g(v)}(x) = \phi_{\phi_v(v)}(x) = \phi_{t(g(v))}(x).$$

Quindi $e = t(v)$ soddisfa la tesi del teorema. □

Il teorema di Kleene permette di dimostrare uno dei più forti risultati negativi della teoria della computabilità e cioè il *Teorema di Rice*; esso, infatti, asserisce che qualunque proprietà non banale delle funzioni calcolabili (e quindi degli algoritmi) è indecidibile.

Teorema (Rice) 3.8.2 *Sia F una famiglia di funzioni calcolabili. L'insieme*

$$S = \{x \in \mathbb{N} \mid \phi_x \in F\}$$

è decidibile se e solo se $F = \emptyset$ oppure F coincide con l'intera classe delle funzioni calcolabili.

Dimostrazione. Se $F = \emptyset$, $S = \emptyset$ è decidibile; se F coincide con la classe di tutte le funzioni calcolabili, $S = \mathbb{N}$ è ancora decidibile. Supponiamo allora che F non sia vuoto né contenga tutte le funzioni calcolabili, e mostriamo che S è indecidibile. Useremo per questo obiettivo un argomento di diagonalizzazione. Ammettiamo per assurdo S decidibile. Le ipotesi su F ci assicurano che ci sono funzioni calcolabili dentro e fuori di F : siano i e j rispettivamente il primo indice per cui $\phi_i \in F$, cioè $i \in S$, il primo indice per cui $\phi_j \notin F$, cioè $j \notin S$. Siccome S è decidibile, possiamo calcolare esplicitamente i e j , ricorrendo a una MdT M

che decide S . Basta che facciamo operare M su 0, poi su 1, e così via finché necessario. Se M stabilisce che 0 è in S , si pone $i = 0$, altrimenti si deduce $j = 0$; dopo di che si passa a 1. Se si è ottenuto $i = 0$, e M dichiara che $1 \notin S$, si pone $j = 1$; altrimenti per $i = 0$ e $1 \in S$, si passa a 2, certi di incontrare prima o poi nella sequenza dei naturali il nostro j . Analogamente si procede per $j = 0$. Di conseguenza la funzione g tale che, per ogni $x \in \mathbb{N}$,

$$g(x) = \begin{cases} i & \text{se } x \notin S \\ j & \text{se } x \in S \end{cases}$$

risulta calcolabile, e anche ovviamente totale. Inoltre si ha, per ogni $x \in \mathbb{N}$, che

$$g(x) \in S \text{ se e solo se } x \notin S,$$

cioè

$$\phi_{g(x)} \in F \text{ se e solo se } \phi_x \notin F.$$

Ma, siccome g è totale calcolabile, il teorema di Kleene ci fornisce un naturale e tale che $\phi_{g(e)} = \phi_e$, e questo conduce alla contraddizione

$$\phi_e \in F \text{ se e solo se } \phi_e \notin F.$$

Dunque S non può essere decidibile. □

Il teorema di Rice conferma l'ind decidibilità di certi linguaggi già incontrati in questo capitolo.

Esempio. L'insieme $T = \{x \in \mathbb{N} : \varphi_x \text{ è totale}\}$ corrisponde alla famiglia F delle funzioni calcolabili totali, che non è vuota ma neppure esaurisce la classe delle funzioni calcolabili. Così il teorema di Rice si applica e garantisce che $T = \{x \in \mathbb{N} : \varphi_x \in F\}$ non è decidibile.

Lo stesso si può affermare di $E = \{x \in \mathbb{N} : \varphi_x = id\}$, per il quale F si riduce alla sola funzione identità.

Le conseguenze del Teorema di Rice in termini di teoria della programmazione sono particolarmente significative. Infatti, in tale contesto, si deve ammettere l'impossibilità di provare esplicitamente specifiche proprietà, come la costanza, la crescita, la monotonia, e così via, delle funzioni calcolate da programmi. Infatti la classe delle funzioni che verificano una di queste proprietà corrisponde alle ipotesi del teorema di Rice.

In particolare, un'interessante applicazione, già discussa precedentemente, riguarda l'impossibilità di decidere la correttezza di un programma. Infatti la questione si traduce nel considerare l'insieme:

$$S = \{x \in \mathbb{N} : \phi_x = f\}$$

dove f è la *specifica* del programma e le varie ϕ_x rappresentano le *implementazioni* di f . Dal teorema di Rice segue che S è indecidibile.

3.9 Esercizi

1. Si dimostri che la funzione

$$f(x) = \begin{cases} 1 & \text{se } \phi_x(0) \downarrow \\ 0 & \text{se } \phi_x(0) \uparrow \end{cases}$$

non è calcolabile (suggerimento: ci si riduca al problema dell'arresto).

2. Si dica se la funzione D tale che, per ogni $x \in \mathbb{N}$,

$$D(x) = \begin{cases} 1 & \text{se Dio esiste,} \\ 0 & \text{altrimenti} \end{cases}$$

è calcolabile.

3. Si discuta la semidecidibilità degli insiemi

$$K' = \{(x, y) \in \mathbb{N}^2 : M_x \downarrow y\}, \quad T = \{x \in \mathbb{N} : \phi_x \text{ è totale}\},$$

$$I = \{x \in \mathbb{N} : \phi_x = id\}, \quad E = \{(x, y) \in \mathbb{N}^2 : \phi_x = \phi_y\}$$

e dei loro complementari $\mathbb{N}^2 - K'$, $\mathbb{N} - T$, $\mathbb{N} - I$, $\mathbb{N}^2 - E$.

4. Si provi che $\emptyset$, $\mathbb{N}$ sono decidibili, e che unioni e intersezioni di insiemi decidibili restano decidibili. Si discutano gli stessi problemi per la semidecidibilità.
5. Sia $\mathcal{U}$ la MdT universale. Allora l'insieme $\{x \in \mathbb{N} : \mathcal{U} \downarrow x\}$ è decidibile o no? Semidecidibile o no?
6. Si dimostri che se t è una funzione calcolabile totale allora esiste un numero infinito di indici x tali che $\phi_x = \phi_{t(x)}$.
7. Si dimostri che esiste un naturale x tale che per ogni naturale y vale $\phi_x(y) = \phi_y(x)$.
8. L'insieme dei naturali x tali che 5 è nella immagine di ϕ_x è o no decidibile?
9. Si dimostri che, se I è un insieme finito o I ha complementare $\mathbb{N} - I$ finito, allora I è un insieme decidibile.
10. È possibile applicare il teorema di Rice all'insieme $\{x \in \mathbb{N} \mid \phi_x \text{ converge in } x \text{ passi su } x\}$? E a $K = \{x \in \mathbb{N} \mid \phi_x(x) \downarrow\}$?

Riferimenti bibliografici [66] presenta molti esempi di problemi risolvibili e problemi non risolvibili algebricamente. Altri interessanti problemi non algebricamente risolvibili sono stati discussi in [27] e più recentemente in [93]. Per una descrizione dettagliata della macchina di Turing Universale, presentata inizialmente sempre in [108], si può fare riferimento a [7]. Per un risultato analogo in un contesto più vicino alla programmazione classica si faccia riferimento al libro [66].

Maggiori notizie sulla Σ di Rado si trovano in [39]. Si veda anche l'articolo di Brady in [52]. Il decimo problema di Hilbert è stato proposto originariamente in [53]. La prova della sua indecidibilità può essere trovata in [77] e in versione più elementare in [28]. Per una completa trattazione del problema si può anche consultare [93]. I suoi più recenti sviluppi sono discussi in [30].

4.1 Calcolabilità secondo Church

Il nostro proposito in questo capitolo è quello di presentare un approccio alla computabilità

- contemporaneo a quello di Turing,
- formalmente diverso da quello di Turing, anzi più astratto e matematico,
- tuttavia completamente equivalente a quello di Turing.

Si basa sulla nozione di *funzione ricorsiva*. Ma prima di introdurre questo nuovo concetto, inquadrriamo nuovamente e brevemente il nostro obiettivo generale: per un fissato alfabeto finito A ed in riferimento all'insieme A^* di tutte le stringhe di simboli di A , intendiamo

1. riconoscere quei linguaggi su A che ammettono un algoritmo di decisione,
2. riconoscere quelle funzioni da A^* (o da sottoinsiemi delle sue potenze cartesiane) ad A^* che ammettono un algoritmo di calcolo.

L'analisi di Turing, sviluppata negli scorsi capitoli, ci ha allora rispettivamente condotto alle corrispondenti nozioni di:

1. linguaggio *decidibile*,
2. funzione *calcolabile*.

Come detto, vogliamo adesso esplorare un approccio alternativo. Ricordiamo che non è restrittivo per i nostri propositi supporre di lavorare con numeri naturali, dunque riferirci ad un qualche alfabeto per i numeri naturali (ad esempio $A = \{0, 1, 2, \dots, 9\}$, ma anche $A = \{0, 1\}$ se preferiamo la rappresentazione dei naturali in base 2) ed assumere conseguentemente, senza perdita di generalità, che A^* coincida con l'insieme $\mathbb{N}$ stesso. Opportune codifiche permettono di ridurre qualunque A a questa situazione. Consideriamo allora

1. insiemi L di naturali, o di k -uple di naturali,
2. funzioni (eventualmente parziali) da $\mathbb{N}^k$ a $\mathbb{N}$

per qualche intero positivo k (ricordiamo che una funzione *parziale* ha dominio contenuto in $\mathbb{N}^k$ ma non necessariamente coincidente con tutto $\mathbb{N}^k$; in quest'ultimo caso la funzione si dice *totale*; ricordiamo anche che una funzione -parziale o totale- da $\mathbb{N}^k$ a $\mathbb{N}$ si dice k -aria). In questo ambito vogliamo esaminare nuove possibili strategie per definire la decidibilità degli insiemi e la calcolabilità delle funzioni. D'altra parte possiamo ricordare che, per $L \subseteq \mathbb{N}^k$, resta definita la *funzione caratteristica* f_L di L , quella funzione (totale) da $\mathbb{N}^k$ a $\mathbb{N}$ che ad ogni $x \in \mathbb{N}^k$ associa

- 1 se $x \in L$,
- 0 altrimenti.

Possiamo evidentemente affermare che

esiste un algoritmo per decidere, per ogni $x \in \mathbb{N}^k$, se $x \in L$ o no

se e solo se

esiste un algoritmo per calcolare $f_L(x)$ per ogni $x \in \mathbb{N}^k$.

Basterà allora per i nostri propositi definire con precisione per quali insiemi $L \subseteq \mathbb{N}^k$

esiste un algoritmo per calcolare $f_L(x)$ per ogni $x \in \mathbb{N}^k$.

In altre parole, possiamo concentrare il nostro lavoro a definire in modo appropriato la calcolabilità delle funzioni (anche parziali): infatti tramite f_L possiamo coinvolgere in questo ambito anche il problema di decidere L . Il nostro obiettivo è dunque quello di precisare per quali funzioni parziali k -arie f

esiste un algoritmo per calcolare $f(x)$ per ogni x nel dominio di f .

Turing ci ha suggerito una possibile strategia, tramite le macchine di Turing. Come detto, iniziamo qui a considerarne una alternativa.

4.2 Le funzioni parziali ricorsive

Consideriamo anzitutto le seguenti funzioni parziali:

- la funzione *successore* s da $\mathbb{N}$ a $\mathbb{N}$, quella che ad ogni naturale x associa $x + 1$;
- la funzione *costante zero* Z da $\mathbb{N}$ a $\mathbb{N}$, quella che ad ogni naturale x associa 0;
- finalmente, per ogni intero positivo k e per ogni $i = 1, \dots, k$, la funzione *proiezione* P_i^k da $\mathbb{N}^k$ a $\mathbb{N}$, quella che ad ogni k -upla $\vec{x} = (x_1, \dots, x_k)$ di naturali associa la sua componente i -ma x_i .

Si noti che il terzo gruppo di funzioni include anche l'identità di $\mathbb{N}$ come P_1^1 . Possiamo senz'altro convenire che tutte queste funzioni ammettono un algoritmo di calcolo: dunque, quale che sia la definizione di calcolabilità che ci proponiamo di concordare, essa non potrà escluderle. Chiamiamo per semplicità *iniziali* le funzioni sin qui considerate, e cioè successore, zero e proiezioni.

Adesso osserviamo che la funzione costante che ad ogni naturale associa 1 ammette ovviamente un algoritmo di calcolo e, del resto, si ottiene come *composizione* delle funzioni zero e successore. Le stesse considerazioni si possono fare a proposito della funzione s^2 (quella che ad ogni naturale x associa $x + 2$, e che è la composizione di s per se stessa), o anche di s^3 , e così via. In generale, possiamo ragionevolmente convenire che la composizione di funzioni che hanno un algoritmo di calcolo ammetta a sua volta un algoritmo di calcolo. Dunque la nozione rigorosa di calcolabilità che cerchiamo di ottenere, quale che essa sia, dovrà preservare la composizione di funzioni. Per fissare opportunamente il contesto ed adattarlo anche al caso di funzioni parziali e di k -uple di naturali, poniamo la seguente:

Definizione. Per k e n interi positivi, siano h una funzione parziale n -aria e $g_1, \dots, g_n$ n funzioni parziali k -arie. Una funzione parziale k -aria f si dice definita per *composizione* da h e $g_1, \dots, g_n$ se e solo se, per ogni $\vec{x} \in \mathbb{N}^k$,

- $\vec{x}$ è nel dominio di f se e solo se $\vec{x}$ è nel dominio di g_i per ogni $i = 1, \dots, n$ e la n -upla $(g_1(\vec{x}), \dots, g_n(\vec{x}))$ è nel dominio di h ;
- in tal caso, $f(\vec{x}) = h(g_1(\vec{x}), \dots, g_n(\vec{x}))$.

Come già osservato, è facilmente accettabile che, se $h, g_1, \dots, g_n$ hanno i loro algoritmi di calcolo, anche f lo ottiene: dato $\vec{x} \in \mathbb{N}^k$, si computano, se possibile, $g_1(\vec{x}), \dots, g_n(\vec{x})$ con i relativi algoritmi, poi la loro immagine in h (cioè $f(\vec{x})$) mediante l'algoritmo di h . Notiamo poi che, se $h, g_1, \dots, g_n$ sono funzioni totali (ovvero sempre definite), anche f lo è.

Un altro metodo familiare per definire una funzione f da $\mathbb{N}$ a $\mathbb{N}$ è quello di ricorrere al *Principio di Induzione*, ovvero a quella fondamentale proposizione sui naturali che ci dice che, se una proprietà è soddisfatta da 0 e si preserva da y a $y + 1$ per ogni naturale y , allora quella proprietà vale per tutto $\mathbb{N}$. Applicandola a f , possiamo dedurre che, una volta specificati

$$f(0) = g_0 \in \mathbb{N}$$

e poi, per ogni naturale y ,

$$f(y + 1) = h(y, f(y))$$

per qualche h 2-aria indipendente da y -dunque $f(y + 1)$ come funzione di y e $f(y)$ tramite h -, allora f risulta definita su tutto $\mathbb{N}$. Possiamo ragionevolmente convenire che, se g_0 è dato esplicitamente e h ammette un suo algoritmo di calcolo, allora anche f ottiene un procedimento effettivo che la computa per ogni naturale y : basta considerare anzitutto $f(0) = g_0$, poi calcolare $f(1) = h(0, f(0))$ da $f(0)$ tramite l'algoritmo di h , e così via finché non si arriva a $f(y)$. Comunque, per fissare opportunamente il discorso nell'ambito più generale (per funzioni parziali e k -uple di elementi), conviene nuovamente che poniamo una

Definizione. Per k intero non negativo, siano g una funzione parziale k -aria e h una funzione parziale $(k + 2)$ -arie. Una funzione parziale $(k + 1)$ -aria f si dice definita per *recursione* da g e h se e solo se, per ogni $\vec{x} \in \mathbb{N}^k$,

- $(\vec{x}, 0)$ è nel dominio di f se e solo se $\vec{x}$ è nel dominio di g e, in tal caso, $f(\vec{x}, 0) = g(\vec{x})$,
- per ogni naturale y , $(\vec{x}, y + 1)$ è nel dominio di f se e solo se $(\vec{x}, y)$ è nel dominio di f e $(\vec{x}, y, f(\vec{x}, y))$ è nel dominio di h e, in tal caso, $f(\vec{x}, y + 1) = h(\vec{x}, y, f(\vec{x}, y))$.

Con un minimo di pazienza, si potrà riconoscere in questo ambito generale anche la situazione particolare prima descritta, quella relativa a una funzione f 1-aria: infatti, per $k = 0$, anzitutto la sequenza $\vec{x}$ è vuota, possiamo poi intendere che g si riduca ad una costante g_0 , che h sia 2-aria e che la funzione f che g e h determinano per recursione sia 1-aria:

$$f(0) = g_0, \quad f(y + 1) = h(y, f(y)) \quad \forall y \in \mathbb{N},$$

appunto. Possiamo poi convenire senza problemi che, anche nel caso della definizione generale, se g e h ammettono un qualche algoritmo di calcolo, anche f lo ottiene (arrangiando opportunamente quelli di g e h). Notiamo finalmente che, se g e h sono totali (cioè definite ovunque), anche f lo è.

Un'altra proprietà che contraddistingue i naturali e si presta per definirne le funzioni è il *Principio del Minimo*, secondo cui ogni insieme non vuoto di naturali ammette un minimo elemento. Corrispondentemente consideriamo la seguente definizione.

Definizione. Per k intero positivo, sia g una funzione **totale** $(k + 1)$ -aria. Una funzione parziale k -aria f si dice definita per *minimalizzazione* da g se e solo se, per ogni $\vec{x} \in \mathbb{N}^k$,

- $\vec{x}$ è nel dominio di f se e solo se c'è qualche naturale y per cui $g(\vec{x}, y) = 0$,
- in tal caso, $f(\vec{x})$ è il minimo naturale con questa proprietà, cioè

$$f(\vec{x}) = \min\{y \in \mathbb{N} : g(\vec{x}, y) = 0\}$$

(che taluni preferiscono indicare anche con la notazione $\mu y(g(\vec{x}, y) = 0)$).

Osserviamo che, se g ha un suo algoritmo di calcolo, anche f lo ottiene, ad esempio nel modo seguente. Sia dato $\vec{x} \in \mathbb{N}^k$;

- si calcola $g(\vec{x}, 0)$ tramite l'algoritmo di g ; siccome g è totale (cioè sempre definita), la computazione ha termine e ci è consentito controllare se $g(\vec{x}, 0) = 0$ o no; se sì, si pone $f(\vec{x}) = 0$;
- altrimenti si passa a calcolare $g(\vec{x}, 1)$ sempre con l'algoritmo di g (e grazie alla totalità di g), si controlla poi se $g(\vec{x}, 1) = 0$ o no; se sì, si pone $f(\vec{x}) = 1$;
- se no, si prosegue, considerando $g(\vec{x}, 2)$, e così via.

Può ovviamente capitare che $g(\vec{x}, y)$ non si annulli per nessun y ; ma questo significa semplicemente che $\vec{x}$ non appartiene al dominio della f .

Le precedenti considerazioni ci mostrano l'importanza di assumere che g sia totale, ma evidenziano anche che la funzione f può benissimo non essere altrettanto totale. Ad esempio, se $g(\vec{x}, y) = 1$ per ogni y , si ha che $\vec{x}$ non è nel dominio di f . In conclusione, possiamo convenire che la nostra nozione di computabilità, quale che essa sia, dovrà preservarsi, oltre che per composizione e per recursione, anche per minimalizzazione.

Dunque la classe di funzioni “calcolabili” che vogliamo formare dovrà includere le funzioni iniziali e restare chiusa per composizione, recursione e minimalizzazione. Poniamo a questo punto la seguente:

Definizione. Una funzione parziale f si dice *parziale ricorsiva* se e solo se esiste una sequenza ordinata finita $f_0, \dots, f_n$ di funzioni parziali delle quali $f = f_n$ è l'ultima e tali che, per ogni $i \leq n$, f_i è iniziale oppure si ottiene da funzioni precedenti per composizione, o per recursione, o per minimalizzazione. Definiamo poi *ricorsiva* una funzione parziale ricorsiva che sia anche totale. Diciamo, finalmente, *ricorsivo* un insieme $S \subseteq \mathbb{N}^k$ la cui funzione caratteristica f_S (totale!) sia ricorsiva.

Una proposta che possiamo riferire ad Alonzo Church e al 1936 (lo stesso anno di Turing) e chiamare almeno provvisoriamente *Tesi di Church* afferma (a livello di slogan)

$$\text{calcolabile} = \text{ricorsivo}.$$

Per la precisione afferma:

Tesi di Church (1936). Una funzione parziale f da $\mathbb{N}^k$ a $\mathbb{N}$ ammette un algoritmo di calcolo se e solo se f è parziale ricorsiva. In particolare, per f totale, f ammette un algoritmo di calcolo se e solo se f è ricorsiva. Finalmente, un sottoinsieme L di $\mathbb{N}^k$ ammette un algoritmo di decisione se e solo se L è ricorsivo.

Come già la Tesi a proposito delle macchine di Turing, anche questa Tesi di Church non si propone come un teorema da dimostrare, ma piuttosto come la affermazione (che si può condividere o rifiutare) che le nozioni di funzione e insieme ricorsivi costituiscono il modo rigoroso e preciso di introdurre il concetto intuitivo di algoritmo. Naturalmente, c'è da chiedersi quanto credito possiamo dare a questa ipotesi. Non ci sono dubbi ad accettare che tutto quanto è ricorsivo ammette un algoritmo di calcolo o di decisione: la definizione stessa di ricorsività si preoccupa di garantire questa situazione. Semmai c'è da chiedersi se è vero anche il contrario, se cioè tutte le funzioni che hanno un algoritmo di calcolo e tutti gli insiemi che hanno un algoritmo di decisione corrispondono alla condizione di ricorsività. Il prossimo paragrafo è dedicato a fornire qualche semplice esempio al riguardo.

Per completare questo paragrafo, osserviamo nuovamente come l'approccio alla calcolabilità tramite la ricorsività, anche se definito nel caso specifico dei numeri naturali, si estende facilmente a funzioni e linguaggi su ogni alfabeto finito, grazie alle usuali procedure di codifica.

4.3 Esempi a volontà

In realtà neppure oggi sono noti casi di funzioni che ammettano un algoritmo di calcolo e non si dimostrino contemporaneamente (parziali) ricorsive. Lo stesso vale per gli insiemi. Del resto, se e quando si mostrerà che la Tesi di Church non è adeguata al progresso dei calcolatori e delle procedure di calcolo, non sarà il caso di farne un dramma: si prenderà atto che il modello della ricorsività è superato e si provvederà a cercarne un altro più aggiornato.

Vediamo comunque adesso alcuni semplici esempi di funzioni e insiemi che hanno manifestamente un algoritmo di calcolo o di decisione e che si dimostrano ricorsivi.

Esempi.

1. Mostriamo che le quattro operazioni elementari dei numeri naturali (addizione, moltiplicazione, sottrazione e divisione) corrispondono alla definizione di funzione ricorsiva. Cominciamo con la addizione. La funzione totale binaria $+$ che associa ad ogni coppia ordinata (x, y) di naturali la loro somma $x + y$ è ricorsiva, infatti $+$ si può ottenere per recursione e composizione da funzioni iniziali nel modo che segue: per ogni scelta di $x, y \in \mathbb{N}$,

$$x + 0 = P_1^1(x),$$

$$x + (y + 1) = s(x + y) = s(P_3^3(x, y, x + y)).$$

2. Anche la moltiplicazione $\cdot$ è ricorsiva perchè si ottiene per recursione e composizione dalle funzioni iniziali e dalla addizione, che già sappiamo essere ricorsiva: infatti, per ogni scelta di $x, y \in \mathbb{N}$,

$$x \cdot 0 = 0 = Z(x),$$

$$x \cdot (y + 1) = x \cdot y + x = P_3^3(x, y, x \cdot y) + P_1^3(x, y, x \cdot y).$$

3. Anche la funzione 2-aria di esponenziazione exp , quella che ad ogni coppia ordinata di naturali (x, y) associa x^y è ricorsiva. In realtà si può discutere se exp sia totale oppure parziale, visto che in genere 0^0 non ha significato. Possiamo comunque convenire in questo caso $exp(0, 0) = 1$. Allora non è difficile controllare in dettaglio che exp si ottiene per recursione e minimalizzazione da funzioni iniziali e da $\cdot$ nel modo che segue: per ogni scelta di $x, y \in \mathbb{N}$,

$$exp(x, 0) = 1 = s(Z(x)),$$

$$exp(x, y + 1) = P_3^3(x, y, exp(x, y)) \cdot P_1^3(x, y, exp(x, y)).$$

4. Il meccanismo della recursione aiuta anche a dimostrare la ricorsività di alcuni semplici esempi di funzioni totali 1-arie, che ci saranno utili nel seguito. Per cominciare, consideriamo la funzione pd (*predecessore*) che ad ogni naturale $y > 0$ associa, appunto, il numero che lo precede, e cioè $y - 1$; conveniamo

poi per semplicità $pd(0) = 0$. Allora pd è ricorsiva, in quando è definita come segue, in accordo col meccanismo di recursione:

$$pd(0) = 0,$$

$$pd(y + 1) = y = P_1^2(y, pd(y)) \quad \forall y \in \mathbb{N}.$$

Allo stesso modo si verifica che la funzione *segno* sg , quella che ad ogni naturale y associa 1 se $y \neq 0$ e 0 se $y = 0$, è ricorsiva: infatti

$$sg(0) = 0,$$

$$sg(y + 1) = 1 = s(Z(P_1^2(y, pd(y)))) \quad \forall y \in \mathbb{N}.$$

Il lettore potrà controllare per esercizio che anche la funzione che viene chiamata *controsegno* ed indicata $\overline{sg}$, quella che ad ogni naturale y associa 0 se $y \neq 0$ e 1 se $y = 0$, è ricorsiva.

5. Passiamo adesso alla funzione $-$ di sottrazione. Essa è manifestamente non totale tra i naturali: ad esempio, non possiamo calcolare in $\mathbb{N}$ $2 - 3$, o in generale $x - y$ quando $x < y$. Per rimediare, possiamo procedere come negli esempi di exp e pd e cercare di convenire valori fittizi per la differenza di x e y quando $x < y$. Si considera allora, ad esempio, la funzione $\ominus$, chiamata *differenza aritmetica* e definita come segue: per ogni scelta di x e y naturali, $x \ominus y$ è $x - y$ se $x - y$ si può fare, cioè se $x \geq y$, 0 altrimenti. La funzione così introdotta è ricorsiva. Per dimostrarlo, ricorriamo all'aiuto della funzione predecessore pd e osserviamo che, per ogni scelta di $x, y \in \mathbb{N}$,

$$x \ominus 0 = x = P_1^1(x),$$

$$x \ominus (y + 1) = pd(P_3^3(x, y, x \ominus y)).$$

Quindi $\ominus$ si ottiene per recursione e composizione da funzioni ricorsive (inclusa pd) e di conseguenza è ricorsiva. Un altro possibile surrogato totale della sottrazione è la così detta *differenza assoluta*, la funzione che ad ogni coppia di naturali x e y associa il valore assoluto della differenza $|x - y|$. Anche questa funzione è ricorsiva perchè è facilmente controllato che, per ogni scelta di x e y ,

$$|x - y| = (x \ominus y) + (y \ominus x).$$

6. Consideriamo finalmente la funzione *sottrazione* $-$, e non più i suoi surrogati. Abbiamo già osservato che, per $x_1, x_2 \in \mathbb{N}$, l'esistenza della differenza $x_1 - x_2$ dipende dal fatto che $x_1 \geq x_2$; d'altra parte, $x_1 \geq x_2$ se e solo se esiste un y tale che $x_2 + y = x_1$, condizione che possiamo anche scrivere, ricorrendo alla differenza assoluta, come $|(x_2 + y) - x_1| = 0$. In questo caso, $x_1 - x_2$ è proprio il minimo (in realtà l'unico) y con questa proprietà. Grazie al meccanismo di minimalizzazione, possiamo allora concludere che anche $-$ è parziale ricorsiva.

7. Lo stesso vale per le funzioni q e r che ad ogni coppia di naturali x_1 e x_2 con $x_2 \neq 0$ associano, rispettivamente, il *quoziente* e il *resto* della divisione di x_1 per x_2 . Come già sottolineato, queste funzioni sono definite se e solo se $x_2 \neq 0$; una maniera un po' involuta di esprimere questa condizione è quella di affermare che qualche multiplo di x_2 prima o poi supera x_1 : infatti, per $x_2 = 0$, l'unico numero divisibile per x_2 è 0 stesso, e non può oltrepassare x_1 ; ma altrimenti la sequenza strettamente crescente $0 \cdot x_2, 1 \cdot x_2, 2 \cdot x_2, \dots$ finisce col diventare maggiore di x_1 . Quindi possiamo affermare che q e r sono definiti su (x_1, x_2) se e solo se esiste y tale che $(y+1) \cdot x_2 > x_1$; quest'ultima condizione si può anche esprimere dicendo che la differenza aritmetica $(y+1) \cdot x_2 \ominus x_1$ è maggiore di 0, ovvero che il suo controsegno è 0

$$\overline{sg}((y+1) \cdot x_2 \ominus x_1) = 0.$$

In questo caso, il quoziente di x_1 per x_2 è proprio il minimo y per cui $(y+1) \cdot x_2 > x_1$ (in altre parole, il massimo per cui $y \cdot x_2 \leq x_1$)

$$q(x_1, x_2) = \min \{y \in \mathbb{N} : \overline{sg}((y+1) \cdot x_2 \ominus x_1) = 0\};$$

possiamo allora concludere per minimalizzazione che q è parziale ricorsiva. Lo stesso vale per il resto r , visto che si ha, per ogni scelta di x_1 e x_2 ,

$$r(x_1, x_2) = x_1 - q(x_1, x_2).$$

8. Abbiamo visto nei punti precedenti (con qualche fatica) che tutte le operazioni numeriche elementari sono (parziali) ricorsive. Proponiamo adesso alcuni esempi di *insiemi* ricorsivi. Tale è anzitutto l'insieme dei *numeri pari*, visto che la sua funzione caratteristica, quella che associa ad ogni naturale x il valore 1 se x è pari e il valore 0 se x è dispari, si può esprimere

$$\overline{sg}(r(x, 2)) = \overline{sg}(r(x, s^2(Z(x))))$$

ed è quindi ricorsiva. Anche la relazione binaria di ordine $\leq$ (intesa come insieme di quelle coppie ordinate (x_1, x_2) di naturali per cui, appunto, $x_1 \leq x_2$) è ricorsiva. Infatti, per ogni scelta di x_1 e x_2 , vale

$$x_1 \leq x_2 \quad \Leftrightarrow \quad x_1 \ominus x_2 = 0 \quad \Leftrightarrow \quad \overline{sg}(x_1 \ominus x_2) = 1,$$

dunque la funzione caratteristica di $\leq$ si può esprimere in modo ricorsivo (come controsegno della differenza aritmetica), e questo assicura che $\leq$ è ricorsivo.

Il lettore può controllare per esercizio che, per ogni intero positivo k ,

- l'insieme vuoto e $\mathbb{N}^*$ sono ricorsivi (*suggerimento*: collegare le relative funzioni caratteristiche alla funzione iniziale Z);

- se $X, Y \subseteq \mathbb{N}^k$ sono ricorsivi, anche la loro intersezione $X \cap Y$, la loro unione $X \cup Y$ e il complemento $\mathbb{N}^k - X$ sono ricorsivi (*suggerimento*: si noti $f_{X \cap Y} = f_X \cdot f_Y$, $f_{X \cup Y} = sg(f_X + f_Y)$, $f_{\mathbb{N}^k - X} = \overline{sg} f_X$);
- come applicazione del punto precedente, le relazioni $\geq$, $>$ e $<$ e l'insieme dei numeri dispari sono ricorsivi;
- anche la relazione di divisibilità tra naturali è ricorsiva.

Notiamo adesso che, tra le funzioni degli esempi precedenti, quelle parziali (sottrazione, quoziente, resto) derivano da un impiego del procedimento di minimizzazione. Del resto, possiamo ricordare che le funzioni iniziali sono totali e che i meccanismi di composizione e recursione preservano la proprietà di essere totali e non possono generare funzioni parziali a partire da quelle iniziali. Diciamo allora che una funzione totale f è *ricorsiva primitiva* se e solo se esiste una sequenza ordinata finita $f_0, \dots, f_n$ di funzioni *totali* delle quali $f = f_n$ è l'ultima e tali che, per ogni $i \leq n$, f_i è iniziale oppure si ottiene da funzioni precedenti per composizione, o per recursione (senza quindi coinvolgere la minimizzazione). È ragionevole chiedersi se, tra le funzioni totali, quelle ricorsive primitive esauriscono l'intera classe delle funzioni ricorsive (in altre parole, se la minimizzazione è superflua per definire le funzioni totali ricorsive). La risposta è negativa. Si consideri infatti il seguente esempio di funzione totale binaria A (chiamata *funzione di Ackermann* dal nome di chi la ideò): per ogni scelta di x_1 e x_2 naturali,

- (a) $A(0, x_2) = x_2 + 1$,
- (b) $A(x_1 + 1, 0) = A(x_1, 1)$,
- (c) $A(x_1 + 1, x_2 + 1) = A(x_1, A(x_1 + 1, x_2))$.

Si può verificare che le tre condizioni (a), (b) e (c) definiscono effettivamente una funzione totale A ed anzi le forniscono un algoritmo esplicito di calcolo. Ad esempio $A(1, 2) = 4$ perchè (c) implica

$$\begin{aligned} A(1, 2) &= A(0, A(1, 1)), \\ A(1, 1) &= A(0, A(1, 0)) \end{aligned}$$

e da (a) e (b) segue

$$A(1, 0) = A(0, 1) = 2;$$

allora $A(1, 1) = A(0, 2) = 3$ e $A(1, 2) = A(0, 3) = 4$. In effetti A (come tutte le funzioni per cui è noto un algoritmo di calcolo) è ricorsiva. D'altra parte, si può provare che A non è ricorsiva primitiva: in altre parole, per dimostrarne la ricorsività, si ha bisogno di utilizzare il meccanismo di minimizzazione: composizione e recursione non sono sufficienti a raggiungerla partendo dalle funzioni iniziali.

4.4 Church o Turing?

L'approccio alla calcolabilità tramite le funzioni ricorsive pare chiaramente diverso da quello mediante le macchine di Turing. Il primo è assai più astratto,

l'altro più concreto, ed anzi dichiaratamente ispirato dal tentativo di simulare il comportamento della mente umana. In questo ambito, anzi, il secondo approccio risulta più convincente, se è vero, come è vero, che scienziati importanti come Gödel, assai dubbiosi a proposito della Tesi di Church, si dichiaravano invece assolutamente persuasi dalla calcolabilità secondo Turing, proprio per questa sua maggiore concretezza. Possiamo allora chiederci a chi credere, se a Church oppure a Turing, dunque quale modello di computabilità preferire.

In realtà la scelta è tutto men che drammatica, perchè è possibile dimostrare che i due approcci, pur così differenti in natura, sono tuttavia tra loro equivalenti e dunque ugualmente accettabili (o rifiutabili). Conseguentemente le due tesi, quella sulle macchine di Turing e quella di Church, sono coincidenti, tanto che le abbiamo già implicitamente congiunte nel Capitolo 2 quando abbiamo parlato tranquillamente di *Tesi di Church-Turing*.

Non va poi trascurato che proprio questa equivalenza, e dunque la coincidenza concentrica di punti di vista tanto diversi verso lo stesso obiettivo, è argomento che corrobora e sostiene ulteriormente i due approcci alla calcolabilità, tanto quello di Turing che quello di Church. Ma passiamo alla enunciazione e alla dimostrazione del teorema di equivalenza.

Teorema 4.4.1 *Sia f una funzione parziale k -aria (dai naturali ai naturali). Allora f è calcolabile (nel senso di Turing) se e solo se f è parziale ricorsiva.*

In particolare, quando f è totale, si può dedurre che f è calcolabile (secondo Turing) se e solo se è ricorsiva. Si può poi dedurre facilmente:

Corollario 4.4.2 *Sia $L \subseteq \mathbb{N}^k$. Allora L è decidibile (secondo Turing) se e solo se L è ricorsivo.*

Basta infatti osservare che un insieme L di k -uple di naturali è decidibile (secondo Turing) se e solo se la sua funzione caratteristica è calcolabile (secondo Turing). Passiamo ora alla dimostrazione del teorema: ne daremo solo i cenni principali, evitando di disperderci in troppi dettagli.

Dimostrazione. Assumiamo dapprima f parziale ricorsiva e verifichiamo che f è calcolabile secondo Turing. Ricordiamo che le funzioni ricorsive sono quelle che si ottengono dalle funzioni iniziali applicando un numero finito di volte i procedimenti di composizione, recursione e minimalizzazione. Ci basta allora dimostrare quanto segue:

- (i) le funzioni iniziali sono calcolabili con MdT,
- (ii) funzioni che si ottengono per composizione, oppure per recursione, oppure per minimalizzazione da funzioni che sono calcolabili con MdT sono a loro volta calcolabili con MdT.

Le relative verifiche sono più noiose che difficili. Del resto, (i) è già stato trattato nel Capitolo 2 almeno a livello di esercizi e quando abbiamo introdotto composizione, recursione e minimalizzazione, abbiamo anche accennato ad algoritmi

capaci di calcolare le funzioni corrispondentemente ottenute. Tutti questi procedimenti si possono adattare con un minimo di pazienza al contesto e al linguaggio delle MdT.

Passiamo allora a controllare l'implicazione inversa: abbiamo una funzione parziale k -aria f che è calcolabile secondo Turing e vogliamo mostrare che f è parziale ricorsiva. Sia M una MdT che computa f . Siano poi $q_0, \dots, q_n$ gli stati di M . Possiamo assumere che M si arresti se e solo se M entra nello stato q_n : se questo non è il caso, ci basta aggiungere a M un nuovo stato (senza istruzioni che lo riguardino) e alla funzione di transizione di M le istruzioni che conducano ogni configurazione di arresto di M al nuovo stato. Definiamo a questo punto quattro funzioni totali $(k + 1)$ -arie

$$q_M, i_M, s_M, d_M$$

tali che, per ogni scelta di $\vec{x} \in \mathbb{N}^k$ e $t \in \mathbb{N}$,

$$q_M(\vec{x}, t), i_M(\vec{x}, t), s_M(\vec{x}, t), d_M(\vec{x}, t)$$

codificano rispettivamente

- lo stato di M ,
- che cosa è scritto sulla cella esaminata da M ,
- che cosa è scritto sul nastro a sinistra della cella esaminata da M ,
- che cosa è scritto sul nastro a destra della cella esaminata da M

al passo t della computazione di M sull'input corrispondente a $\vec{x}$. Ad esempio, $q_M(\vec{x}, t)$ è rappresentato dall'indice 0, o 1, ..., o n dello stato corrispondente; $i_M(\vec{x}, t)$ è una cifra di un qualche alfabeto dei naturali (ad esempio, 0, o 1, ..., o 9 se adoperiamo la rappresentazione decimale) oppure un ulteriore numero (10, se vogliamo) per denotare il carattere bianco $\star$; s_M e d_M richiedono invece una definizione più accurata, che utilizza in modo opportuno le consuete tecniche di codifica ed in particolare il teorema di decomposizione in fattori primi: ne omettiamo qui i dettagli. Osserviamo poi che, se per un certo input $\vec{x}$ M converge in T passi, allora per ogni naturale $t > T$ si conviene

$$q_M(\vec{x}, t) = q_M(\vec{x}, T), i_M(\vec{x}, t) = i_M(\vec{x}, T),$$

$$d_M(\vec{x}, t) = d_M(\vec{x}, T), s_M(\vec{x}, t) = s_M(\vec{x}, T),$$

si cristallizza cioè la situazione al momento dell'arresto di M . In questo modo si ottiene che le nostre quattro funzioni diventino totali. A questo punto si prova che q_M, i_M, s_M, d_M sono ricorsive. Ad esempio si osserva che, per ogni $\vec{x} \in \mathbb{N}^k$,

$$q_M(\vec{x}, 0) = 0,$$

perché al passo iniziale M è nello stato q_0 , e

$$i_M(\vec{x}, 0)$$

è la prima cifra della prima componente x_1 di $\vec{x}$ (quella più a sinistra quando inizia la computazione), mentre, per ogni naturale t ,

$$q_M(\vec{x}, t+1), i_M(\vec{x}, t+1)$$

si desumono in modo effettivo dalle istruzioni della funzione di transizione di M sulla coppia

$$(q_M(\vec{x}, t), i_M(\vec{x}, t))$$

ed inoltre da $s_M(\vec{x}, t)$ e $d_M(\vec{x}, t)$. Analogamente per $s_M(\vec{x}, t+1)$, $d_M(\vec{x}, t+1)$. In questo modo, con qualche maggior attenzione per i dettagli, si mostra che le nostre quattro funzioni sono ricorsive.

A questo punto si nota che, per ogni $\vec{x} \in \mathbb{N}^k$,

- $\vec{x}$ è nel dominio di f se e solo se M converge su $\vec{x}$, e quindi se e solo se, per qualche naturale t , M entra nello stato q_n al passo t della computazione su $\vec{x}$, ovvero $q_M(\vec{x}, t) = n$;
- in tal caso, $f(\vec{x})$ si ottiene da $i_M(\vec{x}, t)$, $s_M(\vec{x}, t)$ e $d_M(\vec{x}, t)$ dove t è il minimo naturale per cui $q_M(\vec{x}, t) = n$.

Si intravede in questa definizione il meccanismo di minimalizzazione applicato a funzioni ricorsive (quali q_M , i_M , s_M e d_M). Se ne deduce che f è, a sua volta, parziale ricorsiva, proprio come volevamo dimostrare. $\square$

Il teorema ci aiuta a caratterizzare in termini di funzioni ricorsive anche la semidecidibilità. Infatti con riferimento al teorema 3.5.1 possiamo considerare linguaggi $L \subseteq \mathbb{N}$ tali che $L = \emptyset$ oppure L è immagine di qualche funzione totale ricorsiva 1-aria f . Chiamiamo *ricorsivamente enumerabile* un tale linguaggio L e deduciamo banalmente:

Corollario 4.4.3 $L \subseteq \mathbb{N}$ è *ricorsivamente enumerabile* se e solo se è *semidecidibile*.

I linguaggi ricorsivamente enumerabili si riconoscono allora anche come i domini delle funzioni parziali ricorsive 1-arie (sempre per il teorema 3.5.1).

4.5 Esercizi

1. Ricorrendo alla definizione di insieme ricorsivo, si provi che $\emptyset$, $\mathbb{N}$ sono ricorsivi e che gli insiemi ricorsivi si preservano per unioni, intersezioni, complementari. Che cosa si può dire a questo proposito degli insiemi ricorsivamente enumerabili?
2. Ancora ricorrendo alla definizione di insieme ricorsivo, si mostri che l'insieme dei numeri primi è ricorsivo.
3. Si dia un esempio di insieme I ricorsivamente enumerabile e non ricorsivo che contiene un insieme J ricorsivo e infinito.

4. Si dimostri che un insieme infinito $I \subseteq \mathbb{N}$ è ricorsivo se e soltanto se I è l'immagine di una funzione totale ricorsiva 1-aria *crescente*.

Riferimenti bibliografici Un'ampia trattazione della teoria delle funzioni ricorsive si trova nell'articolo [39], oppure nei libri [84, 87]. Per approfondimenti sulle funzioni ricorsive e sulle relazioni con la teoria della calcolabilità si può far riferimento anche a [67] e [91].

I legami tra insiemi ricorsivi e insiemi ricorsivamente enumerabili (cioè decidibili e semidecidibili) sono discussi in [89] e [103]. [66] presenta tutta la teoria arricchendola di esempi e collegandola ai teoremi di incompletezza di Gödel in modo accessibile (con un'introduzione molto intuitiva alla Teoria degli insiemi).

Calcolabilità e Grammatiche

5.1 Grammatiche e Automi

Lo studio formale dei linguaggi è una parte fondamentale dell'informatica teorica e trova applicazione in varie direzioni. Tra di esse va certamente compresa l'analisi delle proprietà sintattiche dei programmi, e dunque lo studio dei metodi di definizione della sintassi di un linguaggio di programmazione. Un'altra direzione riguarda i metodi di traduzione da un linguaggio a un altro (tipicamente da un linguaggio di programmazione ad alto livello ad uno di più basso livello come il linguaggio macchina). Non da meno sono le tecniche di verifica formale di proprietà comportamentali dei programmi. La familiarità con i concetti di generazione e riconoscimento dei linguaggi è anche importante nell'elaborazione di dati: si pensi, infatti, alla sequenza di operazioni eseguite da un programma, alla sequenza di invocazioni di metodi in programmazione orientata agli oggetti, alle interazioni con l'ambiente esterno di un sistema, alla descrizione di protocolli di comunicazione, e così via.

Ma quel che ci preme maggiormente sottolineare in queste pagine è il collegamento tra studio dei linguaggi e computabilità. Del resto abbiamo già accennato al fatto che problemi di varia natura (come il calcolo di funzioni, o questioni di ottimizzazione) si riconducano spesso alla situazione standard di decidere l'appartenenza di una stringa a un determinato linguaggio: ad esempio, la computazione di una funzione f da $\mathbb{N}$ a $\mathbb{N}$ si riduce alla decisione del grafico $\{(n, f(n)) : n \in \mathbb{N}\}$ di f e dunque, tramite opportune codifiche, a quello di un linguaggio quale $\{a^n b^{f(n)} \mid n \in \mathbb{N}\}$ sull'alfabeto formato da due numeri primi $a < b$.

I metodi che permettono di definire un linguaggio sono diversi. Tra essi possiamo distinguere:

- un approccio di tipo *generativo*, nel quale le stringhe del linguaggio sono costruite secondo regole prefissate all'interno di “*grammatiche*”, così come in genere capita per le frasi della lingua comune;
- un approccio di tipo *riconoscitivo*, che consiste nell'utilizzare macchine astratte, dette *automi riconoscitori* e atte a esaminare le stringhe di un alfabeto e certificarne l'appartenenza al linguaggio considerato.

In questo capitolo concentreremo l'attenzione sulle grammatiche generative, riservando agli automi riconoscitori soltanto qualche breve cenno finale (il lettore interessato a questo secondo argomento potrà comunque trovarne la trattazione in [8, 54, 55]). Definiremo in dettaglio che cosa è una grammatica e come essa generi un linguaggio. Discuteremo poi in quali casi un linguaggio L così costruito sia decidibile, o semidecidibile.

5.2 Linguaggi

Sia A un alfabeto, cioè un insieme finito e non vuoto. Ricordiamo che un sottoinsieme L di A^* (non necessariamente finito) viene chiamato *linguaggio formale* su A o semplicemente *linguaggio* su A . Nel seguito, ometteremo talora l'indicazione dell'alfabeto A , quando chiara dal contesto. Dunque i linguaggi sono particolari insiemi, e si possono applicare a loro proposito le usuali operazioni insiemistiche: per L, L' linguaggi rispettivamente sugli alfabeti A, A' ,

- l'*unione* $L \cup L'$ di L e L' è il linguaggio su $A \cup A'$ composto dalle stringhe che stanno in L o in L' ;
- l'*intersezione* $L \cap L'$ di L e L' è il linguaggio su $A \cap A'$ formato dalle stringhe che stanno tanto in L quanto in L' ;
- la *differenza* $L - L'$ di L e L' è il linguaggio su A composto dalle stringhe di L che non stanno in L' ;
- $L^c = A^* - L$ (il *complementare* di L) è il linguaggio su A formato dalle stringhe di A^* fuori di L .

Tra i linguaggi su un alfabeto A c'è anche l'insieme vuoto $\emptyset$, quello privo di stringhe, da non confondersi con $\{\lambda\}$, cioè con il linguaggio formato dall'unica stringa vuota λ . Ci sono poi altre operazioni tra linguaggi che non corrispondono a nozioni insiemistiche generali ma derivano dal contesto particolare di stringhe e alfabeti.

Definizione. Siano ancora L e L' due linguaggi, rispettivamente sugli alfabeti A e A' :

- la *composizione* LL' di L e L' è il linguaggio su $A \cup A'$ formato dalle concatenazioni ww' con $w \in L$ e $w' \in L'$;
- le *potenze* L^n di L si definiscono per ogni n naturale ponendo $L^0 = \{\lambda\}$, $L^{n+1} = L^n L$; dunque, per ogni $n > 0$, $L^n = \{w_1 w_2 \dots w_n : w_1, w_2, \dots, w_n \in L\}$ è la composizione di n copie di L ;
- la *chiusura di Kleene* di L , indicata da L^* , è l'unione di tutte le potenze L^n di L al variare di n tra i numeri naturali; se invece ci limitiamo a unire le potenze L^n con $n > 0$, otteniamo la *chiusura positiva* di L , denotata L^+ .

Ovviamente $L^* = L^+ \cup \{\lambda\}$.

5.3 Grammatiche e Linguaggi

Nella lingua comune la grammatica sovrintende alla composizione delle frasi, fissando le regole con cui costruirle. Per esempio, ammette come frase corretta una sequenza *soggetto*, *verbo*, *complemento oggetto*, accetta poi la possibilità che il *soggetto* e il *complemento oggetto* siano nomi propri (come *maria*), ma esclude questa opzione per la categoria *verbo*. Con queste regole si riescono già a comporre frasi quali “*maria canta una romanza*”. Tuttavia, in assenza di ulteriori norme, anche frasi come “*maria canta maria*” diventano ammissibili e dunque altre regole andranno stabilite per evitare queste anomalie. Frasi semplici come le precedenti permettono poi di costruirne altre più complesse, come “*se maria canta una romanza, allora io tappo i miei orecchi*”, secondo nuove ulteriori convenzioni. Cerchiamo comunque di capire con maggior precisione come una grammatica regola il modo di formare una frase. Per schematizzare la situazione appena descritta (quella di maria e della romanza),

- (1) conviene usare simboli A , V , B rispettivamente per *soggetto*, *verbo*, *complemento oggetto* e, ancora, un simbolo di avvio S ,
- (2) d’altro lato, ci servono le comuni lettere dell’alfabeto $a, b, c, \dots$ per costruire un dato oggetto, o un dato verbo, o un dato complemento oggetto, e magari un ultimo simbolo “ $_$ ” a indicare uno spazio vuoto.

Siano N , T gli alfabeti (disgiunti) dei simboli rispettivamente in (1) e in (2). Consideriamo poi le seguenti regole per produrre le frasi a partire dal simbolo di avvio S :

$$\begin{aligned} S &\longrightarrow A V B, \\ A &\longrightarrow _P_, A \longrightarrow un_M_, A \longrightarrow una_F_, \\ B &\longrightarrow un_M, B \longrightarrow una_F, \\ P &\longrightarrow maria, M \longrightarrow orecchio, F \longrightarrow romanza, V \longrightarrow canta. \end{aligned}$$

Le prime sei ci consentono di costruire frasi composte da *soggetto* + *verbo* + *complemento oggetto* e di scegliere il soggetto tra nomi propri, o di genere maschile, o di genere femminile, escludono invece la prima opzione per i complementi oggetto. Le ultime 4 regole delimitano la scelta di nomi e verbi. Il complesso di queste norme determina la grammatica; infatti, possiamo chiamare *frase* ogni stringa di T (cioè dei simboli in (2)) che la grammatica produce a partire da S tramite un uso ripetuto delle regole appena stabilite. Si ottengono allora frasi come

maria_canta_una_romanza,

o anche

maria_canta_un_orecchio, un_orecchio_canta_una_romanza,

ma si vieta, per esempio,

maria_canta_maria.

Un simile approccio *grammaticale* consente di generare i nostri linguaggi formali e di specificare le stringhe che li compongono. In questo capitolo considereremo in particolare le così dette *grammatiche a struttura di frase*, che richiamano già dal nome il precedente paragone linguistico. Le introdurremo e discuteremo, vedremo in particolare come anch'esse possono essere pensate come modelli di calcolo ma che, sotto questo punto di vista, si dimostrano computazionalmente equivalenti alle MdT. Eccone intanto la definizione formale.

Definizione. Una *grammatica a struttura di frase* (o *grammatica di tipo 0*) è una quadrupla $G = \langle N, T, \mathcal{P}, S \rangle$ dove:

- T e N sono alfabeti disgiunti, composti rispettivamente da quelli che sono chiamati *simboli terminali* e *simboli non terminali* di G ;
- $\mathcal{P}$ è un insieme *finito* di coppie ordinate (γ, γ') (che in questo ambito denoteremo anche $\gamma \rightarrow \gamma'$) con γ, γ' parole su $N \cup T$ tali che γ contiene almeno un simbolo non terminale; le coppie di $\mathcal{P}$ sono chiamate *regole di produzione* o, più semplicemente, *produzioni* della grammatica G ;
- S è un simbolo di N , e viene chiamato simbolo iniziale di G .

In genere useremo le lettere maiuscole $A, B, C, \dots$ per indicare i simboli non terminali di G (che comprendono anche il simbolo iniziale S) e quelle minuscole $a, b, c, \dots$ per i simboli terminali; le stringhe composte *soltanto* da simboli terminali sono denotate da $u, v, w, x, y, z, \dots$, le altre, eventualmente aperte *anche* a simboli non terminali, dalle lettere greche $\alpha, \beta, \gamma, \dots$.

Esempio. Siano $N = \{S\}$, $T = \{a, b\}$. Per $\mathcal{P} = \{S \rightarrow aSb, S \rightarrow \lambda\}$, $G = \langle N, T, \mathcal{P}, S \rangle$ è una grammatica di tipo 0. Invece, se $\mathcal{P}$ è l'insieme delle coppie $\{S \rightarrow aSb, S \rightarrow ab, ab \rightarrow S\}$ allora $G = \langle N, T, \mathcal{P}, S \rangle$ non è una grammatica di tipo 0 perchè la parte sinistra della terza coppia non contiene simboli non terminali. Un esempio di grammatica di tipo 0 è, ancora, quello precedente di *maria* e delle *romanze*, come il lettore può facilmente verificare.

Per semplicità di notazione, quando più produzioni di una grammatica G condividono la stessa parte sinistra, come in $\gamma \rightarrow \gamma_1, \dots, \gamma \rightarrow \gamma_n$, scriveremo per sintetizzarle $\gamma \rightarrow \gamma_1 \mid \dots \mid \gamma_n$.

Ogni grammatica G genera un linguaggio fissando con le sue regole di produzione il modo di formarne le stringhe, come adesso descriviamo in dettaglio.

Definizione. Sia $G = \langle N, T, \mathcal{P}, S \rangle$ una grammatica a struttura di frase. Consideriamo stringhe con simboli terminali e non, dunque parole sull'alfabeto $N \cup T$. Diciamo allora che la stringa δ' è *derivabile direttamente* in G dalla stringa δ , e scriviamo $\delta \rightarrow_G \delta'$, se $\delta = \alpha\gamma\beta$ e $\delta' = \alpha\gamma'\beta$ per un'opportuna scelta di stringhe α, β, γ e γ' tali che $\gamma \rightarrow \gamma'$ è una produzione in $\mathcal{P}$. Dunque $\delta' = \alpha\gamma'\beta$ è ottenuta da $\delta = \alpha\gamma\beta$ sostituendovi γ con γ' .

Diciamo poi che δ' è *derivabile* in G da δ , e scriviamo $\delta \rightarrow_G^* \delta'$, se esistono un naturale n e stringhe $\delta_0, \delta_1, \dots, \delta_n$ tali che $\delta = \delta_0$, $\delta' = \delta_n$ e, per ogni $i = 1, \dots, n$, δ_i è derivabile direttamente in G da δ_{i-1} . La sequenza $\delta_0, \delta_1, \dots, \delta_n$ viene chiamata una *derivazione* di δ' da δ in G , e n la sua *lunghezza*.

Si chiama finalmente *linguaggio generato da G* , e si indica con $L(G)$, l'insieme di quelle stringhe w di simboli *terminali* di G che sono derivabili dal simbolo iniziale S in G . Quindi $L(G) = \{w \in T^* \mid S \rightarrow_G^* w\}$ è un linguaggio sull'alfabeto T .

Così, se G è la prima grammatica nell'Esempio precedente, $L(G)$ è formato, oltre che dalla parola vuota λ , dalle stringhe ab , a^2b^2 , a^3b^3 (tutte generate dal ripetuto uso della produzione $S \rightarrow aSb$ oltre che da $S \rightarrow \lambda$); si noti che $L(G)$ è generato anche dalla grammatica G' che aggiunge la produzione $S \rightarrow ab$ a quelle di G , e da altre grammatiche ancora. Un ulteriore esempio: se G è la grammatica di *maria* e delle *romanze*,

maria_canta_una_romanza,

fa parte di $L(G)$.

Definizione. Un linguaggio L in un alfabeto T si dice *di tipo 0* se $L = L(G)$ per qualche grammatica G di tipo 0 per la quale T è l'insieme dei simboli terminali.

Esempio. Ogni linguaggio finito L su un alfabeto T è di tipo 0 perchè è generato da una qualche grammatica G di tipo 0 (in genere non unica): basta infatti assumere $G = \langle N, T, \mathcal{P}, S \rangle$ dove N si riduce al solo simbolo iniziale S e $\mathcal{P}$ si compone di tutte e sole le produzioni $S \rightarrow_G w$ con $w \in L$. Siccome L è finito, anche l'insieme $\mathcal{P}$ è finito. La stessa costruzione non vale, evidentemente, per linguaggi infiniti. In effetti vedremo a fine capitolo che ci sono linguaggi che non sono di tipo 0.

Vale la pena di ribadire che può accadere che due diverse grammatiche G_1 e G_2 generino lo stesso linguaggio $L(G_1) = L(G_2)$.

Prima di proseguire, un avviso al lettore: nel seguito, scriveremo talora semplicemente $\rightarrow$ e $\rightarrow^*$ invece di $\rightarrow_G$ e $\rightarrow_G^*$ quando sarà evidente a quale grammatica G ci si riferisce.

5.4 La Gerarchia di Chomsky

Le grammatiche di tipo 0 sono anche note come *Grammatiche di Chomsky* perchè furono introdotte dal linguista americano Noam Chomsky, con lo scopo di rappresentare i procedimenti sintattici elementari che sono alla base della costruzione delle frasi nella lingua inglese. Pur essendosi presto rivelate uno strumento inadeguato per lo studio del linguaggio naturale, le grammatiche formali hanno un ruolo fondamentale nello studio delle proprietà sintattiche dei programmi e dei linguaggi di programmazione.

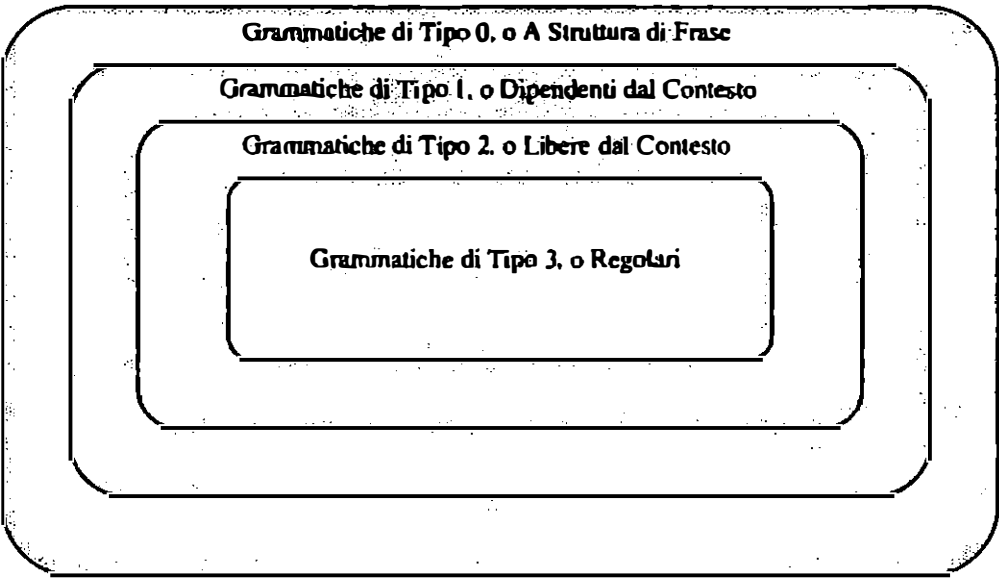


Figura 5.1 Gerarchie di grammatiche

La classe di tutte le grammatiche di tipo 0 contiene alcune sottoclassi notevoli, che adesso descriviamo. Le motivazioni che portano a considerarle sono varie e fanno spesso riferimento a esigenze di programmazione. Per esempio quelle che chiameremo grammatiche regolari sono importanti per la specifica dei token di un linguaggio di programmazione e quindi utili per l’analisi lessicale di un programma; invece quelle che definiremo grammatiche libere dal contesto si applicano all’analisi sintattica dei programmi. Ma è tempo di procedere con l’introduzione di queste sottoclassi.

Definizione. Una *grammatica di tipo 1, o dipendente dal contesto*, è una grammatica a struttura di frase $G = \langle N, T, \mathcal{P}, S \rangle$ tale che ogni produzione in $\mathcal{P}$ è della forma $\gamma A \delta \rightarrow \gamma \beta \delta$ con $\gamma, \delta \in (N \cup T)^*$, $A \in N$, $\beta \in (N \cup T)^+$ (quindi un simbolo non terminale A può essere sostituito da una stringa β purchè si trovi all’interno di un *contesto* della forma $\gamma _ \delta$).
Un linguaggio L si dice di *tipo 1, o dipendente dal contesto*, se $L = L(G)$ per qualche grammatica G di tipo 1.

Definizione. Una *grammatica di tipo 2, o libera dal contesto*, è una grammatica a struttura di frase $G = \langle N, T, \mathcal{P}, S \rangle$ tale che ogni produzione in $\mathcal{P}$ è della forma $A \rightarrow \beta$ con $A \in N$ e $\beta \in (N \cup T)^+$ (cioè, un simbolo non terminale A può essere sostituito dalla stringa β indipendentemente dal contesto in cui si trova).
Un linguaggio è di *tipo 2, o libero dal contesto*, se esiste una grammatica di tipo 2 che lo genera.

Definizione. Una *grammatica di tipo 3, o regolare*, è una grammatica a struttura di frase $G = \langle N, T, \mathcal{P}, S \rangle$ tale che ogni produzione in $\mathcal{P}$ è della forma $A \rightarrow aB$ oppure della forma $A \rightarrow a$ con $A, B \in N$ e $a \in T$ (cioè, un simbolo non terminale A può essere sostituito da un simbolo terminale seguito da uno non terminale oppure direttamente da un simbolo terminale).
Un linguaggio si chiama di *tipo 3, o regolare*, se esiste una grammatica di tipo 3

che lo genera.

È facile verificare che le grammatiche regolari sono anche libere dal contesto e che quelle libere dal contesto sono anche dipendenti dal contesto (basta porre $\gamma = \delta = \lambda$); inoltre quelle dipendenti dal contesto sono anche grammatiche a struttura di frase. È altrettanto semplice convincersi che esistono grammatiche a struttura di frase che non sono dipendenti dal contesto, grammatiche dipendenti dal contesto che non sono libere e, infine, grammatiche libere dal contesto che non sono regolari. Si determina così all'interno della classe di tutte le grammatiche a struttura di frase, rispetto alla relazione di inclusione, una *gerarchia* di grammatiche (usualmente chiamata *gerarchia di Chomsky*), in cui si succedono progressivamente quelle regolari, poi quelle libere dal contesto e, ancora, quelle dipendenti dal contesto.

Vale poi forse la pena di sottolineare che le grammatiche G introdotte in questo paragrafo (quelle dipendenti dal contesto e, di conseguenza, quelle libere dal contesto e quelle regolari) non generano mai con le loro produzioni la stringa vuota λ e dunque la escludono dal linguaggio $L(G)$. Si tratta, ovviamente, di convenzione formale. Comunque, ove si preferisse coinvolgere λ nelle proprie considerazioni, si potrebbe convenire di estendere l'insieme delle produzioni di G ammettendovi anche $S \rightarrow \lambda$ con l'unica precauzione che il simbolo iniziale S non compaia a destra di nessun'altra produzione. Questo adattamento non modifica sostanzialmente il quadro generale delle precedenti definizioni. Nel nostro libro, manterremo tuttavia la scelta di escludere λ dai linguaggi di tipo 1, 2 e 3.

5.5 Alberi di Derivazione

Ogni grammatica a struttura di frase G genera un suo linguaggio $L(G)$ e quindi pone in modo naturale il problema di decidere, per ogni stringa w di simboli terminali, se w appartiene o no a $L(G)$, e cioè se w è o no derivabile in G dal simbolo iniziale S di G . Nel caso di grammatiche G libere dal contesto (quindi anche per grammatiche regolari) e per stringhe $w \in L(G)$ è possibile rappresentare il processo che genera w da S in G tramite un *albero*, nel modo che adesso descriviamo.

Definizione. Siano $G = \langle N, T, \mathcal{P}, S \rangle$ la nostra grammatica libera dal contesto, $w \in L(G)$. L'*albero di derivazione* di w in G è un albero finito (in cui per semplicità è fissato un ordinamento dei nodi) in cui:

- a ogni nodo si assegna come etichetta un simbolo in $N \cup T$,
- in particolare alla radice si assegna il simbolo iniziale S , ai nodi che non sono foglie si associano simboli non terminali (di N) e alle foglie simboli terminali (di T),
- si chiede poi che, se un nodo di etichetta A ha come figli nell'ordine stabilito nell'albero i nodi di etichette $A_1, \dots, A_n$ (terminali o no), allora $A \rightarrow A_1 \dots A_n$ sia una produzione di $\mathcal{P}$,

- d. finalmente, la stringa delle etichette delle foglie (elencate secondo l'ordine fissato nell'albero) è proprio w .

Esempio. Si consideri la grammatica di tipo 0 $G = \langle N, T, \mathcal{P}, S \rangle$, dove $N = \{S\}$, $T = \{a, b\}$ e $\mathcal{P}$ è l'insieme di produzioni

$$S \longrightarrow aSb \mid ab.$$

Si noti che G è libera dal contesto (**esercizio:** G è regolare?). Inoltre a^3b^3 appartiene a $L(G)$. La Figura 5.2 fornisce l'albero di derivazione in G per la stringa a^3b^3 .

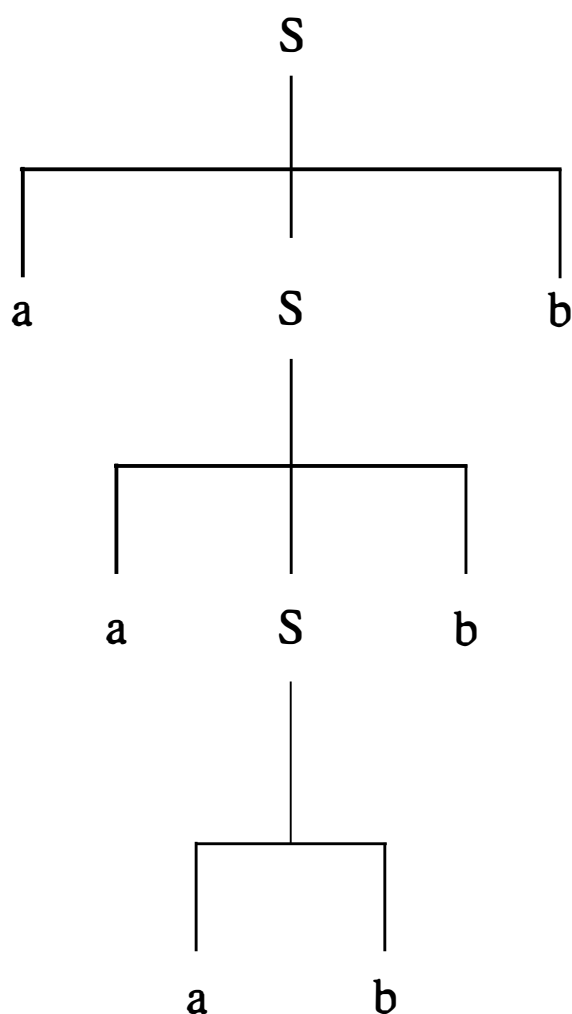


Figura 5.2 Albero di derivazione per $aaabbb$

Nel seguito ci capiterà di incontrare talora anche alberi di derivazione costruiti nel modo sopra descritto, ma liberi di ammettere come radice un qualunque simbolo non terminale, e non soltanto obbligatoriamente S .

Ci si può poi domandare come mai gli alberi di derivazione sono definiti soltanto per grammatiche libere dal contesto e non anche per grammatiche dipendenti dal contesto. D'altra parte, si può ricordare che, nel più vasto ambito dipendente dal contesto, le produzioni ammettono a sinistra non solo singoli elementi di N , e cioè simboli non terminali di G , ma anche stringhe che li contengono, il che rende ovviamente problematica l'assegnazione delle etichette ai vari nodi dell'albero.

5.6 Linguaggi regolari

Diamo anzitutto un esempio di linguaggio regolare.

Esempio. Consideriamo l'alfabeto $\{a, b\}$ e in esso il linguaggio

$$L = \{a^n b^m \mid n, m \in \mathbb{N}, n, m \geq 1\}.$$

Allora L è regolare, perchè si verifica facilmente che L è generato dalla grammatica di tipo 3 $G = \langle N, T, \mathcal{P}, S \rangle$ dove $N = \{S, B\}$, $T = \{a, b\}$ e $\mathcal{P}$ è l'insieme di produzioni dato da:

$$\begin{array}{lcl} S & \longrightarrow & aS \mid aB \\ B & \longrightarrow & bB \mid b. \end{array}$$

Quanto ai linguaggi non regolari, c'è un risultato fondamentale noto come *Teorema uvw* , o *Pumping Lemma* (quest'ultima denominazione inglese è ormai invalsa anche nell'uso italiano) che stabilisce una condizione che un linguaggio regolare deve **necessariamente** soddisfare, e dunque esclude dai linguaggi regolari tutti quelli incapaci di rispettarla.

Teorema uvw (Pumping Lemma per Linguaggi Regolari) 5.6.1 *Sia L un linguaggio di tipo 3 su un alfabeto T . Esiste una costante $n \in \mathbb{N}$ (dipendente da L) tale che, per ogni $z \in L$, se $l(z) \geq n$, allora ci sono stringhe $u, v, w \in T^*$ per cui valgono le seguenti proprietà:*

- (a) $z = uvw$;
- (b) $0 < l(v) \leq n$;
- (c) $\forall i \in \mathbb{N}, uv^i w \in L$.

Dimostrazione. Sia $G = \langle N, T, \mathcal{P}, S \rangle$ una grammatica regolare che genera L , cioè $L = L(G)$. Sia poi n la cardinalità di N , cioè il numero di simboli non terminali della grammatica. Dalla definizione di $L(G)$, segue che la stringa z appartiene a L se e solo se z è derivabile in G dal simbolo iniziale ($S \xrightarrow{*}_G z$) e cioè se e solo se esiste una sequenza di stringhe $w_0, w_1, \dots, w_m$ tali che $S = w_0$, $z = w_m$ e, per ogni $i = 1, \dots, m$, w_i è derivabile direttamente da w_{i-1} in G

$$S = w_0 \longrightarrow_G w_1 \dots \longrightarrow_G w_i \longrightarrow_G \dots \longrightarrow_G w_m = z.$$

Dalla definizione di linguaggio regolare segue poi che ogni w_i deve essere del tipo $w_i = w'_i A_i$ per qualche stringa di simboli terminali $w'_i \in T^*$ e per qualche simbolo non terminale $A_i \in N$; in particolare w'_i è un prefisso proprio di w'_{i+1} e quindi di tutte le stringhe w'_k per $k > i$. Infatti le produzioni di G che generano $w_1, \dots, w_m$ da S sono della forma $A \longrightarrow aB$ oppure $A \longrightarrow a$ con $A, B \in N$ e $a \in T$. Per lo stesso motivo abbiamo che, se vale l'ipotesi $l(z) \geq n$, allora $m \geq n$; di conseguenza, siccome n è il numero totale dei simboli non terminali e $m \geq n$, deve esistere almeno un simbolo non terminale A che si ripete nella

precedente rappresentazione delle stringhe $w_0, \dots, w_m$. Siano $i < k \leq m$ gli indici minimi per cui si osserva questa ripetizione

$$S = w_0 \rightarrow_G w_1 \dots \rightarrow_G w'_i A \rightarrow_G \dots w'_k A \rightarrow_G \dots \rightarrow_G w_m = z.$$

Consideriamo le stringhe w'_i, w'_k e z . Siccome w'_i è un prefisso proprio di w'_k e w'_k è un prefisso di z abbiamo che $w'_k = w'_i \rho$ e $z = w'_k \sigma$ per opportune stringhe ρ e σ in T^* (e per σ eventualmente vuota). A questo punto poniamo $u = w'_i, v = \rho$ e $w = \sigma$. E' facile convincersi che u, v e w così definiti soddisfano le condizioni (a), (b) e (c) del teorema. $\square$

Ribadiamo che il Teorema uvw fornisce soltanto una condizione necessaria perchè un linguaggio sia regolare. Esso non può essere quindi utilizzato per mostrare la regolarità di un linguaggio, ma semmai per escludere che un linguaggio sia regolare, come il seguente esempio mostra.

Esempio. Il linguaggio

$$L = \{a^i b^i \mid i \in \mathbb{N}, i \geq 1\}$$

non è regolare. Supponiamo infatti che lo sia. Applicando il Teorema uvw troviamo $n \in \mathbb{N}$ tale che, $\forall z \in L$, se $l(z) \geq n$ allora $z = uvw$ con $0 < l(v) \leq n$ e $uv^i w \in L$ per ogni naturale i . Prendiamo dunque la stringa $z = a^n b^n$. Ovviamente $z \in L$ e $l(z) = 2n \geq n$ e quindi esistono stringhe u, v e w come descritte dal teorema; in particolare $0 < l(v) \leq n$. A questo punto distinguiamo 3 possibilità:

1. v contiene solo simboli a ,
2. v contiene solo simboli b ,
3. v contiene tanto a quanto b .

In tutti e tre i casi, si osserva facilmente che $uv^2w \notin L$ e cioè che uv^2w non si può scrivere come $a^i b^i$ per nessun naturale i . Ricordiamo infatti che uvw sta in L e quindi si può esprimere in questa forma. Ma allora l'immissione di un'ulteriore sottostringa non vuota v produce

1. più a che b ,
2. più b che a ,
3. alternanze di a, b e ancora a

e quindi situazioni incompatibili con L .

Altre applicazioni del Teorema uvw rivelano importanti proprietà di decidibilità dei linguaggi regolari. Discutiamone una.

Proposizione 5.6.2 *Siano G una grammatica di tipo 3, $L = L(G)$ il corrispondente linguaggio regolare. Sia poi n il numero dei simboli non terminali di G e cioè la costante associata dal Teorema uvw a L . Allora*

- (a) L non è vuoto se e solo se G genera una stringa z di lunghezza $< n$.

(b) L è infinito se e solo se G genera una stringa z tale che $n \leq l(z) < 2n$.

Come vedremo nel prossimo paragrafo, la proposizione si può estendere opportunamente anche alle grammatiche e ai linguaggi liberi dal contesto. Ne presentiamo comunque la dimostrazione nel caso regolare, per illustrarvi il ruolo del Teorema uvw .

Dimostrazione. (a) È chiaro che, se G genera qualche stringa z , allora, a prescindere dalla lunghezza di z , $L = L(G)$ non è vuoto. Viceversa assumiamo che L non sia vuoto ma che tutte le stringhe di L abbiano lunghezza $\geq n$. Sia poi z una stringa di lunghezza minima in L . Poiché $l(z) \geq n$, per il Teorema uvw esistono stringhe u, v e w tale che $z = uvw$, $0 < l(v) \leq n$ e, per ogni naturale i , $uv^i w \in L$. In particolare $uv^0 w = uw \in L$; ma uw ha lunghezza inferiore a $z = uvw$, e questo contraddice la scelta di z . Allora L , quando non è vuoto, deve contenere stringhe di lunghezza $< n$.

(b) Supponiamo dapprima che G generi una stringa z tale che $n \leq l(z) < 2n$. Per il Teorema uvw ci sono u, v, w nell'alfabeto T dei simboli terminali di G tali che $z = uvw$, $v \neq \lambda$ e ogni stringa $uv^i w$ (per i naturale) è in L . Ovviamente $uv^i w \neq uv^j w$ per $i \neq j$. Dunque L ha infiniti elementi. Viceversa, assumiamo L infinito. Siccome il numero delle stringhe di fissata lunghezza su T è comunque finito, L contiene elementi di lunghezza arbitrariamente grande. Sia in particolare z una stringa in L di lunghezza minima tra quelle di lunghezza $\geq n$. Se $l(z) < 2n$ allora abbiamo la stringa che stiamo cercando. Altrimenti applichiamo il Teorema uvw a z come nel caso precedente e troviamo u, v e w in T^* con $0 < l(v) \leq n$, $z = uvw$ e $uv^i w \in L$ per ogni naturale i . In particolare $uw \in L$; inoltre $l(uw) \geq n$ perché $l(z) \geq 2n$ e $l(v) \leq n$; finalmente $l(uw) < l(z)$ perché $l(v) > 0$. Ma così si contraddice la scelta di z . $\square$

Come già osservato nel corso della precedente dimostrazione, le stringhe di T^* di una prefissata lunghezza l sono in numero finito, e dunque si possono enumerare e decidere effettivamente. Lo stesso vale ovviamente anche per stringhe di lunghezza $\leq l$, o compresa tra l e $2l$, per ogni l . Consideriamo allora i problemi di decidere, per ogni grammatica regolare G ,

- (a) se $L(G)$ è vuoto o no,
- (b) se $L(G)$ è finito o no.

In entrambi i casi, una strategia di soluzione consiste nel calcolare anzitutto il numero n dei simboli non terminali di G e poi, se T denota l'alfabeto dei simboli terminali di G , considerare una per una

- (a) le stringhe $z \in T^*$ con $l(z) < n$,
- (b) le stringhe $z \in T^*$ con $n < l(z) \leq 2n$

cercando di stabilire, per ognuna di esse, se z è o non è in $L(G)$. Se l'esame dà esito positivo almeno una volta, allora

- (a) $L(G) \neq \emptyset$,

(b) $L(G)$ è infinito,

altrimenti

(a) $L(G) = \emptyset$,

(b) $L(G)$ è finito.

Dunque i due problemi hanno algoritmo di soluzione **purché** ci sia un procedimento effettivo che sappia decidere, per ogni $z \in T^*$, se $z \in L(G)$ oppure no, in altre parole **purché** $L(G)$ sia decidibile. Vedremo in § 5.8 che in realtà questo è vero non solo per le grammatiche regolari, ma anche per le grammatiche di tipo 1. Se dunque il lettore ha sufficiente fiducia per accettare sin d'ora questo risultato e sufficiente pazienza per aspettarne la dimostrazione in § 5.8, potrà ammettere senza problemi:

Teorema 5.6.3 *Ci sono algoritmi effettivi che sanno decidere, per ogni linguaggio regolare L ,*

(a) *se L è vuoto oppure no,*

(b) *se L è finito oppure no.*

Un'altra proprietà notevole dei linguaggi regolari è la loro capacità di preservarsi rispetto alle usuali operazioni su linguaggi.

Proposizione 5.6.4 *La classe dei linguaggi di tipo 3 è chiusa rispetto alle operazioni di unione, concatenazione, complementazione, intersezione e chiusura di Kleene.*

Dimostrazione. Ci limitiamo a considerare il caso dell'unione. Le altre operazioni di concatenazione, complementazione e chiusura di Kleene si trattano in modo analogo e sono lasciate al lettore come esercizio. La chiusura rispetto all'intersezione segue invece direttamente dalla chiusura rispetto all'unione e alla complementazione: infatti è ben noto che l'intersezione di due insiemi è il complementare dell'unione dei loro complementari. Consideriamo dunque il caso dell'unione. Siano $G_1 = \langle N_1, T_1, \mathcal{P}_1, S_1 \rangle$ e $G_2 = \langle N_2, T_2, \mathcal{P}_2, S_2 \rangle$ due grammatiche regolari, L_1 e L_2 i linguaggi che esse rispettivamente generano. Possiamo assumere per semplicità $N_1 \cap N_2 = \emptyset$, altrimenti si provvede a ridenominare i simboli non terminali in comune tra le due grammatiche in modo da disgiungerne gli insiemi; l'operazione è possibile senza che i simboli terminali di G_1 e G_2 (quelli che interessano L_1 e L_2) abbiano a risentirne sostanzialmente. Vogliamo trovare una grammatica regolare $G_3 = \langle N_3, T_3, \mathcal{P}_3, S_3 \rangle$ tale che $L(G_3) = L_1 \cup L_2$. A tale scopo, possiamo anzitutto porre $N_3 = N_1 \cup N_2 \cup \{S_3\}$, $T_3 = T_1 \cup T_2$. A questo punto, la strategia più immediata per generare le stringhe di L_1 o di L_2 pare quella di aggiungere in $\mathcal{P}_3$ alle produzioni di $\mathcal{P}_1 \cup \mathcal{P}_2$

$$S_3 \longrightarrow S_1 | S_2.$$

In tal modo, infatti, S_3 dà origine, tramite S_1 o S_2 , a tutte le stringhe generate da G_1 o da G_2 . Ma $S_3 \rightarrow S_1|S_2$ non è ammessa nelle grammatiche regolari, che richiedono espressamente un simbolo terminale a destra di ogni produzione. Così dobbiamo procedere in modo lievemente più lungo, accogliendo in $\mathcal{P}_3$ le produzioni in $\mathcal{P}_1$, quelle in $\mathcal{P}_2$ e in più $S_3 \rightarrow \gamma$ se $S_1 \rightarrow \gamma \in \mathcal{P}_1$ oppure $S_2 \rightarrow \gamma \in \mathcal{P}_2$. Non è difficile convincersi che la grammatica G_3 così ottenuta è regolare e che il linguaggio generato da G_3 è l'unione di L_1 e L_2 . $\square$

Dalla precedente proposizione segue direttamente la decidibilità del *problema dell'equivalenza tra grammatiche regolari*, ovvero della questione se due grammatiche regolari generano o no lo stesso linguaggio. Si ha infatti:

Corollario (Problema dell'Equivalenza di Grammatiche Regolari) 5.6.5 *C'è un algoritmo che sa decidere, per ogni scelta di grammatiche regolari G_1 ed G_2 , se $L(G_1) = L(G_2)$ oppure no.*

Dimostrazione. La teoria degli insiemi ci dice che $L(G_1) = L(G_2)$ se e solo se $L' = (L(G_1) \cap L(G_2)^c) \cup (L(G_1)^c \cap L(G_2))$ è vuoto. Ma per la Proposizione 5.6.4 anche L' è regolare, e dunque per il Teorema 5.6.3 possiamo stabilire se L' è vuoto oppure no. $\square$

5.7 Linguaggi Liberi dal Contesto

Molte delle proprietà appena viste per i linguaggi regolari sono soddisfatte anche da quelli liberi dal contesto (magari con gli opportuni adattamenti); altre invece si perdono. Questo paragrafo è appunto dedicato all'analisi dei linguaggi liberi dal contesto. Iniziamo col mostrare che esistono linguaggi liberi dal contesto ma non regolari: un esempio è il seguente.

Esempio. Consideriamo nell'alfabeto $\{a, b\}$ il linguaggio

$$L = \{a^i b^i \mid i \in \mathbb{N}, i \geq 1\}.$$

Già sappiamo dallo scorso paragrafo che L non è regolare. Invece L è libero dal contesto. Infatti si vede facilmente (e in realtà ci è già capitato di osservare) che L è generato dalla grammatica di tipo 2 $G = \langle N, T, \mathcal{P}, S \rangle$ dove $N = \{S\}$, $T = \{a, b\}$ e $\mathcal{P}$ è l'insieme di produzioni dato da:

$$S \rightarrow aSb \mid ab.$$

Una versione del *Pumping Lemma* per linguaggi liberi dal contesto è la seguente.

Teorema $uvwxy$ (Pumping Lemma per Linguaggi Liberi dal Contesto) 5.7.1

Sia L un linguaggio di tipo 2 su un alfabeto T . Allora esiste un intero positivo n dipendente da L tale che, per ogni $z \in L$, se $l(z) > n$, ci sono $u, v, w, x, y \in T^$ per cui valgono le seguenti proprietà:*

- (a) $z = uvwxy$;
- (b) $l(vwx) \leq n$;
- (c) $l(vx) > 0$;
- (d) $\forall i \in \mathbb{N}, uv^iwx^iy \in L$.

Dimostrazione. Sia $G = \langle N, T, \mathcal{P}, S \rangle$ una grammatica libera dal contesto per cui $L = L(G)$ e sia p il numero dei simboli non terminali di G . Consideriamo tutti gli alberi di derivazione di G che hanno l'etichetta di un simbolo non terminale come radice e ammettono profondità $\leq p + 1$ (ricordiamo che la *profondità* di un albero è la lunghezza massima di un suo cammino -cioè di una sua sequenza di lati- dalla radice a una foglia). Otteniamo così un numero finito di alberi: sia n la lunghezza massima delle stringhe generate dai nodi foglia di questi alberi nel modo descritto in § 5.5.

Consideriamo adesso una stringa $z \in L$ tale che $l(z) > n$: z può essere generata solo da un albero di derivazione (con radice S) di profondità maggiore di $p + 1$. Tra tutti gli alberi di derivazione con radice S per la stringa z consideriamone uno con il numero minimo di nodi e scegliamo in questo albero un cammino di lunghezza maggiore di $p + 1$ da S a una foglia. Poiché p è il numero dei simboli non terminali di G , questo cammino deve contenere almeno un simbolo non terminale che si ripete tra le etichette dei suoi nodi; in altre parole, esistono due nodi N_1 e N_2 del cammino tali che:

- N_1 e N_2 hanno la stessa etichetta non terminale, diciamo A ;
- N_2 è discendente di N_1 (cioè è più vicino alla foglia);
- il cammino da N_1 alla foglia è di lunghezza al più $p + 1$.

Per trovare N_1 e N_2 basta percorrere a ritroso il cammino dalla foglia alla radice alla ricerca della prima ripetizione di una etichetta A .

Il sottoalbero di radice N_2 genera una stringa $w \in T^*$ e quindi rappresenta una derivazione $A \xrightarrow{*}_G w$. Da parte sua il sottoalbero di radice N_1 , che comprende N_2 tra i suoi nodi, genera una parola che contiene w come sottostringa e la racchiude come $vw x$ per un'opportuna scelta di v e x in T^* . Allo stesso modo l'intero albero di computazione di z , che include N_1 tra i suoi nodi, genera una stringa che allarga $vw x$ e quindi ha la forma $uvwxy$ per opportune parole u e y in T^* ; ma allora $z = uvwxy$, cioè vale (a).

Se adesso sfruttiamo il fatto che N_1 e N_2 ammettono la stessa etichetta A e sostituiamo nell'albero scelto per la derivazione di z il sottoalbero di radice N_2 con quello di radice N_1 , generiamo con le foglie la stringa che si ottiene rimpiazzando w con $vw x$ in $z = uvwxy$, e cioè uv^2wx^2y . Si prova così che uv^2wx^2y è generata da G e quindi appartiene a L . Allo stesso modo, si dimostra che, per ogni $i > 0$, $uv^iwx^iy \in L$. Se invece nell'albero di z sostituiamo il sottoalbero di radice N_1 con quello di radice N_2 (di nuovo sfruttando il fatto che N_1 e N_2 hanno la stessa etichetta) proviamo che anche $uvw y$ sta in L , cioè $uv^0wx^0y \in L$ anche per $i = 0$. Questo dimostra (d).

Per provare (b), si noti anzitutto che il sottoalbero di radice N_1 ha profondità $\leq p + 1$; quindi la stringa che il sottoalbero genera, e cioè $vw x$, ha lunghezza al

più n per la definizione stessa di n .

Finalmente dimostriamo (c). Se $l(vx) = 0$, e cioè vx è vuota, z coincide con uvw e quindi continua a essere generata anche se sostituiamo nel suo albero di derivazione il sottoalbero di radice N_1 con quello di radice N_2 , e dunque diminuiamo il numero complessivo dei nodi: ma questo contraddice la scelta dell'albero stesso. $\square$

Anche per linguaggi liberi dal contesto, come già per linguaggi regolari, il *Pumping Lemma* può essere usato per costruire esempi negativi, come il seguente.

Esempio. Il linguaggio

$$L = \{a^i b^i c^i \mid i \in \mathbb{N}, i \geq 1\}$$

non è libero dal contesto. Supponiamo infatti che lo sia. Applicando il *Pumping Lemma* troviamo $n \in \mathbb{N}$ tale che, se $z \in L$ e $l(z) > n$, allora ci sono $u, v, w, x, y \in T^*$ per cui $z = uvwxy$, $l(vwx) \leq n$, $l(vx) > 0$ e $\forall i \in \mathbb{N}$ $uv^iwx^i y \in L$. Consideriamo la stringa $z = a^n b^n c^n$. Ovviamente $z \in L$ e $l(z) = 3n > n$. A questo punto il ragionamento dell'esempio dopo il Teorema uvw si ripete e produce una contraddizione, mostrando che L non può essere libero dal contesto. Il lettore può controllarne da solo i dettagli.

La seguente proposizione caratterizza i linguaggi di tipo 2 che sono non vuoti, oppure finiti, estendendo in tal senso i risultati già provati per linguaggi regolari in § 5.5. Anche la dimostrazione è analoga ma, al posto del Teorema uvw , sfrutta il Teorema $uvwxy$.

Proposizione 5.7.2 *Siano G una grammatica di tipo 2, $L(G)$ il linguaggio che G genera. Sia poi n la costante associata a G dal Teorema $uvwxy$. Allora*

- (a) $L(G)$ non è vuoto se e solo se G genera una stringa di lunghezza $\leq n$,
- (b) $L(G)$ è infinito se e solo se G genera una stringa z tale che $n < l(z) \leq 2n$.

Dimostrazione. (a) L'implicazione da destra a sinistra è ovvia. Viceversa supponiamo che $L(G)$ non sia vuoto, ma contenga solo stringhe di lunghezza $> n$. Sia $z \in L(G)$ di lunghezza minima (comunque $> n$). Appliciamo il Teorema $uvwxy$ ed esprimiamo appunto z come $uvwxy$ per opportune stringhe u, v, w, x, y tali che, in particolare, $l(vx) > 0$ e per ogni naturale i $uv^iwx^i y \in L(G)$. In particolare, per $i = 0$ anche $uvw \in L(G)$. Ma $l(uvw) < l(z)$ perchè $l(vx) > 0$, così uvw contraddice la scelta di z . Segue che, se $L(G) \neq \emptyset$, $L(G)$ contiene stringhe di lunghezza $\leq n$.

(b) Se $L(G)$ contiene una stringa z di lunghezza maggiore di n , allora $L(G)$ è infinito: basta applicare il Teorema $uvwxy$ per osservare che z si scrive $uvwxy$ per opportune stringhe u, v, w, x e y tali che, in particolare, $L(G)$ comprende tutte le infinite parole $uv^iwx^i y$ per i che varia tra i numeri naturali. Viceversa, supponiamo che $L(G)$ sia infinito. Allora $L(G)$ contiene stringhe di ogni possibile lunghezza, in particolare anche stringhe z di lunghezza $> n$. Scegliamo z di lunghezza minima (comunque $> n$). Se $l(z) \leq 2n$, siamo a posto. Altrimenti

appliciamo di nuovo il Teorema $uvvwx$ a z e la esprimiamo come $uvvwx$ ricavando u, v, w, x e y dalle parole sull'alfabeto T dei simboli terminali di G : in particolare, $uvw \in L(G)$, $l(uvw) \leq n$ e $l(vx) > 0$. Notiamo $l(uvw) < l(z)$ proprio perchè $l(vx) > 0$; inoltre $l(uvw) \geq l(uv) > n$ perchè $l(uvwx) = l(z) > 2n$ e $l(vwx) \leq n$. Ma allora uvw contraddice la scelta di z . $\square$

Come già nel caso dei linguaggi regolari, possiamo allora ridurre il problema di stabilire se un linguaggio L libero dal contesto è o no vuoto, oppure è o no finito, alla questione della decidibilità di L . Come già preannunciato, dimostreremo in § 5.8 che, in realtà, tutti i linguaggi dipendenti dal contesto (e in particolare quelli liberi dal contesto) sono decidibili. Se accettiamo sin da ora questa conclusione, possiamo enunciare il seguente risultato, che estende il Teorema 5.6.3.

Teorema 5.7.3 *Ci sono algoritmi effettivi che sanno decidere, per ogni linguaggio libero dal contesto L ,*

- (a) *se L è vuoto oppure no,*
- (b) *se L è finito oppure no.*

Esaminiamo adesso il comportamento dei linguaggi liberi dal contesto rispetto alla usuali operazioni tra linguaggi.

Proposizione 5.7.4 *I linguaggi di tipo 2 sono chiusi rispetto alle operazioni di unione, concatenazione e chiusura di Kleene.*

Dimostrazione. La dimostrazione è simile a quella per i linguaggi regolari, e anche più semplice. Siano L_1, L_2 due linguaggi liberi dal contesto, $G_1 = \langle N_1, T_1, \mathcal{P}_1, S_1 \rangle$ e $G_2 = \langle N_2, T_2, \mathcal{P}_2, S_2 \rangle$ le grammatiche tipo 2 che li generano. Di nuovo possiamo assumere che N_1 e N_2 sono disgiunti. Per provare che, per esempio, $L_1 \cup L_2$ è di tipo 2, prendiamo un nuovo simbolo iniziale S_3 e formiamo

$$G_3 = \langle N_1 \cup N_2 \cup \{S_3\}, T_1 \cup T_2, \mathcal{P}_1 \cup \mathcal{P}_2 \cup \{S_3 \rightarrow S_1 | S_2\}, S_3 \rangle,$$

accettiamo dunque $S_3 \rightarrow S_1 | S_2$ nelle produzioni di G_3 , come concesso dalla definizione di grammatica libera dal contesto anche se proibito nelle grammatiche regolari. Si verifica allora che G_3 è davvero una grammatica libera dal contesto e che genera $L_1 \cup L_2$. In modo analogo si procede per concatenazione e chiusura di Kleene. $\square$

Ma i linguaggi di tipo 2, a differenza di quelli regolari, non si preservano per intersezione. Vediamo perchè.

Proposizione 5.7.5 *La classe dei linguaggi di tipo 2 non è chiusa rispetto alla intersezione.*

Dimostrazione. Il linguaggio $L_1 = \{a^i b^j c^j \mid i, j \in \mathbb{N}, i, j \geq 1\}$ è generato dalla grammatica G_1 con produzioni

$$\begin{aligned} S &\longrightarrow AB \\ A &\longrightarrow ab \mid aAb \\ B &\longrightarrow c \mid cB \end{aligned}$$

e il linguaggio $L_2 = \{a^i b^j c^j \mid i, j \in \mathbb{N}, i, j \geq 1\}$ è generato da una grammatica G_2 con produzioni

$$\begin{aligned} S &\longrightarrow AB \\ A &\longrightarrow a \mid aA \\ B &\longrightarrow bc \mid bBc. \end{aligned}$$

Si vede facilmente che le grammatiche G_1 e G_2 sono libere dal contesto, e quindi tali sono anche L_1 e L_2 . Ma la loro intersezione è il linguaggio $\{a^i b^i c^i \mid i \in \mathbb{N}, i \geq 1\}$, che abbiamo visto non essere libero dal contesto. $\square$

Ne segue:

Corollario 5.7.6 *La classe dei linguaggi di tipo 2 non è chiusa rispetto alla complementazione.*

Dimostrazione. Una classe chiusa per unione e complementazione lo è anche per intersezione, visto che l'intersezione di due insiemi è il complemento dell'unione dei loro complementi. $\square$

Si ricordi che la soluzione positiva del problema dell'equivalenza di grammatiche regolari, così come dimostrata nel § 5.6, dipende proprio dal fatto che la classe dei linguaggi regolari si preserva per intersezione e complemento. Come abbiamo appena visto, non altrettanto vale per arbitrari linguaggi liberi dal contesto, e in effetti si può provare che il *problema dell'equivalenza per grammatiche libere dal contesto* rimane indecidibile.

Teorema 5.7.7 *Non c'è algoritmo capace di stabilire, date due grammatiche libere dal contesto G_1 e G_2 , se $L(G_1) = L(G_2)$ oppure no.*

La dimostrazione è elaborata; così rimandiamo ai testi citati in bibliografia per i dettagli.

5.8 Linguaggi Dipendenti dal Contesto

Trattiamo adesso linguaggi dipendenti dal contesto. È anzitutto opportuno fornire un esempio, magari non libero dal contesto. Consideriamo allora il linguaggio

$$L = \{a^i b^i c^i \mid i \in \mathbb{N}, i \geq 1\}$$

sull'alfabeto $\{a, b\}$. Già sappiamo che L non è libero dal contesto. Tuttavia si dimostra che L è dipendente dal contesto. Possiamo infatti osservare che L è generato dalla grammatica $G = \langle N, T, \mathcal{P}, S \rangle$ dove $N = \{S\}$, $T = \{a, b\}$ e $\mathcal{P}$ è l'insieme di produzioni dato da:

$$\begin{array}{ll} S & \longrightarrow aSBC \mid aBC \\ CB & \longrightarrow BC \\ bB & \longrightarrow bb \\ bC & \longrightarrow bc \\ cC & \longrightarrow cc \\ aB & \longrightarrow ab. \end{array}$$

D'altra parte va riconosciuto che questa grammatica G non è a stretto rigore di termini dipendente dal contesto, perchè la produzione $CB \longrightarrow BC$ non può essere della forma prevista $\gamma A \delta \longrightarrow \gamma \beta \delta$ con $\gamma, \delta \in (N \cup T)^*$, $A \in N$ e $\beta \in (N \cup T)^+$. In compenso G ha la proprietà che, per ogni produzione $\alpha \longrightarrow \beta$ di G , si ha $l(\alpha) \leq l(\beta)$. Consideriamo allora tutte le grammatiche che soddisfano questa condizione.

Definizione. Una grammatica a struttura di frase G si dice *monotona* se, per ogni produzione $\alpha \longrightarrow \beta$ in G , si ha $l(\alpha) \leq l(\beta)$.

È evidente che, all'interno della gerarchia di Chomsky, tutte le grammatiche dipendenti dal contesto sono monotone, ma che viceversa si possono costruire grammatiche monotone che non sono dipendenti dal contesto: ne abbiamo appena visto un esempio. Così le grammatiche monotone formano una nuova classe nella gerarchia di Chomsky, che estende propriamente quella delle grammatiche di tipo 1. Tuttavia c'è un risultato fondamentale che collega queste due sottoclassi, o meglio i linguaggi generati dalle rispettive grammatiche:

Teorema 5.8.1 *I linguaggi generati da grammatiche monotone coincidono con quelli dipendenti dal contesto.*

Quindi, per ogni grammatica monotona G , ce n'è una dipendente dal contesto G' tale che $L(G) = L(G')$. La dimostrazione di questa affermazione non è difficile e può trovarsi in [81]. Ne possiamo dedurre, in particolare, che $L = \{a^i b^i c^i \mid i \in \mathbb{N}, i \geq 1\}$, che è generato da una grammatica monotona, risulta dipendente dal contesto.

Ovviamente anche le grammatiche di tipo 2 e 3 sono monotone, in quanto rientrano tra le grammatiche di tipo 1. Il seguente risultato, già preannunciato negli scorsi paragrafi, riguarda allora non solo i linguaggi generati da grammatiche monotone, per le quali viene enunciato, ma anche i linguaggi dipendenti dal contesto, liberi dal contesto e regolari; ribadisce in questi ultimi due casi che i problemi di stabilire se un linguaggio libero dal contesto (in particolare regolare) è o no vuoto, oppure è o no finito, hanno un algoritmo di decisione (si vedano i precedenti Teoremi 5.7.3 e 5.6.3).

Teorema 5.8.2 *Tutti i linguaggi generati da grammatiche monotone sono decidibili.*

Dimostrazione. Sia $G = \langle N, T, \mathcal{P}, S \rangle$ una grammatica monotona. Cerchiamo un algoritmo per decidere, per ogni stringa w su T , se $w \in L(G)$ oppure no. Sia n la lunghezza di w . Consideriamo la seguente successione di insiemi T_i (per $i \in \mathbb{N}$):

- $T_0 = \{S\}$,
- per $i \in \mathbb{N}$, $T_{i+1} = T_i \cup \{\alpha \in (N \cup T)^* \mid \exists \beta \in T_i \text{ tale che } \beta \rightarrow_G \alpha, l(\alpha) \leq n\}$.

Dunque, per ogni $i \in \mathbb{N}$, si costruisce un insieme T_i che estende quelli dei passi precedenti e contiene tutte le stringhe α su $N \cup T$ (in particolare su T) che hanno lunghezza $\leq n$ e sono derivabili da S in G in al più i passi. Si osservi poi che, se $T_i = T_{i+1}$, allora $T_i = T_{i+t}$ per ogni indice $t \geq 0$. D'altra parte, siccome G è monotona, per ogni derivazione $\beta \rightarrow_G \alpha$ deve essere $l(\beta) \leq l(\alpha)$. Questa osservazione, unita al fatto che ogni stringa α di un qualsiasi T_i ha lunghezza $\leq n$, implica che tutte le stringhe usate per derivare α devono avere anch'esse lunghezza $\leq n$. Ne segue che la sequenza di insiemi T_i è limitata superiormente, ovvero esiste un indice m tale che $T_m = T_{m+t}$ per ogni $t \geq 0$. Come limite superiore m possiamo infatti prendere il numero totale delle stringhe in $(N \cup T)^*$ di lunghezza $\leq n$, che è ovviamente finito. L'algoritmo di decisione per l'appartenenza a $L(G)$ di stringhe $w \in T^*$ consiste dunque nel controllare, per ogni $i \leq m$, se $w \in T_i$ e $T_i \neq T_{i+1}$. $\square$

Tuttavia i linguaggi generati da grammatiche monotone (dunque quelli dipendenti dal contesto) non esauriscono quelli decidibili.

Proposizione 5.8.3 *Esistono linguaggi decidibili che non sono generati da grammatiche dipendenti dal contesto.*

Dimostrazione. Usiamo un argomento astratto di diagonalizzazione, che non fornisce esempi espliciti dei linguaggi richiesti, ma ne assicura comunque l'esistenza. Ne accenniamo i punti essenziali. Anzitutto, fissato un alfabeto T , enumeriamo in modo effettivo le grammatiche dipendenti dal contesto che hanno T come insieme di simboli terminali e attingono quelli non terminali da un insieme numerabile prefissato (si procede qui in modo simile a quello seguito per le macchine di Turing). Anche le stringhe di T^* possono essere enumerate effettivamente sulla base di un prefissato ordinamento di T secondo il solito principio lessicografico. Siano

$$G_0, G_1, \dots, G_i, \dots$$

$$w_0, w_1, \dots, w_i, \dots$$

queste enumerazioni (per i che varia tra i naturali). A questo punto formiamo una matrice di dimensioni infinite, in cui le righe corrispondono alle grammatiche dipendenti dal contesto e le colonne alle stringhe su T . Così la riga i etichetta

la grammatica G_i e la colonna i -ma rappresenta la stringa w_i . Si consideri il linguaggio

$$L = \{w_i \in T^* \mid i \in \mathbb{N}, w_i \notin L(G_i)\}.$$

L è decidibile perchè per ogni w_i è possibile anzitutto recuperare effettivamente il corrispondente indice i , determinare conseguentemente la grammatica G_i e verificare finalmente se $w_i \in L(G_i)$ oppure no, sfruttando il fatto che G_i è di tipo 1 e quindi $L(G_i)$ è decidibile. Comunque L non può essere generato da nessuna grammatica dipendente dal contesto. Altrimenti $L = L(G_j)$ per qualche j e di conseguenza

$$w_j \in L \Leftrightarrow w_j \notin L(G_j) \Leftrightarrow w_j \notin L,$$

si ottiene dunque una contraddizione. □

I linguaggi dipendenti dal contesto estendono propriamente quelli liberi dal contesto e quelli regolari, formano in questo senso una classe meno interessante e, del resto, godono minori proprietà. Per esempio si osserva quanto segue.

- Non esiste un algoritmo che sa stabilire se due grammatiche di tipo 1 generano o no lo stesso linguaggio (del resto questa incapacità si rileva già tra le grammatiche di tipo 2, e un procedimento che funzionasse per grammatiche dipendenti dal contesto si applicherebbe anche a quelle libere dal contesto)
- Non esiste un algoritmo che sa stabilire, per ogni linguaggio L di tipo 1, se L è vuoto o no, oppure se è finito o no (invece analoghi procedimenti esistono nel caso libero dal contesto; l'impossibilità di riconoscere $\emptyset$ tra i linguaggi dipendenti dal contesto è provata, per esempio in [29]; entrambi i problemi sono trattati in [54]).

Si verifica poi che i linguaggi di tipo 1 si preservano per le usuali operazioni. Per unione, concatenazione e chiusura di Kleene, la prova non è difficile e riprende quella del caso libero dal contesto. Invece il caso della complementazione, trattato in [58] e [105], non è banale. Comunque si ha:

Teorema 5.8.4 *La classe dei linguaggi dipendenti dal contesto è chiusa per complementazione.*

Siccome allora i linguaggi di tipo 1 si preservano per unione e complementazione, segue immediatamente:

Corollario 5.8.5 *La classe dei linguaggi dipendenti dal contesto è chiusa per intersezione.*

5.9 Linguaggi e Macchine di Turing

In quali casi il linguaggio generato da una grammatica è decidibile, o semidecidibili? Nel precedente paragrafo abbiamo visto che i linguaggi di tipo 1 (e quindi anche quelli di tipo 2 e 3) sono tutti decidibili, ma non esauriscono la classe di tutti

i linguaggi decidibili: ci sono linguaggi decidibili che non sono generati da nessuna grammatica di tipo 1. In questo paragrafo, invece, consideriamo i linguaggi di tipo 0, quelli generati da grammatiche a struttura di frase, e proviamo che essi coincidono esattamente con quelli semidecidibili: forniscono così una ulteriore caratterizzazione della semidecidibilità.

Teorema 5.9.1 *Un linguaggio è di tipo 0 se e solo se è semidecidibile.*

Dimostrazione. Proviamo dapprima che, per ogni grammatica $G = \langle N, T, \mathcal{P}, S \rangle$ di tipo 0, $L(G)$ è semidecidibile. Cerchiamo dunque una MdT M che converga su tutte e sole le stringhe di $L(G)$, e quindi, sostanzialmente, un algoritmo di enumerazione per $L(G)$, come spiegato in § 3.5. Consideriamo allora la sequenza di insiemi T_i (i naturale) definita come segue:

- $T_0 = \{S\}$,
- per $i \in \mathbb{N}$, $T_{i+1} = T_i \cup \{\alpha \in (N \cup T)^* : \exists \beta \in T_i \text{ tale che } \beta \rightarrow_G \alpha\}$.

Allora ogni insieme T_i contiene tutte le stringhe derivabili in G da S in al più i passi e può essere costruito applicando le produzioni di G in tutti i modi possibili alle stringhe degli insiemi che lo precedono nella sequenza. T_i è comunque finito, così come l'insieme delle produzioni di G . Le stringhe dei vari T_i si possono allora enumerare prima in base all'indice i e poi in riferimento alle produzioni che le generano. In particolare le parole del linguaggio $L(G)$ sono quelle stringhe di T^* che sono generate per questa via. Dunque $L(G)$ è semidecidibile.

Dimostriamo adesso che un linguaggio semidecidibile è di tipo 0. Sia L un linguaggio semidecidibile su un alfabeto A . Cerchiamo una grammatica a struttura di frase G_L tale che $L(G_L) = L$. Sia $M = (Q, A, \delta, q_0)$ una MdT che accetta L . La grammatica G_L che adesso definiamo cerca di descrivere e accompagnare le computazioni di M sugli input $w \in A^*$ in modo tale che G_L genera w se e solo se M converge su w . Per questo obiettivo G_L utilizza un alfabeto di simboli terminali $T = A \cup \{\Delta, \nabla, \star\}$ (dove $\star$ è il simbolo bianco di M) e un alfabeto N di simboli non terminali che si compone, oltre che del simbolo iniziale S , di un simbolo $\bar{q}$ per ogni stato $q \in Q$ di M e di tre ulteriori P, q_F, q'_F .

Descriviamo adesso le produzioni di G_L (quelle che simulano il comportamento di M). Ricordiamo allora che, all'inizio di una computazione, M riceve un input di A^* e ne esamina il simbolo più a sinistra nello stato q_0 . Corrispondentemente si pongono in G_L le produzioni

$$\begin{array}{lll} S & \longrightarrow & \Delta \bar{q}_0 P \\ P & \longrightarrow & bP \\ P & \longrightarrow & \nabla \end{array} \quad \forall b \in A$$

che servono a generare ogni possibile configurazione iniziale di M ; in particolare Δ, ∇ hanno funzioni di delimitatori, rispettivamente a sinistra e a destra, $\bar{q}_0$ ricorda lo stato iniziale q_0 e P permette di coinvolgere successivamente ogni elemento di A fino al completamento dell'input, dopo di che P lascia il suo posto a

∇ e scomparire. Passiamo adesso a definire le produzioni di G_L che descrivono il comportamento di M durante i successivi passi delle sue computazioni. Per ogni istruzione di M della forma $\delta(q, a) = (q', a', +1)$ con $q, q' \in Q, a, a' \in A \cup \{\star\}$, aggiungiamo le produzioni

$$\begin{aligned}\bar{q}ab &\longrightarrow a'\bar{q}'b & \forall b \in A \cup \{\star\} \\ \bar{q}a\nabla &\longrightarrow a'\bar{q}'\star\nabla.\end{aligned}$$

Invece ad ogni istruzione $\delta(q, a) = (q', a', -1)$ associamo le produzioni

$$\begin{aligned}b\bar{q}a &\longrightarrow \bar{q}'ba' & \forall b \in A \cup \{\star\} \\ \Delta\bar{q}a &\longrightarrow \Delta\bar{q}'\star a' .\end{aligned}$$

Finalmente, per ogni coppia (q, a) fuori del dominio della funzione di transizione δ , aggiungiamo la produzione

$$\bar{q}a \longrightarrow q_F a .$$

La comparsa di q_F in una stringa generata da G_L ha dunque il ruolo di segnalare che M sta convergendo (l'indice F sta qui a ricordare proprio la fermata di M). Aggiungiamo infine delle produzioni che, non appena M , appunto, si arresta, cancellano i due delimitatori Δ e ∇ e q_F e restituiscono l'esatto output di M :

$$\begin{aligned}bq_F &\longrightarrow q_F b \\ \Delta q_F &\longrightarrow q'_F \\ q'_F b &\longrightarrow bq'_F \\ q'_F \nabla &\longrightarrow \lambda\end{aligned}$$

dove b varia in $A \cup \{\star\}$. In base a queste produzioni, q_F si sposta a sinistra lungo l'output e, raggiunto il delimitatore Δ , lo cancella e lascia campo a q'_F che svolge lo stesso ruolo a destra, mantenendo ancora invariato l'output ed eliminando alla fine tanto il delimitatore destro ∇ quanto se stesso.

Si vede facilmente che G_L è una grammatica di tipo 0. Con un po' di pazienza, si verifica anche che, per ogni stringa $w \in A^*$, M converge su w (e cioè $w \in L$) se solo se $S \xrightarrow{*}_{G_L} w$ (ovvero G_L genera w). $\square$

5.10 Automi di Riconoscimento

Nei precedenti paragrafi abbiamo discusso la possibilità di “generare” un linguaggio come l'insieme delle stringhe derivabili da una *grammatica a struttura di frase*. Abbiamo anche collegato le proprietà di decidibilità di un linguaggio alla espressività delle grammatiche che lo generano e alla forma delle loro produzioni. Ma in questo ambito anche gli automi giocano un ruolo di fondamentale importanza e, in particolare, possono essere adoperati per determinare i linguaggi L in modo alternativo alle grammatiche, volto non a generare, ma a riconoscere e accettare le stringhe di L . Un automa, infatti, legge uno alla volta i simboli di ogni stringa che gli viene proposta e, alla fine dell'esame, “certifica” se la stringa appartiene o

no al linguaggio interessato.

In realtà, si possono formare varie classi di automi, anzi ogni classe di grammatiche nella gerarchia di Chomsky si può collegare a una corrispondente classe di automi in modo tale che differenti poteri espressivi delle classi di grammatiche si rispecchino in differenti poteri riconoscitivi delle relative classi di automi. Del resto abbiamo già visto nello scorso paragrafo che alla classe di tutte le grammatiche a struttura di frase corrisponde quella delle Macchine di Turing, nel senso che un linguaggio è accettato da una MdT se e solo se è generato da una grammatica di tipo 0. Allo stesso modo si vede che:

- alla classe delle grammatiche regolari corrisponde la classe dei così detti *automi a stati finiti*,
- alla classe delle grammatiche libere dal contesto corrisponde la classe degli *automi a pila*,
- alla classe delle grammatiche dipendenti dal contesto corrisponde la classe degli *automi linearmente limitati*.

Diamo adesso qualche breve cenno su queste tre classi di automi e sulle diverse formalità che esse prevedono per l'operazione di riconoscimento delle stringhe di un linguaggio.

Da un punto di vista esteriore, ogni automa, così come ogni MdT, si compone di un dispositivo di controllo con un numero finito di stati, di una testina di lettura e scrittura e di un nastro suddiviso in celle. La stringa in input viene scritta su questo nastro da sinistra verso destra, un carattere per ogni cella. I quadri a sinistra e a destra dell'input sono riempiti dal simbolo bianco $\star$. All'inizio del riconoscimento, la testina di lettura e scrittura esamina il primo simbolo dell'input e il controllo è nello stato iniziale. A questo punto gli automi si differenziano in ragione delle modalità previste per il loro funzionamento.

Per esempio, in un *automa a stati finiti* la stringa in input viene scorsa da sinistra verso destra, un simbolo alla volta: a ogni passo l'automa legge il carattere sul nastro, cambia il suo stato secondo quanto gli ordina una fissata funzione di transizione e sposta finalmente la sua testina di un passo verso destra. Esaminata tutta la stringa in ingresso, l'automa si arresta, ed è proprio lo stato con cui si ferma a stabilire se l'input appartiene o no al linguaggio. Un automa a stati finiti si può dunque pensare come una MdT con capacità ridotte: licenza di leggere ma non di scrivere, permesso di spostarsi solo a destra. Come detto, si mostra che la classe degli automi a stati finiti corrisponde alla classe dei linguaggi di tipo 3: vale a dire, un linguaggio è riconosciuto da un qualche automa a stati finiti se e solo se è regolare.

La limitazione delle capacità di riconoscimento degli automi a stati finiti si riconduce al fatto che essi dispongono soltanto di una memoria finita (quella che è loro assicurata dai loro stati). Per questo motivo essi non sono in grado di riconoscere linguaggi che richiedono invece di ricordare una quantità di informazione illimitata. Si pensi per esempio al linguaggio

$$L = \{a^i b^i : i \in \mathbb{N}, i \geq 1\}$$

già considerato in § 5.6 e all'inizio di § 5.7. Per riconoscere una stringa $a^i b^i$ bisogna ricordare il numero i degli a letti prima di incontrare il primo b , in modo da poter poi controllare che i b che completano la stringa siano altrettanti. Nessun automa a stati finiti ha questa potenzialità di memoria (e del resto L non è regolare). Per migliorare le capacità di riconoscimento degli automi a stati finiti, possiamo pensare a nuovi modelli che dispongano di una memoria ausiliare illimitata e svolgano ogni mossa della loro computazione in dipendenza non solo del carattere letto e dello stato del dispositivo di controllo, ma anche della informazione precedentemente accumulata nella memoria. Un *automa a pila* corrisponde a questi requisiti: ha il suo dispositivo di controllo a stati finiti, la sua testina di lettura e scrittura, il suo nastro e, in più, una memoria ausiliaria a pila di lunghezza infinita cui però può accedere solo dalla testa. Si mostra allora che la classe degli automi a pila corrisponde a quella dei linguaggi di tipo 2: un linguaggio è riconosciuto da un automa a pila (non deterministico) se e solo se è libero dal contesto. Così la disponibilità di una memoria ausiliaria potenzialmente illimitata non è sufficiente a riconoscere ogni linguaggio a struttura di frase: ci sono infatti linguaggi dipendenti dal contesto che nessun automa a pila sa riconoscere. In effetti, quello che è essenziale in questo ambito più generale non è tanto una memoria ausiliaria illimitata, quanto la libertà di poterne disporre senza eccessivi vincoli. Nel caso dei linguaggi dipendenti dal contesto, allora, basta contare su di una memoria linearmente limitata dalla lunghezza dell'input e accessibile senza restrizioni. L'automa che ne risulta viene chiamato *automa linearmente limitato*. La classe degli automi linearmente limitati corrisponde esattamente alla classe dei linguaggi di tipo 1.

5.11 Esercizi

1. Si definiscano, se possibile, grammatiche regolari che generano:
 - (a) le stringhe di lunghezza arbitraria che iniziano con un simbolo assegnato,
 - (b) le stringhe di lunghezza al più 6 che iniziano con un simbolo prefissato.
2. A quale classe nella gerarchia di Chomsky appartiene il linguaggio delle stringhe su $\{a, b, c\}$ che contengono un numero dispari di a , un numero dispari di b e un numero pari di c ?
3. A quale classe nella gerarchia di Chomsky appartengono i seguenti linguaggi?
 - $L_1 = \{a^n a^n \mid n \in \mathbb{N}, n > 0\}$,
 - $L_2 = \{a^n b a^n \mid n \in \mathbb{N}\}$,
 - $L_3 = \{a^n b a^n c a^n \mid n \in \mathbb{N}\}$.
4. Il linguaggio $L = \{a^i b^{i^2} \mid i \in \mathbb{N}, i > 0\}$ è o no libero dal contesto?
5. Sia $G = \langle N, T, \mathcal{P}, S \rangle$ una grammatica libera dal contesto tale che ogni produzione in $\mathcal{P}$ è di una delle forme $A \rightarrow wB$ o $A \rightarrow Bw$ o $A \rightarrow w$, dove $w \in T^*$ e $A, B \in N$. $L(G)$ è regolare?
6. Si provi che ogni grammatica dipendente dal contesto è monotona, e vicever-

sa si dia un esempio di una grammatica monotona che non è dipendente dal contesto.

7. Sia K , al solito, l'insieme dei numeri naturali x tali che la x -ma MdT M_x converge su x . K è un linguaggio di tipo 0? È di tipo 1, o 2, o 3? E che si può dire del suo complementare $\mathbb{N} - K$?

Riferimenti bibliografici Un ottimo riferimento generale per approfondire tutti gli argomenti di questo capitolo è [55], che espone con chiarezza, semplicità e completezza la teoria dei linguaggi formali e degli automi e discute molti problemi di decidibilità per classi di linguaggi e automi. Per una trattazione estesa sulla teoria degli automi si considerino [8] e [54]. Per le grammatiche a struttura di frase si consultino anche [13] e [14].

L'indecidibilità del problema dell'equivalenza per grammatiche libere dal contesto è trattata in [76], quella del problema di riconoscere linguaggi vuoti tra quelli dipendenti dal contesto è in [29]. Il problema della complementazione dei linguaggi dipendenti dal contesto è risolto in [58] e [105]. I fondamenti della teoria degli alberi si possono trovare per esempio in [40].

Segnaliamo anche [81], dove vengono studiate le varie classi della Gerarchia di Chomsky, le loro relazioni con gli automi e le tecniche di parsing dei linguaggi: buona parte del libro è poi dedicata alla descrizione formale dei linguaggi naturali. [81] dimostra anche la coincidenza tra i linguaggi dipendenti dal contesto e quelli generati da grammatiche monotone.

Per le applicazioni della teoria dei linguaggi formali alla definizione sintattica e alla compilazione dei linguaggi di programmazione si consiglia finalmente la lettura dei testi [4, 6].

Per approfondimenti sull'uso degli automi a stati finiti come modelli di sistemi concorrenti e distribuiti si faccia riferimento a [79].

Calcolabilità e Linguaggi di Programmazione

6.1 Il linguaggio WHILE

In questo capitolo presenteremo un nuovo possibile modo di accostare la computabilità: stavolta, infatti, la introdurremo attraverso i moderni linguaggi di programmazione. In effetti, la ricerca di linguaggi *ad alto livello*, capaci cioè di esprimere tutte le funzioni effettivamente computabili, risale già agli anni sessanta e ha prodotto una vasta famiglia di linguaggi come ALGOL e PASCAL, progenitori dei più moderni C, C++ e Java.

Noi però concentreremo qui la nostra attenzione su un semplice linguaggio di programmazione imperativo chiamato WHILE. Si tratta di un linguaggio adeguato a rappresentare tutte le funzioni calcolabili, povero in confronto ad altri linguaggi di alto livello come il PASCAL o il FORTRAN, ma, proprio in virtù della sua semplicità descrittiva, capace di sottolineare in maniera più diretta e immediata il collegamento con la computabilità.

6.2 La Sintassi del Linguaggio WHILE

L'insieme dei caratteri di base del linguaggio WHILE contiene nomi per le variabili, simboli per gli operatori di base che modificano il valore delle variabili, un operatore per confrontare valori di coppie di variabili e un simbolo di programma. La sintassi del linguaggio WHILE è specificata in accordo alle regole di produzione di una grammatica libera dal contesto (presentate nel Capitolo 5) ed è definita come segue.

```

PROGRAM      ::= begin end | begin SEQCOMMANDS end
SEQCOMMANDS ::= COMMAND | SEQCOMMANDS ; COMMAND
COMMAND      ::= ASSIGNMENT | while TEST do COMMAND | PROGRAM
ASSIGNMENT   ::= VAR := 0 | VAR := s (VAR) | VAR := pd (VAR)
TEST         ::= VAR ≠ VAR
VAR          ::= LETTERA | VAR LETTERA | VAR CIFRA
LETTERA      ::= A | B | ... | Z
CIFRA       ::= 0 | 1 | ... | 9

```

Si noti che le variabili sono formate da stringhe finite arbitrarie di lettere maiuscole $A, B, \dots, Z$ e di cifre decimali $0, 1, \dots, 9$ che hanno l'unico obbligo di iniziare con una lettera maiuscola come accade, per esempio, a X e A ma anche a $PIPP01$, $CASA3$ e $P55$.

Definizione. Un programma WHILE è una sequenza ordinata finita (eventualmente vuota) di *comandi*, separati da punti e virgole, preceduta da *begin* e seguita da *end*. I comandi sono definiti induttivamente nel modo che adesso mostriamo.

- Al livello più elementare abbiamo i *comandi di assegnamento*, quelli della forma

```

(Zero): X := 0
(s):    X := s(Y)
(pd):   X := pd(Y)

```

dove 0 è il numero naturale zero, s è la funzione successore ($s(n) = n + 1$ per ogni naturale n) e pd è la funzione predecessore ($pd(n) = n - 1$ se $n > 0$ e $pd(0) = 0$). 0 , s e pd costituiscono gli operatori base del linguaggio. Si noti che nei programmi WHILE i valori assegnati alle variabili saranno sempre numeri naturali.

Si definiscono poi per induzione gli altri comandi, suddivisi tra *comandi composti* e *comandi while*.

- Un *comando composto* è un'istruzione della forma

$$\text{begin } C1; \dots; CN \text{ end}$$

con N naturale e indica che le istruzioni $C1, \dots, CN$ devono essere eseguite in successione, secondo l'ordine assegnato. Ammettiamo anche il caso $N = 0$,

che si riduce sostanzialmente ad un'istruzione che lascia inalterato il valore di tutte le variabili.

- Finalmente un comando `while` ha la forma

$$\text{while } X \neq Y \text{ do COMMAND;}$$

la sua semantica consiste nel ripetere l'esecuzione del comando descritto in `COMMAND` per tutto il tempo in cui la variabile `X` assume un valore diverso dal valore della variabile `Y`.

In particolare, l'unico operatore di confronto tra variabili in un linguaggio `WHILE` è, almeno inizialmente, $\neq$.

Quindi il linguaggio `WHILE` mette a disposizione del programmatore solo pochissimi operatori e comandi. Tuttavia mostriamo adesso come definire varie macroistruzioni che possono agevolare la stesura di un programma; cerchiamo così di convincere il lettore che, proprio grazie a queste macro, il potere espressivo dei programmi `WHILE` non è inferiore a quello di altri usuali linguaggi di programmazione, almeno per quanto concerne l'aspetto computazionale. In particolare i linguaggi `WHILE` riescono a rappresentare tutte le familiari operazioni aritmetiche.

La prima macro che consideriamo riguarda il problema di assegnare il valore di una variabile `Y` a un'altra variabile `X`. Il nostro linguaggio di programmazione non ha, al momento, alcun comando primitivo a questo proposito. E' facile tuttavia convincersi che il programma:

```
begin  X := s(Y) ; X := pd(X)  end
```

permette il passaggio del valore della variabile `Y` alla variabile `X`. Nel seguito, quindi, potremo usare il comando `X := Y` per denotare sinteticamente la precedente macrodefinizione.

Un'altra operazione usuale in programmazione è quella di assegnare il valore di una costante $n > 0$ ad una variabile `U`, `U := n`. La macro che la effettua consiste nella seguente lista di comandi elementari:

```
begin  U := 0 ; U := s(U); ... ; U := s(U)  end
```

dove il comando `U := s(U)` è ripetuto n volte.

Se si vuole sommare il valore della variabile `Y` alla variabile `X`, `X := X + Y`, si può invece usare la macro:

```
begin
  U := 0 ;
  while U ≠ Y do
    begin  X := s(X); U := s(U)  end
end
```

Usando le macro già definite si può poi scrivere il programma:

```
begin
  Z := X ;
  Z := Z + X
end
```

che corrisponde alla istruzione $Z := X + Y$, e dunque alla addizione. Anche l'altra familiare operazione numerica della moltiplicazione si esprime nel linguaggio WHILE tramite un'opportuna macro (che il lettore può provare a definire da solo per esercizio).

Le funzioni descrivibili in WHILE tramite opportune macro includono anche la differenza, la differenza aritmetica, il quoziente, il resto e così via. Consideriamo per esempio il caso della *differenza aritmetica* $Z := X \ominus Y$ di X e Y ; ricordiamo che si conviene che $X \ominus Y$ è $X - Y$ se $X \geq Y$, 0 altrimenti. La macro che la esprime (e del resto corrisponde alla caratterizzazione di $\ominus$ data in § 4.3) è la seguente:

```
begin
  Z:=X ; U:=Y; V:=0
  while U ≠ V do begin  Z:=pd(Z); U:=pd(U)  end
end
```

Di uso frequente in programmazione sono anche gli operatori di confronto. Tuttavia l'unico di essi disponibile almeno inizialmente nel linguaggio WHILE è $X \neq Y$. Ma è spesso necessario considerare anche test del tipo: $X < Y$, $X > 5$, $X \nless Y$, $X = Y$ e via dicendo. Il linguaggio WHILE garantisce queste opportunità se non direttamente almeno tramite macro. Per esempio $X < Y$ può essere coinvolto mediante $(Y \ominus X) \ominus pd(Y \ominus X)$, nel senso che $X < Y$ è vera se e soltanto se $(Y \ominus X) \ominus pd(Y \ominus X) = 1$, mentre $X < Y$ è falsa se e soltanto se $(Y \ominus X) \ominus pd(Y \ominus X) = 0$.

Anzi, questo esempio ci mostra come coinvolgere nei linguaggi WHILE gli usuali connettivi della logica elementare $\neg$ (*non*), $\wedge$ (*e*), $\vee$ (*o*) e quindi esprimere condizioni come quella appena considerata $\neg(X < Y)$, oppure $(X < Y) \vee (X = Y)$ (cioè $X \leq Y$), $(X \neq Y) \wedge \neg(X < Y)$ (cioè $X > Y$) e altre più elaborate come $(\neg(X < Y) \wedge (X = Y)) \vee (X \neq Y)$, in definitiva tutte quelle condizioni che possono ottenersi come *espressioni Booleane* di altre che già conosciamo. Così facendo, $X = Y$ può essere ottenuta come semplice espressione Booleana per $\neg(X \neq Y)$. Basta associare a ciascuna di queste espressioni $\mathcal{E}$ una condizione di test $T_{\mathcal{E}}$ tra variabili in modo tale che $T_{\mathcal{E}}$ ha 0 e 1 come unici valori ammissibili e

- $T_{\mathcal{E}} = 1$ se e soltanto se $\mathcal{E}$ è vera,

e quindi

- $T_{\mathcal{E}} = 0$ se e soltanto se $\mathcal{E}$ è falsa.

Le regole per costruire questi termini sono facili da stabilire; per esempio, usando l'operatore pd e le macro $+$ e $\ominus$, possiamo scriverle come segue.

- $T_{\neg \mathcal{E}} = 1 \ominus T_{\mathcal{E}}$ (dunque 1 se $T_{\mathcal{E}} = 0$ e viceversa),
- $T_{\mathcal{E} \wedge \mathcal{E}'} = pd(T_{\mathcal{E}} + T_{\mathcal{E}'})$ (dunque 1 se $T_{\mathcal{E}} = T_{\mathcal{E}'} = 1$, 0 altrimenti),
- $T_{\mathcal{E} \vee \mathcal{E}'} = T_{\mathcal{E}} + T_{\mathcal{E}'} \ominus pd(T_{\mathcal{E}} + T_{\mathcal{E}'})$ (dunque 1 se $T_{\mathcal{E}} = 1$ o $T_{\mathcal{E}'} = 1$, 0 altrimenti).

Il lettore può provare a usare anche la moltiplicazione per ottenere espressioni più semplici per $\wedge$ e $\vee$, ricordando situazioni analoghe nel Capitolo 4.

In modo simile possiamo catturare nel nostro linguaggio estensioni del comando `while` come

`while EXPRBOOL do COMMAND`

dove `EXPRBOOL` è una generica espressione Booleana $\mathcal{E}$, e non soltanto $X \neq Y$. Per costruire la macroistruzione che corrisponde a questo comando, assumiamo per semplicità che in `EXPRBOOL` compaiano solo variabili e non costanti (del resto espressioni Booleane con costanti si possono ottenere da quelle che hanno solo variabili assegnando le costanti alle variabili nel modo sopra descritto). Allora la macro che ci serve è:

```
begin
  U:= T_EXPRBOOL
  V:=0
  while U ≠ V do begin  COMMAND ; U:= T_EXPRBOOL  end
end
```

dove T_{EXPRBOOL} indica, come sappiamo, la condizione di test associata all'espressione Booleana `EXPRBOOL`.

Un'altra istruzione non direttamente disponibile nel linguaggio dei programmi `WHILE`, ma assai comune in altri linguaggi di programmazione, è il comando

`if EXPRBOOL then COMMAND1 else COMMAND2`

che fa eseguire la sequenza di istruzioni che segue `then`, e cioè `COMMAND1`, se l'espressione Booleana `EXPRBOOL` è vera, e la sequenza di comandi dopo `else`, e cioè `COMMAND2`, se l'espressione Booleana `EXPRBOOL` è falsa. La macro per questo operatore condizionale è la seguente:

```
begin
  U:= T_EXPRBOOL
  V:=0
  while (U = 1) ∧ (V = 0) do begin  COMMAND1 ; V:= 1  end
  while (U = 0) ∧ (V = 0) do begin  COMMAND2 ; V:= 1  end
end
```

Con l'utilizzo di queste macroistruzioni, il linguaggio WHILE raggiunge sostanzialmente l'espressività del PASCAL. Le macro sono quindi di grande aiuto nella stesura di programmi per computazioni complesse; ma quando il nostro obiettivo si sposta a dimostrare proprietà astratte dei programmi, allora la semplicità dell'originario contesto WHILE torna utile per abbreviare e snellire la trattazione.

6.3 La Semantica dei linguaggi WHILE

La sintassi dei linguaggi WHILE ha già insinuato il loro collegamento con le funzioni dell'aritmetica. Ma per apprezzare compiutamente come i programmi WHILE possano computare queste funzioni, abbiamo bisogno di conoscere anche la *semantica* WHILE: un insieme di convenzioni che ci consentiranno, appunto, di associare una funzione parziale ad ogni programma del linguaggio, di formare conseguentemente la classe delle funzioni che sono calcolabili con programmi WHILE e di confrontare finalmente questa classe con le altre che già conosciamo e derivano da approcci alternativi alla computabilità.

Tutti questi sviluppi saranno il tema dei futuri paragrafi. L'obiettivo di quello presente è semplicemente quello di descrivere la semantica WHILE. Cominciamo col fissare con maggiore precisione il nostro contesto.

Consideriamo programmi WHILE P a k variabili, cioè programmi abilitati a utilizzare k variabili $x_1, \dots, x_k$, o eventualmente un loro sottoinsieme. Conveniamo poi che le variabili $x_1, \dots, x_k$ siano ordinate in base ai loro indici $i \in \{1, \dots, k\}$. Chiamiamo *stato di computazione* di P una qualunque k -upla $a = (a_1, \dots, a_k)$ di naturali: si intende allora che in P x_1 assume il valore a_1 , $\dots$, x_k assume il valore a_k . Uno stato di computazione è anche detto *vettore di stato*.

Le definizioni che seguono permettono di introdurre il concetto di *computazione* di un programma WHILE P a k variabili. Useremo le lettere C, C' e così via per denotare termini della categoria sintattica COMMAND (cioè singoli comandi per P) e $C, C', \dots$ per indicare termini della categoria sintattica SEQCOMMAND (cioè sequenze di comandi per P). Si ricordi che P stesso può essere visto come una sequenza di comandi.

Una coppia ordinata $\langle C, a \rangle$ dove C è una sequenza di comandi di P e a è un vettore di stato di P è chiamata *configurazione* di P . La sequenza C descrive, intuitivamente, quali comandi sono da svolgere a un certo punto della sua esecuzione mentre il vettore di stato a precisa il valore delle variabili a cui C si applica. La *relazione di transizione* $\longrightarrow$ tra queste coppie $\langle C, a \rangle$ (sequenza di comandi, vettore di stato), rappresentata in Fig. 6.1, mostra come vengono eseguiti i singoli comandi di un programma e come vengono modificate le variabili del programma stesso. La terminazione di una sequenza di comandi viene denotata con *end*.

$$\begin{array}{c}
 \text{SC}_{\text{zero}} \frac{a = (a_1, \dots, a_k) \text{ e } a' = (a_1, \dots, a_{u-1}, 0, a_{u+1}, \dots, a_k)}{\langle Xu := 0, a \rangle \longrightarrow \langle \text{end}, a' \rangle} \\
 \\
 \text{SC}_s \frac{a = (a_1, \dots, a_k) \text{ e } a' = (a_1, \dots, a_{u-1}, s(a_v), a_{u+1}, \dots, a_k)}{\langle Xu := s(Xv), a \rangle \longrightarrow \langle \text{end}, a' \rangle} \\
 \\
 \text{SC}_{\text{pd}} \frac{a = (a_1, \dots, a_k) \text{ e } a' = (a_1, \dots, a_{u-1}, \text{pd}(a_v), a_{u+1}, \dots, a_k)}{\langle Xu := \text{pd}(Xv), a \rangle \longrightarrow \langle \text{end}, a' \rangle} \\
 \\
 \text{SC}_{\text{while}_1} \frac{a = (a_1, \dots, a_k) \text{ e } a_u \neq a_v}{\langle \text{while } Xu \neq Xv \text{ do } C, a \rangle \longrightarrow \langle C; \text{while } Xu \neq Xv \text{ do } C, a \rangle} \\
 \\
 \text{SC}_{\text{while}_2} \frac{a = (a_1, \dots, a_k) \text{ e } a_u = a_v}{\langle \text{while } Xu \neq Xv \text{ do } C, a \rangle \longrightarrow \langle \text{end}, a \rangle} \\
 \\
 \text{SC}_{\text{program}_1} \frac{}{\langle \text{begin end}, a \rangle \longrightarrow \langle \text{end}, a \rangle} \\
 \\
 \text{SC}_{\text{program}_2} \frac{}{\langle \text{begin } C \text{ end}, a \rangle \longrightarrow \langle C, a \rangle} \\
 \\
 \text{SC}_{\text{seq}_1} \frac{\langle C_1, a \rangle \longrightarrow \langle C'_1, a' \rangle \text{ e } C'_1 \neq \text{end}}{\langle C_1; C_2, a \rangle \longrightarrow \langle C'_1; C_2, a' \rangle} \\
 \\
 \text{SC}_{\text{seq}_2} \frac{\langle C_1, a \rangle \longrightarrow \langle \text{end}, a' \rangle}{\langle C_1; C_2, a \rangle \longrightarrow \langle C_2, a' \rangle}
 \end{array}$$

Figura 6.1 Semantica Operazionale dei Programmi WHILE

Le regole SC_{zero} , SC_s e SC_{pd} descrivono come vengono modificati i valori del vettore di stato quando un comando di assegnamento viene eseguito. Le regole $\text{SC}_{\text{while}_1}$ e $\text{SC}_{\text{while}_2}$ gestiscono invece i comandi di iterazione `while`. In particolare, la regola $\text{SC}_{\text{while}_1}$ mostra come permettere l'esecuzione del corpo del `while` nel caso in cui il test è soddisfatto; la regola $\text{SC}_{\text{while}_2}$ ammette, invece, la terminazione del comando di iterazione quando il test non è soddisfatto. Le regole $\text{SC}_{\text{program}_1}$ e $\text{SC}_{\text{program}_2}$ gestiscono comandi composti (sequenze di comandi tra un `begin` e un `end`). In questo caso, naturalmente, si procede alla valutazione del loro corpo. Infine, le regole SC_{seq_1} e SC_{seq_2} trattano sequenze di comandi e, in particolare, implementano la sequenzializzazione dei comandi specificati.

Definizione. Siano P un programma WHILE a k variabili e $a(0) = (a_1, \dots, a_k)$ un vettore di stato. Una computazione di P è una sequenza, possibilmente infinita,

della forma

$$\langle P, a(0) \rangle \longrightarrow \langle C_1, a(1) \rangle \longrightarrow \langle C_2, a(2) \rangle \longrightarrow \dots$$

dove per ogni $i \geq 1$ $a(i)$ è un vettore stato a k dimensioni e C_i una sequenza di comandi (che corrisponde alla situazione del programma P al passo i -esimo della computazione). Se la sequenza di computazione è finita allora ha la forma

$$\langle P, a(0) \rangle \longrightarrow \langle C_1, a(1) \rangle \longrightarrow \langle C_2, a(2) \rangle \longrightarrow \dots \longrightarrow \langle \text{end}, a(n) \rangle$$

La proposizione che segue è diretta conseguenza della regole operazionali della relazione di transizione $\longrightarrow$ definita nella Fig. 6.1.

Proposizione 6.3.1 *Siano P un programma WHILE a k variabili e $a = (a_1, \dots, a_k)$ un vettore di stato. Allora esiste esattamente una computazione da P che ammette a come vettore di stato iniziale.*

In questo caso a prende il nome di vettore di input. Si conviene poi quanto segue:

- Se la computazione di P su a è infinita, si dice che P *diverge* sull'input a e che il suo output è indefinito.
- Se invece la computazione è finita e della forma

$$\langle P, a(0) \rangle \longrightarrow \langle C_1, a(1) \rangle \longrightarrow \langle C_2, a(2) \rangle \longrightarrow \dots \longrightarrow \langle \text{end}, a(n) \rangle$$

(ovviamente con $a = a(0)$), si dice che P *converge* o *termina* sull'input a e che il suo output è $a(n)$.

6.4 Funzioni WHILE-Calcolabili

Il precedente paragrafo ci ha mostrato come la computazione di un programma WHILE P su un prefissato input è unica, e ci consente quindi di parlare di *funzione calcolata* da P . Sia dunque P un programma a k variabili $x_1, \dots, x_k$ per qualche $k \geq 1$. Si dice *funzione calcolata* da P la funzione parziale f_P da $\mathbb{N}^k$ a $\mathbb{N}$ tale che, per ogni $a \in \mathbb{N}^k$,

- $f_P(a)$ è il valore di x_1 nell'output della computazione di P su a se questa computazione ha fine,
- $f_P(a)$ non è definita altrimenti.

In realtà possiamo convenire che un programma P a k variabili calcoli anche funzioni parziali h -arie per h diverso da k : basta che concordiamo che, per ogni h -upla $a = (a_1, \dots, a_h)$ di naturali, P consideri come input

- per $h > k$, la k -upla troncata $(a_1, \dots, a_k)$ (dunque la sequenza delle prime k componenti di a),

- per $h < k$, la k -upla $(a_1, \dots, a_h, 0, \dots, 0)$ (quella che aggiunge $k - h$ 0 finali ad a)

e poi proceda come appena descritto.

Definizione. Una funzione parziale f da $\mathbb{N}^k$ a $\mathbb{N}$ si dice **WHILE-calcolabile** se f è la funzione calcolata da qualche programma WHILE P .

Dunque la definizione di una funzione WHILE-calcolabile f non richiede espressamente che P abbia esattamente tante variabili quante sono le variabili indipendenti di f .

Esempio. Si vede facilmente che le funzioni ricorsive iniziali Z (zero) e s (successore) sono WHILE-calcolabili perchè la sintassi WHILE prevede espressamente operatori base che le riguardano. Allo stesso modo, ci sono comandi di assegnamento o macro per le usuali operazioni aritmetiche $+$ e $\cdot$, come anche per le funzioni pd , $\ominus$ e via dicendo. Tutte queste funzioni sono quindi WHILE-calcolabili. Il lettore appassionato di programmazione non troverà difficoltà a scrivere programmi WHILE che le calcolano esplicitamente. Anche una proiezione $(a_1, \dots, a_k) \rightarrow a_1$ (per ogni scelta di $(a_1, \dots, a_k) \in \mathbb{N}^k$) è WHILE-calcolabile. Infatti non è difficile convincersi che ogni programma WHILE P del tipo `begin end`, `begin begin end end` e così via è capace di computarla: per ogni vettore input $a = (a_1, \dots, a_k)$, la computazione da $\langle P, a \rangle$ ha termine e la sua configurazione finale è della forma $\langle \text{end}, a \rangle$, non modifica dunque il vettore di stato e assegna alla variabile X_1 il valore a_1 .

6.5 Programmi e Macchine di Turing

In questo paragrafo dimostriamo che le funzioni WHILE-calcolabili coincidono esattamente con quelle calcolabili nel senso di Turing. Dal punto di vista della computabilità, questo risultato fornisce un ulteriore sostegno alla Tesi di Church-Turing e conferma la generalità del modello di Turing. Dal punto di vista della programmazione, invece, ci assicura che le MdT, pur costituendo un formalismo di calcolo a bassissimo livello, formano tuttavia un sistema di programmazione accettabile, visto che la loro espressività è la stessa del linguaggio WHILE.

Teorema 6.5.1 *Una funzione parziale f da $\mathbb{N}^k$ a $\mathbb{N}$ è WHILE-calcolabile se e solo è calcolabile (secondo Turing).*

Dimostrazione. Dimostriamo dapprima che la WHILE-calcolabilità di una funzione f ne implica la computabilità nel senso di Turing. Ricordiamo che un programma WHILE P che calcola f consiste in una sequenza di comandi WHILE. Ci basta allora mostrare che ogni comando di questo genere può essere eseguito da un'opportuna MdT. Infatti, se questo è vero, possiamo anche costruire una MdT che simula l'intero programma P , combinando opportunamente e successivamente le MdT che eseguono i suoi comandi. Ripercorriamo allora la definizione

induttiva dei comandi dei programmi WHILE.

Iniziamo con i *comandi di assegnamento*, quelli che coinvolgono gli operatori 0, s e pd : a loro proposito, già sappiamo dal Capitolo 4 che ci sono MdT che computano le corrispondenti funzioni Z , s e pd .

I passi induttivi della definizione di comando riguardano *comandi composti* come $\text{begin } C_1; \dots; C_N \text{ end}$ oppure *comandi while* della forma $\text{while } x_i \neq x_j \text{ do } C'$.

Concentriamo prima l'attenzione su $\text{begin } C_1; \dots; C_N \text{ end}$ supponiamo che, per ogni $i = 1, \dots, n$, esista una MdT M_i capace di calcolare il programma WHILE C_i . Possiamo assumere che gli insiemi di stati delle macchine $M_1, \dots, M_n$ siano a due a due disgiunti, altrimenti si procede a una ridenominazione del nome degli stati. Allora una MdT M che esegue il programma $\text{begin } C_1; \dots; C_N \text{ end}$ è quella che coinvolge in serie $M_1, M_2, \dots, M_n$, ovvero comincia la sua computazione simulando M_1 a partire dalla sua configurazione iniziale, poi propone la configurazione finale di M_1 come configurazione iniziale di M_2 , e via di seguito.

Consideriamo ora il caso $\text{while } x_i \neq x_j \text{ do } C'$. Per induzione, possiamo supporre che il programma C' sia eseguito da una MdT M' . Sia poi k il numero delle variabili di questo programma. Costruiamo preliminarmente una MdT ausiliaria $M''_{(i,j)}$ che, in ogni vettore di $\mathbb{N}^k$, controlla se l' i -ma componente coincide con la j -ma. Con l'aiuto di M' e $M''_{(i,j)}$ è allora possibile definire una MdT M che esegue il ciclo iterativo richiesto: ogni volta la macchina M controlla il test $x_i \neq x_j$ mediante $M''_{(i,j)}$ e, quando l'esito del test è positivo (e solo in questo caso), simula M' ; altrimenti M termina la sua esecuzione.

Questo conclude la dimostrazione che la WHILE-calcolabilità implica la computabilità (nel senso di Turing). Viceversa, dobbiamo provare che ogni funzione parziale calcolabile secondo Turing è anche WHILE-calcolabile. Ma siccome le funzioni parziali calcolabili secondo Turing coincidono con quelle parziali ricorsive, ci basta mostrare che, se f è una funzione parziale ricorsiva, e cioè si ottiene dalle funzioni ricorsive iniziali (Z , s e proiezioni) applicando un numero finito di volte i procedimenti di composizione, recursione e minimalizzazione, allora c'è un programma WHILE che calcola f . Si procede ancora per induzione, stavolta sulla costruzione di f . Abbiamo già visto negli esempi finali di § 6.4 che le funzioni iniziali sono tutte WHILE-calcolabili. Resta da vedere che la WHILE-calcolabilità si preserva per composizione, recursione e minimalizzazione. Il lettore appassionato di programmazione può provare a verificarlo in dettaglio, usando le definizioni di questi tre procedimenti. $\square$

6.6 Il Teorema di Kleene per i Programmi

Nel Capitolo 3, abbiamo presentato, discusso e dimostrato il Teorema di Kleene nell'ambito delle MdT. Il risultato si basa essenzialmente sulla possibilità di enumerare effettivamente le MdT su un fissato alfabeto A e anzi di costruire una biiezione effettiva tra MdT su A e numeri naturali. Ma anche i programmi WHILE

ammettono una analoga enumerazione effettiva (il lettore può cimentarsi a trovarla, partendo da una codifica effettiva delle istruzioni elementari su cui è definita la sintassi WHILE). Così un analogo del Teorema di Kleene si può formulare e, soprattutto, dimostrare anche per i linguaggi WHILE. Ci riferiamo, come detto, ad una fissata enumerazione effettiva dei programmi WHILE

$$P_0, P_1, \dots, P_i, \dots \quad (i \in \mathbb{N}),$$

accompagnata, per ogni intero positivo h , da una enumerazione delle funzioni parziali h -arie calcolate da questi programmi

$$f_0^{(h)}, f_1^{(h)}, \dots, f_i^{(h)}, \dots$$

(quindi, per ogni i , $f_i^{(h)}$ denota qui la funzione parziale h -aria calcolata da P_i).

Teorema (di Kleene per Programmi WHILE) 6.6.1 *Sia t una funzione calcolabile totale da $\mathbb{N}$ a $\mathbb{N}$. Allora, per ogni intero positivo h , esiste un naturale e tale che i programmi P_e e $P_{t(e)}$ calcolano la stessa funzione h -aria:*

$$f_e^{(h)} = f_{t(e)}^{(h)}.$$

Possiamo qui intendere indifferentemente che t sia calcolabile nel senso classico di Turing, oppure tramite programmi WHILE; infatti, abbiamo appena dimostrato che le due nozioni sono equivalenti. Ma quel che è notevole se si assume il nuovo punto di vista è il fatto che t può essere adesso introdotta come trasformazione di programmi, con esplicito riferimento all'ambito WHILE. Il seguente esempio illustra questa potenzialità.

Esempio. Consideriamo la funzione t da $\mathbb{N}$ a $\mathbb{N}$. Per ogni naturale n , t prende il programma P_n di indice n e vi opera le seguenti sostituzioni:

- (1) t rimpiazza in P_n ogni occorrenza di s con pd e, viceversa, ogni occorrenza di pd con s ;
- (2) t prende poi la prima istruzione I del programma:
 - se I è della forma $X_i := s(X_j)$, allora t la sostituisce con

$\text{begin } X_i := s(X_j); X_i := s(X_j) \text{ end};$

- se I è della forma $X_i := \text{pd}(X_j)$, allora t la sostituisce con

$\text{begin } X_i := \text{pd}(X_j); X_i := \text{pd}(X_j) \text{ end},$

- se I non è né dell'una né dell'altra forma, t inserisce $\text{begin } X_1 := 0 \text{ end}$ immediatamente dopo il primo begin del programma;

- (3) finalmente t prende il programma ottenuto dopo (1) e (2) (dunque una sequenza ordinata finita di istruzioni $C_1, C_2, \dots, C_n$), lo intende come un comando composto

$$\text{begin } C_1; C_2 \dots; C_n \text{ end}$$

e lo scrive a rovescio, trasformandolo in

$$\text{begin } C_n; C_{n-1} \dots; C_1 \text{ end.}$$

Allora $t(n)$ è l'indice del programma ottenuto in questo modo. La funzione t così definita in termini di riscrittura di programmi va da $\mathbb{N}$ a $\mathbb{N}$ ed è calcolabile.

Anche alle funzioni t come quella costruita nell'ultimo esempio si applica il Teorema di Kleene, stabilendo per ogni intero positivo h l'esistenza di un punto fisso e per cui P_e e $P_{t(e)}$ finiscono col calcolare la stessa funzione parziale h -aria. Se ne deducono notevoli conseguenze, quale:

Proposizione 6.6.2 *Esiste un programma WHILE che fornisce in output il proprio indice. In altre parole per ogni h esistono un naturale e (e quindi un programma P_e e una funzione parziale h -aria $f_e^{(h)}$) tali che, per ogni $x \in \mathbb{N}^h$, $f_e^{(h)}(x) = e$.*

Dimostrazione. Definiamo la funzione di riscrittura di programmi $t : \mathbb{N} \rightarrow \mathbb{N}$ che trasforma un arbitrario programma P_n in un programma $P_{t(n)}$ così fatto:

$$P_{t(n)} = \text{begin } X1 := n \text{ end.}$$

Anche t ha un punto fisso, diciamo e , rispetto a h . Quindi per ogni $x \in \mathbb{N}^h$

$$f_e^{(h)}(x) = f_{t(e)}^{(h)}(x) = e.$$

□

6.7 Esercizi

1. Si scriva un programma WHILE che calcoli $\text{pd}(X)$ usando solo assegnamenti della forma $X := 0$ e $X := s(X)$.
2. Si mostri che istruzioni della forma $\text{while } X \neq 0 \text{ do } C$ danno origine alla stessa formulazione di programmi WHILE delle istruzioni $\text{while } X_i \neq X_j \text{ do } C$.
3. Si fornisca una enumerazione esplicita dei programmi WHILE.
4. Si dia una funzione di trasformazione di programmi t tale che, se n è un suo punto fisso e M è l'indice del nuovo programma

$$P_m = \text{begin } X1 := s(X1); X1 := \text{pd}(X1); P_n \text{ end}$$

allora m non è punto fisso per t .

5. Si provi in dettaglio che ogni funzione parziale ricorsiva è WHILE-calcolabile.

Riferimenti bibliografici Un approccio alla teoria della calcolabilità secondo la moderna programmazione è sviluppato in [66] in modo molto chiaro: le nozioni formali e le tecniche vengono introdotte anche in modo intuitivo e illustrate da numerosissimi esempi.

Per un approfondimento sulla semantica dei linguaggi di programmazione si faccia riferimento al testo [31], che discute e confronta vari possibili approcci.

Parte B

Teoria della complessità

7.1 Introduzione

Nella prima parte del libro abbiamo proposto ed approfondito la definizione astratta del concetto di computabilità fornita da Church, Turing e altri. Sulla sua base si possono finalmente distinguere i problemi S che sono computabili da quelli che non lo sono. Ma, anche quando S corrisponde al primo caso, può capitare che i migliori algoritmi che lo risolvono richiedano costi irraggiungibili nella pratica: impiegano tempi proibitivi, o necessitano di risorse di memoria ed energia che nessun calcolatore può assicurare. Questi problemi S , pur computabili in teoria, non lo sono in realtà. Ma allora la nozione di computabilità affermata dalla tesi di Church-Turing va forse rimeditata e riveduta in riferimento ai costi effettivi che le computazioni richiedono. Concentriamo dunque, in questa seconda parte del libro, la nostra attenzione su quei problemi che possono comunque risolversi con opportuni programmi, e vogliamo misurare quali sono le risorse minime richieste da questa soluzione; valutare dunque l'efficienza dei relativi algoritmi e il costo delle loro computazioni.

L'attenzione a questo argomento si è risvegliata in modo naturale con l'avvento e lo sviluppo dei calcolatori. Infatti gli strumenti moderni e la loro potenza computazionale permettono di gestire situazioni sempre più difficili e affrontare input sempre più complicati. Nasce allora l'esigenza di trovare programmi rapidi e poco costosi per trattare queste situazioni. Del resto è facile osservare, come l'uso di tecniche appropriate possa produrre significativi miglioramenti a questo proposito.

Citiamo alcuni esempi abbastanza intuitivi, attinti dalla Matematica elementare.

Esempi

1. Ammettiamo di dover calcolare il valore di un dato polinomio

$$x^7 + 5x^6 + 2x^4 + 12x^3 + 5x^2 + 2x + 21$$

per certi input x . Un semplice controllo mostra come la computazione richie-

de, se il polinomio è proposto nella forma appena descritta,

6 moltiplicazioni (per ottenere x^7),

6 moltiplicazioni (per $5x^6$),

4 moltiplicazioni (per $2x^4$),

3 moltiplicazioni (per $12x^3$),

2 moltiplicazioni (per $5x^2$),

1 moltiplicazioni (per $2x$),

(dunque complessivamente 22 moltiplicazioni) e 6 addizioni. Se però riscriviamo il polinomio come

$$((((x + 5)x^2 + 2)x + 12)x + 5)x + 2)x + 21,$$

ci vengono richieste ancora 6 addizioni, ma solamente 5 moltiplicazioni; il risparmio è ovvio.

2. Ammettiamo di dover calcolare il massimo comun divisore (a, b) di due interi positivi a, b . Immaginiamo naturalmente che a e b siano molto grandi, e si compongano di molte cifre rispetto alla tradizionale base 10.

Il metodo più familiare, quello che si impara sin dalle scuole medie, è

- decomporre a, b in fattori primi,
- ottenere (a, b) come prodotto dei fattori primi occorrenti in entrambe le decomposizioni, presi con il loro esponente minimo.

Il procedimento non è tuttavia facile da svolgere per a, b grandi: infatti, non sono attualmente conosciuti algoritmi soddisfacentemente rapidi di decomposizione in fattori primi.

Funziona invece molto meglio l'algoritmo delle divisioni successive di Euclide, che ricapitoliamo brevemente. Per $a \geq b$,

- dividiamo a per b , con resto r_0 e quoziente q_0 , $a = bq_0 + r_0$, $r_0 < b$,
- poi b per r_0 , $b = r_0q_1 + r_1$, $r_1 < r_0$,
- ancora r_0 per r_1 , $r_0 = r_1q_2 + r_2$, $r_2 < r_1$, ...

finché non si trova, dopo s passi, un resto nullo (come è lecito attendersi, visto che i naturali non possono ammettere una successione decrescente infinita $b > r_0 > r_1 > r_2 > \dots$)

$$r_{s-2} = r_{s-1}q_s, \quad r_s = 0.$$

Ovviamente l'ultimo resto non nullo r_{s-1} è il massimo comun divisore di r_{s-2} e r_{s-1} e non è difficile controllare, osservando tutte le uguaglianze corrispondenti alle varie divisioni, che r_{s-1} è anche il massimo comun denominatore richiesto (a, b) .

Il suo calcolo è costato un numero di divisioni che è ovviamente molto minore di b e si rivela comunque di gran lunga preferibile all'approccio mediante decomposizione in fattori primi, come vedremo in maggior dettaglio più tardi: lo si può svolgere in un numero di passi approssimato in qualche senso dal quadrato del logaritmo in base 10 di a .

3. In algebra lineare si imparano a risolvere sistemi di m equazioni lineari in n incognite. Ammettiamo $m = n$ per semplicità. Per $n = 2$, consideriamo per esempio

$$\begin{cases} 2x + 3y = 5 \\ x - y = 0 \end{cases}$$

Si fa allora riferimento alle due matrici associate al sistema, quella incompleta e quella completa, rispettivamente

$$A = \begin{pmatrix} 2 & 3 \\ 1 & -1 \end{pmatrix}, \quad A' = \begin{pmatrix} 2 & 3 & 5 \\ 1 & -1 & 0 \end{pmatrix}.$$

Possiamo risolvere il sistema calcolando il determinante di A

$$\det \begin{pmatrix} 2 & 3 \\ 1 & -1 \end{pmatrix} = -2 - 3 = -5.$$

Se questo è diverso da 0 (come nel nostro caso), il sistema ammette una e una sola soluzione, che (per la regola di Cramer) è data da

$$x = \frac{1}{-5} \cdot \det \begin{pmatrix} 5 & 3 \\ 0 & -1 \end{pmatrix} = \frac{-5}{-5} = 1$$

$$y = \frac{1}{-5} \cdot \det \begin{pmatrix} 2 & 5 \\ 1 & 0 \end{pmatrix} = \frac{-5}{-5} = 1.$$

È richiesto, dunque, il ripetuto calcolo di determinanti. Per $n = 2$, è esercizio semplice. Per n molto grande, le cose si complicano. Infatti, sfogliando i manuali di algebra lineare, si vede che il determinante di una matrice $n \times n$ è la somma di $n!$ addendi: per $n = 2$, $2! = 2$ è valore innocuo, ma per $n = 10^2, 10^3, 10^{10}, \dots$ sono da attendersi grosse difficoltà. Ricordiamo infatti che, per $n \geq 4$, $n! \geq 2^n$.

C'è comunque un altro metodo di risoluzione del sistema, che estende i ben noti procedimenti di sostituzione, addizione e sottrazione e così via, e fa riferimento al rango delle matrici A, A' . Riducendole in forma triangolare, si arriva alle matrici

$$\begin{pmatrix} 2 & 3 \\ 0 & 5 \end{pmatrix}, \begin{pmatrix} 2 & 3 & 5 \\ 0 & 5 & 5 \end{pmatrix} \text{ (anzi } \begin{pmatrix} 2 & 3 & 5 \\ 0 & 1 & 1 \end{pmatrix})$$

che hanno lo stesso rango 2. Si deduce che ci sono soluzioni, le quali coincidono con quelle del sistema

$$\begin{cases} 2x + 3y = 5 \\ y = 1 \end{cases}$$

Facilmente si ottiene di nuovo $x = y = 1$.

Il numero delle computazioni richieste dal procedimento si valuta approssimativamente in n^3 . Per $n = 2$, $2^3 = 8$ può sembrare livello peggiore del precedente $2!$. Lo stesso vale per $n = 3, 4$. Ma già per $n = 5$, $n^3 = 125$ e $n! = 120$. Per $n \geq 6$, n^3 diviene molto minore di 2^n e perciò di $n!$.

Dunque gli esempi proposti mostrano quanto la scelta di una tecnica invece di un'altra possa incidere significativamente nella soluzione del problema. Vale la pena di approfondire la questione. Un discorso rigoroso richiede comunque che precisiamo:

1. con quale criterio intendiamo misurare il costo delle computazioni, quindi l'efficienza degli algoritmi e in definitiva la complessità di un problema;
2. a quale modello astratto di computazione (una macchina di Turing o qualcosa di più aggiornato) vogliamo fare riferimento.

Si tratta di questioni delicate (soprattutto la prima), meritevoli di approfondimenti futuri. Anticipiamo comunque qualche riflessione.

1. Ci sono vari possibili parametri atti a determinare il livello di risorse necessarie per risolvere un problema:

tempo, memoria, spazio, energia,...

tra loro collegati, ma non direttamente dipendenti.

2. Come sappiamo, il modello della *macchina di Turing* (che anche in questa seconda parte del testo continueremo sovente ad abbreviare con MdT) è ancor oggi perfettamente adeguato quando si parla di computabilità. I problemi che hanno algoritmo di soluzione sono ancora da identificarsi con quelli decisi da macchine di Turing. Quando però l'accento si sposta sulla ricerca di procedure efficienti di computazione (quale che sia il criterio scelto per giudicare questa efficienza) si può dubitare che la MdT sia ancora un modello aggiornato e che miglioramenti e perfezionamenti non siano possibili in ragione di progressi tecnici, o anche dei criteri stessi di efficienza.

Così faremo riferimento in partenza al modello di Turing, ma ci riprometteremo di esplorare nuove prospettive nel seguito.

C'è anche da stabilire quali sono i “problemi” che vogliamo considerare. Lavoriamo, al solito, su un dato alfabeto A , meglio se finito. A meno di codifiche standard, $A = \{0, 1\}$ può essere il nostro punto di riferimento. A^* denota al solito l'insieme delle parole su A . Pensiamo allora a

- (1) problemi di *decisione* volti a considerare linguaggi $S \subseteq A^*$ e a decidere, per ogni parola w su A , se $w \in S$ oppure no; l'input è dunque $w \in A^*$, l'output un *SÌ* o un *NO* (semmai codificati in un modo opportuno).

Formalmente il nostro problema si identifica con il linguaggio S inteso come insieme degli input w per cui si ha output *SÌ*. Anzi, spesso nel seguito useremo “problema” proprio come sinonimo di “linguaggio”. Questo contesto non sarà comunque tale da esaurire ogni possibile interesse. Possiamo infatti pensare a

- (2) problemi di *computazione*: per f funzione da A^* a A^* , eventualmente parziale, cioè ristretta a $S \subseteq A^*$, calcoliamo i valori di f ; l'input è, nuovamente, $w \in A^*$ (semmai w nel dominio di f), l'output richiesto è $f(w)$.

Il panorama non si restringe qui, possiamo pensare a problemi di ottimizzazione, ricerca, e così via. Del resto, le distinzioni tra questi vari obiettivi non sono nette e distinte, matematicamente separate in modo rigoroso. Per semplicità, comunque, ci limiteremo nelle pagine future principalmente ad (1) e talora anche a (2). Possiamo inoltre immaginare di decidere, o computare, coppie, o terne, o sequenze finite di input. Ma opportune funzioni di codifica possono ridurre questi ambiti generalizzati al caso di una sola parola w .

C'è un altro punto che vogliamo sottolineare in questa introduzione. Ci stiamo muovendo per determinare le risorse minime necessarie per risolvere un problema (e dunque per limitare inferiormente la sua difficoltà). Ci attendiamo quindi ragionevolmente

risultati di carattere prevalentemente negativo

atti a certificare che un particolare problema non si risolve senza impegnare una certa quantità di risorse, oppure anche

risultati di confronto

atti a comparare problemi di varia natura e a dichiarare che l'uno è almeno tanto difficile o dichiaratamente più difficile dell'altro; oppure a collegare procedure tese a risolvere lo stesso problema e, nuovamente, a ordinarle in ragione della loro efficienza. La *complessità computazionale* è volta principalmente a questi obiettivi.

7.2 Come misurare la complessità

Cominciamo col valutare la efficienza di un algoritmo (dunque di una macchina di Turing), intendendo misurare poi la complessità di un problema in riferimento ai

migliori algoritmi che lo affrontano. La nostra trattazione sarà ancora vagamente informale, non specificando in dettaglio né il criterio di misura dell'efficienza, né la natura delle procedure seguite. Sono dati l'alfabeto A e, per un problema di decisione, un linguaggio $S \subseteq A^*$. Consideriamo un algoritmo per S . Per valutare adeguatamente la sua efficienza,

- non basta riferirsi ad un **singolo particolare** input $w \in A^*$ e al costo da lui richiesto all'algoritmo;
- ci serve invece un quadro complessivo della situazione, che misuri quanto soddisfacente (o insoddisfacente) sia la computazione **al variare** di w , in ragione della complicazione dell'input w .

La misura della complessità di w può essere ben rappresentata dalla sua lunghezza $l(w)$, cioè dal numero di simboli successivi di w , inteso come parola su A . Possiamo allora considerare una funzione f da $\mathbb{N}$ in $\mathbb{N}$ (eventualmente parziale) che, per ogni naturale n , considera tutti gli input di lunghezza $\leq n$ (che sono comunque un numero finito, visto che l'alfabeto A è finito), segue le corrispondenti computazioni del programma, almeno quelle che hanno buon esito, e va a determinare il loro massimo costo. Questo è $f(n)$. Notiamo che niente esclude di riferirsi ad altre misure di efficienza, per esempio al costo medio (anziché massimo) delle computazioni su input di lunghezza $\leq n$. Manteniamo però qui la scelta di riferirci al costo massimo.

Ci interessano dunque funzioni f di $\mathbb{N}$ in $\mathbb{N}$, eventualmente parziali (cioè non sempre definite), tali comunque da soddisfare le proprietà che andiamo ad introdurre.

- (i) È ragionevole anzitutto attendersi che l'algoritmo da valutare abbia buon esito almeno una volta: se w è il corrispondente input e N la sua lunghezza, f è definita per ogni $n \geq N$, infatti w concorre a stabilire il massimo costo su input di qualsiasi lunghezza $n \geq N$. Dunque

f è definita per ogni $n \geq N$ per un opportuno N .

- (ii) Ovviamente, più grande è n , più numerosi sono gli input da confrontare per calcolare $f(n)$, e dunque più grande è $f(n)$. In termini rigorosi,

f è crescente: per $n \leq n'$ nel dominio di f , $f(n) \leq f(n')$.

- (iii) Specifichiamo meglio quanto detto in (i). In realtà è ragionevole attendersi che l'algoritmo abbia buon esito su input sempre più lunghi, e che il costo massimo cresca effettivamente senza limitazioni all'aumentare della lunghezza n . In termini rigorosi

f non è limitata, $\lim_{n \rightarrow +\infty} f(n) = +\infty$.

Dunque consideriamo funzioni da $\mathbb{N}$ a $\mathbb{N}$ che soddisfano le condizioni (i),(ii), (iii). Ci sono esempi familiari al riguardo.

Esempi

1. Sia $f : \mathbb{N} \rightarrow \mathbb{N}$, $f(n) = n$, o n^2 , o n^3 , o n^k per qualche intero positivo k e per ogni naturale n . Chiaramente valgono (i), (ii), (iii).
2. Possiamo allargare il discorso a funzioni polinomiali, come per esempio

$$f(n) = n^2 - 3n - 4.$$

Più in generale,

$$f(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$$

con k, a_k interi positivi e $a_{k-1}, \dots, a_0$ interi. Ammettiamo dunque coefficienti $a_{k-1}, \dots, a_0$ anche negativi, purché a_k sia positivo (come nel caso di $n^2 - 3n - 4$). È un facile esercizio di analisi verificare (i), (ii), (iii). Per $a_{k-1}, \dots, a_0$ negativi, la risultante funzione f può essere solo parziale cioè non definita per qualche n (quelli per cui il valore del polinomio $a_k n^k + \dots + a_1 n + a_0$ è negativo) oppure non essere crescente; ma quando n è abbastanza grande, f assume valore non negativo, e dunque ammette n nel suo dominio di funzione da $\mathbb{N}$ in $\mathbb{N}$ e inoltre $f(n) < f(n+1)$. Per esempio

$$f(n) = n^2 - 3n - 4$$

assume valori naturali solo per $n \geq 4$, infatti

$$f(0) = -4, f(1) = -6, f(2) = -6, f(3) = -4, f(4) = 0,$$

e $f(n) > 0$ per $n > 4$. Inoltre f è crescente per $n \geq 4$.

3. Anche la funzione $f(n) = 2^n$, per ogni $n \in \mathbb{N}$, soddisfa (i), (ii), (iii).
4. Lo stesso vale per

$$f(n) = 2^{2^n}, f(n) = 2^{2^{2^n}}, f(n) = 2^{2^{2^{2^n}}},$$

o

$$f(n) = 2^{2^n}, f(n) = 2^{n^2}, (n \in \mathbb{N})$$

e così via. Tutte queste funzioni sono ovunque definite, crescenti e illimitate.

5. La funzione $n \mapsto \log_2 n$ è definita per $n > 0$, assume valori reali ≥ 0 per $n \geq 1$, è poi crescente e non limitata. I suoi valori non corrispondono però in genere a numeri naturali, tranne il caso in cui n sia una potenza di 2, cioè $n = 2^t$ con t (allora $\log_2 n = t$). Tuttavia, se sostituiamo, per ogni n , $\log_2 n$ con la parte intera $\lfloor \log_2 n \rfloor$ di $\log_2 n$, cioè con il massimo naturale $\leq \log_2 n$, otteniamo una funzione L ancora definita per $n \geq 1$, crescente e illimitata, tale dunque da soddisfare le condizioni (i), (ii), (iii), e in più a valori naturali. Si ha infatti

$$L(1) = \log_2 1 = 0, L(2) = L(3) = 1, L(4) = \dots = L(7) = 2 \text{ e così via}$$

Naturalmente ci possiamo attendere esempi più “irregolari” di quelli sin qui proposti, come la funzione

$$f(0) = f(1) = f(2) = \dots = f(29) = 1$$

$$f(n) = 2^n \text{ per } n \geq 30,$$

(che coincide con l'esponenziale da 30 in poi, ha comportamento autonomo prima), o casi ancora peggiori. Anche f soddisfa (i), (ii), (iii). Del resto f coincide asintoticamente con l'esponenziale $n \mapsto 2^n$.

Le funzioni che ci interessano sono dunque quelle che obbediscono a (i), (ii), (iii). Infatti, quando esse corrispondono ad un particolare algoritmo nel modo sopra descritto, possono ben costituire un indice della sua efficienza.

Ma in relazione a quanto già detto all'inizio del paragrafo, non ha gran senso lasciarsi condizionare da un singolo valore di f , come $f(10)$ (che si limita comunque ad un campione parzialissimo di input, quelli di lunghezza ≤ 10), ci interessa piuttosto uno studio generale del grafico di f , in particolare il suo comportamento asintotico, quando n tende a farsi grande (e si avvicina a $+\infty$).

In altre parole, date f, g che soddisfano (i), (ii), (iii), per dichiarare f “migliore” di g (e dunque la procedura di f più efficiente di quella di g) non basta notare

$$f(10) \leq g(10),$$

ma occorre qualcosa di più significativo. Si dà a questo proposito la seguente definizione. Assumiamo di trattare funzioni $f, g, \dots$ per cui valgano (i), (ii), (iii).

Definizione Si pone $f = O(g)$ se e solo se esistono un naturale N_0 e un reale positivo c tale che

$$f(n) \leq c \cdot g(n), \text{ per ogni } n \geq N_0.$$

La relazione ora introdotta tra le nostre funzioni stabilisce dunque un confronto asintotico tra f e g ; $f = O(g)$ vale, parlando alla buona, se da un certo N_0 in poi, $g(n)$ domina $f(n)$ a meno della costante c . Tale relazione è chiaramente riflessiva e transitiva (cioè si ha $f = O(f)$ per ogni f , e se $f = O(g)$ e $g = O(h)$, allora $f = O(h)$ per ogni scelta di f, g, h , come è facile verificare). Non è tuttavia antisimmetrica, non costituisce cioè una relazione di ordine parziale. Per esempio, per $f(n) = n$ e $g(n) = 2n$ per ogni $n \in \mathbb{N}$, $f \neq g$ ma

$$\begin{aligned} f &= O(g) \text{ (perché } f(n) \leq g(n) \text{ per ogni } n \in \mathbb{N}), \\ g &= O(f) \text{ (perché } g(n) \leq 2f(n) \text{ per ogni } n \in \mathbb{N}, \text{ dunque per } c = 2). \end{aligned}$$

Si ha comunque il seguente

Teorema 7.2.1 *Siano f, g tali da soddisfare (i), (ii), (iii).*

- 1) *Se $\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} = l \neq 0$, allora $f = O(g)$ e $g = O(f)$.*
- 2) *Se $\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} = 0$, ovvero $\lim_{n \rightarrow +\infty} \frac{g(n)}{f(n)} = +\infty$, allora $f = O(g)$ ma $g \neq O(f)$.*

Dimostrazione. Notiamo che (iii) assicura che $g(n) \neq 0$ e dunque che il rapporto $\frac{f(n)}{g(n)}$ ha senso per ogni n abbastanza grande. Ciò premesso, 1) e 2) sono facili conseguenze della definizione di limite. Vediamo i dettagli.

1) Anzitutto $l > 0$ perché f, g hanno valori naturali. Per ogni reale $\varepsilon > 0$ (senza perdita di generalità, $\varepsilon < l$), esiste $N_\varepsilon > 0$ naturale tale che, per ogni $n \geq N_\varepsilon$,

$$l - \varepsilon < \frac{f(n)}{g(n)} < l + \varepsilon.$$

Moltiplichiamo la seconda disuguaglianza per $g(n)$ (che è positivo) e deduciamo

$$f(n) < (l + \varepsilon) \cdot g(n) \text{ per ogni } n \geq N_\varepsilon,$$

cioè $f = O(g)$ per $c = l + \varepsilon$. Moltiplichiamo poi la prima disuguaglianza per $g(n)$ e dividiamola per $l - \varepsilon > 0$; otteniamo

$$g(n) < \frac{1}{l - \varepsilon} \cdot f(n), \text{ per ogni } n \geq N_\varepsilon,$$

che ancora basta a dedurre $g = O(f)$ per $c = \frac{1}{l - \varepsilon}$.

2) Per ogni $\varepsilon > 0$, esiste N_ε tale che, per ogni $n \geq N_\varepsilon$, $\frac{f(n)}{g(n)} < \varepsilon$, cioè $f(n) < \varepsilon \cdot g(n)$. Allora $c = \varepsilon$ garantisce $f = O(g)$. Invece $g = O(f)$ impone $g(n) \leq c \cdot f(n)$ per $c > 0$ opportuno e per ogni n sufficientemente grande. Ne segue $\frac{f(n)}{g(n)} \geq \frac{1}{c} > 0$ per ogni n grande, e questo esclude $\lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} = 0$. $\square$

Applichiamo il precedente teorema agli esempi sopra considerati (quelli delle funzioni polinomiali, esponenziali o logaritmiche).

Esempi.

1. Sia f una funzione polinomiale

$$f(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$$

con k, a_k interi positivi e $a_{k-1}, \dots, a_0$ interi. Ricordiamo dall'analisi

$$\lim_{n \rightarrow +\infty} \frac{f(n)}{n^k} = a_k \neq 0.$$

Così $f = O(n^k)$ e $n^k = O(f)$, f e $n \mapsto n^k$ asintoticamente vanno a identificarsi tramite O .

2. Sappiamo che, per ogni intero positivo k ,

$$\lim_{n \rightarrow +\infty} \frac{n^k}{2^n} = 0.$$

Così $n^k = O(2^n)$, ma non viceversa. Allo stesso modo

$$\lim_{n \rightarrow +\infty} \frac{2^n}{2^{2^n}} = 0,$$

da cui $2^n = O(2^{2^n})$, ma non viceversa, e così via.

3. Per $L(n) = \text{parte intera del logaritmo di } n$,

$$\lim_{n \rightarrow +\infty} \frac{L(n)}{n^k} = 0$$

per ogni k intero positivo. Dunque $L = O(n^k)$ e non viceversa.

Possiamo così individuare un sottoinsieme di funzioni totalmente ordinato da O

$$L, \dots, n, n^2, \dots, n^k, \dots, 2^n, \dots, 2^{2^n}, \dots$$

Consideriamo tutte queste funzioni, quelle che ad esse si identificano tramite O , o quelle che sono in relazione O con una di loro. Immaginiamole come possibili indici di complessità di un algoritmo. Ci possiamo domandare fino a quale livello in O possiamo ritenere che il corrispondente algoritmo sia efficiente e quando non più. Una risposta a questa domanda richiede che decidiamo con precisione quale criterio di misura intendiamo adottare per la complessità di algoritmi e problemi, se il tempo, o la memoria, o altri parametri ancora. Il prossimo capitolo affronta la questione quando il parametro privilegiato è il tempo.

7.3 Un esempio

Concludiamo questo capitolo discutendo il tema della lunghezza dell'input e del costo di una computazione nel contesto familiare dei numeri naturali. Cominciamo parlando della lunghezza.

Sappiamo tutti che un numero naturale viene comunemente scritto, in base 10, elencando a partire da destra la cifra delle unità, quella delle decine, quella delle centinaia, quella delle migliaia, e così via. Un numero da 0 a 9 (cioè fino a 10 escluso) richiede una sola cifra, uno tra 10 e $100 = 10^2$ (escluso) necessita di due cifre, uno tra 100 e $1000 = 10^3$ (escluso) di 3, e via di seguito. Dunque la lunghezza di un numero intero (positivo) N si riferisce al suo logaritmo in base 10 o meglio alla sua parte intera, che infatti vale

- 0 per numeri fino a 10 escluso,
- 1 per numeri da 10 fino a 10^2 escluso,
- 2 per numeri da 10^2 a 10^3 escluso.

e così via. Il valore preciso della lunghezza di N in base 10 è, infatti

$$\lfloor \log_{10} N \rfloor + 1$$

(dove $\lfloor \dots \rfloor$ denota la parte intera).

Naturalmente, il riferimento alla base 10 non è l'unico possibile, e si potrebbe preferire un'altra base, per esempio 2, e rappresentare così i naturali solo con le cifre 0 e 1, così che

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, \dots$$

diventano

$$0, 1, 10, 11, 100, 101, 110, 111, 1000, 1001, 1010, 1011, 1100, \dots$$

La lunghezza aumenta, ma la regola che la calcola rimane formalmente la stessa, riferita ovviamente alla base 2. In effetti, non è difficile convincersi, anche con pochi semplici esempi, che la lunghezza di N in base 2 è

$$\lfloor \log_2 N \rfloor + 1$$

(e dunque è strettamente collegato a $\log_2 N$). Ricordiamo poi la formula che lega i logaritmi di N in base 10 e 2:

$$\log_{10} N = \log_{10} 2 \cdot \log_2 N.$$

Così anche le lunghezze pur diverse di N nelle basi 10, 2 crescono “quasi proporzionalmente” tramite la costante $\log_{10} 2$. Analoghe considerazioni possono farsi a proposito di altre possibili basi 3, 4, 5, e così via. Notiamo anche che $N = 10^{\log_{10} N} = 2^{\log_2 N}$ è esponenziale rispetto alla sua lunghezza. Così un algoritmo che richiede costo N (o vicino a N) su N è da ritenersi poco efficiente: constatazione che sarà bene ricordare in futuro. In genere, la base cui si fa più spesso riferimento è 2. Le cifre 0,1 che vi ricorrono si chiamano *bit*. Così 1001 (cioè 9 in base 10) si compone di 4 bit (e la sua lunghezza in base 2 è 4).

Abbiamo così trattato il tema della lunghezza degli input. Adesso dobbiamo considerare il costo delle computazioni sui naturali. Anticipando il tema del prossimo capitolo, ammettiamo di misurare la complessità di una computazione tramite il numero complessivo di passi da essa svolto prima di concludere. Ovviamente è da chiarire che cosa si intende per passo di computazione nel nostro ambito, quando si trattano naturali N in notazione binaria e si svolgono le varie operazioni di somma, prodotto e così via. In genere si conviene che ogni singola operazione sui bit degli input N coinvolti corrisponde ad un passo di computazione. Per esempio si assume che l'addizione

$$\begin{array}{r} 1001 \quad + \\ 1110 \quad = \\ \hline 100111 \end{array}$$

(ovvero $9+30=39$ in base 10) richiede 5 passi per svolgere i calcoli sulle varie colonne. In questa ottica, si può notare che sommare due numeri di k bits richiede

al più k passi, ed è dunque al più lineare nella lunghezza k , mentre la moltiplicazione degli stessi numeri si può svolgere in k^2 passi (dunque al più in costo al più quadratico in k). Lo stesso vale per la divisione (intendendo come divisione il calcolo di quoziente e resto, come in genere si conviene tra i naturali).

Illustriamo allora la situazione riferendoci come esempio al calcolo del massimo comun divisore tramite l'algoritmo di Euclide delle divisioni successive. Vogliamo mostrare che secondo i parametri appena introdotti, questo procedimento ha costo al più quadratico rispetto alla lunghezza dell'input (come già anticipato a livello intuitivo nell'introduzione di questo capitolo).

Consideriamo dunque due naturali a, b . Possiamo supporre $a > b > 0$ per semplicità. I due primi passi dell'algoritmo di Euclide corrispondono alle divisioni

$$a = b \cdot q_0 + r_0 \text{ con } r_0 < b, q_0 \geq 1,$$

$$b = r_0 \cdot q_1 + r_1 \text{ con } r_1 < r_0, q_1 \geq 1.$$

Ne deduciamo

$$a = (r_0 \cdot q_1 + r_1)q_0 + r_0 = r_0(q_1 \cdot q_0 + 1) + r_1 \cdot q_0 > r_0 + r_1 > 2r_1.$$

Così due applicazioni dell'algoritmo determinano un resto $r_1 < \frac{1}{2}a$. Non è difficile ripetere l'osservazione e notare che $r_1 > 2r_3$, dunque $a > 2r_1 > 2^2r_3$; in generale, dopo t iterazioni si deduce

$$a > 2^t \cdot r_{2t-1}$$

(dove r_{2t-1} è il resto ottenuto al passo $2t - 1$).

Sia s il passo finale (quello che produce $r_s = 0$ e dunque dichiara $r_{s-1} = (a, b)$). Supponiamo per semplicità che s sia pari, dunque $s = 2T$, per T numero naturale, e deduciamo

$$a > 2^T r_{2T-1}.$$

Siccome $r_{s-1} > 0$, $\frac{a}{2^T} \geq 1$, cioè $a \geq 2^T$ e $T \leq \log_2 a$, da cui $s - 1 \leq 2\log_2 a$.

Così le varie divisioni successive da svolgere sono $O(\log_2 a)$. Inoltre i calcoli sui bit dei vari numeri coinvolti in ciascuna divisione si svolgono certamente in tempo $O(\log_2^2 a)$ perché riguardano interi al più lunghi quanto a , e anzi possono ridursi a $O(\log_2 a)$ con qualche accorgimento. Dunque il numero di passi dell'algoritmo euclideo è $O(\log_2^2 a)$, dunque al più polinomiale (e anzi quadratico) nella lunghezza dell'input a (e conseguentemente anche in quella di b , visto che $b \leq a$).

7.4 Esercizi

1. Quali delle seguenti funzioni da $\mathbb{N}$ in $\mathbb{N}$ soddisfano le condizioni (i), (ii), (iii)?
Per n che varia in $\mathbb{N}$,
 - a) $f(n) = n^2 - 10^{37}$;
 - b) la funzione Σ di Rado;

- c) $f(n) = \max\{2^{2^n}, \Sigma(n)\}$;
 - d) la funzione caratteristica dell'insieme dei numeri pari;
 - e) per K insieme non ricorsivo di naturali, $f(n)$ = numero dei naturali che stanno in K e sono $\leq n$;
 - f) $f(n) = n$ se n è pari; $f(n)$ non è definita altrimenti.
2. Consideriamo la relazione O tra funzioni. Quali delle seguenti affermazioni sono vere?
- a) $2^n = O(\Sigma)$ dove Σ è la funzione di Rado;
 - b) $\Sigma = O(2^n)$;
 - c) $\lfloor \log_{10} n^2 \rfloor = O(\lfloor \log_{10} n^2 \rfloor)$;
 - d) $n = O(n!)$;
 - e) $n! = O(n)$.

Riferimenti bibliografici Una trattazione dettagliata della teoria astratta delle misure di complessità può trovarsi in [5], o anche in [22], [29], [50], [85], [102]. [5] propone e discute anche vari problemi algebrici (per esempio, la moltiplicazione righe per colonne delle matrici) la soluzione dei quali risulta avvantaggiata e accelerata dall'impiego di tecniche opportune. Le informazioni basilari sui naturali (ed in particolare sul massimo comun divisore) possono trovarsi in [69], o anche in [40].

Classi di Complessità Temporale

8.1 La classe P e la Tesi di Edmonds-Cook-Karp

Lavoriamo con un alfabeto finito A e con $S \subseteq A^*$. Ricordiamo che una macchina di Turing M su A decide S se M converge su tutti gli input $w \in A^*$ e,

- se $w \in S$, risponde “SÌ” (produce cioè un output convenzionalmente identificato con “SÌ”),
- se $w \notin S$, risponde “NO” (allo stesso modo).

Si dice invece che M accetta S se, per ogni $w \in A^*$,

- se $w \in S$, M converge su w ,
- se $w \notin S$, M diverge su w .

È da ribadire che, in generale, le due nozioni, decidere e accettare, non coincidono. L'alternativa *convergere/divergere* non può sostituire le risposte *sì/no*. Infatti non è possibile a priori realizzare che una macchina di Turing diverge; un osservatore esterno vede sviluppare la computazione per $1, 10, 10^2, 10^3, \dots, 10^n, \dots$ passi, ma non può prevedere se M convergerà al passo successivo oppure continuerà a lavorare all'infinito.

Ciò premesso, vediamo come sia possibile misurare la complessità di una macchina di Turing M su un alfabeto finito A , privilegiando il parametro *tempo*. Facciamo quindi riferimento al numero dei passi delle computazioni convergenti di M . Specificamente definiamo una funzione c_M (*complessità temporale* di M) come segue: per ogni $n \in \mathbb{N}$, c_M è definita su n se e solo se esiste almeno un input di lunghezza $\leq n$ su cui M converge e, in tal caso, $c_M(n)$ è il massimo numero di passi di una tale computazione di M .

Notiamo che, siccome A è finito, c'è al più un numero finito di parole su A di lunghezza $\leq n$. Quindi c'è un numero finito di computazioni convergenti su tali input e dunque, tra esse, ce n'è una di lunghezza massima. In conclusione c_M è ben definita.

Possiamo anche ragionevolmente ammettere che una tale funzione c_M (da $\mathbb{N}$ a $\mathbb{N}$) soddisfi (i), (ii), (iii). Escludere (i) significa infatti supporre che M non converga mai, su nessun input: situazione ammissibile, ma di nessun interesse pratico. (ii)

è condizione facile da accettare. Quanto a (iii), contraddirla significa ammettere che M , quando converge, lo fa in un numero prefissato di passi, a prescindere dalla lunghezza dell'input che le viene proposto. Di nuovo, si tratta di situazione possibile, ma certamente non interessante dal punto di vista pratico.

Così possiamo assumere (i), (ii), (iii) per c_M .

Si tratta ora di decidere, in riferimento all'analisi finale del paragrafo precedente, quali condizioni su c_M possono certificare che M è efficiente rispetto al tempo (cioè rapida) nelle sue computazioni. C'è una proposta a questo proposito, avanzata a metà degli anni sessanta da Edmonds, anticipata comunque da von Neumann, Rabin e Cobham, e poi ribadita da Cook e Karp, la quale afferma, a livello di slogan, che

rapido = polinomiale.

In termini rigorosi, poniamo la seguente

Definizione. P è la classe dei problemi S su alfabeti finiti A per i quali c'è una macchina di Turing M su A che accetta S e ha complessità $c_M = O(n^k)$ per qualche intero positivo k .

Dunque la MdT M , di fronte ad un input $w \in A^*$ di lunghezza n ,

- se $w \in S$, converge su w in al più $c \cdot n^k$ passi (dove c è una costante reale positiva prefissata);
- se $w \notin S$, diverge su w .

Convienne aprire una breve parentesi a proposito della definizione ora data. Abbiamo infatti tenuto a sottolineare in precedenza la differenza tra *decidere* e *accettare* S . In generale, questa distinzione va ben tenuta presente. Ma, nel caso particolare ora in esame, sappiamo che la MdT M sull'input w , se converge, lo fa entro $c \cdot n^k$ passi, dove n è la lunghezza di w . Così possiamo dedurre che, se non c'è stata convergenza entro il passo $c \cdot n^k$, M certamente divergerà. Dunque è possibile controllare anche la divergenza e, nella situazione che viene a crearsi,

rispondere SÌ/NO

equivale perfettamente a

convergere/divergere.

Ciò premesso, possiamo enunciare con maggior precisione la

Tesi di Edmonds-Cook-Karp. Sia S un insieme di parole su un alfabeto finito A . Allora S ha un algoritmo rapido di decisione se e solo se $S \in P$.

Si stabilisce dunque che un problema è *computabile* quando la sua soluzione richiede tempo al più polinomiale nella lunghezza dell'input. La parola tesi è da intendersi qui come nel caso di Church-Turing, descritto nella prima parte. Non è un teorema, o un assioma; è solo una ipotesi di lavoro con qualche base sperimentale, eventualmente da discutere (ne parleremo tra qualche riga), da accettare finché l'evidenza la sostiene, da rivedere, correggere, adeguare altrimenti.

Commenti sulla Tesi di Edmonds-Cook-Karp.

- La tesi di Edmonds-Cook-Karp si riferisce non tanto agli algoritmi quanto ai problemi. Individua i problemi “rapidamente” risolubili come quelli che ammettono un algoritmo (cioè una macchina di Turing) che li decide (equivalentemente, li accetta) in tempo al più polinomiale nella lunghezza dell’input.
- Dunque la tesi esclude che un problema i cui algoritmi lavorano tutti in tempo almeno esponenziale 2^n possa ritenersi di rapida soluzione: affermazione che pare facilmente condivisibile se si ricorda quanto rapidamente crescono le potenze 2^n di 2 all’aumentare dell’esponente n .
- Viceversa la tesi afferma che, se c’è un algoritmo che decide (o accetta) un problema in tempo al più polinomiale $O(n^k)$, allora il problema ha rapida soluzione. Questa seconda osservazione appare assai più discutibile, per almeno due motivi.
 1. Quanto rapido è un algoritmo che lavora in tempo n^{10^6} , n^{10^9} , oppure $n^{5 \cdot 10^{17}}$ rispetto alla lunghezza dell’input? (Ricordiamo che $5 \cdot 10^{17}$ secondi è il tempo stimato dall’inizio dell’universo ad oggi secondo la teoria del Big Bang).
 2. Non va dimenticato il ruolo della costante c nella definizione di O . Se c è enormemente grande, per esempio $5 \cdot 10^{17}$, anche per k piccolo, quanto rapido può ritenersi un algoritmo che lavora in tempo $c \cdot n^k$ rispetto alla lunghezza n dell’input?

Sotto questo punto di vista, possono esserci ragionevoli riserve all’adozione della tesi di Edmonds-Cook-Karp. D’altra parte, da un punto di vista teorico,

- possiamo ad esempio ammettere che algoritmi che lavorano in tempo al più lineare $O(n)$, o al più quadratico $O(n^2)$ nella lunghezza dell’input siano “rapidi”;
- una volta concordato che algoritmi che impiegano tempo $O(n^k)$ (per un qualche intero positivo k) sono rapidi, quale motivo può indurci ad escludere come “lenti” quelli che richiedono tempo $O(n^{k+1})$?

Così la tesi di Edmonds-Cook-Karp è comunemente accettata, pur con i dubbi sopra riferiti.

Vediamo adesso alcuni esempi, più o meno difficili e famosi, di problemi che stanno in P (e sono dunque da ritenersi di “rapida” soluzione).

Esempi.

1. Grafi 2-colorabili (2COL).

Ricordiamo che un grafo è una struttura $G = (V, E)$ dove V è un insieme non vuoto ed E è una relazione binaria su A irriflessiva e simmetrica (in altre parole, nessun $v \in V$ è in relazione con se stesso e dunque soddisfa $(v, v) \in E$; se poi $v, w \in V$ e $(v, w) \in E$, allora anche $(w, v) \in E$).

Possiamo dunque pensare gli elementi v di V come possibili tappe da raggiungere, e le coppie $(v, w) \in E$ come strade dirette che li congiungono. In effetti,

i punti di V si chiamano *vertici* e le coppie di E *lati*. Nessun lato può unire un vertice a se stesso per la irriflessività, ma ogni lato è a doppio senso (per la simmetria). La sequenza di vertici $v_0, v_1, \dots, v_{n-1}$ viene detta *cammino* di lunghezza n se, per ogni $i < n$, $(v_i, v_{i+1}) \in E$ e $(v_i, v_{i+1}) \neq (v_j, v_{j+1})$ per $i < j < n$.

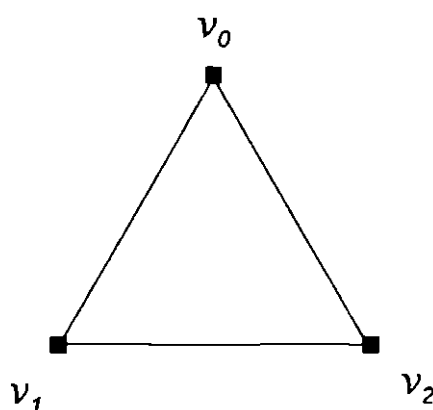
Una *2-colorazione* di un grafo $G = (V, E)$ è una funzione c da V a $\{1, 2\}$ tale che, per ogni scelta di $v, w \in V$ con $(v, w) \in E$, $c(v) \neq c(w)$.

Possiamo pensare 1, 2 come 2 possibili colori con cui dipingere i vertici di V ; c è, appunto, questa colorazione e deve soddisfare la condizione che estremi distinti dello stesso lato hanno colori diversi. Se una tale c esiste, il grafo G si dice *2-colorabile*.

Ci interessa il seguente problema di decisione, chiamato *2COL* (2-colorabilità dei grafi):

- INPUT: un grafo finito $G = (V, E)$;
- OUTPUT: SÌ/NO, a seconda che G sia o no 2-colorabile.

$2COL \in P$. Un possibile algoritmo a questo proposito è il seguente. Prendiamo un vertice $v_0 \in V$, e coloriamolo in qualche modo, per esempio fissiamo $c(v_0) = 1$: la scelta non è restrittiva perché, se una 2-colorazione c di G funziona, anche quella che si ottiene da c permutando i colori di tutti i vertici è ancora valida. Sia dunque $c(v_0) = 1$. A questo punto, tutti i vertici v_1 collegati a v_0 da qualche lato dovranno essere colorati in modo diverso $c(v_1) = 2$; i vertici v_2 collegati ai vari v_1 dovranno a loro volta soddisfare $c(v_2) = 1$, e così via. Ovviamente questo è impossibile se, per esempio, vale già $(v_0, v_2) \in E$ e dunque v_2 deve già avere colore 2:



in questo caso, c non può essere trovato. Altrimenti si continua il procedimento. Si crea così una sorta di effetto domino al termine del quale tutti i vertici collegati a v_0 da una sequenza finita di lati successivi derivano dalla condizione iniziale $c(v_0) = 1$ una colorazione obbligatoria. Se tale colorazione c non è complessivamente possibile, possiamo rispondere NO al quesito della 2-colorabilità di (V, E) . Altrimenti, se la scelta $c(v_0) = 1$ si concilia con tutti i vertici in qualche modo connessi a v_0 , andiamo a considerare i punti v non ancora coinvolti, perché non collegati a v_0 da nessuna sequenza di lati. La loro colorazione è ancora libera, e possiamo applicare da capo il procedimento. Dopo un numero finito di applicazioni positive arriviamo alla conclusione SÌ.

È facile convincersi che il procedimento non è molto più lungo della semplice visita dei singoli vertici del grafo, e dunque si può svolgere in tempo polinomiale rispetto al numero $|V|$ degli elementi di V .

Così $2COL \in P$. Semmai ci si può chiedere che cosa succede quando le ambizioni pittoriche crescono e i colori a disposizione diventano più di 2. Ma di questo avremo modo di riparlare più avanti.

2. 2-Soddisfacibilità (2SAT).

Ammettiamo stavolta di avere un alfabeto potenzialmente infinito

$$p_0, p_1, p_2, \dots$$

e di poterne scrivere le lettere anche in maiuscolo

$$P_0, P_1, P_2, \dots$$

Se vogliamo preservare al nostro alfabeto il carattere finito, possiamo convenire di limitarci a 3 soli simboli $p, P, |$ e scrivere lettere minuscole e maiuscole come

$$p, p|, p||, p|||, \dots, P, P|, P||, P|||, \dots$$

Formiamo parole con 2 lettere

$$p_0P_1, p_0P_2, P_1P_3, p_4P_0, P_4P_5, \dots$$

con l'obbligo di evitare ripetizioni di lettere nella stessa parola: dunque parole come $p_0p_0, p_0P_0, P_0p_0, P_0P_0$ sono tutte vietate. Chiamiamo "clausole" le parole così ottenute. Ovviamente, si possono costruire clausole anche più lunghe come $p_0P_1p_4P_7$, se mai la cosa ci interessa. Resta comunque valida la regola di evitare ripetizioni nella stessa clausola.

Adesso formiamo insiemi finiti di clausole di 2 lettere, come

$$\{p_0P_1, p_0p_2, P_1p_3, p_4P_0, P_4P_5\}.$$

Ci domandiamo se c'è una clausola (eventualmente lunga più di 2 lettere) che le interseca tutte (nel senso che condivide con ciascuna di esse almeno una lettera minuscola oppure maiuscola). Nel caso specifico $p_0P_1p_4P_5$ funziona poiché ha in comune p_0 con p_0P_1 e p_0p_2 , P_1 con P_1p_3 , p_4 con p_4P_0 e, finalmente, P_5 con P_4P_5 .

Il problema che ci interessa generalizza l'esempio appena svolto; si chiama 2-soddisfacibilità (soddisfacibilità per clausole di due lettere) e si indica 2SAT. È una questione di logica elementare. 2SAT (2-soddisfacibilità) ha

- INPUT: un insieme finito di clausole a 2 lettere;
- OUTPUT: SÌ/NO, a seconda che esista una clausola (anche a più lettere) che interseca ciascuna delle clausole assegnate.

Nuovamente si vede che $2SAT \in P$. Per dimostrarlo si può procedere come nel caso della 2-colorabilità. Abbiamo un insieme finito di clausole a due lettere, e dobbiamo formare una parola w che interseca ognuna di queste clausole. Cominciamo con l'azzardare che w includa p_0 (come p_0P_1 e p_0p_2 nell'esempio precedente), ma, per quelle che invece contengono P_0 (come p_4P_0), impone l'inserimento di p_4 in w . Di nuovo le clausole con p_4 risultano così intersecate, quelle con P_4 - come P_4P_5 - obbligano P_5 in w , e così via. Se in questo modo si riesce a formare una parola w che interseca ciascuna clausola allora l'algoritmo risponde *SÌ*, se no si prova a inserire P_0 in w .

Si conclude come nel caso di $2COL$ che questo algoritmo (innescando una sorta di effetto domino) decide $2SAT$ in un tempo che è funzione al più polinomiale nella lunghezza dell'input (indipendentemente da come rappresentiamo l'input: tramite le lettere $p_0, p_1, \dots$, o, in modo più prolisso, mediante $p, P, |$).

$2SAT$ e il conseguente gioco di parole che si intersecano, nasconde in realtà una questione di logica elementare: la soddisfacibilità, appunto. Proviamo infatti a pensare alle lettere minuscole come ad affermazioni del tipo "oggi piove" e alle corrispondenti maiuscole come alle loro negazioni "oggi non piove".

Intendiamo poi informalmente ogni clausola come la disgiunzione delle affermazioni corrispondenti alle sue lettere. Per esempio se p_0 sta per "oggi piove" e p_1 per "domani nevica", p_0P_1 è "oggi piove o domani non nevica".

Si capisce poi perché si escludono ripetizioni come p_0p_0 ("oggi piove oppure oggi piove") o banalità come p_0P_0 ("oggi piove oppure oggi non piove").

Finalmente pensiamo un insieme di clausole come la congiunzione delle clausole che lo compongono. Così, se p_0 sta per "oggi piove", p_1 per "domenica nevica" e p_2 per "lunedì grandina",

$$\{p_0P_1, p_0p_2, \dots\}$$

significa "oggi piove o domani non nevica" e "oggi piove o lunedì grandina" e così via.

In questa ottica, se vogliamo riferirci a $p_0, p_1, p_2, p_3, \dots$ come a proposizioni e a $P_0, P_1, P_2, P_3, \dots$ come alle loro negazioni, una clausola come $p_0P_1p_4P_5$ può intendersi come la scelta di una assegnazione di verità a p_0, p_1, p_4, p_5 (ed eventualmente anche a $p_2, p_3, p_6, \dots$). $p_0P_1p_4P_5$ dice infatti di accettare p_0, p_4 come vere e p_1, p_5 come false (e dunque P_1, P_5 come vere). Consideriamo allora il precedente insieme $\{p_0P_1, p_0p_2, P_1p_3, p_4P_0, P_4P_5\}$ di clausole a due lettere, intersecato da $p_0P_1p_4P_5$ in ogni sua parola; in riferimento alla valutazione appena descritta, possiamo equivalentemente dire che, in ogni clausola $p_0P_1, p_0p_2, P_1p_3, p_4P_0, P_4P_5$ dell'insieme considerato, compare una lettera (cioè una proposizione affermata o negata) che viene ritenuta vera da $p_0P_1p_4P_5$, così che la clausola stessa - l'alternativa delle due singole proposizioni che la compongono - risulta conseguentemente vera.

Fissare la clausola $p_0P_1p_4P_5$ equivale dunque a fissare una funzione v dalle varie p_n ($n \in \mathbb{N}$) in $\{0, 1\}$ (dove 0 sta per falso, 1 per vero): v si chiama

valutazione. Nel caso specifico dell'esempio ora considerato, si ha

$$v(p_0) = v(p_4) = 1, v(p_1) = v(p_5) = 0$$

mentre $v(p_n)$ è libera per gli altri indici n . Notiamo che la v così definita scopre dentro ogni clausola a 2 lettere dell'insieme considerato una lettera cui dà valore 1

$$v(p_0) = v(p_4) = v(P_1) = v(P_5) = 1, \text{ come già osservato.}$$

$2SAT$ si può allora interpretare come il seguente problema:

- INPUT: un insieme finito di clausole a 2 lettere;
- OUTPUT: SÌ/NO, a seconda che ci sia qualche valutazione v che dà valore 1 a qualche lettera in ognuna delle clausole dell'insieme (e in questo senso lo "soddisfa").

Come già visto, $2SAT \in P$. Questo risultato si può anche usare per confermare la conclusione dell'Esempio 1, cioè $2COL \in P$. Infatti $2COL$ si riduce a $2SAT$ come segue.

Ad ogni grafo finito G , si associa un insieme finito di clausole a 2 lettere $I(G)$ tale che

- (i) la costruzione di $I(G)$ da G è "rapida",
- (ii) $G \in 2COL$ se e solo se $I(G) \in 2SAT$.

Se questo è possibile, per verificare che un dato grafo G è 2-colorabile, basta

- prima formare "rapidamente" $I(G)$,
- poi verificare "rapidamente" se $I(G)$ è soddisfacibile (come è possibile perché $2SAT \in P$).

Si decide conseguentemente se G è 2-colorabile.

Vediamo allora come si procede per la costruzione di $I(G)$. Sia dato $G = (V, E)$ dove fissiamo per semplicità $V = \{v_0, \dots, v_{n-1}\}$. Le clausole di $I(G)$ richiedono $2n$ lettere minuscole

$$p_{i1}, p_{i2} \quad (i < n),$$

due per ogni vertice v_i di V , accompagnate dalle corrispondenti maiuscole

$$P_{i1}, P_{i2} \quad (i < n).$$

$I(G)$ si decompone delle seguenti clausole

- per $i < n$, $p_{i1}p_{i2}$,
- per $i < j < n$ e $(v_i, v_j) \in E$, $P_{i1}P_{j1}$ e $P_{i2}P_{j2}$.

Così il numero complessivo delle clausole di $I(G)$ è $|V| + 2|E|$, e cioè la somma tra il numero dei vertici e il doppio del numero dei lati di G . Quindi la costruzione di $I(G)$ può farsi in modo univoco e preciso (come anche si dice, deterministico) e, soprattutto, polinomiale nella lunghezza di G . Vale quindi (i). Proviamo adesso (ii).

Supponiamo dapprima G 2-colorabile. Sia c una 2-colorazione di G . Costruiamo una parola w che, per ogni $i < n$, include

- p_{i1}, P_{i2} se $c(v_i) = 1$,
- P_{i1}, p_{i2} se $c(v_i) = 2$.

Così w ha lunghezza $2n$. Inoltre, per ogni $i < n$, w interseca certamente la clausola $p_{i1}p_{i2}$; se poi $i < j < n$ e $(v_i, v_j) \in E$, si ha $c(v_i) \neq c(v_j)$, dunque w include P_{i1} e P_{j2} oppure P_{i2}, P_{j1} , e comunque interseca sia $P_{i1}P_{j1}$ che $P_{i2}P_{j2}$. Così $I(G)$ è soddisfacibile.

Viceversa, ammettiamo $I(G)$ soddisfacibile. Sia w una parola che interseca tutte le clausole di $I(G)$. Così, per ogni $i < n$, w contiene p_{i1} oppure p_{i2} ; per $i < j < n$ con $(v_i, v_j) \in E$, w contiene P_{i1} o P_{j1} , e poi P_{i2} o P_{j2} . Definiamo una funzione c di V in $\{1, 2\}$ ponendo, per ogni $i < n$,

$$c(v_i) = 1 \text{ se } p_{i1} \text{ è in } w, \quad c(v_i) = 2 \text{ altrimenti.}$$

Mostriamo che c è una 2-colorazione di G . Siano $i < j < n$ tali che $(v_i, v_j) \in E$. Supponiamo per assurdo $c(v_i) = c(v_j)$. Se il colore comune di v_i, v_j in c è 1, allora p_{i1}, p_{j1} sono in w , che dunque esclude tanto P_{i1} quanto P_{j1} , e non interseca la clausola $P_{i1}P_{j1}$. Sia allora $c(v_i) = c(v_j) = 2$, così p_{i1}, p_{j1} non compaiono in w , che quindi include p_{i2} e p_{j2} , e conseguentemente esclude P_{i2}, P_{j2} . Dunque w non riesce a intersecare $P_{i1}P_{j2}$. Così deve essere $c(v_i) \neq c(v_j)$ e, tramite c , G 2-colorabile.

L'esempio mostra come problemi di natura e origine diversa possano talora collegarsi in modo tale che algoritmi soddisfacenti per l'uno riescano ad applicarsi anche all'altro. Approfondiremo nel seguito, anche al di fuori di P , il tema di queste riduzioni.

3. Equazioni lineari a coefficienti interi

Non sempre un'equazione a coefficienti interi ha soluzioni intere. Abbiamo avuto già modo di discuterne parlando del Decimo Problema di Hilbert, nel corso della prima parte. Anche se ci restringiamo a equazioni lineari (cioè di primo grado) in 2 incognite x, y troviamo situazioni disperate. Per esempio

$$2x + 4y = 1$$

non ha soluzioni intere perché 1 è dispari e, per ogni scelta di X, Y interi, $2X + 4Y$ è pari. Invece

$$2x + 3y = 1$$

ha la soluzione intera $X = -1, Y = 1$. Il problema che ci interessa (quello delle equazioni lineari a coefficienti interi) ammette appunto

- INPUT: un'equazione $ax + by = c$ con $a, b, c \in \mathbb{Z}$, a, b non entrambi nulli

e richiede

- OUTPUT: SÌ/NO a seconda che ci siano o no interi X, Y che la soddisfano ($aX + bY = c$).

Un minimo di dimestichezza con l'algebra ci mostra che $ax + by = c$ ha soluzioni intere se e solo se il massimo comun divisore (a, b) di a, b divide c . Infatti, se ci sono $X, Y \in \mathbb{Z}$ per cui $aX + bY = c$, (a, b) , che è divisore di a e b , tramite X, Y divide anche $aX + bY$, cioè c .

Viceversa, l'identità di Bézout assicura che ci sono due interi X, Y per cui

$$aX + bY = (a, b)$$

e la proprietà si trasmette ovviamente ad ogni multiplo c di (a, b) . Anche in vista di successive applicazioni nei prossimi paragrafi, ricordiamo in maggior dettaglio la prova dell'identità di Bézout. Tutto nuovamente si basa sull'algoritmo euclideo delle divisioni successive per la ricerca del massimo comun divisore (a, b) di a, b : per $a \geq b > 0$, si divide a per b , b per l'eventuale resto non nullo r_0 , e così via finché non si trova un resto nullo r_s ; l'ultimo resto non nullo r_{s-1} è (a, b) .

Se la procedura si ferma subito, cioè $(a, b) = b$, b si scrive $a \cdot 0 + b \cdot 1$, dunque si pone $X = 0, Y = 1$. Altrimenti, si procede per induzione sul numero s dei passi necessari per determinare (a, b) . Ammettiamo così di aver ottenuto la decomposizione di (b, r_0) (il cui calcolo richiede un passo in meno rispetto ad (a, b)) e di aver provato $(b, r_0) = bX' + r_0Y'$ per X', Y' interi opportuni; ricordiamo $a - bq_0 = r_0$ per qualche q_0 e deduciamo

$$(a, b) = (b, r_0) = bX' + r_0Y' = bX' + (a - bq_0)Y' = aY' + b(X' - q_0Y'),$$

così $X = Y'$ e $Y = X' - q_0Y'$ soddisfano quanto l'identità di Bézout richiede. Anzi si noti che X, Y così determinati hanno lunghezza polinomialmente limitata da quella dei coefficienti a, b, c . Infatti, la proprietà è ovvia quando $X = 0, Y = 1$, e si preserva ad ogni passo induttivo; così ci basta ricordare che X, Y sono calcolati entro S passi dove $S \leq \log_2 N$.

Torniamo adesso al nostro problema di esistenza di soluzioni intere per $ax + by = c$. È facile tracciare un programma che lo risolve: dati a, b, c ,

- calcoliamo (a, b) (per esempio tramite l'algoritmo euclideo delle divisioni successive),
- controlliamo poi se (a, b) divide o no c .

L'esistenza di soluzioni intere corrisponde infatti ad una risposta positiva all'ultima verifica.

Questa procedura pone il problema in P . Infatti i tempi di lavoro sono più o meno quelli dell'algoritmo di Euclide, visto che la divisione finale di c per (a, b) e il calcolo del relativo resto non incidono significativamente, e sappiamo dal Capitolo 7 che il procedimento di Euclide impiega tempo al più polinomiale, anzi quadratico, rispetto alla lunghezza degli input a, b .

4. *PRIMI*

L'ultimo esempio che vogliamo citare riguarda un problema famoso e datato, che risale ai tempi di Euclide, e dunque a 2 millenni or sono, e anche più.

È dato:

- INPUT: un intero $N > 1$;
mentre
- OUTPUT: consiste nel
 - a) rispondere *SÌ/NO* secondo che N sia primo, o composto;
o addirittura
 - b) decomporre N nei suoi fattori primi.

Ricordiamo che N è primo se non ha divisori naturali diversi da 1 e da N e che ogni intero maggiore di 1 si decompone in modo unico nel prodotto di fattori primi. Come si vede, la questione è duplice. Infatti, per N composto, a) si accontenta di saperlo, b) pretende invece di conoscere i divisori non banali di N . a) è poi un problema di decisione, b), invece, un problema di computazione. Come detto, la questione è classica, eppure ancora attuale, visto che molti protocolli crittografici (usati per esempio per garantire transazioni sicure in rete), si basano sulle attuali conoscenze e ignoranze su algoritmi di primalità e fattorizzazione. Pur tuttavia, metodi molto semplici riescono a soddisfare l'una e l'altra questione. Vediamone uno.

Dato N , si divide N per ogni numero q tale che $1 < q < N$. Se la divisione non è mai precisa (cioè non dà mai resto 0), si può dedurre che N è primo; altrimenti, se $N = q \cdot q'$ per qualche q con $1 < q < N$ e per un opportuno corrispondente q' , ancora soddisfacente $1 < q' < N$, allora si può dichiarare N composto e si hanno informazioni sulla sua decomposizione (anche se, per ottenere tutti i fattori primi di N , bisognerà applicare la procedura a q , q' e poi ulteriormente, finché necessario).

Tutto questo era noto già ai tempi di Euclide e degli antichi greci. Perché dunque l'algoritmo non ci basta? La risposta è che esso richiede $N - 2$ divisioni, quindi un numero di passi almeno esponenziale rispetto al logaritmo di N e quindi, in definitiva, rispetto alla lunghezza di N .

È vero che la procedura si può accelerare; per esempio è sufficiente applicarla ai valori q con $2 < q < \sqrt{N}$ (**Esercizio.** Perché?). Tuttavia queste semplificazioni non bastano ad evitare il carattere esponenziale: infatti $\sqrt{N} = N^{\frac{1}{2}} = 2^{\frac{1}{2} \log_2 N}$ è ancora esponenziale rispetto a $\log_2 N$.

In effetti, solo nell'agosto del 2002 tre informatici indiani (Agrawal, Kayal, Saxena [2], [3]) hanno determinato un algoritmo che risolve il problema di primalità lavorando in tempo al più polinomiale nella lunghezza di N (circa $O(\log_2^{11} N)$, a meno di fattori di minor significato). La procedura di Agrawal, Kayal, Saxena, già denotata AKS dalle iniziali dei suoi autori, è relativamente complicata, richiede conoscenze di algebra astratta e di combinatorica e trascende certamente gli obiettivi di queste note. Comunque colloca finalmente il problema della primalità in P (concludendo in questo modo un itinerario di ricerca durato oltre 2 millenni).

Terminiamo l'esempio con due commenti:

- AKS, pur ottimo in teoria, non è ancora gran che utilizzato nella pratica. Computazioni di grado 11 nella lunghezza dell'input risultano proibitive

nelle applicazioni. Si continuano a preferire, per la primalità, procedure di carattere probabilistico, passibili di errori nelle loro risposte “ N primo” o “ N composto”, pur tuttavia rapide nella pratica e abbastanza affidabili (capaci cioè di minimizzare l’eventualità di sbaglio). Ne parleremo tra poco. L’osservazione comunque mostra quanto la teoria e la pratica possono essere distanti anche per P .

- AKS, pur risolvendo egregiamente in teoria il problema della primalità, non dà sviluppi significativi per l’altra questione, quella della decomposizione in fattori primi. In quest’ultimo ambito, procedure al più polinomiali sono ancora sconosciute (a meno che certi sviluppi della fisica quantistica non permettano di rivedere in modo sostanziale i concetti di calcolatore e computazione: ne accenneremo alla fine di questo libro).

In riferimento all’ultimo esempio (quello dei primi e dei composti) possiamo anche fare la seguente banale osservazione. Per come abbiamo impostato il problema, ci aspettiamo la risposta

- $SÌ$, se il dato input N è primo,
- NO , se N è composto.

L’accento positivo è sulla primalità. Ammettiamo adesso di invertire la domanda, e chiedere “ N è composto o no?”. La risposta adesso sarà

- $SÌ$, se N è composto,
- NO , se N è primo.

Non c’è da immaginare, però, che il cambio della domanda possa mutare i tempi di lavoro, rallentarli, o accelerarli. Infatti, i procedimenti per la primalità funzionano ancora, si tratta solo di invertire la conclusione: se si è risposto $SÌ/NO$ alla domanda se “ N è primo”, si dice $NO/SÌ$ se la domanda diventa “ N è composto”. Tutto questo è assolutamente ovvio. Così, in generale, possiamo anche porre, se vogliamo, la seguente definizione.

Definizione. coP è la classe dei problemi S su alfabeti A tali che $A^* - S$ (il complemento di S) è in P .

D’altra parte, per $w \in A^*$, $w \in A^* - S$ se e solo se $w \notin S$. Così $A^* - S$ ha risposta $SÌ/NO$ esattamente quando S ha risposta $NO/SÌ$. Dunque, banalmente, $coP = P$.

Nel seguito, per $S \subseteq A^*$, denoteremo spesso S^c il complementare $A^* - S$ di S .

Esercizio. Si provi che, se S è un insieme finito di parole su un alfabeto A , allora $S \in P$.

8.2 La classe NP

Ritorniamo all’esempio 2 del precedente paragrafo, quello relativo a $2SAT$. Accettiamo adesso come input insiemi finiti di clausole di lunghezza superiore a 2,

come

$$\{p_0P_1p_2P_3P_4, P_0p_1P_2p_3P_4, P_2p_4P_5p_6\},$$

per valutarne la soddisfacibilità. Il relativo problema si indica con la sigla *SAT*, (dall'inglese *satisfiable*, soddisfacibile) e generalizza *2SAT*. Ma l'algoritmo per *2SAT* e il relativo effetto domino adesso non valgono più perché sono ovviamente legati alla lunghezza 2 delle parole coinvolte; anzi, riesce difficile elaborare procedure alternative di soddisfacente velocità.

In effetti, quel che cerchiamo è una valutazione v di tutte le lettere coinvolte nelle parole in input, che assegna il valore 1 ad una lettera (maiuscola o minuscola) in ogni clausola (equivalentemente una parola che intersechi ogni clausola in almeno una lettera). Ora, se n sono le lettere coinvolte nelle nostre clausole,

- $p_0, p_1, \dots, p_{n-1}$ le relative versioni minuscole,
- $P_0, P_1, \dots, P_{n-1}$ le corrispondenti maiuscole,

le valutazioni v da indagare sono 2^n : ogni v può infatti assegnare, per ogni $i < n$, a p_i valore 0 oppure 1, e conseguentemente 1 o 0 ad P_i . Notiamo che l'indice n è parametro significativo della lunghezza dell'insieme finito di clausole che fa da input. Così una indagine su tutte le possibili v è da ritenersi almeno esponenziale nella lunghezza dell'input, e quindi è chiamata ad esaminare "troppi" casi.

Ammettiamo però di conoscere una valutazione v giusta (seppur esiste). Controllare che v funziona, rilevare cioè che ogni clausola dell'input ha una lettera cui v associa 1, non è significativamente più lungo che leggere l'input stesso. Così la relativa verifica, conoscendo v , è rapida rispetto alla lunghezza dell'input. Ricordiamo poi che le informazioni essenziali su v , cioè i suoi valori su $p_0, p_1, \dots, p_{n-1}$, possono sintetizzarsi in un'unica parola di lunghezza $\leq n$, come abbiamo visto nel precedente paragrafo. Per esempio $p_0p_1P_2$ può rappresentare la valutazione v per cui $v(p_0) = v(p_1) = 1, v(p_2) = 0$. Si noti che $p_0p_1P_2$ interseca le tre clausole dell'insieme proposto poche righe fa, $p_0P_1p_2P_3P_4$ in p_0 , $P_0p_1P_2p_3P_4$ in p_1 e $P_2p_4P_5p_6$ in P_2 .

La classe *NP* è quella che include i problemi che hanno questa stessa caratteristica di *SAT*, nel senso della seguente definizione ($l(w)$ denota qui e in seguito la lunghezza di un input w).

Definizione. *NP* è la classe dei problemi di decisione S su alfabeti finiti A per i quali esistono

- $S' \subseteq A^*, S' \in P$
- un polinomio p_S a coefficienti interi (e a valori positivi)

tali che, per ogni $w \in A^*$, $w \in S$ se e solo se esiste $y \in A^*$ per cui $(w, y) \in S'$ e $l(y) \leq p_S(l(w))$.

Esempi.

1. Come già detto, *SAT*, o anche *nSAT* (la sua variante ristretta ad input costituiti da insiemi finiti di clausole di n lettere) per $n \geq 3$, sono in *NP*. Altri esempi li accompagnano.

2. Consideriamo infatti *3COL* (il problema della 3-colorabilità di grafi finiti). Formalmente, *3COL* può intendersi come l'insieme dei grafi finiti (V, E) per cui esiste una 3-colorazione $c : V \rightarrow \{1, 2, 3\}$ tale che, per $v, w \in V$ e $(v, w) \in E$, $c(v) \neq c(w)$. Così *3COL* è la variante del problema *2COL* (già considerato e collocato in *P*) quando i colori ammessi sono 3.

Non è evidente che *3COL* stia ancora in *P*, l'algoritmo polinomiale di *2COL* è troppo legato all'ipotesi dei 2 colori. Del resto le possibili colorazioni c sono, per $|V| = n$, 3^n , tante quante le funzioni dagli n vertici di V ai 3 colori $\{1, 2, 3\}$; esplorarle una ad una è procedura esponenziale in n . Si può obiettare che, se una colorazione c funziona, tutte le colorazioni c' che si ottengono da c componendola con una permutazione dei colori 1, 2, 3, funzionano ancora. In altre parole, per concludere che $(V, E) \in 3COL$, non è tanto importante come si colora un dato vertice, se di 1, o di 2, o di 3, ma che i vertici adiacenti in E abbiano colori diversi. Così se c soddisfa

$$c(v) \neq c(v') \text{ per ogni scelta di } v, v' \in V \text{ con } (v, v') \in E,$$

e σ permuta 1, 2, 3, allora anche σc mantiene la proprietà

$$\sigma c(v) \neq \sigma c(v') \text{ per ogni scelta di } v, v' \text{ come sopra .}$$

Quindi potremmo limitare la nostra analisi trascurando le possibili σc al variare di σ , quando c è già stata controllata. Purtroppo le permutazioni su 3 oggetti sono $3! = 6$, e dunque l'osservazione riduce il numero dei casi da esplorare da 3^n a $\frac{3^n}{6}$, mantenendolo comunque al livello esponenziale.

D'altra parte, una singola colorazione c è individuata dai colori associati agli n vertici di V e cioè da una n -upla ordinata in $\{1, 2, 3\}$ (e dunque ha lunghezza direttamente collegata ad n). Se poi c funziona, verificarlo, cioè controllare che vertici distinti assumono colori distinti in c , è impegno che si limita ad una "lettura" (ad una osservazione complessiva) del grafo (V, E) , e quindi non ne eccede in modo significativo la lunghezza.

Ne deriva che $3COL \in NP$. Lo stesso vale per $mCOL$ (grafi colorabili a m colori) per ogni $m \geq 3$ come il lettore può formalizzare in dettaglio, se vuole.

3. Consideriamo ora equazioni in 2 incognite a coefficienti interi, come nell'esempio 3 del precedente paragrafo. Ammettiamo tuttavia grado 2, e comunque esaminiamo

- INPUT: un'equazione $ax^2 + by = c$ con a, b, c interi, $a \neq 0$,

e nuovamente cerchiamo

- OUTPUT: SÌ/NO secondo che l'equazione abbia o no soluzioni intere.

Così $x^2 = 3$ (cioè $a = 1, b = 0, c = 3$) ha risposta negativa perché 3 non è quadrato di nessun intero; $x^2 + 4y = 4$ (cioè $a = 1, b = 4, c = 4$) ha risposta positiva (perché risolta da $(\pm 2, 0)$, o da $(0, 1)$, e così via).

Il procedimento usato per le equazioni lineari (cioè sostanzialmente, l'algoritmo euclideo delle divisioni successive) è intrinsecamente legato al grado

1, e non può estendersi alla nuova situazione: varrebbe infatti a determinare l'esistenza di possibili soluzioni intere X', Y' per l'equazione lineare

$$aX' + bY' = c,$$

ma non a garantire che $X' = x^2$ si esprima come quadrato di un intero. Semmai possiamo ricordare quanto osservato nell'esempio 3 del paragrafo 8.1 e cioè che la lunghezza delle possibili soluzioni X', Y' è polinomialmente limitata rispetto a quella dei parametri a, b, c ; tramite X' , quindi, risulta polinomialmente limitata anche la lunghezza di un eventuale intero x per cui $X' = x^2$.

Ritorniamo allora alla nostra equazione di secondo grado $ax^2 + by = c$. Dalle precedenti considerazioni deduciamo che le soluzioni intere, se esistono, hanno lunghezza limitata da quella dei coefficienti. Allora calcoliamo quanti sono gli interi X o Y di una data lunghezza $l > 1$ in base 2. Il loro numero è 2^{l-1} perché essi si compongono di l cifre, ciascuna delle quali può assumere i valori 0, 1, salvo la prima che è ovviamente 1. Per esempio, i numeri di 4 cifre in base 2

$$1000, 1001, \dots, 1111$$

sono $2^3 = 8$.

Possiamo cercare le possibili soluzioni intere della nostra equazione prima fissandone la lunghezza (limitata!) e poi procedendo per tentativi, coinvolgendo cioè ogni possibile coppia (X, Y) di lunghezza lecita l ; ma si tratta di un meccanismo esponenzialmente lungo essendo 2^{l-1} i casi da esaminare per X o per Y , fissata l . Pur tuttavia, se la soluzione (X, Y) è nota, verificarla, controllare cioè $aX^2 + bY = c$, è compito che si può svolgere con rapidità.

In conclusione, $ax^2 + by = c$ ha radici intere se e solo se esistono X, Y di lunghezza polinomialmente limitata da quella di a, b, c per cui $aX^2 + bY = c$. Il problema è quindi in NP .

In previsione di sviluppi futuri, allunghiamo ancora la nostra lista di esempi in NP .

4. Consideriamo adesso grafi finiti $G = (V, E)$. Introduciamo le seguenti definizioni: un sottoinsieme V_0 di V si dice

- *indipendente* se, per ogni scelta di $v, v' \in V_0$, $(v, v') \notin E$;
- un *ricoprimento di vertici* se, per ogni scelta di $v, v' \in V$ con $(v, v') \in E$, $v \in V_0$ o $v' \in V_0$;
- una *cricca* (in inglese, *clique*) se, per ogni scelta di $v, v' \in V_0$ con $v \neq v'$, $(v, v') \in E$.

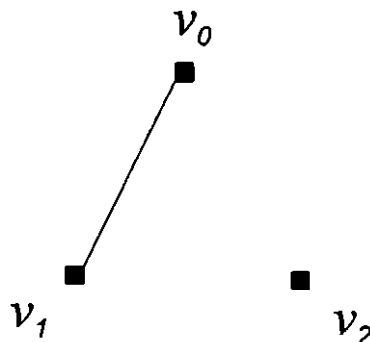
Ricordiamo che la nostra definizione di grafo esclude $(v, v) \in E$ per $v \in V$. Ci interessano i seguenti tre problemi.

a) IS = problema dell'insieme indipendente.

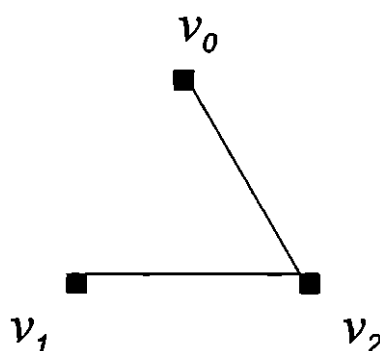
- INPUT: (V, E) grafo finito, k intero positivo minore o uguale della cardinalità $|V|$ di V .

- OUTPUT: *SÌ/NO* secondo che (V, E) abbia un sottoinsieme indipendente V_0 di k vertici.
- b) VC = problema del ricoprimento dei vertici.
 - INPUT: (V, E) , k come sopra.
 - OUTPUT: *SÌ/NO* secondo che (V, E) abbia un ricoprimento di k vertici.
- c) $CLIQUE$ = problema della cricca.
 - INPUT: (V, E) , k come sopra.
 - OUTPUT: *SÌ/NO* secondo che (V, E) contenga una cricca di k vertici.

I tre problemi sono, in realtà, lo stesso, nel senso che segue. Notiamo anzitutto che in un grafo (V, E) un sottoinsieme V_0 è indipendente se e solo se $V - V_0$ è un ricoprimento (il lettore può verificare il motivo con un facile **esercizio**). Inoltre V_0 è indipendente in (V, E) se e solo se V_0 è una cricca nel grafo che si ottiene da (V, E) mantenendone i vertici, ma collegandone due (distinti) tra loro se e solo se non sono uniti da lati in E . Per esempio v_0, v_2 formano un insieme indipendente nel seguente grafo (V, E)



e una cricca nel grafo che se ne ottiene nel modo sopra descritto.



Dunque procedimenti per riconoscere un insieme indipendente si adattano a controllare anche i ricoprimenti dei vertici, o le cricche. Anzi la traduzione di un'istanza di IS a una di VC , o di $CLIQUE$, o viceversa, avviene in modo polinomialmente limitato dalle dimensioni dei grafi coinvolti.

Non è tuttavia chiaro se IS (o VC o $CLIQUE$) siano in P . In effetti, i sottoinsiemi con k elementi di un insieme V con n elementi sono

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

(talora “troppi” rispetto a n e k ; infatti, quando k si accosta a $\frac{n}{2}$, $\binom{n}{k}$ approssima 2^k).

Tuttavia, ciascuno di essi ha, appunto, $k \leq n$ elementi e il controllo di un eventuale V_0 , cioè la verifica che i suoi vertici lo rendono indipendente perché sono comunque scollegati (oppure un ricoprimento perché capaci di coinvolgere ogni lato in E , oppure ancora una cricca perché, se distinti, congiunti da un opportuno lato), si può svolgere semplicemente osservando il grafo (V, E) , e più direttamente V_0 .

Così IS , VC , $CLIQUE$ sono in NP .

5. *KNAPSACK*

Il Problema dello Zaino, o *KNAPSACK* secondo la terminologia ufficiale e la lingua inglese, è la questione che in genere affligge la notte prima della partenza per le vacanze: abbiamo uno zaino di volume V da riempire e vogliamo infilarci n oggetti ognuno con un determinato volume: ciascuno di essi ci pare, infatti, indispensabile per il buon esito delle nostre ferie. Ci chiediamo se possiamo sceglierne alcuni in modo da colmare esattamente lo zaino. Assumiamo poi che lo zaino non sia rigido e si presti ad essere deformato in ogni modo possibile, così che la soluzione dipenda solo dal volume degli oggetti coinvolti, e non dalla loro figura. Formalmente abbiamo

- INPUT: uno zaino di volume V e n oggetti $0, 1, \dots, n-1$, rispettivamente di volume $v(0), \dots, v(n-1)$; assumiamo $n, V, v(0), \dots, v(n-1)$ interi positivi;
- OUTPUT: SÌ/NO secondo che esista un sottoinsieme I di $\{0, \dots, n-1\}$ tale che

$$V = \sum_{i \in I} v(i).$$

L'esperienza comune ci insegna che la questione è tutto meno che semplice: in genere, la sera prima delle ferie, i tentativi si sprecano e le ore volano prima che gli oggetti siano finalmente sistemati, se mai lo possono essere in modo soddisfacente. Del resto, la difficoltà ha un suo logico fondamento e non è evidente se *KNAPSACK* sia in P , visto che le possibili combinazioni di oggetti da infilare nello zaino sono tante quanti i sottoinsiemi di $\{0, 1, \dots, n-1\}$, e cioè 2^n . Non v'è dubbio però che il problema stia in NP : se conosciamo l'opportuno insieme I , verificare che, appunto, $V = \sum_{i \in I} v(i)$ (in relazione agli input $n, V, v(0), \dots, v(n-1)$) è controllo rapido e di poca fatica.

Dopo tanti esempi, ecco una semplice osservazione che collega la classe P considerata nel precedente paragrafo e la nuova classe NP . Ricordiamo che P include tutti e soli i problemi che hanno algoritmo “rapido” di decisione; invece, in base alle osservazioni e agli esempi di questo paragrafo, NP può essere considerata come la classe dei problemi che hanno algoritmo “rapido” di verifica della soluzione. È naturale ammettere che trovare una soluzione sia più difficile che verificarla. In altre parole, si ha:

Teorema 8.2.1 $P \subseteq NP$.

Dimostrazione. Sia $S \in P$. Guardiamo alla definizione di NP e poniamo

$$S' = S, p_S = 1 \text{ (anzi } y = \emptyset \text{ per ogni } w).$$

S', p_S testimoniano $S \in P$. □

Come nel caso di P e coP , possiamo ora introdurre:

Definizione. $coNP$ è la classe dei problemi S su alfabeti finiti A tali che il complementare $S^c = A^* - S$ di S è in NP .

Stavolta, però, non è affatto chiaro che $coNP = NP$. Il lettore può cercare di intuire le difficoltà che emergono a questo proposito, oppure attendere il paragrafo 8.9 dove le descriveremo in maggior dettaglio.

8.3 Un'altra caratterizzazione di NP: le macchine di Turing non deterministiche

Conosciamo già le macchine di Turing M su un alfabeto A . Ricordiamone ancora la definizione. Una MdT M su A nell'insieme degli stati Q può essere intesa come una sequenza di istruzioni (dettate dalla funzione di transizione δ di M) che riguardano possibili coppie (q, a) con $q \in Q$ e con a elemento di A , oppure simbolo muto $\star$, e associa a ciascuna di esse una terna univocamente determinata

$$(q', a', \pm 1)$$

dove $q' \in Q$, $a' \in A \cup \{\star\}$, -1 (spostarsi a sinistra), $+1$ (spostarsi a destra) sono comandi di spostamento.

Quel che conviene nuovamente ribadire è che, ad ogni data coppia (a, q) , o non è associata alcuna terna di istruzioni o, se una terna è associata, essa è unica.

Ammettiamo adesso di far cadere quest'ultima condizione. Otteniamo allora il concetto di *macchina di Turing non deterministica* M su A nell'insieme degli stati Q , da intendersi come insieme di 5-uple

$$(q, a, q', a', \pm 1)$$

con $q, q' \in Q$, $a, a' \in A \cup \{\star\}$, ± 1 come sopra. Ne abbiamo già parlato nella prima parte. Sottolineiamo ancora che la novità è che ad una data coppia (q, a) possono corrispondere più possibili istruzioni tra le quali scegliere quella da eseguire.

In effetti una MdT non deterministica M su A può talora svolgere sullo stesso input più computazioni, anche di esito opposto.

Ovviamente non è vietato, ed è anzi ammesso, che la MdT M abbia al più una istruzione su ogni coppia. Così le “vecchie” macchine di Turing, corrispondenti a questo comportamento, rientrano tra quelle non deterministiche ora introdotte. Per distinguere il loro preciso rigore, le chiameremo *macchine di Turing deterministiche*. Occorre però accogliere con molta attenzione questa terminologia. le

MdT deterministiche non sono quelle che non sono non deterministiche, formano anzi una sottoclasse della classe di tutte le macchine di Turing non deterministiche, caratterizzata, appunto, dalla condizione che ogni possibile coppia (q, a) ammette al più una terna di istruzioni che la riguardano.

È anche da dire che, come già mostrato nella prima parte del libro, ogni MdT non deterministica M su A si può accompagnare con una MdT M' deterministica su A (eventualmente in un maggior numero di stati) con la seguente proprietà: per ogni $w \in A^*$,

$$M' \text{ converge su } w$$

se e solo se

esiste una computazione convergente di M su w .

Sostanzialmente, M' segue in parallelo tutte le possibili computazioni di M su w , accompagnandole passo per passo, e si arresta non appena una di esse si arresta. Scrivere le istruzioni dettagliate di M' è esercizio pesante, ma possibile. Ovviamente i tempi di lavoro di M' sono destinati ad essere più lunghi di quelli di M .

Possiamo ora dare una nuova caratterizzazione di NP , riferita proprio alle MdT non deterministiche. In questo contesto, la caratterizzazione richiama il modo in cui P è definito (tramite le MdT deterministiche).

Teorema 8.3.1 *Sia S un insieme di parole su un alfabeto finito A . Allora $S \in NP$ se e solo se ci sono una MdT non deterministica M su A e un polinomio a coefficienti interi (e valori positivi) q_S , tali che, per ogni $w \in A^*$,*

- *quando $w \in S$, c'è almeno una computazione di M su w che converge impiegando al più $q_S(l(w))$ passi;*
- *quando $w \notin S$, nessuna computazione di M su w converge.*

Dimostrazione. Sia dapprima $S \in NP$. Allora esistono, in base alla definizione che conosciamo, una MdT deterministica M_S su A e due polinomi p_S, t_S tali che, per ogni $w \in A^*$,

- $w \in S$ se e solo se esiste $y \in A^*$ per cui $l(y) \leq p_S(l(w))$ e M_S converge su (w, y) ;
- su ogni possibile input, M_S diverge oppure converge in tempo al più polinomiale limitato da t_S rispetto alla lunghezza dell'input stesso.

Costruiamo allora una MdT non deterministica M su A come segue: per ogni $w \in A^*$,

- M scrive prima tutte le possibili stringhe y con $l(y) \leq p_S(l(w))$,
- per ogni y M simula poi M_S su (w, y) .

Il non determinismo deriva dal fatto che M svolge più possibili computazioni, una per ogni testimone y . Comunque è chiaro che un elemento $w \in A^*$ sta in S

se e solo se, per un y opportuno, c'è una computazione di M su w che converge, impiegando al più $t_S(l(w)) + p_S(l(w))$ passi.

Viceversa, ammettiamo che esistano M , q_S come enunciato. Per $w \in A^*$, M opera su w come segue: se $w \in S$, c'è almeno una computazione di M su w che converge in al più $q_S(l(w))$ passi; altrimenti, per $w \notin S$, ogni computazione di M diverge. Ricordiamo poi, dalla prima parte del libro, che ogni computazione di una MdT su un assegnato input può essere opportunamente codificata da un intero positivo, o anche da una parola y su A . Questo si applica alle computazioni di M su una certa w , anche a quelle di lunghezza $\leq q_S(l(w))$. La lunghezza $l(y)$ di y può essere poi opportunamente limitata da quella di w e dal numero di passi $q_S(l(w))$ della computazione corrispondente. Possiamo allora usare le codifiche y delle computazioni di M di lunghezza $\leq q_S(l(w))$ su una generica w per organizzare il seguente algoritmo deterministico M_S . Data $w \in A^*$, consideriamo le coppie (w, y) in A^* con y come sopra e le proponiamo come input. La MdT M_S si comporta come segue:

- se M converge almeno una volta su w in al più $q_S(l(w))$ passi, e y è la codifica di una tale computazione, M_S converge su (w, y) ;
- altrimenti, M_S diverge su (w, y) .

È esercizio di pazienza scrivere in dettaglio la definizione di M_S . Risulta comunque chiaro che $w \in S$ se e solo se c'è $y \in A^*$ tale che M_S converge su (w, y) . Inoltre la lunghezza dei testimoni y dipende polinomialmente, tramite M e q_S , da quella di w . Infine il tempo di lavoro di M_S si può anch'esso limitare polinomialmente rispetto a $l(w)$. $\square$

Siccome le MdT deterministiche sono particolari MdT deterministiche, si ha dal precedente teorema una nuova conferma che $P \subseteq NP$.

8.4 Il problema $P = NP$

Possiamo domandarci se vale addirittura

$$P = NP$$

(dunque se $NP \subseteq P$).

Commenti.

1. P è, come spesso ricordato, la classe dei problemi che ammettono algoritmo rapido di decisione (dove “rapido” è da intendersi come “al più polinomiale”, ai sensi della tesi di Edmonds-Cook-Karp). Se ci rifacciamo poi alla definizione originaria di NP , quest'ultima può interpretarsi sinteticamente come la classe dei problemi S che hanno algoritmo rapido di verifica della decisione: dato un input w , il testimone y può garantire infatti in modo deterministico e rapido che $w \in S$ perché $(w, y) \in S'$.

La conclusione che $P \subseteq NP$ deriva allora dal fatto che tempi rapidi di decisione implicano tempi (anche più) rapidi di verifica.

Affermare $P = NP$ significa in questa prospettiva che, per ogni problema S , se c'è una procedura rapida di verifica per S , ce n'è una (magari più lenta eppur ancora) rapida (perché al più polinomiale nella lunghezza dell'input) di decisione per S .

2. Riferiamoci adesso alla caratterizzazione di NP fornita nell'ultimo paragrafo. $S \in NP$ se e solo se c'è una MdT non deterministica M su A tale che, per ogni $w \in A^*$,

- se $w \in S$, M_S converge su w almeno una volta con un numero di passi limitato in funzione di $l(w)$ da un polinomio a coefficienti interi q_S che dipende solo da S e non da w ;
- se $w \notin S$, M_S diverge su w in ogni caso.

Notiamo che, se consideriamo MdT deterministiche (capaci dunque di un'unica computazione su ogni input w), la stessa caratterizzazione definisce i problemi $S \in P$. La domanda $P = NP$ può dunque così riformularsi. Supponiamo che S ammetta una MdT non deterministica M_S come sopra. È possibile costruire una MdT deterministica che accetta S in tempo al più polinomiale, limitato da un opportuno polinomio q'_S , magari di grado maggiore di quello di q_S ? Tra l'altro, sappiamo che la MdT non deterministica M_S può essere simulata nelle sue computazioni da una MdT deterministica M'_S (con tempi di lavoro più lunghi). Ci domandiamo se M'_S può essere scelta in modo da avere ancora complessità al più polinomiale.

Ora, è evidente che, se si trova un problema che sta in NP , ma non può appartenere a P (per esempio, se uno dei numerosi esempi proposti in NP nei precedenti paragrafi manifesta questa proprietà), allora $P \neq NP$.

Mostrare, invece, l'uguaglianza $P = NP$ sembra richiedere, almeno a priori, argomenti teorici generali che mostrino che ogni problema in NP ha algoritmo rapido di decisione che lo colloca in P .

Riferirsi ad un particolare esempio S in NP e pretendere che, se $S \in P$, allora P e NP coincidono sembra grossolano errore di logica elementare. Tuttavia esistono alcuni problemi S in NP che hanno questa capacità di rappresentare tutti gli altri, nel senso appena descritto: si chiamano *problemi NP-completi*, e si introducono nel modo che segue.

8.5 Problemi NP-completi

Iniziamo con una

Definizione. Siano S, S' due problemi su alfabeti A, A' rispettivamente. Diciamo che S' si *riduce in tempo polinomiale* a S e scriviamo $S' \leq_p S$ se e solo se c'è una funzione f da A'^* in A^* che si computa in tempo al più polinomiale rispetto alla lunghezza dell'input e soddisfa, per ogni $w \in A'^*$,

$$w \in S' \text{ se e solo se } f(w) \in S.$$

Esercizio. Si provi che la relazione $\leq_p$ è riflessiva e transitiva (dunque una relazione di riducibilità); $\leq_p$ è anche antisimmetrica? (La risposta a quest'ultima domanda sarà evidente tra poche righe).

Esempi.

1. Nel precedente paragrafo 1, esempio 2, abbiamo visto che $2COL \leq_p 2SAT$ tramite la funzione $G \mapsto I(G)$.
2. $IS, VC, CLIQUE$ sono l'uno riducibile all'altro tramite $\leq_p$ (si ricordi quanto osservato nel paragrafo 8.2, esempio 4).

Adesso diciamo:

Definizione. Un problema S è

- NP -arduo se, per ogni $S' \in NP$, $S' \leq_p S$,
- NP -completo se è NP -arduo e sta in P .

È semplice osservare

Proposizione 8.5.1 *Sia S un problema NP -completo. Allora $S \in P$ se e solo se $P = NP$.*

Dimostrazione. È chiaro che, se $P = NP$, allora $S \in P$.

Viceversa, supponiamo $S \in P$, così S ha un algoritmo deterministico di decisione che lo accetta in tempo al più polinomiale rispetto alla lunghezza dell'input. Per ogni $S' \in NP$, dobbiamo trovare un analogo algoritmo per S' . D'altra parte $S' \leq_p S$, cioè esiste una procedura deterministica f che traduce in tempo polinomiale parole dell'alfabeto A' di S' in parole dell'alfabeto A di S in modo tale che le parole di S' corrispondono esattamente a quelle di S . L'algoritmo cercato per S' si ottiene allora combinando quest'ultima procedura e l'algoritmo per S : per $w \in A'^*$, si computa $f(w)$ e si controlla se $f(w) \in S$ o no. Corrispondentemente si deduce che $w \in S'$ o no. $\square$

Così un problema NP -completo S può rappresentare adeguatamente tutta la classe NP a proposito della questione $P = NP$. Ovviamente, se $S \notin P$, allora $P \neq NP$. Ma, sorprendentemente, se $S \in P$, allora $P = NP$. Per usare un famoso riferimento letterario, vale tra i problemi NP -completi il classico patto dei moschettieri di Dumas: “uno per tutti, e tutti per uno”.

Va però mostrato che problemi NP -completi esistono davvero. Eccone allora un esempio (assai artificiale e tecnico, ma comunque sufficiente a confermare che la nozione ha senso).

Proposizione 8.5.2 *Ci sono problemi NP -completi.*

Dimostrazione. Consideriamo il seguente problema $\mathcal{U}$. Gli input di $\mathcal{U}$ sono terne costituite da

- una MdT M su $\{0, 1\}$,
- una parola $w \in \{0, 1\}^*$,
- due interi positivi l, t

(tutti proposti come parole di un opportuno alfabeto, per esempio come numeri naturali): l'output è *SI* o *NO* secondo che esista una parola $y \in \{0, 1\}^*$ tale che $l(y) \leq l$ e M converge sull'input (w, y) in al più t passi.

La definizione stessa di NP assicura che $\mathcal{U} \in NP$. Resta allora da provare che $\mathcal{U}$ è NP -arduo. Sia $S' \in NP$. Senza perdita di generalità possiamo supporre che l'alfabeto di S' sia $\{0, 1\}$. Dunque esistono una MdT deterministica M_S su $\{0, 1\}$ e polinomi q_S, t_S tali che, per ogni $w \in \{0, 1\}^*$,

$$w \in S$$

se e solo se

$$\begin{aligned} &\text{esiste } y \in \{0, 1\}^* \text{ tale che } l(y) \leq q_S(l(w)) \\ &\text{e } M_S \text{ accetta } (w, y) \text{ in al più } t_S(l(w) + l(y)) \text{ passi.} \end{aligned}$$

Poniamo allora, per $w \in \{0, 1\}^*$,

$$f(w) = (M_S, w, t_S(l(w) + q_S(l(w))), q_S(w)).$$

È facile controllare che f riduce S' ad $\mathcal{U}$, come richiesto. In conclusione, $\mathcal{U}$ è NP -completo. $\square$

Quindi ci sono problemi NP -completi. Il prossimo paragrafo ce ne darà esempi meno artefatti e più "concreti".

8.6 Il teorema di Cook-Levin

I concetti di riduzione $\leq_p$ e di NP -completezza furono introdotti nel 1971 da Cook e in modo indipendente negli stessi anni da Levin. Cook e Levin proposero anche i primi esempi naturali di problemi NP -completi (in aggiunta al troppo artificiale $\mathcal{U}$). SAT è il primissimo di questi esempi, capostipite di moltissimi altri. Il presente paragrafo è dedicato proprio alla dimostrazione del teorema di Cook e Levin che afferma la NP -completezza di SAT .

Teorema (Cook-Levin) 8.6.1 SAT è NP -completo.

Dimostrazione. Già sappiamo che $SAT \in NP$. Dobbiamo allora mostrare che SAT è NP -arduo, cioè che ogni problema $S' \in NP$ si riduce in tempo polinomiale a SAT , $S' \leq_p SAT$: esiste dunque una funzione f che traduce in tempo polinomiale parole w sull'alfabeto A' di S' in insiemi $f(w)$ di clausole dell'alfabeto di SAT in modo tale che, per ogni $w \in A'^*$,

$$w \in S' \text{ se e solo se } f(w) \in SAT.$$

Consideriamo $S' \in NP$; esistono una MdT non deterministica M' su A' e un polinomio p' a coefficienti interi tali che, per ogni $w \in A'^*$,

- se $w \in S'$, allora c'è almeno una computazione di M' su w che converge e impiega al più $p'(l(w))$ passi;
- se $w \notin S'$, ogni computazione di M' su w diverge.

La strategia che seguiremo sarà quella di scrivere, per ogni w , un insieme di clausole $f(w)$ che cerca proprio di seguire il lavoro di M' su w e, in particolare, è soddisfatto da qualche valutazione v se e solo se $w \in S'$, cioè se e solo se qualche computazione di M' su w ha buon esito. Per questo converrà prima fissare opportunamente il contesto e la notazione.

Possiamo anzitutto convenire che A' si componga dei simboli $a_1, \dots, a_m$ e che a_0 rappresenti il simbolo $\star$. Possiamo poi supporre che la MdT M' ammetta gli stati $q_0, \dots, q_h$ e, in particolare, si arresti se e solo se entra nello stato q_h .

Consideriamo adesso una parola w su A' . Sia $n = l(w)$. Fissiamo poi una computazione di M' su w ; per $w \in S'$, scegliamo in particolare una computazione che converge in al più $p'(n)$ passi. Notiamo che una computazione di $p'(n)$ passi riguarda al più $2p'(n) + 1$ quadri del nastro, $p'(n)$ a sinistra e $p'(n)$ a destra del quadro di partenza. Infatti le istruzioni di M' possono chiedere di spostarsi costantemente a sinistra, oppure costantemente a destra nei $p'(n)$ passi interessati, nei quali casi i quadri coinvolti saranno i $p'(n)$ a sinistra o i $p'(n)$ a destra di quello di partenza; altrimenti le istruzioni porteranno l'indice di M' talora a destra, talora a sinistra, e solo alcuni dei $2p'(n) + 1$ quadri in questione resteranno toccati dalla computazione, parte a destra e parte a sinistra. Numeriamo allora per semplicità i $2p'(n) + 1$ quadri coinvolti, da 0 fino a $2p'(n)$ procedendo da sinistra verso destra; così il quadro centrale, quello di partenza, riceve il numero $p'(n)$.

Per costruire $f(w)$ e le sue clausole, ci interessano alcune lettere minuscole dell'alfabeto di SAT . Per convenienza le indichiamo

$$c_{i,j,t}, \quad s_{r,t}, \quad d_{i,t}$$

dove

- $i \leq 2p'(n)$ si riferisce ai quadri della computazione,
- $j \leq m$ ai simboli di A' ,
- $r \leq h$ agli stati di M' ,
- $t \leq p'(n)$ ai passi della computazione.

Ovviamente ci possono servire anche le corrispondenti maiuscole

$$C_{i,j,t}, \quad S_{r,t}, \quad D_{i,t}.$$

Tra le tante possibili valutazioni di queste lettere, ne segnaliamo una, V , quella che meglio anticipa i nostri propositi e, in riferimento alla computazione di M' su w precedentemente scelta, assume

$V(c_{i,j,t}) = 1$ se e solo se al passo t sul quadro i c'è scritto a_j ;

$V(s_{r,t}) = 1$ se e solo se al passo t M' è nello stato q_r ;

$V(d_{i,t}) = 1$ se e solo se al passo t M' esamina il quadro i .

Chiameremo V la *valutazione standard*.

Notiamo che le lettere minuscole sin qui coinvolte sono $O(p^2)$, visto che m e h dipendono solo da A' e da M' e dunque sono costanti rispetto a w e a n . Quindi, se g indica il grado di p' , le lettere complessivamente necessarie per il nostro lavoro sono $O(n^{2g})$.

In riferimento alla loro rappresentazione con i simboli $p, |$ illustrata all'inizio dell'esempio 2 nel paragrafo 1, possiamo dire che la massima lunghezza di una lettera, e quindi il tempo massimo necessario per scriverla, è $O(n^{2g})$.

Ci serve introdurre un'ulteriore notazione. Per $p_0, p_1, \dots, p_s$ lettere minuscole, $I_s(p_0, \dots, p_s)$ rappresenta l'insieme delle clausole

- $p_0 \cdots p_s$,
- $P_i P_j$ per $i < j \leq s$.

Ricordiamo che una valutazione v soddisfa $I_s(p_0, \dots, p_s)$ se e solo se in ciascuna delle clausole elencate v assegna il valore 1 ad almeno una lettera; quindi

$$v(p_0) = 1 \text{ o } v(p_1) = 1 \text{ o } \cdots \text{ o } v(p_s) = 1,$$

ma, per ogni scelta di $i < j \leq s$, $v(p_i) = 0 \text{ o } v(p_j) = 0$; in altre parole, v soddisfa una e una sola delle lettere $p_0, \dots, p_s$.

Passiamo adesso a scrivere le clausole di $f(w)$. Le suddividiamo in 7 gruppi, per ciascuno dei quali indichiamo anche in quali casi la valutazione standard V soddisfa le clausole elencate, e quanti passi sono richiesti per scrivere queste clausole. Come già sottolineato, $f(w)$ cerca di accompagnare e descrivere la computazione di M' su w scelta in precedenza. Ricordiamo che $t \leq p'(n)$.

I primi quattro gruppi di clausole descrivono (almeno in riferimento alla valutazione standard V) il comportamento di una generica macchina di Turing, e il suo modo di computare.

1. Per ogni $t \leq p'(n)$, poniamo

$$A_t = I_{2p'(n)}(d_{0,t}, \dots, d_{2p'(n),t}).$$

Così V soddisfa A_t se e solo se, al passo t della computazione di M' su w precedentemente scelta, M' esamina uno e uno solo dei quadri $0, \dots, 2p'(n)$. Siccome ciascuna lettera richiede $O(n^{2g})$ passi per essere scritta, il tempo richiesto per scrivere A_t (e quindi tutte le coppie di lettere maiuscole per $i < j \leq 2p'(n)$) è $O(n^{4g})$.

2. Per $t \leq p'(n)$, sia B_t l'unione per $i \leq 2p'(n)$ degli insiemi $I_m(c_{i,0,t}, \dots, c_{i,m,t})$. V soddisfa B_t se e solo se, al tempo t della computazione di M' su w , ogni quadro contiene uno e un solo simbolo tra $a_0, \dots, a_m$. Il numero di passi richiesto per scrivere B_t è $O(n^{3g})$: $O(n^{2g})$ è infatti il tempo necessario per ogni singola lettera, dobbiamo però successivamente considerare l'unione per $i \leq 2p'(n)$. Invece m non dipende da n , e dunque la costruzione dei singoli $I_m(c_{i,0,t}, \dots, c_{i,m,t})$ non determina ulteriori complicazioni.

3. Ancora per $t \leq p'(n)$, consideriamo

$$C_t = I_h(s_{0,t}, \dots, s_{h,t}).$$

V soddisfa C_t se e solo se, al tempo t della sua computazione su w , M' si trova in uno e un solo stato $q_0, \dots, q_h$. Il numero di passi richiesto per scrivere C_t è $O(n^{2g})$.

4. Per $t < p'(n)$, D_t si compone, per $i < 2p'(n)$, $j \leq m$, delle clausole

$$c_{i,j,t}C_{i,j,t+1}d_{i,t},$$

$$C_{i,j,t}c_{i,j,t+1}d_{i,t}.$$

Un attimo di riflessione mostra che V soddisfa D_t se e solo se nella computazione di M su w , dal passo t al passo $t+1$, solo il simbolo sul quadro che M' considera al passo t può cambiare. Infatti, per i fissato, V soddisfa $c_{i,j,t}C_{i,j,t+1}d_{i,t}$ e $C_{i,j,t}c_{i,j,t+1}d_{i,t}$ al variare di $j \leq m$ se e solo se, per ogni $j \leq m$,

$$V(c_{i,j,t}) = V(c_{i,j,t+1}) = 1,$$

o

$$V(c_{i,j,t}) = V(c_{i,j,t+1}) = 0,$$

cioè il quadro i non cambia simbolo da t a $t+1$, oppure $V(d_{i,t}) = 1$, ovvero i è il quadro in esame al passo t .

Procedendo come prima si vede che il numero di passi richiesto per scrivere D_t è $O(n^{3g})$.

5. Sia $t < p'(n)$. Sia poi

$$U = (q_r, a_j; q_{r'}, a'_{j'}, \varepsilon)$$

una quintupla in M ; quindi $j, j' \leq m$, $r, r' \leq h$, $\varepsilon = \pm 1$. Per $t \leq p'(n)$ consideriamo l'insieme $I(U, t)$ delle seguenti clausole

$$D_{i,t}C_{i,j,t}S_{r,t}d_{i+\varepsilon,t+1},$$

$$D_{i,t}C_{i,j,t}S_{r,t}c_{i,j',t+1},$$

$$D_{i,t}C_{i,j,t}S_{r,t}s_{r',t},$$

per $i \leq 2p'(n)$. Notiamo che la valutazione V soddisfa $I(U, t)$ se e solo se, per ogni $i \leq 2p'(n)$, $V(d_{i,t}) = 0$ o $V(c_{i,j,t}) = 0$ o $V(s_{r,t}) = 0$ o $V(d_{i+\varepsilon,t+1}) = V(c_{i,j',t+1}) = V(s_{r',t}) = 1$, equivalentemente se e solo se, da $V(d_{i,t}) = V(c_{i,j,t}) = V(s_{r,t}) = 1$, segue $V(d_{i+\varepsilon,t+1}) = V(c_{i,j',t+1}) = V(s_{r',t}) = 1$; in altre parole V soddisfa $I(U, t)$ se e solo se, per $i \leq 2p'(n)$, ammesso che M' al passo t della computazione su w esamini il quadro i , vi legga a_j e sia nello stato S , allora al passo $t+1$ M' passa al quadro $i+\varepsilon$, scrive $a_{j'}$ nel quadro i e va nello stato $q_{r'}$, in conclusione esegue U .

Non è difficile a questo punto adoperare i vari $I(U, t)$ per costruire, per $t < p'(n)$, un insieme E_t di clausole tali che V soddisfa E_t se e solo se il passo t

della computazione su w corrisponde, appunto, all'esecuzione delle istruzioni di qualche 5-upla U in M' . Si verifica facilmente che il costo di scrivere le clausole di un singolo $I(U, t)$ è $O(n^{3g})$, e tale resta per E_t .

Adesso cerchiamo di esprimere il fatto che l'input della computazione è w .

6. Sia F l'insieme delle clausole

$$\begin{aligned} & d_{p'(n),0}, \\ & s_{0,0}, \\ & C_{i,0,0} \text{ per } i < p'(n); \\ & C_{i,j(i),0} \text{ per } p'(n) \leq i \leq 2p'(n), \end{aligned}$$

dove $j(i)$ è definito come segue:

- $a_{j(i)}$ è lo $(i - p'(n))$ -simbolo di w se $i - p'(n) < n$,
- $j(i)$ è 0, cioè $a_{j(i)} = a_0 = \star$, altrimenti.

V soddisfa F se e solo se, al tempo 0 della computazione, M' guarda il quadro centrale (quello di partenza), è nello stato q_0 e l'input sul nastro è w (infatti la porzione di nastro a sinistra del quadro centrale è vuota, mentre quella a destra contiene, appunto, w).

Il numero di passi necessari per scrivere le clausole di F è $O(\pi^{3g})$.

7. Finalmente, sia G l'insieme dell'unica clausola

$$s_{h,0}s_{h,1} \cdots s_{h,p'(n)}.$$

V lo soddisfa se e solo se c'è $t \leq p'(n)$ tale che M' entra nello stato q_h al passo t della computazione, e dunque se e solo se M' si arresta entro il passo $p'(n)$. Il costo di scrivere G è $O(n^{3g})$.

Come preannunciato, $f(w)$ è l'unione degli insiemi in

- A_t, B_t, C_t per $t \leq p'(n)$,
- D_t, E_t per $t < p'(n)$,
- F, G .

Scrivere $f(w)$ ha costo complessivo $O(n^{5g})$ ed è dunque polinomiale in $n = l(w)$. Infatti il costo massimo di $A_t, B_t, C_t, D_t, E_t, F, G$ è $O(n^{4g})$, ma la composizione di w' richiede l'unione di A_t, B_t, C_t per $t \leq p'(n)$ e quella di D_t, E_t per $t < p'(n)$, il che porta la spesa complessiva a $O(n^{5g})$.

Finalmente, $w \in S'$ se e solo se $f(w) \in SAT$. Infatti, se $w \in S'$, allora M' converge su w in al più $p'(n)$ passi e dunque V soddisfa $f(w)$, e $f(w) \in SAT$. Viceversa, sia $f(w) \in SAT$, allora c'è una valutazione v che soddisfa $f(w)$. Si possono usare i valori

$$v(c_{i,j,t}), v(s_{\tau,t}), v(d_{i,t})$$

(per $i \leq 2p'(n)$, $j \leq m$, $r \leq h$ e $t \leq p'(n)$) per simulare una possibile computazione di una MdT nel modo già seguito per la valutazione standard; per esempio, interpretiamo

$$v(c_{i,j,t}) = 1$$

nel senso che il quadro i contiene a_j al tempo t della computazione. Il fatto che v soddisfa A_t , B_t , C_t , D_t al variare di t ci dice che, in questo modo, si definisce davvero una MdT, e dall'ulteriore condizione che v soddisfa E_t per $t < f(n)$ deduciamo che la MdT che computa opera esattamente come M' ; l'input è proprio w , perché v soddisfa F ; la computazione converge perché v soddisfa G . In definitiva, $w \in S'$.

Questo chiude la dimostrazione del teorema di Cook e Levin. □

8.7 Altri problemi NP-completi

Come già detto, *SAT* è storicamente il primo esempio naturale di problema *NP*-completo, ma non è certamente l'unico, anzi, in un certo senso, è il capostipite di una lunga progenie di esempi. Già nel 1972, dunque un anno dopo il teorema di Cook-Levin, Karp propose una lista di 21 problemi *NP*-completi S . La strategia usata per riconoscerli tali è meccanica e ripetitiva. Va infatti provato anzitutto che $S \in NP$, poi che S è *NP*-arduo. Relativamente al secondo punto basta mostrare che $SAT \leq_p S$ (oppure che qualche altro problema di cui è nota la *NP*-completezza si riduce polinomialmente a S). Tramite *SAT*, o comunque tramite l'altro esempio di riferimento, si prova allora che tutto *NP* si riduce in $\leq_p$ a S ; infatti $\leq_p$ gode della proprietà transitiva: se un dato S' soddisfa $S' \leq_p SAT$ e inoltre $SAT \leq_p S$, allora $S' \leq_p S$. Dunque S è *NP*-arduo (e *NP*-completo).

In questo modo, Karp procedette per i suoi 21 problemi. Ma la costruzione dei problemi *NP*-completi non si esaurì certamente con la lista di Karp e col 1972. Negli anni successivi, nuovi esempi sono stati scoperti, distribuiti in svariati ambiti di studi e ricerca: in Informatica, in Matematica, in Biologia, in Chimica, in Enigmistica.

Libri classici come [42] ne propongono e discutono un gran numero. Ma la lista complessiva dei problemi *NP*-completi è costantemente aggiornata nel *Journal of Algorithms* a cura di D. Johnson, e anche da siti Internet come quelli citati a fine capitolo.

Qui mostriamo che tutti gli esempi di problemi in NP dati nel paragrafo 8.2, cioè $3SAT$, $3COL$, IS , VC (o $CLIQUE$), $KNAPSACK$, equazioni quadratiche a coefficienti interi, sono NP -completi. Nei casi che più ci interessano in futuro (come $3SAT$, $3COL$, IS), diamo una dimostrazione dettagliata di questa asserzione. Negli altri ci limitiamo a sintetizzare le informazioni essenziali e a fornire possibili riferimenti bibliografici per chi è interessato ad approfondire.

Teorema 8.7.1 $3SAT$ è NP -completo.

Dimostrazione. Sappiamo che $3SAT \in NP$, ci basta mostrare che $3SAT$ è NP -arduo, e anzi, come già spiegato, che $SAT \leq_p 3SAT$. Cerchiamo dunque un algoritmo deterministico che lavora in tempo al più polinomiale e traduce ogni insieme finito I di clausole in un insieme finito $I(3)$ di clausole con 3 lettere tale che

$$I(3) \in 3SAT \text{ se e solo se } I \in SAT.$$

In dettaglio, ad ogni clausola k di I associamo un insieme I_k di clausole con 3 lettere (eventualmente nuove) nel modo che segue.

Se k consta di un'unica lettera p_0 (o P_0), prendiamo due nuove lettere minuscole q_0, q_1 e formiamo

$$I_k = \{p_0q_0q_1, p_0q_0Q_1, p_0Q_0q_1, p_0Q_0Q_1\}.$$

Notiamo che una valutazione v soddisfa I_k se e solo se v soddisfa k , cioè p_0 (infatti in nessun caso v può soddisfare contemporaneamente $q_0q_1, q_0Q_1, Q_0q_1, Q_0Q_1$). Se $k = p_0p_1$ consta di due lettere, prendiamo una nuova lettera minuscola q non usata precedentemente e formiamo

$$I_k = \{p_0p_1q, p_0, p_1Q\}.$$

Di nuovo, è facile notare che una valutazione v soddisfa I_k se e solo se v soddisfa k , cioè p_0 o p_1 .

Se k ha 3 lettere, si pone $I_k = \{k\}$.

Finalmente assumiamo che k consti di $h \geq 4$ lettere,

$$k = p_0p_1 \cdots p_{h-1}.$$

Prendiamo $h - 3$ nuove lettere

$$q_0, \dots, q_{h-4}$$

e formiamo

$$I_k = \{p_0p_1q_0, p_2Q_0q_1, p_3Q_1q_2, \dots, p_{h-3}Q_{h-5}q_{h-4}, p_{h-2}p_{h-1}Q_{h-4}\}.$$

Supponiamo che v sia una valutazione che soddisfa I_k e mostriamo che v soddisfa anche k . Altrimenti $v(p_0) = v(p_1) = \cdots = v(p_{h-1}) = 0$. Da $v(p_0) = v(p_1) = 0$

deduciamo $v(q_0) = 1$ perché $p_0 p_1 q_0 \in I_k$; così $v(Q_0) = 0$ e deve essere $v(q_1) = 1$ perché $p_2 Q_0 q_1 \in I_k$. Ripetendo il procedimento si deduce $v(q_{h-4}) = 1$ e quindi $v(Q_{h-4}) = 0$. Ma anche $v(p_{h-2}) = v(p_{h-1}) = 0$, e questo è impossibile perché $p_{h-2} p_{h-1} Q_{h-4} \in I_k$.

Viceversa, sia v una valutazione che soddisfa k , mostriamo che c'è qualche valutazione v_k che estende v e soddisfa I_k . Per ipotesi esiste $j < h$ tale che $v(p_j) = 1$. Sia j il minimo indice con questa proprietà. Se $j = 0$ o $j = 1$, l'estensione v_k di v per cui

$$v_k(q_i) = 0 \text{ per ogni } i \leq h - 4$$

soddisfa I_k , infatti assegna il valore 1 a p_0 o a p_1 nella prima clausola e a Q_i nelle clausole successive. Allo stesso modo, se $j = h - 2$ o $j = h - 1$,

$$v_k(q_i) = 1 \text{ per ogni } i \leq h - 4$$

funziona. Altrimenti $2 \leq j < h - 3$, e ci basta assumere

$$v_k(q_i) = 1 \text{ se } i < j - 2, v_k(q_i) = 0 \text{ altrimenti.}$$

Notiamo che la traduzione $k \mapsto I_k$ è deterministica e polinomiale rispetto alla lunghezza di k .

Sia ora $I(3) = \bigcup_{k \in I} I_k$. La procedura che definisce $I(3)$ da I è deterministica e polinomiale rispetto a $|I|$ e alla lunghezza delle clausole in I . Inoltre ogni clausola in $I(3)$ ha esattamente 3 lettere.

Sia $I(3) \in 3SAT$ e sia v una valutazione che soddisfa $I(3)$, dunque I_k per ogni clausola k di I . Allora v soddisfa ogni k e complessivamente I . Ne segue $I \in SAT$.

Viceversa, sia $I \in SAT$ e sia v una valutazione che soddisfa I , quindi ogni clausola k in I . Così, per ogni k , c'è una valutazione v_k che soddisfa I_k ed estende v sulle lettere di k . Per clausole di ≤ 3 lettere, anzi, $v_k = v$. In ogni caso, la precedente costruzione mostra che le varie v_k sono definite su lettere distinte (le nuove lettere q_i e Q_i coinvolte per la corrispondente k) e non interferiscono tra loro. Pertanto possiamo cucirle assieme per costruire una valutazione v' (definita sulle lettere di I e poi sulle nuove variabili q_i, Q_i al variare di k) tale che v' estende v e soddisfa complessivamente $I(3)$. Dunque $I(3) \in SAT$. $\square$

Teorema 8.7.2 IS è NP -completo.

Dimostrazione. Ci basta provare che IS è NP -arduo, infatti già sappiamo che $IS \in NP$. Siccome $3SAT$ è NP -completo, è poi sufficiente mostrare $3SAT \leq_p IS$. In particolare proponiamo un algoritmo deterministico che traduce insiemi finiti I di m_I clausole di 3 lettere in grafi finiti $G_I = (V_I, E_I)$ in modo tale che

$$I \in 3SAT \text{ se e solo se } (G_I, m_I) \in IS;$$

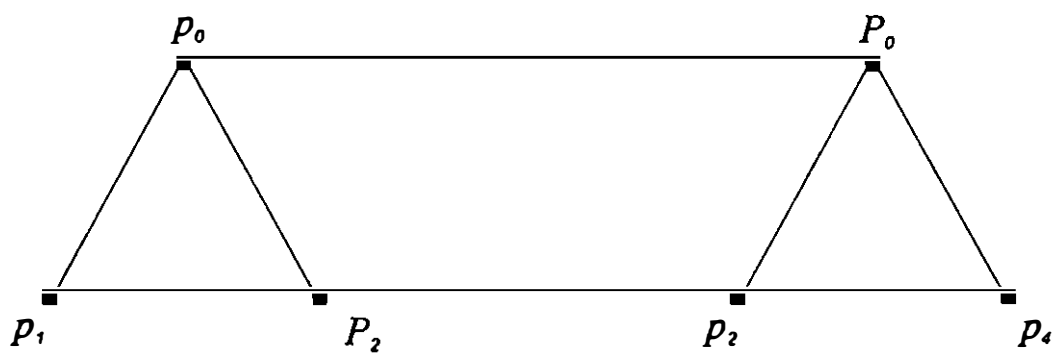
l'algoritmo opera poi in tempo al più polinomiale nella lunghezza di I . In dettaglio G_I è così definito:

- V_I ha $3m_I$ vertici, 3 per ogni clausola in I , anzi, più precisamente uno per ogni lettera in una clausola di I (ovviamente alla stessa lettera in 2 clausole diverse corrispondono due vertici diversi);
- E_I congiunge vertici corrispondenti a lettere della stessa clausola, oppure ogni lettera minuscola con la corrispondente maiuscola.

Per esempio, se I è composto da

$$p_0p_1P_2, P_0p_2p_4,$$

G_I è



È facile ammettere che la traduzione da I a G_I è deterministica e avviene in tempo polinomiale rispetto a $l(I)$.

Resta da provare che $I \in 3SAT$ se e solo se $(G_I, m_I) \in IS$. Supponiamo dapprima che G_I contenga un sottoinsieme V_0 indipendente di m_I vertici. Siccome G_I contiene m_I triangoli (uno per ogni clausola di I) e vertici di uno stesso triangolo sono collegati in E_I , V_0 deve contenere esattamente un vertice per ogni triangolo. Consideriamo una valutazione v che assegni valore 1 alle lettere corrispondenti ai vertici di V_0 . Notiamo che v è ben definita perché V_0 non può contenere vertici corrispondenti ad una minuscola e alla corrispondente maiuscola (la relativa coppia determina un lato in E_I). Dalla costruzione segue che v assegna il valore 1 ad almeno una lettera in ciascuna clausola di I . Così $I \in 3SAT$.

Viceversa, sia $I \in 3SAT$, e sia v una valutazione che lo soddisfa. In ogni clausola di I c'è una lettera (minuscola o maiuscola) cui v dà il valore 1. Scegliamo una tale lettera in ogni clausola e definiamo $V_0 \subseteq V_I$ come l'insieme dei vertici corrispondenti a tali lettere. $|V_0| = m_I$ perché I ha m_I clausole. Due vertici di V_0 non possono essere collegati da E_I perché

- non sono nello stesso triangolo,
- non possono corrispondere l'uno ad una lettera minuscola q , l'altro alla corrispondente maiuscola Q (altrimenti $v(q) = v(Q) = 1$). □

Teorema 8.7.3 $3COL$ è NP -completo.

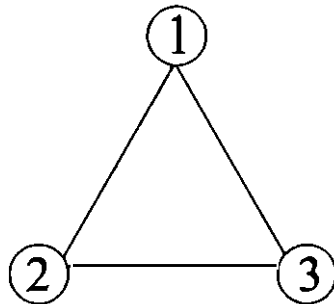
Dimostrazione. Come nel caso precedente basta provare $3SAT \leq_p 3COL$. Stavolta però la riduzione è più complicata. Vale comunque la pena di accennarla per mostrare quali difficoltà combinatorie debbano talora affrontarsi nell'ambito della

NP-completezza. Ricordiamo allora che cerchiamo un algoritmo deterministico che lavora in tempo polinomiale nella lunghezza dell'input, opera su insiemi finiti I di clausole a 3 lettere e li traduce in grafi $G(I) = (V(I), E(I))$ tali che

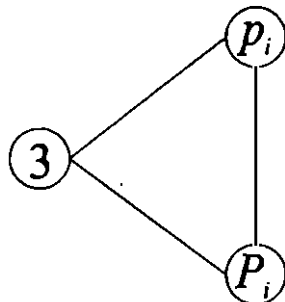
$$I \in 3SAT \text{ se e solo se } G(I) \in 3COL.$$

Per semplicità assumiamo che le clausole di I siano $k_0, \dots, k_{m-1}$ e coinvolgano lettere tra le minuscole $p_0, \dots, p_{n-1}$ e le corrispondenti maiuscole $P_0, \dots, P_{n-1}$. La costruzione di $G(I)$ avviene come segue.

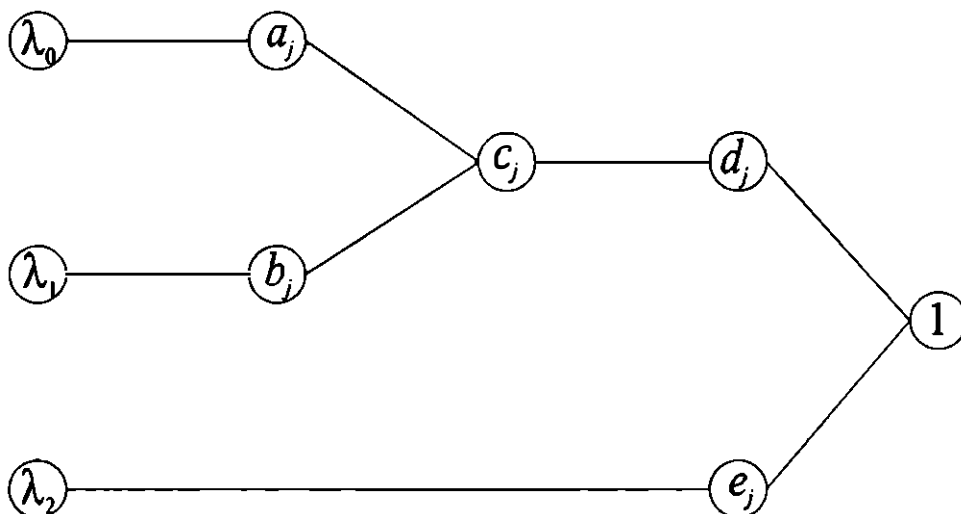
Anzitutto inseriamo in $G(I)$ 3 vertici corrispondenti ai 3 colori 1,2,3, collegati tra loro in $E(I)$.



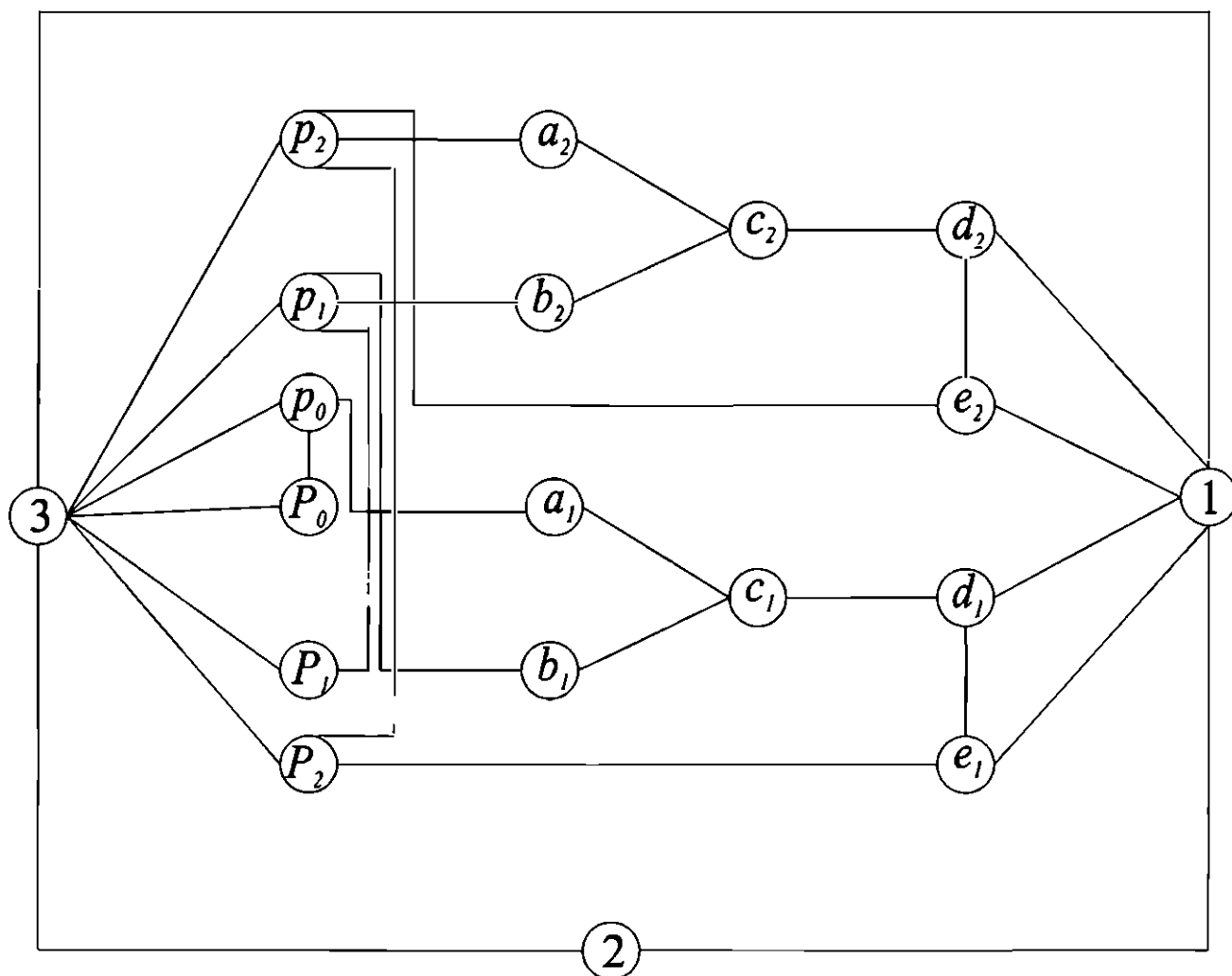
Poi, per ogni $i < n$, aggiungiamo 2 vertici, uno per p_i , l'altro per P_i e li uniamo tra loro e con il vertice del colore 3 tramite $E(I)$.



Finalmente aggiungiamo 5 vertici in $V(I)$ e 9 lati in $E(I)$ per ogni $j \leq m$, cioè per ogni clausola k_j in I . Se k_j è $\lambda_0 \lambda_1 \lambda_2$ (con $\lambda_0, \lambda_1, \lambda_2$ lettere maiuscole o minuscole), la costruzione avviene come mostrato dal disegno (a_j, b_j, c_j, d_j, e_j sono i nuovi 5 vertici per k_j).



Per esempio, supponiamo $I = \{k_0, k_1\}$ con $k_0 = p_0 p_1 P_2$ e $k_1 = P_0 p_1 p_2$. Il grafo $G(I)$ risultante è



e contiene quindi $3+6+10=19$ vertici e 30 lati. Il risultato può sembrare scoraggiante, visto che le clausole di partenza sono solo 2 e le lettere coinvolte solo 6, anzi $m = 2$ e $n = 3$.

D'altra parte, si vede facilmente che, in generale, $G(I)$ viene ad avere

- $3 + 2n + 5m$ vertici,
- $3 + 3n + 9m$ lati.

Così la sua costruzione, oltre ad essere deterministica, richiede tempo polinomiale in m e n , proprio come richiesto.

Dobbiamo però controllare che

$$I \in 3SAT \text{ se e solo se } G(I) \in 3COL.$$

Ci serve un'analisi attenta e minuziosa.

Supponiamo dapprima $G(I) \in 3COL$; sia c una corrispondente colorazione. Sappiamo che c è definita a meno di una permutazione dei colori 1, 2, 3, possiamo allora supporre che c colori i vertici 1, 2, 3 proprio di 1, 2, 3. Allora i vertici p_i, P_i ($i < n$), che sono collegati a 3, hanno colore 1 o 2, anzi uno di loro ha colore 1 e l'altro ha colore 2 perché sono uniti in $E(I)$ anche tra loro. Definiamo una valutazione v ponendo, per ogni $i < n$,

$$v(p_i) = 1 \text{ se e solo se } c(p_i) = 1$$

e $v(p_i) = 0$ altrimenti (analogamente per P_i). La precedente osservazione ci assicura che v è ben definita. Vogliamo mostrare che v soddisfa I (così potremo dedurre $I \in 3SAT$). Dunque va mostrato che, per ogni $j < m$, c'è in k_j almeno una lettera cui c dà il colore 1. Fissiamo j e supponiamo $k_j = \lambda_0 \lambda_1 \lambda_2$, come prima. Siccome d_j, e_j sono collegati tra loro e con 1, $c(d_j), c(e_j)$ assumono i valori 2 e 3: uno dei due ha colore 2, e l'altro colore 3.

Se $c(e_j) = 2$, λ_2 (che è collegato a 3) deve avere colore 1, e siamo a posto. Allora supponiamo $c(e_j) = 3$ e $c(d_j) = 2$. Ne segue che $c(c_j) = 2$ è 1 o 3. Conseguentemente a_j e b_j si distribuiscono, uno per uno, i colori 2 e 3, oppure 2 e 1. In ogni caso, $c(a_j) = 2$ o $c(b_j) = 2$. Se $c(a_j) = 2$, λ_0 , che è collegato ad a_j e 3, riceve colore 1. Se $c(b_j) = 2$, chi riceve il colore 1 è, allo stesso modo, λ_1 . Comunque almeno una lettera di k_j ha colore 1.

Viceversa, data una valutazione v che soddisfa I , è possibile dedurre, usando gli stessi ragionamenti, una colorazione appropriata c per $G(I)$. Il lettore può cercare di sviluppare da solo, per esercizio, i relativi dettagli. $\square$

Esercizio. Si provi che anche $3COL \leq_p 3SAT$ (si usi la strategia già adoperata per $2COL$ e $2SAT$ nell'esempio 2 del paragrafo 8.1 di questo capitolo).

Come detto, molti altri esempi di problemi NP -completi, con relative dimostrazioni e $\leq_p$ -riduzioni, possono trovarsi in [42] e negli altri riferimenti citati a fine capitolo. Per esempio, in [42] è presente la prova che anche il problema dello zaino ($KNAPSACK$) è NP -completo, perché gli si può ridurre $3SAT$. Lo stesso può dirsi dell'altro esempio di problema in NP discusso nel paragrafo 8.2, quello delle equazioni quadratiche a coefficienti interi. Anch'esso è NP -completo, come mostrato da Adleman e Manders nel 1978 e ricordato anche in [42], p. 250. La strategia consiste ancora nel ridurre $3SAT$ a questo problema, ma i dettagli richiedono qualche premessa approfondita di algebra. Conseguentemente tralasciamo un resoconto particolareggiato e ci accontentiamo degli esempi fin qui proposti.

8.8 P e NP: qualche commento ulteriore

Abbiamo considerato nei precedenti paragrafi numerosi esempi di problemi NP -completi. Possiamo domandarci se esistono problemi S che stanno in $NP - P$, ma non sono NP -completi (ammesso, naturalmente, che $P \neq NP$). Tali problemi si dicono NP -intermedi. Si può mostrare che, quand'anche $P \neq NP$, i problemi NP -completi non esauriscono la classe NP , e ci sono davvero problemi NP -intermedi. Dunque provare l'esistenza di problemi NP -intermedi equivale ad escludere $P = NP$. Questa osservazione stimola certamente la ricerca di possibili esempi NP -intermedi. Ci sono questioni di algebra che sembrano candidarsi ragionevolmente in questa prospettiva. Consideriamo per esempio il *problema dell'isomorfismo tra grafi finiti*. Ricordiamo anzitutto:

Definizione. Due grafi finiti $G = (V, E)$ e $G' = (V', E')$ si dicono isomorfi se e solo se c'è una corrispondenza biunivoca f di V su V' tale che, per ogni scelta di

$v, w \in V$,

$(v, w) \in E$ se e solo se $(f(v), f(w)) \in E'$.

f si dice un *isomorfismo* di G su G' .

Problema dell'isomorfismo tra grafi GI .

- INPUT: due grafi finiti $G = (V, E)$ e $G' = (V', E')$;
- OUTPUT: *SÌ*/NO secondo che ci sia o no un isomorfismo dei loro lati.

È facile notare che il problema sta in NP : se l'isomorfismo f è dato, verificare che sia tale è controllo che si può svolgere in tempo polinomiale rispetto alle "dimensioni" di G, G' (cioè rispetto al numero dei loro vertici e dei loro lati). D'altra parte, le possibili corrispondenze biunivoche da V a V' , anche ammesso che V, V' abbiano lo stesso numero n di elementi, sono $n!$, e dunque un controllo di isomorfismo fatto caso per caso, cioè corrispondenza per corrispondenza, può non essere rapido perché richiede l'analisi di $n!$ possibilità (e, per $n > 3$, $n! > 2^n$). Non è dunque chiaro se il problema è in P , almeno per questa via, né vi sono certezze derivate in altro modo in questo ambito. Neppure sembra che il problema sia NP -completo (se $P \neq NP$). Così la questione dell'isomorfismo tra grafi può essere un candidato nella classe dei problemi NP -intermedi.

Altrettanto può dirsi di un'altra questione che già abbiamo avuto modo di illustrare e commentare, e cioè del problema di decomporre un dato intero $N > 1$ nei suoi fattori primi (si ricordi che, invece, il problema di riconoscere se N è primo o no è in P).

A proposito, NPI è la sigla che denota usualmente questa classe, mentre NPC indica quella dei problemi NP -completi.

Un'altra strategia di ricerca di problemi NP -intermedi fa riferimento alla seguente definizione.

Definizione. Sia S un insieme di parole sull'alfabeto finito A . S si dice *rado* se c'è un polinomio p a coefficienti interi tale che, per ogni intero positivo n , S ha al più $p(n)$ parole di lunghezza $\leq n$.

Semplici esempi di problemi radi sono quelli per cui A si riduce ad un unico simbolo, per esempio $A = \{1\}$: in tal caso, per ogni n , l'unica parola in A^* di lunghezza n è $11 \cdots 1$, così vi sono al più n parole di lunghezza $\leq n$, e ogni $S \subseteq A^*$ è rado, si può infatti assumere $p(x) = x$. Un tale problema S si dice *1-ario*. Così lo stesso A^* è *1-ario* (ma appartiene ovviamente a P).

A proposito dei problemi radi, si può provare che:

1. se $P \neq NP$, allora ogni problema rado in $NP - P$ è NP -intermedio;
2. c'è un problema rado in $NP - P$ se e solo se ce n'è qualcuno 1-ario.

Così, una possibile strategia per ottenere problemi NP -intermedi (e dunque per concludere anche $P \neq NP$) è quella di esplorare problemi 1-ari. Ma nessuna certezza è derivata finora dalla loro analisi.

In ogni caso, assumendo $P \neq NP$, i problemi NP -completi sembrano quelli maggiormente complicati all'interno di NP . Del resto sappiamo che due qualunque S, S' tra essi sono riducibili l'uno all'altro in $\leq_p$

$$S \leq_p S', S' \leq_p S.$$

Possiamo allora considerare la seguente nozione.

Definizione. Due problemi S, S' su alfabeti A, A' sono P -isomorfi se c'è una biiezione f di A^* su A'^* tale che

1. f, f^{-1} sono computabili in tempo polinomiale,
2. per ogni $w \in A^*, w \in S$ se e solo se $f(w) \in S'$.

Ci si può conseguentemente chiedere:

Congettura di Berman - Hartmanis. Due problemi NP -completi sono sempre P -isomorfi.

Ma la congettura è ancora aperta. Ovviamente è falsa se $P = NP$. Infatti in tal caso insiemi infiniti di parole, come SAT , accompagnerebbero in P problemi finiti S , e non c'è biiezione possibile tra SAT e un simile S . D'altra parte, la congettura potrebbe risultare falsa anche ammesso $P \neq NP$.

Come si vede, l'indagine su $P = NP$ è delicata e complicata, ricca di sfumature e curiosità, apparentemente lontana da soluzioni. Non per nulla $P = NP$ figura nella lista ufficiale dei così detti "problemi del millennio": un elenco compilato nel 2000 per raccogliere i misteri teorici più complicati nell'ambito della Matematica pura e applicata (all'Informatica). La lista si compone di sette problemi, e $P = NP$ è uno di questi. Chi è interessato a conoscere gli altri sei problemi e a leggere una loro discussione generale può consultare [34].

8.9 coNP e la gerarchia polinomiale

Abbiamo già trattato nel paragrafo 8.1 la classe coP dei problemi S su alfabeti finiti A tali che il complementare $S^c = A^* - S$ appartiene a P . È stato facile osservare $coP = P$. Infatti, per $w \in A^*, w \in S^c$ se e solo se $w \notin S$. Così, quando si deve decidere l'appartenenza di w e si intende farlo in tempo rapido, è indifferente porre l'accento su S o su S^c , chiedere cioè se

$$w \in S^c \text{ o no}$$

invece che se

$$w \in S \text{ o no,}$$

perché una risposta positiva alla prima domanda equivale ad una negativa alla seconda e viceversa.

Consideriamo adesso NP e la classe $coNP$ dei problemi *complementari* di quelli di NP : per $S \subseteq A^*$,

$$S \in coNP \text{ se e solo se } S^c \in NP.$$

Stavolta, però, come già anticipato alla fine del paragrafo 8.2, non è affatto chiaro che $NP = coNP$. Vediamo perché.

Esempio. Sappiamo che SAT sta in NP . SAT raggruppa gli insiemi finiti soddisfacibili K di clausole. La sua appartenenza a NP è facilmente dimostrata: fissato $K \in SAT$,

- si sceglie una valutazione v delle lettere minuscole o maiuscole in K capace di soddisfare K ,
- si nota che la lunghezza di v è polinomialmente limitata dal numero di queste lettere e dunque, in definitiva, dalla lunghezza di K (tramite un polinomio che non dipende da K),
- si verifica finalmente (e rapidamente) che v soddisfa K , si ha quindi tramite il *testimone* v la certezza che K è soddisfacibile.

Questo basta a concludere $SAT \in NP$. Ma adesso consideriamo il problema complementare di SAT , quello essenzialmente formato dagli insiemi finiti di clausole che **non** sono soddisfacibili. Indichiamoli con $UNSAT$ (ancora in omaggio alla lingua inglese e in riferimento all'aggettivo *unsatisfiable*, insoddisfacibile). Stavolta, fissato $K \in UNSAT$, non basta il ricorso ad un singolo testimone v per acquisire la certezza che K è insoddisfacibile. Infatti potremo ancora rapidamente verificare che v non soddisfa K , ma questa conclusione non sarà sufficiente ad escludere a priori che altre valutazioni v' si comportino in modo diverso su K , e lo soddisfino. Soltanto il controllo di tutte le valutazioni possibili (dunque di 2^n casi, se n è il numero delle lettere coinvolte in K) ci garantirà questa certezza. In conclusione, non è chiaro che $UNSAT \in NP$, cioè che $SAT \in coNP$.

Del resto, la definizione stessa di $coNP$ ci rivela come la situazione sia meno limpida che nel caso di P . Ricordiamo infatti che, per $S \subseteq A^*$,

$$S^c \in NP$$

se e solo se esistono un problema $V_S \in P$ e un polinomio a coefficienti interi (e a valori positivi) p_S tali che, per ogni $w \in A^*$, $w \in S^c$ se e solo se

$$\text{esiste } y \in A^* \text{ per cui } l(y) \leq p_S(l(w)) \text{ e } (w, y) \in V_S.$$

Negando quest'ultima condizione, otteniamo che $w \in S$ se e solo se

$$\text{per ogni } y \in A^* \text{ tale che } l(y) \leq p_S(l(w)), \text{ si ha } (w, y) \notin V_S.$$

D'altra parte, il complementare W_S di V_S è ancora in P , poiché $coP = P$. In conclusione si ha la seguente caratterizzazione di $coNP$:

$S \subseteq A^*$ è in $coNP$ se e solo se esistono un problema $W_S \in P$ e un polinomio a coefficienti interi (e a valori positivi) p_S tali che, per ogni $w \in A^*$,

$$w \in S$$

se e solo se

per ogni $y \in A^*$ tale che $l(y) \leq p_S(l(w))$, si ha $(w, y) \in W_S$.

Dunque, il controllo dell'appartenenza a S non si accontenta di interpellare un unico opportuno testimone y , ma richiede di coinvolgerli tutti.

Osservazioni.

1. Ovviamente

$$coNP \supseteq coP = P$$

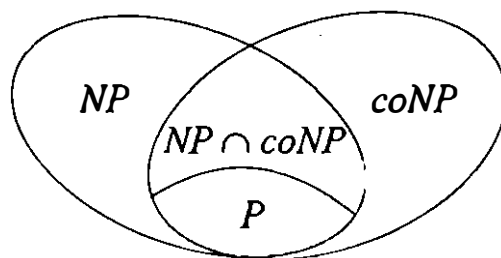
perché $NP \supseteq P$.

2. Inoltre, se $P = NP$, allora $coNP = coP = P = NP$.

3. D'altra parte, allo stato attuale delle conoscenze, non v'è alcuna definitiva sicurezza sulle relazioni tra NP e $coNP$. Potrebbero essere uguali o distinti

$$coNP = NP \text{ o } coNP \neq NP;$$

potrebbe anzi essere $coNP = NP$ anche se $P \neq NP$.



SAT è un esempio di problema di NP di cui non è noto se appartiene a $coNP$. Conseguentemente $UNSAT$ è un esempio di problema di $coNP$ (di cui non è noto se appartiene anche a NP).

Se confrontiamo le definizioni di NP e $coNP$, cogliamo un'unica differenza formale, relativa al quantificatore che accompagna y : per NP si chiede che **esista** y con opportune proprietà, mentre per $coNP$ si domanda che **ogni** y soddisfi gli stessi requisiti. Si tratta, ovviamente, di profonda differenza sostanziale, come l'esempio di SAT ci mostra. Ma possiamo trarne lo spunto per introdurre nuove classi di complessità progressivamente crescente Σ_k^P, Π_k^P per k naturale, nel modo che segue.

Definizione.

1. $k = 0$: poniamo anzitutto $\Sigma_0^P = \Pi_0^P = P$.

2. $k = 1$: Σ_1^P è NP , dunque la classe dei problemi $S \subseteq A^*$ per cui esistono $V_S \in P$ e un polinomio p_S a coefficienti interi (e a valori positivi) tali che, per ogni input $w \in A^*$,

$$w \in S$$

se e solo se

$$\exists y \in A^* \text{ tale che } l(y) \leq p_S(l(w)) \text{ e } (w, y) \in V_S.$$

Π_1^P è $coNP$, cioè la classe dei problemi $S \subseteq A^*$ per i quali esistono $V_S \in P$ e un polinomio p_S a coefficienti interi (e a valori positivi) tali che, per ogni input $w \in A^*$,

$$w \in S$$

se e solo se

$$\forall y \in A^* \text{ con } l(y) \leq p_S(l(w)), \text{ si ha } (w, y) \in V_S.$$

3. Per $k = 2$, si procede come segue. Σ_2^P è la classe dei problemi $S \subseteq A^*$ per cui esistono $V_S \in P$ e un polinomio p_S a coefficienti interi (e a valori positivi) tali che, per ogni input $w \in A^*$,

$$w \in S$$

se e solo se

$$\begin{aligned} &\exists y_1 \in A^* \text{ tale che } l(y_1) \leq p_S(l(w)) \\ &\text{e, } \forall y_2 \in A^* \text{ con } l(y_2) \leq p_S(l(w)), (w, y_1, y_2) \in V_S. \end{aligned}$$

Invece Π_2^P è la classe dei problemi $S \subseteq A^*$ per cui ci sono $V_S \in P$ e un polinomio p_S come sopra tali che, per ogni input $w \in A^*$,

$$w \in S$$

se e solo se

$$\begin{aligned} &\forall y_1 \in A^* \text{ con } l(y_1) \leq p_S(l(w)), \\ &\exists y_2 \in A^* \text{ tale che } l(y_2) \leq p_S(l(w)) \text{ e } (w, y_1, y_2) \in V_S. \end{aligned}$$

4. In generale, per $k \geq 2$, Σ_k^P è la classe dei problemi $S \subseteq A^*$ per cui ci sono $V_S \in P$ e p_S come sopra tali che, per ogni input $w \in A^*$,

$$w \in S$$

se e solo se

$$\begin{aligned} &\exists y_1 \in A^* \forall y_2 \in A^* \exists y_3 \in A^* \cdots Q_k y_k \in A^* \text{ con} \\ &l(y_1), \dots, l(y_k) \leq p_S(l(w)), (w, y_1, \dots, y_k) \in V_S \end{aligned}$$

(dove Q_k rappresenta $\forall, \exists$ secondo che k sia pari o dispari).

Π_k^P è la classe dei problemi $S \subseteq A^*$ per cui ci sono $V_S \in P$ e p_S come sopra tali che, per ogni input $w \in A^*$,

$$w \in S$$

se e solo se

$$\forall y_1 \in A^* \exists y_2 \in A^* \forall y_3 \in A^* \cdots Q_k y_k \in A^* \text{ con } l(y_1), \dots, l(y_k) \leq p_S(l(w)), (w, y_1, \dots, y_k) \in V_S$$

(dove Q_k si introduce come nel caso precedente).

Ecco alcune osservazioni su queste classi.

Osservazioni. Sia $k \in \mathbb{N}$.

1. $\Pi_k^P = co\Sigma_k^P$ e dunque $\Sigma_k^P = co\Pi_k^P$: la verifica è questione di logica elementare, ed estende il fatto che $\Pi_1^P = coNP = co\Sigma_1^P$.
2. Notiamo poi che, per S insieme di parole su A , $S \in \Sigma_{k+1}^P$ se e solo se ci sono $S_k \in \Pi_k^P$ e un polinomio a coefficienti interi p_S tali che, per ogni $w \in A^*$,

$$w \in S \quad \text{se e solo se}$$

$$\exists y_1 \in A^* \text{ tale che } l(y_1) \leq p_S(l(w)) \text{ e } (w, y_1) \in S_k.$$

Allo stesso modo, $S \in \Pi_{k+1}^P$ se e solo se ci sono $S_k \in \Sigma_k^P$ e un polinomio a coefficienti interi p_S tali che, per ogni $w \in A^*$,

$$w \in S \quad \text{se e solo se}$$

$$\forall y_1 \in A^* \text{ con } l(y_1) \leq p_S(l(w)), (w, y_1) \in S_k.$$

3. Per $i \leq k$, $\Sigma_i^P \subseteq \Sigma_k^P$, $\Pi_i^P \subseteq \Pi_k^P$.

Infatti la condizione di appartenenza a Σ_i^P di $S \subseteq A^*$, cioè l'esistenza di $V_S \in P$ e p_S tali che, per ogni $w \in A^*$

$$w \in S \quad \text{se e solo se}$$

$$\forall y_1 \in A^* \exists y_2 \in A^* \forall y_3 \in A^* \cdots Q_i y_i \in A^* \text{ con } l(y_1), \dots, l(y_i) \leq p_S(l(w)), (w, y_1, \dots, y_i) \in V_S$$

si adatta facilmente a mostrare $S \in \Sigma_k^P$ con l'introduzione di nuove parole fittizie $y_{i+1}, \dots, y_k$ come segue: esistono V_S, p_S tali che, per ogni $w \in A^*$,

$$w \in S \quad \text{se e solo se}$$

$$\exists y_1 \in A^* \forall y_2 \in A^* \cdots Q_i y_i \in A^* Q_{i+1} y_{i+1} \cdots Q_k y_k, \text{ tutti di lunghezza } \leq p_S(l(w)), \text{ con } (w, y_1, \dots, y_i) \in V_S.$$

Analogamente si procede per $\Pi_i^P \subseteq \Pi_k^P$.

4. Ancora, per $i < k$, $\Pi_i^P \subseteq \Sigma_k^P$, $\Sigma_i^P \subseteq \Pi_k^P$.

È sufficiente mostrare $\Sigma_i^P \subseteq \Pi_{i+1}^P$ (e $\Pi_i^P \subseteq \Sigma_{i+1}^P$). Di nuovo basta osservare che la condizione di appartenenza a Π_i^P di $S \subseteq A^*$ (appena ripetuta al punto

precedente) si adatta a provare $S \in \Pi_{i+1}^P$ con l’inserimento di una parola iniziale fittizia, nel modo che segue: esistono V_S, p_S tali che, per ogni $w \in A^*$,

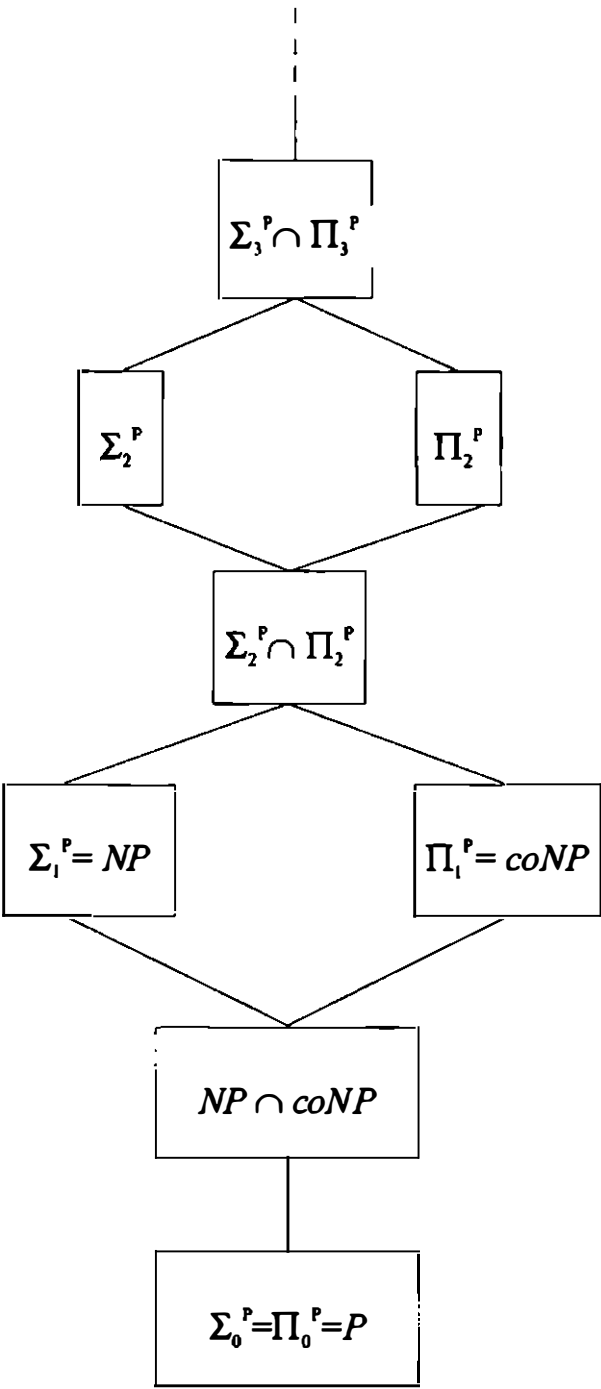
$$w \in S \quad \text{se e solo se}$$

$$\forall y \in A^* \exists y_1 \in A^* \forall y_2 \in A^* \cdots Q_i y_i \text{ con } l(y) \leq p_S(l(w)) \\ \text{(oltre a } l(y_1), l(y_2), \dots, l(y_i) \leq p_S(l(w))) \text{ e } (w, y_1, \dots, y_i) \in V_S.$$

Consideriamo anche $\Sigma_k^P \cap \Pi_k^P$ per $k \in \mathbb{N}$. Per esempio,

$$\Sigma_0^P \cap \Pi_0^P = P \cap coP = P, \quad \Sigma_1^P \cap \Pi_1^P = NP \cap coNP \supseteq P.$$

Le nuove classi Σ_k^P, Π_k^P , e relative intersezioni, si distribuiscono come illustrato nel seguente disegno



Si dice che le nuove classi così definite a partire da P, NP e $coNP$ formano la *gerarchia polinomiale*. Il culmine di questa gerarchia, ovvero l’unione di tutte

queste classi

$$\bigcup_{k \in \mathbb{N}} \Sigma_k^P = \bigcup_{k \in \mathbb{N}} \Pi_k^P,$$

si indica con PH (dall'inglese *polynomial hierarchy*).

La gerarchia polinomiale (e la sua costruzione così complicata) conferma la difficoltà del problema $P = NP$. Infatti, tanto per cominciare, si ha:

Teorema 8.9.1 $P = PH$ se e solo se $P = NP$.

Dimostrazione. Assumiamo infatti $P = PH$, notiamo $P \subseteq NP \subseteq PH$ e deduciamo $P = NP$. Viceversa supponiamo $P = NP$ e mostriamo $P = PH$. Ci basta mostrare che, $\forall k \in \mathbb{N}$, $\Sigma_k^P = P$ (infatti, in tal caso, $PH = \bigcup_{k \in \mathbb{N}} \Sigma_k^P = P$). Per $k = 0$, la cosa è ovvia. Assumiamo allora $\Sigma_k^P = P$ e proviamo $\Sigma_{k+1}^P = P$, cioè $\Sigma_{k+1}^P \subseteq P$, perché l'altra inclusione è chiara. Anzitutto $\Pi_k^P = co\Sigma_k^P = coP = P$. Sia allora $S \in \Sigma_{k+1}^P$. Ci sono quindi $V_S \in \Pi_k^P$ e un polinomio a coefficienti interi p_S tali che, per ogni parola w nell'alfabeto A di S ,

$$w \in S$$

se e solo se

$$\exists y_1 \in A^* \text{ tale che } l(y_1) \leq p_S(l(w)) \text{ e } (w, y_1) \in V_S.$$

Ma $\Pi_k^P = P$, dunque $V_S \in P$. Ne segue $S \in NP = P$. $\square$

Per $P \neq NP$, la situazione risulta assai meno chiara, anche se è possibile osservare che, in ogni caso, si ha:

Teorema 8.9.2

1. Se $\Sigma_k^P = \Pi_k^P$ per qualche $k \in \mathbb{N}$, allora $\Sigma_j^P = \Pi_j^P = \Sigma_k^P$ per ogni $j \geq k$, dunque $PH = \Sigma_k^P$.
2. Allo stesso modo, se $\Sigma_k^P = \Sigma_{k+1}^P$ per qualche $k \in \mathbb{N}$, allora di nuovo la gerarchia polinomiale si stabilizza, cioè $\Sigma_j^P = \Pi_j^P = \Sigma_k^P$ per ogni $j \geq k$ e quindi $PH = \Sigma_k^P$. Lo stesso capita se $\Pi_k^P = \Pi_{k+1}^P$ per qualche $k \in \mathbb{N}$: $\Sigma_j^P = \Pi_j^P = \Pi_k^P$ per ogni $j \geq k$ e dunque $PH = \Pi_k^P$.

Il lettore può cercare di sviluppare la dimostrazione per esercizio sulla base degli argomenti adoperati nelle osservazioni.

Notiamo infine che, come ci sono problemi NP -completi (adesso potremmo dire Σ_1^P -completi), così si può mostrare che, per ogni $k \in \mathbb{N}$, $k > 0$, esistono problemi S che sono Σ_k^P -completi:

- stanno in Σ_k^P ,
- per ogni $S' \in \Sigma_k^P$, $S' \leq_p S$.

In conclusione, anche la gerarchia polinomiale è ancora ben lungi dall'essere capita.

8.10 Oracoli

Nel mondo antico greco-romano, il cittadino alle prese con misteri più grandi di lui andava a consultare gli “oracoli”, a cercare cioè, tramite sibille e veggenti, il consiglio e l’illuminazione degli dei. Il mito di Edipo illustra il meccanismo e i suoi tragici pericoli.

Anche nelle moderne computabilità e complessità si ammette il ricorso agli oracoli. Nel nostro caso, però, un “oracolo” è semplicemente un fissato problema di decisione $\mathcal{O}$. Si immagina allora che una macchina di Turing (o un altro modello astratto di calcolatore) possa talora interpellare l’oracolo $\mathcal{O}$ nel modo che ora descriviamo. La MdT dispone di un nastro ulteriore, ed è libera nel corso della sua computazione di fare domande ad $\mathcal{O}$ scrivendo certe parole x su questo nastro. L’oracolo $\mathcal{O}$ le fornisce in un unico passo una risposta x' per cui $(x, x') \in \mathcal{O}$.

La consultazione di $\mathcal{O}$ può avvenire un qualunque numero di volte, e ogni domanda ad $\mathcal{O}$ costa soltanto un singolo passo di computazione.

È chiaro che, con questa facilitazione, la soluzione dei problemi si può accelerare: possiamo anzi formare a questo proposito nuove classi di complessità. Per esempio, per $\mathcal{O}$ oracolo prefissato,

- $P^{\mathcal{O}}$ è la classe dei problemi che si decidono in tempo al più polinomiale con macchine di Turing che possono ricorrere all’oracolo $\mathcal{O}$,
- $NP^{\mathcal{O}}$ è definito in modo analogo,

e così via. Anzi, possiamo generalizzare ulteriormente e costruire classi come

$$NP^{NP},$$

quella dei problemi che stanno in NP grazie a MdT che possono consultare un oracolo $\mathcal{O}$ ancora in NP .

Un profondo teorema di Baker, Gill e Solovay del 1975 collega P , NP e gli oracoli e sottolinea per questa via la difficoltà della questione $P = NP$. Afferma, infatti, quanto segue:

Teorema (Baker, Gill, Solovay) 8.10.1

1. Esiste un oracolo $\mathcal{O}_1$ tale che $P^{\mathcal{O}_1} = NP^{\mathcal{O}_1}$.
2. Esiste un oracolo $\mathcal{O}_2$ tale che $P^{\mathcal{O}_2} \neq NP^{\mathcal{O}_2}$.

Così, ove si ammetta la consultazione di oracoli diversi, la questione $P = NP$ arriva ad avere risposte definite ma opposte. Ed in effetti, allo stato attuale delle conoscenze, solo un oracolo può rivelarci se davvero $P = NP$ o no. La prova del teorema di Baker, Gill, Solovay, almeno per la parte 1, richiede qualche maggior approfondimento. La rimandiamo allora (per intero) al prossimo Capitolo 10.

8.11 Tempi esponenziali

Quando tutti gli algoritmi che riguardano un dato problema S richiedono tempo almeno esponenziale nella lunghezza dell'input, possiamo ritenere S praticamente intrattabile: infatti, i tempi di attesa delle risposte sono talora eccessivi. Anche in questo ambito negativo, possiamo comunque individuare vari livelli di complessità.

Definizione. $PEXP$ è la classe dei problemi che si risolvono deterministicamente in tempo $O(2^{p(n)})$ dove p è un polinomio (a coefficienti interi e valori positivi).

P sta per polinomiale, ovviamente. Se ci limitiamo a polinomi di grado 1, cioè lineari, formiamo (con l'iniziale L di lineare) la classe seguente.

Definizione. $LEXP$ è la classe dei problemi che si risolvono deterministicamente in tempo $O(2^{cn})$ con c costante intera positiva.

$NPEXP$, $NLEXP$ rappresentano le corrispondenti versioni non deterministiche. Ovviamente

$$\begin{aligned} P &\subseteq LEXP \subseteq PEXP, \\ LEXP &\subseteq NLEXP \subseteq NPEXP, \\ PEXP &\subseteq NPEXP. \end{aligned}$$

Si può anche vedere $PH \subseteq PEXP$: alla fine del Capitolo 10 ne daremo una dimostrazione particolareggiata.

8.12 $P = NP$ nella pratica

$P = NP$ e le difficoltà relative alla sua soluzione possono anche avere applicazioni pratiche. Questo paragrafo ne accenna alcune, riguardanti l'ambito della crittografia. La situazione che ci interessa riguarda dunque la necessità di proteggere da occhi indiscreti comunicazioni riservate; prevede dunque due processi:

- l'uno tramite il quale il mittente cifra il suo messaggio,
- l'altro mediante il quale il destinatario decifra il testo originale.

Un esempio classico e semplice è costituito dal sistema adoperato da Giulio Cesare per comunicare con i suoi luogotenenti durante le campagne militari, essenzialmente basato su una permutazione delle lettere, come quella seguente:

- Cesare cifrava i messaggi spostandone le lettere di 3 passi avanti, dunque A diveniva D , B diventava E , e via dicendo;
- i luogotenenti decifravano muovendo le lettere di 3 passi indietro: D ed E tornavano ad essere rispettivamente A e B , e così via.

Ovvia premessa del procedimento è un accordo preventivo tra Cesare e luogotenenti a proposito della chiave (3) con cui cifrare o decifrare. Infatti, la funzione f che cifra è quella che aggiunge 3, quella che decifra è la funzione f^{-1} inversa di f che toglie 3. Tra l'altro si noti che è facile in questo caso ottenere f^{-1} sapendo f , e viceversa.

Naturalmente nei secoli dopo sono stati elaborati criptosistemi assai più sofisticati e affidabili. L'evoluzione della società moderna ha tuttavia sollevato, anche per la crittografia, nuove esigenze e nuovi orizzonti. Oggi internet consente la possibilità di acquisti, sottoscrizioni, transazioni, certificazioni in rete; deve però assicurare la piena riservatezza di queste operazioni. Ma, in questo ambito, uno scambio preventivo di chiavi, come nel caso di Cesare, non è più prevedibile, perché il sistema coinvolge - e vuole coinvolgere - un ampio numero di utenti. Allora:

- è preferibile che ogni utente disponga di due chiavi, l'uno con cui gli interlocutori cifrano i messaggi a lui destinati, l'altra che gli serve per decifrarli; questa seconda chiave deve restare segreta e personale;
- dunque, chiunque può cifrare, ma solo il destinatario riesce a decifrare; ma allora le due operazioni - cifrare e decifrare, appunto - non devono essere computazionalmente equivalenti.

Allora chi conosce la funzione cifrante f non deve poter ricavare con facilità l'inversa f^{-1} che decifra. Nasce così l'esigenza di funzioni "a senso unico"

- facili da calcolare,
- invertibili,
- ma con inversa f^{-1} difficile da calcolare.

Si tratta, come si vede, di concetto relativamente vago e intuitivo, non completamente precisato: va infatti chiarito che cosa significhi "facile", o "difficile" da calcolare. Il collegamento con la questione $P = NP$ appare comunque evidente. Consideriamo infatti un problema NP -completo S : sappiamo che

- è facile verificarne le soluzioni (se le si conoscono),
- è difficile trovarle (almeno nei casi più sfavorevoli).

Questa disparità tra capacità di scoprire e di controllare le soluzioni di S sembra poter fornire spunti interessanti per funzioni a senso unico. Almeno a livello intuitivo, si possono infatti immaginare situazioni di funzioni f collegate a S , per le quali il calcolo di f^{-1}

- risulta agevole a chi conosce le soluzioni di S , e deve solo controllarle,
- diventa proibitivo a chi, non sapendo le soluzioni deve prima trovarsele.

Se poi si dovesse provare $P \neq NP$, l'esistenza di problemi NP -completi da coinvolgere in queste costruzioni si rivelerebbe non solo contingente, ma destinata a durare.

In realtà, una volta che si è definita in modo rigoroso la nozione di funzione a senso unico, si prova che l'esistenza di una tale funzione equivale non tanto a $P \neq$

NP , quanto alla disuguaglianza $P \neq UP$ dove UP è la classe di quei problemi di NP che sono accettati da MdT non deterministiche M che lavorano in tempo polinomiale e *ammettono per ogni input al più una computazione convergente* (condizione che si traduce dicendo che M non è *ambigua* e corrisponde al fatto che una funzione a senso unico f si suppone invertibile).

Non è chiaro se $P = UP$ o no, e neppure se vale l'equivalenza

$$P = UP \text{ se e solo se } P = NP$$

anche se ovviamente $P \neq UP$ implica $P \neq NP$. Così, da un punto di vista teorico, l'esistenza di funzioni a senso unico si collega in qualche modo a $P = NP$ e all'esistenza di problemi NP -completi. Nella pratica, però, questa connessione risulta meno incisiva e convincente. Infatti, per un problema NP -completo, l'accennata disparità tra

controllare / trovare

soluzioni si riferisce non alla media dei casi possibili, ma ai casi peggiori. Invece, dai sistemi crittografici ci si aspetta forte affidabilità nella media (o addirittura nella totalità) degli eventi, e non in casi "eccezionali".

Così nelle applicazioni crittografiche è stato, almeno finora, più vantaggioso riferirsi ad altre situazioni, per esempio alla più volte ricordata difficoltà di decomporre gli interi in fattori primi, che è il fondamento del più popolare protocollo crittografico ora in uso, l' RSA , e verosimilmente costituisce un problema NP -intermedio, piuttosto che NP -completo. Ma qui il discorso ci porterebbe lontano. Dedichiamo allora il resto del paragrafo ad un semplice esempio di sistema crittografico basato su un problema NP -completo da noi già preso in considerazione nei paragrafi 8.2 e 8.7, più precisamente sul problema dello zaino ($KNAPSACK$).

Il sistema di Merkle-Hellman Consideriamo il problema NP -completo dello zaino:

- INPUT: è dato da interi positivi $n, V, v(0), \dots, v(n-1)$;
- OUTPUT: SÌ/NO secondo che esista una sequenza $e(0), \dots, e(n-1)$ di bit 0,1 per cui

$$\sum_{i < n} e(i) \cdot v(i) = V$$

(il lettore potrà facilmente controllare l'equivalenza tra questa formulazione e quella data in precedenza nel paragrafo 8.8.2: sostanzialmente $I = \{i < n : e(i) = 1\}$). V può intendersi come il volume di uno zaino, $v(0), \dots, v(n-1)$ come volumi di oggetti con cui vogliamo riempirlo. Osserviamo comunque che il Problema dello Zaino può avere talora rapida soluzione, ammesso che la sequenza $v(0), \dots, v(n-1)$ sia opportunamente costruita. Ammettiamo per esempio che $v(0), \dots, v(n-1)$ sia *supercreciente*, cioè tale che, per ogni $i < n$, $v(i)$ sia maggiore della somma dei volumi degli oggetti che lo precedono $\sum_{j < i} v(j)$.

Esercizio. Si mostri che la sequenza 2, 3, 7, 15, 31 è supercreciente, mentre 2, 3, 4, seppur crescente, non è supercreciente.

È facile trovare algoritmi rapidi di soluzione per il Problema dello Zaino per sequenze supercrescenti. Infatti, supponiamo che $v(0), \dots, v(n-1)$ fornino una tale sequenza (e fissiamo anche V). Consideriamo per cominciare $v(n-1)$. Se $v(n-1) > V$, dobbiamo evidentemente escluderlo dallo zaino e quindi porre $e(n-1) = 0$; ma se $v(n-1)$ è invece minore o uguale a V , dobbiamo obbligatoriamente inserirlo e coinvolgerlo nella eventuale successione.

Esercizio. Perché? (Suggerimento: $v(n-1)$ non può essere infatti sostituito dalla somma $\sum_{j < n-1} v(j)$ dei volumi restanti, che gli è minore per ipotesi, dunque non riesce neppure a raggiungere il valore V e riempire in tal modo lo zaino; dunque, se c'è soluzione, $v(n-1)$ vi partecipa ed è $e(n-1) = 1$).

A questo punto consideriamo $v(n-2)$ e confrontiamolo col volume complessivo restante (ancora V se $v(n-1)$ è stato escluso, $V - v(n-1)$ altrimenti), ripetendo la procedura. Applicando questo metodo, recuperiamo alla fine una soluzione del problema, se essa esiste. Anzi, si può osservare che la soluzione, se c'è, è anche unica.

Esercizio. Si consideri la sequenza supercrescente 2, 3, 7, 15, 31 e si fissi $V = 24$. Si provi la soluzione del relativo problema dello zaino (risposta: 2, 7, 15).

In conclusione, il *Problema Supercrescente dello Zaino* ha un algoritmo rapido di soluzione, mentre non si conosce nessuna procedura veloce per il Problema generale dello Zaino (e anzi si paventa che non ci sia).

Passiamo allora al contesto crittografico. Abbiamo due interlocutori **A** e **B** che vogliono comunicare tra loro in modo sicuro, al riparo dal pirata (informatico) **C**. Possiamo anche immaginare che il messaggio di **A** a **B** si componga di sequenze di lunghezza n di bit 0 o 1.

Una possibile strategia che **A** può seguire per organizzare il suo criptosistema consiste nello scegliere anzitutto una sequenza supercrescente $v(0), \dots, v(n-1)$. Ci sono semplici metodi per generare una tale sequenza. Per esempio, **A** può prendere $n+1$ interi positivi arbitrari $z(0), \dots, z(n-1), z(n)$ e calcolarsi

$$v(0) = z(0),$$

$$v(1) = v(0) + z(1),$$

$$v(i) = \sum_{j < i} v(j) + z(i) \text{ per ogni } i < n$$

(che è ovviamente supercrescente), anzi computare anche

$$m = \sum_{j < n} v(j) + z(n).$$

Per esempio, gli interi positivi 2, 1, 2, 3, 4, 22 generano in questo modo la sequenza supercrescente 2, 3, 7, 15, 31 di cui sopra (e inoltre $m = 33$). A questo punto **A** maschera la sequenza prodotta $v(0), \dots, v(n-1)$ moltiplicandone gli elementi

modulo m per un qualche intero a primo con m (dunque A moltiplica ogni $v(i)$ per a e poi ne considera il resto della divisione per m). Siccome a è primo con m , esiste un intero a' , inverso di a modulo m , cioè tale che $a \cdot a' \equiv 1 \pmod{m}$. Nell'esempio, si può scegliere $a = 2$ (infatti 2 è primo con 33, e il suo inverso modulo 33 è 17) e ottenere da 2, 3, 7, 15, 31 la nuova sequenza 4, 6, 14, 30, 29 (modulo 33), che non è più supercrescente, perché la moltiplicazione per 2 ha rimescolato (modulo 33) l'ordine degli elementi coinvolti e in particolare $31 \cdot 2 = 62$ ha resto 29 modulo 33. In generale A , scelto a , computa anzitutto l'inverso a' di a modulo m (operazione facile e veloce, grazie a metodi elementari di teoria dei numeri) e poi soprattutto la sequenza $w(0), \dots, w(n-1)$ definita ponendo, per ogni $i < n$, $0 < w(i) < m$ e $w(i)$ congruo ad $a \cdot v(i)$ modulo m (cioè $v(i)$ congruo ad $a' \cdot w(i)$ modulo m). Ovviamente la nuova sequenza $w(0), \dots, w(n-1)$ può non essere supercrescente, e anzi A avrà comunque cura che non lo sia. A questo punto A comunica a B (e anzi divulga a tutti gli interessati)

- $w(0), \dots, w(n-1)$ (la chiave pubblica di codifica da utilizzare per scrivergli)

mentre mantiene gelosamente segreti

- m, a e $v(0), \dots, v(n-1)$ (che costituiscono la sua chiave privata di decodifica).

La codifica, l'invio e la decodifica di un eventuale messaggio avvengono come segue. Come detto, l'interlocutore B lo scrive sotto forma di sequenze $(e(0), \dots, e(n-1))$ di lunghezza n di bit scelti tra 0 e 1. Per ogni sequenza, B calcola $W = \sum_{i < n} e(i) \cdot w(i)$ e lo trasmette ad A : questa è la procedura f di codifica. Con le informazioni segrete in suo possesso, A può recuperare $(e(0), \dots, e(n-1))$ nel modo che adesso descriviamo. Dapprima A moltiplica il messaggio cifrato ricevuto W per l'inverso a' di a modulo m , e poi calcola il resto V' di questo prodotto nella divisione per m ; poi osserva che

$$V' \equiv a' \cdot W = \sum_{i < n} e(i) \cdot a' \cdot w(i) \quad \text{e} \quad V = \sum_{i < n} e(i) \cdot v(i)$$

hanno lo stesso resto nella divisione per m . Tuttavia, siccome tanto V che V' sono interi positivi minori di m , A deduce $V' = V$. A questo punto A , ricavato $V = \sum_{i < n} e(i) \cdot v(i)$, decodifica facilmente $(e(0), \dots, e(n-1))$ perché la sequenza $v(0), \dots, v(n-1)$ è supercrescente. Invece un pirata C che pure riesca ad intercettare W e intenda recuperare $(e(0), \dots, e(n-1))$, conoscendo soltanto la sequenza (non supercrescente) $w(0), \dots, w(n-1)$, si trova di fronte ad una istanza del Problema generale dello Zaino, con tutte le difficoltà che ne derivano. In conclusione, Knapsack (e altri problemi NP -completi) possono suggerire sistemi crittografici per lo scambio sicuro di messaggi. Quello appena descritto è il crittosistema ideato da Merkle ed Hellman alla fine degli anni settanta. Ad onor del vero, dobbiamo confessare che si tratta di un crittosistema troppo semplice, non affidabile nè sicuro: il lettore può cercare di immaginare il perché (suggerimento: le sequenze pubbliche $w(0), \dots, w(n-1)$, pur non essendo supercrescenti come

quelle private, hanno comunque una natura particolare perché si ottengono da sequenze supercrescenti moltiplicando per un fattore costante modulo un intero m pure costante; ammettono conseguentemente un loro specifico algoritmo rapido di soluzione che, pur senza avere la pretesa di soddisfare il Problema generale dello Zaino, riesce comunque ad infrangere con successo l'intero sistema). Va anche detto che, a seguito di questa osservazione, si studiarono perfezionamenti del criptosistema di Merkle ed Hellman capaci di restaurarne l'affidabilità.

8.13 Esercizi

1. Si dimostri che ogni linguaggio finito è in P , e che altrettanto vale per ogni linguaggio che ha complementare finito.
2. Si provi che unione e intersezione di linguaggi in P sono ancora in P . Si può dire lo stesso di NP ?
3. È corretto affermare che “se $P = NP$, allora ogni problema in NP è NP -completo?” (Suggerimento: si usi l'esercizio 1).
4. Due primi p_1, p_2 si dicono gemelli se $p_2 = p_1 + 2$. Ad esempio 3 e 5 sono primi gemelli, 7 e 11 no. L'insieme delle coppie ordinate di primi gemelli è in P ?
5. Il *problema del matrimonio a due sessi* $2DM$ (dall'inglese *2-dimensional matching*) prevede come input
 - due insiemi U, D disgiunti, tutti con lo stesso numero q di elementi,
 - un insieme S di coppie $U \times D$.

Informalmente, U si può pensare come un insieme di uomini, D come un insieme di donne, S come una relazione di simpatia uomo-donna. L'output è S o NO a seconda che si possano estrarre da S esattamente q coppie che coinvolgano ogni elemento di U e di D (si possono cioè organizzare q matrimoni in modo da soddisfare tutti gli uomini e le donne considerati, senza che nessuno resti single ed evitando casi di poligamia). Si provi che $2DM$ è in P .

6. Si provi che i problemi $IS, VC, CLIQUE$ sono l'uno riducibile all'altro in $\leq_p$.
7. Si mostri che, se $S, \bar{S}$ sono due problemi e $S \leq_p \bar{S}$, allora $S^c \leq_p \bar{S}^c$.
8. Si provi in dettaglio il teorema 8.9 relativo alla gerarchia polinomiale.
9. Si provi che, per ogni intero positivo k , Σ_k^P è chiusa per $\leq_p$.

I seguenti esercizi propongono un itinerario per provare che $KNAPSACK$ è NP -completo, ma non sono affatto facili: il lettore interessato può consultare [42], ad esempio, per trovare in dettaglio le relative dimostrazioni.

10. Il *problema del matrimonio a tre sessi* $3DM$ generalizza il caso dell'esercizio 5. Stavolta si hanno in input
 - tre insiemi U, D, X a due a due disgiunti, tutti con lo stesso numero q di elementi,
 - un insieme S di terne in $U \times D \times X$

e si chiede se è o no possibile estrarre da S q terne che coinvolgano tutti gli elementi di U, D, X . Si provi che $3DM$ è NP -completo (suggerimento: si mostri che $3DM$ è in NP e che $SAT \leq_p 3DM$).

11. Il *problema della partizione* $PART$ considera un insieme A di elementi a ciascuno con un suo peso $p(a)$ - un intero positivo - e chiede se sia o no possibile suddividere A in due parti disgiunte A_0 e A_1 di ugual peso complessivo, cioè tali che

$$\sum_{a \in A_0} p(a) = \sum_{a \in A_1} p(a).$$

Si provi che $PART$ è NP -completo (suggerimento: si cerchi di mostrare che $3DM \leq_p PART$).

12. Si provi finalmente che $KNAPSACK$ è NP -completo (suggerimento: $PART \leq_p KNAPSACK$).

Riferimenti bibliografici Le classi P, NP , la questione $P = NP$, la NP -completezza, la gerarchia polinomiale e gli altri argomenti cruciali del capitolo sono ampiamente discussi da [61] e da testi fondamentali di complessità, quali [22], [42], [76], [102]. Vale la pena di citare anche [48], [49] come contributi fondamentali nell'avvio della storia della complessità. Passiamo ai riferimenti su temi più specifici.

- La distinzione tra algoritmi polinomiali e esponenziali, e la consapevolezza che algoritmi che lavorano in tempo polinomiale danno risposte in tempi ragionevolmente rapidi appare già in [24], [83] e [88], ma la tesi di Edmonds, Cook, Karp è esplicitata in [35], [36] e [37].
- La nozione di NP -completezza (e la $\leq_p$ -riducibilità) furono proposte da Cook [25] e indipendentemente da Levin [75]. Negli stessi riferimenti si trovano le prove originali del teorema di Cook-Levin.
- La prima lista di 21 problemi NP -completi di Karp del 1972 è in [63]. Successivi aggiornamenti si trovano in [42], in [60], o in vari siti internet quali [115]; [42] mostra anche la NP -completezza del Problema dello Zaino, mentre quella del problema delle equazioni quadratiche a coefficienti interi è provata in [1]. [74] introduce e discute in tono divulgativo alcuni di questi problemi.

Il tema dei linguaggi NP -intermedi è trattato, per esempio, in [61]. [70] prova la loro esistenza quando $P \neq NP$. Il teorema di Baker-Gill-Solovay si trova in [12].

Finalmente, le necessarie informazioni sui grafi e numeri naturali possono trovarsi, per esempio, in [21], [40].

Per quanto riguarda i numeri naturali, si consideri anche [69], che tratta il tema del rapporto tra aritmetica, complessità e crittografia moderna. L'algoritmo AKS si trova nell'articolo originale di Agrawal, Kayal, Saxena [2], e nella versione

riveduta [3]; svariate esposizioni sono disponibili in rete; una trattazione dal tono relativamente divulgativo è in [73].

I collegamenti tra complessità e crittografia sono anche l'argomento di [90] e [94]. [69] descrive in dettaglio il criptosistema dello zaino ed i suoi perfezionamenti. [34] discute i sette “problemi del millennio” e, tra essi, anche $P = NP$.

Complessità, Logica e Circuiti

9.1 Un po' di logica

Il capitolo che inizia presenta un altro criterio di misura del costo di una computazione, e quindi della complessità di un algoritmo e di un problema. Esso si riferisce all'alfabeto $A = \{0, 1\}$, al quale, del resto, abbiamo visto potersi ridurre ogni possibile contesto. Osserviamo che una computazione di una macchina di Turing M su $\{0, 1\}$ riceve come input una stringa w di 0, 1 e produce un output della stessa forma: ad esempio, quando si deve decidere se w sta o no in un certo $S \subseteq \{0, 1\}^*$, 1 per dire *SÌ* e 0 per dire *NO*. Restano così determinate funzioni da $\{0, 1\}^n$ a $\{0, 1\}$ che, per ogni naturale $n > 0$, associano ad ogni input $w \in \{0, 1\}^n$ di lunghezza n l'esito 0 o 1 della corrispondente computazione di M .

D'altra parte, si vedrà che ogni tale funzione si può sempre rappresentare come composizione di tre particolari funzioni (che sono denominate *NOT*, *AND*, *OR* e saranno introdotte tra poche pagine). Potremo allora valutare il costo delle computazioni di M su input di lunghezza n a partire dalla complessità di tali rappresentazioni. Questa è l'idea che intendiamo sviluppare. Ma, prima di procedere in questo senso, abbiamo bisogno di chiarire in dettaglio i risultati appena citati sulle funzioni da $\{0, 1\}^n$ a $\{0, 1\}$. Del resto, essi si collegano a questioni di logica elementare, e dunque ci danno l'occasione di sviluppare e discutere alcuni temi in questo altro ambito. Infatti, nei capitoli precedenti abbiamo avuto modo di parlare, con qualche equilibrismo, del problema della soddisfacibilità nella logica proposizionale, riducendolo ad un gioco di lettere minuscole o maiuscole. Ma adesso vale la pena di riproporre il contesto in maniera più rigorosa e adeguata ai dettami e alla notazione della logica proposizionale.

L'alfabeto che ci interessa contiene ancora un'infinità di variabili, atte a indicare "proposizioni":

$$p_0, p_1, p_2, \dots, p_n, \dots \text{ con } n \text{ naturale.}$$

Esse costituiscono anche le formule più semplici tra quelle che intendiamo esaminare. Sappiamo già, però, di poter costruire formule più elaborate

- negando,
- disgiungendo,

- congiungendo

formule già ottenute. Con la notazione usata nei capitoli precedenti, possiamo per esempio considerare

- $P_0, P_1, P_2, \dots$ - le negazioni di $p_0, p_1, p_2, \dots$,
- $p_0 p_1 P_2$ (la disgiunzione di p_0, p_1 e della negazione di p_2), $P_1 p_3$ (la disgiunzione della negazione di p_1 e di p_3), $\dots$
- $\{p_0 p_1 P_2, P_1 p_3\}$ (la congiunzione di $p_0 p_1 P_2$ e $P_1 p_3$), $\dots$

Questa notazione è assai semplice e ci sarà utile nel seguito. Ma conviene anche accompagnarla con un'altra più ufficiale; aggiungere conseguentemente al nostro alfabeto

- le parentesi $(,)$,
- tre simboli $\neg, \vee, \wedge$ a indicare rispettivamente *non* (negazione), *o* (disgiunzione), *e* (congiunzione), e da chiamare *connettivi* in futuro,

definire poi le *formule* come segue:

- ogni variabile proposizionale è una formula,
- se α_0, α_1 sono formule, anche $\neg\alpha_0, \alpha_0 \vee \alpha_1, \alpha_0 \wedge \alpha_1$ lo sono,
- niente altro è una formula;

usare finalmente le parentesi $(,)$ come consueto strumento per ordinare la struttura di una formula, ed evitare ogni confusione nella sua interpretazione.

Con questa nuova notazione

- $P_0, P_1, P_2, \dots$ diventano $\neg p_0, \neg p_1, \neg p_2, \dots$,
- $p_0 p_1 P_2, P_1 p_3$ si scrivono rispettivamente $p_0 \vee p_1 \vee \neg p_2, \neg p_1 \vee p_3$,
- $\{p_0 p_1 P_2, P_1 p_3\}$ si esprime come $(p_0 \vee p_1 \vee \neg p_2) \wedge (\neg p_1 \vee p_3)$.

Sono poi lecite altre formule in cui $\neg, \vee, \wedge$ sono combinati in modo diverso, purché rispettoso della definizione, quali

$$\neg(p_0 \wedge \neg p_1), \neg(p_0 \wedge \neg(p_1 \wedge \neg p_2))$$

e così via.

Adesso ricordiamo il concetto di valutazione, adattando al nuovo contesto e al nuovo modo di scrivere le formule quanto già illustrato nei capitoli precedenti.

Una *valutazione* è definita come una funzione v dall'insieme delle variabili proposizionali a $\{0, 1\}$. L'intuizione è la solita: per n naturale,

- $v(p_n) = 1$ significa che v crede a p_n ,
- $v(p_n) = 0$ il contrario.

La valutazione v si estende poi in modo obbligato a formule arbitrarie: per α_0, α_1 formule, si pone infatti

- $v(\neg\alpha_0) = 1$ se e solo se $v(\alpha_0) = 0$; $v(\neg\alpha_0) = 0$ altrimenti;

- $v(\alpha_0 \vee \alpha_1) = 1$ se e solo se $v(\alpha_0) = 1$ oppure $v(\alpha_1) = 1$; $v(\alpha_0 \vee \alpha_1) = 0$ altrimenti;
- $v(\alpha_0 \wedge \alpha_1) = 1$ se e solo se $v(\alpha_0) = v(\alpha_1) = 1$; $v(\alpha_0 \wedge \alpha_1) = 0$ altrimenti;

quindi in generale

- $v(\neg \alpha_0) = 1 - v(\alpha_0)$,
- $v(\alpha_0 \vee \alpha_1) = v(\alpha_0) + v(\alpha_1) - v(\alpha_0) \cdot v(\alpha_1) = 1 - (1 - v(\alpha_0)) \cdot (1 - v(\alpha_1))$,
- $v(\alpha_0 \wedge \alpha_1) = v(\alpha_0) \cdot v(\alpha_1)$.

Una formula α si dice *soddisfacibile* se c'è almeno una valutazione v per cui $v(\alpha) = 1$.

Ci siamo già interessati al

Problema della soddisfacibilità. Determinare un algoritmo per decidere, per ogni formula α , se α è soddisfacibile oppure no.

In realtà, nei capitoli precedenti abbiamo concentrato la nostra attenzione sul caso particolare di *SAT*, e cioè della soddisfacibilità per quelle formule che si dicono in forma normale congiuntiva (abbreviata *CNF*), quelle che, nella vecchia notazione, si scrivono come insiemi finiti di parole con lettere maiuscole o minuscole

$$\{p_0 p_1 P_2, P_1 p_3\},$$

e adesso si esprimono come congiunzioni di disgiunzioni di variabili proposizionali e negazioni

$$(p_0 \vee p_1 \vee \neg p_2) \wedge (\neg p_1 \vee p_3).$$

A questo proposito, si indica con *3CNF* l'insieme di quelle formule di *CNF* che ammettono 3 lettere in ogni clausola, ovvero sono congiunzioni di disgiunzioni di 3 variabili o loro negazioni. Nell'ambito "ristretto" di *CNF* e *3CNF* abbiamo poi visto il teorema di Cook-Levin 8.6.1 secondo cui *SAT* è *NP*-completo (così come, di conseguenza, $3SAT = 3CNF \cap SAT$).

Del resto, è facile organizzare un programma per convertire qualunque formula della logica proposizionale in *CNF*. Ci si basa sulla seguente definizione e su alcune semplici osservazioni che la riguardano.

Definizione. Le formule α, α' si dicono *equivalenti* se, per ogni valutazione v , $v(\alpha) = v(\alpha')$.

Si osserva facilmente che la relazione binaria così definita tra le formule è davvero di equivalenza: riflessiva, simmetrica e transitiva. Il suo significato è chiaro: l'equivalenza di α, α' significa che α, α' , anche se formalmente diverse, sono ugualmente interpretate da ogni valutazione v . Per esempio p_0 e $\neg\neg p_0$ sono equivalenti poiché, per ogni v ,

$$v(\neg\neg p_0) = 1 - v(\neg p_0) = 1 - (1 - v(p_0)) = v(p_0).$$

Allo stesso modo $\neg(p_0 \wedge p_1)$ e $\neg p_0 \vee \neg p_1$ sono equivalenti, infatti, per ogni v ,

$$v(\neg(p_0 \wedge p_1)) = 1 - v(p_0 \wedge p_1) = 1 - v(p_0) \cdot v(p_1)$$

eguaglia

$$\begin{aligned}
 v(\neg p_0 \vee \neg p_1) &= v(\neg p_0) + v(\neg p_1) - v(\neg p_0) \cdot v(\neg p_1) = \\
 &= 1 - v(p_0) + 1 - v(p_1) - (1 - v(p_0)) \cdot (1 - v(p_1)) = \\
 &= 2 - v(p_0) - v(p_1) - 1 + v(p_0) + v(p_1) - v(p_0) \cdot v(p_1) = \\
 &= 1 - v(p_0) \cdot v(p_1).
 \end{aligned}$$

Generalizzando si vede facilmente che, per ogni scelta di formule $\alpha_0, \alpha_1, \alpha_2$,

$\neg\neg\alpha_0$ è equivalente a α_0 ,

$\neg(\alpha_0 \wedge \alpha_1)$ è equivalente a $\neg\alpha_0 \vee \neg\alpha_1$,

$\neg(\alpha_0 \vee \alpha_1)$ è equivalente a $\neg\alpha_0 \wedge \neg\alpha_1$,

$\alpha_0 \vee (\alpha_1 \wedge \alpha_2)$ è equivalente a $(\alpha_0 \vee \alpha_1) \wedge (\alpha_0 \vee \alpha_2)$,

come del resto l'intuizione conferma.

Notiamo poi che, per α_0, α'_0 tra loro equivalenti, e α_1, α'_1 tra loro equivalenti, restano equivalenti

$$\begin{aligned}
 &\neg\alpha_0, \neg\alpha'_0, \\
 &\alpha_0 \vee \alpha_1, \alpha'_0 \vee \alpha'_1, \\
 &\alpha_0 \wedge \alpha_1, \alpha'_0 \wedge \alpha'_1.
 \end{aligned}$$

Così, siccome $\neg\neg p_0$ equivale a p_0 (e p_1 ovviamente equivale a se stessa)

$$\neg\neg p_0 \wedge p_1 \text{ equivale a } p_0 \wedge p_1.$$

In generale, quindi, se si sostituisce parte di una formula con un “ricambio” ad essa equivalente, la formula risultante resta equivalente a quella che era.

Ecco allora un

Procedimento per la riduzione in CNF. È data una formula arbitraria α .

1. Si elimina in α ogni $\neg$ che non riguardi direttamente una variabile proposizionale (attraverso l'uso dell'equivalenza tra $\neg\neg\alpha_0$ e α_0 , $\neg(\alpha_0 \wedge \alpha_1)$ e $\neg\alpha_0 \vee \neg\alpha_1$, $\neg(\alpha_0 \vee \alpha_1)$ e $\neg\alpha_0 \wedge \neg\alpha_1$ già osservata).
2. Si porta la formula risultante nella forma normale congiuntiva *CNF* usando l'ulteriore equivalenza tra $\alpha_0 \vee (\alpha_1 \wedge \alpha_2)$ e $(\alpha_0 \vee \alpha_1) \wedge (\alpha_0 \vee \alpha_2)$.

Esempio. Sia $\alpha = \neg(p_0 \wedge \neg(p_1 \wedge \neg p_2))$.

1. α equivale successivamente a

$$\neg p_0 \vee \neg\neg(p_1 \wedge \neg p_2),$$

$$\neg p_0 \vee (p_1 \wedge \neg p_2);$$

nell'ultima formula $\neg$ riguarda solo le variabili p_0, p_1 , ma la forma normale congiuntiva non è ancora raggiunta.

2. Tuttavia $\neg p_0 \vee (p_1 \wedge \neg p_2)$ si scrive equivalentemente

$$(\neg p_0 \vee p_1) \wedge (\neg p_0 \vee \neg p_2)$$

che è in CNF .

Ulteriori dettagli ed esempi su questo algoritmo di traduzione in CNF si possono trovare nei manuali di logica (si vedano le note bibliografiche a fine capitolo). Aggiungiamo che l'algoritmo non è efficiente (dal punto di vista del tempo), né si conoscono attualmente procedimenti rapidi di riduzione in CNF . CNF può comunque rappresentare, nel senso ora descritto, l'intera classe delle formule proposizionali rispetto al problema della soddisfacibilità.

Adesso consideriamo funzioni da $\{0, 1\}^n$ a $\{0, 1\}$. Cominciamo col notare che, ad ogni formula α (in CNF o no) nelle variabili proposizionali $p_0, \dots, p_{n-1}$, resta associata in modo naturale proprio una funzione

$$f_\alpha^n : \{0, 1\}^n \rightarrow \{0, 1\},$$

quella che ad ogni n -upla $(x_0, \dots, x_{n-1})$ in $\{0, 1\}$ associa

$$f_\alpha^n(x_0, \dots, x_{n-1}) = v(\alpha)$$

dove v è una valutazione per cui $v(p_i) = x_i$, per ogni $i < n$. Dunque f_α^n deduce $v(\alpha)$ da $(v(p_0), \dots, v(p_{n-1}))$.

Esempi.

1. Sia $\alpha = \neg p_0$. Ovviamente $f_\alpha^1(1) = 0$, $f_\alpha^1(0) = 1$ perché, se v è una valutazione, $v(p_0) = 0$ se e solo se $v(\neg p_0) = 1$. Nel seguito chiameremo *NOT* la funzione f_α^1 così definita.

2. Per $\alpha = p_0 \vee p_1$, $p_0 \wedge p_1$, si ha rispettivamente

- $f_\alpha^2(1, 1) = f_\alpha^2(1, 0) = f_\alpha^2(0, 1) = 1$, $f_\alpha^2(0, 0) = 0$ nel primo caso;
- $f_\alpha^2(1, 1) = 1$, $f_\alpha^2(1, 0) = f_\alpha^2(0, 1) = f_\alpha^2(0, 0) = 0$ nel secondo.

Per esempio, nel caso di $\vee$, si ricorda che, se v è una valutazione,

$$v(p_0 \vee p_1) = 1$$

se e solo se $v(p_0) = 1$ o $v(p_1) = 1$, e dunque

$$v(p_0 \vee p_1) = 0$$

se e solo se $v(p_0) = 0$ e $v(p_1) = 0$. Ci riferiremo alle due funzioni ora introdotte indicandole con *OR*, *AND* nel seguito.

3. Sia $\alpha = \neg p_0 \vee p_1$. Allora una valutazione v dà valore 1 ad α se e solo se dà valore 0 a p_0 oppure 1 a p_1 ; dunque si ha $f_\alpha^2(1, 1) = f_\alpha^2(0, 1) = f_\alpha^2(0, 0) = 1$, $f_\alpha^2(1, 0) = 0$.

4. Sia $\alpha = p_0 \wedge \neg p_1 \wedge p_2$. Allora una valutazione v dà valore 1 ad α se e solo se $v(p_0) = v(p_2) = 1$ e $v(p_1) = 0$. Così

$$f_{\alpha}^3(1, 0, 1) = 1$$

mentre f_{α}^3 vale 0 in tutti gli altri casi.

In riferimento alle funzioni *NOT*, *OR*, *AND* appena introdotte, si può notare quanto segue.

Teorema 9.1.1 *Per ogni formula α (con variabili $p_0, \dots, p_{n-1}$), la funzione f_{α}^n si esprime come composizione di *NOT*, *AND*, *OR*.*

Dimostrazione. Basterà osservare che, per α_0, α_1 formule nelle variabili $p_0, \dots, p_{n-1}$, si ha, per ogni scelta di $x_0, \dots, x_{n-1} \in \{0, 1\}$,

- (i) $f_{\neg\alpha_0}^n(x_0, \dots, x_{n-1}) = NOT(f_{\alpha_0}^n(x_0, \dots, x_{n-1}))$,
- (ii) $f_{\alpha_0 \vee \alpha_1}^n(x_0, \dots, x_{n-1}) = OR(f_{\alpha_0}^n(x_0, \dots, x_{n-1}), f_{\alpha_1}^n(x_0, \dots, x_{n-1}))$,
- (iii) $f_{\alpha_0 \wedge \alpha_1}^n(x_0, \dots, x_{n-1}) = AND(f_{\alpha_0}^n(x_0, \dots, x_{n-1}), f_{\alpha_1}^n(x_0, \dots, x_{n-1}))$.

Vediamo per esempio (i). Sappiamo che, per ogni scelta di $x_0, \dots, x_{n-1} \in \{0, 1\}$,

$$f_{\alpha_0}^n(x_0, \dots, x_{n-1}) = v(\alpha_0), \quad f_{\neg\alpha_0}^n(x_0, \dots, x_{n-1}) = v(\neg\alpha_0)$$

dove v è una valutazione per cui $v(p_i) = x_i$ per ogni $i < n$. Inoltre $v(\neg\alpha_0) = 1 - v(\alpha_0) = NOT(v(\alpha_0))$. La tesi segue ovviamente. I casi $\vee, \wedge$ si trattano in modo analogo.

Applicando ripetutamente la precedente osservazione, si deduce facilmente che ogni funzione f_{α}^n si può ottenere come composizione di *NOT*, *OR*, *AND* a partire dalle variabili $x_0, \dots, x_{n-1}$ cui f_{α}^n si applica. $\square$

Per illustrare l'affermazione, consideriamo il caso dell'esempio 4 di cui sopra: per $x_0, x_1, x_2 \in \{0, 1\}$, si ricava facilmente

$$f_{\alpha}^3(x_0, x_1, x_2) = AND(x_0, AND(NOT(x_1), x_2)),$$

o anche

$$f_{\alpha}^3(x_0, x_1, x_2) = AND(AND(x_0, NOT(x_1)), x_2).$$

Per ogni n naturale > 0 , si costruiscono così alcune funzioni da $\{0, 1\}^n$ a $\{0, 1\}$. Vogliamo mostrare che, in realtà, tutte le funzioni da $\{0, 1\}^n$ a $\{0, 1\}$ si ottengono per questa via.

Teorema 9.1.2 *Sia n un intero positivo. Allora ogni funzione da $\{0, 1\}^n$ a $\{0, 1\}$ si può ottenere come f_{α}^n per qualche formula α nelle variabili $p_0, \dots, p_{n-1}$ (e dunque come composizione di *NOT*, *AND*, *OR*).*

Si noti che la scelta di α e la conseguente rappresentazione $f = f_\alpha^n$ non è necessariamente unica, come appena mostrato nel caso dell'esempio 4.

Dimostrazione. La tesi è banale quando f è la funzione identicamente 1, quella che associa 1 ad ogni n -upla in $\{0, 1\}^n$. Infatti la formula

$$\alpha = p_0 \vee \neg p_0$$

(nella quale la presenza delle ulteriori variabili proposizionali $p_1, \dots, p_{n-1}$ si intende puramente fittizia) ha ovviamente valore 1 in ogni valutazione v , e dunque soddisfa

$$f_\alpha^n(x_0, \dots, x_{n-1}) = 1 \text{ per ogni } x_0, \dots, x_{n-1} \in \{0, 1\}$$

e coincide con f .

In generale, le funzioni da $\{0, 1\}^n$ - un insieme con 2^n elementi - a $\{0, 1\}$ - che di elementi ne ha 2 - sono 2^{2^n} ; escludendo quella identicamente 1, ne restano $2^{2^n} - 1$. Allora ci basta costruire nelle variabili $p_0, \dots, p_{n-1}$ $2^{2^n} - 1$ formule α (addirittura in *CNF*) tali che le funzioni f_α^n sono a due a due distinte e non identicamente 1. Per introdurre le varie α , ci conviene ancora fare riferimento (almeno per una volta) alla loro rappresentazione come insiemi non vuoti di parole su lettere minuscole $p_0, \dots, p_{n-1}$ e sulle loro "negazioni" maiuscole $P_0, \dots, P_{n-1}$. In effetti, le clausole che si possono formare inserendo, per ogni $i < n$, esattamente una lettera tra p_i e P_i (dunque p_0 oppure P_0 , p_1 oppure P_1 , e così via) sono 2^n . Gli insiemi non vuoti - cioè le formule α in *CNF* - che conseguentemente si possono costruire con queste parole sono $2^{2^n} - 1$: infatti i sottoinsiemi di un insieme con 2^n elementi sono in tutto 2^{2^n} , ma dobbiamo escludere l'insieme vuoto.

Consideriamo le $2^{2^n} - 1$ formule α così ottenute. Ci basta dimostrare che ciascuna di esse definisce una funzione $f_\alpha^n \neq 1$ e che parole distinte $\alpha \neq \alpha'$ determinano funzioni distinte $f_\alpha^n \neq f_{\alpha'}^n$.

A questo proposito ci conviene ricordare che una arbitraria valutazione v assegna valore 1 ad una tale formula α se e solo se, in ogni clausola di α , v trova una lettera (minuscola o maiuscola) cui associa 1.

Siano allora $\alpha \neq \alpha'$ due di queste formule. Allora c'è una clausola in α e non in α' , o viceversa. Supponiamo per semplicità

$$p_0 P_1 p_2 \cdots p_{n-1} \in \alpha - \alpha'.$$

In particolare, in ogni clausola di α' compare P_0 o p_1 o P_2 o ... P_n . Sia $\bar{v}$ una valutazione tale che

$$\bar{v}(P_0) = \bar{v}(p_1) = \bar{v}(P_2) = \cdots = \bar{v}(P_{n-1}) = 1,$$

Allora $\bar{v}(\alpha') = 1$, ma $\bar{v}(\alpha) = 0$ perché nella clausola $p_0 P_1 p_2 \cdots p_{n-1} \in \alpha$ non c'è lettera cui $\bar{v}$ assegni il valore 1. Ne segue

$$f_\alpha^n(\bar{v}(p_0), \dots, \bar{v}(p_{n-1})) = 0, \quad f_{\alpha'}^n(\bar{v}(p_0), \dots, \bar{v}(p_{n-1})) = 1,$$

quindi $f_\alpha^n \neq f_{\alpha'}^n$.

Allo stesso modo si prova che, per ogni formula α , $f_\alpha^n \neq 1$: basta fare riferimento a qualche clausola in α e definire $\bar{v}$ in modo tale da assegnare valore 0 ad ogni lettera nella clausola fissata. Ovviamente $\bar{v}(\alpha) = 0$, così $f_\alpha^n \neq 1$. $\square$

9.2 Ancora logica: le formule Booleane quantificate

Prima di proseguire e approfondire il discorso sulle funzioni da $\{0, 1\}^n$ a $\{0, 1\}$, apriamo in questo paragrafo una breve parentesi su un argomento di logica che ci sarà utile nel seguito.

Sia α una formula della logica proposizionale con variabili $p_0, p_1, \dots, p_{n-1}$; magari scriviamo $\alpha(p_0, \dots, p_{n-1})$ a sottolineare questa situazione. Ricordiamo che α è soddisfacibile se è possibile assegnare alle sue variabili $p_0, \dots, p_{n-1}$ valori 0,1 tramite una qualche valutazione v in modo tale che si abbia di conseguenza $v(\alpha) = 1$.

Possiamo allora esprimere la soddisfacibilità di α usando il quantificatore $\exists$ (esiste) per scrivere

$$\exists p_0 \cdots \exists p_{n-1} \alpha(p_0, \dots, p_{n-1}).$$

Si intende che la verità di questa nuova formula corrisponde, appunto, alla soddisfacibilità di α , e dunque è assolutamente indipendente dal riferimento a una qualche particolare valutazione. Infatti

- o c'è in generale una valutazione v di $p_0, \dots, p_{n-1}$ tale che $v(\alpha) = 1$, e in tal caso $\exists p_0 \cdots \exists p_{n-1} \alpha(p_0, \dots, p_{n-1})$ è da intendersi vera,
- o questa valutazione non c'è, e $\exists p_0 \cdots \exists p_{n-1} \alpha(p_0, \dots, p_{n-1})$ è da intendersi falsa.

Possiamo anche coinvolgere il quantificatore $\forall$ (per ogni) per costruire nuove formule di analoga struttura, quale

$$\forall p_0 \cdots \forall p_{n-1} \alpha(p_0, \dots, p_{n-1})$$

da intendersi vera o falsa a seconda che $v(\alpha) = 1$ per ogni valutazione v di $p_0, \dots, p_{n-1}$ oppure no.

In generale possiamo considerare le seguenti nuove formule.

Definizione. Una *formula Booleana quantificata* ha la formula

$$Q_0 p_0 Q_1 p_1 \cdots Q_{n-1} p_{n-1} \alpha(p_0, \dots, p_{n-1})$$

dove α è una formula proposizionale con variabili $p_0, \dots, p_{n-1}$ e $Q_0, \dots, Q_{n-1}$ sono quantificatori $\exists$ o $\forall$.

QBF indica l'insieme delle formule Booleane quantificate (la sigla *QBF* deriva, ovviamente, dall'inglese *quantified Boolean formula*).

Una formula in *QBF* si dice vera o falsa secondo i criteri già anticipati nei casi precedenti, e ribaditi per chiarezza nell'esempio che segue.

Esempio. $\exists p_0 \forall p_1 \forall p_2 \alpha(p_0, p_1, p_2)$ è vera se è possibile assegnare a p_0 un valore 0 o 1 tale che, qualunque valore 0 o 1 si dia a p_1 e p_2 , la valutazione v risultante soddisfa $\alpha(p_0, p_1, p_2)$; falsa altrimenti.

Esercizio. Le seguenti formule di QBF

- $\exists p_0 \forall p_1 \forall p_2 (p_0 \wedge p_1 \wedge p_2),$
- $\forall p_0 \forall p_1 \exists p_2 ((p_0 \wedge p_2) \vee (p_1 \wedge \neg p_2))$

sono vere o false?

È talora utile inserire in QBF due formule senza variabili proposizionali e senza quantificatori

- T (per “*true*”, vero in inglese) da intendersi sempre vera,
- F (per “*false*”, falso) da intendersi sempre falsa.

T, F possono essere coinvolte in altre formule Booleane quantificate, al posto di variabili, come nel caso di

$$\alpha : \forall p_0 \exists p_1 (T \wedge (p_0 \vee p_1)).$$

Questa formula risulta vera se e solo se, per ogni valore di p_0 ce n'è uno di p_1 per cui si soddisfa $T \wedge (p_0 \wedge p_1)$, e cioè $p_0 \vee p_1$. Così basta supporre $v(p_1) = 1$ per dedurre la verità di α .

Sorge ovviamente il problema di riconoscere le formule Booleane quantificate vere e distinguerle dalle false. Indicheremo nel seguito con $QSAT$ l'insieme delle formule Booleane quantificate vere.

9.3 Circuiti

Torniamo adesso a considerare funzioni f da $\{0, 1\}^n$ a $\{0, 1\}$ o anche, più generalmente, da $\{0, 1\}^n$ a $\{0, 1\}^m$ dove m e n sono interi positivi. Esse si chiamano anche *porte logiche*. Già conosciamo le tre porte logiche

- NOT (con $n = m = 1$),
- AND, OR (con $n = 2, m = 1$).

Sappiamo poi che ogni porta logica da $\{0, 1\}^n$ a $\{0, 1\}$ (quindi con $m = 1$) è la composizione di *NOT*, *AND*, *OR*.

In realtà questa affermazione si allarga al caso $m > 1$. Infatti una funzione f da $\{0, 1\}^n$ a $\{0, 1\}^m$ si può identificare con la m -upla delle funzioni da $\{0, 1\}^n$ a $\{0, 1\}$ che associano ad ogni $x \in \{0, 1\}^n$ la prima, la seconda, ..., la m -ma componente di $f(x)$. Così, per determinare $f(x)$, conviene ricopiare preventivamente l'argomento $x \in \{0, 1\}^n$ per m volte e poi, per ogni $j = 1, \dots, m$, adoperare la j -ma copia per calcolare la j -ma componente di $f(x)$, ricomporre finalmente $f(x)$ come la m -upla di queste componenti. Ciascuna di esse si ottiene, come sappiamo, da *NOT*, *AND*, *OR* mediante opportune composizioni. Di conseguenza la costruzione di f richiede *NOT*, *AND*, *OR*, ma anche un'ulteriore funzione di copia, e cioè una porta logica da $\{0, 1\}$ a $\{0, 1\}^2$ che duplica ogni $x_0 \in \{0, 1\}$ e produce (x_0, x_0) : la si chiama *FANOUT*, e un suo uso ripetuto permette di ottenere sequenze ordinate arbitrariamente lunghe di x_0 , in particolare m -uple $(x_0, \dots, x_0)$.

In conclusione, ogni porta logica f si può esprimere come composizione di *NOT*, *AND*, *OR* e *FANOUT*. Si dice allora che *NOT*, *AND*, *OR* e *FANOUT* formano un *insieme universale di porte logiche*.

Esercizio. Si noti che, per ogni scelta di $x_0, x_1 \in \{0, 1\}$,

$$OR(x_0, x_1) = NOT(AND(NOT(x_0), NOT(x_1))).$$

Si deduca che anche *NOT*, *AND* e *FANOUT* costituiscono un insieme universale di porte logiche (cioè che ogni porta logica si rappresenta come loro composizione). Si può dire lo stesso di *NOT*, *OR*, *FANOUT*?

Torniamo ad una generica porta logica da $\{0, 1\}^n$ a $\{0, 1\}^m$. Sia $m = 1$. Allora f si rappresenta tramite *NOT*, *AND*, *OR*, e una sua decomposizione in questa forma si può descrivere graficamente con un opportuno circuito. Ricordiamo infatti:

Definizione. Un *circuito* è un grafo diretto $G = (V, E)$ privo di cicli.

Ricordiamo anche che, in un grafo diretto G , E è una qualunque relazione binaria: per $v, w \in V$, $(v, w) \in E$ significa che c'è un lato - a senso unico - da v a w , ma questo non implica - né esclude - che ci sia un altro lato da w a v , cioè che $(w, v) \in E$. È poi ammessa la possibilità che si abbia $(v, v) \in E$ per qualche $v \in V$.

Ricordiamo infine che un *ciclo* di G è una sequenza $v_0, \dots, v_n$ di vertici con $(v_i, v_{i+1}) \in E$ per ogni $i < n$, $(v_i, v_{i+1}) \neq (v_j, v_{j+1})$ per $i < j \leq n$, e $(v_n, v_0) \in E$.

Vediamo allora alcuni semplici esempi di circuiti associati a funzioni f da $\{0, 1\}^n$ a $\{0, 1\}$, più precisamente alle loro rappresentazioni tramite *NOT*, *OR*, *AND*.

Esempi.

1. La porta logica chiamata *NAND* è la funzione da $\{0, 1\}^2$ a $\{0, 1\}$ che si

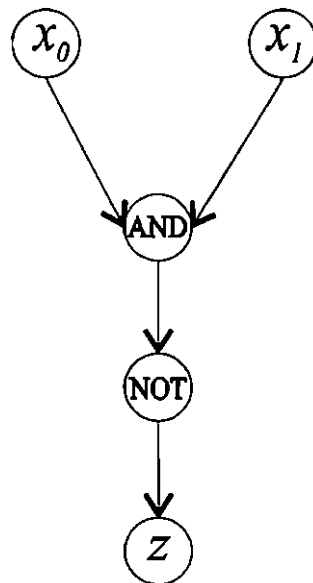
ottiene componendo *NOT* e *AND*: per $x_0, x_1 \in \{0, 1\}$,

$$NAND(x_0, x_1) = NOT(AND(x_0, x_1)).$$

Così $NAND = f_\alpha^2$ dove α è la formula $\neg(p_0 \wedge p_1)$. In dettaglio

$$NAND(1, 1) = 0, \quad NAND(0, 1) = NAND(1, 0) = NAND(0, 0) = 1.$$

Il circuito che descrive *NAND* in riferimento alla rappresentazione come composizione di *NOT* e *AND* è il seguente.



Si riconoscono:

- due vertici input corrispondenti alle variabili x_0, x_1 da cui escono lati di E , ma a cui non arrivano lati di E ;
- due vertici corrispondenti alle porte logiche *NOT*, *AND*; essi ammettono lati di E sia in entrata che in uscita;
- un vertice output, corrispondente al valore $z = NAND(x_0, x_1)$ che ha un lato in entrata, ma nessuno in uscita.

Si osservi anche che, per ogni scelta di $x_0, x_1 \in \{0, 1\}$,

- $NOT(x_0) = NAND(x_0, x_0) = NAND(FANOUT(x_0))$,
- $AND(x_0, x_1) = NOT(NAND(x_0, x_1))$.

Quindi *NAND* e *FANOUT* costituiscono da soli un insieme universale di porte logiche, perché generano *NOT*, *AND*, dunque *OR*, e tutte le loro composizioni

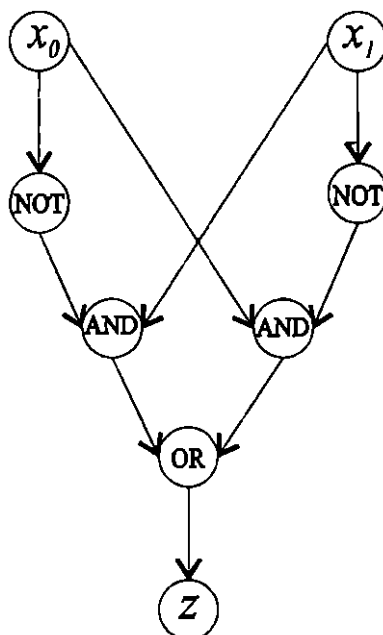
2. Sia ora $f = \oplus$ l'addizione modulo 2 in $\{0, 1\}$, quella per cui

$$1 \oplus 1 = 0 \oplus 0 = 0, \quad 0 \oplus 1 = 1 \oplus 0 = 1$$

e in generale, per ogni $x_0, x_1 \in \{0, 1\}$, $x_0 \oplus x_1 = x_0 + x_1 \pmod{2}$. Si può notare che f coincide con f_α^2 dove $\alpha = (p_0 \wedge \neg p_1) \vee (p_1 \wedge \neg p_0)$ (il lettore può verificare da solo il perché). Dunque

$$x_0 \oplus x_1 = OR(AND(x_0, NOT(x_1)), AND(NOT(x_0), x_1)).$$

Il circuito che ne deriva è il seguente.

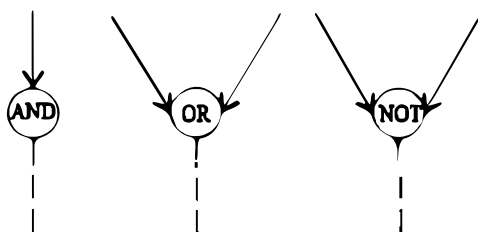


Ci sono

- due vertici input con lati in uscita e non in entrata;
- cinque vertici corrispondenti ai connettivi (due per *NOT*, due per *AND*, uno per *OR*) con lati tanto in uscita che in entrata;
- un vertice output con lati in entrata, ma non in uscita.

In generale, il circuito associato a $f : \{0, 1\}^n \rightarrow \{0, 1\}$ e ad una sua data rappresentazione tramite *NOT*, *OR*, *AND* ammette

- n vertici input per $x_0, \dots, x_{n-1}$, i quali ammettono lati in uscita e non in entrata;
- vertici corrispondenti alle porte logiche *NOT*, *OR*, *AND*; ciascuno di essi ammette lati in entrata (uno per *NOT*, due per *OR* e *AND*) e lati in uscita;
- un vertice output per $z = f(x_0, \dots, x_{n-1})$ con lati in entrata, ma non in uscita.



Nel caso in cui $m > 1$ e cioè f va da $\{0, 1\}^n$ a $\{0, 1\}^m$ con $m > 1$, la descrizione di f si può ridurre a quella delle sue m componenti grazie a *FANOUT*, riproducendo m volte ogni suo argomento x , e quindi si può ottenere con i corrispondenti m circuiti.

Altri esempi ed esercizi.

3. Una porta logica $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ si dice *reversibile* se è iniettiva, e dunque da ogni possibile immagine di f si può risalire ad un'unica retroimmagine. Si mostri che, per $n = m$, f è reversibile se e solo se è una biiezione.

Quali delle porte logiche *NOT*, *AND*, *OR*, $\oplus$, *NAND* sono reversibili? È vero che la composizione di porte logiche reversibili resta reversibile?

4. *XOR* è la porta logica da $\{0, 1\}^2$ a $\{0, 1\}^2$ tale che, per ogni scelta di x_0, x_1 in $\{0, 1\}$,

$$XOR(x_0, x_1) = (x_0, x_0 \oplus x_1).$$

Le sue componenti trasformano la generica coppia (x_0, x_1) in $x_0, x_0 \oplus x_1$ rispettivamente. Il lettore può costruire i due circuiti relativi.

XOR è reversibile? (*XOR* è talora denotata *CNOT*).

5. *SWAP* è la porta logica da $\{0, 1\}^2$ a $\{0, 1\}^2$ tale che, per ogni scelta di x_0, x_1 in $\{0, 1\}$,

$$SWAP(x_0, x_1) = (x_1, x_0).$$

(Del resto *swap* è la parola inglese che traduce *scambio*). *SWAP* è reversibile? Si esprimano le componenti di *SWAP* come composizioni di *NOT*, *AND*, *OR* e si disegnino i relativi circuiti. È possibile ottenere *SWAP* come composizione di *NOT*, *XOR*?

6. Consideriamo ora *FANOUT*, cioè la porta logica da $\{0, 1\}$ a $\{0, 1\}^2$ che “duplica” ogni $x_0 \in \{0, 1\}$, nel senso che $FANOUT(x_0) = (x_0, x_0)$. Si noti che *FANOUT* è reversibile ma non è una biiezione.

7. Consideriamo la porta logica da $\{0, 1\}^3$ a $\{0, 1\}^3$ che associa alla generica terna $(x_0, x_1, x_2) \in \{0, 1\}^3$

$$(x_0, x_1, x_2 \oplus AND(x_0, x_1));$$

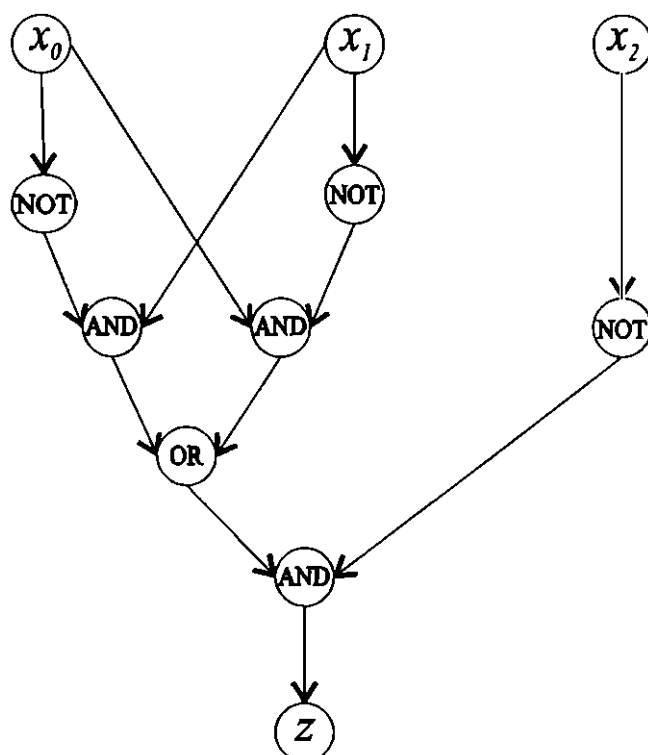
per esempio l'immagine di $(1, 1, 1)$ è $(1, 1, 0)$ perché $AND(1, 1) = 1$ e $1 \oplus 1 = 0$. Essa è detta porta di *Toffoli* (*Toffoli gate*), dal nome di chi - Toffoli appunto - osservò che essa, pur essendo reversibile, non si ottiene come composizione di *NOT* e *XOR*. Sapreste provare queste affermazioni?

La porta di Toffoli ha anche un'altra notevole proprietà: costituisce da sola un insieme universale, capace cioè di generare (semmai con l'aiuto marginale di funzioni costanti e proiezioni dalle terne di $\{0, 1\}^3$ alle tre coordinate in $\{0, 1\}$) qualunque altra porta logica, come provato in [107].

Abbiamo visto come associare a ogni funzione da $\{0, 1\}^n$ a $\{0, 1\}$ un circuito. Viceversa, ogni circuito C della forma sopra descritta determina una funzione f_C da $\{0, 1\}^n$ a $\{0, 1\}$.

Esempio.

8. Consideriamo il circuito C



Esso determina la funzione f_C da $\{0, 1\}^3$ a $\{0, 1\}$ tale che, per ogni scelta di $x_0, x_1, x_2 \in \{0, 1\}$,

$$f_C(x_0, x_1, x_2) = AND(x_0 \oplus x_1, NOT(x_2))$$

e corrisponde dunque alla formula

$$\alpha = ((\neg p_0 \wedge p_1) \vee (\neg p_1 \wedge p_0)) \wedge \neg p_2$$

(in altre parole $f_C = f_\alpha^3$).

Possiamo misurare la complessità di un circuito C (e quindi quella della funzione f_C che gli è associata) con vari criteri:

- possiamo per esempio riferirci alla *profondità* del circuito, e cioè alla massima lunghezza di un cammino da un nodo input ad un nodo output; la profondità è dunque 3 nell'esempio 1, 4 nell'esempio 2, 5 nell'esempio 8;
- alternativamente considerare il numero dei lati del circuito (4, 9, 12 negli esempi 1, 2, 8 rispettivamente);
- oppure, ancora, contare le porte logiche coinvolte nel circuito (2, 5, 7 nei tre esempi sopra citati, in ordine).

Preferiamo qui privilegiare un ulteriore criterio che calcola il numero delle porte logiche diverse da *NOT* (quindi 1, 3, 5 nei tre esempi). Questo numero si chiamerà la *misura* del circuito C . Vi sono infatti motivazioni non infondate per escludere *NOT*:

- anzitutto *NOT* è l'unica porta logica coinvolta che è 1-aria, e cioè richiede un solo lato in entrata; dunque le occorrenze di *NOT* non incidono significativamente sulla "complessità" del circuito, tanto più che ripetizioni *NOT NOT* (cioè $\neg\neg$) non sono ammesse, o comunque si possono evitare e dimenticare;

- di più, per $\alpha \in CNF$, $\neg$ opera al più sulle variabili proposizionali, dunque, come porta logica, *NOT* interviene al più sui vertici input.

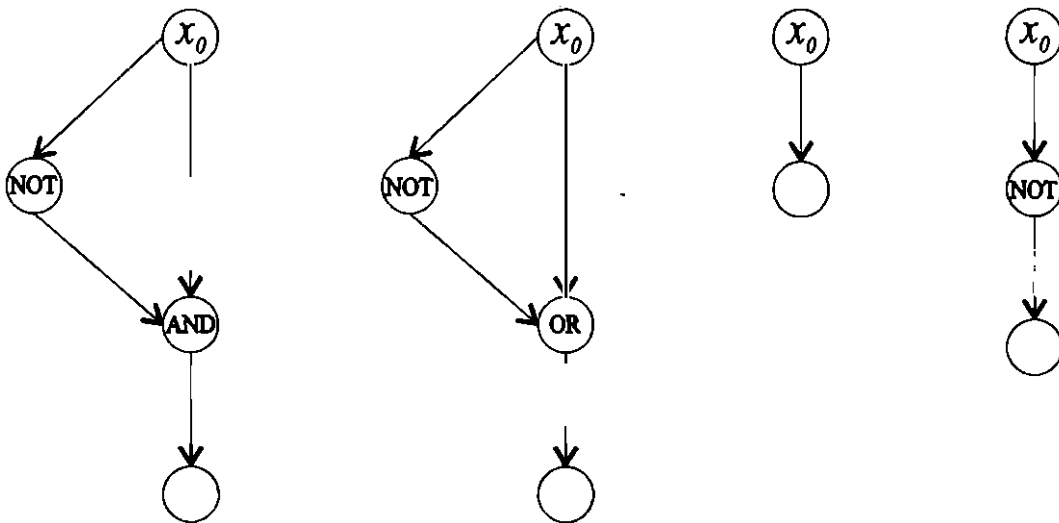
Naturalmente una funzione $f : \{0, 1\}^n \rightarrow \{0, 1\}$ può corrispondere a più circuiti anche di misura diversa in ragione di più possibili decomposizioni con *NOT*, *OR*, *AND*. Sarà opportuno, in questi casi, riferirsi alla rappresentazione che ha un circuito di misura minima. In effetti la misura di questi circuiti - con n vertici input - si può limitare come segue.

Teorema 9.3.1 *Sia s la funzione dagli interi positivi agli interi positivi definita ponendo*

$$s(1) = 1, \quad s(n+1) = 2 \cdot s(n) + 3 \text{ per ogni } n > 0.$$

Allora, per ogni intero positivo n , qualunque porta logica f da $\{0, 1\}^n$ a $\{0, 1\}$ corrisponde ad un circuito di misura $\leq s(n)$.

Dimostrazione. Supponiamo dapprima $n = 1$. Ci sono quattro possibili funzioni da $\{0, 1\}$ a $\{0, 1\}$ (quindi per $n = 1$), rispettivamente le funzioni costanti 0, 1, l'identità e *NOT*: in dettaglio: $f(x_0) = 0$, $f(x_0) = 1$, $f(x_0) = x_0$, $f(x_0) = NOT(x_0)$ per ogni $x_0 \in \{0, 1\}$. Esse corrispondono ai circuiti



di misura massima 1. Così ogni funzione da $\{0, 1\}$ a $\{0, 1\}$ corrisponde ad un circuito di misura $\leq 1 = s(1)$.

Sia ora $n > 0$ e consideriamo $n+1$. Notiamo che una funzione f da $\{0, 1\}^{n+1}$ a $\{0, 1\}$ si può decomporre, per ogni $x_0, x_1, \dots, x_n \in \{0, 1\}$, nel modo che segue

$$\begin{aligned} f(x_0, x_1, \dots, x_n) &= \\ &= OR(AND(x_0, f(1, x_1, \dots, x_n)), AND(NOT(x_0), f(0, x_1, \dots, x_n))). \end{aligned}$$

Infatti, per $x_0 = 1$,

$$\begin{aligned} OR(AND(1, f(1, x_1, \dots, x_n)), AND(0, f(0, x_1, \dots, x_n))) &= \\ = f(1, x_1, \dots, x_n) &= f(x_0, x_1, \dots, x_n) \end{aligned}$$

e, per $x_0 = 0$,

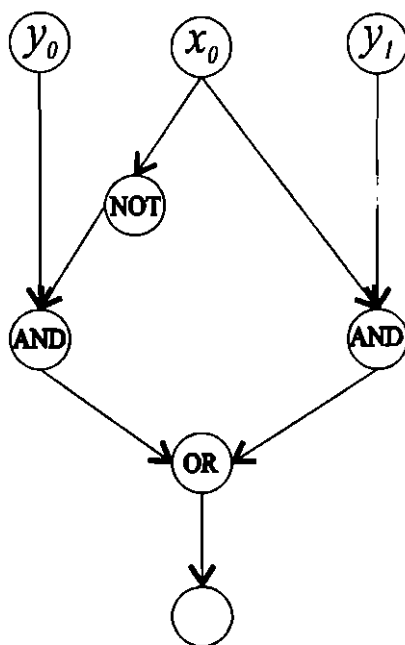
$$\begin{aligned} OR(AND(0, f(1, x_1, \dots, x_n)), AND(1, f(0, x_1, \dots, x_n))) &= \\ = f(0, x_1, \dots, x_n) &= f(x_0, x_1, \dots, x_n). \end{aligned}$$

f viene così riferita a due funzioni da $\{0, 1\}^n$ a $\{0, 1\}$

$$(x_1, \dots, x_n) \mapsto f(0, x_1, \dots, x_n),$$

$$(x_1, \dots, x_n) \mapsto f(1, x_1, \dots, x_n),$$

ambidue corrispondenti a circuiti di misura $\leq s(n)$, C_0, C_1 rispettivamente. D'altra parte, un circuito per f è quello che raggiunge da $x_1, \dots, x_n$ $y_1 = f(1, x_1, \dots, x_n)$ da $x_1, \dots, x_n$ con C_1 , e $y_0 = f(0, x_1, \dots, x_n)$ con C_0 , procede poi come disegnato



La sua misura è $\leq 2 \cdot s(n) + 3$. Così ogni funzione da $\{0, 1\}^{n+1}$ a $\{0, 1\}$ corrisponde ad almeno un circuito di misura $\leq 2 \cdot s(n) + 3 = s(n+1)$. $\square$

9.4 Circuiti e complessità

Torniamo ora a considerare problemi (nel senso di insiemi S di parole su un alfabeto finito A). Come già convenuto, supponiamo che A si riduca a $\{0, 1\}$. Possiamo allora associare a S una collezione di funzioni

$$f_n : \{0, 1\}^n \rightarrow \{0, 1\}$$

al variare di n tra gli interi positivi; per ogni n e per ogni parola w di lunghezza n su $\{0, 1\}$ (cioè per ogni $w \in \{0, 1\}^n$),

$$f_n(w) = 1 \text{ se } w \in S,$$

$$f_n(w) = 0 \text{ se } w \notin S.$$

Ciascuna f_n si può rappresentare (eventualmente in più modi) come composizione di NOT , OR , AND e quindi corrisponde (eventualmente in più modi) a circuiti C per cui $f_n = f_C$. Resta allora giustificata la seguente

Definizione. $S \subseteq \{0, 1\}^*$ è risolto dalla sequenza di circuiti $C_1, C_2, \dots, C_n, \dots$ (n intero positivo) se, per ogni $n > 0$ e per ogni $w \in \{0, 1\}^n$,

$$w \in S \text{ se e solo se } f_{C_n}(w) = 1.$$

Vale la pena di sottolineare che un insieme S risolto da una sequenza di circuiti $C_1, C_2, \dots, C_n, \dots$ può benissimo non essere ricorsivo, e cioè deciso da una macchina di Turing. Per esempio, per K insieme di naturali non ricorsivo, formiamo l'insieme S_K che contiene tutte le parole di lunghezza n al variare di $n \in K$, e non altre parole. Per ogni $n > 0$, possiamo costruire un circuito C_n che su ogni input w di lunghezza n dà output 1 se $n \in K$, e 0 se $n \notin K$. C_n può essere ottenuto di misura 1. Ovviamente S_K è risolto da $C_1, C_2, \dots, C_n, \dots$, ma S_K non può essere deciso da alcuna macchina di Turing.

Poniamo allora. In effetti, un insieme $S \subseteq \{0, 1\}^*$ è ricorsivo se e solo se è risolto da una sequenza “uniforme” di circuiti $C_1, C_2, \dots, C_n, \dots$, nel senso che c'è una MdT deterministica che genera, per ogni intero positivo n , il corrispondente circuito C_n .

Esercizi.

1. Si provi l'ultima affermazione.
2. Si dimostri che l'insieme S_K sopra introdotto non può essere risolto da una sequenza uniforme di circuiti.
3. Si mostri che un insieme $S \subseteq \{0, 1\}^*$ è in P se e solo se è risolto da una famiglia uniforme $C_1, C_2, \dots, C_n, \dots$ di circuiti tale che c'è una MdT deterministica che genera ciascun C_n da n lavorando in tempo polinomiale. Prescindendo dall'ipotesi di uniformità, poniamo comunque:

Definizione. Siano $S \subseteq \{0, 1\}^*$, f una funzione da $\mathbb{N}$ a $\mathbb{N}$. Diciamo che $S \in SIZE(f(n))$ se e solo se S è risolto da una sequenza di circuiti $C_1, C_2, \dots, C_n, \dots$, tale che, per ogni $n > 0$, C_n ha misura $\leq f(n)$.

Per esempio, $S \in SIZE(O(n^k))$ per qualche k significa che S è risolto da una sequenza di circuiti $C_1, C_2, \dots, C_n, \dots$, tale che la funzione che ad ogni $n > 0$, C_n associa la misura di C_n è al più polinomiale.

A proposito, $SIZE(n^{O(1)})$ può denotare per semplicità l'intera classe dei problemi S di $SIZE(O(n^k))$ (infatti una funzione da $\mathbb{N}$ a $\mathbb{N}$ è $O(1)$ se e solo se è costante).

È poi da sottolineare quanto segue.

Osservazione. Ogni problema $S \subseteq \{0, 1\}^*$ è in $SIZE(O(2^n))$.

Infatti abbiamo visto nello scorso paragrafo che ogni funzione $f : \{0, 1\}^n \rightarrow \{0, 1\}$ corrisponde ad un circuito di misura $\leq s(n)$ dove

$$s(1) = 1,$$

$$s(n+1) = 2 \cdot s(n) + 3.$$

Approssimativamente, questa limitazione raddoppia per ogni incremento unitario di n , da n a $n + 1$. Se ne deduce $s(n) = O(2^n)$.

Adesso confrontiamo le nuove classi di complessità per circuiti, ed in particolare $SIZE(n^{O(1)})$, con quelle già introdotte negli scorsi capitoli, come P e NP . È da ricordare che $SIZE(n^{O(1)})$ include anche problemi che nessuna MdT sa decidere. Addirittura, ci sono tali problemi in $SIZE(1)$ come l'esempio S_K sopra accennato mostra. Comunque si ha

Teorema 9.4.1 $P \subseteq SIZE(n^{O(1)})$. Più in generale, se $S \subseteq \{0, 1\}^*$ è un problema accettato deterministicamente in tempo $t(n)$, allora $S \in SIZE(O(t^2(n)))$.

Dimostrazione. Basta provare la seconda tesi, infatti, quando $t(n)$ è polinomiale, anche $t^2(n)$ lo è. Usiamo idee analoghe a quelle del teorema di Cook-Levin. Sia dunque M una MdT deterministica che accetta S in tempo $\leq t(n)$: per ogni input $w \in \{0, 1\}^*$ di lunghezza n ,

$$M \text{ converge su } w \text{ se e solo se } w \in S,$$

e, in tal caso, la computazione di M richiede al più $t(n)$ passi. Chiaramente la porzione di nastro coinvolta in $t(n)$ passi è composta da al più $2t(n) + 1$ quadri, $t(n)$ a destra e $t(n)$ a sinistra del quadro di partenza (oltre ovviamente al quadro di partenza stesso). Ad ogni istante della computazione, ciascun quadro contiene 0, 1 o il simbolo bianco $\star$, la MdT osserva uno e un solo quadro in un particolare stato (attinto da un insieme finito Q). Tutta la computazione può essere descritta da una matrice

$$(t(n) + 1) \times (2t(n) + 1),$$

le cui righe corrispondono ai vari passi del calcolo, da 0 a $t(n)$, e le cui colonne corrispondono ai quadri coinvolti (da 0 a $2t(n)$, intendendo con $t(n)$ quello di partenza). Per ogni $i \leq t(n)$ e per ogni $j \leq 2t(n)$, la matrice contiene al posto (i, j)

- (q, a) se M sta esaminando al tempo i proprio il quadro j , vi legge a ed è allo stato $q \in Q$,
- $(\square, a)$ se a è scritto al tempo i sul quadro j , ma M sta esaminando un altro quadro al tempo i ;

$\square$ ha quindi la funzione di stato muto, paragonabile al ruolo di $\star$ rispetto ad A . Per esempio, la prima riga della matrice, quella relativa al tempo 0, è

$$((\square, \star), (\square, \star), \dots, (\square, \star), (q_0, w_0), (\square, w_1), \dots, (\square, \star), (\square, \star)),$$

dove $w = w_0 w_1 \dots w_{n-1}$ e (q_0, w_0) corrisponde ovviamente alla colonna $t(n)$. Tutte le coppie coinvolte nella matrice si possono codificare come sequenze di 0, 1 di un'opportuna lunghezza k , dipendente dal numero degli elementi di A e degli stati della MdT, ma non da n . Così possiamo pensare che gli elementi

della matrice siano vettori in $\{0, 1\}^k$. Continuiamo comunque, per semplicità, a denotarli come stabilito, come coppie (q, a) con $q \in Q \cup \{\square\}$, $a \in \{\star, 0, 1\}$. Le computazioni di M sugli input di lunghezza n possono quindi descriversi tramite le funzioni f_{ij} (con $i < t(n)$, j di valore assoluto $< 2t(n)$) che, per ogni scelta di i, j , considerano l'elemento di posto i, j della matrice insieme a quelli che lo accompagnano a destra e sinistra sulla stessa riga (dunque ai posti $i, j-1$ e $i, j+1$) - complessivamente una terna di vettori in $\{0, 1\}^k$ e quindi un vettore in $\{0, 1\}^{3k}$ - e gli associamo la terna degli elementi sulle stesse colonne nella riga $i+1$ - ancora un vettore in $\{0, 1\}^{3k}$ -.

Per esempio, se la terna sulla riga i -ma e le colonne $j-1, j, j+1$ è

$$(\square, a_0), (q, a), (\square, a_1) \text{ con } q \in Q,$$

e M ha l'istruzione $(q, a, q', a', -1)$, allora l'immagine in f_{ij} è

$$(q', a_0), (\square, a'), (\square, a_1),$$

e così via.

Per ogni scelta di i, j , f_{ij} è dunque una funzione da $\{0, 1\}^{3k}$ a $\{0, 1\}^{3k}$, ma, come sappiamo, può essere pensata come una sequenza di $3k$ funzioni da $\{0, 1\}^{3k}$ a $\{0, 1\}$, che associano ad ogni elemento di $\{0, 1\}^{3k}$

la prima, la seconda,..., la $3k$ -ma

componente della sua immagine tramite f_{ij} in $\{0, 1\}^{3k}$. Così f_{ij} è determinata da $3k$ circuiti che hanno misura $O(2^k)$ rispetto a k , ma non dipendono da n , e anzi da un unico circuito di misura $\leq 3k \cdot O(2^k) = O(k \cdot 2^k)$ rispetto a k - ancora indipendente da n - che si può ottenere combinando i $3k$ circuiti precedenti in modo opportuno. Le computazioni di M su input di lunghezza n sono allora determinate da

$$t(n) \cdot (2t(n) - 1) \text{ circuiti di misura } O(k \cdot 2^k) \text{ rispetto a } k$$

(uno per ogni $i < t(n)$ e per ogni j con $|j| < 2t(n)$) o, se preferiamo, da un unico circuito complessivo di misura

$$\leq t(n) \cdot (2t(n) - 1) \cdot O(k \cdot 2^k) = O(t^2(n) \cdot k \cdot 2^k) = O(t^2(n))$$

rispetto a n . L'ultimo circuito si costruisce combinando i precedenti con opportuni espedienti tecnici. $\square$

Passiamo adesso a NP . Ci chiediamo se $NP \subseteq SIZE(n^{O(1)})$. A confermare quanto complicata sia la questione $P = NP$, possiamo affermare soltanto che, se NP è incluso, come P , in $SIZE(n^{O(1)})$, allora la gerarchia polinomiale si arresta al secondo livello. Infatti si ha:

Teorema (Karp-Lipton) 9.4.2 *Se $NP \subseteq SIZE(n^{O(1)})$, allora $PH = \sum_2^P$.*

Dimostrazione. Basta provare $\prod_2^P \subseteq \sum_2^P$. Sia allora S un problema (sull'alfabeto $\{0, 1\}$) in $\prod_2^P$. Allora ci sono

- un problema $V_S \in P$,
- un polinomio p_S (a coefficienti interi e valori positivi)

tali che, per ogni w di lunghezza $\leq n$,

$$w \in S$$

se e solo se, per ogni $y_1 \in \{0, 1\}^*$ di lunghezza $\leq p_S(n)$,

($\star$) esiste $y_2 \in \{0, 1\}^*$ di lunghezza $\leq p_S(n)$ tale che $(w, y_1, y_2) \in V_S$.

Notiamo che la condizione ($\star$) su (w, y_1) (con $l(y_1) \leq p_S(l(w))$) determina un insieme $S' \in NP$. Siccome SAT è NP -completo, c'è una funzione computabile in tempo polinomiale g che traduce parole in $\{0, 1\}$ in formule di CNF (opportunamente codificate ancora come parole su $\{0, 1\}$) in modo tale che, per (w, y_1) come sopra,

$$(w, y_1) \in S' \text{ se e solo se } g(w, y_1) \in SAT.$$

Di più, $SAT \in NP$, dunque la nostra ipotesi implica che $SAT \in SIZE(n^{O(1)})$ e ci sono circuiti C_n di misura polinomiale in n tali che, per ogni $w \in \{0, 1\}^*$ di lunghezza n (o anche $\leq n$) e per ogni y_1 di lunghezza $\leq p_S(n)$,

$$g(w, y_1) \in SAT \text{ se e solo se } f_{C_n}(g(w, y_1)) = 1.$$

In realtà, i circuiti in questione si applicano ad arbitrarie formule in CNF di conveniente lunghezza; del resto $g(w, y_1)$ ha lunghezza al più polinomiale rispetto a $l(w)$ e dunque a $l(w, y_1)$. Così potremmo azzardare la seguente eventualità: c'è un polinomio q_S tale che, per $w \in \{0, 1\}^*$ di lunghezza $\leq n$,

$$w \in S$$

se e solo se

($\star$)' esiste un circuito C_n di misura $\leq q_S(n)$ tale che, per ogni $y_1 \in \{0, 1\}^*$ di lunghezza $\leq q_S(n)$, $f_{C_n}(g(w, y_1)) = 1$.

Notiamo che la condizione ($\star$)' certamente corrisponde alla definizione di Σ_2^P , perché C_n ha misura polinomiale in n , e dunque si può codificare con opportuni accorgimenti tecnici come parola su $\{0, 1\}$ di lunghezza dipendente da n tramite un opportuno q_S , y_1 ha lunghezza analoga, g è calcolabile in tempo polinomiale in n e, finalmente, $f_{C_n}(g(w, y_1)) = 1$ determina un problema in P .

Di più, se $w \in S$, la condizione è soddisfatta, come già verificato. Ma potrebbe accadere che $w \notin S$, cioè esiste $\bar{y}_1$ di lunghezza $\leq p_S(n)$ per cui $g(w, \bar{y}_1) \notin SAT$; eppure c'è un circuito C_n corrispondente alla precedente condizione, in particolare tale che, per ogni $y_1 \in \{0, 1\}^*$ di lunghezza $\leq p_S(n)$, $f_{C_n}(g(w, y_1)) = 1$.

Dobbiamo escludere questa eventualità. Ammettiamo allora che un tale C_n esista. In particolare $g(w, \bar{y}_1) \notin SAT$, ma $f_{C_n}(g(w, \bar{y}_1)) = 1$. Ricordiamo che $g(w, \bar{y}_1)$ è una formula in CNF (opportunamente codificata come parola su $\{0, 1\}$). C_n opera in generale su formule di CNF che hanno al più tante variabili proposizionali quante $g(w, \bar{y}_1)$, supponiamo m (non è detto che $m = n$, ma m è polinomialmente dipendente da n). Generalizziamo allora la situazione. Abbiamo un circuito C_n che pretende di riconoscere le formule α di SAT con al più m variabili proposizionali, assegnando loro il valore 1, ma può sbagliare

- perché $f_{C_n}(\alpha) = 1$ ma $\alpha \notin SAT$,
- oppure perché $f_{C_n}(\alpha) = 0$ ma $\alpha \in SAT$.

Nel caso sopra descritto, solo il primo errore è possibile per $\alpha = g(w, \bar{y}_1)$. Mostriamo allora come definire un circuito C'_n (ancora di misura polinomiale in m) che evita proprio gli errori del primo tipo. Questo è ovviamente sufficiente.

C'_n considera la formula $\alpha \in CNF$ con variabili proposizionali $p_0, \dots, p_{m-1}$. Immaginiamo per comodità α come insieme finito non vuoto di clausole, come nei capitoli precedenti. C'_n cerca una valutazione v per cui $v(\alpha) = 1$. C'_n considera p_0 e azzarda almeno momentaneamente la valutazione 1 per p_0 ; in altre parole

- dimentica le clausole di α che contengono p_0 (in quanto già soddisfatte tramite p_0),
- cancella la negazione P_0 nelle clausole che la includono (in quanto P_0 è valutata 0 e la soddisfazione va cercata altrove).

Si ottiene una formula α_0 nelle variabili $p_1, \dots, p_{m-1}$ che può essere proposta a C_n . Se $f_{C_n}(\alpha_0) = 1$, C'_n assume $v(p_0) = 1$ e procede con p_1 , e poi con $p_2, \dots, p_{m-1}$ allo stesso modo. Se no, C'_n opta per $v(p_0) = 0$ o prosegue. Alla fine si determina v , e C'_n controlla che $v(\alpha)$ davvero eguaglia 1. Se sì, conclude $f_{C'_n}(\alpha) = 1$. Se no, pone $f_{C'_n}(\alpha) = 0$.

C'_n può ancora sbagliare - come C_n - perché può essere $f_{C'_n}(\alpha) = 0$ e $\alpha \in SAT$. Ma certamente, se $f_{C'_n}(\alpha) = 1$, allora $\alpha \in SAT$ perché il lavoro di C'_n ha incluso la verifica $v(\alpha) = 1$.

Che C'_n mantenga misura polinomiale in n al variare di n e come scrivere C'_n come circuito sono punti non banali, che comunque possono essere controllati con successo. Ne omettiamo i dettagli per non appesantire la dimostrazione. $\square$

C'è un particolare non secondario da sottolineare nel confronto tra $SIZE(n^{O(1)})$ e le altre classi di complessità legate ai circuiti, e le classi P, NP (e altre classi che presto introdurremo in relazione al parametro spazio, o altri ancora). Nella definizione di P, NP , e così via, si fa riferimento ad una macchina di Turing che gestisce input di lunghezza arbitraria. Per $SIZE(\dots)$, invece, si chiede, per ogni $n > 0$, un circuito C_n che esamini gli input di lunghezza n o $\leq n$. La complessità di P, NP è, in questo senso, *uniforme*, capace cioè di controllare tutti gli input con un unico modello di computazione (la macchina di Turing) a prescindere dalla loro lunghezza. Invece, come già parzialmente sottolineato, la complessità per circuiti non condivide questa caratteristica di uniformità: input di

lunghezza diversa n sono trattati da circuiti diversi C_n . Resta comunque definita una funzione complessità per ogni famiglia $\{C_n : n > 0\}$ di circuiti, quella che associa ad ogni n la misura di C_n . È ragionevole riconoscerle le proprietà (i), (ii), (iii) a suo tempo stabilite.

9.5 Circuiti e NP-completezza

Terminiamo il capitolo accennando brevemente ad un problema *NP*-completo che nasce tra i circuiti. Anzi, lo si può intendere come un equivalente di *SAT* in questo ambito. Si tratta infatti della soddisfacibilità per circuiti (denotata con la sigla *CIRCUITSAT*).

Definizione. *CIRCUITSAT* è l'insieme dei circuiti C (con n vertici input) per i quali c'è una parola $w \in \{0, 1\}^n$ tale che $f_C(w) = 1$.

Non è difficile convincersi (e neppure provare in dettaglio) che si ha:

Teorema 9.5.1 *CIRCUITSAT* è *NP-completo*.

9.6 Esercizi

1. Si considerino le seguenti formule α della Logica proposizionale

$$\neg(p_0 \wedge (p_1 \vee p_2)), \quad p_0 \wedge \neg(p_1 \vee \neg p_2), \quad \neg(p_0 \wedge p_1 \wedge p_2) \vee p_3.$$

In ogni caso si costruiscano le corrispondenti funzioni f_α^B e i circuiti che le rappresentano.

2. Si considerino le seguenti formule in *QBF*

$$\forall p_0 \exists p_1 ((p_0 \wedge \neg p_1) \vee (p_1 \wedge \neg p_0)), \quad \forall p_0 \forall p_1 (p_0 \wedge \neg p_1), \quad \exists p_0 \forall p_1 (p_0 \wedge \neg p_1), \\ \forall p_0 \exists p_1 (F \wedge p_0 \wedge \neg p_1).$$

Quali di esse appartengono a *QSAT*?

3. È data la porta logica f da $\{0, 1\}^2$ a $\{0, 1\}^3$ tale che, per ogni scelta di $x_0, x_1 \in \{0, 1\}$,

$$f(x_0, x_1) = (NOT(x_0 \oplus x_1), AND(SWAP(x_0, x_1)), x_1).$$

Si rappresentino le tre componenti di f tramite *NOT*, *AND*, *OR*. La porta f è reversibile?

4. Sia *NOR* la porta logica da $\{0, 1\}^2$ a $\{0, 1\}$ che ad ogni coppia $(x_0, x_1) \in \{0, 1\}^2$ associa

$$NOR(x_0, x_1) = NOT(OR(x_0, x_1)).$$

Si provi che *NOR* e *FANOUT* formano un insieme universale di porte logiche.

5. Si dimostri che la porta logica $\oplus$ non costituisce da sola un insieme universale perché per esempio non sa generare AND . Che si può dire di XOR ?
6. Si provi che $P \neq SIZE(n^{O(1)})$.

Riferimenti bibliografici La logica proposizionale è usualmente considerata da tutti i manuali di Logica Matematica (si veda [23], per esempio). I circuiti sono trattati in [21], [76], [107]. Il teorema di Karp-Lipton è provato in [65]. La NP -completezza di $CIRCUITSAT$ è mostrata, per esempio, in [76].

Classi di Complessità Spaziale

10.1 Il parametro spazio

Il tempo non è, ovviamente, l'unico criterio possibile per la misura del costo di una computazione e, dunque, della complessità di una procedura e di un problema. A questo fine, si può infatti fare alternativo riferimento alle risorse di *memoria*, di accumulo di informazioni di cui un algoritmo necessita, e cioè, in definitiva, allo *spazio* impiegato. Possiamo così ritenere un problema *risolubile* quando la sua computazione richiede risorse accessibili di memoria. Naturalmente, per formulare una definizione rigorosa, dobbiamo precisare a quale modello di calcolatore ci riferiamo e che cosa intendiamo per *risorse accessibili di memoria*. Nel caso delle macchine di Turing, pare ragionevole considerare il numero dei quadri richiesti per computare. La complessità spaziale di una MdT M si potrebbe quindi definire come la funzione che associa ad ogni naturale n il numero massimo di quadri necessari per una computazione convergente su un input di lunghezza $\leq n$. D'altra parte, è ovvio constatare che la sola scrittura di un input di lunghezza n richiede, appunto, n quadri. Di conseguenza la funzione appena definita è almeno lineare rispetto alla lunghezza dell'input. Ci sono però casi in cui il numero dei quadri richiesti da una computazione **dopo** la lettura dell'input, cioè durante il successivo lavoro, risulta men che lineare.

Esempio. Supponiamo di dover decidere se 2 numeri naturali, proposti in base 2 come sequenze di 1 e 0, siano o no uguali. Un possibile procedimento ne confronta i bit di ugual posto, a partire da quello più a destra: se i 2 bit, 0 o 1, differiscono, l'algoritmo si arresta e dichiara i 2 input diversi, se invece coincidono, passa a esaminare i bit al posto successivo (a sinistra). In questo secondo caso, può cancellare i calcoli già svolti e utilizzare di nuovo i quadri di lavoro già impiegati. In conclusione, a prescindere dalla lunghezza dei due naturali da confrontare (e della lunghezza temporale della procedura), lo spazio di computazione occupa un numero costante - e ridotto - di quadri: quello necessario al confronto di due bit.

Allora, per apprezzare correttamente le risorse di memoria richieste da un programma, conviene distinguere

- lo spazio necessario per l'input,
- lo spazio usato dallo sviluppo della computazione.

Per questa finalità, è opportuno riferirsi ad un modello lievemente riveduto di macchina di Turing, consentendo la possibilità di più nastri, come già spiegato nel paragrafo 2.10 della prima parte del libro. Più precisamente, nel nostro caso specifico ammettiamo:

- un nastro di sola lettura su cui collocare l'input,
- un nastro ausiliario di lavoro su cui svolgere la computazione (e raccogliere, se necessario, tutte le informazioni di memoria utili)

ed eventualmente

- un terzo nastro di sola scrittura, da usare una sola volta per l'output.

Ad ogni passo della computazione, la MdT esamina

- un quadro sul nastro dell'input,
- la situazione del nastro di lavoro,
- il proprio stato

ed esegue le eventuali istruzioni corrispondenti a questa terna di premesse.

In tutto questo capitolo facciamo riferimento a questo modello di macchina di Turing.

Possiamo allora convenire quanto segue.

Definizione. La *complessità spaziale* di una MdT deterministica M (come appena descritta) è la funzione s_M che associa ad ogni naturale n il numero massimo di quadri impiegato da M sul nastro di lavoro nelle computazione convergenti su input di lunghezza $\leq n$.

La nozione si può estendere anche a MdT non deterministiche M . In questo caso, però, lo stesso input w può originare più di una computazione. Porremo allora, per ogni naturale n ,

$s_M(n)$ = valore massimo al variare degli input w di lunghezza $\leq n$ del minimo numero di quadri occupati da M sul nastro di lavoro in una computazione convergente su w .

La complessità spaziale di una MdT non può essere inferiore a quella temporale. Infatti una computazione che richiede m passi non può coinvolgere più di m quadri, uno per passo. Possiamo poi ragionevolmente convenire che, per ogni MdT M di un qualche interesse, s_M soddisfi la condizione (i), (ii), (iii) che definiscono le funzioni di complessità (si veda il paragrafo 7.2 di questa seconda parte).

Sia ora M una MdT (deterministica o anche non deterministica) di complessità spaziale s_M . Quando M opera su un input di lunghezza $\leq n$, possiamo assumere che la sua computazione coinvolga

- al più n quadri sul nastro di lettura,
- al più $s_M(n)$ quadri sul nastro di lavoro,

(quelli su cui M converge, se converge). Ogni configurazione di M in questo contesto è data dalla terna che esprime

- il quadro in esame sul nastro di scrittura,
- la sequenza che compare sugli $s_M(n)$ quadri di lavoro,
- lo stato di M .

Per esempio la configurazione di partenza è quella in cui

- il quadro in esame sul nastro di scrittura è quello più a sinistra,
- la sequenza che compare sul nastro di lavoro è bianca,
- lo stato di M è q_0 .

Contiamo allora il numero delle possibili configurazioni di una macchina di Turing M su input di lunghezza n :

- vi sono n possibili scelte per a);
- le possibilità relative a c) corrispondono al numero degli stati di M , che è prefissato, dipende da M , ma non dall'input w e da n ;
- il numero delle possibili scelte per b) è, invece, funzione di n e anche della cardinalità dell'alfabeto A di M ; infatti ognuno degli $s_M(n)$ quadri coinvolti può ospitare un elemento di A , o il simbolo bianco $\star$; complessivamente, risultano allora al più

$$(|A| + 1)^{s_M(n)}$$

possibili scelte.

In conclusione, il numero di configurazioni che M può assumere nelle sue computazioni su input di lunghezza n è

$$k \cdot n \cdot (|A| + 1)^{s_M(n)}$$

dove k è una costante che rivela il numero degli stati di M . D'altra parte

$$k \cdot n \cdot (|A| + 1)^{s_M(n)} = n \cdot 2^{\log_2 k + s_M(n) \cdot \log_2 (|A| + 1)} = n \cdot 2^{O(s_M(n))}.$$

Così il numero delle configurazioni è

$$n \cdot 2^{O(s_M(n))}.$$

Per esempio, se $s_M(n) = O(n)$, allora il numero delle configurazioni è

$$n \cdot 2^{O(n)} = 2^{n+O(n)} = 2^{O(n)}$$

(eventualmente inferiore a 2^n); se invece $s_M(n) = O(\log_2 n)$, il numero diviene

$$n \cdot 2^{O(\log_2 n)} = 2^{\log_2 n + O(\log_2 n)} = 2^{O(\log_2 n)} \leq 2^{c \cdot (\log_2 n)}$$

per qualche costante c , e quindi $O(n^c)$.

Di più, ogni singola configurazione può essere descritta con $O(s_M(n))$ informazioni (quella relativa al quadro i in esame, quella sullo stato e , finalmente, la descrizione del nastro di lavoro), dunque in spazio $O(s_M(n))$.

La computazione di M su un particolare input w può essere descritta da un grafo diretto $G(M, w) = (V(M, w), E(M, w))$ costruito come segue. Sia $n = l(w)$.

- I vertici di $V(M, w)$ sono le configurazioni di M nella computazione su w (codificate nel modo sopra descritto) per una porzione di $s_M(n)$ quadri del nastro di lavoro.
- Se c, c' sono due di queste configurazioni, assumiamo che ci sia un lato in $E(M, w)$ da c a c' se e solo se c' è un'istruzione di M che permette il passaggio da c a c' .

È utile supporre che $V(M, w)$ abbia un ulteriore vertice $t(M, w)$, cui ogni configurazione di arresto di M su w confluisce in $E(M, w)$. Così, per $n = l(w)$, $G(M, w)$ ha $n \cdot 2^{O(s_M(n))}$ vertici (e, ovviamente, un numero di lati al più uguale al quadrato del numero dei vertici, cioè al numero delle coppie ordinate dei vertici). Di più, ogni vertice di $G(M, w)$ è descritto da $O(s_M(n))$ bit di informazione, e ogni lato dalle $O((s_M(n))^2)$ informazioni derivanti dai suoi vertici di partenza e di arrivo.

Il riferimento a questi grafi sarà più volte utile nel seguito. Le considerazioni svolte ci servono anche a provare:

Proposizione 10.1.1 *Sia M una MdT (deterministica o anche non deterministica) con complessità spaziale $s_M(n)$. Allora M ha complessità temporale $n \cdot 2^{O(s_M(n))}$.*

Dimostrazione. Sia w un input di lunghezza $\leq n$. Seguiamo una computazione convergente di n su w che occupa al più $s_M(n)$ quadri del nastro di lavoro. Se c'è una ripetizione nella sequenza delle relative configurazioni (stesso quadro in esame per l'input, stessa situazione sul nastro di lavoro, stesso stato), questo significa che la MdT entra in un ciclo e dunque diverge su w . Ma allora, se M converge su w , la sua computazione non ha ripetizioni nelle configurazioni, e quindi si deve svolgere entro $n \cdot 2^{O(s_M(n))}$ passi. $\square$

10.2 PSPACE e L

In analogia al caso temporale, possiamo dedicarci ai problemi che sono risolti da macchine di Turing che hanno complessità spaziale al più polinomiale.

Definizione. *PSPACE* è la classe dei problemi di decisione S che sono accettati da macchine di Turing (deterministiche) M tali che $s_M(n) = O(n^k)$ per qualche intero positivo k .

Dunque, c'è un polinomio p di grado k a coefficienti interi e valori positivi tale che, per w parola sull'alfabeto A di S e $n = l(w)$,

- se $w \in S$, M converge su w occupando un numero di quadri limitato da $p(n)$ sul suo nastro di lavoro,
- se $w \notin S$, M diverge su w .

Dalle considerazioni del paragrafo precedente segue facilmente:

Osservazione. $P \subseteq PSPACE \subseteq PEXP$.

Più precisamente la prima inclusione segue da quanto notato subito dopo la definizione di complessità spaziale s_M , la seconda dalla proposizione finale 10.1.1.

Nel caso temporale, la tesi di Edmonds-Cook-Karp sostiene “*rapido = polinomiale*” e quindi identifica in P la classe dei problemi che hanno algoritmo “veloce” di soluzione. Ma adesso, se privilegiamo il parametro spazio, può capitare di incontrare problemi non banali che possono essere risolti in spazio men che polinomiale. Ci sono esempi opportuni che testimoniano questa evenienza. Visto il carattere tecnico, non ci attardiamo a discuterli: del resto essi possono trovarsi nei riferimenti elencati a fine capitolo.

Ha dunque senso pensare a restrizioni significative di $PSPACE$, atte a selezionare problemi la cui soluzione richiede risorse di spazio e memoria men che polinomiali. Introduciamo allora la classe L (L abbrevia qui la sigla $LOGSPACE$ e significa, ovviamente, spazio logaritmico; L non va dunque confuso in questo ambito con un arbitrario linguaggio, come avveniva nella prima parte del libro).

Definizione. L è la classe dei problemi di decisione S che sono accettati da macchine di Turing (deterministiche) M tali che $s_M(n) = O(\log_2 n)$.

In realtà la base (2, oppure 10, oppure $e \dots$) rispetto a cui il logaritmo è calcolato è del tutto irrilevante, purché > 1 . Infatti, si ha, per esempio, per ogni n , $\log_2 n = \log_2 10 \cdot \log_{10} n$, così $\log_2 n = O(\log_{10} n)$ e viceversa.

Dalla proposizione 10.1.1, segue direttamente

Proposizione 10.2.1 $L \subseteq P$.

Dimostrazione. Basta ricordare che, per $s_M(n) = O(\log_2 n)$, $n \cdot 2^{O(s_M(n))} = O(n^k)$ per qualche costante positiva k . $\square$

Così abbiamo

$$L \subseteq P \subseteq PSPACE.$$

Ancora, si hanno appropriati argomenti tecnici che provano l'esistenza di problemi in $PSPACE - L$ e dunque mostrano $L \neq PSPACE$. Di conseguenza,

$$P \neq PSPACE \text{ o } P \neq L.$$

Può anche succedere $P \neq PSPACE, L$. Un'investigazione degli esempi conosciuti di problemi in P e le difficoltà che talora si riscontrano per dotarli di procedure che li decidono in spazio al più logaritmico avvalorano la

Congettura. $P \neq L$.

10.3 NPSPACE e NL

Come P è accompagnato da NP tra le classi di complessità temporale, così anche $PSPACE$ e L ammettono le loro varianti non deterministiche, introdotte come segue.

Definizione. $NPSPACE$ è la classe dei problemi di decisione che sono accettati da macchine di Turing non deterministiche M in spazio $O(n^k)$ per qualche intero positivo k .

Definizione. NL (abbreviazione di $NLOGSPACE$) è la classe dei problemi di decisione che sono accettati da macchine di Turing non deterministiche M in spazio $O(\log_2 n)$.

Così $s_M(n) = O(n^k)$ nel caso di $NPSPACE$, $s_M(n) = O(\log_2 n)$ per NL . In altre parole, un problema S sull'alfabeto A è in $NPSPACE$ se e solo se ci sono una MdT non deterministica M su A e un polinomio p a coefficienti interi e valori positivi tali che, per ogni $w \in A^*$, con $n = l(w)$,

- se $w \in S$, allora almeno una computazione di M su w converge occupando al più $p(n)$ quadri del nastro di lavoro,
- se $w \notin S$, nessuna computazione di M su w converge.

Lo stesso vale per L , salvo sostituire il polinomio p con $k \cdot \log_2 n$, dove k è un qualche intero positivo.

È ovvio constatare

- $PSPACE \subseteq NPSPACE$,
- $L \subseteq NL$,

basta ricordare che le MdT deterministiche sono particolari MdT non deterministiche. Si ha poi

- $NP \subseteq NPSPACE$.

perché una computazione non può occupare più quadri dei passi svolti.

Più sottile e intrigante è la proprietà seguente.

Teorema 10.3.1 $NL \subseteq P$.

(In altre parole, spazio non deterministico logaritmico implica tempo deterministico polinomiale).

Dimostrazione. Sia $S \in NL$, così esiste una MdT non deterministica M che accetta S in spazio al più logaritmico. Per ogni parola w nell'alfabeto A di S , consideriamo il grafo diretto $G(M, w)$ associato alla computazione di M su w . Un algoritmo in P per S si ottiene come segue: per ogni input w di lunghezza n , si costruisce il grafo $G(M, w)$ della computazione di M su w e poi si controlla se in $G(M, w)$ il vertice terminale $t(M, w)$ è estremo di qualche lato. Ovviamente $w \in S$ se e solo se un tale lato esiste. D'altra parte, per $s_M(n) = O(\log_2 n)$, $G(M, w)$

ha un numero di vertici $O(n^k)$ per qualche k (e un numero di lati $O(n^{2k})$) e ogni vertice è descritto in spazio $O(\log_2 n)$. Dunque la costruzione di $G(M, w)$ richiede tempo al più polinomiale in n , così come il controllo dei lati di $G(w)$ e dell'eventualità che $t(M, w)$ sia loro estremo. $\square$

$NPSPACE$ e NL sono stati introdotti in riferimento alla seconda caratterizzazione di NP , quella che fa uso di MdT non deterministiche, e non sulla base della definizione originaria di NP . Ci si può domandare se l'equivalenza dei due approcci a NP si mantiene per $NPSPACE$ e NL , in particolare se NL coincide anche con la classe di problemi S (definiti su alfabeti finiti A) per i quali esistono $V_S \in L$ e un polinomio a coefficienti interi e valori positivi tali che, per ogni $w \in A^*$,

$$w \in S$$

se e solo se

$$\exists y \in A^* \text{ che ha lunghezza } \leq p_S(l(w)) \text{ e soddisfa } (w, y) \in V_S.$$

Ci sono però fondati dubbi a questo proposito. Per esempio, si vede che SAT corrisponde a quest'ultima caratterizzazione, vi sono infatti algoritmi che, dato un insieme K di clausole, sono capaci di controllare in spazio logaritmico rispetto a K che una certa valutazione v soddisfa o no K (si noti che v ha comunque lunghezza polinomialmente limitata da quella di K). Dunque, se la nuova possibile caratterizzazione equivale alla precedente definizione di NL , si deduce $SAT \in NL$ e, dal teorema precedente, che $SAT \in P$, risolvendo così in modo positivo la congettura $P = NP$.

Nell'ambito dello spazio logaritmico, la questione che corrisponde a $P = NP$ è

Problema. $L = NL$?

Per apprezzarne la difficoltà, può essere utile individuare problemi che giocano lo stesso ruolo chiave svolto dalle questioni NP -complete nel contesto $P = NP$. Naturalmente la nozione di NP -completezza va adeguata a NL , dunque alla nuova prospettiva del parametro spazio e delle misure logaritmiche. Poniamo allora:

Definizione. Siano S, S' due problemi su alfabeti finiti A, A' rispettivamente. Scriviamo

$$S \leq_{\log} S'$$

se e solo se c'è una funzione f da A^* a A'^* che è computabile deterministicamente in spazio logaritmico e soddisfa, per ogni $w \in A^*$, $w \in S$ se e solo se $f(w) \in S'$.

Ci si aspetta ragionevolmente che $\leq_{\log}$ rispetti le due proprietà seguenti:

- L è chiuso per $\leq_{\log}$, e cioè, se $S' \in L$ e $S \leq_{\log} S'$, allora anche $S \in L$ (e lo stesso vale per NL);
- $\leq_{\log}$ è riflessiva: $S \leq_{\log} S$ per ogni S ;
- $\leq_{\log}$ è transitiva e cioè, se $S \leq_{\log} S'$ e $S' \leq_{\log} S''$, allora $S \leq_{\log} S''$;

in particolare che $\leq_{\log}$ sia una relazione di riducibilità. Ricordiamo che le corrispondenti affermazioni per $\leq_p$ e P risultano banalmente soddisfatte. In effetti, anche per $\leq_{\log}$ e L le cose funzionano e in particolare la riflessività è evidente, ma la dimostrazione delle altre due proprietà è tutto men che semplice, come adesso vediamo: l'ostacolo maggiore è nuovamente costituito dallo spazio più che logaritmico richiesto per scrivere un input.

Proposizione 10.3.2 *Siano S, S', S'' problemi definiti su alfabeti finiti A, A', A'' rispettivamente.*

1. *Se $S' \in L$ e $S \leq_{\log} S'$, allora $S \in L$. Lo stesso vale per NL al posto di L .*
2. *Se $S \leq_{\log} S'$ e $S' \leq_{\log} S''$, allora $S \leq_{\log} S''$.*

Dimostrazione.

1. Consideriamo il caso di L , NL si tratta analogamente. Siano

- f la funzione che riduce S a S' (ai sensi della definizione di $\leq_{\log}$),
- M_f una MdT che computa f in spazio logaritmico,
- M la MdT su A che accetta S ancora in spazio logaritmico.

La prima strategia che viene in mente per costruire M è quella di comporre M_f e M' : data una parola w su A ,

- prima si calcola $f(w)$ tramite M_f ,
- poi si lavora su $f(w)$ con M' .

Ovviamente $w \in S$ se e solo se $f(w) \in S'$, dunque se e solo se il precedente procedimento converge.

C'è però un ostacolo da considerare: la MdT M appena descritta può impiegare spazio più che logaritmico rispetto alla lunghezza n di w . Infatti l'input di M , quello che viene posto all'inizio sul nastro di lettura della computazione, è, appunto, w . Ma poi, al passo intermedio della computazione, quando M_f ha terminato il calcolo di $f(w)$ e M' le subentra per svolgere la sua parte, c'è una nuova parola $f(w)$ da considerare. Per M' , $f(w)$ è l'input e dunque viene scritto sul nastro di lettura di M' ; una sua lunghezza eventualmente eccessiva non va a sconvolgere lo spazio logaritmico a disposizione sul nastro di lavoro di M' . Ma per M le cose cambiano, $f(w)$ non è l'input di M , dunque non va scritto sul nastro di lettura di M , ma su quello di lavoro, dove può intaccare i limiti logaritmici previsti e occupare una porzione eccessiva.

Il rimedio per ovviare a questa difficoltà consiste nella strategia seguente: si alterna in M il lavoro di M_f e di M' ,

- M_f fornisce $f(w)$ non in un'unica soluzione, ma progressivamente, simbolo per simbolo, per la porzione che via via interessa M' .
- M' , a sua volta, opera non più su tutto $f(w)$, ma solo su sue parti di lunghezza appropriata.

Si riesce così a preservare lo spazio logaritmico a disposizione.

2. Si presentano analoghe difficoltà e analoghi rimedi. Il lettore interessato può curare i dettagli per **esercizio**. $\square$

Possiamo adesso introdurre la nozione di NL -completezza.

Definizione. Un problema $\bar{S}$ si dice

- NL -arduo se ogni $S \in NL$ soddisfa $S \leq_{log} \bar{S}$,
- NL -completo se sta in NL ed è NL -arduo.

Così, per $\bar{S}$ NL -completo, se $\bar{S} \notin L$, si deduce ovviamente $L \neq NL$ ma, se $\bar{S} \in L$, si può anche concludere $L = NL$ (**Esercizio:** perché?).

Tra i problemi NL -completi, SAT e la sua variante $3SAT$ sono quelli principali: i primi storicamente, e anche quelli a cui la maggioranza degli altri usualmente si riduce. Vogliamo cercare esempi altrettanto significativi per la NL -completezza. Il collegamento tra computazioni e grafi diretti osservato nel paragrafo 10.1 può fornire suggerimenti al riguardo. Consideriamo infatti il seguente

Problema dell'accessibilità nei grafi (GAP , *graph accessibility problem*). Sono dati

- un grafo diretto finito $G = (V, E)$,
- due vertici $t_0, t_1 \in V$.

Si chiede di determinare se t_1 è o no *accessibile* da t_0 in V , nel senso che c'è un cammino di E che conduce da t_0 a t_1 in G .

Sulla base delle considerazioni del paragrafo 10.1, è quasi immediato dedurre:

Teorema 10.3.3 GAP è NL -completo.

Dimostrazione. Dobbiamo provare:

- $GAP \in NL$,
- GAP è NL -arduo.

a) Ecco una procedura non deterministica che accetta GAP in spazio al più logaritmico. Dati $G = (V, E)$, t_0, t_1 , esploriamo ogni possibile vertice v per cui c'è una serie di lati di E da t_0 a v (questo è il passo non deterministico dell'algoritmo). Se $v = t_1$, concludiamo che t_1 è accessibile da t_0 , se $v \neq t_1$, passiamo a considerare un altro vertice v' . Il controllo di uguaglianza si può svolgere in spazio al più logaritmico nella lunghezza dell'input, in particolare nella cardinalità di V , in maniera analoga all'esempio del paragrafo 10.1.

b) Sia $S \in NL$, c'è dunque una MdT non deterministica M che accetta S in spazio al più logaritmico $s_M(n) = O(\log_2 n)$. Dobbiamo provare $S \leq_{log} GAP$. Consideriamo allora la funzione che ad ogni parola w nell'alfabeto A di S associa anzitutto il grafo diretto $G(M, w)$ che descrive la computazione di M su w , e poi due vertici t_0, t_1 di seguito descritti:

- $t_0 =$ vertice della configurazione iniziale di M su w ,
- $t_1 = t(M, w)$.

Ovviamente si ha che $w \in S$ se e solo se M accetta w e dunque se e solo se t_1 è accessibile da t_0 in $G(M, w)$. Inoltre la costruzione di $G(M, w)$, t_0, t_1 a partire da w si può svolgere in spazio al più logaritmico rispetto alla lunghezza n di w . Infatti, come sappiamo, per $s_M(n) = O(\log_2 n)$, $G(M, w)$ ha $O(n^k)$ vertici e $O(n^{2k})$ lati (per k opportuna costante), ma ogni vertice - e ogni lato, inteso come coppia ordinata di vertici - si descrive in spazio $O(\log n)$. Così, costruito un vertice, o un lato, di $G(M, w)$ sul nastro di lavoro in spazio $O(\log n)$, lo si ricopia sul nastro output e si può liberare il nastro di lavoro per formarvi i vertici e i lati successivi. $\square$

10.4 Il teorema di Savitch

Il teorema di Savitch del 1970 collega le minime risorse di memoria necessarie per risolvere deterministicamente o non deterministicamente un problema S ; mostra infatti che, se S può essere accettato non deterministicamente in spazio $s(n)$ per opportune funzioni $s(n) \geq \log_2 n$, allora S è accettato deterministicamente in spazio $O(s^2(n))$; implica in particolare che

- ogni $S \in NL$ si accetta deterministicamente in spazio $O(\log^2 n)$

e anche

- $PSPACE = NPSPACE$.

Si tratta dunque di risultato di grande interesse. Il presente paragrafo è dedicato alla sua dimostrazione. Cominciamo col caso particolare di NL .

Teorema 10.4.1 *Sia $S \in NL$. Allora S può essere accettato deterministicamente in spazio $O(\log^2 n)$.*

Dimostrazione. Ci basta provare la tesi quando $S = GAP$. Infatti ogni problema S di NL si riduce a GAP tramite $\leq_{\log}$ ed è facile arrangiare la dimostrazione della proposizione 10.3.2 in modo da dedurre che, se c'è un procedimento deterministico per GAP che impiega spazio $O(\log^2 n)$, lo stesso vale per $S \in NL$ (basta combinare in modo appropriato, come suggerito dalla proposizione 10.3.2, l'algoritmo deterministico per GAP e la riduzione deterministica di S a GAP assicurata da $\leq_{\log}$).

Dunque cerchiamo una procedura deterministica per svolgere in spazio $O(\log^2 n)$ quanto segue: dati un grafo diretto finito $G = (V, E)$ e due vertici $t_0, t_1 \in V$,

- la procedura converge se c'è un cammino di G che conduce da t_0 a t_1 ,
- la procedura diverge altrimenti.

Possiamo riferirci alla funzione Acc (per *Accessibile*) definita come segue: per $v, v' \in V$ e k intero positivo

$$Acc(v, v', k) = \begin{cases} 1 & \text{se } v' \text{ è accessibile da } v \text{ tramite un cammino} \\ & \text{di } \leq k \text{ lati consecutivi di } E, \\ \text{non definito} & \text{altrimenti.} \end{cases}$$

Così t_1 è accessibile da t_0 se e solo se vale $Acc(t_0, t_1, k) = 1$ per qualche intero positivo k . Ci basta allora un programma che accetta quest'ultima condizione in spazio al più logaritmico rispetto alla lunghezza dell'input.

Ovviamente, se $|V| = n$, basta considerare $k \leq n$, o anche k minore o uguale della minima potenza di 2 che è $\geq n$ (dunque $2 = 2^1$ se V ha ≤ 2 vertici, $4 = 2^2$ se V ha ≤ 4 vertici, e così via). Vediamo come si computa Acc . Per $k = 1 = 2^0$, si ha

$$Acc(v, v', 1) = \begin{cases} 1 & \text{se } (v, v') \in E, \\ \text{non definito} & \text{se } (v, v') \notin E. \end{cases}$$

Sia ora $k \geq 2$. Supponiamo che k sia una potenza di 2, $k = 2^{h+1}$ per qualche intero $h \geq 0$. Per il calcolo di $Acc(v, v', 2^{h+1})$ possiamo far riferimento a possibili tappe intermedie $u \in V - \{v, v'\}$ di una sequenza di k lati da v a v' . Ricordiamo che la metà di $k = 2^{h+1}$ è 2^h , ancora una potenza di 2. Allora, al variare di $u \in V - \{v, v'\}$, consideriamo

- $Acc(v, u, 2^h)$,
- $Acc(u, v', 2^h)$.

Se ambedue sono definite per qualche u , $Acc(v, v', 2^{h+1})$ lo è (e vale 1). Altrimenti $Acc(v, v', 2^{h+1})$ resta indefinita. Lo spazio richiesto per questo calcolo necessita di:

- $O(\log_2 n)$ per ricorrere a u ,
- lo spazio per $Acc(v, u, 2^h)$, $Acc(u, v', 2^h)$.

Poniamo allora, per ogni k potenza di 2,

$$\sigma(k) = \text{massimo spazio richiesto dalla computazione di } Acc(\cdot, \cdot, k).$$

Dalla definizione di $Acc(\cdot, \cdot, 1)$ deriva facilmente

$$\sigma(1) = O(\log_2 n).$$

Per quanto riguarda invece $k \geq 2$, e dunque $k = 2^{h+1}$ con $h \geq 0$, si è visto

$$\sigma(2^{h+1}) = O(\log_2 n) + \sigma(2^h).$$

Otteniamo complessivamente

$$\sigma(2^{h+1}) = O(\log_2 n) \cdot (h + 1)$$

dove $k = 2^{h+1}$ implica $h = O(\log_2 k)$. Quando $k \leq n$, o anche quando k è la minima potenza di 2 che è $\geq n$,

$$O(\log_2 k) = O(\log_2 n),$$

dunque

$$\sigma(k) = O(\log_2^2 n).$$

In conclusione, l'accessibilità di t_1 da t_0 si decide in spazio $O(\log_2^2 n)$. $\square$

La dimostrazione per NL si può adattare al caso generale nel modo che segue.

Teorema 10.4.2 *Sia S un problema che si accetta non deterministicamente in spazio s dove $s(n) \geq \log_2 n$ è una funzione a sua volta computabile in spazio $O(s(n))$. Allora S è accettato deterministicamente in spazio $O(s^2(n))$.*

Dimostrazione (Cenni). Sappiamo che c'è una MdT non deterministica M che accetta S in spazio $s_M(n) = s(n)$ e cerchiamo un procedimento deterministico che ottiene lo stesso obiettivo, eventualmente impiegando maggior spazio $O(s^2(n))$. Non possiamo più riferirci a GAP , ma, per ogni parola w nell'alfabeto A di S , possiamo intanto costruire il grafo diretto $G(M, w)$ della computazione di M su w , impiegando sul nastro di lavoro spazio $O(s(n))$ - quello necessario a identificare ogni vertice e ogni lato di $G(M, w)$ - come già visto nella dimostrazione del teorema 10.3.3.

Si ha che $w \in S$ se e solo se c'è una computazione di M su w che converge in spazio $s(n)$, dunque se e solo se in $G(M, w)$ il vertice terminale $t_1 = t(M, w)$ è accessibile dalla configurazione iniziale di M su w . Ci basta allora un procedimento deterministico che ottiene questo risultato in spazio $O(s^2(n))$. Ma l'algoritmo proposto nel caso di NL funziona ancora egregiamente: all'ipotesi che M lavora in spazio $O(\log_2 n)$ - valida per NL - abbiamo sostituito quella su $s(n) \geq \log_2 n$, che ancora basta a far sì che lo spazio complessivamente richiesto - incluso quello per costruire $G(M, w)$ - sia $O(s^2(n))$. Tuttavia, per poter ripetere i ragionamenti su Acc , ci serve anche conoscere - per limitare k - il numero complessivo dei vertici di $G(M, w)$, e dunque $s(n)$. L'ipotesi che $s(n)$ sia computabile in spazio $O(s(n))$ vale a garantire che anche questa parte del procedimento soddisfa i limiti di spazio previsti. Il lettore può provare a controllare per esercizio i dettagli, se vuole. $\square$

Il precedente teorema su NL si può dedurre da questo, più generale: infatti, per $S \in NL$, $s(n) = \log_2 n$ e si può vedere che $\log_2 n$ è computabile in spazio $O(\log_2 n)$. Lo stesso vale per ogni polinomio e quindi un'altra rilevante conseguenza riguarda $NPSPACE$ e afferma:

Teorema 10.4.3 $NPSPACE = PSPACE$.

Dimostrazione. Per $S \in NPSPACE$, s è un polinomio; ma allora anche s^2 è un polinomio. $\square$

Così l'equivalenza tra $PSPACE$ e $NPSPACE$, corrispondente ai problemi aperti $P = NP$ e $L = NL$, ha soluzione positiva relativamente semplice.

10.5 $NL = coNL$

Nel caso di P e NP , l'uguaglianza tra NP e $coNP$ resta misteriosa, e anzi il tentativo di chiarirla conduce alla costruzione della complicata gerarchia polinomiale e di PH . Per L e NL , invece, non ci sono dubbi. Poniamo infatti

Definizione. $coNL$ è la classe dei problemi S tali che $S^c \in NL$.

(Ricordiamo che, per S problema sull'alfabeto A , S^c denota il problema complementare $A^* - S$).

Così $S \in coNL$ significa che c'è una MdT non deterministica M su A che lavora in spazio al più logaritmico $s_M(n) = O(\log_2 n)$ e accetta S^c , dunque, per $w \in A^*$ di lunghezza n ,

- se $w \notin S$, c'è almeno una computazione convergente di M su w (che occupa $O(\log_2 n)$ quadri);
- se $w \in S$, nessuna computazione di M su w converge.

Un risultato di Immerman e Szelepcsényi del 1988 prova comunque:

Teorema 10.5.1 $NL = coNL$.

Dimostrazione. Anzitutto, ci basta provare $NL \subseteq coNL$, infatti l'inclusione inversa $coNL \subseteq NL$ è ovvia conseguenza: per $S \in coNL$, $S^c \in NL \subseteq coNL$, dunque $S \in NL$.

Possiamo poi fare riferimento al problema NL -completo GAP e limitarci a mostrare che $GAP \in coNL$. Infatti ogni $S \in NL$ è riducibile in $\leq_{log}$ a GAP , c'è dunque una funzione f che lavora in spazio logaritmico e traduce parole w dell'alfabeto di S in istanze di GAP in modo tale che

$$w \in S \text{ se e solo se } f(w) \in GAP$$

e dunque

$$w \in S^c \text{ se e solo se } f(w) \in GAP^c.$$

Così si ha anche $S^c \leq_{log} GAP^c$ ancora grazie a S . Se sappiamo $GAP \in coNL$, deduciamo $GAP^c \in NL$, da cui $S^c \in NL$ e $S \in coNL$.

Abbiamo allora da provare $GAP \in coNL$, cioè $GAP^c \in NL$. Cerchiamo quindi una procedura non deterministica che lavora in spazio logaritmico per determinare, dati un grafo diretto finito $G = (V, E)$ e due vertici $t_0, t_1 \in V$, se

$$t_1 \text{ non è accessibile da } t_0 \text{ in } G.$$

Cerchiamo anzitutto di calcolare

$$r = \text{numero di vertici di } V \text{ accessibili da } t_0 \text{ in } G.$$

La strategia che seguiremo consiste infatti nell'individuare quei vertici di G che sono accessibili da t_0 , o almeno il loro numero r , e poi controllare che t_1 non sia uno di loro. La difficoltà è quella di sviluppare questa procedura in spazio logaritmico rispetto a $n = |V|$: questa restrizione, infatti, ci permette di scrivere

in memoria, sul nastro di lavoro, il numero r , ma può impedirci di elencarvi tutti gli r vertici che si raggiungono da t_0 lungo i lati di G .

Comunque, iniziamo col calcolare r . Ci arriviamo fissando anzitutto, per ogni naturale $i < n$,

r_i = numero dei vertici di V accessibili da t_0 tramite una serie di al più i lati.

Notiamo che $r = r_{n-1}$ perché i vertici di V diversi da t_0 sono $n - 1$. Possiamo quindi raggiungere r computando prima r_0 e poi, per ogni $i < n - 1$, r_{i+1} da r_i . Ovviamente $r_0 = 1$, infatti solo t_0 è accessibile da t_0 senza muoversi, quando $i = 0$. Notiamo anche che

r_1 = numero dei vertici $v \in V$ per cui $(t_0, v) \in E$.

Supponiamo ora di conoscere r_i e cerchiamo di ottenere r_{i+1} . Come già sottolineato, lo spazio logaritmico che abbiamo a disposizione ci consente di scrivere in memoria il valore r_i , ma non di elencare gli r_i vertici che da t_0 raggiungono in al più i lati (questi vertici potrebbero essere occupare troppo spazio).

Possiamo ovviare a questo ostacolo con un procedimento non deterministico che può risultare enormemente lungo quanto a tempo, ma rispetta il “poco” spazio a disposizione. Sia allora $v \in V$. Durante la computazione su v cerchiamo anche una nuova stima r'_i di r_i , come vedremo.

Fissiamo non deterministicamente $u \in V$ e una serie di $\leq i$ lati da t_0 . Può darsi che questi lati non conducano ad u ; in questo caso interpelliamo una nuova serie di lati. Può darsi che, viceversa, i lati raggiungano u . In questo caso, incrementiamo anzitutto di 1 il valore presumibile r'_i di r_i dedotto dalla computazione su v . Poi controlliamo se $u = v$ oppure se $(u, v) \in E$. Se sì, incrementiamo di 1 il valore presumibile di r_{i+1} e passiamo ad un altro vertice v' . Se no, consideriamo la stima r'_i fin qui ottenuta di r_i . Ricordiamo che disponiamo di r_i in memoria. Possiamo allora confrontare r'_i e r_i . Se $r'_i = r_i$, abbiamo la certezza che tutti gli u accessibili da t_0 in al più i lati sono stati coinvolti per v , e possiamo escludere v dal computo di r_{i+1} e interpellare un nuovo v' . Se $r'_i \neq r_i$, continuiamo ad esaminare v , ma interpelliamo un nuovo vertice u' invece di u .

Alla fine, r_{i+1} è ottenuto. Come detto, è da prevedere che il procedimento comporti un gran numero di fallimenti su u e sulla relativa serie di lati. Ma il lettore può verificare che lo spazio logaritmico non viene violato, il numero dei vertici coinvolti ad ogni passo è opportunamente limitato.

Arriviamo così a $r_{n-1} = r$, che collochiamo in memoria. Di nuovo, non possiamo accompagnarlo con l'elenco dei vertici accessibili da t_0 . Possiamo comunque controllare che t_1 non fa parte di questi r elementi ripetendo l'argomento precedente, a partire da t_1 e da r . Durante la verifica su t_1 , cerchiamo anche una nuova stima r' di r . Procediamo come segue.

Scegliamo non deterministicamente $u \in V$. Sempre in modo non deterministico fissiamo una sequenza di $< n$ lati da t_0 . Se la sequenza non raggiunge u in nessuna tappa, passiamo ad un lato successivo. Se invece la sequenza conduce ad u , aumentiamo intanto di 1 la stima r' di r . Vediamo poi se $u = t_1$. Se sì, escludiamo che t_1 non sia accessibile da t_0 . Se no, controlliamo se $r' = r$.

In caso negativo, esploriamo un nuovo u (e un nuovo itinerario di lati). In caso positivo, se t_1 non è stato ancora raggiunto, possiamo concludere che t_1 **non** è accessibile da t_0 .

Così, l'ultima fase del procedimento ripete le precedenti, con gli opportuni adattamenti: ne mantiene i difetti (una prevedibile abbondanza di fallimenti, e dunque lunghi tempi di lavoro), ma anche, e soprattutto, i pregi: il rispetto dello spazio logaritmico. $\square$

10.6 PSPACE-completezza

Il Teorema di Savitch ci dice $PSPACE = NPSPACE$, e in questo modo diminuisce l'interesse per la classe $NPSPACE$: non ha senso, in particolare, cercare problemi $NPSPACE$ -completi. Possiamo comunque introdurre il concetto di $PSPACE$ -completezza riferito a $\leq_p$. Ricordiamo che, per $S, \bar{S}$ problemi su alfabeti $A, \bar{A}$ rispettivamente,

$$S \leq_p \bar{S}$$

significa che c'è un procedimento deterministico che lavora in tempo al più polinomiale sulla lunghezza dell'input per tradurre parole w su A in parole $\bar{w}$ su $\bar{A}$ in modo tale che

$$w \in S \text{ se e solo se } \bar{w} \in \bar{S}.$$

Ricordiamo poi che un algoritmo che impiega tempo al più polinomiale occupa spazio al più polinomiale, ma non viceversa. Poniamo allora:

Definizione. $\bar{S}$ è

- $PSPACE$ -arduo se, per ogni $S \in PSPACE$, $S \leq_p \bar{S}$;
- $PSPACE$ -completo se $PSPACE$ -arduo e in $PSPACE$.

Un esempio di problema $PSPACE$ -completo è $QSAT$, l'insieme costituito dalle formule Booleane quantificate che sono vere.

Teorema (Stockmeyer-Meyer) 10.6.1 $QSAT$ è $PSPACE$ -completo.

Dimostrazione. Anzitutto mostriamo che $QSAT$ è in $PSPACE$. Un algoritmo deterministico che decide $QSAT$ in spazio al più polinomiale è il seguente. Sia data una formula Booleana quantificata α . Lavoriamo per induzione sul numero n di variabili proposizionali in α (cioè sul numero n di quantificatori all'inizio di α). Se $n = 0$, cioè α non ha variabili proposizionali (e quindi neppure quantificatori), α è T , e quindi è vera, oppure F , e allora non è vera. Supponiamo adesso che l'algoritmo sia capace di decidere nel modo stabilito formule con n variabili e consideriamo formule α con $n+1$ variabili; possiamo supporre che α inizi con $\exists p_1$ o con $\forall p_1$. In ogni caso possiamo costruire due nuove formule da α cancellando

$\exists p_1$ o $\forall p_1$ e sostituendo ogni successivo p_1 uniformemente con T oppure con F . Siano α_T o α_F le formule così determinate. Per esempio, se α è

$$\exists p_1 \forall p_2 \exists p_3 (p_1 \wedge (p_2 \vee p_3)),$$

α_T è

$$\forall p_2 \exists p_3 (T \wedge (p_2 \vee p_3)),$$

e α_F è

$$\forall p_2 \exists p_3 (F \wedge (p_2 \vee p_3)).$$

Ovviamente α_T e α_F sono formule Booleane quantificate, ma hanno una variabile proposizionale in meno rispetto a α e dunque n variabili se α ne ha $n + 1$. α_T corrisponde ad assumere p_1 vera, α_F a supporla falsa. Così, se α inizia con $\exists p_1$, α è vera se e solo se almeno una tra α_T e α_F è vera, mentre nel caso in cui α cominci con $\forall p_1$, α è vera se e solo se α_T e α_F lo sono entrambe.

L'algoritmo procede allora così. Prima scrive α_T e ne controlla la verità, poi segna il risultato sì o no in memoria, cancella α_T e la relativa computazione e passa ad α_F , operando allo stesso modo. Alla fine, dagli esiti su α_T , α_F , si deduce la validità di α . Notiamo che, anche se i casi da esaminare raddoppiano (non c'è più α , ma ci sono α_T e α_F), lo spazio richiesto non raddoppia, ma resta sostanzialmente lo stesso perché della prima computazione, quella su α_T , si conserva in memoria solo il risultato. Così il ricorso all'induzione non costringe a spazi esponenziali in n , ma mantiene spazi al più polinomiali.

Adesso dobbiamo mostrare che $QSAT$ è $PSPACE$ -arduo, e dunque che ogni $S \in PSPACE$ soddisfa $S \leq_p QSAT$. L'idea della dimostrazione si riferisce al Teorema di Cook-Levin (vedi 8.6.1). Infatti, sappiamo in partenza che c'è una MdT M deterministica che in tempo al più polinomiale decide S : per ogni input w nell'alfabeto A di S , M converge su w e risponde sì o no secondo che $w \in S$ o no; impiega spazio $p(n)$ e dunque tempo $2^{O(p(n))}$ rispetto alla lunghezza n di w , dove p è un opportuno polinomio non dipendente da w . Semmai, per variare la dimostrazione rispetto allo schema seguito per il Teorema di Cook-Levin, riferiamoci stavolta al grafo diretto $G(M, w) = (V(M, w), E(M, w))$ della computazione di M su w . Ricordiamo che i vertici di $V(M, w)$ corrispondono alle configurazioni di M su w , ma c'è un ulteriore vertice $t(M, w)$; esiste un lato in $E(M, w)$ tra due configurazioni se esse si succedono direttamente nella computazione di M su w , e un lato che unisce l'eventuale configurazione di arresto a $t(M, w)$. Indichiamo poi con $s(M, w)$ la configurazione di partenza. Così $w \in S$ se e solo se il grafo ha un cammino da $s(M, w)$ a $t(M, w)$ di lunghezza $2^{O(p(n))}$. Cerchiamo di descrivere quest'ultima condizione con una corrispondente formula Booleana quantificata. Dunque dobbiamo esprimere: in G c'è un cammino $\leq 2^{p(n)}$ lati che conduce da $s(M, w)$ a $t(M, w)$. Possiamo utilizzare l'idea del Teorema di Savitch: c'è un cammino come richiesto se e solo se c'è una tappa intermedia tra $s(M, w)$ e $t(M, w)$ con $\leq 2^{p(n)-1}$ lati da $s(M, w)$ a metà percorso e $\leq 2^{p(n)-1}$ lati dopo, fino a $t(M, w)$. In genere, per v_k, v'_k vertici di G , possiamo esprimere

“c’è un cammino di al più 2^k lati da v_k a v'_k ”

dicendo che

“c’è un vertice w_k con al più 2^{k-1} lati da v_k a w_k e al più 2^{k-1} lati da w_k a v'_k ”.

Il difetto di questa strategia è che la riduzione da k a $k - 1$ raddoppia la lunghezza della formula (da costruire), e dunque il tempo di scrittura. In questo modo, siccome possiamo attenderci $p(n)$ successive riduzioni, il tempo complessivo di scrittura può arrivare a $2^{p(n)}$ passi, troppi se vogliamo che la nostra traduzione in *QSAT* sia polinomiale in n . Un trucco usuale in logica elementare ci aiuta: sostanzialmente, anziché dire

“c’è un vertice w_k con al più 2^{k-1} lati da v_k a w_k e al più 2^{k-1} lati da w_k a v'_k ”, diciamo

“c’è un vertice w_k tale che, per ogni v_{k-1}, v'_{k-1} ,
se $v_{k-1} = v_k$ e $v'_{k-1} = w_k$ oppure $v_{k-1} = w_k$ e $v'_{k-1} = v_k$,
allora c’è una serie di al più 2^{k-1} lati da v_{k-1} a v'_{k-1} ”.

Il procedimento aumenta la lunghezza della frase risultante in ragione della premessa su w_k, v_{k-1}, v'_{k-1} , ma in modo costante rispetto a k ; comunque non la raddoppia.

Adesso occorre scrivere formalmente la nostra formula per k . Ci servono variabili proposizionali $p_{k,0}, p_{k,1}, p_{k,2}, q_k$, cui diamo un significato informale nello stile del teorema di Cook-Levin, come segue.

- $p_{k,0}$: w_k è un elemento su un cammino da v_k a v'_k ,
- $p_{k,1}$: $v_{k-1} = v_k$ e $v'_{k-1} = w_k$,
- $p_{k,2}$: $v_{k-1} = w_k$ e $v'_{k-1} = v_k$,
- q_k : c’è un cammino di al più 2^{k-1} lati tra v_{k-1} e v'_{k-1} .

Scriviamo allora

$$\forall p_{k,0} \forall p_{k,1} \forall p_{k,2} \exists q_k (\neg p_{k,0} \vee \neg (p_{k,1} \vee p_{k,2}) \vee q_k).$$

Il significato informale è

“comunque la si pensi sul fatto w_k è un elemento su un cammino da v_k a v'_k , che $v_{k-1} = v_k$ e $v'_{k-1} = v_k$ oppure $v_{k-1} = w_k$ e $v'_{k-1} = v'_k$, c’è modo di credere che c’è un cammino di al più 2^{k-1} lati tra v_{k-1} e v'_{k-1} ”.

La costruzione dell’intera formula corrispondente a w richiede ovviamente che, per ogni $k \leq p(n)$, q_k sia poi sostituita dall’analogia formula al livello $k - 1$. Per $k = 0$, però, ci serve una “effettiva” variabile q_0 col significato informale

$$“v_1 = v'_1 \text{ oppure } (v_1, v'_1) \in E”.$$

Un po’ di pazienza permette di compilare l’intera formula e di verificare che soddisfa tutte le vostre richieste. I dettagli, che qui omettiamo, ripetono, come già detto, le idee del Teorema di Cook-Levin. $\square$

Adesso, per $h \geq 1$, consideriamo l'insieme QBF_h delle formule di QBF che hanno h variabili proposizionali $p_1, \dots, p_h$ e h alternanze di quantificatori, come sotto descritto

$$\exists p_1 \forall p_2 \exists p_3 \forall p_4 \cdots Q_h p_h \alpha(p_1, \dots, p_h)$$

dove Q_h è, ovviamente, $\exists$ se h è dispari e $\forall$ se h è pari, $\alpha(p_1, \dots, p_h)$ è poi una formula proposizionale nelle variabili proposizionali $p_1, \dots, p_h$.

Sia poi $QSAT_h$ l'insieme delle formule vere in QBF_h , cioè

$$QSAT_h = QBF_h \cap QSAT.$$

La struttura delle formule di QBF_h richiama in modo naturale la definizione della gerarchia polinomiale e in particolare Σ_h^P . In effetti, non è difficile mostrare che

$$QSAT_h \in \Sigma_h^P.$$

Il lettore potrà convincersene per **esercizio** confrontando le definizioni di Σ_h^P e di QBF_h e ricordando che il problema di controllare se una data valutazione soddisfa una formula $\alpha(p_1, \dots, p_h)$ è in P , e che le variabili $p_1, \dots, p_h$ di α hanno lunghezza ovviamente limitata da quella di α .

Inoltre, adattando ancora opportunamente la dimostrazione del Teorema di Cook-Levin, si può vedere che $QSAT_h$ è arduo, quindi completo, in Σ_h^P rispetto a $\leq_p$: per ogni $S \in \Sigma_h^P$, $S \leq_p QSAT_h$. Trascuriamo i dettagli e ci limitiamo a enunciare:

Teorema 10.6.2 *Per ogni intero $h \geq 1$, $QSAT_h$ è Σ_h^P -completo.*

Adesso ricordiamo che, per ogni $h \geq 1$, $QSAT_h = QBF_h \cap QSAT$, e che le formule di QBF_h si riconoscono facilmente alla semplice lettura tra quelle di QBF . Così l'immersione di QBF_h in QBF prova che $QSAT_h \leq_p QSAT$. Si deduce allora:

Teorema 10.6.3 *Per ogni intero $h \geq 1$, $\Sigma_h^P \subseteq PSPACE$. Dunque $PH \subseteq PSPACE$.*

Dimostrazione. Infatti ogni $S \in \Sigma_h^P$ è riducibile in $\leq_p$ a $QSAT_h$ e, tramite $QSAT_h$, a $QSAT \in PSPACE$. Questo prova la prima affermazione, la seconda deriva invece dall'uguaglianza $PH = \bigcup_{h \geq 1} \Sigma_h^P$. $\square$

In particolare, siccome $PSPACE \subseteq PEXP$, segue anche $PH \subseteq PEXP$.

L'esistenza di problemi $PSPACE$ -completi, insieme all'eguaglianza $PSPACE = NPSPACE$, ci permette finalmente di dare la dimostrazione completa del teorema 8.10.1 di Baker, Gill, Solovay. Eccone i dettagli.

Teorema (Baker, Gill, Solovay) 10.6.4

1. Esiste un oracolo $\mathcal{O}_1$ tale che $P^{\mathcal{O}_1} = NP^{\mathcal{O}_1}$.
2. Esiste un oracolo $\mathcal{O}_2$ tale che $P^{\mathcal{O}_2} \neq NP^{\mathcal{O}_2}$.

Dimostrazione.

1. Sia $\mathcal{O}_1$ un qualunque problema $PSPACE$ -completo (per esempio $\mathcal{O}_1 = SAT$). La $PSPACE$ -completezza di $\mathcal{O}_1$ implica

$$P^{\mathcal{O}_1} = P^{PSPACE},$$

d'altra parte

$$P^{PSPACE} = PSPACE$$

perché $P \subseteq PSPACE$. Così $P^{\mathcal{O}_1} = PSPACE$. Allo stesso modo $NP^{\mathcal{O}_1} = NP^{PSPACE} \subseteq NPSPACE$. Dal teorema di Savitch, $PSPACE = NPSPACE$, così $NP^{\mathcal{O}_1} \subseteq PSPACE = P^{\mathcal{O}_1}$. Segue $NP^{\mathcal{O}_1} = P^{\mathcal{O}_1}$.

2. Lavoriamo per semplicità sull'alfabeto $A = \{0, 1\}$. Per ogni $S \subseteq A^*$, formiamo un insieme S' , composto dalle stringhe 0^n con n intero positivo tale che S ha almeno una parola di lunghezza n . È chiaro che, per ogni scelta di S , $S' \in NP^S$, per esempio mediante la seguente procedura: per n intero positivo,

- scegliamo non deterministicamente una parola w di lunghezza n su A ,
- chiediamo all'oracolo S se $w \in S$ o no,
- se sì, poniamo 0^n in S' .

Adesso costruiamo un insieme $S (= \mathcal{O}_2)$ tale che $S' \notin P^S$. Così $P^S \neq NP^S$, come richiesto. La dimostrazione non richiede qui novità particolari, ma solo un po' di ingegnosità e un classicissimo argomento di diagonalizzazione. Per questo enumeriamo tutte le macchine di Turing deterministiche che lavorano in tempo polinomiale limitato con accesso ad oracolo su A

$$M_0, M_1, M_2, \dots, M_k, \dots,$$

si intende che le computazioni di ciascuna M_k prevedono domande ad un oracolo (che può variare caso per caso). Siano poi

$$p_0, p_1, \dots, p_k, \dots,$$

polinomi che limitano la lunghezza delle computazioni delle M_k . Per ogni naturale k definiamo

$$S_k \subseteq A^*, n_k \in \mathbb{N}$$

come segue.

Per $k = 0$, si pone $S_0 = \emptyset$, $n_0 = 0$.

Per $k > 0$, sia anzitutto n_k il minimo intero positivo tale che:

- (i) n_k supera la lunghezza massima delle parole coinvolte da $M_0, \dots, M_{k-1}$ su input di lunghezza $l \leq n_{k-1}$ nei primi $p_0(n_{k-1}), \dots, p_{k-1}(n_{k-1})$ passi della computazione;
- (ii) $p_k(n_k) < 2^{n_k}$.

Si noti che, in particolare, $n_k > n_{k-1}$.

Se poi 0^{n_k} è accettato da M_k quando attinge all'oracolo S_{k-1} , si pone $S_k = S_{k-1}$. Altrimenti scegliamo una parola w_k di lunghezza n_k che non è impiegata da M_k nei primi $p_k(n_k)$ passi della computazione sull'input 0^{n_k} con l'uso dell'oracolo S_{k-1} : (ii) assicura che w_k esiste, perché ci sono 2^{n_k} parole su A di lunghezza n_k ; poniamo allora $S_k = S_{k-1} \cup \{w_k\}$. Finalmente poniamo

$$S = \bigcup_{k \in \mathbb{N}} S_k.$$

Mostriamo che $S' \notin P^S$, come richiesto. Altrimenti c'è un naturale k tale che S' è l'insieme accettato da M_k quando ricorre all'oracolo S . Così, in particolare, $0^{n_k} \in S'$ se e solo se

($\star$) 0^{n_k} è accettato da M_k attingendo all'oracolo S .

D'altra parte, la condizione ($\star$) è equivalente a

($\star\star$) 0^{n_k} è accettato da M_k attingendo a S_k .

Infatti ($\star\star$) implica ($\star$) perché $S_k \subseteq S$. Viceversa, ($\star$) implica ($\star\star$) perché una computazione convergente di M_k su input di lunghezza $\leq n_k$, come 0^{n_k} , non può coinvolgere parole di lunghezza $\geq n_{k+1}$, e quindi stringhe in $S - S_k$ (che hanno tutte lunghezza $\geq n_{k+1}$).

D'altra parte ($\star\star$) equivale ad escludere parole di lunghezza n_k in S (visto che l'unica parola di tale lunghezza ammissibile in S è w_k) e quindi a concludere $0^{n_k} \notin S'$. Si giunge così alla contraddizione

$$0^{n_k} \in S' \text{ se e solo se } 0^{n_k} \notin S'.$$

Segue che $S' \notin P^S$. □

10.7 Esercizi

1. L (cioè $LOGSPACE$) include i linguaggi finiti?
2. L'insieme dei numeri naturali pari è in NL ? E quello dei multipli di 3?
3. È vero che $NPSPACE = coNPSPACE$?
4. Si mostri che $PSPACE$ è chiuso per $\leq_p$.
5. Si cerchi di mostrare in dettaglio che, per ogni $h \geq 1$, $QSAT_h$ è Σ_h^P -completo.
6. Si provi che, se $PH = PSPACE$, allora $\Sigma_h^P = \Sigma_{h+1}^P$ per qualche h . (Suggerimento: se $PH = PSPACE$, allora $QSAT$ è in PH e quindi in Σ_h^P per qualche h).

Riferimenti bibliografici L'idea di usare lo spazio (e dunque le risorse di memoria di una MdT) come misura di complessità è già in [48] e [49]. Esempi di problemi in *PSPACE* e non in *L* sono discussi, per esempio, in [60] e [61]. La dimostrazione originaria del teorema di Savitch è in [95]. Il teorema di Immerman e Szelepsc'enyi che stabilisce l'uguaglianza di *NL* e *coNL* è provato in [58] e [105], quello di Stockmeyer e Meyer sulla *PSPACE*-completezza di *QSAT* in [104]. Ricordiamo anche il riferimento [43] per il teorema di Baker, Gill, Solovay. Tutti questi argomenti sono comunque considerati nei testi generali già elencati nelle note bibliografiche del Capitolo 8.

Classi di Complessità Probabilistiche

11.1 Probabilmente primi

Talora si può accelerare la rapidità di una risposta sacrificandone la precisione. Se l'errore non è troppo probabile, o comunque è tollerabile in ragione del risparmio di tempo, la scelta può anche essere proficua. Consideriamo per esempio il caso dei numeri primi. Sappiamo che il loro insieme è in P . Infatti dal 2002 è noto un algoritmo deterministico, denominato *AKS* dalle iniziali dei suoi creatori - Agrawal, Kayal, Saxena - capace di stabilire se un dato naturale $N \geq 2$ è primo o composto in tempo più o meno polinomiale di grado 11 rispetto a $\log N$. Inutile sottolineare che *AKS* è perfettamente affidabile e degno di fede, sia quando dichiara "*N primo*" che quando risponde "*N composto*". Tuttavia, ancor oggi, nelle situazioni pratiche in cui è utile riconoscere i primi, si continuano a preferire algoritmi meno precisi e talora fallibili, comunque più rapidi di *AKS* e del grado 11 da esso richiesto. I più famosi di questi procedimenti sono quelli di

- Solovay-Strassen,
- Miller-Rabin

e risalgono entrambi agli anni settanta. Vediamo per esempio come funziona il secondo dei due, e perché si preferisce adoperarlo.

L'algoritmo di Miller-Rabin. Ci si basa su due semplici proprietà di un primo N .

- (i) Anzitutto si ricorda che, per un primo N , le uniche soluzioni di

$$x^2 \equiv 1 \pmod{N}$$

sono ± 1 . Infatti, se N è primo e divide $x^2 - 1 = (x - 1)(x + 1)$, allora N divide uno dei due fattori $x - 1, x + 1$, cioè $x \equiv 1 \pmod{N}$ o $x \equiv -1 \pmod{N}$. Si noti che la proprietà non sempre si estende ai composti. Per esempio, per $N = 8$, i quattro valori distinti $\pm 1, \pm 3$ soddisfano tutti $x^2 \equiv 1 \pmod{8}$.

- (ii) C'è poi da ricordare il classico risultato attribuito a Fermat, e anzi chiamato *Piccolo Teorema di Fermat*, secondo il quale, se N è primo e r è un qualunque intero primo con N , allora

$$r^{N-1} \equiv 1 \pmod{N}.$$

Purtroppo però, la condizione non vale solo per i primi. Per esempio $N = 561 = 3 \cdot 11 \cdot 17$ è composto, ma per ogni intero r primo con N si ha

$$r^{560} \equiv 1 \pmod{561}.$$

In realtà $N = 561$ è il più piccolo controesempio a questo proposito, ma è accompagnato da infiniti altri N che condividono la stessa situazione (e sono chiamati *pseudoprimi di Carmichael*).

Comunque il metodo di Miller-Rabin combina (i) e (ii) per ricavare un metodo di primalità rapido anche se fallibile. Vediamo come.

È dato $N \geq 2$. Possiamo supporre N dispari, perché, se N è pari, $N = 2$ oppure N è ovviamente composto, in quanto maggiore di 2 e divisibile per 2. Così $N - 1$ è pari e possiamo scrivere

$$N - 1 \equiv 2^s \cdot d$$

con s, d interi positivi, d dispari. Per esempio

- per $N = 13$, $N - 1 = 12 = 2^2 \cdot 3$,
- per $N = 7$, $N - 1 = 6 = 2 \cdot 3$,
- per $N = 561$, $N - 1 = 560 = 2^4 \cdot 35$.

Fissiamo un intero r con $1 < r < N$ e calcoliamo il massimo comun divisore (r, N) di r e N , per esempio col (rapido) algoritmo delle divisioni successive di Euclide. Se $(r, N) \neq 1$, abbiamo trovato un divisore proprio (r, N) di N e possiamo dichiarare “ N COMPOSTO”. Altrimenti, se $(r, N) = 1$, calcoliamo r^{N-1} modulo N . Se $r^{N-1} \not\equiv 1 \pmod{N}$, possiamo nuovamente dedurre, grazie al piccolo teorema di Fermat, “ N COMPOSTO”. Altrimenti

$$r^{N-1} \equiv 1 \pmod{N},$$

cioè

$$r^{2^s \cdot d} \equiv 1 \pmod{N}.$$

Del numero $r^{2^s \cdot d}$ possiamo calcolare la radice quadrata, quella quarta, quella ottava, fino alla 2^s -ma

$$r^{2^{s-1} \cdot d}, r^{2^{s-2} \cdot d}, \dots, r^d \text{ modulo } N.$$

Se N è primo, (i) garantisce che

- tutti questi valori coincidono con 1 modulo N

oppure che

- il primo di essi che è $\not\equiv 1$ modulo N è -1 .

Così, se queste eventualità non si verificano, possiamo correttamente dedurre “ N COMPOSTO”. Se invece uno dei due casi si attua, possiamo azzardare “ N PRIMO”, per la precisione dichiarare “ N PROBABILMENTE PRIMO”; infatti niente è emerso che contraddica la primalità di N , ma non c’è neppure la certezza che N sia primo. In effetti, la prima risposta “ N COMPOSTO” è sicura, la seconda “ N PROBABILMENTE PRIMO” no.

Illustriamo la situazione e il procedimento di Miller-Rabin con alcuni semplici esempi relativi a valori di N assai piccoli, utili per capire (ma lontani da quelli a cui l’algoritmo si applica nella pratica).

Esempi.

1. Supponiamo $N = 13$. Così $N - 1 = 2^2 \cdot 3$. Affidiamoci a $r = 2$, che è primo con 13. Notiamo

$$2^{2^2 \cdot 3} = 2^{12} \equiv 1 \pmod{13},$$

$$2^{2^2 \cdot 3} = 2^6 = 64 = 65 - 1 \equiv -1 \pmod{13}.$$

Così N è dichiarato “PROBABILMENTE PRIMO” (e infatti è primo).

2. Sia ora $N = 7$. Così $N - 1 = 2 \cdot 3$. Ancora in riferimento a $r = 2$, si ha

$$2^{2^2 \cdot 3} = 2^6 \equiv 1 \pmod{7},$$

$$2^3 \equiv 1 \pmod{7}.$$

Così N risulta “PROBABILMENTE PRIMO” (e infatti è primo).

3. Adesso consideriamo $N = 561 = 3 \cdot 11 \cdot 17$, che primo non è. Si ha $560 = 2^4 \cdot 35$. Se prendiamo un qualunque b primo con 561, sappiamo già che vale

$$b^{560} \equiv 1 \pmod{561}$$

perché N è pseudoprimo di Carmichael. Così, se scegliamo $r = b^{2^4}$, deduciamo

$$r^{2^4 \cdot 35} = r^{560} = (b^{560})^{2^4} \equiv 1 \pmod{561},$$

$$r^{2^3 \cdot 35} = r^{280} = (b^{560})^{2^3} \equiv 1 \pmod{561},$$

$$r^{2^2 \cdot 35} = r^{140} = (b^{560})^{2^2} \equiv 1 \pmod{561},$$

$$r^{2 \cdot 35} = r^{70} = (b^{560})^2 \equiv 1 \pmod{561},$$

$$r^{35} = b^{560} \equiv 1 \pmod{561}.$$

Se ne conclude che $N = 561$ è “probabilmente primo”, mentre in realtà è composto. Dunque $r = b^{2^4}$ è testimone inattendibile della primalità di N

per ogni b primo con N . D'altra parte, basta far riferimento a $r = 2$ per avere invece (tramite l'algoritmo di Miller-Rabin) la conferma sicura che N è composto, infatti

$$\begin{aligned} 2^{560} &\equiv 1 \pmod{561}, \\ 2^{280} &\equiv 1 \pmod{561}, \\ 2^{140} &\not\equiv \pm 1 \pmod{561}, \end{aligned}$$

come si può controllare con l'uso degli algoritmi rapidi di elevamento a potenza attualmente noti.

In effetti si può mostrare con argomenti non banali che, per ogni input N composto,

- c'è almeno un testimone onesto r che lo dichiara tale e non afferma scorrettamente che N è probabilmente primo,
- anzi, almeno $\frac{3}{4}$ degli interi r interpellati è attendibile, e dunque la probabilità di errore per un singolo r è al più $\frac{1}{4}$.

Ovviamente il ricorso a k testimoni r e una loro concorde risposta abbassano la probabilità di errore a $\leq \frac{1}{4^k}$. Inoltre si vede che tutti i calcoli richiesti per un certo r si svolgono in tempi rapidi rispetto a $\log_2 N$ (certamente preferibili a quelli di *AKS*), in particolare, i valori

$$r^{2^{s-1} \cdot d}, r^{2^{s-2} \cdot d}, \dots, r^d$$

eventualmente da calcolare modulo N sono al più s , e s è ovviamente minore o uguale di $\log_2 N$ (infatti 2^s è la massima potenza di 2 che divide N).

Ricapitolando, l'algoritmo di Miller-Rabin funziona così. È dato un input $N \geq 2$ dispari. Si scrive $N - 1 = 2^s \cdot d$ con s, d interi positivi, d dispari. Si interPELLA poi un intero r con $1 < r < N$, scelto casualmente.

- si calcola (r, N) ; se $(r, N) \neq 1$, si conclude "*N COMPOSTO*";
- se $(r, N) = 1$ e $r^{N-1} = r^{2^s \cdot d} \not\equiv 1 \pmod{N}$ si conclude nuovamente "*N COMPOSTO*";
- altrimenti, per $(r, N) = 1$ e $r^{N-1} \equiv 1 \pmod{N}$, calcoliamo (finché necessario)

$$r^{2^{s-1} \cdot d}, r^{2^{s-2} \cdot d}, \dots, r^d \text{ modulo } N.$$

Se c'è qualche valore $\not\equiv 1 \pmod{N}$, e il primo di questi valori è anche $\not\equiv -1 \pmod{N}$, diciamo "*N COMPOSTO*"; altrimenti diciamo "*N PROBABILMENTE PRIMO*".

Come detto, il procedimento è rapido, ma può mentire. Più precisamente

- se l'algoritmo dichiara "*N COMPOSTO*", N è certamente composto, cioè la probabilità di errore è 0;
- se l'algoritmo dichiara "*N PROBABILMENTE PRIMO*", può essere che N sia invece composto, ma questa eventualità ha probabilità $\leq \frac{1}{4}$ (riducibile a $\leq \frac{1}{4^k}$ dopo k controlli su testimoni r distinti).

11.2 Montecarlo o Las Vegas?

Possiamo allora ritenere *computabile* ciò che è computabile in tempi rapidi, cioè al più polinomiali, e, se non proprio con esattezza assoluta, almeno con un alto livello di precisione, al di là di un margine tollerabile di imperfezione. Testimonianze illustri ci sostengono in questa scelta; per esempio quella, autorevole, di un grande matematico come Emil Borel, secondo cui “*un evento che ha probabilità minore di 10^{-50} non accadrà mai e, se accadrà, non sarà mai osservato*”.

Anche il caso dei numeri primi avvalora questa impostazione. In effetti il procedimento di Miller e Rabin è un esempio di algoritmo probabilistico, più precisamente di un algoritmo di tipo *Montecarlo*, nel senso che

- fornisce sempre risposta,
- talora sbagliata,
- ma con una probabilità di errore conosciuta.

Anche l'algoritmo di Solovay-Strassen sopra menzionato ha le medesime caratteristiche. Ci sono però, anche per la primalità, algoritmi probabilistici con caratteristiche diverse, denominati di tipo *Las Vegas*, per i quali

- è ammessa la facoltà di non rispondere,
- l'eventuale risposta è sempre corretta,
- ed è nota la probabilità di silenzio.

Il rischio di errore o di silenzio deve essere ovviamente bilanciato da un'accelerazione dei tempi di lavoro rispetto agli usuali procedimenti deterministici “*sicuri*”. Quando questa ulteriore esigenza di rapidità è soddisfatta, un algoritmo di tipo Montecarlo può intendersi capace di

fornire risposte probabilmente vere in tempi certamente brevi,

uno di tipo Las Vegas, invece, sa

fornire risposte certamente vere in tempi probabilmente brevi.

Per un trattamento teorico di questi algoritmi, ci occorre il modello formale delle *macchine di Turing probabilistiche*. Si tratta di MdT M su alfabeti A che, per computare un input “reale” w ricorrono all'aiuto di un input “casuale” r (dipendente da w). Intendiamo che M lavora in tempo polinomiale su w se

- anzitutto l'input casuale r ha lunghezza polinomialmente limitata da quella di w ,
- inoltre la computazione di M su (w, r) avviene in tempo polinomialmente limitato da $l(w)$.

Ovviamente i polinomi che regolano la lunghezza di r e quella della computazione su (w, r) in funzione di $l(w)$ non devono dipendere da w , ma solo da M . La computazione di M su w non è quindi unica, perché coinvolge l'input casuale r , e anche la sua conclusione può variare con r . Viene così naturale associare a

w la probabilità, al variare di r , che la computazione di M su w (e r) abbia un particolare esito. Scriveremo allora

$$Pr_r(M \text{ converge su } w),$$

$$Pr_r(M \text{ ha output } w' \text{ su } w),$$

e così via, a intendere la probabilità che l'evento in parentesi si verifichi, al variare di r . Per esempio

$$Pr_r(M \text{ converge su } w) = 1$$

significa che M converge su w (e r) per ogni scelta di r ,

$$Pr_r(M \text{ converge su } w) = 0$$

significa che M diverge sempre su w (e r), quale che sia il testimone interpellato r . $Pr_r(M \text{ converge su } w)$ - e le altre probabilità coinvolte - assumono comunque valori reali tra 0 e 1.

11.3 La classe PP, e altre variazioni sul tema

Un primo, relativamente ovvio, tentativo di introdurre una classe di complessità probabilistica è quello seguente.

Definizione. Un insieme S di parole su A si dice *computabile probabilisticamente in tempo polinomiale* se esiste una MdT probabilistica M su A che lavora in tempo polinomiale per decidere S , nel senso ora specificato: per ogni input $w \in A^*$ e per ogni testimone casuale r , M converge su (w, r) rispondendo “ $w \in S$ ” o “ $w \notin S$ ”, e

- se $w \in S$, allora $Pr_r(M \text{ dichiara } w \in S) > \frac{1}{2}$,
- se $w \notin S$, allora $Pr_r(M \text{ dichiara } w \notin S) > \frac{1}{2}$.

La classe così definita si chiama PP (sigla che sta ad indicare “*Probabilistic Polynomial time*”, tempo probabilistico polinomiale).

Come già insinuato, si tratta di un modello abbastanza grossolano e poco utile nella pratica. In effetti, la MdT M può mentire con probabilità abbastanza grande. Un ulteriore sostegno a questa considerazione è fornito dal prossimo risultato, il quale, tra l'altro, ci dà una prima informazione sulla collocazione di PP rispetto a NP (e a P).

Teorema 11.3.1 $NP \subseteq PP$.

Dimostrazione. Sia S un insieme di parole sull'alfabeto finito A tale che S è in NP . Quindi esiste una macchina di Turing non deterministica M su A che accetta

S in tempo polinomiale: per ogni parola w su A , se $w \notin S$, allora ogni computazione di M su w diverge, mentre, se $w \in S$, allora esiste una computazione di M su w che converge in un numero di passi strettamente limitato da $p(l(w))$ per un opportuno polinomio $p(t)$ a coefficienti interi. Vogliamo trasformare M in una MdT probabilistica M' su A che lavora in tempo polinomiale per decidere S ai sensi della definizione di PP . Non c'è perdita di generalità a supporre che:

- (i) M si arresta se e solo se entra in un fissato stato q_H ;
- (ii) per ogni stato $q \neq q_H$ di M e per ogni $a \in A \cup \{\star\}$, la coppia (q, a) ammette in M esattamente due istruzioni.

Così ogni configurazione di M su w in uno stato diverso da q_H ne genera due successive. Aggiungendo stati e istruzioni opportuni (eventualmente fittizi) possiamo sempre ottenere questo obiettivo. Adesso adattiamo M inserendo due istruzioni anche per ogni coppia (q_H, a) con $a \in A \cup \{\star\}$: per esempio, possiamo ordinare di ricopiare a e poi di spostarsi a destra oppure a sinistra, mantenendosi nello stato q_H . Così le vecchie computazioni divergenti di M su un input w , quelle che non raggiungono mai lo stato q_H , restano invariate; le altre, quelle convergenti perché capaci di arrivare a q_H , si prolungano indefinitamente. Dunque il nuovo programma diverge in ogni sua computazione su w . Conveniamo però di bloccare tutte queste computazioni al passo $p(l(w))$. Si determinano così $2^{p(l(w))}$ rami r di computazione, ognuno di lunghezza $p(l(w))$. Di più, si ha che $w \in S$ se e solo se almeno un ramo si trova nello stato q_H al passo $p(l(w))$. Notiamo anche che tutti i rami r di computazione su w hanno la stessa probabilità

$$\frac{1}{2^{p(l(w))}}$$

di essere eseguiti. Costruiamo adesso la nostra MdT probabilistica M' . Per ogni $w \in A^*$,

- (1) M' lancia idealmente una moneta non truccata;
- (2) se esce testa, M' lancia la moneta per altre $p(l(w)) + 1$ volte, e M' dà output NO se e solo se esce sempre testa, e SI altrimenti;
- (3) se esce croce, allora M' segue uno dei rami r su w .

In M' il testimone casuale τ' è costituito non solo dalla scelta di r , ma anche dai lanci di moneta (1 oppure $p(l(w)) + 2$ a seconda dei casi); comunque questa ulteriore informazione relativa agli esiti dei lanci si scrive ancora al più in spazio $p(l(w)) + 2$. Così i testimoni casuali interpellati da M' hanno lunghezza al più polinomiale rispetto a quelli dell'input. Anche la computazione di M' su w ha la stessa caratteristica.

Finalmente, notiamo che per $w \notin S$, la probabilità che M' dichiari $w \notin S$ è

$$\frac{1}{2^{p(l(w))+2}}$$

se esce testa al primo lancio e

$$\frac{1}{2}$$

se esce croce al primo lancio. Infatti la probabilità che esca testa è $\frac{1}{2}$, e l'eventualità che esca testa altre $p(l(w)) + 1$ volte ha probabilità

$$\frac{1}{2} \cdot \frac{1}{2^{p(l(w))+1}} = \frac{1}{2^{p(l(w))+2}}.$$

Anche la probabilità che esca croce è $\frac{1}{2}$, ma, se $w \notin S$, ogni computazione di M su w risponde no. Quindi in totale

$$Pr_{r'}(M \text{ dichiara } w \notin S) = \frac{1}{2} + \frac{1}{2^{p(l(w))+2}} > \frac{1}{2}.$$

Invece, per $w \in S$, la probabilità che M' dichiari $w \in S$ è esattamente

$$\frac{1}{2} - \frac{1}{2^{p(l(w))+2}}$$

se esce testa al primo lancio, e almeno

$$\frac{1}{2^{p(l(w))+1}}$$

se esce croce al primo lancio: ricordiamo infatti che, per $w \in S$, almeno un ramo r entra nello stato q_H e che tutti i rami hanno la stessa probabilità

$$\frac{1}{2^{p(l(w))}}.$$

In complesso

$$Pr_{r'}(M \text{ dichiara } w \in S) \geq \frac{1}{2} - \frac{1}{2^{p(l(w))+2}} + \frac{1}{2^{p(l(w))+1}} > \frac{1}{2}.$$

Notiamo infine che ogni computazione di M' lavora in tempo polinomiale rispetto alla lunghezza dell'input. Quindi M' è la MdT richiesta. $\square$

Non è noto se $NP = PP$, ma si congettura di no. Comunque l'inclusione $NP \subseteq PP$ basta a catalogare PP come classe troppo vasta, tale da operare, sì, in tempi brevi, ma con probabilità di mentire troppo alta. Ci servono classi probabilistiche più convincenti, con più bassi livelli di errore. Si definisce allora, per esempio, la classe BPP degli insiemi computabili probabilisticamente in tempo polinomiale con errore limitato (la sigla BPP sta, appunto, a sottolineare le iniziali della corrispondente espressione inglese *Bounded error Probabilistic Polynomial time*).

Definizione. Un insieme S di parole su un alfabeto finito A si dice *computabile probabilisticamente in tempo polinomiale con errore limitato* se esistono una MdT

probabilistica M su A che lavora in tempo polinomiale e un reale ϵ con $0 < \epsilon < \frac{1}{2}$ tali che, per ogni parola w su A :

- se $w \in S$, $Pr_r(M \text{ dichiara } w \in S) \geq \frac{1}{2} + \epsilon$,
- se $w \notin S$, $Pr_r(M \text{ dichiara } w \notin S) \geq \frac{1}{2} + \epsilon$.

Può sembrare che il ricorso ad un ϵ verosimilmente piccolo crei poca differenza rispetto a PP , e che BPP possa coincidere senza problemi con PP . In realtà, si può vedere che l'intervento di ϵ è decisivo: per $S \in BPP$, si possono costruire MdT che decidono S lavorando in tempo polinomiale e hanno una probabilità di errore arbitrariamente piccola, anzi tendente a 0 al crescere della lunghezza dell'input. Più precisamente si ha:

Teorema 11.3.2 *Sia S un insieme di parole su un alfabeto finito A . Fissiamo un qualunque intero positivo h . Allora $S \in BPP$ se e solo se c'è una MdT probabilistica M su A che lavora in tempo polinomiale e, per ogni $w \in A^*$, soddisfa:*

- se $w \in S$, $Pr_r(M \text{ dichiara } w \in S) \geq 1 - \frac{1}{2^{l(w)^h}}$,
- se $w \notin S$, $Pr_r(M \text{ dichiara } w \notin S) \geq 1 - \frac{1}{2^{l(w)^h}}$.

Dimostrazione. L'implicazione da destra a sinistra è ovvia. Dobbiamo mostrare il contrario, considerare cioè un problema $S \in BPP$ e costruire una MdT M come richiesto dall'enunciato del teorema. Dalla definizione di BPP sappiamo che ci sono una MdT M_0 su A che lavora in tempo polinomiale e un reale positivo $\epsilon < \frac{1}{2}$ tali che

- se $w \in S$, $Pr_r(M_0 \text{ dichiara } w \in S) \geq \frac{1}{2} + \epsilon$,
- se $w \notin S$, $Pr_r(M_0 \text{ dichiara } w \notin S) \geq \frac{1}{2} + \epsilon$.

Usiamo la stessa strategia già illustrata nella trattazione dell'algoritmo di Miller-Rabin, volta ad abbassare la probabilità di errore tramite la consultazione di più testimoni casuali r . In altre parole, operiamo con M_0 su k input casuali $r_1, \dots, r_k$ (insieme all'input "reale" w). Senza perdita di generalità, supponiamo k dispari. Sia $M(k)$ il programma che risulta se conveniamo, per ogni $w \in A$, quanto segue:

- $M(k)$ dichiara " $w \in S$ " se e solo se tale è l'esito della computazione di M_0 su w per la maggioranza dei k input casuali interpellati $r_1, \dots, r_k$,
- $M(k)$ dichiara " $w \notin S$ " altrimenti.

Intuitivamente, la probabilità di errore per $M(k)$ su w è $< \frac{1}{2}$. In realtà il suo valore risulta assai minore. Per provarlo abbiamo bisogno - non sorprendentemente - di risultati non banali di calcolo delle probabilità, che si chiamano *limiti di Chernoff* e che cerchiamo di illustrare nel nostro caso specifico. Abbiamo un input assegnato e le corrispondenti probabilità

$$Pr_r(M_0 \text{ dichiara } w \in S), \quad Pr_r(M_0 \text{ dichiara } w \notin S)$$

entrambe dipendenti ovviamente da w . Chiaramente, se p denota $Pr_{\tau}(M_0 \text{ dichiara } w \in S)$, allora

$$Pr_{\tau}(M_0 \text{ dichiara } w \notin S) = 1 - p.$$

Possiamo supporre $0 < p < 1$. Come detto, ricorriamo a k testimoni casuali $r_1, \dots, r_k$. Per ogni i ,

$$Pr_{r_i}(M_0 \text{ dichiara } w \in S) = p, \quad Pr_{r_i}(M_0 \text{ dichiara } w \notin S) = 1 - p,$$

dunque le due probabilità non dipendono da i . Comunque le computazioni di M su w con r_1 , o con r_2 , ..., o con r_k possono avere esito diverso: conviene allora associare all'evento

$$E_i: "M_0 \text{ dichiara } w \in S \text{ quando interpella } r_i"$$

una variabile casuale X_i che assume valore

- 1 se l'esito si verifica,
- 0 altrimenti.

Le variabili $X_1, \dots, X_k$ sono tra di loro indipendenti perché la computazione di M su (w, r_i) non incide su quelle relative agli altri testimoni casuali. Adesso supponiamo $w \in S$ (e quindi $p \geq \frac{1}{2} + \epsilon$). Allora

$$M(k) \text{ dà risposta corretta su } w$$

se e solo se il numero degli indici i per cui E_i si verifica, cioè $X_i = 1$, è $> \frac{k}{2}$

$$\sum_{i=1}^k X_i > \frac{k}{2};$$

conseguentemente

$$M(k) \text{ dà risposta sbagliata su } w$$

se e solo se

$$\sum_{i=1}^k X_i < \frac{k}{2}.$$

Così la probabilità che $M(k)$ sbagli su w al variare di $r_1, \dots, r_k$ è

$$\begin{aligned} & Pr_{r_1, \dots, r_k}(M(k) \text{ dichiara } w \notin S) = \\ & = Pr_{r_1, \dots, r_k}\left(\frac{\sum_{i=1}^k X_i}{k} < \frac{1}{2}\right) \leq \\ & \leq Pr_{r_1, \dots, r_k}\left(\frac{\sum_{i=1}^k X_i}{k} - p < \frac{1}{2} - \left(\frac{1}{2} + \epsilon\right)\right) = \\ & = Pr_{r_1, \dots, r_k}\left(\frac{\sum_{i=1}^k X_i}{k} - p < -\epsilon\right) \end{aligned}$$

(infatti $p \geq \frac{1}{2} + \epsilon$, e dunque $-p \leq -(\frac{1}{2} + \epsilon)$). I limiti di Chernoff si applicano a questo contesto generale:

- un reale positivo ε ,
- k variabili casuali indipendenti $X_1, \dots, X_k$, tutte in $\{0, 1\}$,
- una comune probabilità $p = Pr(X_i = 1)$ per $i = 1, \dots, k$.

In tutti questi casi (e dunque nel nostro ambito particolare), esse affermano

$$Pr_{r_1, \dots, r_k} \left(\frac{\sum_{i=1}^k X_i}{k} - p < -\varepsilon \right) < e^{-\varepsilon^2 \cdot \frac{k}{2p(1-p)}}.$$

Ne deduciamo che, per $w \in S$,

$$Pr_{r_1, \dots, r_k} (M(k) \text{ dichiara } w \notin S) < e^{-\varepsilon^2 \cdot \frac{k}{2p(1-p)}}.$$

Analoga conclusione si ottiene per $w \notin S$, in riferimento alla probabilità che $M(k)$ dichiari scorrettamente $w \in S$.

Ora è chiaro che, aumentando opportunamente k , possiamo rendere

$$\varepsilon^2 \cdot \frac{k}{2p(1-p)}$$

arbitrariamente grande e quindi

$$e^{\frac{-\varepsilon^2 \cdot k}{2p(1-p)}}$$

arbitrariamente piccolo, cioè vicino a 0. Più precisamente, ricordiamo che, a prescindere da $l(w)$, p - che pure è funzione di w - soddisfa $0 < p < 1$ e quindi, con calcoli relativamente semplici, $0 < 2p(1-p) \leq \frac{1}{2}$. Dunque

$$\frac{\varepsilon^2 \cdot k}{2p(1-p)} \geq 2\varepsilon^2 \cdot k.$$

Così per ottenere

$$Pr_{r_1, \dots, r_k} (M(k) \text{ dichiara il falso su } w) < 2^{-l(w)^h},$$

come richiesto, e quindi

$$e^{-\frac{\varepsilon^2 \cdot k}{2p(1-p)}} < 2^{-l(w)^h},$$

ci basta

$$2\varepsilon^2 \cdot k \geq \log_2 e \cdot l(w)^h,$$

cioè $k \geq c \cdot l(w)^h$ dove

$$c = \frac{\log_2 e}{2\varepsilon^2}$$

non dipende da w .

In conclusione, M si può costruire come segue: dato $w \in A^*$, si calcola $k = c \cdot l(w)^h$, si considera la corrispondente MdT $M(k)$ e con essa si opera su w .

Siccome k è polinomiale in $l(w)$ e $M(k)$ procede su w come M_0 , riferendosi però a k testimoni casuali diversi, M lavora in tempo polinomiale rispetto a $l(w)$; soddisfa poi ovviamente le altre condizioni richieste. $\square$

Il teorema precedente ci dice quanto poco rilevante sia la scelta di ε nella definizione di BPP . Così possiamo anche fissare $\varepsilon (= \frac{1}{4} \text{ o } \frac{1}{3})$ e definire BPP come la classe dei problemi S per cui esiste una MdT probabilistica M che lavora in tempo polinomiale in modo che, per ogni w ,

- se $w \in S$, $Pr_r(M \text{ dichiara } w \in S) \geq \frac{3}{4}$ ($\frac{2}{3}$ rispettivamente),
- se $w \notin S$, $Pr_r(M \text{ dichiara } w \notin S) \leq \frac{1}{4}$ ($\frac{1}{3}$ rispettivamente).

Il teorema precedente sottolinea anche le differenze tra PP e BPP . Così, mentre $PP \supseteq NP$, non è chiaro se $BPP \subseteq NP$, o viceversa. Invece è evidente che $BPP \subseteq PP$, e che inoltre $P \subseteq BPP$ perché ogni MdT deterministica è anche probabilistica (non ha bisogno di ricorrere a testimoni casuali). Non è noto se $P = BPP$. Del resto, se questo non fosse il caso, ci troveremmo di fronte ad una situazione notevole: esisterebbero problemi fuori di P , quelli appunto in $BPP-P$, che si possono comunque risolvere grosso modo in tempo polinomiale, dal momento che la probabilità di errore di una computazione può essere resa arbitrariamente piccola, e dunque anche minore della probabilità che si verifichi un guasto nell'hardware di un calcolatore.

Esempio. Consideriamo l'insieme S dei numeri primi. L'algoritmo di Miller e Rabin testimonia la sua appartenenza a BPP : infatti, per ogni naturale $N > 2$, il procedimento attesta correttamente la primalità di N se N è primo, mentre la probabilità che l'algoritmo affermi correttamente che N è composto quando N è composto è di almeno $\frac{3}{4}$. Si noti però l'asimmetria della situazione: per N primo, l'algoritmo non mentisce mai; per N composto, talora sì.

L'ultima osservazione giustifica l'introduzione di una ulteriore classe di problemi: la classe RP degli insiemi *computabili probabilisticamente in tempo polinomiale in modo casuale*, meglio confacente alla situazione dell'algoritmo di Miller-Rabin. RP sono qui le iniziali della corrispondente espressione inglese *Random Polynomial time*.

Definizione. Un insieme S di parole su un alfabeto A è in RP se esiste una MdT probabilistica M su A che lavora in tempo polinomiale in modo che, per ogni $w \in A^*$,

- se $w \in S$, $Pr_r(M \text{ dichiara } w \in S) \geq \frac{1}{2}$,
- se $w \notin S$, $Pr_r(M \text{ dichiara } w \notin S) = 1$.

Dunque, se M dice che $w \in S$, la sua risposta è pienamente affidabile; al contrario M può dichiarare che $w \notin S$ anche se $w \in S$, ma la probabilità di questo errore è ridotta a $\leq \frac{1}{2}$. Si osservi che questa è esattamente la situazione dell'algoritmo di Miller e Rabin per l'insieme S dei numeri composti. Vediamo le relazioni tra questa nuova classe e quelle già introdotte.

Teorema 11.3.3 $RP \subseteq NP$, $RP \subseteq BPP$.

Dimostrazione. Sia S un insieme di parole su un alfabeto finito A tale che $S \in RP$. Proviamo dapprima che $S \in NP$. Sia M una MdT probabilistica polinomialmente limitata su A che garantisca, appunto, l'appartenenza di S a RP . Allora, per ogni input $w \in A$:

- se $w \in S$, c'è (almeno) un testimone casuale r di lunghezza polinomialmente limitata da $l(w)$ tale che M converge su (w, r) in tempo ancora polinomialmente limitato da $l(w)$ e dichiara " $w \in S$ " (in realtà la cosa vale per almeno la metà degli r interpellati);
- se invece $w \notin S$, nessun testimone r induce M a dichiarare " $w \in S$ ".

In questa luce M si presta ad essere interpretato come programma atto a collocare S in NP , proprio in base alla definizione di NP . Dunque $S \in NP$. Infatti, fissato r , M lavora in modo deterministico sulla coppia (w, r) . In conclusione $S \in NP$.

Adesso, sempre per $S \in RP$, vogliamo provare che $S \in BPP$. Consideriamo ancora una MdT probabilistica M che lavora in tempo polinomiale e garantisce $S \in RP$ secondo la definizione di RP : per ogni input, w

- se $w \in S$, $Pr_r(M \text{ dichiara } w \in S) \geq \frac{1}{2}$
- se $w \notin S$, $Pr_r(M \text{ dichiara } w \notin S) = 1$.

Definiamo un nuovo algoritmo probabilistico M' come segue. Data una parola w sull'alfabeto A di S , si operano due successive computazioni di M su w facendo riferimento a testimoni casuali r diversi. Se almeno una volta otteniamo l'output $S\hat{I}$ (cioè la conclusione che $w \in S$), confermiamo questo output; altrimenti rispondiamo NO (cioè dichiariamo $w \notin S$). Notiamo che, se $w \notin S$, entrambe le computazioni di M su w concordano che $w \notin S$; quindi M' risponde NO in ogni caso, dunque con probabilità $1 \geq \frac{3}{4}$. Se invece $w \in S$, la probabilità che ambedue le computazioni di M su w rispondano $w \notin S$ è minore o uguale a $\frac{1}{4} = \frac{1}{2} \cdot \frac{1}{2}$, e dunque la probabilità che il nuovo algoritmo M' risponda $S\hat{I}$ è $\geq \frac{3}{4}$. Così M' assicura $S \in BPP$. $\square$

Si osservi che, essendo $RP \subseteq BPP$, ogni insieme S in RP ammette algoritmi probabilistici che lavorano in tempo polinomiale con una possibilità di errore che può essere resa arbitrariamente piccola, e anzi tende a 0 al crescere della lunghezza dell'input; questi algoritmi mantengono però la particolarità dell'asimmetria osservata per il procedimento di Miller-Rabin. Più precisamente si può provare che, per ogni insieme S di parole su un alfabeto finito A , $S \in RP$ se e solo se, per ogni intero positivo h , esiste una macchina di Turing probabilistica polinomialmente limitata M su A tale che, per ogni parola w su A ,

- se $w \in S$, $Pr_r(M \text{ dichiara } w \in S) \geq 1 - \frac{1}{2^{l(w)^h}}$
- se $w \notin S$, $Pr_r(M \text{ dichiara } w \notin S) = 1$.

Notiamo adesso che il procedimento di Miller-Rabin (e altri legati alla primalità) hanno una comune caratteristica: possono mentire ma rispondono sempre, secondo il modello già denominato come *Montecarlo*. Sappiamo però che esistono anche algoritmi probabilistici di natura diversa, cioè *Las Vegas*: quando rispondono, rispondono correttamente SI oppure NO ; ma possono non rispondere o, equivalentemente, affermare $NON SO$. Corrispondentemente introduciamo una nuova classe ZPP , quella degli insiemi *computabili probabilisticamente in tempo polinomiale con errore zero* (il motivo della sigla ZPP è facilmente intuibile, derivando dall'inglese “Zero error Probabilistic Polynomial time”).

Definizione. Un insieme S di parole su un alfabeto finito A è in ZPP se esiste una MdT probabilistica polinomialmente limitata M su A con tre possibili output SI , NO , $?$ tale che, per ogni parola w su A ,

- $Pr_r(M \text{ risponde } ? \text{ su } w) \leq \frac{1}{2}$,
- per ogni input casuale r per cui M dà risposta diversa da $?$ su (w, r) , $w \in S$ esattamente quando M dà output SI .

Dunque, per $S \in ZPP$, c'è una MdT probabilistica che lavora in tempo polinomiale ma, su un generico input w può dare come output, oltre alle usuali risposte SI e NO , altri output $?$ interpretabili come $NON SO$; la probabilità di queste risposte incerte è comunque bassa al variare di r , e le risposte certe, quando fornite, sono assolutamente sicure.

Chiaramente $P \subseteq ZPP$ (sempre per il motivo che le MdT deterministiche si possono intendere come MdT probabilistiche che non hanno bisogno di testimoni). Si ha poi il seguente teorema.

Teorema 11.3.4 $ZPP \subseteq RP$.

Dimostrazione. Sia S un insieme di parole su un alfabeto finito A . Supponiamo che S appartenga a ZPP grazie alla MdT probabilistica polinomialmente limitata M . Modifichiamo M in modo da controllare, al termine di ogni computazione su un input w , quale è l'output scritto sul nastro; se esso è SI o NO , lo si lascia inalterato, altrimenti, nel caso dubbio $?$, lo si sostituisce con NO . La MdT M' ottenuta in questo modo è ancora probabilistica e lavora in tempo polinomiale. Inoltre, se $w \in S$, allora $Pr_r(M \text{ dichiara } w \in S) \geq \frac{1}{2}$ e quindi $Pr_r(M' \text{ dichiara } w \in S) \geq \frac{1}{2}$. Se invece $w \notin S$, $Pr_r(M' \text{ dichiara } w \notin S) = 1$. Perciò M' garantisce $S \in RP$. $\square$

Volendo riassumere le inclusioni sin qui provate tra le classi di complessità introdotte, possiamo scrivere:

$$P \subseteq ZPP \subseteq RP \subseteq BPP \subseteq PP,$$

e anche

$$RP \subseteq NP \subseteq PP.$$

L'analogia con i casi RP e BPP suggerisce che, per $S \in ZPP$, si possono trovare MdT probabilistiche polinomialmente limitate che testimoniano, appunto, $S \in ZPP$ e per le quali la probabilità di una risposta del tipo *NON SO* si può rendere arbitrariamente piccola (in modo analogo a quel che capita, appunto, a RP e BPP). Effettivamente è possibile provare questo risultato con l'ovvia strategia di ripetere il ricorso a testimoni casuali.

C'è però un'altra caratterizzazione di ZPP che ci piace considerare. Finora, quando abbiamo valutato l'efficienza degli algoritmi e, conseguentemente, la complessità dei problemi rispetto al parametro "tempo", abbiamo fatto sempre riferimento al numero massimo di passi delle computazioni convergenti e dunque, in definitiva, ai casi peggiori che possono verificarsi. Anche trattando le classi probabilistiche abbiamo privilegiato la rapidità di tutte le computazioni, incluse le peggiori, anche a costo di sacrificare l'esattezza. D'altra parte, possiamo sempre recuperare il requisito della precisione. Ammettiamo infatti di avere $S \subseteq A^*$ e di voler controllare se un dato input $w \in A^*$ è in S o no. Supponiamo poi $S \in ZPP$, o $S \in BPP$, tramite una MdT probabilistica M .

1. Per $S \in ZPP$, interpellando più di metà dei possibili testimoni r , ne troviamo almeno uno che dà output diverso da ?, ed è dunque assolutamente affidabile.
2. Anche per $S \in BPP$, se più di metà dei possibili testimoni r concorda che $w \in S$ o $w \notin S$, possiamo correttamente dedurre che $w \in S$ o no.

Dunque se ci affidiamo non più ad un singolo input casuale, ma al complesso di tutti i testimoni o anche solo alla loro maggioranza, abbiamo risposte certe. Ma il problema è che, per $|A| \geq 2$ e p polinomio che limita $l(r)$ rispetto a $l(w) = n$, i testimoni complessivi - cioè le parole di lunghezza $p(n)$ su A - sono almeno $2^{p(n)}$, e la loro metà $\frac{2^{p(n)}}{2} = 2^{p(n)-1}$ è esponenziale in n . Dunque il nuovo testimone casuale - il complesso dei precedenti - è troppo lungo, e tale risulta anche la sua consultazione. Si recupera l'esattezza, ma si perde di rapidità *almeno nei casi peggiori*. Tuttavia, almeno nel caso di ZPP può capitare che già il primo dei testimoni interpellati r si sbilanci in una risposta diversa da ? e permetta di chiudere la computazione. Dunque, se non i tempi peggiori, almeno i tempi medi di lavoro (al variare di r) sembrano ancora ragionevoli. Precisiamo meglio il contesto. Abbiamo a che fare, per la precisione, con macchine di Turing M che, insieme agli input reali w , coinvolgono testimoni casuali r ; per ogni w , calcoliamo il tempo medio delle computazioni di M su (w, r) al variare di r , consideriamo poi, per ogni intero positivo n , il massimo valore di questi tempi medi al variare di w . Ammettiamo che la funzione così ottenuta sia $O(n^k)$ per qualche intero positivo k , e diciamo in questo caso che M lavora in *tempo medio polinomiale*. Stabiliamo allora che un problema $S \subseteq A^*$ ha un algoritmo di decisione che lavora in tempo medio polinomiale se c'è una MdT M su A che lavora in tempo medio polinomiale e decide S nel senso seguente: dato $w \in A^*$

- se $w \in S$, ogni computazione di M su w e su un input casuale r dichiara " $w \in S$ ";

- se $w \notin S$, ogni computazione di M su w e su un input casuale r dichiara “ $w \notin S$ ”

(come nel caso di $S \in ZPP$ quando consultiamo se necessario fino a metà dei testimoni). Si ha in effetti

Teorema 11.3.5 *ZPP è la classe dei problemi che hanno un algoritmo di decisione che lavora in tempo medio polinomiale.*

Dimostrazione. Sia $S \in ZPP$. Dunque c'è una MdT probabilistica M che lavora in tempo polinomiale con output ?, SI, NO e soddisfa

- per ogni input w , $\Pr_r(M \text{ dà output ? su } w) \leq \frac{1}{2}$
- per ogni input reale w e per ogni testimone casuale r per cui M dà output diverso da ?, si ha che $w \in S$ se e solo se M dà output SI su (w, r) .

Costruiamo una nuova MdT M' che imita M ma, come già sopra accennato, quando M arriva all'output ? su un dato input w , ricomincia la computazione con un altro testimone casuale r' . La risposta di M' (quando arriva) è dunque corretta, e comunque è destinata ad arrivare perché almeno metà dei testimoni r per M conduce M a rispondere. Purtroppo può capitare, nei casi peggiori, che l'attesa di un testimone r per M privo di dubbi su w si prolunghi eccessivamente fino a $N(n) \geq 2^{p(n)-1}$ consultazioni (con la notazione sopra adoperata per p). D'altra parte sappiamo che M impiega per ogni r un tempo (al più) polinomiale $T(n)$ sugli input w di lunghezza $\leq n$. In questa situazione, cerchiamo di valutare il tempo **medio** di lavoro di M' . È quindi dato w con $l(w) \leq n$. Quando M interPELLa un primo testimone casuale r , c'è probabilità almeno $\frac{1}{2}$ che M dia risposta certa e corretta su w (in tempo $\leq T(n)$), conseguentemente M' si arresta su w entro $T(n)$ passi in almeno metà dei casi. Altrimenti, M' coinvolge un secondo testimone casuale tramite M ; la probabilità di ricevere risposta certa in questa maniera diventa $\frac{1}{4}$, il tempo complessivo di lavoro $2T(n)$. In generale, k testimoni richiedono a M' tempo $\leq kT(n)$ e assicurano risposta per almeno $\frac{1}{2^k}$ dei casi totali. Ripetiamo poi che il numero di testimoni da interPELLare è finito, $N(n) \leq 2^{p(n)-1}$. Così M' converge comunque su w e il tempo medio di lavoro di M' per input di lunghezza $\leq n$, come w , è al più $\frac{1}{2}T(n) + \frac{1}{2^2}2T(n) + \frac{1}{2^3}3T(n) \cdots + \frac{1}{2^N}NT(n) = \sum_{k=1}^N \frac{1}{2^k}kT(n)$ che è comunque $O(T(n))$, dunque al più polinomiale in n , come richiesto.

Viceversa, sia S un problema deciso da una MdT M che lavora in tempo medio polinomiale $T(n)$ su input w di lunghezza $\leq n$. Per mostrare che $S \in ZPP$, facciamo lavorare M solo per $2T(n)$ passi, accettandone l'esito se M converge entro tale limite e convenendo l'output ? se la computazione si prolunga. Si noti che si può assumere che gli input casuali interPELLati da M su input reali di lunghezza n abbiano lunghezza polinomialmente limitata da n , e anzi $\leq 2T(n)$ (tale è infatti la loro porzione coinvolta nella computazione). È facile vedere che si ottiene

un procedimento probabilistico adeguato a ZPP : lavora in tempo polinomiale $2T(n)$ rispetto alla lunghezza n dell'input w ; quando dà risposte diversa da ?, le dà corrette; resta nel dubbio ? in al più la metà dei casi possibili, altrimenti, in almeno metà dei testimoni casuali r , richiede tempo di lavoro $\geq 2T(n)$, e dunque ha tempo medio $> \frac{1}{2} \cdot 2T(n) = T(n)$, il che è assurdo. $\square$

Concludiamo il paragrafo con alcuni cenni sulle classi complementari di PP , BPP , ZPP e RP . La simmetria che si osserva nelle definizioni di PP , BPP , ZPP tra il caso in cui $w \in S$ e quello in cui $w \notin S$ permette facilmente di dedurre

$$coPP = PP, coBPP = BPP, coZPP = ZPP.$$

Non altrettanto vale per RP . Infatti si ha che un insieme S di parole su un alfabeto finito A è in $coRP$ se e solo se il suo complementare S^c è in RP e dunque se e solo se c'è una MdT probabilistica M' su A che lavora in tempo polinomiale in modo tale che, per ogni input $w \in A^*$,

- se $w \in S$, $Pr_r(M' \text{ dichiara } w \in S) = 1$,
- se $w \notin S$, $Pr_r(M' \text{ dichiara } w \notin S) \geq \frac{1}{2}$.

Così, se M' conclude $w \notin S$, la sua risposta è completamente affidabile, mentre può capitare che M' dichiari che $w \in S$ anche quando $w \notin S$, ma la probabilità di questo errore è $\leq \frac{1}{2}$. Un confronto con la definizione di RP mostra che la nuova classe $coRP$ è introdotta in modo sostanzialmente diverso: per $S \in RP$, infatti, la conclusione $w \in S$ è sicura e quella $w \notin S$ ha probabilità di errore $\leq \frac{1}{2}$. Quindi non è manifesto che $RP = coRP$. Per esempio l'algoritmo di Miller-Rabin, che colloca l'insieme dei numeri composti in RP , dimostra che l'insieme dei primi è in $coRP$, perché, per ogni $N \geq 2$, dichiara con sicurezza che N è primo quando N è effettivamente primo, ma può rispondere che N è composto, sia pure con probabilità $\leq \frac{1}{4} < \frac{1}{2}$, quando N in realtà è primo. L'algoritmo di Miller-Rabin non serve invece a provare né che l'insieme dei composti è in $coRP$, né che l'insieme dei primi è in RP (naturalmente, sappiamo che tanto l'insieme dei primi quanto quello dei composti stanno sia in RP che in $coRP$, perché si trovano addirittura in P : ma non è tramite l'algoritmo di Miller-Rabin che possiamo arrivare a questa conclusione). Si può comunque mostrare

Teorema 11.3.6 $RP \cap coRP = ZPP$.

Dimostrazione. Già sappiamo che $ZPP \subseteq RP$. Ne deduciamo $coZPP \subseteq coRP$. Ma $coZPP = ZPP$, dunque $ZPP \subseteq RP \cap coRP$.

Dobbiamo adesso mostrare che un problema S in RP e in $coRP$ sta in ZPP . Per ipotesi ci sono due MdT probabilistiche M e M' sull'alfabeto A di S che lavorano in tempo polinomiale in modo tale che, per ogni $w \in A^*$, al variare di r testimone casuale,

- se $w \in S$,

$$Pr_\tau(M \text{ dichiara } w \in S) \geq \frac{1}{2}, \quad Pr_\tau(M' \text{ dichiara } w \in S) = 1,$$

- se $w \notin S$,

$$Pr(M \text{ dichiara } w \notin S) = 1, \quad Pr_\tau(M' \text{ dichiara } w \notin S) \geq \frac{1}{2}.$$

Costruiamo una nuova MdT probabilistica M'' su A che lavora nel modo che adesso illustriamo. Per ogni input $w \in A^*$, M'' segue le computazioni di M e M' su w e r ;

- se M dichiara “ $w \in S$ ”, M'' dichiara ugualmente “ $w \in S$ ”;
- se M' dichiara “ $w \notin S$ ”, M'' dichiara allo stesso modo “ $w \notin S$ ”;
- negli altri casi, quando M dice che “ $w \notin S$ ” e M' dice che “ $w \in S$ ”, e dunque una delle due forzatamente sbaglia, M'' non risponde, cioè dà output ? (NON SO).

È facile osservare che M'' lavora in tempo polinomiale. Certamente se M dice “ $w \in S$ ”, allora $w \in S$ e, se M' dice “ $w \notin S$ ”, ovviamente $w \notin S$. Resta da calcolare la probabilità che M'' dia esito incerto?: ora la probabilità che una delle due MdT, M o M' , sbaglia è $\leq \frac{1}{2}$. $\square$

11.4 BPP e NP

Abbiamo mostrato nel precedente paragrafo come RP sia inclusa tanto in BPP quanto in NP . Ci si può domandare quale sia la relazione diretta tra BPP e NP . Abbiamo già sottolineato che niente di definitivo è noto a questo proposito. Comunque il seguente teorema, anche se non risponde specificamente alla domanda, colloca tuttavia BPP entro il secondo livello della gerarchia polinomiale.

Teorema (Sipser-Gacs-Lautemann) 11.4.1 $BPP \subseteq \Sigma_2^P \cap \Pi_2^P$.

Dimostrazione. Ci basta provare che $BPP \subseteq \Sigma_2^P$. In tal caso, infatti, l'inclusione $coBPP \subseteq \Pi_2^P$ vale per le classi dei problemi complementari. Ma $BPP = coBPP$, e così si deduce $BPP \subseteq \Pi_2^P$. La dimostrazione che $BPP \subseteq \Sigma_2^P$ è impegnativa e può basarsi sulla seguente nuova caratterizzazione (solo necessaria) di BPP .

Lemma. Sia S un problema in BPP . Allora c'è una MdT probabilistica M sull'alfabeto A che lavora in tempo polinomiale e, per ogni input $w \in A^*$, soddisfa

$$Pr_\tau(M \text{ dà risposta scorretta su } w) \leq \frac{1}{3^{p(l(w))}}$$

(dove p è il polinomio che limita la lunghezza dei testimoni casuali r in funzione di $l(w)$).

Dimostrazione del lemma. C'è una macchina di Turing probabilistica $\hat{M}$ su A che colloca, appunto, S in BPP ; in particolare, per ogni input w , se denotiamo con $\hat{r}$ il corrispondente testimone casuale, possiamo supporre

$$Pr_{\hat{r}}(\hat{M} \text{ dà risposta scorretta su } w) \leq \frac{1}{3}.$$

Inoltre, per un opportuno polinomio $\hat{p}$, si ha $l(\hat{r}) \leq \hat{p}(l(w))$. Adesso costruiamo M . Adoperiamo ancora la strategia già adoperata in passato, e conveniamo che, sul generico input w di lunghezza n , M opera come $\hat{M}$ interpellando tuttavia k testimoni distinti $\hat{r}$, tutti ovviamente di lunghezza $\leq \hat{p}(n)$; $k = k(n)$ dipende da n , è dunque funzione di n , ma sarà precisato meglio in seguito; assumiamo comunque $k(n)$ dispari per semplicità. Notiamo che l'input casuale r consultato da M ha lunghezza $\leq k(n) \cdot \hat{p}(n)$. Al solito, assumiamo che

M conclude “ $w \in S$ ” oppure “ $w \notin S$ ”

secondo che la maggioranza dei testimoni $\hat{r}$ interpellati tramite $\hat{M}$ dichiara “ $w \in S$ ” o “ $w \notin S$ ”, rispettivamente. L'ipotesi che $k(n)$ è dispari serve ad escludere soluzioni ex aequo. Usando i limiti di Chernoff come nel teorema 11.3.2, si deduce

$$Pr_r(M \text{ dà una risposta scorretta su } w) \leq \frac{1}{2^{ck(n)}},$$

dove c è un'opportuna costante indipendente da n . Adesso scegliamo, per ogni n , $k(n)$ in modo che $2^{ck(n)} \geq k(n) \cdot \hat{p}(n)$ e quindi

$$\frac{1}{2^{ck(n)}} \leq \frac{1}{3k(n) \cdot \hat{p}(n)}$$

e otteniamo M come cercata. Si conclude così la prova del lemma.

Torniamo adesso a $BPP \subseteq \Sigma_2^P$. Sia $S \in BPP$. Per semplicità supponiamo che l'alfabeto di S sia $A = \{0, 1\}$; del resto sappiamo che opportune codifiche permettono di ridursi sempre a questo caso. Sia poi M la MdT probabilistica appena costruita per S nel lemma. Per ogni input w di lunghezza n , M ricorre a testimoni casuali di lunghezza $\leq p(n)$, per p polinomio opportuno. Per provare che $S \in \Sigma_2^P$, ci basta mostrare che la condizione di appartenenza di un input w di lunghezza n a S si può esprimere come segue:

($\star$) esistono $r_1, \dots, r_{p(n)} \in \{0, 1\}^{p(n)}$ tali che, per ogni $z \in \{0, 1\}^{p(n)}$, M dichiara “ $w \in S$ ” per qualche $i = 1, \dots, p(n)$ quando ricorre a $r_i \oplus z$.

Qualche parola di maggiore spiegazione è necessaria. Quanto ($\star$) esprime è che $w \in S$ se e solo se è possibile determinare $p(n)$ testimoni casuali $r_1, \dots, r_{p(n)}$

- equivalentemente, una sequenza $(r_1, \dots, r_{p(n)})$ di lunghezza $p^2(n)$ in $\{0, 1\}$ - tale che, per ogni ulteriore input casuale $z \in \{0, 1\}^{p(n)}$, quando si formano

$$r_1 \oplus z, \dots, r_{p(n)} \oplus z$$

- nuove sequenze in $\{0, 1\}^{p(n)}$ e quindi nuovi possibili testimoni casuali per w e M - e M fa, appunto, riferimento ad essi per decidere se w è in S o no, almeno una volta M conclude “ $w \in S$ ”. Tra parentesi, ricordiamo che $\oplus$ denota la somma modulo 2; così, per $i = 1, \dots, p(n)$, $r_i \oplus z$ è il vettore in $\{0, 1\}^{p(n)}$ che, ad ogni posto $j = 1, \dots, p(n)$, ha

- 0 se le componenti j -me di r_i e z coincidono,
- 1 altrimenti.

Non è difficile controllare poi che la condizione $(\star)$ corrisponde pienamente alla definizione di Σ_2^P e quindi, se dimostrata, implica $S \in \Sigma_2^P$, come richiesto. Passiamo dunque a verificarla. Supponiamo dapprima $w \in S$. Dobbiamo trovare $r_1, \dots, r_{p(n)}$. Ci basta provare che la probabilità p_1 , al variare di $r_1, \dots, r_{p(n)}$ in $\{0, 1\}^{p(n)}$, che per ogni $z \in \{0, 1\}^{p(n)}$ M dichiari “ $w \in S$ ” per qualche $i = 1, \dots, p(n)$ quando ricorre a $r_i \oplus z$ soddisfa

$$p_1 > 0.$$

Equivantemente, possiamo controllare che la probabilità, rispetto a $r_1, \dots, r_{p(n)}$, dell'evento complementare (e cioè che ci sia $z \in \{0, 1\}^{p(n)}$ tale che M dichiari “ $w \notin S$ ” quando ricorre al testimone casuale $r_i \oplus z$, per ogni $i = 1, \dots, p(n)$) è < 1 ,

$$1 - p_1 < 1.$$

Infatti quest'ultima probabilità $1 - p_1$ è maggiorata da

$$\sum_{z \in \{0, 1\}^{p(n)}} P_{r_1, \dots, r_{p(n)}}(M \text{ dichiara “} w \in S \text{” consultando } r_i \oplus z, \forall i = 1, \dots, p(n))$$

$$\leq 2^{p(n)} \cdot \frac{1}{(3^{p(n)})^{p(n)}} = \left(\frac{2}{3^{p(n)}} \right)^{p(n)} < 1;$$

basta ricordare che gli z coinvolti sono $2^{p(n)}$ e che, per ciascuno di essi e per ogni $i = 1, \dots, p(n)$, la conseguente probabilità che M dichiari “ $w \notin S$ ” riferendosi a $r_i \oplus z$ è $\leq \frac{1}{3^{p(n)}}$ per il lemma e quindi proprio per la scelta di M .

In modo analogo si procede quando $w \notin S$. Stavolta si fissano $r_1, \dots, r_{p(n)}$ e si va a valutare la probabilità p_2 rispetto a z che, per ogni $i = 1, \dots, p(n)$, M dichiari “ $w \notin S$ ” consultando $r_i \oplus z$. Si vuole mostrare $p_2 > 0$ per dedurre l'esistenza di un qualche z come richiesto. Si passa nuovamente all'evento complementare (cioè che M dichiari “ $w \in S$ ” per qualche $i = 1, \dots, p(n)$, interpellando $r_i \oplus z$)

e alla sua probabilità $1 - p_2$ rispetto a z . Va verificato che questa probabilità è < 1 . Infatti, usando nuovamente il lemma, si ottiene

$$\begin{aligned} 1 - p_2 &\leq \sum_{i=1, \dots, p(n)} Pr_z(M \text{ dichiara } "w \in S" \text{ ricorrendo a } r_i \oplus z) \leq \\ &\leq p(n) \cdot \frac{1}{3^{p(n)}} = \frac{1}{3} < 1. \end{aligned}$$

□

Un altro risultato che ribadisce quanto sia ancora dubbia la relazione tra BPP e NP è il seguente.

Teorema 11.4.2 $BPP \subseteq SIZE(n^{O(1)})$.

Vi si afferma che i problemi in BPP si possono trattare con circuiti di dimensione polinomiale. A questo proposito, vale la pena di ricordare il teorema di Karp-Lipton che dice che, se altrettanto accade per NP , e cioè $NP \subseteq SIZE(n^{O(1)})$, allora la gerarchia polinomiale si ferma al secondo livello: $PH = \Sigma_2^P$. Dunque, mostrare

$$NP \subseteq BPP$$

ha questa conseguenza, $PH = \Sigma_2^P$, appunto, e in questo senso pare poco plausibile. Anche l'inclusione inversa

$$BPP \subseteq NP$$

è congettura ancora aperta. Ma passiamo a provare il teorema sopra enunciato.

Dimostrazione. Sia $S \in BPP$. Ammettiamo ovviamente di lavorare sull'alfabeto $A = \{0, 1\}$. Sia $S \subseteq \{0, 1\}^*$, $S \in BPP$. Allora, come sappiamo, c'è una macchina di Turing probabilistica M che lavora in tempo polinomiale e, per ogni input w di lunghezza n , soddisfa

$$Pr_r(M \text{ dà una risposta scorretta su } w) \leq \frac{1}{2^{n+1}}.$$

M ricorre a testimoni casuali r con $l(r) \leq p(n)$ dove p è un opportuno polinomio indipendente da w e da n . Ammettiamo che sia possibile trovare $r \in \{0, 1\}^{p(n)}$ tale che, per ogni $w \in \{0, 1\}^n$, M dà risposta corretta su w se ricorre ad r . In questo caso è possibile simulare la computazione di M sul generico input w di lunghezza n e quest'unico testimone r di lunghezza $p(n)$ con un circuito C_n di $SIZE$ polinomiale in n . Va quindi mostrata l'esistenza di r . Usiamo ancora argomenti legati alla probabilità. Infatti, basta provare

$$Pr_r(M \text{ dà una risposta corretta su ogni } w \in \{0, 1\}^n) > 0.$$

Ma questo si deduce passando all'evento complementare e notando

$$\begin{aligned} Pr_r(\text{esiste } w \in \{0, 1\}^n \text{ su cui } M \text{ dà una risposta sbagliata}) &\leq \\ &\leq \sum_{w \in \{0, 1\}^n} Pr_r(M \text{ dà una risposta sbagliata su } w) \leq \\ &\leq \sum_{w \in \{0, 1\}^n} \frac{1}{2^{n+1}} = \frac{2^n}{2^{n+1}} = \frac{1}{2} < 1. \end{aligned}$$

□

11.5 Esercizi

1. Si usi l'algoritmo di Miller e Rabin per controllare se 19, 21, 23 sono o no primi.
2. La classe PP è chiusa per $\leq_p$?
3. Si dimostri che RP è chiusa per intersezioni.
4. Si provi che $PP = coPP$, $BPP = coBPP$, $coZPP = ZPP$.

Riferimenti bibliografici Un buon riferimento di Teoria dei Numeri, che include una trattazione degli algoritmi di primalità di Solovay-Strassen e di Miller-Rabin e delle loro basi matematiche, è [69], che dedica anche alcune pagine agli pseudoprimi di Carmichael. Per l'algoritmo AKS si rimanda, invece, agli articoli già segnalati nelle note bibliografiche del Capitolo 8.

I necessari fondamenti di Calcolo delle Probabilità, in particolare i limiti di Chernoff, possono trovarsi in [92], [97].

La genesi e le motivazioni delle varie classi di complessità probabilistiche sono discusse in [43]; si veda anche [61]. Del resto, tutti i riferimenti generali già segnalati negli scorsi capitoli descrivono queste classi e le confrontano con le altre già considerate, come per esempio NP .

Finalmente, la prova originaria del teorema di Sipser, Gacs e Lautemann si può trovare in [72], [101].

12.1 Soddisfacibilità unica

Conosciamo bene il problema SAT , quello che si preoccupa di determinare, per ogni formula α in CNF , se c'è o no qualche valutazione v che la soddisfa. Sappiamo che SAT è NP -completo, ma ignoriamo se appartiene a P . Infatti, in linea di principio, se α ha n variabili proposizionali $p_0, \dots, p_{n-1}$, ci sono troppe (per la precisione 2^n) valutazioni v da esplorare per vedere se qualcuna soddisfa $v(\alpha) = 1$. Il punto chiave è, dunque, identificare rapidamente una valutazione v giusta, ammesso che esista: infatti, la verifica di $v(\alpha) = 1$ è veloce da compiersi (del resto, $SAT \in NP$). Ci si può chiedere se la situazione migliora quando

- manteniamo una generica $\alpha \in CNF$ come input,
- precisiamo la domanda e ci domandiamo se α ha **esattamente** una valutazione che la soddisfa.

Così la risposta NO può corrispondere a due motivi:

- la mancanza di valutazioni v tali che $v(\alpha) = 1$,
- l'abbondanza di queste valutazioni

(nel senso che ce ne sono almeno due che funzionano); la risposta $SÌ$ si ha invece quando esiste una e una sola valutazione v che associa 1 a α .

Il problema che ne deriva si denota $USAT$ (per “*unique satisfiability*”, soddisfacibilità unica). Le precedenti considerazioni possono insinuare l'idea che $USAT$ sia più abbordabile di SAT . In realtà, per certi versi è più ingarbugliato, poiché si verifica che

- $USAT$ è NP -arduo,
- tuttavia non è chiaro se $USAT$ sta in NP , e dunque se $USAT$ è NP -completo; si sa solo che $USAT \in NP$ quando $NP = coNP$;
- di conseguenza, non è nemmeno chiaro se $USAT \in P$, e neppure se l'appartenenza di $USAT$ a P implica o no $P = NP$.

Solo la versione “casuale” della questione, quella che si ottiene sostituendo P con RP , ha, per ora, risposta positiva, vale infatti:

Teorema (Valiant-Vazirani) *Se $USAT \in RP$, allora $RP = NP$.*

Questo capitolo è dedicato a provare il teorema di Valiant-Vazirani, ma anche a discutere la questione che andiamo a introdurre, e che generalizza $USAT$. Torniamo infatti a SAT . Per ogni $\alpha \in SAT$ ci si può chiedere

quante valutazioni v soddisfano $v(\alpha) = 1$,

se una (è il caso in cui $\alpha \in USAT$), o due, o più ancora, o addirittura 2^n . Si tratta quindi di calcolare il numero delle soluzioni v per il nostro problema. In realtà la domanda si può porre per ogni $S \in NP$. Infatti dalla definizione di NP sappiamo che ci sono un problema $V_S \in P$ e un polinomio p_S (a coefficienti interi e valori positivi) tali che, per ogni parola w nell'alfabeto A di S ,

$w \in S$ se e solo se esiste $y \in A^*$ tale che $l(y) \leq p_S(l(w))$ e $(w, y) \in V_S$.

Conseguentemente, ci possiamo domandare

quanti testimoni y confermano che $w \in S$.

12.2 Contare

Introduciamo l'opportuna notazione per affrontare la questione sollevata al termine dello scorso paragrafo. Siano dati un problema $S \in NP$ e i relativi $V_S \in P$ e p_S . Chiamiamo $\#S$ (o anche, più esattamente, $\#V_S$) il seguente problema, non più di decisione, ma di calcolo:

- l'input è $w \in A^*$,
- l'output è il numero degli $y \in A^*$ con $l(y) \leq p_S(l(w))$ per cui $(w, y) \in V_S$.

In particolare $\#(S, w)$ denota questo numero.

Notazione. $\#P$ è la classe dei problemi $\#S$ così definita.

Esempi.

1. $\#SAT$ si riferisce al problema NP -completo SAT e chiede di stabilire, per ogni formula $\alpha \in CNF$, quante valutazioni v delle variabili in α soddisfano $v(\alpha) = 1$.
Allo stesso modo si definisce $\#3SAT$ a partire dal problema NP -completo $3SAT$: stavolta l'input è una formula $\alpha \in 3CNF$ e, di nuovo, si chiede per quante valutazioni v delle variabili di α si ha $v(\alpha) = 1$.
2. $\#CIRCUITSAT$ si riferisce al problema NP -completo $CIRCUITSAT$; considera un circuito C e tende a stabilire, se n è il numero dei vertici input di C , quante n -uple w in $\{0, 1\}^n$ danno output 1, cioè soddisfano $f_C(w) = 1$.

Definizione. Per $S, \bar{S}$ problemi in NP negli alfabeti $A, \bar{A}$ rispettivamente, poniamo

$$\#S \leq_{par} \#\bar{S}$$

se esiste una funzione f da A^* a $\bar{A}^*$ tale che, per ogni $w \in A^*$,

$$\#(S, w) = \#(\bar{S}, f(w)).$$

Dunque il numero dei testimoni per $w \in S$ eguaglia quello di $f(w) \in \bar{S}$. “ $\#_{par}$ ” viene chiamata riduzione “*parsimoniosa*”: è facile verificare che è una relazione di riducibilità (riflessiva e transitiva).

Definizione. Per $\bar{S} \in NP$, $\#\bar{S}$ è detto $\#P$ -completo se è completo in $\#P$ per $\leq_{\#}$, e cioè

- $\#\bar{S} \in \#P$,
- per ogni problema $S \in NP$ (dunque tale che $\#S \in \#P$), $\#S \leq_{par} \#\bar{S}$.

Tanto $\#3SAT$ quanto $\#CIRCUITSAT$ sono $\#P$ -completi. Vediamo perché.

Teorema 12.2.1 $\#3SAT$ e $\#CIRCUITSAT$ sono $\#P$ – completi per $\leq_{par}$.

Dimostrazione. Già sappiamo che $\#3SAT$ e $\#CIRCUITSAT$ sono entrambi in $\#P$. Allora è sufficiente provare

- per ogni $S \in NP$, $\#S \leq_{par} \#CIRCUITSAT$,
- $\#CIRCUITSAT \leq_{par} \#3SAT$.

(i) Lavoriamo per semplicità nell’alfabeto $\{0, 1\}$. Sia $S \subseteq A^*$, $\#S \in \#P$. In particolare fissiamo $V_S \in P$ e un polinomio p_S tali che, per ogni parola w su $\{0, 1\}$, $w \in S$ se e solo se esiste

$$\text{esiste } y \in \{0, 1\}^* \text{ tale che } l(y) \leq p_S(l(w)) \text{ e } (w, y) \in V_S.$$

Sia allora M_S una MdT deterministica che accetta V_S in tempo polinomiale in $l(w, y)$ e quindi in $l(w)$. Abbiamo già visto nel Capitolo 10, dimostrando $P \subseteq SIZE(n^{O(1)})$, che le computazioni di M_S su input (w, y) di assegnata lunghezza $n + p_S(n)$ si possono rappresentare con un circuito C_n di misura polinomiale in n . Per ogni input $w \in \{0, 1\}^*$ di lunghezza n possiamo estrarre da C_n il circuito C_w che lavora solo su al più $p_S(n)$ vertici input (quelli relativi a y) perché fissa w negli altri vertici input. In particolare C_w dà output 1 su y se e solo se C_n dà output 1 su (w, y) . Così il numero degli y per cui C_w dà output 1 eguaglia i testimoni y per $w \in S$ (quelli di lunghezza $\leq p_S(n)$ che soddisfano $(w, y) \in V_S$). Allora la funzione f che ad ogni $w \in \{0, 1\}^*$ associa C_w è la riduzione richiesta (ove w e C_w siano opportunamente codificati come naturali). Infatti non è difficile controllare che f è computabile in tempo polinomiale rispetto a $l(w)$.

(ii) Ad ogni circuito C associamo una formula $\alpha_C \in 3CNF$ tale che il numero delle valutazioni v per cui $v(\alpha_C) = 1$ coincide con il numero degli input cui C dà output 1 (e la computazione $C \mapsto \alpha_C$ avviene in tempo polinomiale rispetto alla misura di C). Supponiamo che C ammetta

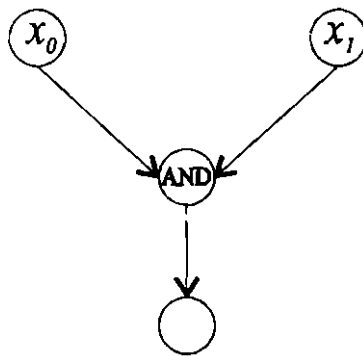
- n vertici input $x_0, \dots, x_{n-1}$,
- m vertici corrispondenti a porte logiche $g_0, \dots, g_{m-1}$ relative a OR, AND .

Allora la formula α_C ha $n + m$ variabili proposizionali

- n corrispondenti agli input di C (denotate $p_0, \dots, p_{n-1}$),
- m corrispondenti alle porte con OR, AND (indicate $q_0, \dots, q_{m-1}$),

e al più $7m$ clausole, 7 per ogni porta OR , 5 per ogni porta AND , ciascuna ovviamente con 3 lettere. La costruzione di α_C avviene come segue. Iniziamo con due semplici esempi collegati proprio a $\wedge, \vee$ rispettivamente.

Supponiamo dapprima che C sia il circuito con due vertici input e un solo vertice di porta logica g_0 corrispondente a AND .



Allora α_C , scritta come insieme finito di clausole, è

$$\{p_0 p_1 q_0, p_0 p_1 Q_0, p_0 P_1 q_0, p_0 P_1 Q_0, P_0 p_1 q_0, P_0 p_1 Q_0, P_0 P_1 q_0\}.$$

È facile vedere che l'unica valutazione v di p_0, p_1, q_0 che soddisfa α_C è, appunto,

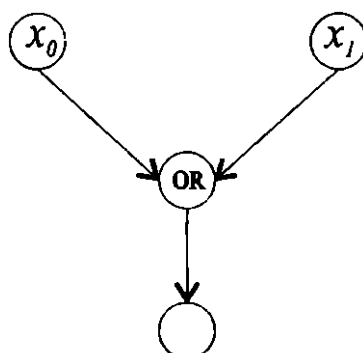
$$v(p_0) = v(p_1) = v(q_0) = 1$$

(per esempio, la possibilità $v(p_0) = v(p_1) = v(Q_0) = 1$ è esclusa dalla clausola $P_0 P_1 q_0$ e così via), e in effetti C ha un unico input

$$x_0 = x_1 = 1$$

per cui f_C assume il valore 1.

Passiamo ora a OR , e al circuito con due vertici input e un solo vertice di porta logica g_0 corrispondente a OR .



Stavolta basta una formula α_C con cinque clausole: per la precisione

$$\alpha_C = \{p_0p_1Q_0, p_0p_1q_0, p_0P_1q_0, P_0p_1q_0, P_0P_1q_0\}.$$

α_C è soddisfatta esattamente da tre valutazioni come tre, e cioè $(1, 1), (1, 0), (0, 1)$, sono le coppie in $\{0, 1\}^2$ per cui f_C ha il valore 1. Le valutazioni che soddisfano α_C sono

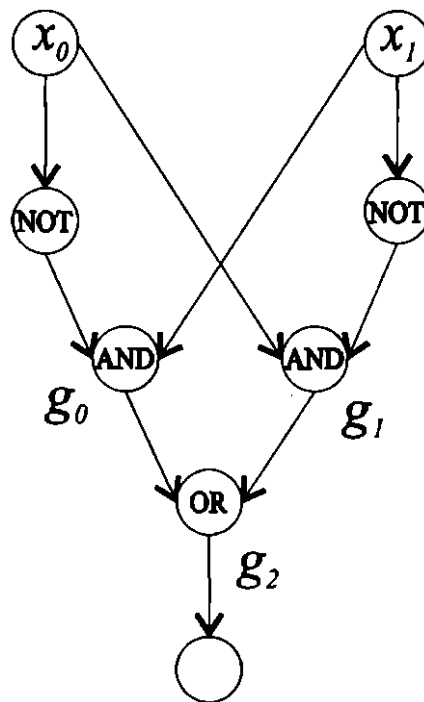
$$v(p_0) = v(p_1) = 1$$

$$v(p_0) = v(P_1) = 0$$

$$v(P_0) = v(p_1) = 0$$

e $v(q_0) = 1$; le prime due clausole servono infatti ad escludere $v(P_0) = v(P_1) = 1$, le altre a fissare $v(q_0) = 1$ ed escludere $v(Q_0) = 1$ (ovviamente i ruoli di Q_0 e q_0 possono essere qui invertiti, dobbiamo solo permettere una sola scelta tra $v(q_0) = 1$ e $v(Q_0) = 1$).

Sulla base dei due semplici esempi appena proposti, ogni circuito C può essere accompagnato con una adeguata $\alpha_C \in 3CNF$; basta trattare ogni porta AND , OR in C come sopra mostrato; la presenza di eventuali NOT richiede solo di adattare opportunamente le 7 (o 5) clausole appena indicate. Per esempio, supponiamo che C sia il circuito con due vertici input per x_0, x_1 , due porte g_0, g_1 con AND , una porta g_2 con OR (e due ulteriori porte NOT).



La formula α_C si compone allora di 19 clausole, 7 per g_0 , 7 per g_1 , 5 finalmente per g_2 , dedotte dai due esempi precedenti. In dettaglio

- le clausole per g_0 sono

$$P_0p_1q_0, P_0p_1Q_0, P_0P_1q_0, P_0P_1Q_0, p_0p_1q_0, p_0p_1Q_0, p_0P_1q_0$$

(la presenza di NOT tra x_0 e AND inverte i ruoli di P_0, p_0 rispetto all'esempio di AND);

- le clausole per g_1 sono

$$p_0P_1q_0, p_0P_1Q_0, p_0p_1q_0, p_0p_1Q_0, P_0P_1q_0, P_0P_1Q_0, P_0p_1Q_0;$$

- le clausole per g_2 , finalmente, sono

$$q_0q_1Q_2, q_0q_1q_2, q_0Q_1q_2, Q_0q_1q_2, Q_0Q_1q_2.$$

Si controlla facilmente che α_C ha tante valutazioni che la soddisfano quante sono le coppie di $\{0, 1\}$ cui f_C associa 1. La funzione che dal generico C produce α_C è computabile in tempo polinomiale rispetto alla lunghezza di C . Così $\#CIRCUITSAT \leq_{par} \#3SAT$. $\square$

12.3 Approssimare

Non sempre contare è rapido e facile, o comunque si rivela tale allo stato attuale delle conoscenze. Per esempio si può provare che il problema $\#P$ -completo di calcolo $\#3SAT$ è fuori di PH (opportunamente ridefinito per accogliere non solo problemi di decisione, ma anche problemi di calcolo). Si tratta di risultato di non facile dimostrazione che ci accontentiamo qui di enunciare; testimonia tuttavia quanto poco immediato sia computare $\#3SAT$. Comunque, quando non è possibile calcolare esattamente il numero delle soluzioni di un problema, possiamo almeno cercare di approssimarle il meglio possibile. Trattiamo l'argomento anche per illustrare l'uso di strumenti di probabilità nell'ambito della complessità. Enunciamo allora in dettaglio il risultato:

Teorema (di approssimazione per $\#P$) 12.3.1 *Per ogni $S \in NP$ e per ogni reale $\varepsilon > 0$, c'è un algoritmo che, per ogni input w dell'alfabeto A di S , computa con alta probabilità un valore N tale che*

$$(1 - \varepsilon) \cdot \#(S, w) \leq N \leq (1 + \varepsilon) \cdot \#(S, w),$$

impiegando tempo polinomiale in $l(w)$ e $\frac{1}{\varepsilon}$, e attingendo ad un oracolo in NP .

Per la precisione, l'algoritmo di approssimazione per $\#S$ è in BPP^{NP} , cioè è in BPP (e dunque opera in tempo polinomiale nella lunghezza dell'input, con una probabilità di successo sempre $\geq \frac{3}{4}$) salvo poter accedere ad un oracolo in NP ; BPP e NP sono qui ovviamente riadattati per accogliere anche problemi di calcolo. Ricordiamo che $BPP \subseteq \Sigma_2^P$ e $NP = \Sigma_1^P$. Se ne deduce che il problema di approssimare $\#S \in \#P$ è in Σ_3^P . In particolare

$$\text{approssimare } \#3SAT \text{ è in } \Sigma_3^P$$

(mentre ci sono ragioni per pensare che calcolare $\#3SAT$ è fuori di PH). Passiamo ora alla dimostrazione del teorema di approssimazione.

Dimostrazione. Anzitutto, fissiamo con precisione il nostro obiettivo.

1. Ci basta provare la tesi per $S = 3SAT$, infatti $\#3SAT$ è $\#P$ -completo (il lettore può convincersene per **esercizio**).
2. In realtà, ci è sufficiente trovare un algoritmo M per $\#3SAT$ e una costante positiva c tali che, per ogni formula $\alpha \in 3CNF$, se $M(\alpha)$ è l'output di M su α ,

$$(\star) \quad \frac{1}{c} \cdot \#(3SAT, \alpha) \leq M(\alpha) \leq c \cdot \#(3SAT, \alpha).$$

Infatti, data α , possiamo costruire $\alpha^k : \alpha \wedge \alpha \wedge \dots \wedge \alpha$, congiunzione di k formule che coincidono con α salvo che usano variabili proposizionali sempre nuove; ad esempio, se α è $p_0 \vee p_1 \vee p_2$, α^2 è $(p_0 \vee p_1 \vee p_2) \wedge (p_3 \vee p_4 \vee p_5)$. α^k è ancora in $3CNF$ ma ha un numero di variabili che è k volte quello di α ; quindi, se $\#(3SAT, \alpha) = t$, cioè α ha t valutazioni di soddisfacibilità, allora α^k ne ha t^k . Così, se vale $(\star)$, si ha

$$\frac{1}{c} \cdot t^k \leq M(\alpha^k) \leq c \cdot t^k,$$

da cui si deduce

$$\frac{1}{\sqrt[k]{c}} \cdot t^k \leq \sqrt[k]{M(\alpha^k)} \leq \sqrt[k]{c} \cdot t,$$

dunque $\sqrt[k]{M(\alpha^k)}$ approssima $t = \#(3SAT, \alpha)$ come richiesto dal teorema, purché sia $k \geq \frac{1}{\varepsilon}$, così che $\frac{1}{\sqrt[k]{c}}$ e $\sqrt[k]{c}$ approssimano in maniera ragionevole $1 - \varepsilon$, $1 + \varepsilon$ rispettivamente.

Cerchiamo allora la procedura M . Ci fondiamo sulla seguente funzione *a-comp* (la sigla *a-comp* sta per *computazione approssimata*): per $\alpha \in 3CNF$ e s intero positivo,

$$a-comp(\alpha, s) = \begin{cases} 1 & \text{se } \#(3SAT, \alpha) \geq 2s \\ 0 & \text{se } \#(3SAT, \alpha) \leq \frac{s}{2}, \end{cases}$$

mentre negli altri casi, quando $\frac{s}{2} < \#(3SAT, \alpha) < 2s$, *a-comp* assume valore arbitrario.

Per approssimare $t = \#(3SAT, \alpha)$ secondo $(\star)$, con c opportuno, possiamo allora procedere come segue:

- computiamo successivamente, finché necessario, $a-comp(\alpha, 1)$, $a-comp(\alpha, 2)$, $a-comp(\alpha, 2^2)$, $\dots$, $a-comp(\alpha, 2^i)$, $\dots$
- se $a-comp(\alpha, 1)$ è già 0 (cioè non è 1), allora

$$\#(3SAT, \alpha) \not\geq 2 \cdot 1 = 2,$$

dunque

$$\#(3SAT, \alpha) \text{ è } 0 \text{ oppure } 1,$$

e possiamo chiederne il valore all'oracolo in NP ;

- se $a\text{-comp}(\alpha, 1) = a\text{-comp}(\alpha, 2) = \dots = a\text{-comp}(\alpha, 2^i) = 1$ ma $a\text{-comp}(\alpha, 2^{i+1}) = 0$, allora si dichiara output 2^{i+1} .

La procedura lavora in tempo polinomiale rispetto alla lunghezza di α , purché $a\text{-comp}$ si possa calcolare in tempo polinomiale. Lasciamo comunque momentaneamente da parte $a\text{-comp}$ e mostriamo che l'output $M(\alpha)$ davvero soddisfa $(\star)$ per qualche c , come promesso.

Se $a\text{-comp}(\alpha, 1) = 0$, l'output è assolutamente corretto (anche se ha bisogno di ricorrere all'oracolo NP).

Assumiamo allora che valga il secondo caso: $a\text{-comp}(\alpha, 1) = a\text{-comp}(\alpha, 2) = \dots = a\text{-comp}(\alpha, 2^i) = 1$ e $a\text{-comp}(\alpha, 2^{i+1}) = 0$.

Si deduce dalla definizione di $a\text{-comp}$

- $\sharp(3SAT, \alpha) > \frac{1}{2} \cdot 2^i$, altrimenti $a\text{-comp}(\alpha, 2^i) = 0 \leq \frac{2^i}{2}$,
- $\sharp(3SAT, \alpha) < 2 \cdot 2^{i+1}$, altrimenti $a\text{-comp}(\alpha, 2^{i+1}) = 1$,

e quindi

$$\frac{1}{4} \cdot 2^{i+1} < \sharp(3SAT, \alpha) < 2 \cdot 2^{i+1};$$

ma allora

$$\frac{1}{2} \cdot \sharp(3SAT, \alpha) < 2^{i+1} < 4 \cdot \sharp(3SAT, \alpha);$$

in particolare vale $(\star)$ per $c = 4$.

Ci resta da trovare una procedura per $a\text{-comp}$ che stia in BPP^{NP} , cioè che lavori in tempo polinomiale con probabilità di successo $\geq \frac{3}{4}$, purché le sia consentito di consultare un oracolo in NP . Si tratta di obiettivo non banale, che necessita di qualche ulteriore premessa teorica e di strumenti del calcolo delle probabilità.

Rivediamo allora il nostro contesto: $a\text{-comp}$ è definito in coppie (α, s) con $\alpha \in 3CNF$, s intero positivo. Ammettiamo che α contenga n variabili $p_0, \dots, p_{n-1}$. Allora una valutazione v di α può essere individuata da una sequenza di 0,1 di lunghezza n , più esattamente da

$$(v(p_0), \dots, v(p_{n-1})).$$

D'altra parte, per ogni intero positivo m è possibile costruire una famiglia H di funzioni

$$h : \{0, 1\}^n \rightarrow \{0, 1\}^m$$

tali che, per ogni scelta di $x, x' \in \{0, 1\}^n$ con $x \neq x'$ e per ogni scelta di $y, y' \in \{0, 1\}^m$, la probabilità al variare di $h \in H$ che $h(x) = x'$ e $h(y) = y'$ soddisfa

$$Pr_{h \in H}(h(x) = x' \text{ e } h(y) = y') = \left(\frac{1}{2}\right)^{2^m}$$

(in altre parole, al variare di $h \in H$, $h(x), h(x')$ sono distribuiti indipendentemente e uniformemente tra i 2^m possibili valori in $\{0, 1\}^m$). H viene in genere

chiamata una famiglia di funzioni *hash indipendenti* (“hash” è parola inglese che sta per l’italiano “mescolanza” ed è ormai invalsa nell’uso tecnico comune).

Per costruire H , basta considerare tutte le possibili matrici A a m righe, n colonne, coefficienti 0,1 e porre, per ogni A ,

$$h_A(x) = A \cdot x$$

dove $x \in \{0,1\}^n$, $\cdot$ è il prodotto righe per colonne tra matrici e quindi x è da intendersi come vettore colonna (a n righe) secondo le regole dell’algebra lineare. C’è un risultato profondo di probabilità sulle famiglie di funzioni hash, che dice:

Teorema (Impagliazzo-Levin-Luby, 1989) 12.3.2 *Sia H una famiglia di funzioni hash indipendenti $h : \{0,1\}^n \rightarrow \{0,1\}^m$, e sia ϵ un reale positivo. Fissiamo un sottoinsieme $S \subseteq \{0,1\}^n$ e supponiamo che $|S| \geq 4 \cdot \frac{2^m}{\epsilon^2}$ (dove $|\dots|$ denota il numero degli elementi dell’insieme interessato). Allora*

$$Pr_{h \in H} \left(\left| |\{v \in S : h(v) = 0\}| - \frac{|S|}{2^m} \right| \geq \epsilon \frac{|S|}{2^m} \right) \leq \frac{1}{4}$$

(dunque c’è probabilità al più $\frac{1}{4}$, al variare di $h \in H$, che il valore assoluto della differenza tra

- il numero degli elementi $v \in S$ su cui h si annulla,
- $\frac{|S|}{2^m}$,

superi $\epsilon \cdot \frac{|S|}{2^m}$).

Ritorniamo adesso alla ricerca della procedura per *a-comp*. Siano dati

- $\alpha \in 3CNF$ (con n variabili),
- s intero positivo;

L’obiettivo, come sappiamo, è calcolare

$$a\text{-comp}(\alpha, s) = \begin{cases} 1 & \text{se } \sharp(3SAT, \alpha) \geq 2s, \\ 0 & \text{se } \sharp(3SAT, \alpha) \leq \frac{s}{2}, \\ \text{a caso} & \text{altrimenti.} \end{cases}$$

Operiamo come segue.

- 1) Se $s < 2^5 = 32$, allora controlliamo direttamente se $\sharp(3SAT, \alpha) \geq 2s$, o $\leq \frac{s}{2}$, o corrisponde al terzo caso.

2) Se invece $s \geq 2^5$, prendiamo una famiglia H di funzioni hash indipendenti $h : \{0, 1\}^n \rightarrow \{0, 1\}^m$ dove m soddisfa

$$2^m \leq \frac{s}{2^4} < 2^{m+1}$$

(quindi $m = 1$ per $2^5 \leq s \leq 2^6$, $m = 2$ per $2^6 \leq s < 2^7$, e così via). Scegliamo $h \in H$ e diamo output 1 se e solo se ci sono $> \frac{s}{2^m}$ valutazioni v (intese come elementi di $\{0, 1\}^n$ nel senso sopra descritto) tali che $v(\alpha) = 1$ e $h(v) = 0$.

Notiamo che, per $\frac{s}{2^m} < 2^5$, cioè $\frac{s}{2^4} < 2^{m+1}$, il test finale in 2) si può fare consultando un oracolo in NP . Lo stesso vale per la verifica richiesta in 1).

Mostriamo ora che l'algoritmo descritto è in BPP^{NP} , nel senso che, consultando l'oracolo in NP , produce in tempo polinomiale nella lunghezza dell'input una risposta che ha probabilità almeno $\frac{3}{4}$ di essere corretta. Ovviamente basta controllare 2). Sia allora $S \subseteq \{0, 1\}^n$ l'insieme delle valutazioni v per cui $v(\alpha) = 1$.

Caso 1. $|S| \geq 2s$ (così la risposta corretta per $a\text{-comp}(\alpha, s)$ è 1).

Osserviamo

$$|S| \geq 2s > s \geq 16 \cdot 2^m = \frac{4 \cdot 2^m}{\epsilon^2}$$

per $\epsilon = \frac{1}{2}$. Applichiamo allora il Teorema di Impagliazzo-Levin-Luby che, per $\epsilon = \frac{1}{2}$, implica

$$Pr_{h \in H} \left(\left| |\{v \in S : h(v) = 0\}| - \frac{|S|}{2^m} \right| \geq \frac{1}{2} \cdot \frac{|S|}{2^m} \right) \leq \frac{1}{4};$$

quindi

$$Pr_{h \in H} \left(|\{v \in S : h(v) = 0\}| \leq \frac{1}{2} \cdot \frac{|S|}{2^m} \right) \leq \frac{1}{4}$$

e in definitiva riserva al caso di fallimento (cioè all'eventualità che ci siano al più $\frac{s}{2^m}$ valutazioni v per cui $v(\alpha) = 1$ e $h(v) = 0$)

$$Pr_{h \in H} \left(|\{v \in S : h(v) = 0\}| \leq \frac{s}{2^m} \right) \leq \frac{1}{4}.$$

Così l'algoritmo ha successo con probabilità $\geq \frac{3}{4}$.

Caso 2. $|S| \leq \frac{s}{2}$ (così la risposta corretta per $a\text{-comp}(\alpha, s)$ è 0).

Sia $S' \supseteq S$, $|S'| = \frac{s}{2}$. Allora l'eventualità di insuccesso è quella che vi siano più di $\frac{s}{2^m}$ valutazioni v che stanno in S e soddisfano $h(v) = 0$. Si ha

$$Pr_{h \in H} \left(|\{v \in S : h(v) = 0\}| > \frac{s}{2^m} \right) \leq$$

$$\begin{aligned} &\leq \Pr \left(|\{v \in S' : h(v) = 0\}| > \frac{s}{2^m} \right) \leq \\ &\leq \Pr \left(\left| |\{v \in S' : h(v) = 0\}| - \frac{|S'|}{2^m} \right| > \frac{|S'|}{2^m} \right) \leq \frac{1}{4}, \end{aligned}$$

per il teorema di Impagliazzo-Levin-Luby applicato a $\varepsilon = 1$; infatti, in questo caso, $|S'| = \frac{s}{2}$ assicura $|S'| \geq 4 \cdot 2^m$ per $s \geq 8 \cdot 2^m$ come assicurato dalla scelta di m . Quindi anche stavolta la probabilità di successo dell'algoritmo è $\geq \frac{3}{4}$, come richiesto. $\square$

12.4 Il teorema di Valiant-Vazirani

Quest'ultimo paragrafo del capitolo è dedicato a provare il teorema già annunciato e discusso nel paragrafo 12.1 su *USAT*.

Teorema (Valiant-Vazirani) 12.4.1 *Se $USAT \in RP$, allora $RP = NP$.*

Dimostrazione. Siccome già sappiamo che $RP \subseteq NP$, ci basta provare l'inclusione inversa $NP \subseteq RP$. Possiamo anzi fare riferimento a *3SAT* come prototipo di problema *NP*-completo e dunque limitarci a mostrare

$$3SAT \in RP,$$

cercare dunque un algoritmo probabilistico di tipo *RP* per *3SAT*, capace cioè di lavorare in tempo polinomiale nella lunghezza della formula $\alpha \in 3CNF$ che viene proposta per input, e inoltre

- se $\alpha \in 3SAT$, riconoscerlo con probabilità di successo $> \frac{1}{2}$,
- se $\alpha \notin 3SAT$, riconoscerlo senza errore.

Sappiamo che un algoritmo analogo per *USAT* è disponibile. In realtà, per la nostra dimostrazione ci basta un algoritmo probabilistico che lavora in tempo polinomiale nella lunghezza della formula $\alpha' \in CNF$ che gli viene proposta e

- se $\alpha' \in USAT$, cioè esiste una e una sola valutazione v che la soddisfa, allora risponde sì con probabilità di successo $> \frac{1}{2}$,
- se $\alpha' \notin USAT$ perché non ha valutazioni che la soddisfino, dichiara senza possibilità di errore che, appunto, α' non è soddisfacibile

(in altre parole, è indifferente per i nostri propositi quel che l'algoritmo risponde quando $\alpha' \notin USAT$ perché ci sono almeno 2 valutazioni che soddisfano α'). Ma ritorniamo a *3SAT*. Sia $\alpha \in 3CNF$ con variabili $p_0, \dots, p_{n-1}$. Notiamo che le possibili valutazioni v per queste variabili, e dunque per α , sono 2^n (per ogni $i < n$, $v(p_i)$ può assumere uno qualunque tra i valori 0 e 1). Ricordiamo

anche che ognuna di queste valutazioni può essere codificata da una sequenza di lunghezza n di $0,1$, più precisamente da

$$(v(p_0), \dots, v(p_{n-1}));$$

anzi, si ottiene in questo modo una corrispondenza biunivoca tra l'insieme delle possibili valutazioni e $\{0,1\}^n$.

Adesso scegliamo casualmente:

- un naturale $k < n$,
- una matrice A con $k + 2$ righe, n colonne e coefficienti in $\{0,1\}$.

Ricordiamo che A definisce una funzione

$$h_A : \{0,1\}^n \rightarrow \{0,1\}^{k+2},$$

per cui

$$h_A(x) = A \cdot x \text{ per ogni } x \in \{0,1\}^n.$$

A questo proposito, sarà anche bene ripetere che, al variare di A , le diverse h_A formano una famiglia di funzioni hash indipendenti da $\{0,1\}^n$ a $\{0,1\}^{k+2}$.

La nostra strategia è di costruire, a partire da α , in riferimento a k e A , una formula $\alpha' \in 3CNF$, eventualmente in più di n variabili, tale che

il numero delle valutazioni che soddisfano α'

coincide con

il numero delle valutazioni v per cui $v(\alpha) = 1$ e inoltre $h_A(v) = 0$.

Rimandiamo momentaneamente la costruzione di α' che tratteremo sotto in dettaglio; anticipiamo solo che il procedimento che produce α' da α è deterministico e impiega tempo polinomiale nella lunghezza di α (così anche la lunghezza di α' è polinomiale rispetto a quella di α). Procediamo invece con α . L'algoritmo probabilistico che proponiamo è il seguente: ottenuta α , applichiamo il procedimento RP per $USAT$ ad α' e

- se esso risponde *no*, estendiamo la risposta ad α (e al quesito se $\alpha \in 3SAT$),
- se esso risponde *sì*, altrettanto diciamo per $\alpha \in 3SAT$.

I tempi richiesti da tutta la procedura per α sono, come voluto, polinomiali nella lunghezza di α . Dobbiamo però discutere quanto sia ragionevole affidarsi ad α' ; in altre parole, dobbiamo provare che l'algoritmo proposto per $3SAT$ e α è ancora in RP . Fissiamo $t = \#(3SAT, \alpha)$.

- Sia dapprima $t = 0$, cioè $\alpha \notin 3SAT$. Allora non ci sono valutazioni v che soddisfano α , meno che mai valutazioni v per cui $v(\alpha) = 1$ e in più $h_A(v) = 0$. Così, per come α' è costruita, non ci sono affatto valutazioni che soddisfano α' . Il procedimento RP per $USAT$ dichiara questa situazione senza possibilità di errore. La risposta negativa si trasmette al quesito $\alpha \in 3SAT$, ed è assolutamente corretta.

- Sia ora $t > 0$, cioè $\alpha \in 3SAT$. Ricordiamo che $t \leq 2^n$, e notiamo che c'è probabilità $\geq \frac{1}{n}$ di scegliere $k < n$ (per cui $2^k \leq t \leq 2^{k+1}$). Fissato k , resta comunque da scegliere A ; ci chiediamo quale è la probabilità Pr che, tra le t valutazioni v che soddisfano α , esattamente una rispetti anche la condizione $h_A(v) = 0$. In tal caso, infatti, c'è una e una sola valutazione che soddisfa α' , e l'algoritmo per $USAT$ dichiara $\alpha' \in USAT$ con probabilità $\geq \frac{1}{2}$.

Assumiamo di sapere $Pr > \frac{1}{8}$. Se così accade, la probabilità che l'algoritmo per $3SAT$ dichiari $\alpha \in 3SAT$ (per una scelta opportuna di k, A) supera

$$\frac{1}{2} \cdot \frac{1}{n} \cdot \frac{1}{8} = \frac{1}{16n}$$

($\frac{1}{2}$ per l'algoritmo di $USAT$, $\frac{1}{n}$ per k , $\frac{1}{8}$ per A). Ovviamente $\frac{1}{16n}$ non raggiunge il livello $\frac{1}{2}$ richiesto per algoritmo RP . Ma un numero opportuno di iterazioni della procedura ($8n$ in tutto) ottengono anche questo risultato.

Così si è provato $3SAT \in RP$, come richiesto, purché si chiariscano i due punti rimasti, e cioè.

- (1) la costruzione di α' da α ,
- (2) $Pr > \frac{1}{8}$.

Ecco le relative trattazioni.

(1) Qui cerchiamo un programma deterministico che lavora in tempo al più polinomiale per produrre, in riferimento alla matrice A , per ogni formula $\alpha \in 3CNF$ nelle variabili $p_0, \dots, p_{n-1}$, una formula $\alpha' \in 3CNF$ tale che il numero delle valutazioni che soddisfano α' eguaglia quello delle valutazioni v tali che $v(\alpha) = 1$ e

$$A \cdot \begin{pmatrix} v(p_0) \\ \vdots \\ v(p_{n-1}) \end{pmatrix} = 0.$$

La costruzione di α' è tecnica, ma non banale. La situazione ci conduce ovviamente al sistema lineare $A \cdot x = 0$ (dove x è un vettore colonna in $\{0, 1\}^n$ e $= 0$ è da intendersi modulo 2, visto che i soli valori ammessi sono 0,1). Ricordiamo che A è una matrice di $k+2$ righe, da 0 a $k+1$, n colonne, da 0 a $n-1$, e ha coefficienti in $\{0, 1\}$. A proposito, per $i < k+2$ e $j < n$, indichiamo con a_{ij} l'elemento di posto (i, j) in A . Così il sistema $A \cdot x = 0$ si compone in dettaglio delle equazioni

$$(\star) \quad a_{i0}x_0 + a_{i1}x_1 + \dots + a_{in-1}x_{n-1} \equiv 0 \pmod{2}$$

dove $i < k+2$, come detto, e $x_0, x_1, \dots, x_{n-1}$ sono, ovviamente, le componenti di x . Ricorrendo a nuove variabili y_{ij} ($i < k+2, j < n$) in $\{0, 1\}$, possiamo sostituire a ciascuna di queste equazioni $(\star)$ un sistema equivalente

$$(\star\star) \quad \begin{cases} y_{i0} \equiv a_{i0}x_0 \pmod{2} \\ y_{ij+1} \equiv a_{i,j} + a_{i,j+1}x_{j+1} \pmod{2} \text{ per } 1 \leq j < n-1 \\ y_{in-1} \equiv 0 \pmod{2} \end{cases}$$

ancora per $i < k + 2$. È evidente, infatti, che $x \in \{0, 1\}^n$ soddisfa $A \cdot x = 0$ se e solo se esistono $y_{ij} \in \{0, 1\}$ (per $i < k + 2, j < n$) che, insieme a x , risolvono tutte le equazioni di $(\star\star)$ per $i < k + 2$. In realtà, per ogni $i < k + 2$, le equazioni di $(\star\star)$, esclusa l'ultima, servono a determinare univocamente $y_{i0}, y_{i1}, \dots, y_{in-1}$, uno dietro l'altro, in funzione di x e dei precedenti; invece, l'ultima equazione è quella che traduce effettivamente $(\star)$.

In particolare, per v valutazione di $p_0, \dots, p_{n-1}$ (tale che $v(\alpha) = 1$), e per $i < k + 2$, si ha che

$$v(p_0), \dots, v(p_{n-1}) \text{ soddisfano } (\star)$$

se e solo se ci sono $y_{i0}, y_{i1}, \dots, y_{in-1} \in \{0, 1\}$, univocamente definiti, che insieme a $v(p_0), \dots, v(p_{n-1})$ risolvono $(\star\star)$. Possiamo in tal caso associare a $y_{i0}, y_{i1}, \dots, y_{in-1}$ nuove variabili proposizionali $q_{i0}, q_{i1}, \dots, q_{in-1}$, ed estendere v ad una valutazione v' di queste nuove variabili tale che, appunto,

$$v'(p_j) = v(p_j), \text{ per ogni } j < n$$

e inoltre

$$v'(q_{ij}) = y_{ij} \text{ per ogni } j < n.$$

Adesso procediamo finalmente alla costruzione di α' . Essenzialmente, α' congiunge α con clausole in $p_0, \dots, p_{n-1}$ e nelle nuove variabili $q_{i0}, q_{i1}, \dots, q_{in-1}$ ($i < k + 2$) tali che, per ogni valutazione v di $p_0, \dots, p_{n-1}$,

$$v(\alpha) = 1 \text{ e } v(p_0), \dots, v(p_{n-1}) \text{ risolvono } A \cdot x = 0 \text{ (e cioè ogni sistema } (\star\star) \text{ con i corrispondenti } y_{i0}, y_{i1}, \dots, y_{in-1})$$

se e solo se

$$v' \text{ (come sopra definito) soddisfa } \alpha'.$$

Mostriamo la costruzione di α' in un caso particolare. Ammettiamo che α ammetta 2 sole variabili p_0, p_1 , cioè che $n = 2$, e che $A = (1, 1)$, cioè che il sistema $A \cdot x = 0$ si riduca all'unica equazione

$$(\star) \quad x_0 + x_1 \equiv 0 \pmod{2}.$$

Il corrispondente sistema $(\star\star)$ è

$$(\star\star) \quad \begin{cases} y_0 \equiv x_0 \pmod{2} \\ y_1 \equiv y_0 + x_1 \pmod{2} \\ y_1 \equiv 0 \pmod{2} \end{cases}$$

in particolare coinvolge 2 nuove variabili y_0, y_1 in $\{0, 1\}$. Bastano allora due nuove variabili proposizionali q_0, q_1 per α' . In dettaglio α' contiene in aggiunta ad α le seguenti 7 clausole (2 per la prima equazione, 1 per l'ultima, 4 per l'intermedia):

- (i) $p_0 Q_0, P_0 q_0,$
- (ii) $P_0 Q_0 Q_1, P_0 q_0 q_1, p_0 Q_0 q_1, p_0 q_0 Q_1,$
- (iii) $Q_1.$

(i) traduce la prima equazione in ($\star\star$), infatti, per ogni v , $\{p_0Q_0, P_0q_0\}$ è soddisfatta da v' se e solo se $v'(p_0) = v'(q_0)$ (cioè $v'(p_0) = 1$ e $v'(q_0) = 1$ oppure $v'(P_0) = 1$ e $v'(Q_0) = 1$).

(ii) traduce la seconda equazione in ($\star\star$): per ogni v, v' soddisfa

$$\{P_0Q_0Q_1, P_0q_0q_1, p_0Q_0q_1, p_0q_0Q_1\}$$

se e solo se $v'(q_1) = v'(q_0) + v'(p_0)$.

(iii) finalmente corrisponde all'ultima equazione in ($\star\star$): fissata v, v' soddisfa $\{Q_1\}$ se e solo se $v'(q_1) = 0$.

Si può rilevare che non tutte le clausole di α' hanno esattamente 3 lettere, come in genere si richiede a una formula in $3CNF$, talora si riducono a 2 lettere, e perfino a una sola lettera. Ma abbiamo visto, provando la NP -completezza di $3SAT$ (nel teorema 8.7.1) come ovviare a questa (piccola) difficoltà, e come scrivere tutta α' rigorosamente in $3CNF$.

La costruzione di α' nel caso generale estende l'idea dell'esempio e garantisce le seguenti situazioni:

- per ogni valutazione v di $p_0, \dots, p_{n-1}$ tale che $v(\alpha) = 0$ e $v(p_0), \dots, v(p_{n-1})$ risolvono $A \cdot x = 0$, c'è un'unica espansione v' di v a $q_{i0}, q_{i1}, \dots, q_{in-1}$ ($i < k + 2$) tale che $v'(\alpha) = 1$;
- viceversa, per ogni valutazione v' di $p_0, \dots, p_{n-1}, q_{i0}, q_{i1}, \dots, q_{in-1}$ ($i < k + 2$) tale che $v'(\alpha') = 1$, la restrizione v di v' a $p_0, \dots, p_{n-1}$ soddisfa $v(\alpha) = 1$ (e inoltre $A \cdot x = 0$ tramite $v(p_0), \dots, v(p_{n-1})$).

In particolare il numero delle valutazioni v' tali che $v'(\alpha') = 1$ coincide con il numero delle valutazioni v tali che $v(\alpha) = 1$ e $v(p_0), \dots, v(p_{n-1})$ soddisfano $A \cdot x = 0$.

Si verifica che il numero delle clausole di α' (non già in α) è polinomiale - anzi lineare - in n . Così la costruzione di α' , oltre che essere deterministica, avviene anche in tempo al più polinomiale rispetto a $l(\alpha)$, proprio come richiesto.

(2) Adesso è da provare il seguente risultato.

Lemma Siano $\alpha \in 3CNF$ con variabili $p_1, \dots, p_n$, $t = \sharp(3SAT, \alpha)$. Supponiamo $t > 0$ e fissiamo $k < n$ tale che $2^k \leq t < 2^{k+1}$. Sia poi H una famiglia di funzioni hash indipendenti da $\{0, 1\}^n$ a $\{0, 1\}^{k+2}$. Allora

$$Pr_{h \in H} (c'è esattamente una valutazione v per cui $v(\alpha) = 1, h(v) = 0) \geq \frac{1}{8}.$$$

Dimostrazione del lemma. La dimostrazione richiede argomenti tutto sommato elementari di calcolo delle probabilità. Eccone i particolari. Consideriamo dapprima una singola valutazione v tale che $v(\alpha) = 1$. Vogliamo calcolare

$$Pr_{h \in H} (v \text{ è l'unica valutazione per } \alpha \text{ tale che } v(\alpha) = 1 \text{ e } h(v) = 0)$$

e cioè la probabilità rispetto a $h \in H$ che $h(v) = 0$, ma $h(v') \neq 0$ per ogni altra valutazione v' di α per cui $v'(\alpha) = 1$. Chiaramente la probabilità cercata è il prodotto di

- (i) $Pr_{h \in H}(h(v) = 0)$,
- (ii) la probabilità rispetto a $h \in H$ che $h(v') \neq 0$ per ogni $v' \neq v$ tale che $v'(\alpha) = 1$, condizionata però a $h(v) = 0$.

È chiaro che $Pr_{h \in H}(h(v) = 0) = \frac{1}{2^{k+2}}$, visto che $h(v)$ è elemento arbitrario in $\{0, 1\}^{k+2}$ (e H è famiglia di funzioni hash indipendenti). Passiamo allora a (ii): nuovamente per il motivo che H è famiglia di funzioni hash indipendenti, si ha che la condizione $h(v) = 0$ non interferisce sul valore di $h(v')$ per $v \neq v'$. Così ci basta calcolare

$$Pr_{h \in H}(\text{per ogni valutazione } v' \neq v \text{ per cui } v'(\alpha) = 1, h(v') \neq 0).$$

Possiamo anche riferirci all'evento complementare, e quindi calcolare

$$Pr_{h \in H}(\text{esiste una valutazione } v' \neq v \text{ per cui } v'(\alpha) = 1 \text{ e } h(v') = 0)$$

che eguaglia

$$\sum_{v \neq v', v(\alpha)=1} Pr_{h \in H}(h(v') = 0).$$

Già sappiamo che $Pr_{h \in H}(h(v') = 0) = \frac{1}{2^{k+2}}$ per ogni v' . D'altra parte il numero delle valutazioni v' coinvolte è $t - 1 < t \leq 2^{k+1}$, così la probabilità cercata è

$$(t - 1) \cdot \frac{1}{2^{k+2}} < 2^{k+1} \cdot \frac{1}{2^{k+2}} = \frac{1}{2}.$$

Ne segue

$$Pr_{h \in H}(\text{per ogni valutazione } v' \neq v \text{ per cui } v'(\alpha) = 1, h(v') = 0) > \frac{1}{2},$$

e quindi

$$\begin{aligned} Pr_{h \in H}(v \text{ è l'unica valutazione per } \alpha \text{ tale che } v(\alpha) = 1 \text{ e } h(v) = 0) &> \\ &> \frac{1}{2^{k+2}} \cdot \frac{1}{2} = \frac{1}{2^{k+3}}. \end{aligned}$$

Adesso passiamo a

$$\begin{aligned} Pr_{h \in H}(\text{esiste esattamente una valutazione } v' \neq v \\ \text{per cui } v'(\alpha) = 1 \text{ e } h(v') = 0). \end{aligned}$$

Essa coincide con

$$\sum_{v(\alpha)=1} Pr_{h \in H}(v \text{ è l'unica valutazione di } \alpha \text{ tale che } v(\alpha) = 1 \text{ e } h(v) = 0),$$

$$\text{quindi è } > t \cdot \frac{1}{2^{k+3}} \geq 2^k \cdot \frac{1}{2^{k+3}} = \frac{1}{8}.$$

□

12.5 Esercizi

1. Si dimostri che $\leq_{par}$ è una relazione di riducibilità.
2. Si enunci il problema $\#KNAPSACK$. Se $(V, n, v_0, \dots, v_{n-1})$ è un'istanza di $KNAPSACK$ e la sequenza $(v_0, \dots, v_{n-1})$ è supercrescente, quali valori può assumere $\#(KNAPSACK, (V, n, v_0, \dots, v_{n-1}))$?
3. Si enunci il problema $\#IS$. Se $G = (V, E)$ è un grafo e k un intero positivo, quali valori può assumere $\#(IS, (G, k))$? E se V è una cricca?

Riferimenti bibliografici $\#P$ è introdotto in [109]. Il teorema di Valiant-Vazirani si trova in [111]. Il teorema di Impagliazzo, Levin, Luby usato nella dimostrazione del teorema di approssimazione del paragrafo 12.3 è in [59]; a questo proposito, qualche evidenza che $\#3SAT$ è fuori di PH deriva dal teorema di Toda [106] secondo cui $PH \subseteq P^{\#P}$ (si veda anche [50]).

Un quadro delle applicazioni della complessità alla crittografia può trovarsi in [90] oppure in [94].

13.1 Artù e Merlino

Ricordiamo ancora una volta la definizione di NP : un problema S è in questa classe se e solo se ci sono

- un problema $V_S \in P$,
- un polinomio p_S

tali che, per ogni input w nell'alfabeto A di S , $w \in S$ se e solo se esiste $y \in A^*$ tale che $l(y) \leq p_S(l(w))$ e $(w, y) \in V_S$.

Così il controllo dell'appartenenza di w a S si può pensare come un gioco di due persone

- l'uno A deve convincersi al di là di ogni ragionevole dubbio che $w \in S$, o $w \notin S$,
- l'altro M deve fornirgliene la prova evidente.

In effetti, per $S \in NP$ e $w \in S$, M ha una semplice strategia per raggiungere il suo obiettivo, e cioè produrre un testimone y , dopo di che sta ad A verificare in modo deterministico e rapido che $(w, y) \in V_S$. Se invece $w \notin S$, M non ha modo di convincere A del contrario, né gli serve ricorrere ad alcun testimone y .

Esempio. Sia $S = 3COL$ il problema (NP -completo) della 3-colorabilità dei grafi: è dato un grafo finito $G = (V, E)$ e ci si chiede se è possibile colorarlo con 3 colori, in modo che vertici di V collegati da E ricevano colori diversi. Una ovvia strategia che M ha per convincere A è proprio quella di mostrargli una 3-colorazione di G (ammesso che esista); a quel punto, A deve soltanto controllare che vertici adiacenti abbiano davvero colori differenti.

Le iniziali A e M usate per indicare i due interlocutori del gioco non sono prese a caso, si riferiscono infatti ai romanzi del ciclo della Tavola Rotonda: in particolare

- A è il giovane Artù, ancora inesperto e bisognoso di formazione,
- M è il mago Merlino, che tutto sa e (quasi) tutto può, e si prende cura dell'educazione di Artù.

Immaginiamo che sia dato un problema S e organizziamo la seguente interazione tra A e M : per ogni parola w nell'alfabeto A di S ,

- M deve convincere A al di là di ogni ragionevole dubbio del fatto che $w \in S$ o no,
- A , da parte sua, dispone soltanto di ridotte capacità di calcolo (potremmo dire, di risorse polinomialmente limitate dalla lunghezza di w).

Come già detto nel caso di $S = 3COL$ e, più in generale, per $S \in NP$, basta un'unica comunicazione da M a A - quella riguardante y -, per di più polinomialmente limitata da $l(w)$, per chiudere il discorso se $w \in S$ (per $w \notin S$, nessun testimone y può aiutare M a persuadere A del contrario): conoscendo y , A applica un procedimento deterministico polinomiale per V_S , e conclude $w \in S$.

Possiamo però concepire modelli più raffinati che ammettono una più intensa interazione tra A e M , consentono in particolare ad A di rivolgere domande a M , e obbligano M a rispondere. Quindi, A e M si scambiano alternativamente messaggi in cui

- A chiede a M ulteriori spiegazioni tramite domande di lunghezza polinomialmente limitata da quella della corrispondenza precedente e dunque, in definitiva, da $l(w)$;
- M risponde ad A senza inganni fornendogli dimostrazioni anch'esse di lunghezza polinomialmente limitata da $l(w)$;

anche il numero totale delle comunicazioni intercorse tra A e M deve essere polinomialmente limitato da $l(w)$. Alla fine della corrispondenza, A utilizza l'informazione appresa y proponendola come input (insieme a w) ad una procedura deterministica V_S che lavora in tempo polinomiale per stabilire tramite y se $w \in S$ oppure no. Indichiamo momentaneamente con $\overline{NP}$ la classe dei problemi S che ammettono una tale procedura. In essi, l'interazione consente a M di convincere A "al di là di ogni ragionevole dubbio" che w appartiene o non appartiene a S , nel senso che segue:

- (i) se $w \in S$, M lo prova con assoluta certezza;
- (ii) se $w \notin S$, M non ha modo di persuadere A del contrario.

In genere il requisito enunciato in (i) si chiama *completezza*, quello di (ii) *validità*. Così, in questo ambito, l'espressione

"convincere al di là di ogni ragionevole dubbio"

significa

"dimostrare senza possibilità di errore".

D'altra parte, è facile notare che la classe $\overline{NP}$ così ottenuta va a coincidere con NP .

Osservazione. $\overline{NP} = NP$.

Infatti, già sappiamo che $\overline{NP} \supseteq NP$, visto che i problemi di NP corrispondono al caso di una singola interazione da M ad A . Viceversa, supponiamo che un problema S sia in $\overline{NP}$, ammetta quindi un algoritmo interattivo come sopra descritto e che la corrispondenza tra M e A preveda, per ogni dato w , più possibili messaggi. M può comunque concentrarne tutto il contenuto in un singolo messaggio da inviare in un'unica soluzione ad A ; in particolare, M può mantenere il vincolo sulla lunghezza di questo messaggio, perché tutte le interazioni precedenti e il loro stesso numero sono polinomialmente limitati in funzione di $l(w)$. Segue che $S \in NP$; infatti, per ogni w , y è proprio il contenuto del messaggio riassuntivo da M ad A .

Così la semplice interazione tra i due interlocutori A e M non aggiunge nulla di nuovo a NP , almeno finché conveniamo che A si limiti a chiedere ulteriori spiegazioni e M risponda in modo preciso. Nozioni più interessanti si ottengono se, oltre alla interazione, consentiamo una qualche dose di *casualità*, intendiamo cioè l'espressione

“convincere al di là di ogni ragionevole dubbio”

in modo meno rigoroso e vincolante, non più come “dimostrare senza possibilità di errore o di inganno”, ma come

“persuadere con alta probabilità”.

Si costruiscono su questa base varie classi di complessità, le cui differenze derivano dalle porzioni di *interazione* e *causalità* che esse ammettono. Consideriamo per esempio la situazione dei seguenti problemi S . Per un dato input w , si ammette ancora che A e M si corrispondano con messaggi di lunghezza polinomialmente limitata da quella di w e in numero polinomialmente limitato da $l(w)$. Stavolta però si ammette che M possa ingannare A . Conseguentemente A non si fida e, sia pure nella sua ignoranza, cerca di acquisire maggiori certezze in un modo che M non possa prevedere, né fronteggiare, a meno che non conosca davvero la verità su w . Da un punto di vista formale, si può allora pensare che A , in ogni suo intervento, lanci una monetina un certo numero di volte (comunque polinomialmente limitato da $l(w)$) e ne comunichi l'esito a M . M deve rispondere a tono (gli esempi successivi chiariranno meglio questa situazione). Si noti che l'intervento di A è del tutto casuale e non dipende dalla conversazione precedente. Al termine della corrispondenza, A procede come prima, considera cioè il contenuto complessivo y dei messaggi intercorsi (dipendente dalla casualità delle sue domande) e lo affida insieme all'input iniziale w ad un programma deterministico (indipendente da w e dalla corrispondenza stessa); il programma lavora in tempo al più polinomiale in $l(w)$ per stabilire se $w \in S$ o no.

Si dirà che $S \in AM[Poly]$ se si possono determinare, appunto,

- un problema $V_S \in P$,
- opportuni polinomi (atti a limitare lunghezza e numero dei messaggi, e quindi anche lunghezza del contenuto complessivo y , in funzione della lunghezza dell'input)

tali che, per ogni w sull'alfabeto di S ,

- (i) se $w \in S$, $Pr_y((w, y) \in V_S) \geq \frac{2}{3}$,
- (ii) se $w \notin S$, $Pr_y((w, y) \in V_S) \leq \frac{1}{3}$.

(i) e (ii) dicono come le richieste di *completezza*, *validità* (rispettivamente) vengono adattate al nuovo contesto. Dunque

- se $w \in S$, M ha la probabilità $\geq \frac{2}{3}$ di convincere A , rispondendo opportunamente alle sue domande,
- se $w \notin S$, M ha la probabilità $\leq \frac{1}{3}$ di ingannare A con risposte adeguate e persuaderlo del contrario.

L'interazione tra A e M che produce i possibili y si dice un *protocollo Artù-Merlino*.

Esempio. Torniamo al problema $3COL$. Già conosciamo un ovvio algoritmo interattivo che deriva dal fatto che $3COL \in NP$ e consente quindi a M di convincere A senza possibilità di inganno della 3-colorabilità di un dato grafo $G = (V, E)$. D'altra parte, il procedimento obbliga M a rivelare l'eventuale 3-colorazione di G . Supponiamo allora che M voglia ancora persuadere A , ma senza manifestargli la colorazione. Un algoritmo che gli assicura questo obiettivo e si trova in $AM[Poly]$ è quello che adesso illustriamo in modo vagamente informale. Ci si basa sul fatto che una 3-colorazione di G , se esiste, non è unica, ma è definita a meno di una permutazione dei colori. Siccome le permutazioni su 3 oggetti sono $3! = 6$, ogni 3-colorazione di G è accompagnata da altre 5 possibili soluzioni, semplicemente ottenute permutando i colori. Si procede di conseguenza come ora descritto.

- A lancia una moneta o dei dadi, determina comunque in modo casuale una permutazione dei 3 colori e un lato del grafo;
- M mostra i 2 vertici di questo lato con la colorazione che loro deriva dalla permutazione scelta (rispetto alla 3-colorazione originaria);
- A verifica che i 2 vertici hanno colori diversi.

Il procedimento può ripetersi più volte. È evidente che, se G è 3-colorabile e M sa come farlo, M risponde a tono con assoluta certezza ad ogni domanda di A ; se invece G non è 3-colorabile, si può fare in modo, con qualche maggiore cautela (illustrata per esempio in [85]), di ridurre la probabilità che M riesca ad ingannare A .

Finalmente, A non viene a sapere la totale colorazione del grafo G (almeno quando G è abbastanza "complicato") perché M ha la facoltà di permutarla ad ogni nuova interazione. Infatti chiunque sappia prevedere il lato che A indicherà è capace di rispondergli soddisfacentemente, anche se ignora la 3-colorazione di G . Così l'algoritmo tende a fornire ad A certezze, più che conoscenze.

Avremo modo di riparlare di $AM[Poly]$ nel prossimo paragrafo. Qui preferiamo trattare alcune sue sottoclassi, e precisamente quelle per cui il numero dei possibili messaggi tra A e M è addirittura costante rispetto alla lunghezza dell'input.

Definizione. Sia k un intero positivo.

- $MA[k]$ è la classe dei problemi S risolubili da protocolli Artù-Merlino in cui M parla per primo e la corrispondenza complessiva consiste esattamente in k messaggi.
- $AM[k]$ si definisce allo stesso modo, con l'unica variazione che stavolta il primo interlocutore a intervenire è A .

Esempi.

1) $MA[1] = NP$.

Infatti il protocollo si riduce ad un'unica comunicazione di M ad A , nella quale tutta l'informazione y viene trasmessa senza margini di casualità: quel che ne deriva è, appunto, la definizione di NP .

2) $AM[1] = BPP$.

Stavolta A rivolge a M una domanda, ma M non ha il tempo di rispondere. L'informazione y è dunque la sequenza completamente casuale scelta da A : quel che ne deriva corrisponde chiaramente alla definizione di BPP .

Si noti che, per ogni k , tanto $AM[k]$ quanto $MA[k]$ sono inclusi in $AM[k+1]$ e $MA[k+1]$. A noi comunque interessano in modo particolare

$$MA = MA[2],$$

$$AM = AM[2],$$

e cioè le classi corrispondenti a due interazioni, prima di M e poi di A per MA , a ruoli invertiti per AM . Infatti è possibile osservare che, per ogni $k > 2$, $MA[k] = AM[k] = AM$. Consideriamo per esempio $MA[3]$ e $AM[3]$. Nel primo caso l'algoritmo interattivo si sviluppa come segue: per ogni input w ,

- M spedisce ad A un primo messaggio;
- A lancia un certo numero di volte una moneta e comunica l'esito a M ;
- M risponde ad A .

Si intende che tutti i messaggi tra A e M rispettano i vincoli polinomiali prefissati. Alla fine della corrispondenza, A ne fissa il contenuto complessivo y , dopo di che propone (w, y) a una verifica deterministica polinomiale. D'altra parte, il testo del primo messaggio di M - indipendente dal lancio successivo - può essere ritardato e inserito nella risposta finale. Ne deriva un procedimento per $AM = AM[2]$.

Consideriamo ora il caso di $AM[3]$. Stavolta, per un certo w ,

- A lancia più volte una moneta e dice l'esito a M ;
- M risponde;
- A lancia ancora una moneta.

A questo punto **A** raccoglie l'esito y della corrispondenza e procede. D'altra parte, siccome il secondo lancio casuale di **A** non ha replica, possiamo anticiparlo scambiandolo con la precedente risposta di **M**. Di nuovo, se ne ottiene un procedimento in $AM = AM[2]$.

Si può provare, in generale, che per ogni intero $k \geq 3$, $MA[k] = AM[k] = AM$. Così due interazioni, la prima delle quali da parte di **A**, possono riassumere un qualunque numero ≥ 3 di messaggi. Ma la situazione a proposito di AM e MA è lungi dall'essere chiara. Ecco alcuni spunti a questo proposito.

Osservazioni.

1. È evidente che $NP \subseteq MA \subseteq AM$. La prima inclusione dipende da $NP = MA[1]$. Circa la seconda inclusione, notiamo che quando **M** ha diritto al primo intervento, come nel caso di MA , allora la sua comunicazione iniziale è indipendente da **A**, e possiamo sempre inserirla nella risposta alla prima domanda di **A**, riconducendoci così a AM .
2. Si conoscono anche limitazioni superiori per MA e AM nella gerarchia polinomiale, precisamente

$$MA \subseteq \Sigma_2^P \cap \Pi_2^P, \quad AM \subseteq \Pi_2^P.$$

3. Anche se AM contiene NP , non è chiaro se

$$AM \supseteq coNP.$$

Se sì, si prova che la gerarchia polinomiale si stabilizza ad AM , vale cioè $PH = AM$.

Proponiamo un ulteriore esempio a questo proposito.

Esempio (Goldwasser, Micali, Rackoff, [45]). Consideriamo il problema GI dell'isomorfismo tra grafi: sono dati due grafi finiti $G_1 = (V_1, E_1)$ e $G_2 = (V_2, E_2)$, dobbiamo decidere se c'è un isomorfismo tra G_1 e G_2 , e cioè una biiezione $\pi : V_1 \rightarrow V_2$ tale che, per ogni scelta di $u_1, v_1 \in V_1$,

$$(u_1, v_1) \in E_1 \text{ se e solo se } (\pi(u_1), \pi(v_1)) \in E_2.$$

Sappiamo dal paragrafo 8.8 che GI appartiene certamente a NP , mentre non è chiaro se si trova anche in $coNP$. Allora, se consideriamo il problema complementare GI^c , quello del non-isomorfismo tra grafi, che

- ammette come input $G_1 = (V_1, E_1), G_2 = (V_2, E_2)$,
- dà output $SÌ/NO$ secondo che si possa escludere o meno un isomorfismo π tra G_1 e G_2 ,

possiamo affermare che sta certamente in $coNP$, e non è chiaro se appartenga a P . Tuttavia, $GI^c \in AM$. Ecco il protocollo relativo:

- **A** sceglie casualmente un indice j tra 1 e 2 e il corrispondente grafo $G_j = (V_j, E_j)$, permuta poi, nuovamente in modo casuale, l'insieme V_j dei vertici di G_j , ottenendo un nuovo grafo $G'_j = (V_j, E'_j)$ con gli stessi vertici, ma con nuovi lati a determinare E'_j ; spedisce G'_j a **M** chiedendogli a quale dei due grafi originari G_1 e G_2 esso corrisponde;
- **M** risponde.

A questo punto, se **M** replica correttamente, **A** si convince che G_1, G_2 non sono isomorfi; altrimenti **A** crede che G_1, G_2 sono isomorfi. Spieghiamo meglio la procedura. Notiamo anzitutto che G'_j è isomorfo a G_j tramite la relativa permutazione dei vertici; **M**, però, non conosce j , ma solo G'_j . Comunque:

- se G_1, G_2 non sono isomorfi, solo uno dei due può essere isomorfo a G'_j e **M** può riconoscerlo con le sue infinite risorse; allora una risposta corretta dà ad **A** la possibilità di accettare con probabilità $1 \geq \frac{2}{3}$;
- se G_1, G_2 sono isomorfi, **M** non può distinguerli tramite G'_j e la sua probabilità di risposta scorretta è $\frac{1}{2}$; allora la possibilità di accettare il non-isomorfismo è per **A** uguale a $\frac{1}{2}$ (riducibile a $\frac{1}{2^k}$ dopo k esperimenti e dunque ad un valore $\leq \frac{1}{3}$ dopo due ripetizioni della procedura).

Notiamo anche che **A** non viene a sapere dall'algoritmo sui grafi G_1, G_2 niente di più di quanto già conosceva; in particolare non ha argomenti/dimostrazioni per convincere un terzo interlocutore che G_1, G_2 (non) sono isomorfi. In altri termini, l'interazione ha prodotto certezze, ma non conoscenze (come già nel caso di *3COL*).

13.2 La classe IP

Modelli più raffinati di interazione si hanno quando si consente ad **A** un ruolo più partecipe e consapevole. Possiamo, per esempio, ammettere che ogni messaggio di **A** a **M** derivi da qualche calcolo autonomo che **A** svolge in funzione dell'input, di eventuali testimoni casuali e della sua corrispondenza precedente con **M**. **A** è dunque capace di autonome riflessioni e strategie, e l'interazione con **M** è più profonda ed effettiva (anzi, c'è chi preferisce abbandonare i personaggi dello scettico e ingenuo Artù e dell'onnisciente Merlino per rappresentare i due interlocutori e parlare piuttosto, rispettivamente, di

- un “verificatore” da convincere,
- un “dimostratore” che tenta di convincere;

noi preferiamo mantenere qui, se non il riferimento esplicito ad Artù e Merlino, almeno le lettere **A**, **M** per denotare, appunto, verificatore e dimostratore).

Si introduce comunque la nuova classe *IP* dei problemi con prova interattiva, che è formalmente definita come $AM[Poly]$, salvo che si ammette per **A** la capacità computazionale sopra descritta e che l'unico vincolo sulla interazione è il tempo

polinomialmente limitato dalla lunghezza dell'input che viene complessivamente concesso alle computazioni di $\mathbf{A}$. Così I, P sono da intendersi come le iniziali di *interattivo* e *polinomiale* rispettivamente.

La classe IP fu introdotta da Goldwasser, Micali e Rackoff nel 1985. Ovviamente $IP \supseteq AM[Poly]$. In modo per certi versi sorprendente, si ha addirittura che

$$IP = AM[Poly],$$

come provato da Goldwasser e Sipser nel 1986: dunque, quando la limitazione sui tempi di lavoro è polinomiale e non più costante, la maggiore capacità computazionale di $\mathbf{A}$ non provoca sostanziali miglioramenti nella soluzione dei problemi. È anche possibile intuirne i motivi: infatti le informazioni che derivano nel modello di IP dal maggior impegno computazionale riconosciuto ad $\mathbf{A}$ (al di là del semplice ricorso al lancio di una moneta e dunque al caso), si possono trasferire all'attività di risposta di $\mathbf{M}$, mantenendo i vincoli polinomiali sulla corrispondenza di $\mathbf{A}$ e $\mathbf{M}$, secondo i canoni di $AM[Poly]$. Non ci attardiamo comunque su questa uguaglianza. Osserviamo semmai che $NP \subseteq IP$. Ricordiamo poi che NP fu descritto a suo tempo come la classe dei problemi che hanno una procedura efficiente di verifica. Ma la plausibilità di questa affermazione dipende, ovviamente, da che cosa si intende per "verifica". Nel caso di NP , il termine ha il significato di "convincimento rapido", originato dalla singola notifica di un opportuno testimone y da parte di $\mathbf{M}$. Se però preferiamo intenderla come una prova interattiva, in cui domande, risposte e controlli possono succedersi anche dopo la prima comunicazione di $\mathbf{M}$, IP si propone ragionevolmente come la classe che riunisce i problemi che hanno una procedura efficiente di "verifica" in questo senso. Comunque, la questione se $NP = IP$ è aperta.

Proponiamo adesso un esempio diretto ed esplicito di problema risolubile con prova interattiva.

Esempio. Come già detto $NP \subseteq AM \subseteq AM[Poly] \subseteq IP$, e dunque $3SAT \in IP$, infatti $3SAT \in NP$ ed è anzi NP -completo. Consideriamo allora il problema complementare di $3SAT$, quello che decide, per ogni formula $\alpha \in 3CNF$, se α è insoddisfacibile oppure no. Possiamo denominarlo $3UNSAT$ (a sottolineare l'insoddisfacibilità - *UNSATisfiability* - richiesta). Si tratta di un tipico problema in $coNP$, del quale non è noto se appartiene anche a NP . Vogliamo mostrare

$$3UNSAT \in IP,$$

producendo la relativa prova interattiva.

La strategia che seguiamo per ottenere questo obiettivo prevede due passi.

Passo 1. Dapprima "aritmetizziamo" il problema, associamo cioè ad ogni formula $\alpha \in 3CNF$ due interi N_α, C_α tali che $\alpha \in 3UNSAT$ se e solo se N_α divide C_α , vale cioè la congruenza $C_\alpha \equiv 0 \pmod{N_\alpha}$.

Passo 2. Poi forniamo una prova interattiva che, per ogni α , decide la congruenza $C_\alpha \equiv 0 \pmod{N_\alpha}$ (e, tramite essa, la insoddisfacibilità di α).

Passo 1. Sia $\alpha \in 3CNF$ con

- n variabili $p_1, \dots, p_n$,
- m clausole.

Pensiamo dunque α come insieme finito di clausole, ciascuna ovviamente composta da tre lettere. Fissiamo un numero primo $N_\alpha > 2^n \cdot 3^m$ e consideriamo l'insieme $\mathbb{Z}/N_\alpha\mathbb{Z}$ delle classi di resto modulo N_α . Ricordiamo che $\mathbb{Z}/N_\alpha\mathbb{Z}$ è un campo rispetto alle usuali operazioni di somma e prodotto modulo N_α . Adesso associamo alla formula α un polinomio $f_\alpha(x_1, \dots, x_n)$ a coefficienti interi. Procediamo come segue.

- Per $i = 1, \dots, n$, facciamo corrispondere ad ogni variabile p_i il polinomio x_i e alla negazione P_i il polinomio $1 - x_i$ (in ambedue i casi, il polinomio così formato ha grado 1).
- Ad ogni clausola di α associamo poi la somma dei polinomi corrispondenti alle sue lettere (quindi un polinomio ancora di grado 1). Per esempio, alla clausola $p_1 P_2 P_3$ facciamo corrispondere

$$x_1 + (1 - x_2) + (1 - x_3) = x_1 - x_2 - x_3 + 2.$$

- Finalmente ad α associamo il prodotto dei polinomi corrispondenti alle sue clausole, dunque un polinomio di grado m . Per esempio, se $\alpha = \{p_1 P_2 P_3, P_1 p_2 p_4, p_2 p_3 p_4\}$, allora

$$f_\alpha(x_1, x_2, x_3, x_4) = (x_1 - x_2 - x_3 + 2) \cdot (-x_1 + x_2 + x_4 + 1) \cdot (x_2 + x_3 + x_4).$$

Mostriamo adesso che, per ogni valutazione v di $p_1, \dots, p_n$, si ha

$$v(\alpha) = 0 \text{ se e solo se } f_\alpha(v(p_1), \dots, v(p_n)) \equiv 0 \pmod{N_\alpha}.$$

Procediamo come segue.

- Per $i = 1, \dots, n$ si ha
 - $v(p_i) = 0$ se e solo se $v(p_i)$ annulla il polinomio x_i corrispondente a p_i ,
 - $v(P_i) = 0$ se e solo se $v(p_i) = 1$ e dunque se e solo se $v(p_i)$ annulla il polinomio $1 - x_i$ corrispondente a P_i ,

Si noti che, se il polinomio corrispondente a p_i o P_i non è annullato da $v(p_i)$, allora ha valore $1 < N_\alpha$.

- Per k clausola di α , $v(k) = 0$ se e solo se, per ogni lettera λ in k , $v(\lambda) = 0$ e quindi se e solo se $v(p_1), \dots, v(p_n)$ annullano i tre polinomi corrispondenti alle clausole in α . Ma questo avviene se e solo se $v(p_1), \dots, v(p_n)$ annullano la somma di questi tre polinomi, e cioè il polinomio associato a k , modulo N_α . Infatti è chiaro che, se si annullano i tre addendi, anche la somma si annulla (e quindi, in particolare, $\equiv 0 \pmod{N_\alpha}$). Altrimenti, almeno un addendo vale 1 e dunque la somma vale 1 o 2 o 3. Ma $3 < N_\alpha$ e di conseguenza né 1 né 2 né 3 coincide con 0 modulo N_α . Per inciso, si noti che i possibili valori che $v(p_1), \dots, v(p_n)$ assegnano al polinomio corrispondente a k sono 0, 1, 2, 3.

(iii) Finalmente mostriamo che

$$v(\alpha) = 0 \text{ se e solo se } f_\alpha(v(p_1), \dots, v(p_n)) \equiv 0 \pmod{N_\alpha},$$

come promesso. Infatti $v(\alpha) = 0$ se e solo se c'è almeno una clausola $k \in \alpha$ per cui $v(k) = 0$, ovvero il polinomio associato a k è annullato modulo N_α da $v(p_1), \dots, v(p_n)$. Ma questo equivale a dire che c'è almeno un fattore di grado 1 di $f_\alpha(v(p_1), \dots, v(p_n))$ che viene annullato modulo N_α da $v(p_1), \dots, v(p_n)$ e quindi, in definitiva, che

$$f_\alpha(v(p_1), \dots, v(p_n)) \equiv 0 \pmod{N_\alpha}.$$

Infatti, in un campo - come $\mathbb{Z}/N_\alpha\mathbb{Z}$ - un prodotto si annulla se e solo se si annulla almeno uno dei suoi fattori.

Per inciso, si osservi che, in generale, $f_\alpha(v(p_1), \dots, v(p_n))$ è prodotto di m fattori i cui valori tramite $v(p_1), \dots, v(p_n)$ sono 0, 1, 2 o 3. Segue che, in ogni caso, a prescindere dal fatto che si abbia o no $v(\alpha) = 0$,

$$0 \leq f_\alpha(v(p_1), \dots, v(p_n)) \leq 3^m < N_\alpha.$$

Adesso poniamo $C_\alpha = \sum_v f_\alpha(v(p_1), \dots, v(p_n))$ dove v varia tra le valutazioni di $p_1, \dots, p_n$. Notiamo anzitutto che le valutazioni v di $p_1, \dots, p_n$ sono 2^n , quindi $0 \leq C_\alpha \leq 2^n \cdot 3^m < N_\alpha$, perché C_α è la somma di 2^n addendi, tutti compresi tra 0 e 3^m . Anzi

$$C_\alpha \equiv 0 \pmod{N_\alpha} \text{ se e solo se tutti questi addendi sono } \equiv 0 \pmod{N_\alpha}.$$

Può essere utile osservare anche che, al variare di v , $(v(p_1), \dots, v(p_n))$ descrive ogni possibile n -upla in $\{0, 1\}$; quindi

$$C_\alpha = \sum_{(c_1, \dots, c_n) \in \{0, 1\}^n} f_\alpha(c_1, \dots, c_n).$$

Adesso mostriamo che

$$\alpha \in 3UNSAT \text{ se e solo se } C_\alpha \equiv 0 \pmod{N_\alpha},$$

come promesso. Infatti, si ha che $\alpha \in 3UNSAT$ se e solo se, per ogni valutazione v , $v(\alpha) = 0$, e cioè se e solo se, per ogni v , $f_\alpha(v(p_1), \dots, v(p_n)) \equiv 0 \pmod{N_\alpha}$; ma, per quanto sopra osservato, questo equivale a sostenere $C_\alpha \equiv 0 \pmod{N_\alpha}$, come richiesto.

Questo conclude l'“aritmetizzazione” del problema. Ma, a questo proposito, vale la pena di aggiungere che:

- la costruzione di $f_\alpha(x_1, \dots, x_n)$ a partire da α richiede tempo polinomiale rispetto a $l(\alpha)$;

- se poi fissiamo $N_\alpha = 2^{O(n+m)}$ e quindi ogni elemento di $\mathbb{Z}/N_\alpha\mathbb{Z}$ si può scrivere con $O(n+m)$ cifre in base 2, il calcolo dei valori di f_α modulo N_α si può eseguire ancora in tempo polinomiale rispetto a $l(\alpha)$.

Passo 2. Adesso descriviamo un protocollo con cui **M** può convincere **A** che la generica formula $\alpha \in 3CNF$ è insoddisfacibile, ovvero che $C_\alpha \equiv 0 \pmod{N_\alpha}$.

Prima interazione: **M** spedisce ad **A**

- l'intero N_α ,
- il polinomio $f_1(x_1) = \sum_{c_2, \dots, c_n \in \{0,1\}} f_\alpha(x_1, c_2, \dots, c_n)$,

o, almeno, un numero naturale e un polinomio che egli sostiene essere N_α , $f_1(x_1)$ rispettivamente. Si noti comunque che $f_1(x_1)$ ha grado $\leq m$ e

$$f_1(0) + f_1(1) = \sum_{c_1, c_2, \dots, c_n \in \{0,1\}} f_\alpha(c_1, c_2, \dots, c_n) = C_\alpha \equiv 0 \pmod{N_\alpha}.$$

Seconda interazione: **A** verifica che:

- il numero pervenuto N_α è primo,
- il polinomio pervenuto davvero soddisfa $f_1(0) + f_1(1) \equiv 0 \pmod{N_\alpha}$.

Ricordiamo per inciso che sono disponibili procedure rapide di controllo della primalità che **A** può applicare a N_α . Se una delle due verifiche è negativa, **A** rifiuta di procedere perché ha la certezza che **M** lo ha ingannato. Altrimenti **A** procede nel protocollo, sceglie $r_1 \in \{0, 1, \dots, N_\alpha - 1\}$ e lo invia a **M**.

Terza interazione: **M** costruisce

$$f_2(x_2) = \sum_{c_3, \dots, c_n \in \{0,1\}} f_\alpha(r_1, x_2, c_3, \dots, c_n)$$

e lo spedisce ad **A** (o comunque invia ad **A** un polinomio che spaccia per $f_2(x_2)$). Si noti che $f_2(x_2)$ ha grado $\leq m$ e soddisfa (modulo N_α)

$$f_2(0) + f_2(1) = \sum_{c_2, \dots, c_n \in \{0,1\}} f_\alpha(r_1, c_2, \dots, c_n) = f_1(r_1).$$

Quarta interazione. **A** controlla che il polinomio arrivato soddisfi $f_2(0) + f_2(1) = f_1(r_1)$ modulo N_α . Se no, **A** interrompe la procedura. Se sì, **A** prosegue, sceglie $r_2 \in \{0, 1, \dots, N_\alpha - 1\}$ e lo spedisce a **M**.

Il protocollo prosegue secondo questo schema. In generale, **M** spedisce ad **A** il polinomio di grado $\leq m$

$$f_i(x_i) = \sum_{c_{i+1}, \dots, c_n \in \{0,1\}} f_\alpha(r_1, \dots, r_{i-1}, x_i, c_{i+1}, \dots, c_n),$$

o comunque un polinomio che spaccia per tale. **A** controlla se $f_i(0) + f_i(1) = f_{i-1}(r_{i-1})$ modulo N_α . Se no, rifiuta; se sì, sceglie $r_i \in \{0, 1, \dots, N_\alpha - 1\}$ e la manda a **M**. All'ultimo scambio,

- **M** spedisce a **A** $f_n(x_n) = f_\alpha(r_1, \dots, r_{i-1}, x_n)$ (ancora un polinomio di grado $\leq m$);
- **A** controlla se $f_n(0) + f_n(1) = f_{n-1}(r_{n-1})$ modulo N_α ; se no, rifiuta; se sì, sceglie $r_n \in \{0, 1, \dots, N_\alpha - 1\}$.

Verifica finale. **A** controlla deterministicamente se $f_n(r_n) = f_\alpha(r_1, \dots, r_n)$ modulo N_α , come gli è consentito dal fatto che conosce α e quindi può calcolare il polinomio $f_\alpha(x_1, \dots, x_n)$ e il suo valore $f_\alpha(r_1, \dots, r_n)$ (addirittura in tempo polinomialmente limitato da $l(\alpha)$).

Si noti che i messaggi tra **M** e **A** sono di lunghezza polinomiale in $l(\alpha)$ perché riguardano

- elementi di $\mathbb{Z}/N_\alpha\mathbb{Z}$ (che si scrivono con $O(n + m)$ cifre),

oppure

- polinomi di grado $\leq m$ a coefficienti tra $0, 1, \dots, N_\alpha - 1$.

Anche il numero dei messaggi scambiati è $O(n)$, dunque polinomialmente limitato da $l(\alpha)$.

Adesso controlliamo che la prova interattiva appena descritta mostra $3UNSAT \in IP$.

- Sia dapprima $\alpha \in 3UNSAT$. Allora **M** ne convince **A** con probabilità 1: tutti i controlli di **A** hanno esito positivo se **M** spedisce ad **A** le informazioni corrette di cui può disporre.
- Viceversa, supponiamo $\alpha \in 3SAT$. Dobbiamo mostrare che, indipendentemente dalla strategia di **M** e dalla sua possibilità di fornire informazioni scorrette, **A** rifiuta con alta probabilità. Ricordiamo infatti che, per α soddisfacibile,

$$C_\alpha = \sum_{c_1, \dots, c_n \in \{0, 1\}} f_\alpha(c_1, \dots, c_n) \not\equiv 0 \pmod{N_\alpha}.$$

Così, se **M** invia correttamente $f_1(x_1)$ alla prima interazione, **A** può constatare $f_1(0) + f_1(1) \not\equiv 0 \pmod{N_\alpha}$ e rifiutare. Ovviamente, **M** può cercare di inviare un altro polinomio $f'_1(x_1) \neq f_1(x_1)$ (sempre di grado $\leq m$). Ma $f_1(x_1), f'_1(x_1)$ coincidono per al più m valori in $\mathbb{Z}/N_\alpha\mathbb{Z}$, dunque $f_1(r_1) \neq f'_1(r_1)$ con probabilità $\geq 1 - \frac{m}{N_\alpha}$ rispetto a r_1 . Così, quando **M** spedisce $f_2(x_2)$ correttamente, **A** può osservare che con alta probabilità $1 - \frac{m}{N_\alpha}$

$$f_2(0) + f_2(1) = f_1(r_1) \not\equiv f'_1(r_1) \pmod{N_\alpha}$$

e rifiutare. Di nuovo, **M** deve rimediare spedendo $f'_2(x_2) \neq f_2(x_2)$; ma $f'_2(r_2) \neq f_2(r_2)$ con probabilità $\geq 1 - \frac{m}{N_\alpha}$.

Iterando il procedimento, si vede che la probabilità totale che **A** si convinca che $\alpha \in 3UNSAT$ anche se $\alpha \in 3SAT$ si limita ad essere $\leq \frac{n \cdot m}{N_\alpha}$. $\square$

L'esempio sintetizza le caratteristiche delle prove interattive che sono richieste per IP . Permette anche di dedurre facilmente:

Corollario 13.2.1 $coNP \subseteq IP$.

Dimostrazione. Sia $S \in coNP$, allora il problema complementare di S è in NP e conseguentemente si riduce tramite $\leq_p$ a $3SAT$. Così S si riduce al complementare di $3SAT$, cioè $S \leq_p 3UNSAT$. Sapendo $3UNSAT \in IP$, si deduce facilmente $S \in IP$. $\square$

Ricordiamo comunque che $coNP \subseteq PH$ e che $PH \subseteq PSPACE$. Allora il precedente corollario si ottiene facilmente dal teorema di Shamir che dimostreremo nel prossimo paragrafo e che costituisce un sorprendente collegamento tra IP e la classe $PSPACE$: afferma infatti che le due classi addirittura coincidono, e cioè che $IP = PSPACE$.

13.3 Il teorema di Shamir

Il paragrafo è, appunto, dedicato a provare che i problemi che ammettono prova interattiva, secondo la definizione di IP , coincidono esattamente con quelli che si risolvono deterministicamente in spazio polinomiale, e cioè stanno in $PSPACE$.

Teorema (Shamir) 13.3.1 $IP = PSPACE$.

Dimostrazione. È relativamente semplice provare che $IP \subseteq PSPACE$, è sufficiente infatti adattare la prova che $P \subseteq PSPACE$ data nel Capitolo 10, fondata sul fatto che computazioni che richiedono tempo polinomialmente limitato non possono coinvolgere spazio polinomialmente limitato. Altrettanto vale per le prove interattive di IP , per le quali sia il numero dei messaggi che la lunghezza di ogni singolo messaggio sono polinomialmente limitati dalla lunghezza dell'input, così come la computazione finale del verificatore A .

Proviamo allora che $PSPACE \subseteq IP$. Possiamo riferirci al problema $QSAT$, che sappiamo essere $PSPACE$ -completo e, in tal senso, adeguato a rappresentare per i nostri propositi tutta la classe $PSPACE$ (il lettore può controllare il motivo in dettaglio, come **esercizio**, rifacendosi alla definizione di $PSPACE$ -completezza). Ci basta quindi mostrare che $QSAT \in IP$. L'esperienza di $3UNSAT$ ci suggerisce di procedere in due fasi:

- 1) "aritmetizzare" $QSAT$, associare in particolare ad ogni formula $\phi \in QBF$ un intero $f_\phi \in \{0, 1\}$ in modo tale che

$$\phi \in QSAT \text{ se e solo se } f_\phi = 1;$$

- 2) sviluppare una prova interattiva che $f_\phi = 1$, piuttosto che $\phi \in QSAT$.

1) Abbiamo una formula φ in QBF , dunque φ è

$$Q_1 p_1 \cdots Q_n p_n \alpha(p_1, \dots, p_n)$$

dove ciascun Q_i (per $i = 1, \dots, n$) è $\forall$ o $\exists$ e $\alpha = \alpha(p_1, \dots, p_n)$ è una formula della logica proposizionale con n variabili $p_1, \dots, p_n$.

Inoltre nel precedente paragrafo, provando che $3UNSAT \in IP$, abbiamo visto come associare ad α (almeno per $\alpha \in 3CNF$) un polinomio $f_\alpha(x_1, \dots, x_n)$ a coefficienti interi tale che, per ogni valutazione v di $p_1, \dots, p_n$,

$$v \text{ non soddisfa } \alpha \text{ se e solo se } f_\alpha(x_1, \dots, x_n) \equiv 0 \pmod{N_\alpha},$$

dove N_α è un primo abbastanza grande: per la precisione, N_α è un primo maggiore di $2^n \cdot 3^m$ dove n è, come detto, il numero delle variabili di α e m il numero delle sue clausole; di più f_α ha grado m . Possiamo anche supporre che il termine noto di f_α sia 0 o 1: se questo non è il caso, basta dividere f_α per l'inverso del suo termine noto modulo N_α .

In realtà la costruzione di f_α si può ripetere per ogni possibile formula proposizionale α , anche fuori di $3CNF$ (ai lettori interessati accenniamo che:

- se α è una variabile p_i , le si può associare x_1 ($i = 1, \dots, n$);
- se α è $\neg\beta$ per qualche β , si pone $f_\alpha(x_1, \dots, x_n) = 1 - f_\beta(x_1, \dots, x_n)$;
- se α è $\beta \vee \gamma$ per opportune β, γ , si pone $f_\alpha(x_1, \dots, x_n) = f_\beta(x_1, \dots, x_n) + f_\gamma(x_1, \dots, x_n)$;
- finalmente, se α è $\beta \wedge \gamma$ per qualche scelta di β, γ , si pone $f_\alpha(x_1, \dots, x_n) = f_\beta(x_1, \dots, x_n) \cdot f_\gamma(x_1, \dots, x_n)$;

si procede, ovviamente, modulo un primo abbastanza grande N_α ; se necessario, si divide il polinomio costruito per il suo termine noto modulo N_α).

Così ogni formula $\alpha(p_1, \dots, p_n)$ ottiene il corrispondente polinomio $f_\alpha(x_1, \dots, x_n)$; ma, per semplicità di notazione, continuiamo pure a riferirci a $\alpha \in 3CNF$. Quel che vogliamo fare adesso è estendere il procedimento che produce f_α da α al caso in cui anche i quantificatori $\forall, \exists$ intervengono nella formula, e dunque, in definitiva, a $\varphi \in QBF$. Procediamo per gradi, e cominciamo col considerare

$$\alpha_{n-1} : Q_n p_n \alpha(p_1, \dots, p_n).$$

Distinguiamo le due possibilità $Q_n = \forall$ o $Q_n = \exists$. Notiamo che le variabili in α_{n-1} non vincolate da Q_n sono $p_1, \dots, p_{n-1}$.

Caso 1. $Q_n = \forall$. Sia data una valutazione v di $p_1, \dots, p_{n-1}$. In realtà non abbiamo ancora definito che cosa significhi che v soddisfa, o non soddisfa α_{n-1} . Possiamo però ragionevolmente convenire che, per $Q_n = \forall$,

$$v \text{ non soddisfa } \alpha_{n-1}$$

se e solo se c'è modo di estendere v a p_n , assegnando valore 0 o 1, tale che la valutazione così ottenuta non soddisfa α

e quindi se e solo se

$$f_{\alpha}(v(p_1), \dots, v(p_{n-1}), 0) \equiv 0 \pmod{N_{\alpha}}$$

oppure

$$f_{\alpha}(v(p_1), \dots, v(p_{n-1}), 1) \equiv 0 \pmod{N_{\alpha}}$$

dunque se e solo se

$$f_{\alpha}(v(p_1), \dots, v(p_{n-1}), 0) \cdot f_{\alpha}(v(p_1), \dots, v(p_{n-1}), 1) \equiv 0 \pmod{N_{\alpha}}.$$

In conclusione, se indichiamo

$$f_{\alpha_{n-1}}(x_1, \dots, x_{n-1}) = f_{\alpha}(x_1, \dots, x_{n-1}, 0) \cdot f_{\alpha}(x_1, \dots, x_{n-1}, 1),$$

si ha che, per ogni valutazione v di $p_1, \dots, p_{n-1}$,

$$v \text{ non soddisfa } \alpha_{n-1}$$

se e solo se

$$f_{\alpha_{n-1}}(v(p_1), \dots, v(p_{n-1})) \equiv 0 \pmod{N_{\alpha}}.$$

Si noti che $f_{\alpha_{n-1}}$ ha $n - 1$ indeterminate, e grado $\leq 2 \cdot m$ (come prodotto di due polinomi di grado $\leq m$).

Caso 2. $Q_n = \exists$. Di nuovo, riferiamoci ad una valutazione v di $p_1, \dots, p_{n-1}$. Possiamo facilmente convenire che stavolta, per $Q_n = \exists$,

$$v \text{ non soddisfa } \alpha_{n-1}$$

se e solo se,

comunque si estenda v a p_n , sia assumendo $v(p_n) = 0$ che ponendo $v(p_n) = 1$, si ha che la corrispondente valutazione non soddisfa α

e cioè se e solo se

$$f_{\alpha}(v(p_1), \dots, v(p_{n-1}), 0) \equiv f_{\alpha}(v(p_1), \dots, v(p_{n-1}), 1) \equiv 0 \pmod{N_{\alpha}}.$$

Poniamo allora

$$f_{\alpha_{n-1}}(x_1, \dots, x_{n-1}) = 1 - (1 - f_{\alpha}(x_1, \dots, x_{n-1}, 0)) \cdot (1 - f_{\alpha}(x_1, \dots, x_{n-1}, 1)).$$

È facile controllare che, per ogni valutazione v di $p_1, \dots, p_{n-1}$,

$$v \text{ non soddisfa } \alpha_{n-1}$$

se e solo se

$$f_{\alpha_{n-1}}(v(p_1), \dots, v(p_{n-1})) \equiv 0 \pmod{N_{\alpha}}.$$

Anche stavolta, $f_{\alpha_{n-1}}$ ha $n - 1$ indeterminate e grado $\leq 2 \cdot m$. Di più, possiamo facilmente ottenere che $f_{\alpha_{n-1}}$ abbia termine noto 0 o 1.

Adesso affrontiamo $Q_{n-1}p_{n-1}$ e dunque $\alpha_{n-2} : Q_{n-1}p_{n-1}\alpha_{n-1}$. Procedendo come sopra si ottiene un polinomio $f_{\alpha_{n-1}}(x_1, \dots, x_{n-2})$ a coefficienti interi, $n - 2$ indeterminate, grado $\leq 2^2 \cdot m = 2 \cdot (2m)$ e termine noto 0 o 1 tale che, per ogni valutazione v di $p_1, \dots, p_{n-1}$, si ha

v non soddisfa α_{n-2}

se e solo se

$$f_{\alpha_{n-2}}(v(p_1), \dots, v(p_{n-2})) \equiv 0 \pmod{N_\alpha}.$$

Si prosegue allo stesso modo per gli altri quantificatori. Così, per $\alpha_2 = Q_2 p_2 \cdots Q_n p_n \alpha(p_1, \dots, p_n)$ si ottiene un polinomio $f_{\alpha_2}(x_1)$ a coefficienti interi, una indeterminata x_1 , grado $\leq 2^{n-1} \cdot m$ e termine noto 0 o 1 (modulo N_α) tale che, per ogni valutazione v di p_1 ,

v non soddisfa α_2

se e solo se

$$f_{\alpha_2}(v(p_1)) \equiv 0 \pmod{N_\alpha}.$$

Alla fine, per $\alpha_1 = \varphi = Q p_1 \cdots Q_n p_n \alpha(p_1, \dots, p_n)$ si ottiene una costante f_φ tra 0 e 1 tale che

$$\varphi \text{ è falsa se e solo se } f_\varphi \equiv 0 \pmod{N_\alpha}$$

e cioè se e solo se

$$f_\varphi = 0;$$

in altre parole

$$\varphi \in QSAT \text{ se e solo se } f_\varphi = 1:$$

infatti una formula di QBF è vera o falsa in assoluto, a prescindere dal riferimento a una particolare valutazione.

2) Terminata l'aritmetizzazione, passiamo alla prova interattiva per f_ϕ . L'obiettivo è individuare una procedura tale che:

- se $f_\phi = 1$ e cioè ϕ è vera, **M** ne convince **A** con probabilità 1;
- se $f_\phi = 0$ e cioè ϕ è falsa, la probabilità che **M** convinca **A** del contrario è sufficientemente ridotta.

Si può procedere in maniera analoga al caso precedente di $3UNSAT$. Ecco come **M** può muoversi e convincere **A** che $f_\varphi = 1$.

Prima interazione: **M** elimina $Q_1 p_1$ da f_ϕ , cioè considera α_2 , calcola

$$G_1(x_1) = f_{\alpha_2}(x_1)$$

e lo spedisce a **A** (e comunque invia un polinomio che contrabbanda per $f_{\alpha_2}(x_1)$). Notiamo che $G_1(0) \cdot G_1(1) = f_\phi$ se $Q_1 = \forall$, e $1 - (1 - G_1(0)) \cdot (1 - G_1(1)) = f_\phi$ se $Q_1 = \exists$.

Seconda interazione: **A**, ricevuto $G_1(x_1)$, controlla se $G_1(0) \cdot G_1(1) = 1$ oppure se $1 - (1 - G_1(0)) \cdot (1 - G_1(1)) = f_\phi$, a seconda che Q_1 sia $\forall$ o $\exists$. Se l'uguaglianza non vale, allora rifiuta. Altrimenti sceglie un valore casuale r_1 , con $0 \leq r_1 < N_\alpha$, calcola $s_1 = G_1(r_1)$ modulo n_α e invia r_1 a **M**.

Terza interazione: ora **M** deve convincere **A** che s_1 è il valore di $f_{\alpha_3}(r_1, x_2)$. Allora **M** trasmette ad **A**

$$G_2(x_2) = f_{\alpha_3}(r_1, x_2)$$

o un polinomio contrabbandato per tale. Ma, se $G_2(x_2)$ è davvero quel che promette di essere, si ha che $s_1 = G_1(x_1)$ coincide modulo n_α con

- $G_2(0) \cdot G_2(1)$ se Q_2 è $\forall$,
- $1 - (1 - G_1(0)) \cdot (1 - G_2(1))$ se Q_2 è $\exists$

e **A** può verificarlo.

Quarta interazione: **A** compie il suo controllo, rifiuta in caso di esito negativo, altrimenti sceglie $0 \leq r_2 < N_\alpha$ e lo spedisce a **M**.

Il protocollo interattivo prosegue su questa linea, in modo analogo a quello di *3UNSAT*.

Il numero dei messaggi tra **M** e **A** è $2n$, dunque risulta polinomialmente limitato dalla lunghezza di φ (e di α). Come nel caso di *3UNSAT* si vede che, con questa strategia,

- se $f_\varphi = 1$, **M** ha addirittura probabilità 1 di convincere **A**;
- se $f_\varphi = 0$, **M** ha probabilità bassa di persuadere **A** del contrario.

Ma c'è un "ma": in realtà, il procedimento ora descritto non rispetta tutte le clausole previste da *IP*, in particolare può essere che la lunghezza dei messaggi da **M** ad **A** ecceda i vincoli polinomiali previsti. Infatti, già alla prima interazione **M** invia ad **A** un polinomio $G_1(x_1)$ che può avere grado $2^{n-1} \cdot m$, e dunque una stringa di coefficienti di lunghezza esponenziale nel numero n delle variabili di α . La stessa situazione si ripete nella terza interazione, quando **M** comunica un polinomio che può raggiungere grado $2^{n-2} \cdot m$.

Urge pertanto rivedere e correggere la procedura. Al posto di

$$f_{\alpha_n}, f_{\alpha_{n-1}}, \dots, f_{\alpha_2}$$

è necessario costruire polinomi che giochino lo stesso ruolo ma abbiano grado basso, possibilmente ≤ 1 in ogni variabile. Ci può aiutare l'osservazione che i valori assegnati alle indeterminate di $f_{\alpha_n}, \dots, f_{\alpha_2}$ nella nostra procedura corrispondono a valutazioni di variabili, e dunque si riducono a 0 oppure a 1. Quindi, in astratto, ci interessa la situazione di un polinomio $f(x_1, \dots, x_n)$ in cui qualche variabile ha grado alto ≥ 2 , e si vuole determinare un polinomio $f'(x_1, \dots, x_n)$ che ha grado basso ≤ 1 in ogni variabile e gli stessi valori di $f(x_1, \dots, x_n)$ quando $x_1, \dots, x_n$ variano in $\{0, 1\}$. Ciò risulta possibile se si pensa che, per $x_i \in \{0, 1\}$, $x_i = x_i^2 = x_i^3 = \dots$, per $i = 1, \dots, n$. Così si può porre, ad esempio, per

$$f(x_1, x_2, x_3) = x_1^3 x_2^4 + 5x_1 x_2^3 + x_2^6 x_3^2,$$

$f'(x_1, x_2, x_3) = x_1x_2 + 5x_1x_2 + x_2x_3$. Formalmente, da $f(x_1, \dots, x_n)$ si costruisce in generale

$$f_n(x_1, \dots, x_{n-1}, 1) = x_n f(x_1, \dots, x_{n-1}, 1) + (1 - x_n) f(x_1, \dots, x_{n-1}, 0),$$

dunque un polinomio di grado ≤ 1 in x_n , e si nota che:

- se $x_n = 1$, allora

$$\begin{aligned} f_n(x_1, \dots, x_n) &= 1 \cdot f(x_1, \dots, x_{n-1}, 1) + 0 \cdot f(x_1, \dots, x_{n-1}, 0) = \\ &= f(x_1, \dots, x_{n-1}, 1) = f(x_1, \dots, x_{n-1}, x_n); \end{aligned}$$

- se $x_n = 0$, allora

$$\begin{aligned} f_n(x_1, \dots, x_n) &= 0 \cdot f(x_1, \dots, x_{n-1}, 0) + 1 \cdot f(x_1, \dots, x_{n-1}, 1) = \\ &= f(x_1, \dots, x_{n-1}, 1) = f(x_1, \dots, x_{n-1}, x_n). \end{aligned}$$

Si ripete poi il procedimento sulle variabili $x_{n-1}, \dots, x_1$ fino a giungere a un polinomio

$$f'(x_1, \dots, x_n)$$

il quale

- ha grado ≤ 1 in ogni variabile $x_1, \dots, x_n$,
- assume gli stessi valori di $f(x_1, \dots, x_n)$ per $x_1, \dots, x_n \in \{0, 1\}$.

A questo punto, nel precedente procedimento, si sostituiscono i polinomi

$$f_{\alpha_n}, f_{\alpha_{n-1}}, \dots, f_{\alpha_2}$$

con

$$f'_{\alpha_n}, f'_{\alpha_{n-1}}, \dots, f'_{\alpha_2}.$$

Su questa base si riformula tutto il protocollo interattivo adeguandolo ai dettami di IP . □

Esercizio. Dimostrare che $\#P \subseteq IP$ (suggerimento: $\#P \subseteq PSPACE$?).

13.4 Un tentativo di conclusione

Nei capitoli precedenti il concetto di “computabilità” proposto da Church, Turing ed altri ed esposto nella prima parte del libro è stato oggetto di una qualche rimediazione e revisione. Infatti abbiamo preferito concentrare il nostro interesse solo su quei problemi che sono risolvibili “a costi accessibili” e quindi trascurare come inattuabili quei problemi che, pur risolvibili in teoria, richiedono risorse indisponibili nella pratica. Questa nuova prospettiva ci ha condotto, negli scorsi capitoli, ad introdurre svariate classi di problemi, suddivisi in base

- alla risorsa scelta per misurarne il costo (il tempo, la memoria, la casualità, o altro),
- a che cosa si intende per *accessibile*,
- al modello di calcolo che sviluppa le computazioni,

e talora anche a criteri più sfumati e meno palpabili. Pur tuttavia, il nostro resoconto è rimasto gravemente inadempiente sotto questo punto di vista, e le classi che abbiamo presentato sono ben lontane dell'esaurire tutte le possibilità sin qui escogitate. In realtà, gli esperti del settore sanno che il mondo delle classi di complessità computazionale è assai più vasto ed articolato, tanto che oggi include oltre 400 esempi. A loro proposito si parla ormai di “*zoo della complessità*”, a sottolinearne la molteplice varietà. Anzi, c'è un sito Internet, denominato, appunto, www.complexityzoo.com, che si preoccupa di aggiornarne accuratamente l'elenco, accompagnandolo con la precisa definizione di ogni classe e con un grafico che ne descrive le varie inclusioni: a dicembre 2004 vi risultano censite 412 classi. Ma, al di là della costruzione di questa larga messe di esempi, ci sono altre questioni fondamentali in cui la complessità gioca un ruolo significativo:

- l'attenzione al costo della soluzione di un problema, e alle minime risorse che essa richiede;
- la conseguente ricerca di tecniche di risparmio, capaci di ottimizzare le soluzioni, magari solo in casi particolari,
- e ancora, la coscienza delle connessioni sottili che possono talora collegare problemi di natura assai diversa, e ridurli l'uno all'altro, anche e soprattutto negli algoritmi di soluzione.

A questi aspetti abbiamo cercato di dare l'enfasi necessaria.

C'è poi da dire che la difficoltà di realizzare le precise relazioni tra varie classi computazionali (come P e NP , per citare il caso più famoso) e l'ostinata resistenza che spesso si incontra a fare un minimo di chiarezza in questo ambito sono spesso sorgente di ispirazione per la costruzione di vari procedimenti crittografici per interazioni in rete. Consideriamo ad esempio il tema trattato in questo capitolo, quello dei protocolli interattivi, volti a convincere il recalcitrante Artù delle verità del Mago Merlino. Al di là del riferimento ai romanzi cavallereschi, questi algoritmi si possono applicare anche a situazioni reali. Per citare un caso banale, immaginiamo un inventore che trova un procedimento rivoluzionario e vuole da un lato persuadere un qualche magnate a finanziargliene la fabbricazione e la diffusione, dall'altro mantenere il segreto sui dettagli del suo lavoro; dovrà allora trasmettere al suo interlocutore la certezza della sua scoperta, ma non la sua conoscenza (ad evitare di indurlo nella tentazione di sottrargliela); convincerlo in tempo rapido, al di là di ogni ragionevole dubbio, proprio come per gli algoritmi dei problemi di IP . La tematica dei protocolli interattivi si rivela allora ricca di applicazioni nella vita moderna: ci sono così tecniche che consentono la possibilità di un *poker telefonico* (per chi, ammalato di influenza, non vuol rinunciare ad una partita a distanza con gli amici, ma intende comunque cautelarsi da bluff ed inganni) oppure si applicano ad argomenti di più vasta portata sociale, ad esempio

a concepire meccanismi elettorali che concilino le potenzialità di un voto telematico con le giuste esigenze di riservatezza dei singoli: [94] descrive in dettaglio queste situazioni. I limiti di questo libro ci hanno impedito di approfondire il tema di queste applicazioni e, più in generale, dei legami tra complessità e crittografia; ma ci piace almeno accennarlo in queste note finali, e semmai rimandare il lettore interessato ai riferimenti bibliografici che sviluppano il tema e ne mostrano le prospettive. V'è poi da dire che nuovi scenari nella computazione e nella complessità (incluse le applicazioni alla crittografia) possono derivare dall'idea di *calcolatori quantistici*, basati su recenti sviluppi della ricerca fisica. Ma di questo argomento sarà dato resoconto nella terza parte del libro.

13.5 Esercizi

1. Si provi che, per ogni intero $k \geq 3$, $AM[k] = MA[k] = AM$.
2. La classe IP è chiusa per $\leq_p$?
3. Si cerchi di mostrare in dettaglio che $AM[Poly] = IP$.
4. Si mostri che IP contiene PH .

Riferimenti bibliografici Il modello di Artù e Merlin e le classi $AM[\dots]$ e $MA[\dots]$ furono introdotti da Babai in [10] (si veda anche [11]), mentre IP fu proposta da Goldwasser, Micali e Rackoff in [45]. L'equivalenza tra i due approcci, in particolare l'uguaglianza $IP = AM[Poly]$, furono provate da Goldwasser e Sipser in [46]. La prova interattiva per il problema del non-isomorfismo tra grafi risale a [44]. Finalmente il teorema di Shamir è provato in [96].

Quanto alle applicazioni alla crittografia accennate nel paragrafo conclusivo, esse sono descritte, per esempio, in [90] e [94].

Parte C

Teoria quantistica della computazione

If we spoke a different language we would perceive somewhat different world.
- L. Wittgenstein

Nell'ultimo ventennio si è sviluppato un nuovo approccio alla computazione. Esso è essenzialmente basato sull'osservazione che "l'informazione è fisica" (R. Landauer), infatti essa (nell'accezione più ampia della parola) viene sempre codificata su sistemi fisici ed elaborata per mezzo di operazioni fisiche. Pertanto, la descrizione della manipolazione di informazione non può prescindere dalle leggi fisiche.

Ad oggi la teoria fisica più completa che si ha a disposizione è la *Meccanica Quantistica*. La rivisitazione dei concetti di computazione in termini della meccanica quantistica, *Computazione Quantistica*, ha aperto nuove ed inaspettate possibilità di calcolo.

L'approccio quantistico alla computazione è stato motivato anche da considerazioni di carattere tecnologico. Infatti, in base alla legge (empirica) di G. Moore, ogni diciotto mesi circa, a parità di performances, le dimensioni di un processore si dimezzano. Estrapolando questo andamento ci troveremmo nel volgere di un decennio a codificare e manipolare informazione su scale subatomiche, dove le tradizionali leggi della fisica classica non valgono più e subentrano fenomeni descrivibili solo dalla meccanica quantistica.

Introdurremo in questo capitolo il quadro formale che è alla base della meccanica quantistica senza preoccuparci delle sue implicazioni ed interpretazioni fisiche.

14.1 Nozioni preliminari

Da un punto di vista matematico la meccanica quantistica è basata sull'*algebra lineare*. Ne ricapitoliamo brevemente gli aspetti che ci interessano.

Gli oggetti principali dell'algebra lineare sono gli *spazi vettoriali*. Consideriamo uno spazio vettoriale V (di dimensione finita d_V) sui complessi e denotiamo un generico vettore come $|v\rangle$ (notazione di Dirac, per la quale i vettori sono anche denominati *ket*). Rispetto ad una base $|0\rangle \cdots |d_V - 1\rangle$ il vettore $|v\rangle$ può essere scritto

come $|v\rangle = \sum_i v_i |i\rangle$ con $v_i \in \mathbb{C}$ (se non espressamente indicato, la somma è estesa su tutti gli elementi della base, dunque $i = 0, \dots, d_V - 1$). La rappresentazione

matriciale di tale vettore rispetto alla suddetta base sarà $\begin{pmatrix} v_0 \\ \vdots \\ v_{d_V-1} \end{pmatrix}$.

Definizione Un *operatore lineare* tra due spazi vettoriali V e W (di dimensione d_V e d_W), è una qualunque funzione $A : V \rightarrow W$ tale che $A(\sum_i v_i |i\rangle) = \sum_i v_i A|i\rangle$ (useremo la terminologia “operatore A su V ” qualora $A : V \rightarrow V$). L’operatore identità $\mathbb{I}$, definito in modo che sia $\mathbb{I}|v\rangle = |v\rangle$, $\forall |v\rangle \in V$, è chiaramente lineare.

Definizione Uno *spazio di Hilbert* $\mathcal{H}$ è uno spazio vettoriale V (di dimensione finita d_V) sui complessi dotato di *prodotto interno* $(\cdot, \cdot) : V \times V \rightarrow \mathbb{C}$ tale che:

1. $(\cdot, \cdot)$ è lineare nel secondo argomento,

$$\left(|v\rangle, \sum_i v'_i |i\rangle \right) = \sum_i v'_i (|v\rangle, |i\rangle) ;$$

2. $(|v\rangle, |v'\rangle) = (|v'\rangle, |v\rangle)^*$,

$\forall |v\rangle, |v'\rangle \in V$, con $*$ che indica l’operazione di coniugazione complessa;

3. $(|v\rangle, |v\rangle) \geq 0$,

$\forall |v\rangle \in V$, ed $= 0 \iff |v\rangle = 0$ (vettore nullo).

In meccanica quantistica la notazione standard per il prodotto interno di due vettori $|v\rangle$ e $|v'\rangle$ è $\langle v|v'\rangle$. $\langle v|$ indica il vettore *duale* (denominato anche *bra* nella notazione di Dirac) di $|v\rangle$. Il duale è praticamente un operatore lineare dallo spazio V in $\mathbb{C}$ definito come $\langle v|(|v'\rangle) \equiv \langle v|v'\rangle \equiv (|v\rangle, |v'\rangle)$, cosicché il prodotto interno $\langle v|v'\rangle$ viene anche detto *bra-c-ket*.

I vettori $|v\rangle$ e $|v'\rangle$ saranno ortogonali qualora $\langle v|v'\rangle = 0$. La norma del vettore $|v\rangle$ è definita $\| |v\rangle \| \equiv \sqrt{\langle v|v\rangle}$. Dati due qualsiasi vettori $|v\rangle, |v'\rangle$ vale la disuguaglianza (Cauchy-Schwarz) $|\langle v|v'\rangle| \leq \| |v\rangle \| \cdot \| |v'\rangle \|$. Un vettore $|v\rangle$ si dice normalizzato se $\| |v\rangle \| = 1$. Infine, un insieme di vettori $\{|i\rangle\}$, $i = 0, \dots, d_V - 1$, è ortonormale se e solo se per ogni scelta di i, j si ha $\langle i|j\rangle = \delta_{i,j}$ (il simbolo $\delta_{i,j}$, conosciuto come delta di Kronecker, sta per 1 se $i = j$, 0 altrimenti). Ogni spazio vettoriale di dimensione finita ammette una base ortonormale.

Consideriamo ora la rappresentazione matriciale del prodotto interno.

Siano $|v\rangle = \sum_i v_i |i\rangle$ e $|v'\rangle = \sum_j v'_j |j\rangle$ le rappresentazioni di $|v\rangle, |v'\rangle \in V$ rispetto ad una base ortonormale $\{|i\rangle : 0 \leq i < d_V - 1\}$. Allora, poichè $\langle i|j\rangle = \delta_{i,j}$, si ha

$$\langle v|v'\rangle = \left(\sum_i v_i |i\rangle, \sum_j v'_j |j\rangle \right) = \sum_{ij} v_i^* v'_j \delta_{i,j}$$

$$= \sum_i v_i^* v'_i = (v_0^*, \dots, v_{d_V-1}^*) \begin{pmatrix} v'_0 \\ \vdots \\ v'_{d_V-1} \end{pmatrix}, \quad (14.1)$$

dove chiaramente il bra $\langle v|$ corrisponde al vettore riga le cui componenti sono i complessi coniugati del vettore colonna rappresentante il ket $|v\rangle$.

Definizione Dati due spazi vettoriali V e W (di dimensione d_V e d_W) e due vettori $|v\rangle \in V$, $|w\rangle \in W$, definiamo il *prodotto esterno* $|w\rangle\langle v|$ come un operatore lineare da V in W tale che $\forall |v'\rangle \in V$ si abbia

$$\underbrace{(|w\rangle\langle v|)}_{\text{operatore}} (|v'\rangle) = |w\rangle \underbrace{\langle v|v'\rangle}_{\text{numero complesso}} = |w\rangle. \quad (14.2)$$

Sia $\{|i\rangle : 0 \leq i < d_V\}$ una base ortonormale di V tale che $|v\rangle = \sum_i v_i |i\rangle$ e $\langle i|v\rangle = v_i$, allora

$$\left(\sum_i |i\rangle\langle i| \right) |v\rangle = \sum_i |i\rangle\langle i|v\rangle = \sum_i v_i |i\rangle = |v\rangle. \quad (14.3)$$

Poiché ciò è vero $\forall |v\rangle \in V$, allora si ha la *relazione di completezza*

$$\sum_i |i\rangle\langle i| = \mathbb{I}. \quad (14.4)$$

Dato un sottospazio $K \subset V$ (di dimensione $d_K < d_V$), è sempre possibile costruire una base ortonormale $\{|0\rangle, \dots, |d_K-1\rangle\}$ di K tale che $\{|0\rangle, \dots, |d_K-1\rangle\}$ sia una base ortonormale di K . Allora l'operatore lineare $P_K = \sum_{i=0}^{d_K-1} |i\rangle\langle i|$ è il *proiettore* sul sottospazio K . Supponiamo che $A : V \rightarrow W$ sia un operatore lineare e $\{|i\rangle_V : 0 \leq i < d_V\}$, $\{|j\rangle_W : 0 \leq j < d_W\}$ siano basi ortonormali per V, W rispettivamente, allora

$$A = \mathbb{I}_W A \mathbb{I}_V = \sum_{ij} |j\rangle_W \langle j|A|i\rangle_V \langle i| = \sum_{ij} \underbrace{w \langle j|A|i\rangle_V}_{\text{elementi di matrice } A} |j\rangle_W \langle i|. \quad (14.5)$$

Definizione Un *autovettore* di un operatore lineare A è un vettore $|v\rangle$ (non nullo) tale che $A|v\rangle = v|v\rangle$, dove v è un numero complesso conosciuto come *autovalore* corrispondente a $|v\rangle$.

Gli autovalori possono essere trovati per mezzo dell'equazione caratteristica $\det[A - \lambda \mathbb{I}] = 0$ per λ , dove $\det$ indica il determinante. L'autospazio individuato dall'autovalore v è l'insieme dei vettori che hanno autovalore v .

Un operatore A viene detto diagonalizzabile se ammette una rappresentazione diagonale, ovvero $A = \sum_i \lambda_i |i\rangle\langle i|$ dove i vettori $|i\rangle$ formano un insieme ortonormale di autovettori di A con corrispondenti autovalori λ_i .

Definizione Dato un operatore lineare A sullo spazio di Hilbert V , allora esiste un unico operatore $A^\dagger$ su V tale che

$$(|v\rangle, A|v'\rangle) = (A^\dagger|v\rangle, |v'\rangle), \quad \forall |v\rangle, |v'\rangle \in V, \quad (14.6)$$

$A^\dagger$ è detto *aggiunto* o *hermitiano coniugato* di A . Per convenzione si pone $|v\rangle^\dagger \equiv \langle v|$ quindi $(A|v\rangle)^\dagger = \langle v|A^\dagger$ e $(|v'\rangle\langle v|)^\dagger = |v\rangle\langle v'|$. In termini di rappresentazione matriciale $A^\dagger \equiv (A^*)^T$ con T che indica la trasposizione (ovviamente $(A^\dagger)^\dagger = A$).

Se $A^\dagger \equiv A$, l'operatore viene definito *auto-aggiunto* o *hermitiano*.

Un operatore A è detto *normale* se $AA^\dagger = A^\dagger A$ (ovviamente un operatore hermitiano è anche normale).

Un importante teorema è il

Teorema di decomposizione spettrale 14.1.0.1 *Un operatore lineare su V è normale se e solo se è diagonalizzabile.*

Ciò implica che se A è un operatore normale, esiste una base $\{|a_i\rangle\}$ tale che $A = \sum_i a_i |a_i\rangle\langle a_i|$, con $|a_i\rangle$, a_i rispettivamente autovettori ed autovalori di A (notare che $|a_i\rangle\langle a_i|$ risulta essere il proiettore sull'autospazio individuato da $|a_i\rangle$).

Definizione Un operatore U è detto *unitario* se $U^\dagger U = \mathbb{I}$.

In tal caso $U^{-1} \equiv U^\dagger$. La principale caratteristica degli operatori unitari è quella di preservare il prodotto interno.

Il *prodotto tensoriale* è un modo di assemblare spazi vettoriali distinti per formare uno spazio vettoriale più grande.

Definizione Supponiamo che V e W siano spazi vettoriali di dimensione d_V e d_W , allora $V \otimes W$ è uno spazio vettoriale di dimensione $d_V \times d_W$ i cui elementi sono le possibili combinazioni lineari dei vettori $|v\rangle \otimes |w\rangle$ con $|v\rangle \in V$ e $|w\rangle \in W$. La notazione: $|v\rangle \otimes |w\rangle$, o $|v\rangle|w\rangle$, o $|v, w\rangle$, o $|vw\rangle$ verrà usata indistintamente. Se A e B sono operatori lineari su V e W possiamo definire l'operatore lineare $A \otimes B$ su $V \otimes W$ come

$$(A \otimes B)(|v\rangle \otimes |w\rangle) \equiv A|v\rangle \otimes B|w\rangle. \quad (14.7)$$

Qualora si debbano assemblare n spazi vettoriali identici V , si userà la notazione $V^{\otimes n}$ per indicare lo spazio di prodotto tensoriale. Sia $|\psi\rangle \in V$ ed $A : V \rightarrow V$, allora

$$|\psi\rangle^{\otimes n} \equiv \underbrace{|\psi\rangle \otimes |\psi\rangle \otimes \cdots |\psi\rangle}_{n\text{-volte}} \in V^{\otimes n}, \quad (14.8)$$

$$A^{\otimes n} \equiv \underbrace{A \otimes A \otimes \cdots A}_{n\text{-volte}} : V^{\otimes n} \rightarrow V^{\otimes n}. \quad (14.9)$$

La rappresentazione matriciale del prodotto tensoriale di operatori sfrutta il cosiddetto *prodotto di Kronecker*. Date le matrici $A (m \times n)$ e $B (p \times q)$ allora

$$A \otimes B \equiv \begin{pmatrix} A_{11}B & \cdots & A_{1n}B \\ \vdots & \ddots & \vdots \\ A_{m1}B & \cdots & A_{mn}B \end{pmatrix}, \quad (14.10)$$

dove A_{ij} sono gli elementi di A e $A_{ij}B$ denota una sottomatrice $p \times q$.

Definizione Siano $A = \sum_i a_i |a_i\rangle \langle a_i|$ la decomposizione spettrale di un operatore normale e $f : \mathbb{C} \rightarrow \mathbb{C}$, allora definiamo la funzione di operatore $f(A)$ come

$$f(A) \equiv \sum_i f(a_i) |a_i\rangle \langle a_i|. \quad (14.11)$$

14.2 I postulati della Meccanica Quantistica

Siamo ora in grado di introdurre i concetti fondamentali della meccanica quantistica mediante postulati.

Postulato 14.2.1 *Associato ad ogni sistema fisico isolato vi è uno spazio di Hilbert che rappresenta lo spazio dei possibili stati del sistema. Ad ogni istante il sistema è caratterizzato da uno stato e quindi è completamente descritto da un vettore di stato, ovvero un vettore di norma unitaria nel suddetto spazio di Hilbert.*

Il più semplice sistema quantistico è quello cui è associato uno spazio di Hilbert di dimensione 2, isomorfo a $\mathbb{C}^2$. In esso possiamo considerare una base ortonormale $\{|0\rangle, |1\rangle\}$ rappresentabile come

$$|0\rangle \equiv \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle \equiv \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \quad (14.12)$$

In termini di *informazione* lo spazio (isomorfo a) $\mathbb{C}^2$ costituisce il *qubit* (contrazione di *quantum bit*) ed è l'analogo quantistico del *bit* classico. Oltre ai due stati di base $|0\rangle, |1\rangle$, il qubit ha infiniti altri possibili stati descritti dalle sovrapposizioni lineari $|\psi\rangle = c_0|0\rangle + c_1|1\rangle$ con $c_0, c_1 \in \mathbb{C}$. Questi ultimi rappresentano 4 parametri reali. Tuttavia, se li consideriamo scritti nella forma polare $c_j = r_j e^{i\phi_j}$, $j = 0, 1$, possiamo eliminare un fattore di fase ($e^{i\phi_0}$) comune a tutte le componenti¹, giungendo a $|\psi\rangle = r_0|0\rangle + r_1 e^{i(\phi_1 - \phi_0)}|1\rangle$. Imponendo poi che $|\psi\rangle$ abbia

¹Come risulterà chiaro dal Postulato 14.2.4, un fattore di fase che moltiplica tutte le componenti di uno stato non dà luogo ad effetti *osservabili*. Questi ultimi infatti coinvolgono sempre il bra ed il ket dello stato come $\langle\psi|e^{-i\phi_0} \dots e^{i\phi_0}|\psi\rangle \equiv \langle\psi|\dots|\psi\rangle$. È lecito quindi per i nostri scopi, assumere che $e^{i\phi_0}|\psi\rangle \equiv |\psi\rangle$. Tuttavia, si noti che i fattori di fase relativi tra le varie componenti di un vettore hanno importanza rilevante.

norma unitaria, possiamo scrivere

$$|\psi\rangle = \cos \frac{\theta}{2} |0\rangle + e^{i\phi} \sin \frac{\theta}{2} |1\rangle, \quad (14.13)$$

dove lo stato è ora parametrizzato dagli angoli $\theta, \phi = \phi_1 - \phi_0$. Questi rappresentano le coordinate sferiche di un punto sulla superficie di una sfera a raggio unitario S^2 , chiamata *sfera di Bloch*. Le corrispondenti coordinate cartesiane saranno $x \equiv \cos \phi \sin \theta, y \equiv \sin \phi \sin \theta, z \equiv \cos \theta$. Pertanto, lo spazio di Hilbert degli stati di un singolo qubit può essere parametrizzato tramite i punti sulla superficie della sfera S^2 . In particolare, i punti che rappresentano gli stati $|0\rangle$ “polo nord” e $|1\rangle$ “polo sud” corrispondono al bit classico. Ogni altro punto sulla sfera non ha un corrispondente classico e rappresenta una sovrapposizione non banale degli stati di base.

Può sembrare che l'informazione contenuta in un qubit sia infinita in confronto al contenuto di informazione di un bit classico. Infatti, mentre ad un dato istante, un bit può assumere solo uno dei due valori 0, 1, il qubit può essere in uno degli infiniti punti della superficie della sfera di Bloch. Benché questa possibilità costituisca l'ingrediente essenziale per il parallelismo quantistico, come vedremo più avanti, è bene specificare che l'informazione accessibile relativamente un qubit è comunque limitata ad un bit!

Postulato 14.2.2 *Lo spazio degli stati di un sistema composto è il prodotto tensoriale degli spazi degli stati dei singoli sottosistemi.*

Lo spazio di Hilbert di n qubits è il prodotto tensoriale $\mathbb{C}^2 \otimes \dots \otimes \mathbb{C}^2 = \mathbb{C}^{2^n}$, ed i suoi naturali vettori di base sono $|0\rangle \otimes \dots \otimes |0\rangle \equiv |0\dots 0\rangle, |0\rangle \otimes \dots \otimes |1\rangle \equiv |0\dots 1\rangle, \dots, |1\rangle \otimes \dots \otimes |1\rangle \equiv |1\dots 1\rangle$. Per questa base, conosciuta come *base computazionale*, se opportuno scriveremo $|z\rangle$ per denotare $|z_1\dots z_n\rangle$, con $z \equiv \sum_{l=1}^n z_l 2^{n-l}$ (espansione binaria). Per esempio, $|5\rangle \equiv |0\dots 0101\rangle$. Quindi la base computazionale può equivalentemente rappresentare sia le stringhe di n bits, sia gli interi compresi tra 0 e $2^n - 1$ (in termini di teoria dei gruppi diremo che essa può equivalentemente rappresentare sia gli elementi di $\mathbb{Z}_2^n$ che quelli di $\mathbb{Z}_{2^n}$).

È da notare che la sovrapposizione di stati della base permette di ottenere stati *non separabili* (*entangled*), per i quali non si può attribuire uno stato ben preciso ad ogni singolo sottosistema. Per esempio, considerando due qubits possiamo avere stati del tipo $|\psi\rangle = |0\rangle|0\rangle$ (separabile), oppure $|\psi\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ (non separabile).

Un generico stato di n qubits $|\Psi\rangle$, può essere espresso tramite una espansione sulla base computazionale come

$$|\Psi\rangle = \sum_{z=0}^{2^n-1} c_z |z\rangle, \quad c_z \in \mathbb{C}, \quad \sum_{z=0}^{2^n-1} |c_z|^2 = 1. \quad (14.14)$$

Postulato 14.2.3 *Ogni processo fisico (evoluzione temporale) riguardante un sistema isolato è descritto da una trasformazione unitaria sullo spazio degli stati.*

Quindi lo stato $|\psi\rangle$ di un sistema al tempo t è legato allo stato $|\psi'\rangle$ del sistema al tempo t' da un operatore unitario U tale che $|\psi'\rangle = U|\psi\rangle$.

Una *porta logica quantistica* che agisce su di un insieme di n qubits, *registro quantistico*, non è altro che un operatore unitario nello spazio di Hilbert associato $\mathbb{C}^{2^n}$. Di converso l'evoluzione di ogni sistema isolato può essere rivista come un processo computazionale!

Nello spazio di un qubit i più importanti operatori unitari sono i cosiddetti *operatori di Pauli* che nella base computazionale (14.12) possono essere rappresentati come

$$\begin{aligned} \mathbb{I} &\equiv \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}; & X &\equiv \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}; \\ Y &\equiv \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}; & Z &\equiv \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \end{aligned} \quad (14.15)$$

Essi generano delle trasformazioni unitarie molto interessanti.

Siano $\mathbf{n} \equiv (n_x, n_y, n_z)$ un versore di $\mathbb{R}^3$ e $\boldsymbol{\sigma} \equiv (X, Y, Z)$, allora sfruttando la (14.11) si ha

$$\exp(-i\theta \mathbf{n} \cdot \boldsymbol{\sigma}/2) = \cos \frac{\theta}{2} \mathbb{I} - i \sin \frac{\theta}{2} (n_x X + n_y Y + n_z Z). \quad (14.16)$$

La verifica dell'uguaglianza (14.16) è lasciata come esercizio.

Le trasformazioni (14.16) formano un gruppo denominato $SU(2)$, dove S sta per speciale poiché le matrici hanno determinante 1, U per unitario e 2 per la dimensione. Chiaramente $SU(2)$ è un sottogruppo di $U(2)$, il gruppo delle trasformazioni unitarie nello spazio di dimensione 2.

Le trasformazioni unitarie $\exp(-i\theta \mathbf{n} \cdot \boldsymbol{\sigma}/2) \equiv \mathcal{R}_{\mathbf{n}}(\theta)$ rappresentano delle rotazioni sulla sfera di Bloch di un angolo θ attorno alla direzione $\mathbf{n}^2$.

Esempio

$$\mathcal{R}_x(\theta)|0\rangle = \left[\cos \frac{\theta}{2} \mathbb{I} - i \sin \frac{\theta}{2} X \right] |0\rangle = \cos \frac{\theta}{2} |0\rangle + e^{-i\pi/2} \sin \frac{\theta}{2} |1\rangle, \quad (14.17)$$

rappresenta una rotazione del “polo nord” attorno all'asse x ($\phi = -\pi/2$).

$$\mathcal{R}_y(\theta)|0\rangle = \left[\cos \frac{\theta}{2} \mathbb{I} - i \sin \frac{\theta}{2} Y \right] |0\rangle = \cos \frac{\theta}{2} |0\rangle + \sin \frac{\theta}{2} |1\rangle, \quad (14.18)$$

²Alla luce di ciò si potrebbe pensare che il gruppo $SU(2)$ sia isomorfo al gruppo delle rotazioni nello spazio $\mathbb{R}^3$ (gruppo $O(3)$) rappresentabile con matrici ortogonali 3×3 . Tuttavia, tale conclusione è errata. Si può infatti notare come rotazioni di 2π e 4π , nel linguaggio di $O(3)$ siano rappresentate dalla matrice identità (3×3), viceversa nel linguaggio di $SU(2)$ sono rappresentate da $\mp$ l'identità (2×2) rispettivamente. Si dice dunque che i due gruppi sono solo localmente isomorfi.

rappresenta una rotazione “polo nord” attorno all’asse y ($\phi = 0$).

$$\mathcal{R}_z(\theta)|0\rangle = \left[\cos \frac{\theta}{2} \mathbb{I} - i \sin \frac{\theta}{2} Z \right] |0\rangle = e^{-i\theta/2} |0\rangle, \quad (14.19)$$

rappresenta una rotazione “polo nord” attorno all’asse z , ovvero una trasformazione che lo lascia inalterato (a meno di un fattore di fase).

Postulato 14.2.4 *La misura di un osservabile³ M è descritta tramite un operatore hermitiano M nello spazio degli stati. Sia*

$$M = \sum_m m P_m,$$

la sua decomposizione spettrale, con P_m proiettori sugli autospazi di M . Allora i possibili risultati della misura saranno i corrispondenti autovalori m .

Misurando l’osservabile M quando il sistema si trova nello stato $|\psi\rangle$, la probabilità di ottenere il risultato m è

$$Pr(m) = \langle \psi | P_m | \psi \rangle, \quad (14.20)$$

mentre lo stato del sistema immediatamente dopo la misura risulta essere

$$|\psi'\rangle = \frac{P_m |\psi\rangle}{\sqrt{p(m)}}. \quad (14.21)$$

È da notare che si può scrivere $P_m = |m\rangle\langle m|$ con $\{|m\rangle\}$ base di misura (autovettori di M) e $\sum_m P_m = \mathbb{I}$, quindi $Pr(m) = |\langle \psi | m \rangle|^2$. In generale, ci si riferisce alle $\langle \psi | m \rangle \in \mathbb{C}$ come un’*ampiezza di probabilità*.

Inoltre, il valore medio del risultato della misura è dato da

$$\sum_m m Pr(m) = \sum_m m \langle \psi | P_m | \psi \rangle = \langle \psi | M | \psi \rangle \equiv \langle M \rangle. \quad (14.22)$$

Esempio Consideriamo l’osservabile $Z \equiv P_0 - P_1 \equiv |0\rangle\langle 0| - |1\rangle\langle 1|$ e lo stato del sistema $|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$.

I possibili risultati della misura sono

$$0 \longrightarrow Pr(0) = \langle \psi | 0 \rangle \langle 0 | \psi \rangle = \frac{1}{2}; \quad |\psi'\rangle = |0\rangle. \quad (14.23)$$

$$1 \longrightarrow Pr(1) = \langle \psi | 1 \rangle \langle 1 | \psi \rangle = \frac{1}{2}; \quad |\psi'\rangle = |1\rangle. \quad (14.24)$$

³Una qualsiasi grandezza fisica misurabile.

Per comodità associamo agli autovalori $(+1, -1)$ di Z la stessa etichetta degli autovettori $(0, 1)$. È chiaro ora che, relativamente ad un qubit, l'informazione accessibile con una misura è limitata ad un bit (i due possibili autovalori dell'osservabile). Generalizzando il formalismo di (14.23), (14.24) al caso di un registro di n qubits nello stato (14.14), si ha che la misura nella base computazionale, cioè dell'osservabile $Z^{\otimes n}$, fornirà il risultato z con probabilità $Pr(z) = |c_z|^2$.

Abbiamo appena visto che la meccanica quantistica ha una natura intrinsecamente probabilistica, tuttavia non è da confondere con il caso classico dove la probabilità si introduce qualora non sia definito con certezza lo stato di un sistema. Ad esempio, lo stato $|\psi\rangle = (|0\rangle + |1\rangle) / \sqrt{2}$, non rappresenta banalmente un sistema che può essere con probabilità $1/2$ nello stato $|0\rangle$ e con probabilità $1/2$ nello stato $|1\rangle$ (come si potrebbe erroneamente evincere dalla misura nella base computazionale). Questa sarebbe una semplice miscela statistica classica! La differenza sostanziale risiede nel fatto di avere ampiezze di probabilità che possono dare luogo ad *interferenza quantistica*. Per comprendere meglio tale concetto consideriamo un esempio.

Esempio Supponiamo di avere un sistema di due qubits, inizialmente nello stato $|1\rangle \equiv |01\rangle$ (ricordiamo la notazione $|0\rangle \equiv |00\rangle$, $|1\rangle \equiv |01\rangle$, $|2\rangle \equiv |10\rangle$, $|3\rangle \equiv |11\rangle$), che viene sottoposto ad una prima trasformazione unitaria U tale che

$$|1\rangle \xrightarrow{U} c_{11}|1\rangle + c_{13}|3\rangle, \quad c_{ij} \in \mathbb{C}. \quad (14.25)$$

Quindi il sistema sia sottoposto ad una seconda trasformazione unitaria U' tale che

$$\begin{Bmatrix} |1\rangle \\ |3\rangle \end{Bmatrix} \xrightarrow{U'} \begin{Bmatrix} c'_{11}|1\rangle + c'_{12}|2\rangle \\ c'_{33}|3\rangle + c'_{32}|2\rangle \end{Bmatrix}, \quad c'_{ij} \in \mathbb{C}. \quad (14.26)$$

Combinando le due trasformazioni possiamo vedere che

$$|1\rangle \xrightarrow{U'U} c_{11} (c'_{11}|1\rangle + c'_{12}|2\rangle) + c_{13} (c'_{33}|3\rangle + c'_{32}|2\rangle). \quad (14.27)$$

Ora, la probabilità di avere “2” come risultato di una misura nella base computazionale, ovvero la probabilità di trovare il sistema nello stato $|2\rangle$, è

$$Pr(2) = |c_{11}c'_{12} + c_{13}c'_{32}|^2 = |c_{11}|^2|c'_{12}|^2 + |c_{13}|^2|c'_{32}|^2 + 2\text{Re}(c_{11}^*c'_{12}^*c_{13}c'_{32}), \quad (14.28)$$

dove Re sta per parte reale. La corrispondente evoluzione probabilistica classica in due passi successivi dà invece

$$Pr(2) = Pr_{11}Pr'_{12} + Pr_{13}Pr'_{32}; \quad (14.29)$$

$$Pr_{11} = |c_{11}|^2, Pr'_{12} = |c'_{12}|^2, Pr_{13} = |c_{13}|^2, Pr'_{32} = |c'_{32}|^2.$$

Confrontando (14.28) e (14.29) è facile rendersi conto che il termine di interferenza quantistica $2\text{Re}(c_{11}^*c'_{12}^*c_{13}c'_{32})$ può giocare un ruolo molto importante.

14.3 Qubits versus Bits

Riepiloghiamo ora le diverse caratteristiche dei qubits (quali sistemi quantistici) rispetto ai bits (quali sistemi classici) alla luce dei postulati appena introdotti.

- Con riferimento alla sfera di Bloch $\mathbb{S}^2$, abbiamo visto che un bit corrisponde ad un esiguo sottospazio di un qubit, due soli punti (“polo nord” e “polo sud”) rispetto alla totalità dei punti della superficie sferica. Potremmo quindi descrivere un bit usando lo stesso formalismo vettoriale introdotto, ma limitandoci agli stati della base computazionale, $|0\rangle$ e $|1\rangle$, ed evitando qualsiasi loro combinazione lineare. Estrapolando il ragionamento al caso di n bits, avremmo a che fare solo con stati del tipo $|z\rangle$, $0 \leq z < 2^n - 1$, mentre per n qubits i possibili stati sono del tipo $\sum_z c_z |z\rangle$ con $c_z \in \mathbb{C}$ e $\sum_z |c_z|^2 = 1$.
- Un sottinsieme di un insieme di n bits ha sempre lo stato specificato dal valore dei rispettivi bits (ad esempio nel caso di due bits nello stato $|01\rangle$ si può affermare che il primo bit sia nello stato $|0\rangle$). Viceversa, la possibilità di stati quantici *entangled* non permette, in generale, di assegnare uno stato definito ad un sottinsieme di qubits (ad esempio nel caso di due qubits nello stato $(|00\rangle + |11\rangle)/\sqrt{2}$ non possiamo affermare quale sia lo stato del primo qubit).
- Se descriviamo i bits per mezzo degli stati della base computazionale, le operazioni su di essi devono essere tali da trasformare uno stato della base computazionale in un nuovo stato della stessa (evitando in particolare qualsiasi sovrapposizione lineare degli stati). Quindi le operazioni permesse saranno le sole *permutazioni* degli stati della base computazionale, ovvero un ristretto sottoinsieme delle trasformazioni unitarie relative ai qubits⁴.
- L'informazione che si può estrarre con la misura dei bits è il valore stesso dei bits, ovvero z . Dai qubits, invece si può ottenere z solo con probabilità $|c_z|^2$. Ne consegue che dalle misure nella base computazionale, si può risalire allo stato dei bits, ma non allo stato dei qubits, proprio perché da $|c_z|^2$ non si può dedurre c_z !
Infine, lo stato dopo la misura nel caso dei bits rimane inalterato, cioè $|z\rangle$, mentre nel caso di qubits cambia da $\sum_z c_z |z\rangle$ a $|z\rangle$ a causa della proiezione.

14.4 Esercizi

1. Sia

$$|\psi\rangle = c_{00}|00\rangle + c_{01}|01\rangle + c_{10}|10\rangle + c_{11}|11\rangle, \quad c_{ij} \in \mathbb{C}, \quad i, j = 0, 1,$$

un generico stato sul prodotto tensoriale di due qubits. Trovare una semplice condizione sui coefficienti c_{ij} che è soddisfatta se e solo se lo stato $|\psi\rangle$ risulta

⁴Una matrice di permutazione contiene solo “0” ed “1”; per ogni riga e per ogni colonna contiene solo un “1”.

separabile, cioè può essere scritto come

$$(\alpha|0\rangle + \beta|1\rangle) \otimes (\gamma|0\rangle + \delta|1\rangle), \quad \alpha, \beta, \gamma, \delta \in \mathbb{C}.$$

Si considerino i seguenti stati per un singolo qubit:

$$|v\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle) \quad \text{e} \quad |w\rangle = \frac{1}{2}(\sqrt{3}|0\rangle + |1\rangle).$$

Volendo rappresentarli come punti sulla sfera di Bloch, si trovino i versori $\mathbf{n}_v, \mathbf{n}_w \in \mathbb{R}^3$ corrispondenti alle loro coordinate sferiche. Dimostrare inoltre che

$$|\langle v|w\rangle|^2 = \frac{1}{2}(1 + \mathbf{n}_v \cdot \mathbf{n}_w).$$

Si considerino i seguenti stati per un singolo qubit:

i) *miscela statistica*

$$|0\rangle, \Pr(|0\rangle) = \frac{1}{2}, \quad \text{o} \quad |1\rangle, \Pr(|1\rangle) = \frac{1}{2};$$

ii) *sovrapposizione*

$$\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle).$$

Si applichi la trasformazione unitaria $U = (X + Z)/\sqrt{2}$ agli stati i), ii) e si valuti nei due casi la probabilità di ottenere “0” come risultato della misura nella base computazionale.

Riferimenti bibliografici Una introduzione alla computazione quantistica ed alle nuove prospettive da essa aperte è fornita in [112] e [113]. Le nozioni di algebra lineare si possono trovare in [57]. Metodi e concetti della meccanica quantistica sono trattati approfonditamente nel libro [86]. Il passaggio dal concetto dei bits a quello dei qubits è discusso in maniera semplice in [78].

Modelli di Computazione Quantistica

It is only through our laws of physics that the nature of computation can be understood.

- D. Deutsch

Il modello classico di computazione è rappresentato dalla macchina di Turing (MdT) e le potenzialità ultime di calcolo sono riassunte nella *tesi di Church-Turing*. L'approccio quantistico alla computazione ha portato ad un nuovo modello di calcolo, la *macchina di Turing quantistica*, ovvero una macchina di Turing che opera con le regole della meccanica quantistica. Ciò ha fatto sì che la tesi di Church-Turing venisse trasformata in un principio fisico, *principio di Church-Turing*.

In questo capitolo analizzeremo tali concetti introducendo nuove classi di complessità, quelle relative a macchine di Turing quantistiche. Passeremo quindi dal modello astratto di macchina quantistica al più pratico modello circuitale quantistico mostrandone le caratteristiche.

15.1 Dalla macchina di Turing classica a quella quantistica

Storicamente ci sono stati diversi passaggi per arrivare alla formulazione del concetto di macchina di calcolo quantistica. L'importante punto di partenza è costituito dal *principio di Landauer* per il quale l'azzeramento di informazione è necessariamente un processo dissipativo; ovvero, per effettuare una generica computazione, che contempli la possibilità di azzerare informazione, dobbiamo fornire energia al sistema di calcolo (il processo è dunque *irreversibile*)¹.

Il principio ispirò l'idea di computazione *reversibile* e quindi privo di dissipazione. Di fatto si è mostrato che una qualunque MdT deterministica può essere

¹Il processo di "overwriting" di un bit causa la dissipazione di almeno $kT \ln 2$ joules di energia (qui k è la costante di Boltzmann e T la temperatura assoluta).

simulata efficientemente da una MdT deterministica reversibile, provando, come conseguenza, anche l'esistenza di una MdT reversibile universale.

15.1.1 **Macchina di Turing reversibile**

Sostanzialmente una MdT deterministica *reversibile* è una MdT deterministica (paragrafo 2.3). nella quale:

- ogni quintupla (q, a, q', a', x) è completamente caratterizzata non solo dalle sue prime due coordinate q, a ma anche dalle sue ultime tre q', a', x ;
- oltre a quintuple “standard” del tipo “read-write-shift” sono presenti anche quintuple del tipo “shift-read-write” (x, q, a, q', a') (se la macchina è nello stato q allora si sposta di x e legge il nastro; a quel punto se è presente il simbolo a allora essa scrive a' e passa nello stato q').

In generale adottando i due punti precedenti è possibile definire una MdT deterministica reversibile con m nastri.

Particolarmente importante è il caso della MdT deterministica reversibile a 3 nastri con la quale si dimostra semplicemente come una generica computazione irreversibile (effettuata su una MdT deterministica ad 1 nastro) può essere sempre riorganizzata su di essa come una computazione reversibile in 3 fasi (vedere tabella 15.1).

Nella prima fase tale macchina effettua gli stessi passaggi della corrispondente computazione irreversibile tranne per il fatto che essa salva tutti i risultati intermedi (*history* o *workspace*) nel nastro II (evitando quindi l'operazione irreversibile di azzeramento). La seconda fase consiste nel fare una copia dell'output della computazione sul nastro III (anch'essa operazione reversibile se il nastro usato è inizialmente bianco).

Finalmente la terza fase elimina reversibilmente tutti gli indesiderati risultati intermedi (*history*) ripercorrendo all'indietro i passi della prima fase e riportando quindi la macchina alla sua condizione iniziale (a parte la copia dell'output nel nastro III).

La configurazione finale della macchina contiene allora soltanto l'output della computazione assieme ad una copia ricostruita dell'input. Possiamo notare che

Tabella 15.1

Fase di lavoro	nastro I	nastro II	nastro III
COMPUTE	INPUT		
	OUTPUT	HISTORY	
COPY	OUTPUT	HISTORY	OUTPUT
UNCOMPUTE	INPUT		
	OUTPUT		

la prima fase della computazione è sufficiente di per sé a generare l'output desiderato in maniera reversibile, ma al fine del riutilizzo della macchina per una nuova computazione, risulta indispensabile eliminare l'informazione immagazzinata nel workspace. È chiaro che un semplice azzeramento di tale informazione produrrebbe alla fine una spesa energetica (principio di Landauer) rendendo complessivamente irreversibile il processo di computazione. Ma tale problema viene superato agevolmente aggiungendo la seconda e terza fase con le quali il workspace viene ripulito non dissipativamente, copiando prima l'output e poi invertendo la computazione della prima fase.

Dunque, a patto di allungare la computazione di un fattore circa 2, possiamo evitare la spesa energetica di azzeramento e realizzare una computazione completamente reversibile. In questo stile una MdT reversibile definisce rigorosamente un algoritmo deterministico reversibile *effettivamente* non dissipativo.

Più in dettaglio, una qualunque computazione convergente di n passi producendo un output di lunghezza l su di una MdT deterministica irreversibile ad 1 nastro può essere effettuata in maniera reversibile su di una MdT deterministica reversibile a 3 nastri in $O(n) + O(l)$ passi. Se invece dei 3 nastri consideriamo una MdT deterministica reversibile a 1 nastro (diviso in 3 regioni che riproducono i 3 differenti nastri) allora tale computazione viene effettuata in $O(n^2)$ passi.

Comunque sia, l'incremento temporale della computazione risulta essere polinomiale (così come la quantità di memoria necessaria per il workspace) e dunque si ha una *simulazione efficiente* della MdT deterministica irreversibile presa in esame. In generale possiamo simulare anche una MdT universale e realizzare quindi una MdT universale reversibile.

Dal punto di vista pratico sappiamo dal paragrafo 9.4 che il modello della MdT equivale al modello circuitale tramite il concetto di *famiglia o sequenza uniforme di circuiti*. Corrispondentemente alla trasformazione MdT $\rightarrow$ MdT reversibile, un qualunque circuito irreversibile, contenente le porte universali *NAND* e *FANOUT*, può essere reso reversibile sostituendo le suddette con la porta *Toffoli* reversibile ed universale (paragrafo 9.3).

15.1.2 Macchina di Turing quantistica

La dimostrazione dell'esistenza di un modello di computazione reversibile permise la nascita di un modello di computazione quantistico (*computer quantistico*). Tale modello infatti è in linea teorica basato su trasformazioni *unitarie*, le quali sono particolari trasformazioni reversibili (cioè dotate di inversa).

Tuttavia, fu una interpretazione fisica della tesi di Church-Turing, che portò D. Deutsch al modello di macchina di Turing quantistica (1985).

Il suo punto di partenza fu una critica alla tesi di Church-Turing che considerava molto vaga rispetto ai principi fisici fondamentali. In particolare, egli si propose di rendere più concreto il riferimento "funzioni che possono essere considerate naturalmente (intuitivamente) computabili" presente nella tesi di Church-Turing (paragrafo 2.9). Identificò quindi tali funzioni con quelle che possono essere computate da reali sistemi fisici (la cosa può sembrare persino ovvia, visto che è diffi-

cile credere che qualcosa sia naturalmente computabile se non può essere computato in Natura). Pertanto Deutsch propose in luogo della tesi di Church-Turing un *principio fisico*:

Principio di Church-Turing 15.1.2.1 *Ogni sistema fisico finitamente realizzabile può essere simulato da un modello di macchina computazionale universale che opera con mezzi finiti.*

Deutsch introdusse la *Macchina di Turing Quantistica (QTM)* partendo dal modello di macchina classica e sostituendo alcune componenti dell'ordinaria MdT (e.g. bits), con elementi quantistici (e.g. qubits).

Definizione Una MdT quantistica è una macchina a stati finiti che ha tre componenti fondamentali: un *processore finito*, una infinita *unità di memoria* (della quale solo una porzione finita è usata di volta in volta) e un *cursore* (o *testina*). La descrizione dei tre componenti è la seguente:

1. *Processore Finito*: È un'unità di controllo come in una MdT ma consiste di un numero finito P di qubits. Denotiamo lo spazio di Hilbert degli stati del processore come

$$\mathcal{H}_P \quad \text{con base} \quad \{\otimes_i |p_i\rangle : p_i = 0, 1\}_{i=0}^{P-1}. \quad (15.1)$$

2. *Nastro di Memoria*: Esso ha funzioni simili a quello di una MdT, ma consiste in un numero infinito di qubits. Idealmente ci sarà un qubit per cella, e solo un numero finito di essi sarà attivo in ogni passo computazionale. Denotiamo lo spazio di Hilbert degli stati della memoria come

$$\mathcal{H}_M \quad \text{con base} \quad \{\otimes_i |m_i\rangle : m_i = 0, 1\}_{i=-\infty}^{+\infty}. \quad (15.2)$$

3. *Cursore*: Rappresenta la componente interagente tra l'unità di controllo e il nastro di memoria. La sua posizione è individuata da una variabile $x \in \mathbb{Z}$, e lo spazio di Hilbert associato è

$$\mathcal{H}_C \quad \text{con base} \quad \{|x\rangle : x \in \mathbb{Z}\}. \quad (15.3)$$

Complessivamente, lo spazio di Hilbert che descrive lo spazio degli stati della MdT quantistica è

$$\mathcal{H}_{QTM} = \mathcal{H}_C \otimes \mathcal{H}_P \otimes \mathcal{H}_M. \quad (15.4)$$

I vettori di base in $\mathcal{H}_{QTM}$ sono della forma

$$|x; \mathbf{p}; \mathbf{m}\rangle = |x; p_0, p_1, \dots, p_{P-1}; \dots, m_{-1}, m_0, m_1, \dots\rangle, \quad (15.5)$$

e rappresentano gli stati della base computazionale.

Ora potremmo chiederci quale relazione intercorre tra le caratteristiche della MdT discusse nel Capitolo 2 e quelle della MdT quantistica. All'insieme degli stati Q corrisponde lo spazio di Hilbert $\mathcal{H}_P$. Lo spazio del qubit $\mathbb{C}^2$ sostituisce l'alfabeto A . Riguardo l'insieme delle istruzioni (o funzione di transizione) di una MdT, abbiamo bisogno di specificare come opera una MdT quantistica.

Una MdT quantistica opera in passi di durata fissa T , e durante ogni passo solo il processore ed una parte finita della memoria interagiscono attraverso il cursore. L'insieme delle istruzioni di una MdT è quindi sostituito da una evoluzione temporale unitaria degli stati quantici $|\Psi\rangle \in \mathcal{H}_{QTM}$. Dopo n di passi computazionali, lo stato della MdT quantistica risulterà

$$|\Psi(nT)\rangle = U^n |\Psi(0)\rangle, \quad (15.6)$$

con U operatore unitario. Ovviamente, un valido programma quantistico richiede solo un numero finito di passi. Pertanto ad ogni MdT quantistica è associato un operatore unitario U per eseguire un certo programma, così come la MdT ha un insieme unico di istruzioni, ed ogni MdT esegue un certo compito. Per specificare lo stato iniziale $|\Psi(0)\rangle$, poniamo la posizione del cursore sullo zero ($x = 0$) e prepariamo gli stati del processore con $|\mathbf{p} = \mathbf{0}\rangle$. Invece, gli stati della memoria $|\mathbf{m}\rangle$ saranno preparati per allocare i dati di input e le istruzioni del programma, il tutto codificato in una stringa finita di qubits, mentre ognuno dei restanti qubits della memoria sarà posto in $|0\rangle$. Lo stato iniziale delle MdT quantistica sarà dunque

$$|\Psi(0)\rangle = \sum_{\mathbf{m}} c_{\mathbf{m}} |0; \mathbf{0}; \mathbf{m}\rangle, \quad \text{con} \quad \sum_{\mathbf{m}} |c_{\mathbf{m}}|^2 = 1. \quad (15.7)$$

Il concetto di MdT quantistica operante con “mezzi finiti” che entra nel Principio di Church-Turing significa che la macchina non può eseguire infinite operazioni in un dato istante, né in posizioni arbitrarie lungo il nastro di memoria. Ciò si traduce nelle seguenti condizioni per gli elementi di matrice dell'operatore di evoluzione della MdT quantistica:

$$\begin{aligned} \langle x'; \mathbf{p}'; \mathbf{m}' | U | x; \mathbf{p}; \mathbf{m} \rangle = & \left[\delta_{x', x+1} U^+(\mathbf{p}', m'_{x'} | \mathbf{p}, m_x) \right. \\ & \left. + \delta_{x', x-1} U^-(\mathbf{p}', m'_{x'} | \mathbf{p}, m_x) \right] \prod_{x' \neq x \pm 1} \delta_{m'_{x'}, m_x}. \end{aligned} \quad (15.8)$$

Il prodotto continuo sulla destra assicura che soltanto un qubit di memoria (x -esimo) partecipa in un singolo passo computazionale. I termini $\delta_{x', x \pm 1}$ assicurano che durante ogni passo computazionale la posizione x della testina non possa cambiare più di una unità avanti, indietro o in entrambe le direzioni. Il modo di operare della macchina è quindi *locale* nel nastro della memoria. Chiamiamo $U^\pm(\mathbf{p}', m'_{x \pm 1} | \mathbf{p}, m_x)$, le parti di U che rappresentano il moto in avanti (+) ed indietro (−) in x . In pratica esse rappresentano gli operatori $P_{x \pm 1} U P_x$ nella base computazionale, dove P_x è il proiettore sul sottospazio di Hilbert $\mathcal{H}_{QTM}$ costituito dagli stati con il cursore nella posizione x . L'unitarietà di U impone $(U^\pm)^\dagger U^\mp =$

$0, (U^+)^\dagger U^+ + (U^-)^\dagger U^- = \mathbb{I}$. Ogni operatore unitario $U \in \{U^-, U^+\}$ definisce una MdT quantistica.

Possiamo descrivere la computazione di una MdT quantistica $\mathcal{Q}$ nella maniera seguente. Ad un certo passo computazionale $\mathcal{Q}$ esamina il quadro di indirizzo x e legge lo stato m_x del quadro x . Se lo stato interno è p allora $\mathcal{Q}$ pone il quadro x nello stato m'_x , passa nello stato interno p' e sposta la testina di una unità a destra o a sinistra o coerentemente in entrambe le direzioni. In generale anche partendo da un vettore della base $|0; 0; m\rangle$ si ottiene, dopo alcuni passi computazionali, uno stato collettivo di sovrapposizione coerente tra diversi stati interni della macchina, varie posizioni del cursore e vari stati del nastro (stato *entangled*). Proprio questa proprietà di coerenza distingue la MdT quantistica dalla MdT classica, permettendo il cosiddetto *parallelismo quantistico* ovvero la capacità di poter esplorare contemporaneamente tutte le diramazioni binarie di un *albero computazionale*. Una volta terminata una computazione convergente l'output della computazione scritto sul nastro viene letto effettuando una misura sul nastro stesso. In generale tale processo 'proietta' (postulato 14.2.4) lo stato del nastro su uno dei possibili risultati, ognuno dei quali si ottiene con una probabilità legata alle ampiezze di transizione dei passi computazionali. In tal senso la MdT quantistica è analoga al modello classico di MdT probabilistica. Possiamo ora chiederci se una MdT quantistica estende in qualche modo la nozione di MdT classica, ovvero se la nozione di MdT quantistica include quella di MdT classica. Matematicamente, si ottiene che una MdT quantistica riproduce il funzionamento di una MdT classica e reversibile imponendo:

$$U^\pm(p', m'_{x\pm 1} | p, m_x) = \quad (15.9)$$

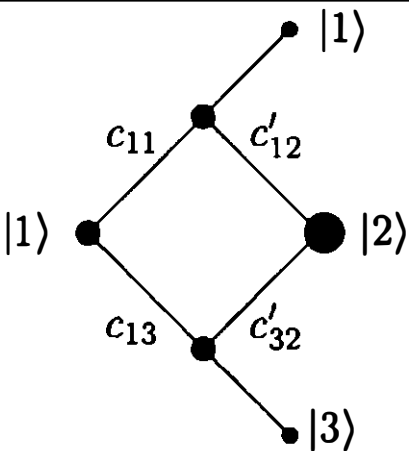
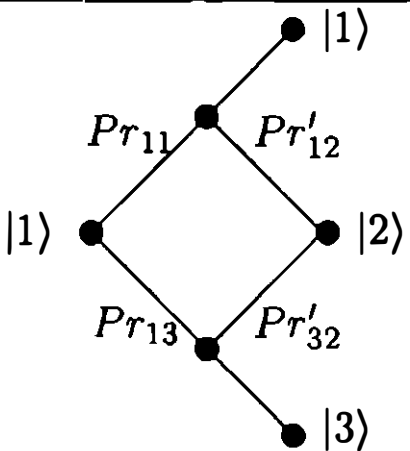
$$\delta_{p', A(p, m_x)} \delta_{m'_{x\pm 1}, B(p, m_x)} \frac{1}{2} [1 \pm C(p, m_x)] \quad (15.10)$$

dove A, B, C sono funzioni da $\mathbb{Z}_2^P \times \prod_{-\infty}^{+\infty} \mathbb{Z}_2$ in $\mathbb{Z}_2^P, \mathbb{Z}_2$ e $\{-1, 1\}$, rispettivamente.

Questa forma della dinamica garantisce che la MdT quantistica rimane negli stati della base computazionale al termine di ogni passo. Questo è esattamente il modo di operare di una MdT nella quale non si riescono a creare sovrapposizioni coerenti (in analogia con quanto discusso nel paragrafo 14.3). La reversibilità è garantita richiedendo che la funzione $(p, m) \mapsto (A(p, m), B(p, m), C(p, m))$ sia biiettiva.

Parallelismo quantistico. La capacità di una MdT quantistica di essere contemporaneamente in più stati della base computazionale (sovrapposizione) è detta *parallelismo quantistico*, ed è la caratteristica saliente di una MdT quantistica. La nozione classica di parallelismo si riferisce alla possibilità di fare "molte cose allo stesso tempo", ed è ora sostituita in ambito quantistico dal concetto di essere in "molti stati allo stesso tempo". Inoltre, classicamente non è sufficiente la semplice connessione di processori in parallelo per ottenere una computazione efficiente, è anche necessaria una appropriata sincronizzazione. Allo stesso modo il parallelismo quantistico non è di per sé sufficiente a garantire il successo della computazione. Dato il carattere probabilistico della meccanica quantistica, in

generale avremo bisogno di ripetere il processo computazionale diverse volte al fine di ottenere un valore attendibile della funzione da computare. Ovviamente, il numero delle ripetizioni dovrebbe essere ridotto al minimo. Ciò dipende dall'operatore $U\{U^+, U^-\}$ e dallo stato iniziale di memoria $|m\rangle$. Un buon algoritmo deve fare in modo che il massimo della distribuzione di probabilità per gli stati di output si abbia in corrispondenza del risultato desiderato, mentre i rimanenti risultati siano molto meno probabili. Questa possibilità risiede nell'operatore unitario $U\{U^+, U^-\}$ come fenomeno dell'*interferenza costruttiva* della ampiezze. Tale concetto era già stato evidenziato nel paragrafo 14.2. Con riferimento alle equazioni (14.25)-(14.29) possiamo pensare ad una MdT quantistica per la quale ad esempio, dato lo stato $|1\rangle$ della configurazione iniziale, dopo due passi computazionali la probabilità di essere nello stato $|2\rangle$ della configurazione finale sia incrementata dal termine di interferenza rispetto al caso di una MdT probabilistica come schematizzato qui di seguito ('1', '2' e '3' sono usati come etichette di configurazioni).

TM quantistica	TM probabilistica
	
$Pr(2) = c_{11}c'_{12} + c_{13}c'_{32} ^2$	$Pr(2) = Pr_{11}Pr'_{12} + Pr_{13}Pr'_{32}$

Macchina di Turing quantistica Universale. La nozione di macchina universale può essere estesa alla MdT quantistica. Una MdT quantistica è capace di eseguire solo il lavoro per cui è stata progettata. Ciò perché l'operatore unitario $U\{U^+, U^-\}$ e lo stato della memoria $|m\rangle$ sono scelti per eseguire uno specifico compito. Tuttavia, gli elementi $U\{U^+, U^-\}$ e $|m\rangle$ di una MdT quantistica possono essere definiti per simulare una qualsiasi altra macchina quantistica. Questo è il concetto di *Macchina di Turing Quantistica Universale*. Quest'ultima rappresenta pertanto un computer quantistico programmabile.

Sia f la generica funzione che vogliamo calcolare con la MdT quantistica universale. Poniamo come $\pi(f)$ il programma (quantistico) che realizza questo obiettivo. Il computer quantistico prende il programma $\pi(f)$ insieme ad un dato valore di input i e calcola il desiderato valore $f(i)$. In corrispondenza di un intero positivo

n si ha la computazione di $f(i)$

$$U^n |0; 0; \pi(f), i, 0\rangle = |0; 0; \pi(f), i, f(i), 0\rangle. \quad (15.11)$$

In questa espressione assumiamo che gli stati iniziali di memoria siano $|\mathbf{m}_i\rangle \equiv |\pi(f), i, 0\rangle$, mentre gli stati finali di memoria contengano la risposta $f(i)$: $|\mathbf{m}_f\rangle \equiv |\pi(f), i, f(i), 0\rangle$ (in entrambi 0 denota lo stato dei qubits che non contengono i dati di interesse).

Una tale macchina è in grado di simulare un qualunque sistema fisico (quantistico e isolato) e quindi in particolare una qualunque MdT quantistica. Vale in particolare il

Principio di Church-Turing forte 15.1.2.2 *Ogni sistema fisico finitamente realizzabile può essere simulato in un tempo al più polinomiale nel tempo di evoluzione fisica del sistema.*

Dobbiamo sottolineare il fatto che, in alcuni casi, l'evoluzione di un sistema fisico equivale al calcolo di una funzione parziale, ma l'insieme delle funzioni parziali, che siamo in grado di calcolare simulando sistemi fisici, non è più grande dell'insieme delle funzioni parziali ricorsive. Quindi, il modello computazionale quantistico non è più potente di quello classico dal punto di vista della decidibilità.

15.2 Classi di complessità quantistiche

Il concetto di MdT quantistica ha molteplici implicazioni, tra le quali ricordiamo una revisione delle classi di complessità. Quando la macchina usata nella computazione è una MdT quantistica, le associate classi di complessità hanno il prefisso "Quantum". Citiamo qui di seguito le più rilevanti.

- Classe EQP (Exact Quantum Polynomial). Contiene quei problemi (decisionali) risolubili esattamente in tempo polinomiale con MdT quantistica (è dunque l'equivalente quantistico di P).

Un insieme S di parole sull'alfabeto finito A si dice *computabile quantisticamente in maniera esatta in tempo polinomiale* (ovvero $S \in EQP$) se esiste una MdT quantistica Q su A tale che, in tempo polinomiale rispetto alla lunghezza dell'input, per ogni parola $w \in A^*$ e in riferimento a un testimone casuale y ,

- se $w \in S$ allora $Pr_y(Q \text{ dice SI a } w) = 1$,
- se $w \notin S$ allora $Pr_y(Q \text{ dice NO a } w) = 1$.

L'inclusione propria $P \subsetneq EQP$ è stata provata e rappresenta un risultato notevole. Significa cioè che un computer quantistico può risolvere efficientemente più problemi di un computer classico.

- Classe ZQP (Zero-error probability Quantum Polynomial). Contiene quei problemi (decisionali) risolvibili con una MdT quantistica con zero probabilità di errore in un tempo polinomiale (è dunque l'equivalente quantistico di ZPP). Un insieme S di parole sull'alfabeto finito A si dice *computabile quantisticamente in tempo polinomiale con errore zero* (ovvero $S \in ZQP$) se esistono due MdT quantistiche Q, Q' su A tali che, in tempo polinomiale rispetto alla lunghezza dell'input, per ogni parola $w \in A^*$ e in riferimento a un testimone casuale y ,

- se $w \in S$ allora $Pr_y(Q \text{ dice SI a } w) \geq \frac{1}{2}$ e $Pr_y(Q' \text{ dice SI a } w) = 1$,
- se $w \notin S$ allora $Pr_y(Q \text{ dice NO a } w) = 1$ e $Pr_y(Q' \text{ dice NO a } w) \geq \frac{1}{2}$.

- Classe BQP (Bounded-error Quantum Polynomial). Contiene quei problemi risolvibili su di una MdT quantistica con probabilità di errore $< 1/2$ in tempo polinomiale (è dunque l'equivalente quantistico di BPP).

Un insieme S di parole sull'alfabeto finito A si dice *computabile quantisticamente in tempo polinomiale con errore limitato* (ovvero $S \in BQP$) se esiste una MdT quantistica Q su A tale che, in tempo polinomiale rispetto alla lunghezza dell'input, per ogni parola $w \in A^*$ e in riferimento a un testimone casuale y ,

- se $w \in S$ allora $Pr_y(Q \text{ dice SI a } w) \geq \frac{1}{2} + \epsilon$,
- se $w \notin S$ allora $Pr_y(Q \text{ dice NO a } w) \geq \frac{1}{2} + \epsilon$.

con ϵ reale positivo $< \frac{1}{2}$.

Dal momento che una MdT quantistica può simulare in tempo polinomiale una MdT probabilistica, si ha che

$$BPP \subseteq BQP. \quad (15.12)$$

Attualmente non si conosce se vale il viceversa e quindi se vale $BPP = BQP$. Per ora i risultati ottenuti portano a congetturare $BPP \subset BQP$, affermando la superiorità del modello computazionale quantistico. L'esempio più eclatante viene dall'algoritmo di Shor riguardo il problema della fattorizzazione (paragrafo 16.4), che ricade in BQP , mentre lo stato dell'arte non permette di affermare che esso sia in BPP . Un'altra questione aperta è la relazione tra BQP e NP (del resto lo è anche quella tra BPP e NP).

Infine sussiste l'inclusione $BQP \subseteq PSPACE$, la quale implica che è possibile classicamente simulare macchine quantistiche con opportune risorse di memoria, benché la simulazione possa essere esponenzialmente rallentata nel tempo². Questo è anche il motivo per cui non esistono problemi risolvibili con una MdT quantistica che escano dal dominio delle MdT deterministiche. Ovvero, la computazione quantistica non contraddice la tesi di Church-Turing. Quindi le MdT quantistiche possono sopravanzare le MdT classiche solo in efficienza.

²In realtà sussiste anche l'inclusione più stringente $BQP \subseteq P^{NP}$.

15.3 Porte logiche quantistiche

La MdT quantistica è un modello per la computazione quantistica, tuttavia non è molto pratica per studiare le potenzialità di un computer quantistico. È altresì molto più utile l'approccio circuitale. L'idea è quella di decomporre il funzionamento di un computer quantistico nelle più semplici operazioni primitive (porte logiche). Nella computazione classica, l'identificazione di porte logiche universali è stata di grande aiuto (paragrafo 9.3). Infatti una porta logica universale come il *NAND* (per la computazione irreversibile) o il *Toffoli* (per la computazione reversibile) agisce localmente su di un ristretto numero di bits, ma si vede che combinando opportunamente un certo numero di esse possiamo eseguire una arbitraria computazione su un arbitrario numero di bits.

L'azione di un computer quantistico corrisponde ad una predeterminata evoluzione (operazione) unitaria su di una serie di qubits (una stringa finita di qubits). Questi ultimi formano il registro di memoria della macchina (*registro quantistico*). Lo spazio di tutti i possibili stati del registro è lo spazio di Hilbert associato. Se $\mathcal{H}$ è lo spazio di Hilbert dell' i -esimo qubit e $|z_i\rangle \in \mathcal{H}$, $z_i = 0, 1$, la sua base computazionale, allora i vettori di base del registro sono dati da tutti i possibili stati del tipo

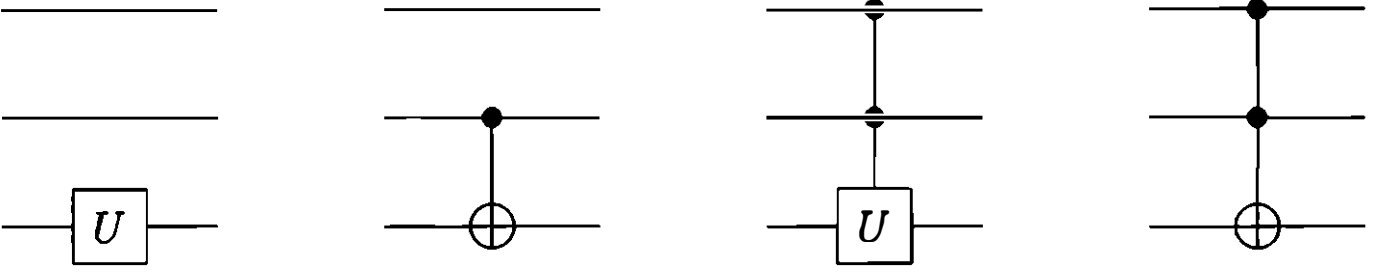
$$|z\rangle \equiv |z_1\rangle \otimes \dots \otimes |z_n\rangle \in \mathcal{H}^{\otimes n}, \quad z_i = 0, 1, \quad i = 1, \dots, n \quad (15.13)$$

dove $z \equiv \sum_{l=1}^n z_l 2^{n-l}$. Ovviamente, un registro di memoria quantistico può immagazzinare anche sovrapposizioni come la (14.14).

Una *porta logica quantistica* è un operatore unitario che agisce sugli stati di un certo insieme di qubits. Se il numero di tali qubits è n , la porta logica è rappresentata da una matrice $2^n \times 2^n$ del gruppo unitario $U(2^n)$. Essa è pertanto una operazione *reversibile*. Una generica porta logica quantistica può avere un numero finito qualsiasi di input qubits, ma per quanto detto sopra siamo interessati solo alle porte logiche elementari e quindi quelle che hanno un piccolo numero di input qubits. Una porta quantistica è rappresentabile come una “black box” dove avviene l'operazione e un certo numero di linee in input ed in output (in eguale numero data la reversibilità) rappresentanti i qubits coinvolti nell'operazione. Vediamo in dettaglio alcuni casi emblematici.

Le porte logiche sono specificate dalla loro azione sulla base computazionale data la loro linearità in quanto trasformazioni unitarie (o equivalentemente dalla loro rappresentazione matriciale sulla base computazionale). In tutti i diagrammi adotteremo la convenzione che il tempo proceda da sinistra verso destra e che i qubits scritti in una sequenza di prodotto tensoriale da sinistra verso destra siano rappresentati con linee dall'alto verso il basso.

Ad esempio, qui di seguito presentiamo alcuni diagrammi di porte logiche su di un registro a 3 qubits.



Da sinistra verso destra abbiamo una porta logica U a singolo qubit, un control-not (CX), un control-control- U (C_2U), ed un control-control-not (C_2X).

Porte logiche ad un qubit Le porte logiche definite come gli operatori $\mathbb{I}$ e X di (14.15) sono equivalenti alle due sole possibili operazioni (reversibili) su un bit, ovvero l'identità ed il "bit flip" o *NOT GATE*. Si noti infatti che $X|z\rangle = |\neg z\rangle$. Tuttavia è possibile disegnare porte logiche che non hanno un analogo classico. Ad esempio $\sqrt{\text{NOT GATE}}$ per cui $U_{\sqrt{\text{NOT}}} = \frac{1}{\sqrt{2}} e^{i\pi/4} (\mathbb{I} - iX)$. Oppure:

HADAMARD GATE

$$U_H \equiv H \equiv \frac{1}{\sqrt{2}}(Z + X) = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad H|z\rangle = \frac{1}{\sqrt{2}} [(-)^z|z\rangle + |\neg z\rangle]; \quad (15.14)$$

PHASE GATE

$$U_\phi \equiv e^{i\phi/2} \left(\mathbb{I} \cos \frac{\phi}{2} - iZ \sin \frac{\phi}{2} \right) = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{bmatrix}, \quad U_\phi|z\rangle = e^{iz\phi}|z\rangle. \quad (15.15)$$

Rispetto al caso classico abbiamo infinite porte logiche a singolo qubit. Ognuna di esse può essere scritta in termini di rotazioni sulla sfera di Bloch, come vedremo più avanti.

Porte logiche a due qubits Anche nel caso di due qubits è possibile considerare porte logiche che hanno un equivalente classico, ad esempio *SWAP GATE*

$$U_{\text{SWAP}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad U_{\text{SWAP}}|z_1\rangle|z_2\rangle = |z_2\rangle|z_1\rangle. \quad (15.16)$$

Introduciamo ora le operazioni controllate CU tali che

$$|c\rangle|t\rangle \mapsto |c\rangle U^c|t\rangle, \quad (15.17)$$

ovvero, porte logiche in cui la trasformazione U agisce sul *target* qubit $|t\rangle$ in maniera condizionata al *control* qubit $|c\rangle$. In particolare, risulta $U^c = U$ per

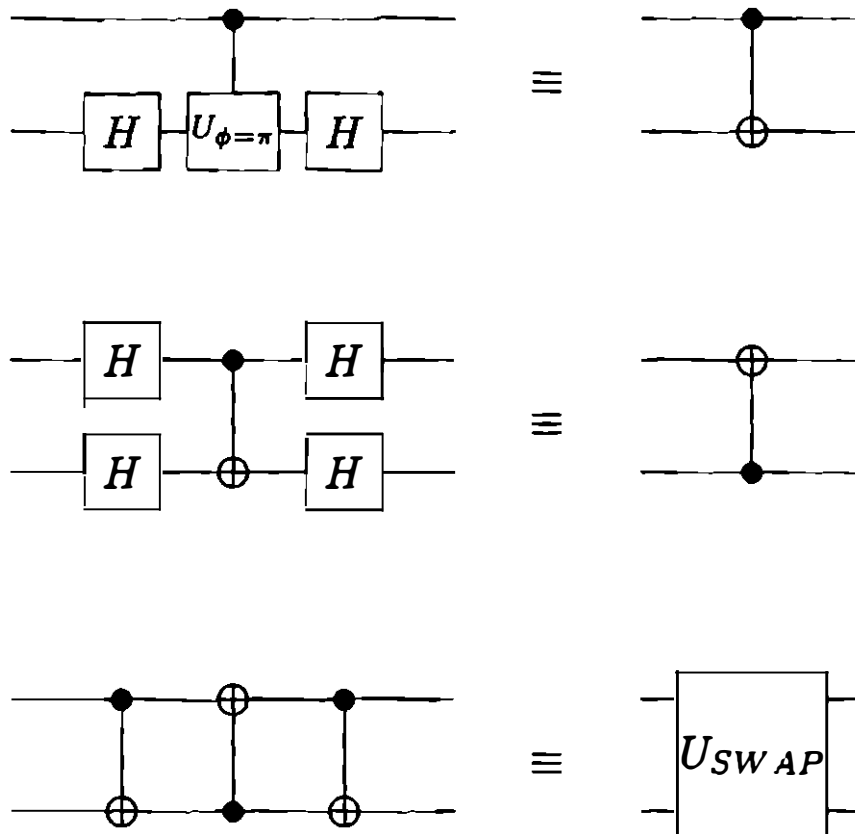
$c = 1$ e $U^c = \mathbb{I}$ per $c = 0$. Consideriamo allora il *CNOT GATE*

$$CX = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad CX|z_1\rangle|z_2\rangle = |z_1\rangle|z_1 \oplus z_2\rangle. \quad (15.18)$$

Esso ha un analogo classico, *XOR* o *OR-esclusivo*, ma anche in questo caso è possibile disegnare porte logiche che non hanno un analogo classico, ad esempio *CPHASE GATE*

$$CU_\phi = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\phi} \end{bmatrix}, \quad CU_\phi|z_1\rangle|z_2\rangle = e^{iz_1 z_2 \phi}|z_1\rangle|z_2\rangle. \quad (15.19)$$

Esempio È facile verificare l'equivalenza delle sequenze di porte logiche di seguito rappresentate, considerando semplicemente la loro azione sulla base computazionale.



Porte logiche a tre qubits Il concetto di operazioni controllate può essere esteso al caso di due qubits di controllo, C_2U (controlled-controlled- U)

$$|c_1, c_2\rangle|t\rangle \mapsto |c_1, c_2\rangle U^{c_1 c_2}|t\rangle, \quad (15.20)$$

con $U^{c_1 c_2} = U$ per $c_1 = c_2 = 1$ altrimenti $U^{c_1 c_2} = \mathbb{I}$.

Si noti che se $U = X$ otteniamo la versione quantistica della porta *Toffoli*.

In generale possiamo avere $C_k U_n$: un'operazione U su n qubits controllata da k qubits.

15.4 Circuiti quantistici

Le semplici porte logiche precedentemente introdotte possono essere assemblate in una rete che permetta l'esecuzione di operazioni quantistiche più complicate. Questa è l'idea base del *circuito quantistico* che Deutsch elaborò generalizzando il modello di circuito classico reversibile.

Nel paragrafo 9.4 si è discussa l'equivalenza tra i modelli classici di MdT e di circuiti logici. Nella teoria quantistica della computazione un teorema dovuto a Yao mostra un risultato analogo. Questo teorema permette quindi un approccio circuitale alla computazione quantistica in luogo di quello basato sulla MdT quantistica.

Definizione Sia K un circuito logico quantistico con n qubits in input. Supponiamo che $|\Psi_x\rangle = \sum_{z \in \{0,1\}^n} c_x(z) |z\rangle$ sia lo stato finale di K per un input $x \in \{0,1\}^n$. La *distribuzione generata* da K per l'input x è definita come la funzione $Pr_x : z \in \{0,1\}^n \mapsto |c_x(z)|^2$. Il circuito quantistico K è detto (n, t) -*simulatore* di una MdT quantistica Q se la famiglia delle distribuzioni di probabilità Pr_x , $x \in \{0,1\}^n$, coincide con le distribuzioni di probabilità delle configurazioni di Q dopo t passi con input x .

Teorema di Yao 15.4.0.3 *Siano Q una macchina di Turing quantistica e n, t interi positivi. Allora esiste un circuito logico quantistico K , di dimensioni polinomiali in n, t , che è (n, t) -simulatore di Q .*

Questo risultato implica che circuiti quantistici possono simulare una MdT quantistica in tempo polinomiale e viceversa. Pertanto, il modello circuitale rappresenta un modello sufficiente per studiare la computazione quantistica.

15.4.1 Insiemi universali

Classicamente, la porta *NAND* è sufficiente per l'universalità, ma essa non è reversibile. Una porta universale reversibile è la porta *Toffoli*, per la quale esiste una versione quantistica. Tuttavia, l'universalità classica è garantita anche dal *FANOUT*.

Esiste una versione quantistica del *FANOUT*? La risposta è negativa e può essere spiegata nei termini del seguente teorema.

Teorema del No-Cloning 15.4.1.1 *Dato uno spazio di Hilbert $\mathcal{H}$ di dimensione ≥ 2 , è impossibile costruire una "macchina quantistica" che clona un arbitrario*

stato $|\psi\rangle \in \mathcal{H}$, ovvero trovare una trasformazione unitaria U su $\mathcal{H}^{\otimes 2}$ tale che $U(|\psi\rangle|\alpha\rangle) = (|\psi\rangle|\psi\rangle)$, $\forall |\psi\rangle \in \mathcal{H}$ con $|\alpha\rangle \in \mathcal{H}$ un fissato stato ancilla.

Dimostrazione. Supponiamo per assurdo che esista una trasformazione unitaria U tale che $U(|\psi\rangle|\alpha\rangle) = (|\psi\rangle|\psi\rangle)$, $\forall |\psi\rangle \in \mathcal{H}$ ed $|\alpha\rangle \in \mathcal{H}$ sia lo stato ancilla (opportunamente preparato) che funge da “foglio bianco” sul quale viene copiato $|\psi\rangle$. Allora, presi $|\psi_1\rangle$ e $|\psi_2\rangle \in \mathcal{H}$ si ha $U(|\psi_1\rangle|\alpha\rangle) = (|\psi_1\rangle|\psi_1\rangle)$ e $U(|\psi_2\rangle|\alpha\rangle) = (|\psi_2\rangle|\psi_2\rangle)$ rispettivamente. Calcolando il prodotto interno (membro a membro) nei due casi si ottiene l’equazione $\langle\psi_1|\psi_2\rangle = (\langle\psi_1|\psi_2\rangle)^2$ che è soddisfatta solo se i due vettori sono identici od ortogonali. $\square$

Il concetto di *insieme universale di porte quantistiche* è centrale nella teoria quantistica della computazione. Un insieme $\mathcal{G} \equiv \{G_{1,n_1}, \dots, G_{r,n_r}\}$ di porte quantistiche G_{j,n_j} agenti su n_j qubits, $j = 1, \dots, r$, è detto universale se ogni trasformazione unitaria U_N su N qubits può essere decomposta nel prodotto di successive azioni G_{j,n_j} su differenti sottoinsiemi dei qubits di input.

Tale nozione di universalità è *esatta* nel senso che la generica trasformazione U_N è intesa come riprodotta esattamente in termini di un numero finito di elementi in $\mathcal{G}$ (in tal caso usiamo la notazione $\mathcal{G}_{\text{ex}}$). Tuttavia, in pratica può essere più conveniente considerare delle simulazioni approssimate di porte logiche.

Definizione Una porta quantistica U_N si dice approssimata da un’altra porta U'_N con un errore $< \epsilon$ quando la distanza $d(U_N, U'_N) \equiv \max_{|\psi\rangle \in \mathcal{H}} \|(U_N - U'_N)|\psi\rangle\|$ è $< \epsilon$.

Con tale definizione potremmo considerare anche la nozione di insieme universale di porte quantistiche in senso *approssimato* (in tal caso useremo la notazione $\mathcal{G}_{\text{p}}$).

Teorema di universalità delle porte logiche quantistiche 15.4.1.2 *Le porte logiche ad un qubit (in pratica il gruppo $U(2)$ delle trasformazioni unitarie in uno spazio di Hilbert di dimensione 2) unitamente alla porta CNOT costituiscono un insieme universale, nel senso esatto, cioè $\mathcal{G}_{\text{ex}} \equiv \{U \in U(2), \text{CNOT}\}$.*

Questo risultato, ulteriore manifestazione del principio di sovrapposizione, è alquanto sorprendente dato che nella computazione classica il CNOT (XOR) non è di per sé sufficiente all’universalità (e, considerando anche la reversibilità non lo sono le porte con meno di tre bits in input).

Dimostrazione. La dimostrazione del teorema 15.4.1.2, può essere specificata in quattro parti.

Parte 1

Lemma 15.4.1.3 *Una porta logica CU può essere scritta in termini di CNOT e operazioni a singolo qubit.*

Dimostrazione. Dalla teoria dei gruppi si ha che ogni matrice $U \in U(2)$ può essere decomposta come:

$$U = e^{i\varphi} \bar{U}, \quad \bar{U} = \mathcal{R}_z(\alpha)\mathcal{R}_y(\beta)\mathcal{R}_z(\gamma), \quad (15.21)$$

dove φ è la fase ($\text{mod}\pi$) del fattore $U(1)$ di $U(2)$ e α, β, γ sono gli angoli (di Eulero) che parametrizzano le rotazioni (14.16) che definiscono la matrice $\bar{U} \in SU(2)$. Con l'aiuto della decomposizione (15.21) possiamo mostrare che per ogni matrice unitaria $\bar{U} \in SU(2)$ esistono matrici $U_1, U_2, U_3 \in SU(2)$ tali che

$$U_1 U_2 U_3 = \mathbb{I}, \quad U_1 X U_2 X U_3 = \bar{U}. \quad (15.22)$$

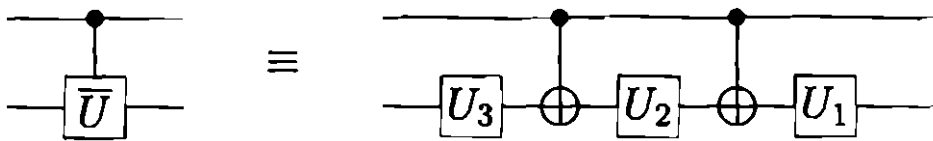
La prova è per costruzione con

$$U_1 = \mathcal{R}_z(\alpha)\mathcal{R}_y(\beta/2), \quad (15.23)$$

$$U_2 = \mathcal{R}_y(-\beta/2)\mathcal{R}_z(-(\alpha + \gamma)/2), \quad (15.24)$$

$$U_3 = \mathcal{R}_z((-\alpha + \gamma)/2). \quad (15.25)$$

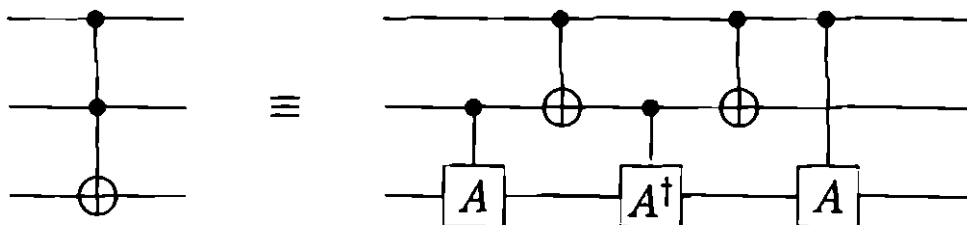
Usando tali risultati è facile mostrare l'equivalenza dei circuiti



considerando in input tutti i possibili stati della base computazionale. Tenendo poi conto che il fattore $e^{i\varphi}$ può essere introdotto con un *PHASE GATE*, si è dimostrato il Lemma.

Parte 2

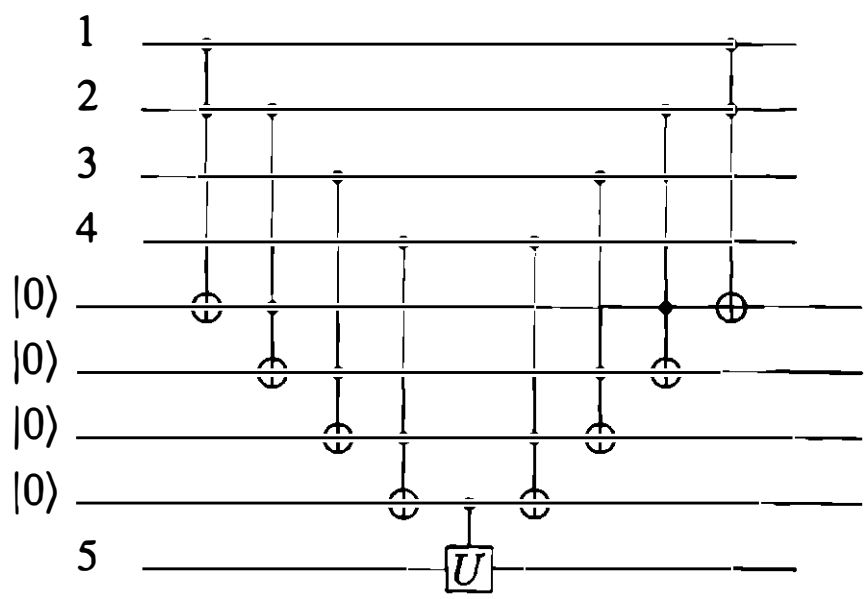
Una porta Toffoli può essere scritta in termini di CA e CX come mostra la seguente figura



dove $A \equiv (1 - i)(\mathbb{I} + iX)/2$. L'equivalenza dei due circuiti è facilmente dimostrabile considerando l'azione sugli stati della base computazionale.

Parte 3

Il seguente circuito



mostra come realizzare una trasformazione $C_{n-1}U$ (nel caso particolare $n = 5$) tramite porte di tipo Toffoli e CU (unitamente a $O(n)$ qubit ausiliari inizializzati in $|0\rangle$). Il costo è pari a $O(n)$ porte Toffoli ed una porta CU , e quindi in virtù dei risultati di Parte 1 e 2, il costo è di $O(n)$ porte di $\mathcal{G}_{\text{ex}}$.

Parte 4

Un'arbitraria matrice unitaria agente su n qubits (quindi $2^n \times 2^n$, $n \geq 2$) può essere decomposta nel prodotto di al più $2^{n-1}(2^n - 1)$ matrici unitarie a due livelli (una matrice a due livelli è una generica rotazione $U(2)$ nel sottospazio individuato da due stati della base computazionale che lascia inalterati tutti gli altri stati della base).

Supponiamo che U sia una matrice a due livelli su n qubits, e che essa agisca sul sottospazio individuato dagli stati $|s\rangle$ e $|t\rangle$, dove $s = s_1 \dots s_n$ e $t = t_1 \dots t_n$ sono le stringhe espansioni binarie di s e t . Definiamo quindi come $\tilde{U}$ la sottomatrice non banale 2×2 . Essa può essere pensata come operazione unitaria su singolo qubit da utilizzare unitamente a porte $CNOT$ per realizzare U .

A tal uopo consideriamo un *gray code* che connette s e t , cioè una sequenza (numerata da 1 a m) di stringhe binarie che inizia con s e si conclude con t ed è tale che le stringhe adiacenti differiscono solo per un bit. Esempio:

1	00111010	s
2	00111011	
3	00111111	
4	00110111	
5	00100111	t

Allora, le operazioni che coinvolgono passi adiacenti di questo codice richiedono una semplice modifica delle porte controllate C_{n-1} . Ovvero gli $(n - 1)$ controllori che rimangono inalterati non saranno tutti condizionati ad $|1\rangle$, ma eventualmente anche a $|0\rangle$. Ciò può essere ottenuto con l'uso di porte NOT prima e dopo l'applicazione della C_{n-1} operazione. Pertanto, la desiderata trasformazione U può essere costruita come segue. Dapprima si scambiano gli stati lungo il gray code, eseguendo le permutazioni $(1,2), (2,3), (3,4), \dots (m-2,m-1)$ [nella tabella dell'e-

sempio precedente $m = 5$]. Ognuna di queste permutazioni è eseguita da una operazione $C_{n-1}X$ modificata. Quindi la desiderata trasformazione $\tilde{U}$ è eseguita tramite l'operazione $C_{n-1}\tilde{U}$ modificata, che coinvolge gli stati $(m-1)$ and (m) . Infine, si applicano permutazioni nell'ordine inverso: $(m-1, m-2)$, $(m-2, m-3)$, ... $(3, 2)$, $(2, 1)$.

Il numero delle operazioni elementari necessarie per eseguire i passi di cui sopra può essere facilmente stimato. Occorrono $(n-1)$ operazioni controllate per scambiare $|s\rangle$ con $|t\rangle$ ed altrettante per la permutazione inversa. Ognuna di queste operazioni controllate può essere realizzata con $O(n)$ porte di $\mathcal{G}_{\text{ex}}$. Pertanto per implementare U occorreranno $O(n^2)$ porte elementari. Inoltre abbiamo detto che un'arbitraria operazione unitaria nello spazio di n qubits può essere scritta come il prodotto di $O(2^{2n})$ matrici unitarie a due livelli. Combinando questi risultati si ottiene che un arbitraria operazione unitaria su n qubits può essere realizzata con un circuito contenente $O(n^2 4^n)$ porte elementari. $\square$

Abbiamo quindi mostrato come sia possibile implementare un generico circuito quantistico usando semplicemente *CNOT* e porte a singolo qubit. Pertanto $\mathcal{G}_{\text{ex}} = \{U \in U(2), \text{CNOT}\}$, costituisce un insieme universale di porte logiche.

La complessità della costruzione di un circuito quantistico a partire dalle porte elementari è una questione molto importante. Combinando insieme i risultati delle Parti 1-4 si ottiene il costo per simulare esattamente un'arbitraria trasformazione su n qubits in $U(2^n)$ che è dell'ordine $O(n^2 4^n)$.

È da notare tuttavia che $\mathcal{G}_{\text{ex}}$ corrisponde ad un insieme infinito di porte. In pratica si vorrebbe avere un insieme universale finito di porte. In tal caso si potrebbe simulare una generica trasformazione entro un certo errore ϵ . Inoltre un insieme di porte universali $\mathcal{G}_{\text{ap}}$ è nello spirito del principio di Church-Turing che impone che una macchina debba operare con mezzi finiti.

Siccome le trasformazioni a singolo qubit corrispondono a delle rotazioni sulla sfera di Bloch S^2 , due rotazioni in direzioni diverse sono sufficienti per poter approssimare qualsiasi altra rotazione. Pertanto possiamo considerare $\mathcal{G}_{\text{ap}} = \{H, W, \text{CNOT}\}$, con H HADAMARD GATE e $W = \text{diag}(1, e^{i\pi/4})$ (da notare che $W \equiv e^{i\pi/8} \mathcal{R}_{(0,0,1)}(\pi/4)$ e $H \equiv e^{-i\pi/2} \mathcal{R}_{(1/\sqrt{2}, 0, 1/\sqrt{2})}(\pi)$).

Tuttavia siamo anche interessati all'efficienza di simulazioni approssimate di una generica porta U . La proprietà di universalità di $\mathcal{G}_{\text{ap}}$ significa che, data un'arbitraria operazione $U \in U(2^n)$ e $\epsilon > 0$, possiamo sempre costruire U' con porte di $\mathcal{G}_{\text{ap}}$ tale che $d(U, U') < \epsilon$.

Approssimare ogni porta a singolo qubit richiede intuitivamente $O(1/\epsilon)$ porte H e W dato che queste generano rotazioni che riempiono densamente ed in maniera uniforme S^2 . Allora, dato un circuito, trasformazione unitaria $U \in U(2^n)$, che contenga m porte tipo *CNOT* e singolo qubit, il numero di porte di $\mathcal{G}_{\text{ap}}$ richieste per approssimare U sarà $O(m^2/\epsilon)$ visto che l'errore per singola porta deve essere ϵ/m . In realtà si può fare molto meglio. Un teorema, noto come *teorema di Solovay e Kitaev* mostra che è possibile un miglioramento esponenziale

dell'approssimazione. Ovvero, un'arbitraria porta a singolo qubit può essere approssimata entro un'accuratezza ϵ usando $O(\log^c(1/\epsilon))$ porte di $\mathcal{G}_{ap}$, dove $c \approx 2$. Pertanto, in base a tale teorema, approssimare un circuito con m porte ($CNOT$ e singolo qubit) richiede $O(m \log^c(m/\epsilon))$ porte di $\mathcal{G}_{ap}$.

Tuttavia, se vogliamo approssimare una generica trasformazione unitaria $U \in U(2^n)$, dobbiamo combinare il teorema di Solovay e Kitaev con la complessità di simulare trasformazioni con $\mathcal{G}_{ex}$, e con la linearità nella propagazione dell'errore nel numero di porte, ne segue che una generica porta unitaria $U \in U(2^n)$ può essere approssimata entro un errore ϵ con $O(n^2 4^n \log^c(n^2 4^n/\epsilon))$ porte in $\mathcal{G}_{ap}$. Si noti come ciò rappresenti una complessità esponenziale nel numero di qubits.

15.4.2 Aritmetica con circuiti quantistici

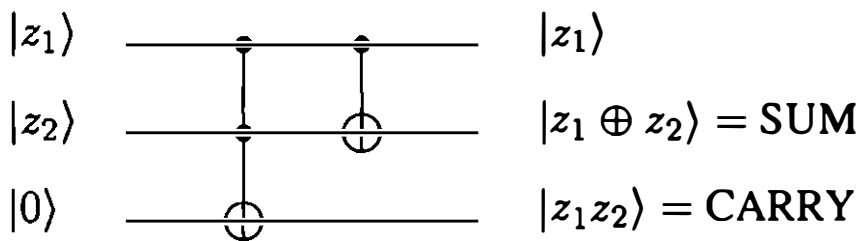
Tipicamente con un computer si è interessati a fare delle operazioni aritmetiche, quindi vediamo come si può realizzare dell'aritmetica quantistica con un insieme di porte universali.

Come visto nella Parte 2 della dimostrazione del teorema 15.4.1.2 una porta *Toffoli* può essere realizzata tramite $CNOT$ ed operazioni a singolo qubit. D'altro canto la porta Toffoli consente di realizzare l'operazione AND in maniera reversibile (analogamente al caso classico):

$$C_2X|z_1, z_2, z_3 = 0\rangle = |z_1, z_2, z_1 \wedge z_2\rangle, \tag{15.26}$$

A questo punto è chiaro che un insieme di porte universali permette la realizzazione dei connettivi logici fondamentali NOT , AND e $CNOT$. Tramite essi le operazioni come addizione, moltiplicazione ed esponenziazione modulare possono essere realizzate in modo del tutto analogo alla computazione classica.

Esempio La concatenazione di una porta Toffoli con una porta CX fornisce un semplice "addizionatore quantistico":



Inoltre, i connettivi logici fondamentali permettono di valutare ogni funzione Booleana $f : \{0, 1\}^n \mapsto \{0, 1\}^m$. Ad esempio, possiamo rivedere l'azione della porta Toffoli come la valutazione della funzione $f : \{0, 1\}^2 \mapsto \{0, 1\}$ definita come $f(z_1, z_2) = z_1 \wedge z_2$, che avviene nel modo seguente

$$|z_1, z_2\rangle|y\rangle \mapsto |z_1, z_2\rangle|y \oplus (z_1 \wedge z_2)\rangle, \quad y \in \{0, 1\}. \tag{15.27}$$

Generalizzando, possiamo pensare alla valutazione di una funzione $f : \mathbb{Z}_2^n \mapsto \mathbb{Z}_2^m$ come realizzata dalla trasformazione unitaria U_f che implementa l'azione di

f . A tal fine richiediamo due registri: il primo di n qubits per immagazzinare l'argomento di f , ed il secondo di m qubits per immagazzinare i valori di f . Quindi:

$$|z\rangle|y\rangle \xrightarrow{U_f} |z\rangle|y \oplus f(z)\rangle, \quad y \in \mathbb{Z}_2^m, \quad (15.28)$$

dove $\oplus$ indica l'addizione $\text{mod } 2$ fatta bit per bit sulle stringhe y ed $f(z)$. Qualora invece sia $f : \mathbb{Z}_{2^n} \rightarrow \mathbb{Z}_{2^m}$, considereremo la trasformazione

$$|z\rangle|y\rangle \xrightarrow{U_f} |z\rangle|(y + f(z)) \bmod 2^m\rangle, \quad y \in \mathbb{Z}_{2^m}. \quad (15.29)$$

e la corrispondente rappresentazione binaria degli stati dei registri.

Esempio Sia $f : \mathbb{Z}_{2^2} \rightarrow \mathbb{Z}_{2^3}$ tale che $f(z) = z^2 \bmod 8$. Il circuito che calcola f agisce come segue

$$\begin{aligned} |00\rangle|000\rangle &\rightarrow |00\rangle|000\rangle, & |10\rangle|000\rangle &\rightarrow |10\rangle|100\rangle, \\ |01\rangle|000\rangle &\rightarrow |01\rangle|001\rangle, & |11\rangle|000\rangle &\rightarrow |11\rangle|001\rangle, \end{aligned}$$

che corrisponde a $|z\rangle|0\rangle \mapsto |z\rangle|z^2 \bmod 8\rangle$ con la scelta $y = 0$ nella (15.29).

L'uso di due registri è dettato dal fatto che una trasformazione unitaria deve preservare l'ortonormalità; una funzione non biiettiva valutata su un singolo registro $|z\rangle \rightarrow |f(z)\rangle$ potrebbe trasformare due stati ortonormali in due stati non-ortonormali.

Si noti che in generale si ha, $\sum_z |z\rangle|0\rangle \xrightarrow{U_f} \sum_z |z\rangle|f(z)\rangle$, ovvero la valutazione di f avviene per tutti gli z in un singolo passo (parallelismo quantistico).

Sovente non si è interessati a come realizzare esplicitamente la trasformazione unitaria U_f , ma semplicemente alla valutazione della funzione. In tali casi la U_f viene considerata come una *black box* da usare ogni qualvolta si voglia conoscere il valore della funzione per un dato input. Ciò equivale al ricorso ad un *oracolo* come discusso nel Capitolo 8. Pertanto, la valutazione della funzione per un dato input può essere considerata come una chiamata, *query*, di un oracolo.

15.5 Esercizi

1. Si supponga di avere un computer quantistico con n qubits disposti in linea e numerati da 1 ad n . Oltre alle operazioni a singolo qubit si ha la possibilità di implementare solamente *CNOT* tra qubits adiacenti, cioè un *CNOT* tra qubits i e j è possibile solo se $|i - j| = 1$. Mostrare che questo è comunque un computer quantistico *universale*, con il quale cioè si può ottenere l'insieme $\mathcal{C}_x$ in maniera efficiente.
2. Data una generica trasformazione unitaria U su singolo qubit, si mostri che $\forall \epsilon > 0$ esistono $n_1, n_2, n_3 \in \mathbb{N}$, tali che

$$d(U, [\mathcal{R}_n(\theta)]^{n_1} [H\mathcal{R}_n(\theta)H]^{n_2} [\mathcal{R}_n(\theta)]^{n_3}) < \epsilon$$

con $\mathcal{R}_n(\theta) \equiv (W)(HWH)$ composizione di due rotazioni attorno all'asse z ed x rispettivamente.

(Suggerimento: si usi la proprietà $d(U_m \dots U_1, V_m \dots V_1) \leq \sum_{j=1}^m d(U_j, V_j)$).

3. Si consideri un computer quantistico con n qubits operante una computazione rappresentabile come sequenza di $p(n)$ porte logiche elementari (ad esempio appartenenti a $\mathcal{G}_{\text{ex}}$) dove $p(n)$ indica un polinomio in n . Sia $|0\rangle$ lo stato iniziale, si valuti l'ampiezza di probabilità di terminare la computazione in uno stato $|y\rangle$, $y \in \{0, 1\}^n$. Dimostrare che essa può essere stimata da un computer classico con risorse spaziali polinomiali in n . Usare il risultato per validare l'affermazione $BQP \subseteq PSPACE$.

Riferimenti bibliografici Il principio di Landauer si può trovare in [71]. Il problema della reversibilità nella computazione è affrontato in [16]. Il principio di Church-Turing e la formulazione della macchina di Turing quantistica sono stati introdotti in [32]. L'inclusione $P \subsetneq EQP$ è stata provata in [20], mentre $BQP \subseteq PSPACE$ in [19]. La dimostrazione del teorema di Yao si può trovare in [14]. Il problema dell'universalità di un insieme di porte quantistiche è trattato in dettaglio in [82].

Quantum mechanics helps in searching for a needle in a haystack.

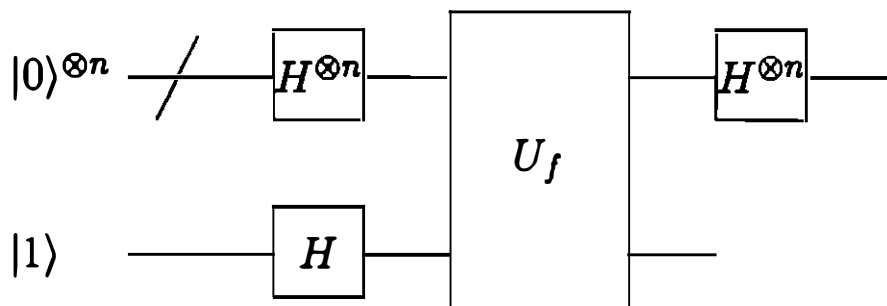
- L. K. Grover

L'approccio quantistico alla computazione non è fine a se stesso, ma conferisce ad essa ulteriori potenzialità. Queste ultime sono diventate palesi dopo lo sviluppo di algoritmi quantistici che permettono la risoluzione di taluni problemi in maniera più efficiente rispetto all'approccio classico. Così, problemi come la fattorizzazione, ritenuti non trattabili classicamente (perlomeno allo stato attuale delle conoscenze), sono divenuti trattabili in ambito quantistico, dove si risolvono con un guadagno esponenziale rispetto al caso classico. Altri problemi invece, come quello della ricerca su un database non strutturato, ammettono "solo" un guadagno polinomiale. Altri ancora, non ammettono invece nessun guadagno. In questo capitolo descriveremo gli algoritmi quantistici più rilevanti da un punto di vista storico ed applicativo. Infine, accenneremo ai limiti della computazione quantistica. Lasceremo al di fuori di questa trattazione le possibili implementazioni di computer quantistici (hardware) che coinvolgono esclusivamente aspetti fisici e tecnologici.

16.1 Algoritmo di Deutsch-Jozsa

Questo algoritmo introdotto da Deutsch ha rappresentato il primo esempio di come un computer quantistico possa battere in efficienza un computer classico. Presentiamo qui una versione estesa dell'algoritmo dovuta a Deutsch e Jozsa (1992). Supponiamo di avere un oracolo che su richiesta valuti la funzione $f : \{0, 1\}^n \rightarrow \{0, 1\}$. Assumiamo che f sia *costante* [$\forall x_1, x_2 \in \{0, 1\}^n, f(x_1) = f(x_2)$] o *bilanciata* [$\#f^{-1}(0) = \#f^{-1}(1)$, cioè il numero di argomenti che hanno immagine 0 è uguale al numero di argomenti che hanno immagine 1].

Per determinare il tipo di funzione (costante o bilanciata) con certezza, classicamente abbiamo bisogno di $2^{n-1} + 1$ queries (o valutazioni di funzione). Quantisticamente avremo bisogno solo di una query! Infatti, consideriamo il seguente circuito



costituito da due registri, uno ad n qubits¹ per immagazzinare l'argomento della funzione f e l'altro ad un qubit per immagazzinare il valore della stessa.

1. Essi sono inizializzati come segue:

$$|0\rangle^{\otimes n} |1\rangle. \quad (16.1)$$

È da notare, come generalizzazione della (15.14), che

$$H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} (-1)^{x \cdot z} |z\rangle, \quad (16.2)$$

dove $x \cdot z = x_1 z_1 \oplus \dots \oplus x_n z_n$ è il prodotto scalare delle componenti binarie di x e z . Pertanto, dopo l'applicazione delle porte H allo stato (16.1) avremo

$$\sum_x \frac{|x\rangle}{\sqrt{2^n}} \left[\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right]. \quad (16.3)$$

2. Quindi la valutazione della funzione dà

$$\sum_x \frac{(-1)^{f(x)} |x\rangle}{\sqrt{2^n}} \left[\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right]. \quad (16.4)$$

3. Infine, un'ulteriore applicazione delle porte H sul primo registro conduce a

$$\sum_z \sum_x \frac{1}{2^n} \left[(-1)^{x \cdot z + f(x)} |z\rangle \right] \frac{1}{\sqrt{2}} [|0\rangle - |1\rangle]. \quad (16.5)$$

4. Consideriamo ora una misura sulla base computazionale del primo registro, ed in particolare la proiezione su $|z\rangle = |0\rangle^{\otimes n}$;

- se f è costante, allora l'ampiezza per $|0\rangle^{\otimes n}$ risulta ± 1 a seconda del valore che $f(x)$ assume; tutte le altre ampiezze risultano invece zero;

¹Una linea orizzontale recante il simbolo “/” sta per n linee orizzontali.

- se f è bilanciata, allora i contributi positivi e negativi all'ampiezza di $|0\rangle^{\otimes n}$ si elidono.

In definitiva, se la misura nella base computazionale dà tutti 0, la funzione è costante, altrimenti la funzione è bilanciata.

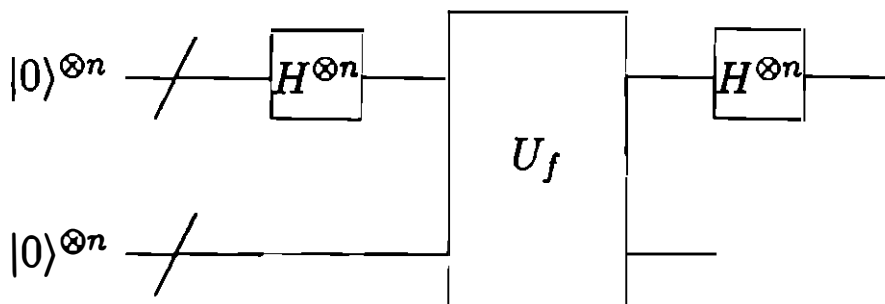
Si noti che il risultato ottenuto rimane inalterato qualora si consideri preliminarmente una misura sul secondo registro, dopo la valutazione della funzione (ad esempio dopo il passo 2.), con un qualunque output.

È chiaro come il problema delle funzioni costanti o bilanciate, se affrontato in maniera classica deterministica, ricada al di fuori della classe P , mentre con l'algoritmo di Deutsch-Jozsa possiamo porlo a tutti gli effetti in EQP . Ciò conferma che $P \subsetneq EQP$.

16.2 Algoritmo di Simon

L'algoritmo di Simon (1994) utilizza molti degli elementi dell'algoritmo di Deutsch-Jozsa. Esso affronta il problema del *periodo* di una funzione Booleana. Nello specifico, supponiamo di avere un oracolo che su richiesta valuti la funzione $f : \{0, 1\}^n \mapsto \{0, 1\}^n$ soggetta alla seguente condizione (promessa): esiste un vettore non nullo $p \in \{0, 1\}^n$ chiamato *periodo* di f tale che $f(x) = f(x')$ se e solo se $x = x'$ o $x = x' \oplus p$.

Mostreremo come quantisticamente sia possibile determinare il periodo p dopo un numero $O(n)$ di valutazioni della funzione, mentre gli algoritmi classici conosciuti richiedono un numero esponenziale di queries. Consideriamo il seguente circuito



costituito da due registri, ad n qubits, uno per immagazzinare l'argomento della funzione f e l'altro per immagazzinare il valore della stessa.

1. Essi sono inizializzati come segue:

$$|0\rangle^{\otimes n} |0\rangle^{\otimes n}. \quad (16.6)$$

Pertanto, dopo l'applicazione delle porte H (16.2) sul primo registro avremo

$$|\psi_1\rangle = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle |0\rangle. \quad (16.7)$$

2. Quindi la valutazione della funzione dà

$$\frac{1}{\sqrt{2^n}} \sum_x |x\rangle |f(x)\rangle. \quad (16.8)$$

3. Consideriamo ora la misura nella base computazionale del secondo registro e supponiamo che sia z_0 il risultato. Sul primo registro, lo stato corrispondente alla proiezione (misura) sarà allora

$$\frac{1}{\sqrt{2}} (|x_0\rangle + |x_0 \oplus p\rangle), \quad (16.9)$$

dove $z_0 = f(x_0) = f(x_0 \oplus p)$.

4. L'ulteriore applicazione delle porte H sul primo registro conduce a

$$\frac{1}{\sqrt{2}} \frac{1}{\sqrt{2^n}} \sum_y \left[(-1)^{x_0 \cdot y} + (-1)^{(x_0 \oplus p) \cdot y} \right] |y\rangle. \quad (16.10)$$

5. Consideriamo infine la misura sulla base computazionale del primo registro. Essa fornirà solo valori di y (tutti equiprobabili) tali che $p \cdot y = 0$ (per convincersene basta ripetere il ragionamento al punto 4. dell'algoritmo di Deutsch-Jozsa).

Ripetendo i precedenti passi M volte otterremo M vettori $y_{(i)}$ tali che

$$p \cdot y_{(i)} = 0, \quad i = 1, \dots, M. \quad (16.11)$$

Risolvendo questo sistema lineare (con algoritmi classici) otterremo il periodo p con probabilità sufficientemente alta a patto che $M = O(n)$ (occorrono almeno n stringhe y linearmente indipendenti per poter risolvere il sistema).

Classicamente una banale ricerca del periodo p implicherebbe $2^{n-1} + 1$ chiamate dell'oracolo. L'algoritmo di Simon mostra un guadagno esponenziale rispetto al caso classico riducendo il problema in BQP .

16.3 Trasformata di Fourier quantistica

Alcune considerazioni preliminari.

Definizione Dato un gruppo Abeliano finito G di ordine $|G| = N$ (che scriviamo per semplicità in forma additiva), un *carattere* di G è una funzione $\chi : G \rightarrow \mathbb{C} \setminus \{0\}$, tale che $\chi(g + g') = \chi(g)\chi(g')$, $\forall g, g' \in G$. Qui $\mathbb{C} \setminus \{0\}$ denota il gruppo moltiplicativo dei complessi non nulli.

Rispetto al prodotto, i caratteri di G formano a loro volta un gruppo Abeliano $\hat{G}$ (gruppo dei caratteri) che fornisce una rappresentazione (irriducibile) del gruppo G stesso.

Per i nostri scopi ci limitiamo ad elencare le proprietà salienti dei caratteri.

- Siccome $\chi(g)^N = \chi(Ng) = \chi(0) = 1$, ogni valore di un carattere è una radice N -esima dell'unità.
- Ci sono sempre esattamente N differenti funzioni χ_k (con $k = 0, \dots, N-1$) soddisfacenti la condizione di carattere.
- Vale la seguente relazione di ortogonalità

$$\frac{1}{N} \sum_{k=0}^{N-1} \chi_k(g_i) \chi_k^*(g_j) = \delta_{i,j} \quad \text{con } g_i, g_j \in G. \quad (16.12)$$

Definizione Siano $G = \{g_0, g_1, \dots, g_{N-1}\}$ un gruppo Abeliano, $\{\chi_0, \dots, \chi_{N-1}\}$ i suoi caratteri ed f una funzione $f : G \rightarrow \mathbb{C}$. La trasformata di Fourier discreta (DFT) di f è una funzione $\tilde{f}$ definita come

$$\tilde{f}(g_k) = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \chi_k^*(g_j) f(g_j). \quad (16.13)$$

Consideriamo ora un sistema quantistico capace di rappresentare gli elementi di un gruppo Abeliano $G = \{g_0, g_1, \dots, g_{N-1}\}$, ovvero sia $\{|g\rangle : g \in G\}$ una base ortonormale dello spazio di Hilbert $\mathcal{H}$ associato al sistema quantistico. Siccome in $\mathcal{H}$ un generico stato $c_0|g_0\rangle + c_1|g_1\rangle + \dots + c_{N-1}|g_{N-1}\rangle$ ($c_i \in \mathbb{C}$) può essere visto come il risultato dell'azione di una funzione $f : G \rightarrow \mathbb{C}$ tale che $f(g_i) = c_i$, allora possiamo porre:

Definizione La trasformata di Fourier quantistica (QFT) è l'operazione che trasforma la base ortonormale $\{|g_0\rangle, |g_1\rangle, \dots, |g_{N-1}\rangle\}$ come segue:

$$|g_j\rangle \longrightarrow \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \chi_k^*(g_j) |g_k\rangle, \quad (16.14)$$

dimodoché su un arbitrario stato sia

$$\sum_{j=0}^{N-1} f(g_j) |g_j\rangle \longrightarrow \sum_{k=0}^{N-1} \tilde{f}(g_k) |g_k\rangle. \quad (16.15)$$

La QFT così definita è un operatore lineare ed unitario sullo spazio di Hilbert $\mathcal{H}$.

QFT in $\mathbb{Z}_2^n$ Prendiamo in esame il caso in cui $G = \mathbb{Z}_2^n$. Si può facilmente constatare che i 2^n caratteri sono

$$\chi_y(x) = (-1)^{x \cdot y}, \quad y \in \{0, 1\}^n, \quad (16.16)$$

dove $x \cdot y \equiv x_1 y_1 \oplus \dots \oplus x_n y_n$ è il prodotto scalare delle componenti binarie di x e y . La QFT (16.14) corrisponde alla trasformazione

$$|x\rangle \longrightarrow \frac{1}{\sqrt{2^n}} \sum_{y \in \{0, 1\}^n} (-1)^{x \cdot y} |y\rangle, \quad (16.17)$$

che possiamo facilmente identificare con la trasformata di Hadamard $H^{\otimes n}$ (si veda (16.2)).

QFT in $\mathbb{Z}_{2^n}$ Consideriamo ora $G = \mathbb{Z}_{2^n}$. In tal caso i 2^n caratteri risultano

$$\chi_y(x) = \exp \left[\frac{2\pi i x y}{2^n} \right], \quad y \in \{0, 1, \dots, 2^n - 1\}. \quad (16.18)$$

La QFT (16.14) corrisponde alla trasformazione

$$|x\rangle \longrightarrow \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} \exp \left[\frac{2\pi i x y}{2^n} \right] |y\rangle. \quad (16.19)$$

In questo caso però non è immediato capire come realizzare tale trasformazione. Vediamolo in dettaglio.

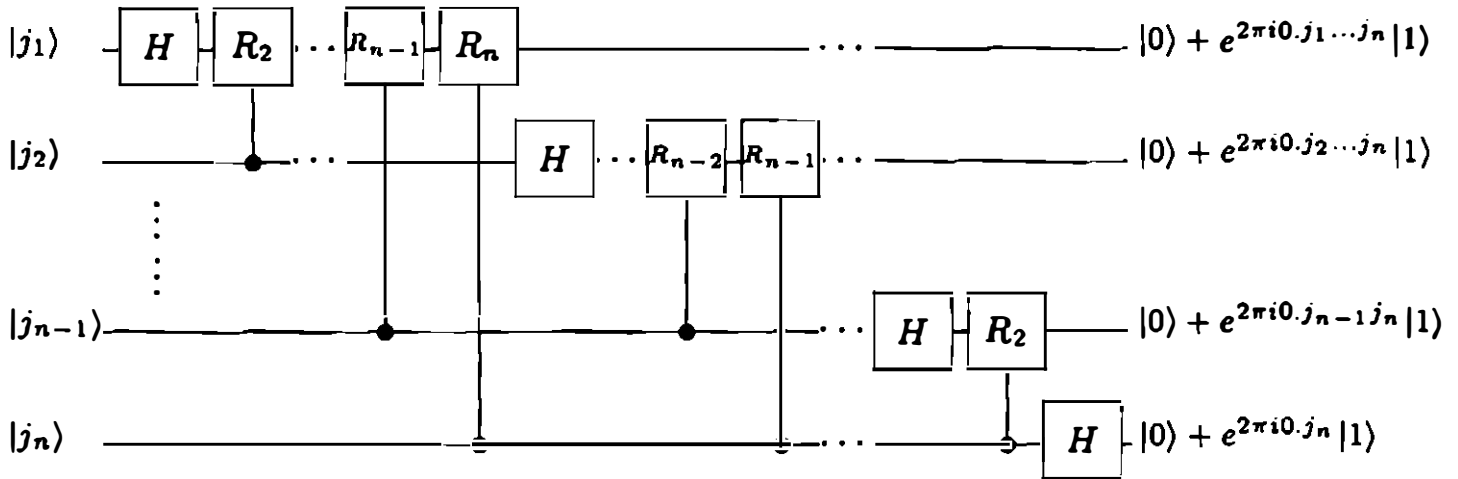
Sia $j_1 j_2 \dots j_n$ la rappresentazione binaria di $j \in \mathbb{Z}_{2^n}$ ($j \equiv \sum_{l=1}^n j_l 2^{n-l}$), introduciamo la *frazione binaria*

$$0.j_1 j_{l+1} \dots j_n \equiv \frac{j_l}{2} + \frac{j_{l+1}}{4} + \dots + \frac{j_n}{2^{n-l+1}}, \quad (16.20)$$

allora la (16.19) può essere riscritta come

$$\begin{aligned} |j\rangle &\longrightarrow \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} \exp \left(2\pi i j \frac{k}{2^n} \right) |k\rangle \\ &= \frac{1}{2^{n/2}} \sum_{k_1=0}^1 \dots \sum_{k_n=0}^1 \exp \left(2\pi i j \left(\sum_{l=1}^n k_l 2^{-l} \right) \right) |k_1 \dots k_n\rangle \\ &= \frac{1}{2^{n/2}} \sum_{k_1=0}^1 \dots \sum_{k_n=0}^1 \bigotimes_{l=1}^n \exp \left(2\pi i j \frac{k_l}{2^l} \right) |k_l\rangle \\ &= \frac{1}{2^{n/2}} \bigotimes_{l=1}^n \left[\sum_{k_l=0}^1 \exp \left(2\pi i j \frac{k_l}{2^l} \right) |k_l\rangle \right] \\ &= \frac{1}{2^{n/2}} \bigotimes_{l=1}^n \left[|0\rangle + e^{2\pi i j / 2^l} |1\rangle \right] \\ &= \frac{1}{2^{n/2}} (|0\rangle + e^{2\pi i 0.j_n} |1\rangle) (|0\rangle + e^{2\pi i 0.j_{n-1}j_n} |1\rangle) \dots \\ &\quad \dots (|0\rangle + e^{2\pi i 0.j_1 j_2 \dots j_n} |1\rangle). \end{aligned} \quad (16.21)$$

Consideriamo ora il *PHASE GATE* (15.15) con $\phi_k = 2\pi/2^k$, $k \in \mathbb{N}$ e poniamo $U_{\phi_k} \equiv R_k$. Allora, la QFT può essere implementata tramite il seguente circuito



La sequenza delle operazioni da sinistra verso destra implica le seguenti trasformazioni per il primo qubit:

$$\begin{aligned}
 |j_1 \dots j_n\rangle &\longrightarrow H \\
 \frac{1}{2^{1/2}} (|0\rangle + e^{2\pi i 0 \cdot j_1} |1\rangle) |j_2 \dots j_n\rangle &\longrightarrow CR_2 \\
 \frac{1}{2^{1/2}} (|0\rangle + e^{2\pi i 0 \cdot j_1 j_2} |1\rangle) |j_2 \dots j_n\rangle &\longrightarrow CR_3 \dots CR_n \\
 \frac{1}{2^{1/2}} (|0\rangle + e^{2\pi i 0 \cdot j_1 j_2 \dots j_n} |1\rangle) |j_2 \dots j_n\rangle &. \quad (16.22)
 \end{aligned}$$

La stessa procedura sul secondo qubit conduce a:

$$\frac{1}{2^{2/2}} (|0\rangle + e^{2\pi i 0 \cdot j_1 j_2 \dots j_n} |1\rangle) (|0\rangle + e^{2\pi i 0 \cdot j_2 \dots j_n} |1\rangle) |j_3 \dots j_n\rangle. \quad (16.23)$$

Continuando in questo modo per ogni qubit, arriviamo a:

$$\frac{1}{2^{n/2}} (|0\rangle + e^{2\pi i 0 \cdot j_1 j_2 \dots j_n} |1\rangle) (|0\rangle + e^{2\pi i 0 \cdot j_2 \dots j_n} |1\rangle) \dots (|0\rangle + e^{2\pi i 0 \cdot j_n} |1\rangle). \quad (16.24)$$

Quindi, con operazioni SWAP (non riportate per semplicità nel circuito) possiamo invertire l'ordine dei qubits per ottenere

$$\frac{1}{2^{n/2}} (|0\rangle + e^{2\pi i 0 \cdot j_n} |1\rangle) (|0\rangle + e^{2\pi i 0 \cdot j_{n-1} j_n} |1\rangle) \dots (|0\rangle + e^{2\pi i 0 \cdot j_1 j_2 \dots j_n} |1\rangle). \quad (16.25)$$

Per valutare la complessità dell'algoritmo usato per implementare la QFT, vediamo il costo in termini di porte elementari. Sul primo qubit agiscono un H e $(n-1)$ CR per un totale di n porte; sul secondo qubit agiscono un H e $(n-2)$ CR per un totale di $n-1$ porte; e così via fino ad arrivare all'ultimo qubit per un totale di $n(n+1)/2$ porte. Inoltre, sono richiesti $n/2$ $SWAP$ (ognuno dei quali si può facilmente mostrare essere realizzato con tre $CNOT$).

Pertanto il circuito mostrato (risalente al 1994) realizza la QFT con costo $O(n^2)$, risultando quindi efficiente. Si noti che in contrasto algoritmi classici per computare DFT su $\mathbb{Z}_{2^n}$ richiedono $O(n2^n)$ porte (è il caso ad esempio dell'algoritmo

noto come “Fast Fourier Transform”). Tuttavia, l'inconveniente della QFT è che non può essere usato come algoritmo fine a se stesso, poiché nel processo di misura si ottengono le probabilità e non le ampiezze di probabilità. Ciò nonostante la QFT costituisce un ingrediente fondamentale in altri algoritmi quantistici.

16.4 Algoritmo di fattorizzazione (Shor)

L'algoritmo quantistico per la decomposizione di un intero N in fattori primi dovuto a P. Shor (1994) fa uso della computazione quantistica per risolvere in maniera efficiente un problema equivalente.

Dato l'intero N , si scelga in maniera casuale un intero $y < N$ e primo con N [dunque $(y, N) = 1$].

Definizione Si definisce *ordine* di $y \pmod{N}$ il minimo esponente intero positivo r di y tale che

$$y^r = 1 \pmod{N}. \quad (16.26)$$

Nella terminologia della teoria dei gruppi, esso è l'ordine dell'elemento y nel gruppo moltiplicativo $\mathbb{Z}_N^*$. È noto che $r \leq N$. Si osservi come r sia anche il periodo della funzione

$$F_N(a) = y^a \pmod{N}, \quad (16.27)$$

ovvero $F_N(a+r) = F_N(a)$ per ogni $a \in \mathbb{Z}$ (si noti che a differenza del problema di Simon qui si ha una funzione $F_N : \mathbb{Z} \mapsto \mathbb{Z}_N$, poi in pratica limiteremo $\mathbb{Z}$ a $\mathbb{Z}_{2^n}$). Vediamo allora come il problema della fattorizzazione può essere ridotto a quello di trovare il periodo della (16.27), ovvero l'ordine di $y \pmod{N}$.

Consideriamo l'equazione quadratica $x^2 = 1 \pmod{N}$, allora $x = \pm 1 \pmod{N}$ rappresentano le soluzioni banali. Se N è un primo dispari, allora esse sono anche le uniche soluzioni. Tuttavia, se N è composto allora ci possono anche essere altre coppie di soluzioni non banali, diciamo $x = \pm s \pmod{N}$.

Esempio La congruenza $x^2 = 1 \pmod{91}$, ha soluzioni banali $x = \pm 1 \pmod{91}$ e soluzioni non banali $x = \pm 27 \pmod{91}$.

Supponiamo infatti per semplicità che $N = n_1 \times n_2$ con $(n_1, n_2) = 1$ e consideriamo i sistemi di congruenze

$$a) \begin{cases} x_1 = 1 \pmod{n_1} \\ x_1 = 1 \pmod{n_2} \end{cases}, \quad b) \begin{cases} x_2 = -1 \pmod{n_1} \\ x_2 = -1 \pmod{n_2} \end{cases}, \quad (16.28)$$

$$c) \begin{cases} x_3 = 1 \pmod{n_1} \\ x_3 = -1 \pmod{n_2} \end{cases}, \quad d) \begin{cases} x_4 = -1 \pmod{n_1} \\ x_4 = 1 \pmod{n_2} \end{cases}. \quad (16.29)$$

In tutti i casi si ha $x_i^2 = 1$ modulo n_1 e modulo n_2 , $i = 1, 2, 3, 4$; pertanto, ogni x_i soddisfa l'equazione $x^2 = 1 \pmod{N}$. D'altra parte il ricorso alla teoria dei

numeri e al *teorema del resto cinese* ci dice che ogni sistema di equazioni $a), b), c), d)$ ha un'unica soluzione modulo N . Da $a)$ e $b)$ otteniamo

$$x_1 = 1 \text{ e } x_2 = -1 \pmod{N} \quad (\text{soluzioni banali}),$$

mentre da $c)$ e $d)$ avremo altre due soluzioni (non banali)

$$x_3 = s \text{ e } x_4 = -s \pmod{N},$$

con $1 < s < N - 1$. Si avrà poi $s^2 - 1 = 0 \pmod{N}$, cioè

$$(s + 1)(s - 1) = 0 \pmod{N}, \quad (16.30)$$

e quindi $(N, s \pm 1)$ saranno fattori non banali di N perchè $0 < s \pm 1 < N$.

Esempio Nell'esempio precedente si è visto che $x^2 = 1 \pmod{91}$ ha soluzioni non banali $x = \pm 27 \pmod{91}$. Si noti allora che $(26, 91) = 13$ e $(28, 91) = 7$ sono fattori non banali di 91.

Supponiamo ora che r sia pari, allora ponendo

$$x = y^{r/2} \quad (16.31)$$

abbiamo $x^2 = 1 \pmod{N}$, quindi x è un candidato per una soluzione non banale dell'equazione $x^2 = 1 \pmod{N}$, che permette di trovare fattori di N tramite la (16.30).

Il suddetto processo può fallire se il valore scelto di y ha un ordine r dispari, o se, pur essendo r pari, $y^{r/2}$ risulta essere una soluzione banale dell'equazione $x^2 = 1 \pmod{N}$. Tuttavia, in generale vale il seguente:

Teorema 16.4.1 *Sia N intero dispari > 1 con fattorizzazione in numeri primi*

$$N = p_1^{\alpha_1} p_2^{\alpha_2} \dots p_k^{\alpha_k}. \quad (16.32)$$

Supponiamo che $y < N$ sia scelto in maniera casuale e che soddisfi la condizione $(y, N) = 1$. Sia r l'ordine di $y \pmod{N}$, allora

$$Pr_y \left(r \text{ pari e } y^{r/2} \neq \pm 1 \pmod{N} \right) \geq 1 - \frac{1}{2^{k-1}}. \quad (16.33)$$

Abbiamo escluso valori pari di N poiché tutti i fattori uguali a 2 in N sono facili da riconoscere. Lo stesso dicasi per le potenze di primi. In definitiva, se N non è della forma p^α o $2p^\alpha$, in base alla (16.33) possiamo affermare che $Pr_y \left(r \text{ pari e } y^{r/2} \neq \pm 1 \pmod{N} \right) \geq \frac{1}{2}$.

Inoltre, dal *teorema sui numeri primi* della teoria dei numeri, si ha che il numero di primi minori di N , $\pi(N)$, tende a $\frac{N}{\log N}$ per N sufficientemente grande. Mentre

il numero degli interi positivi primi con N minori di N , $\phi(N)$ (ϕ è la funzione di Eulero), soddisfa $\phi(N) \geq \pi(N)$. Ne deriva che la probabilità di trovare uno di tali interi è $\frac{\phi(N)}{N} > \frac{1}{\log N}$. Quindi, se y è scelto casualmente, con $1 \leq y \leq N$, si ha

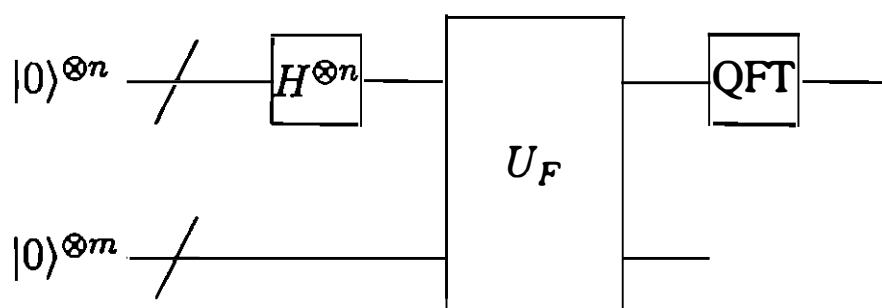
$$Pr_y((y, N) = 1) \geq \frac{1}{\log N}. \quad (16.34)$$

Pertanto, assumendo che si riesca a trovare in maniera efficiente r , otterremo un fattore non banale di N dal procedimento di cui sopra, con probabilità $\geq \frac{1}{2 \log N}$ come risulta dalla (16.33) e (16.34).

Esempio Fattorizzazione di $N = 91$.

Selezioniamo $y < 91$ tale che $(y, 91) = 1$. Sia $y = 3$, allora il periodo di $3^a \bmod 91$ è $r = 6$, sicché $3^6 = 1 \bmod 91$. Ponendo $x = y^{r/2} = 27$ si ottengono i due fattori come $(x - 1, N) = 13$ e $(x + 1, N) = 7$.

In ambito classico non esistono algoritmi efficienti per trovare l'ordine (né tanto meno per trovare il periodo). Shor ha invece elaborato un algoritmo quantistico probabilistico efficiente per questo problema (1994). Esso è schematizzato nel seguente circuito che ricalca quello dell'algoritmo di Simon (paragrafo 16.2) con la differenza di utilizzare la QFT in $\mathbb{Z}_{2^n}$ anziché in $\mathbb{Z}_2^n$ (o $H^{\otimes n}$), a causa della funzione F_N .



1. Dato N si scelga in maniera casuale $y < N$. Si consideri quindi un registro quantistico contenente n qubits tale che $N^2 < 2^n < 2N^2$. Sia il registro preparato nello stato $|0\rangle^{\otimes n}$. Applicando la porta Hadamard a tutti i qubits si ottiene

$$\frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} |a\rangle, \quad (16.35)$$

dove si è posto $2^n = q$.

2. Si valuta quindi la funzione $y^a \pmod{N}$, immagazzinando il risultato in un secondo registro (15.29), preventivamente inizializzato a $|0\rangle^{\otimes m}$ (m sia il numero dei qubits necessari per immagazzinare i valori di F_N)

$$\frac{1}{\sqrt{q}} \sum_{a=0}^{q-1} |a\rangle |y^a \pmod{N}\rangle. \quad (16.36)$$

Ciò può essere eseguito in maniera efficiente facendo ricorso ad algoritmi classici per l'esponenziazione modulare.

3. Si esegue poi una misura sulla base computazionale del secondo registro. Supponiamo che il risultato sia

$$z = y^l \pmod{N}, \quad l \leq r. \quad (16.37)$$

Ovviamente sarà anche

$$y^l = y^{jr+l} \pmod{N}, \quad \forall j \in \mathbb{N}. \quad (16.38)$$

Pertanto, la misura selezionerà i valori

$$a = l, l + r, l + 2r, \dots, l + Ar, \quad (16.39)$$

dove $A = \lfloor (q-l)/r \rfloor$ (il simbolo $\lfloor \bullet \rfloor$ sta al solito per il più grande intero minore di $\bullet$). Notare che $l \leq r$ e $q \sim O(N^2)$ implicano $A \sim q/r$.

Lo stato del primo registro dopo la misura risulterà essere

$$|\Psi_l\rangle = \frac{1}{\sqrt{A+1}} \sum_{j=0}^A |jr+l\rangle. \quad (16.40)$$

Consideriamo ora il semplice caso in cui $r|q$, cosicché $A = q/r - 1$ e

$$|\Psi_l\rangle = \sqrt{\frac{r}{q}} \sum_{j=0}^{\frac{q}{r}-1} |jr+l\rangle \equiv \sum_{a=0}^{q-1} f(a) |a\rangle, \quad (16.41)$$

con

$$f(a) = \begin{cases} \sqrt{r/q} & \text{per } (a-l) \text{ multiplo di } r, \\ 0 & \text{altrimenti.} \end{cases} \quad (16.42)$$

4. Eseguendo ora la QFT sul primo registro, otteniamo

$$|\Psi_l\rangle \mapsto \sum_{c=0}^{q-1} \tilde{f}(c) |c\rangle \quad (16.43)$$

dove $\tilde{f}(c)$ è la DFT di $f(a)$,

$$\tilde{f}(c) = \frac{\sqrt{r}}{q} \sum_{j=0}^{q/r-1} \exp \left[\frac{2\pi i(jr + l)c}{q} \right], \quad (16.44)$$

ovvero

$$\tilde{f}(c) = \begin{cases} \exp(2\pi i l c / q) / \sqrt{r} & \text{per } c \text{ multiplo di } q/r, \\ 0 & \text{altrimenti.} \end{cases} \quad (16.45)$$

Scrivendo $c = jq/r$ lo stato finale del primo registro diventa

$$\frac{1}{\sqrt{r}} \sum_{j=0}^{r-1} \exp \left(\frac{2\pi i l j}{r} \right) \left| j \frac{q}{r} \right\rangle. \quad (16.46)$$

5. A questo punto, una misura con proiezione sullo stato contrassegnato da c darà come risultato un multiplo $\lambda q/r$ con valori $\lambda = 0, \dots, r-1$ equiprobabili (è da notare come la traslazione iniziale l sia scomparsa come conseguenza della proprietà di invarianza traslazionale della QFT).

Come risultato della misura otteniamo dunque il valore c che soddisfa $c/q = \lambda/r$ (q è conosciuto). Se $(\lambda, r) = 1$, possiamo determinare r riducendo c/q ai minimi termini. Poiché il valore di λ è casuale, allora $Pr_{\lambda}((\lambda, r) = 1) \geq \frac{1}{\log r}$ per $r \rightarrow \infty$. Quindi la ripetizione della computazione $O(\log r) < O(\log N)$ volte garantisce una probabilità di successo tendente a 1 e di conseguenza una efficiente determinazione di r .

È da notare per confronto che i migliori algoritmi classici per la fattorizzazione richiedono risorse temporali dell'ordine di

$$\exp \left[(\log N)^{1/3} (\log \log N)^{2/3} \right].$$

Quindi, il problema della fattorizzazione, classicamente in NPI (Capitolo 8), con l'algoritmo di Shor si posiziona in BQP .

16.4.2 Il caso generale

Torniamo al caso generale in cui r non divide q , allora dalla (16.44) la probabilità di ottenere il valore c come risultato della misura è

$$Pr(c) = \frac{1}{q} \frac{1}{A+1} \left| \sum_{j=0}^A \exp \left(\frac{2\pi i j(r c \bmod q)}{q} \right) \right|^2. \quad (16.47)$$

Nel caso semplice precedente una interferenza costruttiva avveniva per c soddisfacente $rc \bmod q = 0$. Nella (16.47) cerchiamo quindi interferenza costruttiva

considerando c tale che $rc \pmod{q}$ sia sufficientemente piccolo. In tal caso, i termini nella somma saranno tutti raggruppati in un settore del cerchio unitario $\mathbb{S}^1$. Infatti, se ogni intero della classe di congruenza $rc \pmod{q}$ è compreso tra $-r/2$ ed $r/2$, condizione che per brevità scriveremo

$$-r/2 \leq rc \pmod{q} \leq r/2, \quad (16.48)$$

allora i termini nella (16.47) saranno distribuiti al massimo su di un semicerchio di $\mathbb{S}^1$. Ci sono esattamente r valori di $c \pmod{q}$ soddisfacenti l'equazione (16.48). Per vedere ciò consideriamo i multipli di q

$$0, q, 2q, \dots, rq,$$

ed i multipli cr di r

$$0, r, 2r, \dots, qr,$$

segnati su una stessa retta. I multipli di r sono spazati di r tra di loro, così, a ognuno degli r multipli di q , sarà associato esattamente un multiplo di r entro una distanza $\pm r/2$. Ciò dà le r soluzioni. Stimiamo dunque $Pr(c)$ per c soddisfacente (16.48). Scriviamo $\theta_c = 2\pi(rc \pmod{q})/q$, allora (16.47) rappresenta la serie geometrica di $e^{i\theta_c}$. Considerando i termini di tale serie come vettori in $\mathbb{C}$, possiamo vedere che la distanza totale dall'origine diminuisce all'aumentare di θ_c , pertanto la probabilità $Pr(c)$ relativa ad un generico θ_c sarà maggiore della stessa probabilità relativa al massimo di θ_c , ovvero

$$Pr(c) \geq Pr(c, \max \theta_c).$$

Tuttavia, $\max \theta_c \equiv \bar{\theta}_c = \pi r/q$, quindi

$$Pr(c) \geq \frac{1}{q} \frac{1}{A+1} \left| \sum_{j=0}^A \left(e^{i\bar{\theta}_c} \right)^j \right|^2 = \frac{1}{q} \frac{1}{A+1} \frac{\sin^2 [(A+1)\bar{\theta}_c/2]}{\sin^2 (\bar{\theta}_c/2)}, \quad (16.49)$$

che con l'approssimazione $A \approx q/r - 1$ diventa

$$Pr(c) \geq \frac{r}{q^2} \frac{1}{\sin^2 \frac{\pi r}{2q}} \approx \frac{4}{\pi^2} \frac{1}{r}, \quad (16.50)$$

per $r/q \rightarrow 0$. Poiché ci sono r di tali valori di c , la probabilità di osservare uno che soddisfa la (16.48) è maggiore di $4/\pi^2$.

Infine, vogliamo estrarre il valore di r dato il valore di c . Notiamo che la (16.48) implica

$$|rc - c'q| \leq \frac{r}{2} \quad (16.51)$$

per qualche $0 \leq c' \leq r-1$. Gli r differenti valori di c' sono associati con gli r possibili valori di c , cosicché

$$Pr(c') \geq \frac{4}{\pi^2} \frac{1}{r}. \quad (16.52)$$

Inoltre

$$\left| \frac{c}{q} - \frac{c'}{r} \right| \leq \frac{1}{2q}. \quad (16.53)$$

Qui c e q sono conosciuti, e $r \leq N$, $q \geq N^2$, pertanto, ci sarà esattamente una frazione c'/r con denominatore al massimo N che soddisfa (16.53). Essa può essere trovata in maniera efficiente come un convergente nell'espansione in frazioni continue di c/q nel senso che adesso spieghiamo.

Frazioni continue Una frazione continuata (finita, semplice) è una espressione della forma

$$a_0 + \frac{1}{a_1 + \frac{1}{a_2 + \frac{1}{\dots + \frac{1}{a_N}}}}. \quad (16.54)$$

dove $a_0, \dots, a_N$ sono interi positivi. Abbreviamo la (16.54) con la notazione

$$[a_0, \dots, a_N], \quad (16.55)$$

e definiamo l' n -esimo convergente di questa espressione come $[a_0, \dots, a_n]$ per $0 \leq n \leq N$. Possiamo scrivere l' n -esimo convergente come rapporto tra due interi P_n/Q_n , cioè $[a_0, \dots, a_n] = P_n/Q_n$, dove valgono le seguenti relazioni di ricorrenza

$$\begin{aligned} P_0 &= a_0, & P_1 &= a_1 a_0 + 1, & P_n &= a_n P_{n-1} + P_{n-2}, \\ Q_0 &= 1, & Q_1 &= a_1, & Q_n &= a_n Q_{n-1} + Q_{n-2}. \end{aligned} \quad (16.56)$$

I convergenti P_n/Q_n calcolati con questo metodo iterativo sono sempre primi tra loro, $(P_n, Q_n) = 1$. In pratica ogni numero razionale x (positivo per semplicità) può essere rappresentato tramite una frazione continuata. Il metodo è il seguente:

$$\begin{aligned} \text{poniamo } a_0 &= [x], & x &= a_0 + \xi_0, & 0 &\leq \xi_0 \leq 1; \\ \text{se } \xi_0 \neq 0 &\text{poniamo } a_1 &= [1/\xi_0], & 1/\xi_0 &= a_1 + \xi_1, & 0 &\leq \xi_1 \leq 1; \\ \text{se } \xi_1 \neq 0 &\text{poniamo } a_2 &= [1/\xi_1], & 1/\xi_1 &= a_2 + \xi_2, & 0 &\leq \xi_2 \leq 1; \text{ etc.} \end{aligned} \quad (16.57)$$

Relativamente alle frazioni continue, siamo interessati in particolare al seguente:

Teorema 16.4.3 Sia P/Q un numero razionale che soddisfa

$$\left| x - \frac{P}{Q} \right| < \frac{1}{2Q^2}. \quad (16.58)$$

Allora P/Q è un convergente della frazione continuata di x .

Questo teorema può essere direttamente applicato per trovare la frazione c'/r nella (16.53), dove il valore conosciuto c/q gioca il ruolo di x . Pertanto, se $(c', r) = 1$, otterremo il valore desiderato di r .

Esempio Con riferimento agli esempi precedenti in cui $N = 91$ ed $y = 3$, supponiamo che sia $L = 14$ dimodoché $N^2 \leq q \leq 2N^2$ con $q = 2^{14} = 16384$. Allora, dalla (16.43) possiamo ottenere come risultato della misura $0 \leq c \leq 16383$ con probabilità data dalla (16.47). Supponiamo che sia $c = 13453$. A questo punto, si considera l'espansione in frazioni continue di c/q come dalla (16.56) e ad ogni passo si determina un convergente in base alle relazioni (16.57). Ad ogni passo si verifica anche se il valore Q_n corrispondente all' n -esimo convergente è il desiderato r , si valuta cioè $F_N(Q_n)$ e se $\neq 1 \pmod{N}$ si procede al passo successivo. Nel nostro caso specifico:

$$\begin{aligned} n = 0, a_0 &= \left\lfloor \frac{13453}{16384} \right\rfloor = 0, \xi_0 = \frac{13453}{16384}, Q_0 = 1, y^{Q_0} = 3 \neq 1 \pmod{91}; \\ n = 1, a_1 &= \left\lfloor \frac{16384}{13453} \right\rfloor = 1, \xi_1 = \frac{2931}{13453}, Q_1 = 1, y^{Q_1} = 3 \neq 1 \pmod{91}; \\ n = 2, a_2 &= \left\lfloor \frac{13453}{2931} \right\rfloor = 4, \xi_2 = \frac{1729}{2931}, Q_2 = 5, y^{Q_2} = 61 \neq 1 \pmod{91}; \\ n = 3, a_3 &= \left\lfloor \frac{2931}{1729} \right\rfloor = 1, \xi_3 = \frac{1202}{1729}, Q_3 = 6, y^{Q_3} = 1 \pmod{91}. \end{aligned}$$

Pertanto $Q_3 = r$ e quindi la procedura si ferma. Si noti che l'uso delle frazioni continue per tale scopo costituisce un algoritmo classico efficiente.

Il procedimento fallisce se $(c', r) \neq 1$, tuttavia ci sono $\phi(r)$ valori c' primi con r , pertanto si ha

$$Pr_{c'}(c' \text{ primo con } r) \geq \frac{4}{\pi^2} \frac{\phi(r)}{r}. \quad (16.59)$$

Per valori sufficientemente grandi di r , sappiamo che $\frac{\phi(r)}{r} > \frac{1}{\log r} > \frac{1}{\log N}$, cosicché, con probabilità maggiore di $4/(\pi^2 \log N)$ otterremo r (ovvero un fattore di N). Ripetendo questa procedura probabilistica $O(\log N)$ volte avremo un algoritmo di fattorizzazione efficiente.

Vale la pena infine sottolineare che l'algoritmo di Shor ha pesanti ripercussioni sulla crittografia, poiché i più comuni protocolli di comunicazione basano la loro sicurezza sulla difficoltà di risolvere il problema della fattorizzazione o problemi analoghi (ad esempio "Discrete Log") per i quali comunque si sono individuati algoritmi quantistici in BQP . Va da sé quindi che la eventuale implementazione di tali algoritmi porterebbe ad una facile rottura degli attuali codici crittografici. Tuttavia, la meccanica quantistica fornisce anche la possibilità di ovviare a tale problema mediante la cosiddetta *crittografia quantistica*, ma la discussione di tale argomento esula degli scopi di questo libro.

16.4.4 Il problema del sottogruppo nascosto

Abbiamo appena visto come il problema dell'ordine (o del periodo in $\mathbb{Z}_{2^n}$) può essere risolto in maniera efficiente in ambito quantistico (ammette un guadagno esponenziale rispetto al caso classico). Lo stesso accadeva per il problema delle funzioni costanti o bilanciate, o per il periodo in $\mathbb{Z}_2^n$. Essi sono particolari tipi di problemi conosciuti come *problemi con promessa* (*promise problems*), ovvero problemi legati alla valutazione di una funzione per la quale si hanno delle informazioni a priori (promesse), ad esempio che è periodica, oppure che è costante o bilanciata, etc. È pertanto possibile riformulare tali problemi in una maniera più generale che permetta l'utilizzo di strumenti matematici più potenti come la teoria dei gruppi.

Definizione del problema del sottogruppo nascosto (*hidden subgroup problem*)

Dati in Input:

un gruppo finito G ed una funzione $f : G \rightarrow X$ dove X è un insieme finito;

ed una **Promessa:**

esiste un sottogruppo non banale $K \subseteq G$ tale che f è costante su ogni classe laterale di K (ovvero su ogni sottinsieme $g + K$ con $g \in G$ e G inteso in senso additivo) ed assume valori distinti su classi laterali di K distinte;

determinare in **Output:**

un insieme di generatori di K (ovvero lo stesso sottogruppo K).

Ad esempio nel problema delle funzioni costanti o bilanciate si ha $G = \mathbb{Z}_2^n$ con l'operazione $\oplus$, $X = \{0, 1\}$ e $K \subseteq \mathbb{Z}_2^n$: $|K| = |\mathbb{Z}_2^n|$ o $|K| = |\mathbb{Z}_2^n|/2$.

Invece, nel problema di Simon si ha $G = \mathbb{Z}_2^n$ con operazione $\oplus$, $X = \mathbb{Z}_2^n$ e $K = \{0, p\}$.

Infine, nel problema dell'ordine si ha $G = \mathbb{Z}_q$ con l'operazione $+(mod\ q)$, $X = \{y^a (mod\ N)\}$ con $a \in \mathbb{Z}_r$ e $r \in G$ tale che $y^r = 1 (mod\ N)$, quindi $K = \{0, r, 2r, \dots\}$.

Nel caso di gruppi G Abeliani (come quelli degli esempi sopracitati) si dimostra che il problema del sottogruppo nascosto ammette una soluzione (algoritmo quantistico) efficiente. In generale l'algoritmo quantistico coinvolge in sequenza operazioni del tipo *HADAMARD*, valutazione di funzione (ovvero uso di un oracolo $\mathcal{O}$), e QFT (si osservino gli schemi dei paragrafi 16.1, 16.2, 16.4 e si ricordi quanto discusso nel paragrafo 16.3).

È invece tuttora aperta la questione dell'esistenza di algoritmi quantistici efficienti per la risoluzione del problema del sottogruppo nascosto per gruppi non Abeliani (ciò è di estremo interesse poiché, ad esempio, il problema dell'isomorfismo dei grafi può essere ricondotto al problema del sottogruppo nascosto per gruppi non Abeliani).

16.5 Algoritmo di ricerca (Grover)

Supponiamo di voler cercare M elementi su di uno spazio (database) non strutturato contenente N elementi ($1 \leq M \leq N$). Piuttosto che cercare gli elementi

direttamente, concentriamoci su degli indici $0, \dots, N-1$ (assumeremo $N = 2^n$). Il problema può pertanto essere rappresentato tramite una funzione f che abbia in input $x \in \{0, \dots, N-1\}$, e soddisfi $f(x) = 1$ se x è soluzione (indice corrispondente ad un elemento cercato) ed $f(x) = 0$ se x non è soluzione (indice non corrispondente ad un elemento cercato).

Supponiamo di avere a disposizione un oracolo $\mathcal{O}$ che sappia riconoscere le soluzioni del problema (supponiamo di saper valutare la funzione predetta). Classicamente avremo bisogno di $O(N/M)$ chiamate dell'oracolo per risolvere il problema. Cosa accade quantisticamente? Ricordiamo dall'algoritmo di Deutsch-Jozsa (16.3)-(16.4) che l'azione dell'oracolo può essere sintetizzata come

$$|x\rangle \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right) \xrightarrow{\mathcal{O}} (-1)^{f(x)} |x\rangle \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right). \quad (16.60)$$

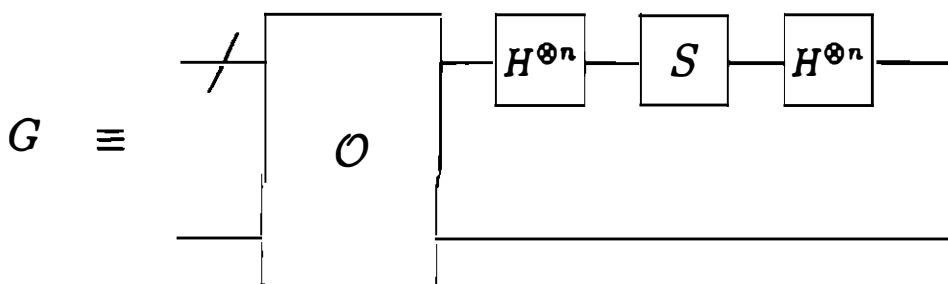
Poiché il qubit dell'oracolo non cambia possiamo ometterlo e scrivere

$$|x\rangle \xrightarrow{\mathcal{O}} (-1)^{f(x)} |x\rangle. \quad (16.61)$$

L'algoritmo quantistico di ricerca consiste nella ripetuta applicazione di una subroutine G conosciuta come *operatore di Grover* dal nome del suo inventore (1996). L'algoritmo inizia con un registro di n qubits inizializzato in $|0\rangle^{\otimes n}$, al quale viene applicata la trasformata di Hadamard $H^{\otimes n}$, ottenendo

$$|\psi\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle, \quad (16.62)$$

quindi G è applicato ripetutamente. L'operatore G è schematizzato nel seguente circuito (la linea in basso rappresenta il qubit dell'oracolo o messo nei calcoli)



L'azione di G può essere suddivisa come segue:

1. Applicazione dell'oracolo $\mathcal{O}$ tale che

$$|x\rangle \rightarrow (-1)^{f(x)} |x\rangle.$$

2. Applicazione di $H^{\otimes n}$.

3. Realizzazione di uno sfasamento condizionato S tale che

$$|x\rangle \rightarrow e^{i(1-\delta_{x,0})\pi} |x\rangle.$$

Come operazione condizionata su n qubits questa può essere implementata con $O(n)$ porte elementari (sezione 15.4). La corrispondente trasformazione unitaria è data da $2|0\rangle\langle 0| - \mathbb{I}$, infatti

$$(2|0\rangle\langle 0| - \mathbb{I}) |x\rangle = 2\langle 0|x\rangle|0\rangle - |x\rangle = \begin{cases} |0\rangle & x = 0 \\ -|x\rangle & x \neq 0 \end{cases}.$$

4. Applicazione di $H^{\otimes n}$.

L'effetto combinato dei punti 2, 3, 4 può essere riassunto come

$$H^{\otimes n} (2|0\rangle\langle 0| - \mathbb{I}) H^{\otimes n} = 2|\psi\rangle\langle\psi| - \mathbb{I}. \quad (16.63)$$

Pertanto

$$G \equiv (2|\psi\rangle\langle\psi| - \mathbb{I}) \mathcal{O}. \quad (16.64)$$

16.5.1 Interpretazione geometrica

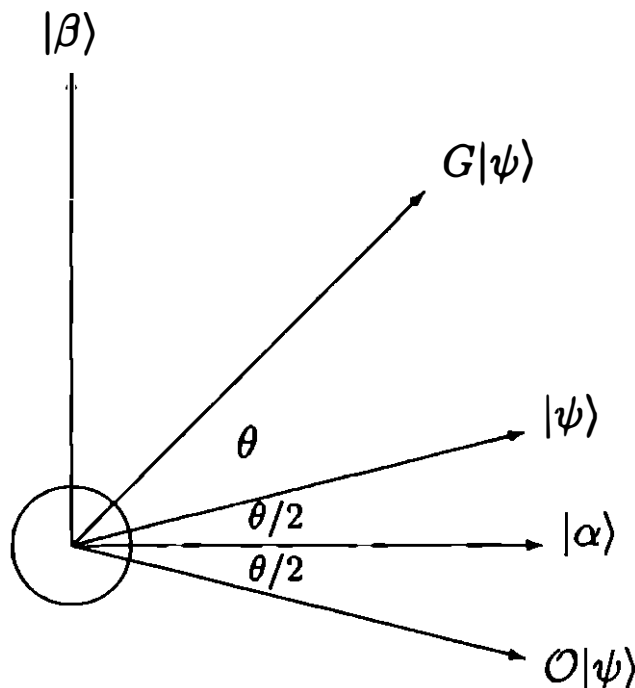
Denotiamo con $\sum'_x$ la somma su tutti i valori di x soluzioni del problema e $\sum''_x$ la somma su tutti i valori di x che non sono soluzioni. Definiamo quindi

$$|\alpha\rangle \equiv \frac{1}{\sqrt{N-M}} \sum''_x |x\rangle, \quad |\beta\rangle \equiv \frac{1}{\sqrt{M}} \sum'_x |x\rangle, \quad (16.65)$$

allora, dalla (16.62)

$$|\psi\rangle = \sqrt{\frac{N-M}{N}} |\alpha\rangle + \sqrt{\frac{M}{N}} |\beta\rangle, \quad (16.66)$$

cioè lo stato iniziale del computer vive in uno spazio generato da due soli vettori ortonormali, $|\alpha\rangle$ e $|\beta\rangle$. Siccome $\bullet|\alpha\rangle = |\alpha\rangle$ e $\bullet|\beta\rangle = -|\beta\rangle$, l'oracolo rappresenta una riflessione rispetto al vettore $|\alpha\rangle$. Analogamente, $2|\psi\rangle\langle\psi| - \mathbb{I}$ rappresenta una riflessione rispetto a $|\psi\rangle$. Pertanto, G sarà la composizione di due riflessioni, ovvero una rotazione nel piano $\{|\alpha\rangle, |\beta\rangle\}$ come mostrato dalla seguente figura.



Ponendo

$$\cos \frac{\theta}{2} = \sqrt{\frac{N-M}{N}} \quad (16.67)$$

e quindi

$$|\psi\rangle = \cos \frac{\theta}{2} |\alpha\rangle + \sin \frac{\theta}{2} |\beta\rangle, \quad (16.68)$$

si ha

$$G^k |\psi\rangle = \cos \left(\frac{2k+1}{2} \theta \right) |\alpha\rangle + \sin \left(\frac{2k+1}{2} \theta \right) |\beta\rangle, \quad (16.69)$$

e dunque $G^k |\psi\rangle$ rimane nello spazio generato da $|\alpha\rangle, |\beta\rangle \forall k \in \mathbb{N}$.

La performance dell'algoritmo può essere stabilita considerando quante volte G deve essere ripetuto affinché $|\psi\rangle$ venga ruotato il più vicino possibile a $|\beta\rangle$. Ruotando $|\psi\rangle$ di $\arccos(\sqrt{M/N})$ esso andrà a coincidere esattamente con $|\beta\rangle$. Allora, ripetendo G un numero di volte pari a

$$R = \left\lceil \frac{\arccos(\sqrt{M/N})}{\theta} \right\rceil, \quad (16.70)$$

(il simbolo $\lceil \bullet \rceil$ sta al solito per l'intero più piccolo maggiore di $\bullet$), si ruoterà $|\psi\rangle$ entro un angolo $\theta/2 \leq \pi/4$ attorno a $|\beta\rangle$. Ne consegue che una misura sulla base computazionale darà la soluzione cercata con probabilità $\geq 1/2$. Inoltre, M molto minore di N implica $\theta \simeq \sin \theta \simeq 2\sqrt{M/N}$, e quindi un errore $\theta/2 \simeq \sqrt{M/N}$ con probabilità $\simeq M/N$. Infine, dalla (16.70) $R \leq \lceil \pi/2\theta \rceil$ ed assumendo $M < N/2$ si ha

$$\frac{\theta}{2} \geq \sin \frac{\theta}{2} = \sqrt{\frac{M}{N}} \implies R \leq \left\lceil \frac{\pi}{4} \sqrt{\frac{N}{M}} \right\rceil. \quad (16.71)$$

In conclusione, $O(\sqrt{N/M})$ chiamate dell'oracolo devono essere fatte per ottenere l'esatta soluzione con alta probabilità. Questo risultato rappresenta un miglioramento di tipo quadratico rispetto al caso classico ($O(\sqrt{N/M})$ versus $O(N/M)$).

L'algoritmo Grover così come presentato fallisce qualora sia $M \geq N/2$. In generale non si è interessati a tale situazione, tuttavia si può ovviare a questo problema usando una versione modificata dell'algoritmo. Essa consiste nel raddoppiare il numero degli elementi nello spazio di ricerca aggiungendo altri N elementi nessuno dei quali sia soluzione del problema. Questo può essere fatto aggiungendo semplicemente un qubit nel registro che rappresenta gli indici di ricerca ($n \rightarrow n+1 \rightarrow 2^{n+1} = 2N$). Un nuovo oracolo $\mathcal{O}'$ può essere quindi costruito in maniera da marcare le soluzioni mentre l'extra bit viene posto a zero.

Abbiamo appena visto che l'algoritmo di Grover permette un guadagno polinomiale (quadratico) rispetto al caso classico, per il problema della ricerca in un insieme non strutturato. Dato che gli algoritmi quantistici visti nelle Sezioni 16.1 e 16.4 permettono un guadagno esponenziale rispetto alle attuali conoscenze classiche, ci si può chiedere se anche il problema della ricerca non ammetta un guadagno migliore in ambito quantistico, magari anch'esso di tipo esponenziale. Tale questione è di fondamentale importanza per la risoluzione dei problemi NP -completi. Infatti, questi ultimi possono essere rifrasiati in termini di esistenza di una soluzione per un problema di ricerca.

Se un problema NP -completo è specificato da n -bit di input, allora esso ammette testimoni che possono essere specificati con $p(n)$ bits, dove $p(n)$ è un qualche polinomio in n . La ricerca attraverso tutti i $2^{p(n)}$ possibili testimoni rivelerà la soluzione del problema. Si noti che non avendo alcuna informazione a priori sullo spazio dei testimoni esso è assimilabile ad un database non strutturato.

Supponiamo che il problema ammetta una sola soluzione. È ovvio che con l'algoritmo di Grover avremmo un guadagno del tipo $O(2^{p(n)}) \rightarrow O(2^{p(n)/2})$, ovvero permane la difficoltà esponenziale. Se invece il guadagno nell'algoritmo di ricerca fosse esponenziale avremmo $O(2^{p(n)}) \rightarrow O(p(n))$, che significherebbe aver ridotto la classe NP in BQP !

Sfortunatamente vedremo che il guadagno quadratico dell'algoritmo di Grover è ottimale, ovvero la ricerca quantistica su N elementi non può usare meno di $\Omega(\sqrt{N})$ queries.

16.6 I limiti della computazione quantistica

Abbiamo descritto il problema della ricerca di un elemento su un insieme non strutturato di N elementi come il problema di trovare un intero x ad n -bit ($N = 2^n$) tale che per la funzione $f : \{0, 1\}^n \rightarrow \{0, 1\}$ sia $f(x) = 1$. Equivalentemente potremmo considerare il problema decisionale dell'esistenza o meno di x tale che $f(x) = 1$. Questo problema può essere espresso come il calcolo della funzione Booleana $F(X)$ che applica OR a una sequenza $X \equiv (X_0, X_1, \dots, X_{N-1})$ dove $X_k \equiv f(k)$ con $k = 0, 1, \dots, N-1$ (e quindi computa $OR(X_0, OR(X_1, \dots))$). Più in generale potremmo considerare una qualsiasi funzione Booleana e chiederci, dato un oracolo per il calcolo di f , quante queries necessitano per la valutazione di F ?

Consideriamo ad esempio il problema *PARITY*: determinare la parità di una funzione $f : \{0, 1\}^n \rightarrow \{0, 1\}$, ovvero calcolare $F(X) = X_0 \oplus X_1 \oplus \dots \oplus X_{N-1}$. Per la risoluzione di tale problema si va dalle N queries classiche alle $N/2$ queries quantistiche (essenzialmente nessun guadagno).

In generale, si può mostrare che nel calcolo di funzioni Booleane, gli algoritmi quantistici possono solo apportare un guadagno di tipo polinomiale (nel migliore dei casi) rispetto agli algoritmi classici (o più in generale nessun guadagno).

All'uopo consideriamo la complessità di algoritmi che coinvolgono chiamate di un oracolo.

Definizione La complessità deterministica $D(F)$ è il numero minimo di chiamate dell'oracolo di cui necessita un computer classico per calcolare F con certezza. $Q(F)$ è l'equivalente quantistico di $D(F)$, ovvero il numero minimo di ricorsi all'oracolo di cui necessita un computer quantistico per calcolare F con certezza. Considerata la natura intrinsecamente probabilistica di un computer quantistico vale la pena definire anche $\tilde{Q}(F)$ come il numero minimo di chiamate dell'oracolo di cui necessita un computer quantistico per calcolare F con probabilità $> \frac{2}{3}$.

Vale ovviamente la seguente relazione: $\tilde{Q}(F) \leq Q(F) \leq D(F) \leq N$. Inoltre, è nota dalla teoria classica la possibilità di rappresentare funzioni Booleane per mezzo di polinomi.

Esempio Possiamo definire un polinomio $p : \mathbb{R}^N \rightarrow \mathbb{R}$ che rappresenti F come

$$p(X) = \sum_{Y \in \{0,1\}^N} F(Y) \prod_{k=0}^{N-1} [1 - (Y_k - X_k)^2]. \quad (16.72)$$

Definizione Un polinomio $p : \mathbb{R}^N \rightarrow \mathbb{R}$ rappresenta F se $p(X) = F(X) \forall X \in \{0,1\}^N$. Il grado minimo di tale rappresentazione di F si indica con $\deg(F)$.

Ad esempio, è noto che $\deg(OR) = \deg(AND) = \deg(PARITY) = N$. Inoltre, vale la relazione

$$D(F) \leq 2 [\deg(F)]^4. \quad (16.73)$$

Definizione Un polinomio $p : \mathbb{R}^N \rightarrow \mathbb{R}$ approssima F se $|p(X) - F(X)| \leq \frac{1}{3} \forall X \in \{0,1\}^N$. Il grado minimo di tale rappresentazione di F si indica con $\widetilde{\deg}(F)$.

Ad esempio, è noto che $\widetilde{\deg}(OR) = \widetilde{\deg}(AND) = \Theta(\sqrt{N})$ e $\widetilde{\deg}(PARITY) = N$. Inoltre, vale la relazione

$$D(F) \leq 216 [\widetilde{\deg}(F)]^6. \quad (16.74)$$

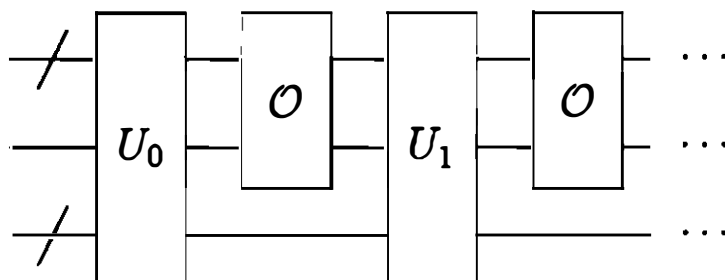
Si può ora enunciare il seguente

Teorema 16.6.0.1 Siano $f : \{0,1\}^n \rightarrow \{0,1\}$ una arbitraria funzione per la quale si dispone di un oracolo e Q un algoritmo (circuit) quantistico che calcola una funzione Booleana $F(X)$ con $X \equiv \{X_0, X_1, \dots, X_{N-1}\}$, $N = 2^n$ ed $X_k \equiv f(k)$. Si ha:

- se Q calcola F con probabilità 1 e con T queries, allora $T \geq \deg(F)/2$;
- se Q calcola F con probabilità $\geq \frac{2}{3}$ e con T queries, allora $T \geq \widetilde{\deg}(F)/2$.

Dimostrazione. Possiamo pensare di realizzare Q come nello schema seguente, usando tre registri: il primo ad n qubits con cui si interroga l'oracolo, il secondo

ad un singolo qubit su cui l'oracolo lascia il suo responso, ed il terzo contenente qubits addizionali sui quali non interviene l'oracolo. L'algoritmo Q può essere considerato come la sequenza di operazioni $U_T \mathcal{O} U_{T-1} \mathcal{O} \dots U_1 \mathcal{O} U_0$ dove $\mathcal{O}$ è l'oracolo e le U_k sono generiche trasformazioni unitarie. Assumiamo inoltre che Q dia il valore di F tramite il risultato della misura di un qubit del terzo registro, ad esempio l'ultimo (la scelta è del tutto arbitraria).



Possiamo scrivere lo stato in input alla prima chiamata dell'oracolo come

$$|\psi_0\rangle = \sum_{ik} (a_{i0k}|i\rangle|0\rangle + a_{i1k}|i\rangle|1\rangle) |k\rangle, \quad (16.75)$$

dove gli indici si riferiscono al primo, secondo e terzo registro rispettivamente. Dopo la risposta dell'oracolo $|\psi_0\rangle$ si trasforma in

$$|\psi_1\rangle = \sum_{ik} (a_{i0k}|i\rangle|X_i\rangle + a_{i1k}|i\rangle|X_i \oplus 1\rangle) |k\rangle \quad (16.76)$$

$$= \sum_{ik} [((1 - X_i) a_{i0k} + X_i a_{i1k}) |i\rangle|0\rangle + ((1 - X_i) a_{i1k} + X_i a_{i0k}) |i\rangle|1\rangle] |k\rangle.$$

Si noti che in $|\psi_0\rangle$ le ampiezze sono di grado 0 in X , mentre in $|\psi_1\rangle$ esse sono di grado 1. Quindi possiamo arguire che ad ogni chiamata dell'oracolo il loro grado aumenta al più di uno. Viceversa, ogni operazione unitaria che Q fa prima o dopo la chiamata dell'oracolo non cambia il grado del polinomio. Concludiamo che dopo T chiamate le ampiezze saranno polinomi di grado al più T .

Scriviamo l'output di Q dopo T chiamate dell'oracolo come $\sum_{i,j,k} c_{ijk} |i\rangle|j\rangle|k\rangle$. Allora, le ampiezze c_{ijk} sono polinomi di grado al più T nelle variabili $X_0, X_1, \dots, X_{N-1}$. Come conseguenza, la misura nella base computazionale produrrà un risultato ijk con probabilità $P_{ijk}(X) = |c_{ijk}|^2$ che sarà un polinomio di grado al più $2T$. La probabilità totale $P(X)$ di ottenere uno come output dell'algoritmo (quindi sull'ultimo qubit del terzo registro) sarà $\sum_{i,j} P_{ij1}(X)$ e quindi anch'essa di grado massimo $2T$.

Nel caso in cui Q produce la risposta corretta con certezza, sarà $P(X) = F(X)$ e quindi $2T \geq \deg(F)$. Invece nel caso in cui Q dà una risposta con probabilità di errore limitata, $P(X)$ approssima $F(X)$ e quindi si ha $2T \geq \widetilde{\deg}(F)$. $\square$

Nel caso del problema OR , il teorema 16.6.0.1 sancisce che il numero di chiamate dell'oracolo deve essere $\geq \Theta(\sqrt{N})$, da cui segue l'ottimalità dell'algoritmo di Grover.

Inoltre, dal teorema 16.6.0.1 si evincono immediatamente le relazioni $Q(F) \geq \deg(F)/2$ e $\tilde{Q}(F) \geq \widetilde{\deg}(F)/2$, che combinate con (16.73) e (16.74) danno

$$Q(F) \geq \left[\frac{D(F)}{32} \right]^{1/4}, \quad \tilde{Q}(F) \geq \left[\frac{D(F)}{13824} \right]^{1/6}. \quad (16.77)$$

Esse mostrano che nel calcolo di funzioni Booleane mediante *black box* gli algoritmi quantistici possono solo apportare un guadagno di tipo polinomiale nel migliore dei casi (e neanche questo è generalmente possibile come accade per il problema *PARITY*). Ciò è dovuto essenzialmente al fatto che non si hanno ulteriori informazioni sulla struttura della funzione valutata dall'oracolo a differenza di quanto accadeva per i *promise problems*.

16.7 Conclusioni

Lo sviluppo degli algoritmi descritti in questo capitolo ha inserito a tutti gli effetti il campo della computazione quantistica nella scienza dei computer. Tuttavia, non era inizialmente chiaro se la computazione quantistica fosse un modello realizzabile fisicamente. Particolarmente preoccupante era il fatto che in natura gli effetti quantistici sono raramente osservabili a causa dei processi di rumore. Questi ultimi, infatti, tendono a distruggere rapidamente le relazioni di fase nelle sovrapposizioni (vedere ad esempio (14.13)), fenomeno che va sotto il nome di *decoerenza*. Per poter ovviare a tale problema si è sviluppata una versione quantistica della teoria della correzione degli errori². Si è mostrato che, sotto opportune condizioni, una computazione quantistica “fault-tolerant” è possibile (1996). Tra le assunzioni vi sono:

- i) la possibilità di individuare uno spazio di Hilbert per il registro quantistico di computazione;
- ii) la possibilità di inizializzare lo stato del registro;
- iii) la possibilità di realizzare su tale registro trasformazioni unitarie che appartengono ad un insieme di porte universali;
- iv) la possibilità di effettuare misure quantistiche sulla base computazionale di tale registro;
- v) la possibilità di realizzare ogni operazione logica con un errore al di sotto di una certa soglia (che in generale è molto piccola, dipendente dal codice di correzione degli errori adottato, ma comunque finita).

Come conseguenza di tale profondo risultato, oggi numerosi sono gli sforzi a livello sperimentale votati alla realizzazione di computers quantistici in una varietà di

²La teoria della correzione degli errori (o teoria dei codici) è ampiamente utilizzata nei computers classici che non sono immuni da errori (bit flip), ed è basata essenzialmente sulla ridondanza, cioè sull'uso di più bit fisici per rappresentare un solo bit logico. Ciò permette, in caso di errore che interviene su di un bit fisico, di non perdere l'informazione sul bit logico come avverrebbe se ci fosse una corrispondenza uno ad uno tra bit fisico e bit logico.

sistemi fisici. La fisica atomica, molecolare ed ottica consente la manipolazione ed il controllo di sistemi quantistici individuali quali elettroni, ioni, atomi neutri o fotoni con i quali si stanno realizzando i primi esempi di porte logiche quantistiche. In particolare, la consolidata tecnica della risonanza magnetico-nucleare ha già permesso il conseguimento di promettenti risultati, come l'implementazione dell'algoritmo di Shor per la fattorizzazione del numero 15 (2003). Tali sistemi sono prototipi elementari ed hanno il problema della scalabilità, cioè dell'estensione ad un arbitrario numero di qubits. Problema che si pensa di risolvere con il ricorso a sistemi di materia condensata, ovvero una versione molto più sofisticata degli attuali dispositivi a stato solido.

Possiamo concludere dicendo che la computazione quantistica rappresenta la tecnologia del "day after tomorrow", ma già oggi costituisce un sapere imprescindibile per chi si inoltra nell'informatica teorica.

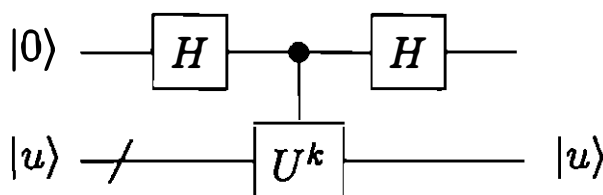
16.8 Esercizi

1. In un circuito che implementa l'algoritmo di Deutsch-Jozsa si consideri $w \in \{0, 1\}^n$ e la funzione $f : \{0, 1\}^n \rightarrow \{0, 1\}$ tale che

$$f(x) = w \cdot x \equiv \sum_{j=0}^{n-1} w_j x_j \pmod{2},$$

con $w_j, x_j \in \{0, 1\}$ componenti binarie di w e x rispettivamente.

- i) Determinare se la funzione f è costante, bilanciata o di altro tipo.
 - ii) Quali stati $|\phi\rangle$ si ottengono con certezza quando si misurano gli n qubits del primo registro ?
 - iii) Può l'algoritmo essere utile per determinare il valore $w \in \{0, 1\}^n$?
2. Data una lista di quattro elementi disordinati ed equiprobabili $\mathcal{L} = \{0, 1, 2, 3\}$ si usi l'algoritmo di Grover per trovare l'elemento $x_0 = 3$. In particolare, si consideri un registro di 2 qubits.
 - i) Calcolare gli operatori di riflessione che compongono l'operatore di Grover.
 - ii) Fornire la rappresentazione matriciale di G nella base computazionale.
 - iii) Usare G in una sovrapposizione di tutti gli stati della lista $\mathcal{L}$ aventi la stessa ampiezza di probabilità.
 - iv) Quante iterazioni m dell'operatore G sono necessarie per ottenere il desiderato elemento $x_0 = 3$? Qual è la probabilità di ottenerlo ?
 3. Si consideri il circuito quantistico



dove $|u\rangle$ è un autostato di U con autovalore $e^{2\pi i\varphi}$. Calcolare la probabilità di ottenere “0” come risultato della misura sul qubit del primo registro. Mostrare che il numero di applicazioni ripetute del circuito ed il valore di $k \in \mathbb{N}$ determinano la precisione con cui si può valutare φ (ovvero l’autovalore di U).

Riferimenti bibliografici L’algoritmo di Deutsch-Jozsa è stato presentato in [33], mentre quello di Simon in [100]. Le nozioni di teoria dei gruppi e la trasformata di Fourier sui gruppi si possono trovare in [41]. L’algoritmo quantistico per la trasformata di Fourier è stato introdotto da D. Coppersmith in [26], mentre algoritmi classici (e.g. Fast Fourier Transform) si possono trovare in [68]. L’algoritmo di Shor è stato presentato in [98], ma la trattazione usata in questo testo ricalca quella di [38]. Per le nozioni di teoria dei numeri si può consultare l’autorevole testo [69]. Una formulazione unitaria di diversi problemi in termini di sottogruppo nascosto è fornita in [62]. L’algoritmo di Grover è stato presentato in [47]. Il fatto che esso sia ottimale per la ricerca su di un database non strutturato, è stato mostrato in [17]. I limiti della computazione quantistica sono discussi in [15]. La possibilità di una computazione quantistica “fault-tolerant” è stata mostrata in [99]. Infine, alcune possibili implementazioni fisiche di computers quantistici sono discusse in [82].

- [1] Adleman L. M., Manders K., *NP*-complete decision problems for binary quadratics, *J. Comput. System Sci.* 16 (1978), 168-184.
- [2] Agrawal M., Kayal N., Saxena N., PRIMES is in *P*, Preprint, 8 agosto 2002 (l'articolo è reperibile nel sito www.cse.iitk.ac.in/news/primality.html).
- [3] Agrawal M., Kayal N., Saxena N., PRIMES is in *P*, Preprint 2003 (l'articolo è reperibile nel sito www.cse.iitk.ac.in/news/primality.html).
- [4] Aho A. V., Ullman J. D., *The Theory of Parsing, Translation and Compiling*, (Volume I and II), Prentice-Hall, (1972).
- [5] Aho A. V., Hopcroft J. E., Ullman J. D., *The Design and Analysis of Computer Algorithms*, Addison-Wesley, (1974).
- [6] Aho A. V., Ullman J. D., *Principles of Compiler Design*, Addison-Wesley, (1977).
- [7] Aiello M., Albano A., Attardi G., Montanari U., *Teoria della Computabilità, Logica, Teoria dei Linguaggi Formali*, Materiali Didattici ETS, Pisa, (1976).
- [8] Arbib M. A., *Theories of Abstract Automata*, Prentice Hall, (1969).
- [9] Ausiello G., d'Amore F., Gambosi G., *Linguaggi, Modelli, Complessità*, Franco Angeli Editore, (2003).
- [10] Babai L., Trading Group Theory For Randomness, *Proc. 17th ACM Symp. on Theory Of Computing* (1985), 421-429.
- [11] Babai L., Moran S., Arthur-Merlin games: A randomized proof system, and a hierarchy of complexity classes, *J. Comput. System Sci.* 36 (1988), 254-276.
- [12] Baker T., Gill J., Solovay R., Relativizations of the $P = NP$ question, *SIAM J. Comput.* 4 (1975), 431-442.
- [13] Bar-Hillel Y., Perles M., Shamir E., On formal properties of simple phrase structure grammars, *Z. Phonetik, Sprachwissen. Kommunikationsforsch.* 14 (1961), 143-172.
- [14] Bar-Hillel Y., *Language and Information*, Addison-Wesley, (1965).
- [15] Beals R., Buhrman H., Cleve R., Mosca M., de Wolf R., Quantum lower bounds by polynomials, *Proc. 39th Annual IEEE Symp. on Foundations of Computer Science - FOCS* (1998), 352-361.
- [16] Bennett C. H., Logical reversibility of computation, *IBM J. Res. Dev.*, 17 (1973), 525-532.
- [17] Bennett C. H., Bernstein E., Brassard G., Vazirani U., Strengths and weaknesses of quantum computing, *SIAM J. Comput.*, 26 (1997), 1510-1523.

- [18] Berman L., Hartmanis J., On Isomorphisms and Density of NP and other Complete Sets, *SIAM J. Comput.* 6 (1977), 305-322.
- [19] Bernstein E., Vazirani U., Quantum complexity theory, *Proc. 25th Ann. ACM Symp. on the Theory of Computing* (1993), 11-20.
- [20] Berthiaume A., Brassard G., The quantum challenge to structural complexity theory, *Proc. 7th Ann. IEEE Conference on Structure in Complexity Theory* (1992), 132-137.
- [21] Biggs N. L., *Discrete Mathematics*, Oxford University Press, (1999).
- [22] Bovet D.P., Crescenzi P., *Teoria della Complessità Computazionale*, Franco Angeli, (1991).
- [23] Cintoli P., Toffalori C., *Logica Matematica*, McGraw-Hill, (2000).
- [24] Cobham A., The intrinsic Computational Difficulty of Functions, Yehoshua Bar-Hillel, editor, *Proc. 1964 International Congress for Logic, Methodology, and Philosophy of Science* (1964), 24-30.
- [25] Cook S. A., The Complexity of Theorem-Proving Procedures, *Proc. 3rd ACM Symp. on Theory of Computing* (1971), 151-158.
- [26] Coppersmith D., An approximate Fourier transform useful in quantum factoring, *IBM Research Report RC 19642*, (1994).
- [27] Davis M. (a cura di), *The Undecidable*, Raven Press, (1965).
- [28] Davis M., Hilbert's Tenth Problem is Unsolvable, *American Mathematical Monthly* 80 (1973), 233-269.
- [29] Davis M., Sigal R., Weyuker E., *Computability, Complexity and Languages: Fundamentals of Theoretical Computer Science*, Academic Press, (1983).
- [30] Denef J., Lipshitz L., Pheidas T., Van Geel J (a cura di), *Hilbert's Tenth Problem*, Contemporary Mathematics 270, American Mathematical Society, (2000).
- [31] De Nicola R., Piperno A., *Semantica Operazionale e Denotazione dei Linguaggi di Programmazione*, Città Studi Edizioni, UTET Libreria, (1999).
- [32] Deutsch D., Quantum theory, the Church-Turing Principle and the universal quantum computer, *Proc. R. Soc. London A*, 400 (1985), 97-117.
- [33] Deutsch D., Jozsa R., Rapid solutions of problems by quantum computation, *Proc. R. Soc. London A*, 439 (1992), 553-558.
- [34] Devlin K., *I problemi del millennio*, Longanesi, (2004).
- [35] Edmonds J., Paths, Trees, and Flowers, *Can. J. Math.* 17 (1965), 449-467.
- [36] Edmonds J., Systems of distinct representatives and linear algebra, *J. Res. Nat. Bur. Standards* 71B (1967), 241-245.
- [37] Edmonds J., Optimum branchings, *J. Res. Nat. Bur. Standards* 71B (1967), 233-240.
- [38] Ekert A., Jozsa R., Quantum computation and the Shor's factoring algorithm, *Rev. Mod. Phys.* 68 (1996), 733-753.
- [39] Enderton H., *Elements of Recursion Theory*, Handbook of Mathematical Logic, North Holland, Amsterdam, 527-566, (1977).
- [40] Facchini A., *Algebra e Matematica Discreta*, Decibel Zanichelli, (2000).
- [41] Fässler A., Stiefel E., *Group Theoretical Methods and their Applications*, Birkhäuser, Boston, (1992).

- [42] Garey M., Johnson D., *Computers and Intractability: A guide to the Theory of Completeness*, Freeman, (1979).
- [43] Gill J., Computational Complexity of Probabilistic Turing Machines, *SIAM J. Comput.* 6 (1977), 675-695.
- [44] Goldreich O., Micali S., Wigderson A., Proofs that Yield Nothing but their Validity of the Assertion and the Methodology of Cryptographic Protocol Design, *Proc. 27th Symp. on Foundations of Computer Science* (1986), 174-187.
- [45] Goldwasser S., Micali S., Rackoff C., The Knowledge Complexity of Interactive Proof-Systems, *Proc. 17th ACM Symp. on Theory of Computing* (1985), 291-304; oppure *SIAM J. Comp.* 18 (1989), 186-208.
- [46] Goldwasser S., Sipser M., Arthur-Merlin Games versus Interactive Proof Systems, *Proc. 18th Annual ACM Symp. on the Theory of Computing* (1986), 59-68.
- [47] Grover L. K., A fast quantum-mechanical algorithm for database search, *Proc. 28th Ann. ACM Symp. on Theory of Computing - STOC* (1996), 212-219.
- [48] Hartmanis J., Lewis P. M. II, Stearns R. E., Hierarchies of Memory Limited Computations, *Proc. 6th Annual IEEE Symp. on Switching Circuit Theory and Logical Design* (1965), 179-190.
- [49] Hartmanis J., Stearns R. E., On the Computational Complexity of Algorithms, *Trans. Am. Math. Soc.* 117 (1965), 285-306.
- [50] Hemaspaandra L. A., Ogihara M., *The Complexity Theory Companion*, Springer-Verlag, (2002).
- [51] Hennie F. C., *Introduction to Computability*, Addison-Wesley, (1977).
- [52] Herken R. (a cura di), *The Universal Turing Machine: a Half-Century Survey*, Springer, (1994).
- [53] Hilbert D., Mathematische Probleme, *Proc. Int. Congress of Mathematicians in Paris* (1900), 253-297 (traduzione inglese in: *Bullettin American Mathematical Society*, 2 (1901-1902), 437-479; ristampato in: *Bullettin American Mathematical Society*, 37 (2000), 407-436).
- [54] Hopcroft J. E., Ullman J.D., *Formal Languages and their Relations to Automata*, Addison-Wesley, (1969).
- [55] Hopcroft J. E., Ullman J.D., *Introduction to automata, languages and computation*, Addison-Wesley, (1979).
- [56] Hopcroft J. E., Motwani R., Ullman J. D., *Automi, Linguaggi e Calcolabilità*, Addison-Wesley, (2003).
- [57] Horn R. A., Johnson C. R., *Topics in Matrix Analysis*, Cambridge University Press, (1991).
- [58] Immerman N., Nondeterministic Space is Closed under Complementation, *SIAM J. Comput.* 17 (1988), 935-938.
- [59] Impagliazzo R., Levin L.A., Luby M., Pseudorandom Generation from One-Way Functions, *Proc. 21st ACM Symp. on Theory of Computing* (1989), 12-24.
- [60] Johnson D. S., The NP-Completeness Column: An Ongoing Guide. *J. Algorithms*, 13 (1992), 502-524.

- [61] Johnson D. S., A Catalog of Complexity Classes, 67-161 in [110].
- [62] Jozsa R., Quantum algorithms and the Fourier transform, *Proc. Royal Soc. London A* 454, (1998) 323-337.
- [63] Karp R. M., Reducibility among Combinatorial Problems, *Complexity of Computer Computations*, a cura di R. E. Miller and J. W. Thatcher, Plenum, 1972, 85-103.
- [64] Karp R. M., On the Computational Complexity of Combinatorial Problems, *Networks* 5 (1975), 45-68.
- [65] Karp R. M., Lipton R. J., Turing Machines that take Advice, *Enseign. Math.* 28 (1982), 191-209.
- [66] Kfoury A. J., Moll R. N., Arbib M. A., *A Programming Approach to Computability*, Springer, (1982) (traduzione italiana: *Programmazione e Computabilità*, ETAS Libri, (1986)).
- [67] Kleene S. C., General Recursive Functions of Natural Numbers, *Mathematische Annalen* 112 (1936), 727-742.
- [68] Knuth D. E., *The art of Computer Programming, Volume 2: Seminumerical Algorithms*, Addison-Wesley, (1981).
- [69] Koblitz N., *A Course in Number Theory and Cryptography*, Springer, (1994).
- [70] Ladner R. E., On the Structure of Polynomial Time Reducibility, *J. Ass. Comput. Mach.* 22 (1975), 155-171.
- [71] Landauer R., Irreversibility and heat generation in the computing process, *IBM J. Res. Dev.*, 5 (1961), 183-191.
- [72] Lautemann C., *BPP* and the Polynomial Hierarchy, *Information Processing Letters* 17 (1983), 215-217.
- [73] Leonesi S., L'Innocente S., Marconi M., Toffalori C., Primi e Segreti, *Lettera Matematica Pristem* 52 (2004).
- [74] Leonesi S., L'Innocente S., Toffalori C., Misteri Informatici, *Lettera Matematica Pristem* in corso di stampa.
- [75] Levin L. A., Universal sorting problems, *Problems of Information Transmission* 9 (1973), 265-266.
- [76] Lewis H. R., Papadimitriou C. H., *Elements of the Theory of Computation*, Prentice-Hall, (1981).
- [77] Matiyasevich Y., Enumerable Sets are Diophantine, *Doklady Akademii Nauk SSSR* 191 (1970), 279-282, in russo (traduzione inglese in: *Soviet Mathematics-Doklady*, 11 (1970), 354-357).
- [78] Mermin N. D., From Cbits to Qbits: Teaching computer scientists quantum mechanics, *Amer. J. Phys.*, 71 (2003), 23-30.
- [79] Milner R., *Communication and Concurrency*, Prentice Hall, (1989).
- [80] Minsky M. L., *Computation: Finite and Infinite Machines*, Prentice-Hall, (1967).
- [81] Moll R. N., Arbib M. A., Kfoury A. J., *An Introduction to Formal Language Theory*, Springer, (1988).
- [82] Nielsen M. A., Chuang I. L., *Quantum Computation and Quantum Information*, Cambridge University Press, (2000).

- [83] Neumann J. von. A certain Zero-Sum Two-Person Game equivalent to the Optimal Assignment Problem. a cura di H. W. Kahn and A. W. Tucker, *Contributions to the Theory of Games II*, Princeton Univ. Press, (1953).
- [84] Odifreddi P., *Classical Recursion Theory*, North Holland, (1992).
- [85] Papadimitriou C.H., *Computational Complexity*, Addison-Wesley, (1994).
- [86] Peres A., *Quantum Theory: Concepts and Methods*, Kluwer, (1993).
- [87] Péter R., *Recursive Functions*, Academic Press, (1967).
- [88] Rabin M., Real time computation, *Israel J. Math.* 1 (1963), 203-211.
- [89] Rice H. G., Classes of Recursively Enumerable Sets and Their Decision Problems, *Trans. Amer. Math. Soc.* 74 (1953), 358-366.
- [90] Rivest R., Cryptography, 719-755 in [110].
- [91] Rogers H., *Theory of Recursive Functions and Effective Computability*, McGraw-Hill, (1967).
- [92] Ross S.M., *Calcolo delle probabilità*, Apogeo, (2004).
- [93] Rozenberg G., Salomaa A., *Cornestones of Undecidability*, Prentice-Hall, (1994).
- [94] Salomaa A., *Public-Key Cryptography*, Springer, (1996).
- [95] Savitch W. J., Relationships between Non Deterministic and Deterministic Tape Classes, *J. Comput. System Sci.* 4 (1970), 177-192.
- [96] Shamir A., $IP = PSPACE$, *J. Ass. Comput. Mach.* 39 (1992), 869-877.
- [97] Shiryayev A. N., *Probability*, Springer, (1996).
- [98] Shor P., Algorithms for quantum computation: discrete log and factoring, *Proc. 35th Ann. IEEE Symp. on Foundations of Computer Science - FOCS* (1994), 20-22.
- [99] Shor P., Fault-tolerant quantum computation, *Proc. 37th Ann. IEEE Symp. on Foundations of Computer Science - FOCS* (1996), 56-65.
- [100] Simon D., On the power of quantum computation, *Proc. 35th Ann. IEEE Symp. on Foundations of Computer Science - FOCS* (1994), 116-123.
- [101] Sipser M., A Complexity Theoretic Approach to Randomness, *Proc. ACM Symp. on Theory of Computing* (1983), 330-335.
- [102] Sipser M., *Introduction to the Theory of Computation*, Course Technology, (1996).
- [103] Soare R. J., *Recursively Enumerable Sets and Degrees*, Springer, (1988).
- [104] Stockmeyer L. J., Meyer A. R., Word Problems requiring Exponential Time, *Proc. 5th Ann. ACM Symp. on Theory of Computing* (1973), 1-9.
- [105] Szelepcs'enyi R., The method of forced enumeration for nondeterministic automata, *Acta Informatica* 26 (1988), 279-284.
- [106] Toda S., PP is as Hard as the Polynomial-Time Hierarchy. *SIAM J. Comput.* 20 (1991), 865-877.
- [107] Toffoli T., Bicontinuous Extensions of Invertible Combinatorial Functions, *Math. System Theory* 14 (1981), 13-23.
- [108] Turing A. M., On Computable Numbers, with an Application to the Entscheidungsproblem, *Proc. London Mathematical Society* (1936), 230-265.
- [109] Valiant L. G., The Complexity of Computing the Permanent, *Theor. Comp. Science* 8 (1979), 189-201.

- [110] Van Leeuwen J., *Handbook of Theoretical Computer Science, vol A: Algorithms and Complexity*, Elsevier, (1990).
- [111] Valiant L. G., Vazirani V. V., *NP is as Easy as Detecting Unique Solutions*, *Theor. Comp. Science* 47 (1986), 85-93.
- [112] Williams C. P., Clearwater S. H., *Explorations in Quantum Computing*, Springer, (1998).
- [113] Williams C. P., Clearwater S. H., *Ultimate Zero and One. Computing at the Quantum Frontiers*, Springer, (2000).
- [114] Yao A. C., Quantum circuit complexity, *Proc. 34th Ann. IEEE Symp. on Foundations of Computer Science - FOCS* (1993), 352-361.
- [115] www.csc.liv.ac.uk/~ped/teachadmin/COMP202/annotated_np.html.

- 2COL, 136
- 2DM, 180
- 2SAT, 137
- 3CNF, 185
- 3COL, 145
- 3DM, 180
- 3SAT, 144
- 3UNSAT, 276
- AKS, 142, 229
- AM, 273
- AM[*Poly*], 271
- AM[*k*], 273
- AND, 187, 343
- BPP, 240
- BQP, 311, 334, 342
- CIRCUITSAT, 204
- CLIQUE, 147
- CNF, 185
 - riduzione in, 186
- CNOT, 195
- EQP, 310, 325
- FANOUT, 192
- GAP, 215
- GI, 166
- GI^c , 274
- IP, 275
- IS, 146
- KNAPSACK, 148, 165, 177
- L, 211
- LEXP, 175
- LOGSPACE, 211
- MA, 273
- MA[*k*], 273
- $M \downarrow w$, 18
- $M \uparrow w$, 18
- NAND, 192
- NL, 212
- NL-arduo, problema, 215
- NL-completo, problema, 215
- NLOGSPACE, 212
- NOT, 187
- NP, 144, 342
- NP-arduo, problema, 153
- NP-completo, problema, 153
- NP-intermedio, 165
- NPC, 166
- NPI, 166, 334
- NPSPACE, 212
- $O(f)$, 126
- OR, 187, 343
- PARITY, 343
- PART, 181
- PEXP, 175
- PH, 173
- PP, 234
- PSPACE, 210
- PSPACE-arduo, problema, 221
- PSPACE-completo, problema, 221
- QBF, 190
- QBF_h , 224
- QSAT, 191
- $QSAT_h$, 224
- RP, 240
- RSA, 177
- SAT, 144
- SIZE, 199
- SWAP, 195
- S^c , 143
- UNSAT, 168
- UP, 177
- USAT, 251
- VC, 147
- XOR, 195
- ZPP, 242

- ZQP , 311
- $\leq_{log}$, 213
- $\leq_{par}$, 253
- $\leq_p$, 152
- $\oplus$, 193, 248
- $\#(S, w)$, 252
- $\#3SAT$, 252
- $\#CIRCUITSAT$, 252
- $\#P$, 252
- $\#P$ -completo, problema, 253
- $\#S$, 252
- $\#SAT$, 252
- $\#V_S$, 252
- $\vdash_M^*$, 18
- $\vdash_M$, 18
- c_M , 133
- $coBPP$, 245
- $coNP$, 168
- coP , 143
- $coPP$, 245
- $coRP$, 245
- $coZPP$, 245
- $l(w)$, 144
- $nSAT$, 144
- s_M , 208
- 2-colorabilità, 136
- 2-soddisfacibilità, 137
- 3-colorabilità, 145
- 3-soddisfacibilità, 144
- accettare, 20, 38
- Agrawal, 142
- Agrawal, Kayal, Saxena, 229
- albero
 - di derivazione, 83
- alfabeto, 12
- algoritmo
 - Solovay-Strassen, 229
 - di Deutsch-Jozsa, 323, 339
 - di Grover, 339, 342, 344
 - di Shor, 311, 330, 337, 346
 - di Simon, 325
 - interattivo, 269
 - Las Vegas, 233
 - Miller-Rabin, 229
 - Montecarlo, 233
- ampiezze di probabilità, 298, 299
- Artù, 269
- Artù-Merlino
 - protocollo, 272
- automa
 - a pila, 99
 - a stati finiti, 99
 - linearmente limitato, 99
- autovalore, 293
- autovettore, 293
- Baker, 174, 225
- Baker, Gill, Solovay, 174, 225
- base computazionale, 296, 308
- Berman, 167
- bit, 129
- black box, 345
- cammino
 - di un grafo, 136
- Cantor, 26
- Carmichael
 - pseudoprimi di, 230
- cella, 15
- Chernoff
 - limiti di, 237
- chiusura
 - di Kleene, 78
- Chomsky, 81
- Church, 31
- circuiti
 - sequenza uniforme, 199
- circuito, 192
 - misura, 196
 - quantistico, 315
- classe di complessità
 - BPP , 236, 240
 - BQP , 311, 334, 342
 - EQP , 310, 325
 - L , $LOGSPACE$, 211
 - $LEXP$, 175
 - NL , $NLOGSPACE$, 212
 - NP , 144, 342
 - NPC , 166
 - NPI , 166, 334
 - $NPSPACE$, 212

- PEXP*, 175
- PH*, 173
- PP*, 234
- PSPACE*, 210
- RP*, 240
- ZPP*, 242
- ZQP*, 311
- coNP*, 168
- clausola, 137
- clique, 146
- colorabilità
 - 2COL*, 136
 - 3COL*, 145
 - mCOL*, 145
- complessità
 - s_M*, 208
 - c_M*, 133
 - spaziale, 208
 - temporale, 133
- composizione, 65
 - di linguaggi, 78
- computazione
 - di una MdT, 19
- computazione quantistica, 291, 305
- computer quantistico, 312, 345
- concatenazione, 13
- configurazione
 - finale, 19
 - iniziale, 18
- configurazione istantanea
 - di una MdT, 18
- Congettura di
 - Berman - Hartmanis, 167
- connettivi, 184
- convergere, 38
- Cook, 134
- correzione degli errori, 345
- cricca, 146
- crittografia, 175, 337
- decidere, 20
- decidibile, linguaggio, 21
- Decimo problema di Hilbert, 6, 56
- decoerenza, 345
- derivazione, 80
- Deutsch D., 305, 323
- dimostratore, 275
- divergere, 38
- Edmonds, 134
- entanglement, 296
- Eulero, 332
- fattorizzazione, 311, 323, 334
- Fermat
 - Piccolo Teorema, 230
- forma normale congiuntiva, 185
- formula
 - Booleana quantificata, 190
 - della logica proposizionale, 184
 - soddisfacibile, 185
- formule
 - equivalenti, 185
- frazione binaria, 328
- frazioni continue, 336
- funzione
 - WHILE-calcolabile, 110
 - parziale, 111
 - calcolabile secondo Turing, 22
 - caratteristica, 42
 - computabile secondo Turing, 22
 - definita per
 - composizione, 65
 - minimalizzazione, 66
 - recursione, 65
 - di Ackermann, 71
 - di transizione, 17
 - parziale ricorsiva, 67
 - proiezione, 39, 64
 - ricorsiva, 67
 - successore, 39, 64
 - zero, 39, 64
- funzioni hash, 259
- Gödel, 6, 26
- Gacs, 246
- gerarchia
 - di Chomsky, 83
- gerarchia polinomiale, 172
- Gill, 174, 225
- Goldwasser, 274, 276
- grafi

- isomorfismo tra, 165
- grafo, 135
- grammatica
 - a struttura di frase, 80
 - di Chomsky, 81
 - di tipo 0, 80
 - di tipo 1, 82
 - di tipo 2, 82
 - di tipo 3, 82
 - dipendente dal contesto, 82
 - libera dal contesto, 82
 - monotona, 94
 - regolare, 82
- gray code, 318
- Grover L. K., 339
- Hartmanis, 167
- hash
 - funzioni, 259
- Hilbert, 6
 - decimo problema, 6, 56
- Impagliazzo, 259
- insieme
 - computabile probabilisticamente in tempo polinomiale, 234
 - con errore limitato, 236
 - con errore zero, 242
 - in modo casuale, 240
 - indipendente, 146
 - numerabile, 28
 - ricorsivamente enumerabile, 74
 - ricorsivo, 67
 - universale, 192, 316, 319
- interferenza quantistica, 299, 309
- isomorfismo
 - tra grafi, 166
- istruzione
 - di una MdT, 17
- Jozsa R., 323
- Karp, 134, 201, 249
- Karp-Lipton
 - teorema, 249
- Kayal, 142
- Kleene, 59
- Landauer R., 291, 303
- Las Vegas
 - algoritmo, 233
- lato
 - di un grafo, 136
- Lautemann, 246
- Levin, 259
- limiti di Chernoff, 237
- linguaggio
 - WHILE, 103
 - ad alto livello, 103
 - decidibile secondo Turing, 21
 - di tipo 0, 81
 - di tipo 2, 82
 - di tipo 3, 83
 - dipendente dal contesto, 82
 - formale, 15, 78
 - generato, 81
 - indecidibile secondo Turing, 21
 - libero dal contesto, 82
 - regolare, 83
 - riconosciuto da MdT, 21
 - ricorsivamente enumerabile, 74
 - semidecidibile secondo Turing, 21
- Lipton, 201, 249
- Luby, 259
- lunghezza
 - dell'input, 144
- macchina di Turing
 - a più nastri, 34
 - deterministica, 17, 149
 - non deterministica, 38, 149
 - probabilistica, 233
 - quantistica, 303, 305, 306
 - quantistica universale, 309
 - reversibile, 304
 - Universale, 44
- Matijasevic, 56
- Matijasevic, teorema, 57
- matrice funzionale, 16
- MdT, vedi Macchina di Turing, 15
- meccanica quantistica, 291

- Merlino, 269
- Micali, 274, 276
- Miller, 240, 242
- Miller-Rabin
 - algoritmo di, 229
- minimalizzazione, 66
- misura
 - di un circuito, 196
 - nella base computazionale, 299
 - quantistica, 298
- Montecarlo, 242
 - algoritmo, 233
- Moore G., 291

- operatori di Pauli, 297
- operatore, 292
 - hermitiano, 294
 - unitario, 294, 297
- operazioni controllate, 313, 314
- oracolo, 174, 321, 339, 340, 343
- ordinamento
 - lessicografico, 14
- ordine, 330

- parallelismo quantistico, 308, 321
- parola, 12
- permutazioni, 300
- Piccolo Teorema di Fermat, 230
- polinomi, 343
- polinomiale
 - gerarchia, 172
- porta logica, 191
 - CNOT*, 195, 314
 - CPHASE*, 314
 - CU*, 313, 317
 - CX*, 313
 - C₂U*, 314
 - C₂U*, 313
 - C₂X*, 313
 - C_{n-1}U*, 318
 - FANOUT*, 192
 - HADAMARD*, 313
 - NAND*, 192
 - NOT*, 313
 - PHASE*, 313, 328
 - SWAP*, 195, 313
 - XOR*, 195
 - $\oplus$, 193
 - $\sqrt{NOT}$, 313
 - quantistica, 297, 312
 - reversibile, 194
 - Toffoli, 195, 305, 312, 315, 317, 320
- Post, teorema, 51
- Postulati
 - della meccanica quantistica, 295
- potenza
 - di un linguaggio, 78
- prefisso, 13
 - proprio, 13
- primalità, 141
- principio
 - del minimo, 66
 - di Church-Turing, 303, 306, 307
 - di Church-Turing forte, 310
 - di induzione, 65
- problema
 - NL*-arduo, 215
 - NL*-completo, 215
 - NP*-arduo, 153
 - NP*-completo, 153
 - PSPACE*-arduo, 221
 - PSPACE*-completo, 221
 - Σ_k^P -completo, 173
 - $\#P$ -completo, 253
 - della soddisfacibilità, 185
 - con promessa, 338, 345
 - del matrimonio
 - a due sessi, 180
 - a tre sessi, 180
 - del periodo, 325, 330
 - del sottogruppo nascosto, 338
 - dell'accessibilità nei grafi, 215
 - dell'arresto, 45
 - dell'equivalenza tra MdT, 47
 - dell'isomorfismo tra grafi, 166
 - dell'ordine, 330
 - della fattorizzazione, 311, 330
 - della partizione, 181
 - della ricerca, 339
 - della soddisfacibilità unica, 251

- delle funzioni costanti o bilanciate, 323
- dello zaino, 148, 165, 177
 - supercrecente, 178
- rado, 166
- problema, vedi linguaggio formale, 15
- problemi
 - P*-isomorfi, 167
- prodotto
 - esterno, 293
 - interno, 292
 - tensoriale, 294
- produzione
 - della grammatica, 80
- programma
 - WHILE, 104
- proiezione, funzione, 39
- protocollo Artù-Merlino, 272
- pseudoprimi di Carmichael, 230
- Pumping Lemma
 - per linguaggi liberi da contesto, 89
 - per linguaggi regolari, 85
- quadro, 15
- qubit (quantum bit), 295, 300
- Rabin, 240, 242
- Rackoff, 274, 276
- Rado, 52
- recursione, 65
- registro quantistico, 297, 312
- regola
 - di produzione, 80
 - di transizione, 17
- Rice, H. G., 59
- ricoprimento
 - di vertici, 146
- riducibilità, 152
- risoluzione
 - mediante circuiti, 199
- Savitch, teorema, 218
- Saxena, 142
- semidecidibile, linguaggio, 21
- sequenza uniforme
 - di circuiti, 199
- sfera di Bloch, 296
- Shamir, 281
- Shor P., 330, 332
- simboli
 - non terminali, 80
 - terminali, 80
- simbolo
 - iniziale, 80
- Simon D., 325
- Sipser, 246, 276
- soddisfacibilità, 137
 - problema della, 185
- soddisfacibilità unica
 - problema della, 251
- Solovay, 174, 225
- Solovay-Strassen
 - algoritmo di, 229
- sovrapposizione di stati, 296, 308
- spazio di Hilbert, 292
- Stockmeyer-Meyer
 - teorema, 221
- stringa, 12
 - derivabile, 80
 - direttamente, 80
 - vuota, 12
- successore, funzione, 39
- suffisso, 13
 - proprio, 13
- supercrecente
 - sequenza, 177
- tempo medio polinomiale, 243
- Teorema
 - uvw*, 85
 - uvwxy*, 89
- Teorema di
 - Cook-Levin, 154
 - approssimazione per $\#P$, 256
 - Baker, Gill, Solovay, 174, 225
 - decomposizione spettrale, 294
 - Immerman-Szelepcs'enyi, 219
 - Impagliazzo-Levin-Luby, 259
 - Karp-Lipton, 201
 - Kleene, 59

- per linguaggi WHILE, 113
 - Matjasevic. 57
 - No-Cloning. 315
 - Post. 51
 - Rado. 54
 - Rice. 59
 - Savitch. 218
 - Sipser-Gacs-Lautemann, 246
 - Solovay e Kitaev, 319
 - Stockmeyer-Meyer, 221
 - universalità, 316
 - Valiant-Vazirani, 252, 261
 - Yao, 315
- Tesi
 - di Church, 67
 - di Church-Turing, 31
 - di Edmonds-Cook-Karp, 134
- Toffoli, porta logica, 195, 305, 312, 315, 317, 320
- trasformata
 - di Fourier discreta, 327
 - di Fourier quantistica, 327, 333
- trasformazione unitaria, 294, 297, 305, 312
- Turing, 15, 31
- unique satisfiability, *USAT*, 251
- Valiant, 252, 261
- valutazione, 184
 - di funzione, 320
- Vazirani, 252, 261
- verificatore, 275
- vertice
 - di un grafo, 136
- zero, funzione, 39